Machine Learning Handbook — Vol 2

deep learning, recommenders, reinforcement learning, ML system design, production, case studies, responsible ML, cheat sheet, question bank (Ch 11–21)

11 parts · one place to study: DesignGurus courses + classic books + the web, combined, beginner to intermediate

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 11 — CNNs and Computer Vision**
   - 11.1 Convolution
   - 11.2 Pooling and the CNN architecture
   - 11.3 A small CNN on 8×8 digits (PyTorch, CPU)
   - 11.4 Famous architectures — a timeline
   - 11.5 Transfer learning and fine-tuning
   - 11.6 Data augmentation for images
   - 11.7 Vision tasks: classification, detection, segmentation
   - 11.8 Putting it together — trade-offs, mistakes, interview
   - Chapter TL;DR
2. **Chapter 12 — Sequences: RNNs, LSTMs and Attention**
   - 12.1 Sequence data
   - 12.2 Recurrent neural networks (RNNs)
   - 12.3 Vanishing and exploding gradients through time
   - 12.4 LSTM and GRU
   - 12.5 Sequence-to-sequence (encoder-decoder) models
   - 12.6 The attention mechanism
   - 12.7 Self-attention and the transformer (high level)
   - 12.8 Text preprocessing and tokenization
   - 12.9 Word embeddings
   - 12.10 Time-series forecasting — an introduction
   - Chapter TL;DR
3. **Chapter 13 — Recommender Systems and Embeddings**
   - 13.1 The recommendation problem
   - 13.2 Explicit vs implicit feedback
   - 13.3 Similarity measures
   - 13.4 Content-based vs collaborative filtering
   - 13.5 Matrix factorisation
   - 13.6 Embeddings
   - 13.7 Two-tower retrieval
   - 13.8 The funnel: candidate generation → ranking → re-ranking
   - 13.9 Approximate nearest neighbour (ANN) search
   - 13.10 Cold start
   - 13.11 Evaluation
   - 13.12 Popularity bias and feedback loops
   - 13.13 Real-world systems
   - Chapter TL;DR
4. **Chapter 14 — Reinforcement Learning**
   - 14.1 What reinforcement learning is
   - 14.2 Markov decision processes, returns and discounting
   - 14.3 Policies, value functions and the Bellman equations
   - 14.4 Exploration vs exploitation — multi-armed bandits
   - 14.5 Dynamic programming — planning when the model is known
   - 14.6 Learning from experience: Monte Carlo vs temporal difference
   - 14.7 Q-learning and SARSA — tabular control
   - 14.8 Deep reinforcement learning — DQN
   - 14.9 Policy gradients, REINFORCE and actor-critic
   - 14.10 RLHF — the bridge to large language models
   - 14.11 Real-world RL
   - 14.12 Pitfalls
   - Chapter TL;DR
5. **Chapter 15 — ML System Design: The Interview Framework**
   - 15.1 ML design vs classic system design
   - 15.2 The three question families
   - 15.3 The seven-step framework
   - 15.4 Capstone — scoping a vague AI question
   - 15.5 Common mistakes (the whole interview)
   - Chapter TL;DR
6. **Chapter 16 — ML in Production: Pipelines, Serving, Evaluation, Monitoring and MLOps**
   - 16.1 The five pieces and the offline/online divide
   - 16.2 Training vs serving
   - 16.3 Where training data comes from
   - 16.4 Features and feature stores
   - 16.5 Batch vs online vs streaming inference
   - 16.6 Model serving basics
   - 16.7 Offline evaluation: choosing the metric
   - 16.8 Monitoring, drift and retraining
   - 16.9 Online experiments and A/B testing
   - 16.10 MLOps: the glue that makes it repeatable
   - 16.11 Capstone — an ML platform for ETA prediction
   - Chapter TL;DR
7. **Chapter 17 — ML Case Studies I: Recommendations, Feed Ranking, Search**
   - 17.0 The case-study template
   - 17.1 Case study: video recommendations
   - 17.2 Case study: social feed ranking
   - 17.3 Case study: e-commerce search ranking
   - 17.4 Case study: text-to-video search
   - Chapter comparison (17.1–17.4)
   - Chapter TL;DR
8. **Chapter 18 — ML Case Studies II: Ads, Fraud, Moderation, PYMK, Music, Visual Search, Blurring, Events, Similar Listings**
   - 18.1 Case study: ad click prediction
   - 18.2 Case study: payment fraud detection
   - 18.3 Case study: harmful content detection (and running it at platform scale)
   - 18.4 Case study: People You May Know (PYMK)
   - 18.5 Case study: music discovery (Spotify-style)
   - 18.6 Case study: visual search for a second-hand marketplace
   - 18.7 Case study: privacy blurring in street imagery
   - 18.8 Case study: event recommendation
   - 18.9 Case study: similar listings from session embeddings
   - Comparison across all thirteen case studies (Chapters 17–18)
   - Chapter TL;DR
9. **Chapter 19 — Responsible ML: Fairness, Explainability, Privacy and Security**
   - 19.1 Where bias comes from
   - 19.2 Fairness definitions and the impossibility trade-off
   - 19.3 Measuring fairness in code
   - 19.4 Mitigating bias: before, during and after training
   - 19.5 Explainability
   - 19.6 Privacy
   - 19.7 Security: attacks on ML models
   - 19.8 Documentation: model cards and datasheets
   - 19.9 Regulation overview
   - 19.10 Real-world incidents
   - 19.11 Responsible-ML checklist
   - 19.12 Interview Q&A
   - Chapter TL;DR
10. **Chapter 20 — Cheat Sheets and Glossary**
   - 20.1 The ML workflow (§1.9, §4.10, §6.11, §15.3, §16.1)
   - 20.2 Math essentials (Ch 2)
   - 20.3 NumPy, pandas and scikit-learn API (Ch 3, §4.10)
   - 20.4 Data preparation and feature engineering (Ch 4)
   - 20.5 Every algorithm in one table (Ch 5, 7–14)
   - 20.6 Loss functions (Ch 2, 5, 8, 10, 13, 14)
   - 20.7 Metrics: formula and when to use (Ch 6, 9, 11, 13, 16)
   - 20.8 Validation and tuning (Ch 4, 6)
   - 20.9 Deep learning: layers, activations, optimizers, regularisation, debugging (Ch 10)
   - 20.10 CNN, RNN and attention: shapes and formulas (Ch 11, 12)
   - 20.11 Recommenders (Ch 13, 17.1)
   - 20.12 Reinforcement-learning update rules (Ch 14)
   - 20.13 ML system design: the 7-step framework and napkin math (Ch 15, 17, 18)
   - 20.14 Production and MLOps checklist (Ch 16)
   - 20.15 Responsible-ML checklist (Ch 19)

- 20.16 Decision tables: which algorithm, which metric
- 20.17 Top 50 interview questions
- 20.18 Glossary (A–Z)
- Chapter TL;DR

11. **Chapter 21 — Interview Question Bank**
    - How to use this bank
    - 1. ML basics and the workflow
    - 2. Bias–variance, overfitting and regularisation
    - 3. Data, feature engineering, leakage and imbalance
    - 4. Evaluation, metrics, cross-validation and tuning
    - 5. Linear and logistic regression
    - 6. Decision trees, random forests and boosting
    - 7. k-NN, SVM and naive Bayes
    - 8. Unsupervised learning: clustering and dimensionality reduction
    - 9. Probability, statistics and math
    - 10. Deep learning fundamentals
    - 11. CNNs, RNNs, attention and transformers
    - 12. Recommender systems and embeddings
    - 13. ML system design
    - 14. MLOps: deployment, monitoring, drift and A/B testing
    - 15. Responsible ML
    - 16. Coding tasks commonly asked
    - Sources of the question lists

Chapter 11 — CNNs and Computer Vision

An image is a grid of numbers — a 224×224 colour photo is 224 × 224 × 3 = 150,528 of them. Dense networks (Chapter 10) handle that badly: far too many weights, and no notion that neighbouring pixels belong together or that a cat shifted ten pixels is still a cat. **Convolutional neural networks (CNNs)** fix both with one idea: slide a small learnable filter across the image and reuse it everywhere. That idea plus depth took vision from "barely works" to superhuman on some **benchmarks** (standard shared datasets and scoring rules) within a decade.

What you'll learn: convolution (kernels, stride, padding, channels, output size); pooling and CNN architecture; receptive field; the famous architectures LeNet → ViT; transfer learning; augmentation; detection (IoU, NMS, YOLO) and segmentation (U-Net); a small PyTorch CNN on scikit-learn's 8×8 digits, trained on CPU in seconds.

Prerequisites: Chapter 10 — backprop/gradients, optimizers (SGD, Adam), BatchNorm, dropout, the PyTorch training loop. All code is in `code/`; outputs are real.

11.1 Convolution

What it is

A **convolutional layer** slides a small grid of weights — a **kernel** or **filter**, typically 3×3 — over the image. At each position it multiplies the kernel element-wise with the pixel patch underneath and sums to one number; doing this everywhere produces a **feature map** (activation map). A layer has many kernels, each producing its own feature map and detecting a different pattern.

The problem it solves

Problem with dense layers on images	How convolution fixes it
Huge parameter counts (150k inputs × 1,000 units = 150M weights for one layer)	**Weight sharing**: one 3×3×3 kernel has 27 weights (+1 bias) regardless of image size
Ignores spatial structure — pixel order could be shuffled	**Local connectivity**: each output looks only at a small neighbourhood
A pattern learned in one place must be re-learned everywhere	**Translation equivariance**: shift the input and the feature map shifts the same way

From `code/conv_by_hand.py`:

```
Conv2d(3->16, 3x3) params: 448  vs Linear(32*32*3 -> 16) params: 49168
```

(3·3·3 + 1) · 16 = 448 versus (3,072 + 1) · 16 = 49,168 — and the conv count doesn't change with image size.

How it works — worked 3×3 example

Input X (5×5, one channel) and a vertical-edge kernel K:

```
X =  1 2 0 1 3       K =  1 0 -1
     0 1 3 2 1            1 0 -1
     2 1 0 1 0            1 0 -1
     1 0 2 3 1
     0 2 1 0 2
```

1. Top-left patch [[1,2,0],[0,1,3],[2,1,0]]: left column (1+0+2)·1 + middle ·0 + right column (0+3+0)·(−1) = 3 − 3 = **0**.
2. Slide one step right; at the end of the row move one down.
3. Centre patch [[1,3,2],[1,0,1],[0,2,3]]: left column 2, right column 6 → **−4** — brighter on the right than the left: a vertical edge.
4. Add the bias (0 here) and, in a real network, apply an activation such as ReLU.

Full output, stride 1, no padding:

```
stride=1 padding=0 -> output (3, 3) (formula 3), matches torch: True
[[ 0.  0. -1.]
 [-2. -4.  3.]
 [ 0. -1.  0.]]
```

Strictly, deep-learning "convolution" is **cross-correlation** — the kernel isn't flipped. Since kernels are learned, it doesn't matter; PyTorch's `F.conv2d` does exactly this loop:

```python
# code/conv_by_hand.py (core)
def conv2d(x, k, stride=1, padding=0):
    x = np.pad(x, padding)
    kh, kw = k.shape
    oh = (x.shape[0] - kh) // stride + 1
    ow = (x.shape[1] - kw) // stride + 1
    out = np.zeros((oh, ow))
    for i in range(oh):
        for j in range(ow):
            patch = x[i * stride:i * stride + kh, j * stride:j * stride + kw]
            out[i, j] = np.sum(patch * k)          # element-wise multiply, then sum
    return out
```

Stride and padding

- **Stride** s: pixels the kernel jumps per move. Stride 2 halves height and width — a cheap way to **downsample**.

- **Padding** p: extra (usually zero) border. Without it each 3×3 conv shrinks the image by 2 and under-uses edge pixels. **"Same"** padding (p = 1 for 3×3, p = 2 for 5×5) keeps size at stride 1; **"valid"** = no padding.

Output size (per spatial dimension): **out = ⌊(n + 2p − k) / s⌋ + 1**

n	k	s	p	out	Check
5	3	1	0	(5 − 3)/1 + 1 = 3	valid conv shrinks by 2
5	3	2	0	⌊2/2⌋ + 1 = 2	stride 2 downsamples
5	3	1	1	(5 + 2 − 3)/1 + 1 = 5	same padding
224	7	2	3	⌊(224 + 6 − 7)/2⌋ + 1 = 112	ResNet's first layer

```
stride=2 padding=0 -> output (2, 2) (formula 2), matches torch: True
[[ 0. -1.]
 [ 0.  0.]]

stride=1 padding=1 -> output (5, 5) (formula 5), matches torch: True
[[-3. -2.  0. -1.  3.]
 [-4.  0.  0. -1.  4.]
 [-2. -2. -4.  3.  6.]
 [-3.  0. -1.  0.  4.]
 [-2. -2. -1.  0.  3.]]
```

Channels

Images have 3 **channels** (RGB); hidden feature maps have many (64, 128, 512…). A kernel always spans **all input channels**: a 3×3 kernel on 64 channels is 3×3×64 = 576 weights producing one output channel; C_out kernels give C_out channels.

- Conv weights: **k × k × C_in × C_out** (+ C_out biases).
- PyTorch tensor layout: **N × C × H × W** (batch, channels, height, width). `nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0)`.
- A **1×1 convolution** mixes channels at each pixel without looking at neighbours — cheaply changes channel count (Inception, ResNet **bottlenecks**: layers that temporarily squeeze to fewer channels to save compute).
- A **depthwise-separable convolution** = per-channel k×k spatial conv + 1×1 channel-mixing conv (MobileNet, Xception, EfficientNet). Cost per output pixel: standard k²·C_in·C_out vs separable k²·C_in + C_in·C_out; ratio 1/C_out + 1/k² — for k = 3, C_in = C_out = 256: ≈ 0.115, about 8.7× cheaper.

What do kernels detect?

`code/edge_filters.py` applies classic hand-designed kernels to a digit. A CNN **learns** kernels like these — trained first-layer filters look like edge, colour-blob and stripe detectors.



Figure — A "0" digit and the feature maps from Sobel vertical/horizontal edge detectors (red = positive, blue = negative), a blur and a sharpen kernel.

Common mistakes

- Mixing up N×C×H×W (PyTorch) and N×H×W×C (TensorFlow/Keras, NumPy images) — fix with `permute(0, 3, 1, 2)`.
- Forgetting padding, then wondering why the image shrinks layer after layer.
- Computing the **flatten** size (C×H×W unrolled into one vector for a dense layer) wrongly by hand — pass a dummy tensor through and print the shape, or use `nn.LazyLinear`.

In the interview

30-second answer: "A conv layer slides small learned kernels, each spanning all input channels, across the image and sums element-wise products into a feature map. Weight sharing makes the parameter count independent of image size; locality plus sharing give translation equivariance. Output size is ⌊(n + 2p − k)/s⌋ + 1; stride downsamples, padding preserves size; 1×1 convs mix channels cheaply."

- *Q: 224×224×3 through `Conv2d(3, 64, 7, stride=2, padding=3)`?* A: 64 × 112 × 112; weights 7·7·3·64 + 64 = 9,472.
- *Q: Is convolution translation invariant?* A: Equivariant — the feature map shifts with the input; pooling and global pooling add (approximate) invariance.
- *Q: Weights in a 3×3 conv, 64 → 64 channels, no bias?* A: 3·3·64·64 = 36,864.

TL;DR

Shared k×k×C_in kernels slide over the input; out = ⌊(n + 2p − k)/s⌋ + 1; weights = k·k·C_in·C_out. Weight sharing + locality = few parameters and translation equivariance.

11.2 Pooling and the CNN architecture

What it is

A **pooling layer** downsamples each feature map independently by summarising small windows:

- **Max pooling** (usually 2×2, stride 2): keep the largest value — "was the feature present in this window?"
- **Average pooling**: the mean.
- **Global average pooling (GAP)**: average each whole feature map to one number — used at the end of modern CNNs instead of big dense layers.

Pooling has **no parameters**. It halves H and W (2×2/2), cutting compute, growing the receptive field, and adding tolerance to small shifts (**local translation invariance**).

Max-pool 2×2/2 of the 5×5 padded-conv output above (last row/column dropped — windows don't fit):

```
max-pool 2x2/2 of the padded-conv output (5x5 -> 2x2):
 [[0. 0.]
 [0. 3.]]
avg-pool via torch:
 [[-2.25 -0.5 ]
 [-1.75 -0.5 ]]
```

Top-left window [[−3, −2], [−4, 0]] → max 0, mean −2.25. Many modern networks use **stride-2 convolutions** (learned downsampling) instead and pool only at the end (GAP).

The standard CNN architecture



1. **Backbone (feature extractor)**: repeated conv → BatchNorm → activation blocks with periodic downsampling. Deeper = spatial size **shrinks**, channels **grow** (64 → 128 → 256 → 512) — keeping capacity and roughly constant compute per layer while features become more abstract and numerous.
2. **Head (classifier)**: flatten or GAP, then dense layer(s) producing class **logits**; softmax + cross-entropy for training.

Learned features form a hierarchy (Zeiler & Fergus, 2013): **edges and colours → textures and corners → object parts → whole objects**.

Receptive field

The **receptive field (RF)** of a unit is the input region that can influence it. Track RF and the **jump** (product of strides so far — input pixels per output step):

$RF_{new} = RF_{old} + (k - 1) \cdot jump_{old}$, $jump_{new} = jump_{old} \cdot s$

VGG pattern: conv3 → 3; conv3 → 5; maxpool 2/2 → 6, jump 2; conv3 → 10; conv3 → 14. `code/receptive_field.py`:

```
Three 3x3 convs (stride 1) vs one 7x7 conv, on a 32x32 input
layer                  out size   RF  jump
conv3x3                      32    3     1
conv3x3                      32    5     1
conv3x3                      32    7     1
layer                  out size   RF  jump
conv7x7                      32    7     1
weights (C=64 channels in and out, no bias): three 3x3 = 110,592  vs one 7x7 = 200,704

VGG-style block pattern: conv, conv, pool, ... on 224x224
layer                  out size   RF  jump
conv3x3                     224    3     1
conv3x3                     224    5     1
maxpool2                    112    6     2
conv3x3                     112   10     2
conv3x3                     112   14     2
maxpool2                     56   16     4
conv3x3                      56   24     4
conv3x3                      56   32     4
maxpool2                     28   36     8
```

Key insight (VGG, 2014): three stacked 3×3 convs see the same 7×7 region as one 7×7 conv but use $27C^2$ instead of $49C^2$ weights and add two extra nonlinearities — hence 3×3 kernels everywhere. Downsampling makes the RF grow much faster (jump doubles per pool), which is how deep layers "see" whole objects. The *effective* RF (where influence is actually strong) is smaller and roughly Gaussian.

Self-check

1. 32×32 input, conv(k=5, s=1, p=0)? → 28×28.
2. Parameters of `Conv2d(64, 128, 3)`? → 3·3·64·128 + 128 = 73,856.
3. RF of two stacked 3×3 convs (stride 1)? → 5×5.

TL;DR

2×2 max-pool halves H and W with no parameters. CNN = backbone of conv-BN-ReLU blocks with downsampling + head of GAP and dense layers. Stacked 3×3 convs grow the receptive field cheaply.

11.3 A small CNN on 8×8 digits (PyTorch, CPU)

scikit-learn's `load_digits`: 1,797 grayscale 8×8 handwritten digits (pixel values 0–16) — trains on a laptop CPU in seconds.

```python
# code/digits_cnn.py (core)
class SmallCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1),   # 1x8x8  -> 16x8x8
            nn.BatchNorm2d(16), nn.ReLU(),
            nn.MaxPool2d(2),                              # 16x8x8 -> 16x4x4
            nn.Conv2d(16, 32, kernel_size=3, padding=1),  # -> 32x4x4
            nn.BatchNorm2d(32), nn.ReLU(),
            nn.MaxPool2d(2),                              # -> 32x2x2
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),                                 # -> 128
            nn.Dropout(0.3),
            nn.Linear(32 * 2 * 2, 10),                    # 10 class logits
        )
    def forward(self, x):
        return self.classifier(self.features(x))

X = (X / 16.0).reshape(-1, 1, 8, 8).astype(np.float32)   # N x C x H x W, scaled to [0, 1]
# ... stratified 70/30 split, DataLoader(batch_size=64, shuffle=True)
opt = torch.optim.AdamW(model.parameters(), lr=3e-3, weight_decay=1e-3)
sched = torch.optim.lr_scheduler.OneCycleLR(opt, max_lr=3e-3, total_steps=epochs * len(train_dl))
# ... standard loop: forward, cross_entropy, zero_grad, backward, step, sched.step()
```

AdamW = Adam with decoupled weight decay; **OneCycleLR** ramps the learning rate up then down over training. The script trains for 40 epochs an **MLP** with a similar parameter budget, the CNN, and the CNN with **shift augmentation** (each training image randomly moved up to 1 pixel), then tests on the normal test set and on the test set **shifted 1 pixel right**.

```
train (1257, 1, 8, 8)  test (540, 1, 8, 8)
MLP                        params= 7210  test accuracy=0.9759
CNN                        params= 6186  test accuracy=0.9907
CNN + shift augmentation   params= 6186  test accuracy=0.9852
MLP                        accuracy on test images shifted 1px right = 0.4722
CNN                        accuracy on test images shifted 1px right = 0.8574
CNN + shift augmentation   accuracy on test images shifted 1px right = 0.9796
misclassified (count, true, predicted): [(2, 8, 1), (2, 3, 9), (1, 9, 7), (1, 9, 1), (1, 8, 9)]
```

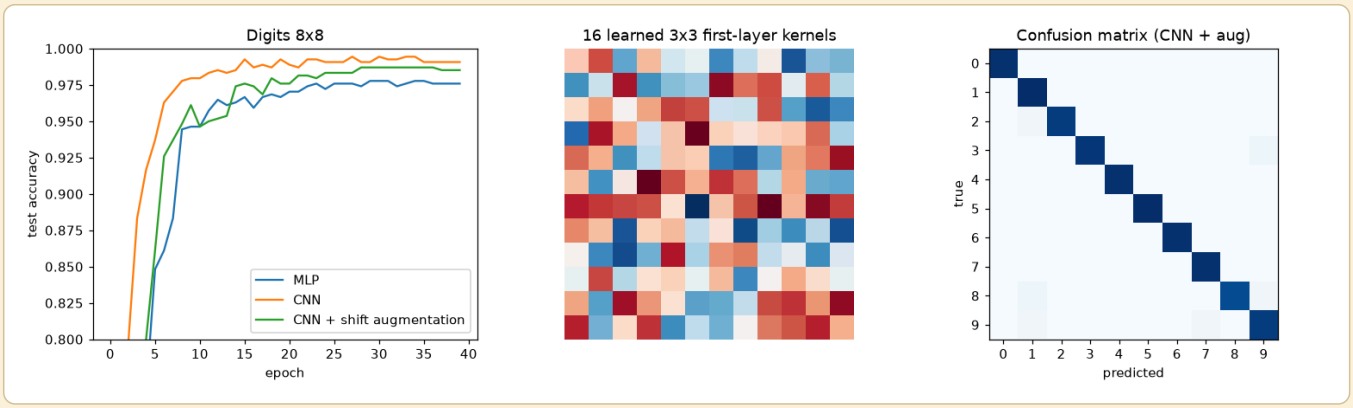


Figure — Left: test accuracy per epoch. Middle: the 16 learned first-layer kernels. Right: confusion matrix — nearly all on the diagonal; errors are 8↔1 and 3↔9 type.

- **Fewer parameters, higher accuracy**: CNN (6.2k) beats MLP (7.2k), 99.1% vs 97.6%.
- **Built-in spatial bias**: shift every test image one pixel and the MLP collapses to 47% (every "pixel feature" moved); the CNN keeps 86% thanks to shared kernels and pooling.
- **Augmentation buys robustness**: with random shifts the CNN keeps 98% on shifted images, at a half-point cost on unshifted test — a typical trade-off on a tiny dataset.

For real images (28×28 MNIST, 32×32 CIFAR-10, 224×224 ImageNet) the same code works with more blocks and channels, `torchvision.datasets` for loading, and a GPU.

Common mistakes

- Forgetting the channel dimension (8×8 must become 1×8×8).
- Evaluating in `train()` mode — call `model.eval()`: dropout off, BatchNorm uses its stored running averages.
- Too much downsampling for small images: 8×8 can only be halved three times.

TL;DR

On 8×8 digits a two-block CNN beats a larger MLP (99.1% vs 97.6%) and is far more shift-robust; shift augmentation makes it nearly shift-proof.

11.4 Famous architectures — a timeline

What it is

Most are winners of the **ImageNet Large Scale Visual Recognition Challenge (ILSVRC)** — 1.28M training images, 1,000 classes — each introducing an idea still in use. **Top-5 error** = true class not in the model's 5 best guesses.

Year	Model	Depth / params	Key idea	Result
1998	**LeNet-5** (LeCun et al.)	7 layers, ~60k	Conv → pool → conv → pool → dense; backprop	Read handwritten digits on bank cheques in production
2012	**AlexNet** (Krizhevsky, Sutskever, Hinton)	8 layers, ~60M	ReLU, dropout, heavy augmentation, 2 GPUs	Top-5 error 15.3% vs 26.2% runner-up — started the deep-learning boom
2014	**VGG-16/19** (Simonyan & Zisserman)	16–19 layers, ~138M (VGG-16)	Only stacked 3×3 convs; very uniform	ILSVRC 2014 runner-up, 7.3% top-5; still a popular feature extractor
2014	**GoogLeNet / Inception** (Szegedy et al.)	22 layers, ~7M	Parallel 1×1, 3×3, 5×5 branches; 1×1 bottlenecks; GAP instead of big dense layers	ILSVRC 2014 winner, 6.7% top-5
2015	**ResNet** (He et al.)	18–152 layers (ResNet-50 ≈ 25.6M)	**Residual (skip) connections** + BatchNorm	ILSVRC 2015 winner, 3.57% top-5 (ensemble); 100+ layer nets trainable
2017	**MobileNet** (Howard et al.)	small	Depthwise-separable convs for phones	Efficient on-device vision
2019	**EfficientNet** (Tan & Le)	B0–B7 (5.3M–66M)	**Compound scaling** of depth, width, resolution; SiLU (smooth ReLU-like) activation, MBConv (mobile inverted-bottleneck) blocks	B7: 84.3% top-1, 8.4× smaller than previous best (GPipe)
2020	**Vision Transformer (ViT)** (Dosovitskiy et al.)	86M+ (ViT-Base)	16×16 patches as tokens into a transformer (Chapter 12)	Beats CNNs when pre-trained on very large datasets
2022	**ConvNeXt** (Liu et al.)	ResNet-sized	ResNet modernised with transformer-era design choices	Matches ViTs — CNNs remain competitive

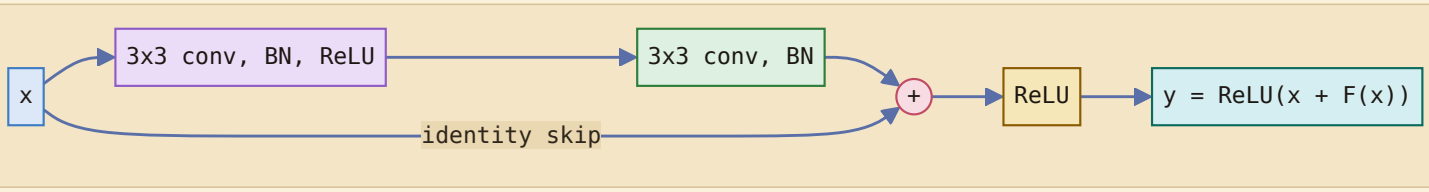
ResNet and skip connections

The problem. Before 2015, plain CNNs deeper than ~20 layers got *worse* — even on the **training** set. This "degradation" isn't overfitting (that hurts only test accuracy); very deep plain stacks are just hard to optimise.

The idea. Let a block learn the **residual** F(x) = H(x) − x and add the input back:

y = ReLU(x + F(x)), where F = conv → BN → ReLU → conv → BN

If doing nothing is best, the block learns F ≈ 0 — easy. In backprop ∂y/∂x includes an identity term, so gradients get a direct **highway** to early layers. When shapes differ (stride 2 or more channels) the skip path uses a 1×1 conv.




```python
# code/resnet_block.py (core)
class ResidualBlock(nn.Module):
    def __init__(self, c):
        super().__init__()
        self.f = nn.Sequential(
            nn.Conv2d(c, c, 3, padding=1, bias=False), nn.BatchNorm2d(c), nn.ReLU(),
            nn.Conv2d(c, c, 3, padding=1, bias=False), nn.BatchNorm2d(c))
    def forward(self, x):
        return torch.relu(x + self.f(x))                # the skip connection
```

Gradient norm reaching the first conv, and 30-conv-layer plain vs residual nets trained on digits for 5 epochs:

```
conv layers  plain |grad|  residual |grad|
          2      2.58e-02         4.78e-02
         10      1.11e-01         1.18e-01
         30      3.54e+00         4.14e-01
         60      3.08e+02         9.41e-01

Training 30-conv-layer nets on digits (SGD+momentum lr 0.05, 5 epochs):
  plain    training accuracy 0.575  training loss 1.121
  residual training accuracy 0.957  training loss 0.159
```

With BatchNorm the plain net's gradients don't vanish here — they become erratic and **explode** with depth (3×10^2 at 60 layers), while the residual net's stay near 1. Same settings: plain reaches 58% **training** accuracy (degradation), residual 96%. Skip connections are now everywhere: DenseNet, U-Net, every transformer block.

EfficientNet in one paragraph

Given more compute, grow deeper, wider, or higher-resolution? EfficientNet scales all three together with fixed ratios (depth × α^φ, width × β^φ, resolution × γ^φ), giving far better accuracy per FLOP than scaling any one. Base B0 came from **neural architecture search** (automated search over candidate designs); B1–B7 are compound-scaled.

Vision Transformer (brief)

ViT cuts an image into patches (16×16 → a 224×224 image gives 14 × 14 = 196), linearly projects each into a token, adds learned **position embeddings** and a learnable [CLS] token (whose final vector summarises the image), and runs a standard transformer encoder (Chapter 12). Lacking locality and translation equivariance, ViTs need much more data (the original used 300M-image pre-training) but scale extremely well. Hierarchical designs (Swin) and modern CNNs (ConvNeXt) compete closely; for small datasets, fine-tune a pretrained CNN or ViT.

Trade-offs

Need	Good choice
Strong, well-understood baseline	ResNet-50 (ResNet-18 for speed)
Mobile / edge devices (phones, cameras, embedded chips)	MobileNetV3, EfficientNet-B0
Best accuracy with large pretraining	ViT / Swin / ConvNeXt, EfficientNetV2
Tiny dataset	Any pretrained backbone, fine-tuned

In the interview

30-second answer: "LeNet showed conv-pool stacks work; AlexNet scaled them with ReLU, dropout and GPUs and won ImageNet 2012; VGG showed stacked 3×3 convs suffice; Inception used parallel branches and 1×1 bottlenecks; ResNet's identity skips let 100+ layer nets train — each block learns a residual and gradients flow through the identity; EfficientNet scales depth, width and resolution together; ViT applies a transformer to patches and wins with large-scale pretraining."

- *Q: Why do residual connections help?* A: Identity is easy to represent (F = 0) and gradients get a direct path — fixing degradation of deep plain nets.
- *Q: What is a 1×1 convolution for?* A: Mixing channels and changing channel count cheaply (bottlenecks).
- *Q: Why does ViT need more data than a CNN?* A: It lacks the CNN's **inductive biases** (built-in assumptions: locality, weight sharing), so it must learn them from data.

TL;DR

LeNet → AlexNet (ReLU, dropout, GPUs) → VGG (3×3 stacks) → Inception (1×1, multi-branch) → ResNet (y = x + F(x)) → MobileNet/EfficientNet (efficiency, compound scaling) → ViT (patches + transformer).

11.5 Transfer learning and fine-tuning

What it is

Transfer learning reuses a model trained on a large **source** task (typically ImageNet) as the starting point for your **target** task. Early layers learn general features — edges, textures, shapes — so you needn't relearn them from a few hundred images.

Why it matters

Training ResNet-50 from scratch on ImageNet takes days of GPU time and 1.28M labels. Fine-tuning a pretrained one on 2,000 of your images takes minutes and usually beats training from scratch on those 2,000 by far. It's the default for practical vision.

How it works — the two strategies

1. **Feature extraction (frozen backbone).** Load pretrained weights, **freeze** the backbone (`requires_grad = False`), replace the final layer with a new head, train only the head. Fast, little data, low overfitting risk.
2. **Fine-tuning.** Also update some or all backbone layers with a **small learning rate** (often 10× smaller than the head's — "discriminative learning rates") so features are adjusted, not destroyed. Common recipe: train the head with the backbone frozen, then unfreeze the top blocks (or all) at a low LR.

Your data	Similar to ImageNet	Very different (X-rays, satellite, spectrograms)
Small	Feature extraction	Features from earlier layers, or gentle fine-tune; augment heavily
Large	Fine-tune all layers	Fine-tune all layers (pretraining still usually helps convergence)

With torchvision (needs a weights download — not run here):

```python
import torch
from torch import nn
from torchvision import models

model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)   # ImageNet-pretrained
for p in model.parameters():
    p.requires_grad = False                                         # 1) freeze backbone
model.fc = nn.Linear(model.fc.in_features, num_classes)             # new head (trainable)
opt = torch.optim.AdamW(model.fc.parameters(), lr=1e-3)
# ... train a few epochs, then 2) unfreeze and fine-tune with a smaller lr:
for p in model.parameters():
    p.requires_grad = True
opt = torch.optim.AdamW([{"params": model.fc.parameters(), "lr": 1e-3},
                         {"params": [p for n, p in model.named_parameters() if not n.startswith("fc.")], "lr": 1e-4}])
# Inputs must be preprocessed like the pretraining data: resize/crop to 224 and normalise with
# ImageNet mean (0.485, 0.456, 0.406) and std (0.229, 0.224, 0.225) - the weights object's
# .transforms() gives the exact pipeline.
```

A runnable demonstration (no downloads)

`code/transfer_learning.py`: pretrain a CNN on **clean** digits (half the dataset), then adapt to the other half **"scanned by a different device"** (Gaussian noise on every pixel) using only k labels per class; averaged over 3 seeds. The script's `make_model(strategy, pretrained)` loads the backbone only (new head) for "frozen backbone" and freezes it, or loads the whole source model for "fine-tune all" with lr 1e-4.

```
source model: train acc on clean digits 0.996, acc on noisy target digits with NO adaptation 0.839

labels/class     from scratch  frozen backbone    fine-tune all   (mean test acc over 3 seeds)
           2            0.530            0.650            0.840
           5            0.673            0.766            0.834
          10            0.738            0.779            0.847
          30            0.810            0.824            0.857

trainable params: from scratch / fine-tune = 6090 | frozen backbone = 1290
```

- With **2 labels per class**: scratch 53%, frozen backbone + new head 65%, gentle full fine-tune 84%.
- The advantage **shrinks as target data grows** (30/class: 81% vs 86%) — transfer matters most when labels are scarce.
- **Measure the pretrained model before adapting**: it already scores 83.9% with no training. Fine-tuning with a *large* LR or a randomly re-initialised head can do *worse* than that baseline — hence small LRs.
- Not magic: in a variant (not in the script) pretraining on digits 0–4 and adapting to 5–9, pretraining didn't help on these tiny images. Real ImageNet backbones, trained on 1,000 diverse classes, transfer far more broadly.

When not to use it

- Inputs nothing like natural images *and* plenty of data (pretraining may still help convergence — check).
- The weights' license forbids your use.

Common mistakes

- Not preprocessing exactly like the pretraining data (size, normalisation).
- Fine-tuning everything with a large LR on a small dataset → **catastrophic forgetting** (overwriting learned features).
- Letting BatchNorm update running statistics on tiny fine-tuning batches (often better frozen / eval mode).

Real-world examples

Medical-imaging classifiers, retail product recognition, factory defect detection; detectors (YOLO, Faster R-CNN) use pretrained classification backbones. LLMs (GPT, Claude) follow the same pretrain-big, fine-tune-small pattern (AI Handbook, Chapter 8).

In the interview

30-second answer: "Take a model pretrained on something like ImageNet, replace its head, and either freeze the backbone and train only the head (small data) or fine-tune some or all layers with a small learning rate (more data or a different domain). Preprocess like the pretraining data. It beats training from scratch whenever labels are limited."

- *Q: Which layers transfer best?* A: Early layers (generic edges/textures); later layers are task-specific, so unfreeze them first.
- *Q: Why a smaller LR for the backbone?* A: To adjust, not overwrite, pretrained features.
- *Q: Why did transfer help less with 30 labels/class than with 2?* A: With more target data, from-scratch catches up.

TL;DR

Pretrained weights; new head; freeze then fine-tune with a small LR; preprocess like pretraining; biggest gains when labels are scarce.

11.6 Data augmentation for images

What it is

Creating new training images by **label-preserving** transformations — a flipped, slightly rotated or recoloured cat is still a cat. More variety → less memorisation, more robustness to real-world variation.

How it works — common transforms (`torchvision.transforms.v2`)

Transform	What it does	Watch out
`RandomResizedCrop(224)`	Random crop of random size/aspect, resized	The standard ImageNet training transform
`RandomHorizontalFlip()`	Mirror left-right	Not for text, digits, or where left/right matters
`RandomRotation(15)`	Small rotations	Large rotations turn 6 into 9
`ColorJitter(0.4, 0.4, 0.4)`	Brightness, contrast, saturation	Not if colour is the label (ripe vs unripe fruit)
`RandomErasing()` / Cutout	Blank a random rectangle	Forces use of several object parts
`GaussianBlur`, noise	Blur or noise	Mimic camera quality
Mixup / CutMix	Blend two images (or paste a patch) and blend their one-hot labels in the same ratio	Strong regularizers for large models
RandAugment / AutoAugment / TrivialAugment	Automatically chosen transform policies	Good defaults in modern recipes

```
from torchvision.transforms import v2 as T   # (torchvision not needed for the rest of this chapter)
train_tf = T.Compose([T.RandomResizedCrop(224), T.RandomHorizontalFlip(), T.ColorJitter(0.2, 0.2, 0.2),
                      T.ToImage(), T.ToDtype(torch.float32, scale=True),
                      T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])])
test_tf = T.Compose([T.Resize(256), T.CenterCrop(224), T.ToImage(), T.ToDtype(torch.float32, scale=True),
                     T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])])
```

Augmentation runs **on the fly** in the training `Dataset`/`DataLoader`, differently each epoch — training only. Validation/test use a fixed deterministic resize/crop. **Test-time augmentation (TTA)** — averaging predictions over a few flipped/cropped copies — adds a little accuracy at extra inference cost. In §11.3, shift augmentation lifted shifted-digit accuracy from 86% to 98%. For **detection and segmentation**, geometric transforms must be applied to boxes/masks too (torchvision v2, Albumentations).

Common mistakes

- Randomly augmenting validation/test data.
- Label-changing augmentations (flipping digits — a mirrored 2 isn't a digit; mirroring road signs).
- Ignoring that augmentation is CPU-bound — use `num_workers` in `DataLoader` (parallel batch-preparing processes).

In the interview

30-second answer: "Random label-preserving transforms — crops, flips, colour jitter, small rotations, Mixup/CutMix — applied on the fly to training images so the model sees more variety and overfits less. Validation and test use a fixed resize and crop. For detection and segmentation the same geometric transform must hit the boxes and masks."

TL;DR

Random label-preserving transforms on training images only — the cheapest regularizer in vision.

11.7 Vision tasks: classification, detection, segmentation

What it is

Task	Question	Output	Typical models
Image classification	What is in this image?	One label per image	ResNet, EfficientNet, ViT
Object detection	What objects are where?	Box (x, y, w, h) + class + confidence per object	YOLO, SSD, Faster R-CNN, DETR
Semantic segmentation	Which class is each pixel?	A class per pixel (all people = "person")	U-Net, DeepLab, FCN
Instance segmentation	Which object does each pixel belong to?	A mask per object (person 1, person 2)	Mask R-CNN, YOLO-seg, SAM (promptable)

Object detection

IoU (Intersection over Union) = intersection area ÷ union area. Used to match predictions to ground truth (e.g. correct if IoU ≥ 0.5) and in NMS.

Worked example (`code/iou_nms.py`): ground truth (50, 50)–(150, 150), prediction (70, 60)–(170, 160). Each area 10,000. Intersection 80 × 90 = 7,200. Union 20,000 − 7,200 = 12,800. **IoU = 0.5625.**

NMS (non-maximum suppression) removes duplicate detections: sort by confidence; keep the top box; delete remaining boxes with IoU above a threshold (e.g. 0.5) with it; repeat.

```python
def my_nms(boxes, scores, thr):
    order = sorted(range(len(boxes)), key=lambda i: -scores[i])
    keep = []
    while order:
        best = order.pop(0)
        keep.append(best)
        order = [i for i in order if iou(boxes[best], boxes[i]) < thr]   # drop heavy overlaps
    return keep
```

```
IoU worked example: 0.562
pairwise IoU of the 5 boxes:
[[1.   0.89 0.65 0.   0.  ]
 [0.89 1.   0.69 0.   0.  ]
 [0.65 0.69 1.   0.   0.  ]
 [0.   0.   0.   1.   0.74]
 [0.   0.   0.   0.74 1.  ]]
NMS (IoU threshold 0.5) keeps boxes [0, 3] with scores [0.9, 0.8]
```

Five raw boxes around two objects → two detections.

Two families of detectors:

- **Two-stage** (R-CNN → Fast R-CNN → **Faster R-CNN**, 2015): a Region Proposal Network suggests candidate boxes; a second stage classifies and refines each. Accurate, slower.
- **One-stage** (**YOLO**, SSD, RetinaNet): all boxes in one forward pass. Real-time capable.

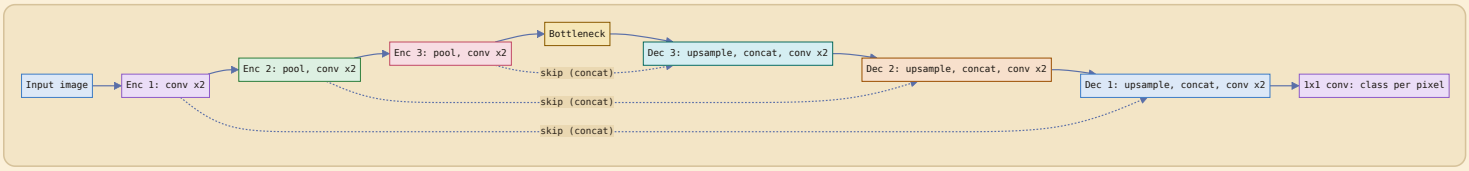
The YOLO idea ("You Only Look Once", Redmon et al., 2016): divide the image into an S × S grid; each cell predicts a few boxes (x, y, w, h, objectness confidence) and class probabilities for an object whose **centre** falls in that cell — one tensor from one CNN pass. Then threshold and NMS. The original ran at ~45 fps. Later versions (YOLOv3 onward, today's Ultralytics releases) added multi-scale predictions, **anchor boxes** (preset shapes that predictions adjust; later anchor-free heads) and better backbones — still the go-to for real-time detection. **DETR** (2020) uses a transformer to predict a set of boxes directly, removing NMS. Full detector-based system (face/plate blurring, where per-person **recall** matters more than mAP): case study 18.7.

Metric — mAP (mean average precision): area under the precision–recall curve per class, averaged over classes. PASCAL VOC uses IoU ≥ 0.5; COCO averages over IoU thresholds 0.5 to 0.95.

Segmentation — the U-Net idea

Per-pixel prediction needs to go down to a compact representation (*what*) and back up to full resolution (*where*). **U-Net** (Ronneberger et al., 2015, biomedical):

1. **Encoder (contracting path)**: conv blocks + downsampling — space shrinks, channels grow.
2. **Decoder (expanding path)**: **upsampling** with transposed convolutions (learned upsampling, roughly conv in reverse) back to input resolution.
3. **Skip connections**: each encoder feature map is **concatenated** onto the decoder's at the same level, restoring fine spatial detail lost in downsampling.
4. Final 1×1 conv gives per-pixel class scores; train with per-pixel cross-entropy, often plus **Dice loss** (overlap-based; copes with objects covering few pixels).



U-Nets also power image-generation **diffusion models** (generate by starting from noise and removing it step by step), e.g. Stable Diffusion v1/v2's denoiser.

Real-world examples

- **Classification**: photo search, retail product recognition, content moderation.
- **Detection**: self-driving perception, warehouse robots, shelf auditing, phone-camera face detection.
- **Segmentation**: tumour/organ outlines, video-call background removal, satellite land-use maps, portrait mode.

In the interview

30-second answer: "Classification gives one label per image; detection gives boxes plus classes; segmentation a label per pixel. Detection uses IoU to match boxes and NMS to remove duplicates; two-stage detectors like Faster R-CNN propose regions then classify, one-stage ones like YOLO predict all boxes in one pass over a grid — real-time. Segmentation uses encoder-decoders like U-Net whose skip connections restore spatial detail. Metrics: accuracy/top-k, mAP at IoU thresholds, IoU/Dice for masks."

- *Q: Why is NMS needed?* A: Detectors emit many overlapping boxes per object; NMS keeps the most confident and removes overlaps above an IoU threshold.
- *Q: In YOLO, which cell owns an object?* A: The cell containing its centre.

TL;DR

Detection → boxes (IoU, NMS; YOLO = one pass over a grid, Faster R-CNN = propose then classify; mAP). Segmentation → per-pixel labels (U-Net = encoder-decoder + skip connections).

11.8 Putting it together — trade-offs, mistakes, interview

When not to use a CNN

- Tabular data — use gradient-boosted trees (Chapter 7).
- Very small image datasets **without** a pretrained model — collect more data or use classical features.
- When a pretrained **foundation model** (large, broadly pretrained, reused for many tasks — e.g. a ViT or CLIP-style model; CLIP matches images to text) already solves it **zero-shot** (classes described in words, no task-specific training examples).

Common mistakes (whole chapter)

Training from scratch when a pretrained backbone exists; wrong tensor layout or normalisation; label-changing augmentations; judging a detector by accuracy instead of mAP.

In the interview — big picture

> **30-second answer:** "CNNs slide shared small kernels over the image — far fewer parameters and translation equivariance. Stacked 3×3 convs with downsampling build a feature hierarchy and a growing receptive field; global pooling plus a dense layer classifies. ResNet's skips enable very deep nets. In practice: start from a pretrained backbone, fine-tune with augmentation, and pick the head by task — classification, YOLO-style detection, or U-Net-style segmentation."

- *Q: First thing to try for a new 5,000-image classification problem?* A: Fine-tune a pretrained backbone (ResNet-50, EfficientNet) with augmentation.
- *Q: What does global average pooling replace?* A: The large flattened dense layers at the end of older CNNs.

Chapter TL;DR

- **Convolution**: shared k×k kernels over all input channels; out = ⌊(n + 2p − k)/s⌋ + 1; params = k·k·C_in·C_out + C_out.
- **Pooling**: 2×2 max-pool halves H and W, no parameters; GAP before the classifier.
- **Architecture**: conv-BN-ReLU blocks, shrinking space and growing channels, then GAP + dense; stacked 3×3 convs grow the RF cheaply.
- **History**: LeNet → AlexNet → VGG → Inception → **ResNet (skip connections)** → MobileNet/EfficientNet → ViT.
- **Transfer learning**: pretrained backbone + new head; freeze, then fine-tune with a small LR.
- **Augmentation**: label-preserving random transforms, training only.
- **Detection**: IoU, NMS, YOLO, Faster R-CNN, mAP. **Segmentation**: U-Net.

Next: Chapter 12 — sequences: RNNs, LSTMs and attention.

Chapter 12 — Sequences: RNNs, LSTMs and Attention

Text, speech, prices, sensor readings, click streams, DNA — much data comes as **sequences**, where order carries meaning ("dog bites man" ≠ "man bites dog"). Chapters 10–11 take a fixed-size input at once; sequence models read **step by step** (RNNs) or compare **all positions against each other** (attention), handling variable lengths and dependencies across time.

The chapter follows history, each step fixing the last one's weakness: RNNs → vanishing gradients → LSTM/GRU gates → seq2seq → the bottleneck → attention → self-attention and the transformer. Plus tokenization, word embeddings and an introduction to time-series forecasting. Runnable PyTorch: an LSTM on two toy tasks, seq2seq with and without attention, a tiny word2vec. The step-by-step transformer is in the AI Handbook, Chapter 3 — Transformers and Embeddings.

All code is in `code/`, runs on CPU in seconds; every output shown is real.

12.1 Sequence data

What it is

A **sequence** is an ordered list x_1, x_2, …, x_T where position matters and T varies. Each element (**time step** or **token**) may be a number, a vector (a video frame's features) or a symbol (a word).

Data	Element	Typical task
Text	word / subword token	classification, translation, generation
Speech	audio frame (10–25 ms)	speech recognition
Time series	measurement per time unit	forecasting, anomaly detection
User events	click, purchase	next-item recommendation, churn
Biology	nucleotide / amino acid	structure and function prediction

Why ordinary networks struggle

- **Variable length**: an MLP needs a fixed input size.
- **Order and context**: "bank" depends on nearby words; a bag of features loses order.
- **Parameter sharing across time**: "not good" should be recognised wherever it appears, like a CNN kernel shared across positions.

Input/output patterns

Pattern	Example
One-to-many	Image captioning
Many-to-one	Sentiment classification, "is this transaction sequence fraud?", next-value forecast
Many-to-many, aligned	Part-of-speech tagging, per-frame video labels
Many-to-many, not aligned (seq2seq)	Translation, summarisation, speech-to-text

TL;DR

Ordered, variable-length data; models must share parameters across positions and carry context along.

12.2 Recurrent neural networks (RNNs)

What it is

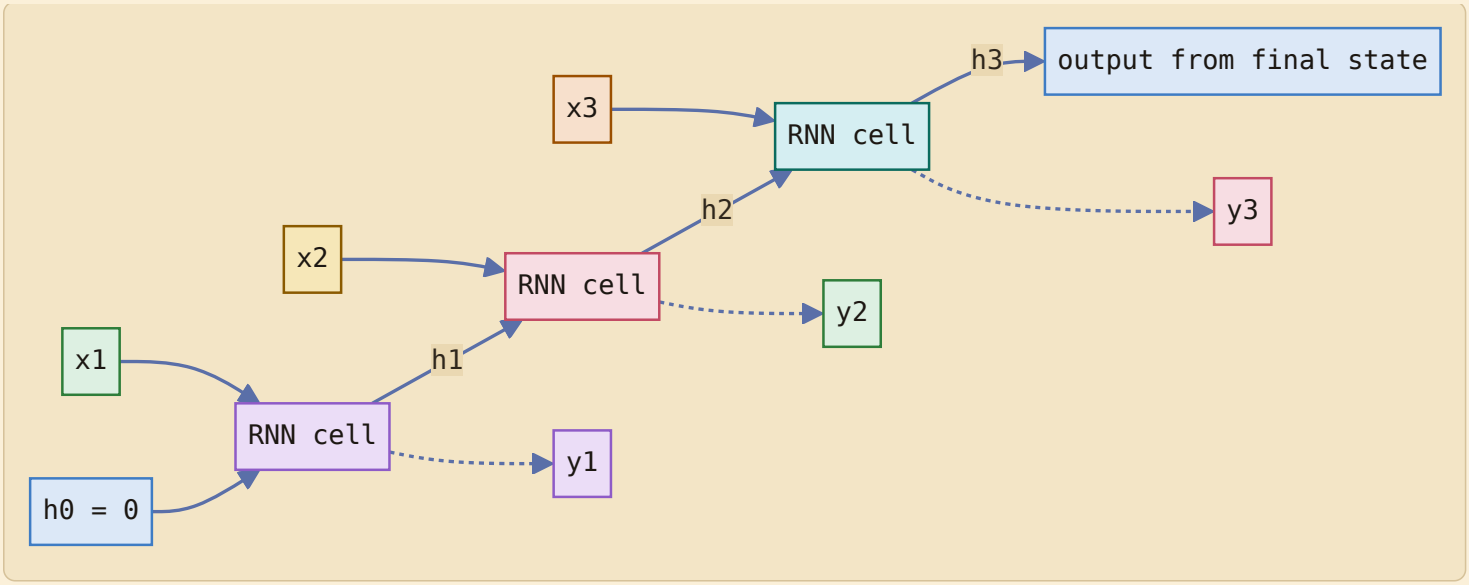
A **recurrent neural network** reads one step at a time and keeps a **hidden state** h_t — a vector summarising everything read so far — updated with the **same weights** at every step:

h_t = tanh(W_xh x_t + W_hh h_{t−1} + b_h) **ŷ_t = W_hy h_t + b_y** (an output, if the task needs one at this step)

tanh squashes into (−1, 1); ŷ is the prediction.

How it works — unrolling

"Unrolling" draws the loop as a chain of cell copies, one per step, all sharing W_xh, W_hh, b_h (Elman, 1990, the "simple RNN").



Worked numbers (`code/rnn_by_hand.py`) — 1 input, 2 hidden units, W_xh = [0.5, −0.3]ᵀ, W_hh = [[0.4, 0.1], [−0.2, 0.3]], b = 0, inputs 1.0, 0.5, −1.0:

t	x_t	W_xh x_t + W_hh h_{t−1}	h_t = tanh(…)
1	1.0	(0.5, −0.3) + 0	(0.4621, −0.2913)
2	0.5	(0.25, −0.15) + (0.4·0.4621 + 0.1·(−0.2913), −0.2·0.4621 + 0.3·(−0.2913)) = (0.4057, −0.3298)	(0.3848, −0.3184)
3	−1.0	(−0.3779, 0.1275)	(−0.3609, 0.1268)

```
torch.nn.RNN final hidden: [-0.3609  0.1268]
```

PyTorch's `nn.RNN` with the same weights agrees. **Parameters don't grow with length**: 2 + 4 + 2 = 8 numbers handle sequences of any length (1,000 steps too).

Backpropagation through time (BPTT)

Unroll over the sequence, sum the loss over steps with targets, and run ordinary backprop through the unrolled graph. Because weights are shared, each weight's gradient is the **sum** over time steps: $\partial L/\partial W_{hh} = \Sigma_t$ ($\partial L/\partial h_t$ · local derivative at t), where $\partial L/\partial h_t$ collects contributions from all later steps. **Truncated BPTT** backpropagates only k steps (e.g. 100) while still carrying the hidden state forward — cheaper, but can't learn dependencies longer than k.

Variants

- **Stacked (deep) RNNs**: one layer's hidden-state sequence is the next layer's input (`num_layers=2`).
- **Bidirectional RNNs**: one RNN left-to-right, one right-to-left, states concatenated — each position sees past and future. Good for tagging/classification; impossible for real-time generation (the future isn't known).

PyTorch: `nn.RNN(input_size, hidden_size, num_layers=1, batch_first=True, bidirectional=False)`; input (batch, time, features); returns `output` (every step's top-layer h) and `h_n` (final state per layer).

TL;DR

Same cell at every step carrying a hidden state; unroll and backprop through time; shared weights = fixed parameter count for any length.

12.3 Vanishing and exploding gradients through time

The problem

The gradient of a late loss w.r.t. an early hidden state is a **product of T Jacobians** (matrix of how each output changes with each input), one per step: $\partial h_T/\partial h_0 = \prod_t \text{diag}(\tanh'(\cdot)) \cdot W_{hh}$. Like depth in Chapter 10, but "depth" is sequence length — often hundreds. If W_hh's largest singular value (how much it stretches vectors) combined with tanh' ≤ 1 is below 1, gradients shrink **exponentially** with distance; above 1 they can grow exponentially (Bengio, Simard & Frasconi, 1994). In practice a simple RNN struggles beyond a few tens of steps.

`code/rnn_by_hand.py` measures $\|\partial h_T/\partial h_0\|$ in a 32-unit tanh RNN for different recurrent-weight scales:

```
scale 0.5: |d h_T / d h_0| at T=1: 2.64e+00  T=10: 6.51e-03  T=60: 3.54e-17
scale 1.0: |d h_T / d h_0| at T=1: 5.13e+00  T=10: 5.22e+00  T=60: 2.58e+01
scale 1.5: |d h_T / d h_0| at T=1: 9.44e+00  T=10: 2.15e+01  T=60: 1.90e+02
scale 3.0: |d h_T / d h_0| at T=1: 1.60e+01  T=10: 3.90e+02  T=60: 1.29e+07
```

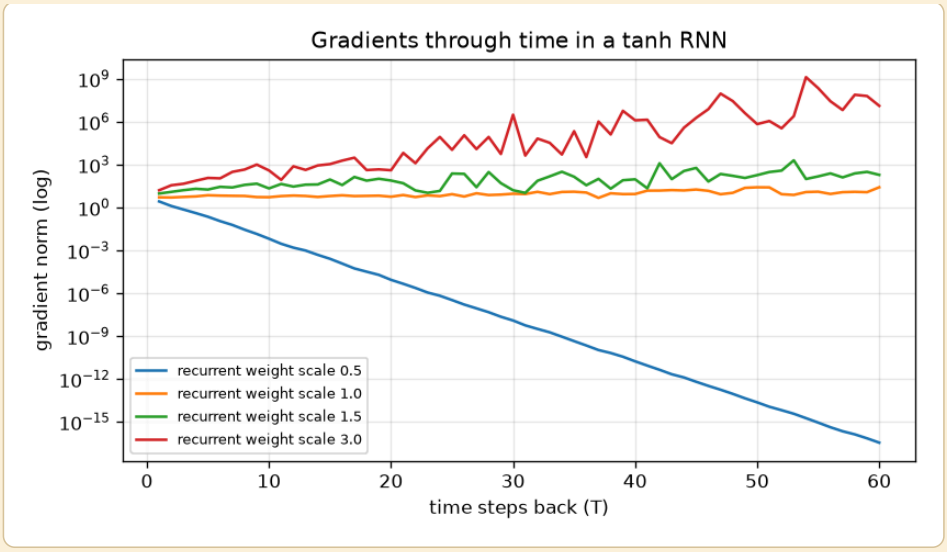


Figure — Small recurrent weights: the gradient from 60 steps back is ~10^{-17} (vanished); large: 10^{7} (exploded). Only a narrow band between is usable, and training rarely stays there.

Fixes

Problem	Fix
Exploding	**Gradient clipping**: rescale the gradient when its norm exceeds a threshold — `torch.nn.utils.clip_grad_norm_(params, 1.0)`. Standard for recurrent models and transformers. Does **not** fix vanishing.
Vanishing	**Gated cells (LSTM, GRU)** with an additive memory path; careful init (orthogonal W_hh, which preserves vector lengths); shorter paths via **attention** or skip connections.

In the interview

30-second answer: "BPTT multiplies one Jacobian per time step, so gradients from distant steps shrink or grow exponentially — vanishing or exploding, exactly like depth. Exploding is fixed by gradient clipping. Vanishing is fixed architecturally: LSTM/GRU gates create an additive path, and attention connects every position directly."

TL;DR

Gradients through time are products of per-step factors → vanish or explode. Clip for exploding; gates or attention for vanishing.

12.4 LSTM and GRU

What it is

Long Short-Term Memory (Hochreiter & Schmidhuber, 1997; forget gate added by Gers et al., 2000) adds a separate **cell state** c_t — a memory "conveyor belt" — and three **gates** deciding what to erase, write and reveal. A **gate** is a vector in (0, 1) (from a sigmoid) multiplied element-wise into another vector: 0 = block, 1 = pass — a soft switch.

How it works — LSTM equations

[h_{t−1}, x_t] = previous hidden state and current input stacked; σ = sigmoid; ⊙ = element-wise product.

Step	Equation	Meaning
Forget gate	f_t = σ(W_f [h_{t−1}, x_t] + b_f)	Which memory entries to keep (1) or erase (0)
Input gate	i_t = σ(W_i [h_{t−1}, x_t] + b_i)	Which entries to update
Candidate	*c̃t = tanh(W_c [h*{t−1}, x_t] + b_c)	Proposed new content, in (−1, 1)
Cell update	**c_t = f_t ⊙ c_{t−1} + i_t ⊙ c̃_t**	Keep part of old memory, add part of new
Output gate	o_t = σ(W_o [h_{t−1}, x_t] + b_o)	Which parts of memory to expose
Hidden state	h_t = o_t ⊙ tanh(c_t)	What the rest of the network sees

Why it fixes vanishing gradients: the cell update is **additive**. Along the cell path ∂c_t/∂c_{t−1} = f_t — no repeated tanh' squashing or W_hh multiplication. With f near 1, information and gradients flow across many steps almost unchanged (the "constant error carousel"); the network *learns* when to remember and forget.

One unit: previous c = 2.0; f = 0.9, i = 0.2, c̃ = 0.5 → c_new = 0.9·2.0 + 0.2·0.5 = 1.9 (mostly kept). With o = 0.7, h = 0.7·tanh(1.9) = 0.7·0.956 = 0.669.

An LSTM has **4 weight blocks** (f, i, c̃, o) — about 4× a simple RNN's parameters at the same width.

GRU

The **Gated Recurrent Unit** (Cho et al., 2014) merges cell and hidden state, with two gates:

Step	Equation	Meaning
Update gate	z_t = σ(W_z [h_{t−1}, x_t] + b_z)	How much to replace the old state
Reset gate	r_t = σ(W_r [h_{t−1}, x_t] + b_r)	How much old state to use when proposing new content
Candidate	*h̃t = tanh(W_h [r_t ⊙ h{t−1}, x_t] + b_h)*	Proposed new state
New state	h_t = (1 − z_t) ⊙ h_{t−1} + z_t ⊙ h̃_t	Interpolate old and new

(Cho et al. and PyTorch swap z and 1 − z — same model, gate meaning flipped; PyTorch also applies r after multiplying h_{t−1} by its weights.) 3 weight blocks — fewer parameters than an LSTM, often similar accuracy.

Code — an LSTM on two toy tasks

`code/lstm_toy.py`.

A) Forecasting a noisy signal. Read the last 30 values of two summed sine waves plus noise; predict the next (many-to-one regression). Train on the first 900 windows, test on the **later** part (never shuffle time across the train/test boundary).

```
class Forecaster(nn.Module):
    def __init__(self, hidden=32):
        super().__init__()
        self.lstm = nn.LSTM(input_size=1, hidden_size=hidden, batch_first=True)  # (batch, time, feat)
        self.head = nn.Linear(hidden, 1)

    def forward(self, x):                       # x: (batch, 30, 1)
        out, (h_n, c_n) = self.lstm(x)          # out: every step's h; h_n/c_n: final hidden & cell state
        return self.head(out[:, -1])            # predict from the last hidden state
```

B) Long memory. Read T random symbols (1–8) and output the **first** — the answer must survive T steps of distraction, isolating vanishing gradients. Each model embeds symbols (`nn.Embedding`, a learned id → vector table), runs a 32-unit RNN/LSTM/GRU, classifies from the last hidden state; gradients clipped at norm 1. The LSTM's forget-gate bias is initialised to 1 ("remember by default", Jozefowicz et al., 2015) — f ≈ 0.73 at start, so memory and gradients persist early in training.

```
if cell == "LSTM":                                  # forget-gate bias = 1: "remember by default"
    with torch.no_grad():                           # PyTorch gate order: input, forget, cell, output
        self.rnn.bias_ih_l0[hidden:2 * hidden].fill_(1.0)
        self.rnn.bias_hh_l0[hidden:2 * hidden].fill_(0.0)
```

```
A) LSTM test MSE 0.0190  vs naive last-value MSE 0.0442  (noise variance = 0.0100)
B) recall the first of T symbols (test accuracy, chance 0.125)
    T     RNN    LSTM     GRU
    5   1.000   1.000   1.000
   15   0.505   1.000   1.000
recurrent-layer parameters (hidden 32): {'RNN': 1600, 'LSTM': 6400, 'GRU': 4800}
saved img/lstm_sine_forecast.png
```

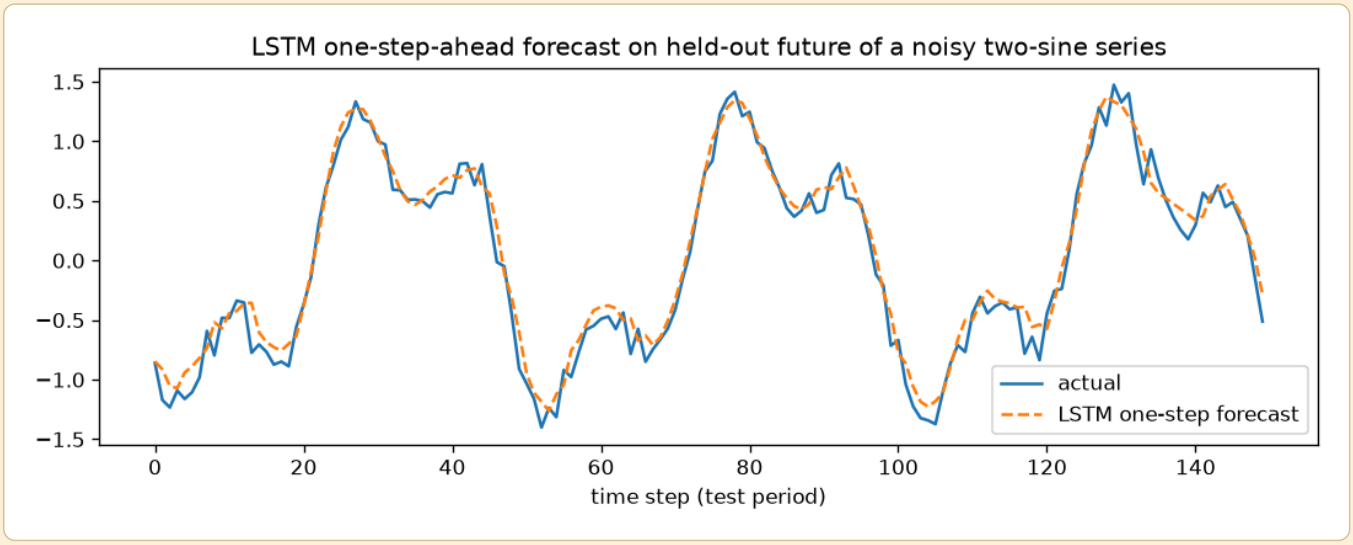


Figure — One-step-ahead LSTM forecasts on the held-out future. MSE 0.019 is under half the naive "next = last" baseline and approaches the noise floor (0.010), which no model can beat.

Part B: at 5 steps all three cells solve it; at 15 the plain RNN stalls near 50% (the gradient back to the first symbol vanished) while LSTM and GRU hit 100%. With much longer gaps even gated cells need more training — where attention shines. Parameter counts confirm the 1 : 4 : 3 ratio (16·32 + 32·32 + 2·32 = 1,600 for the RNN).

Trade-offs

	Simple RNN	LSTM	GRU	Transformer (12.7)
Long-range memory	Poor (a few tens of steps)	Good	Good	Excellent (direct connections)
Parameters (same width)	1×	4×	3×	Different design
Training parallelism across time	None	None	None	Full
Inference memory, long sequences	Constant	Constant	Constant	Grows with length (KV cache)
Typical use today	Teaching	Small/streaming, time series, on-device	Same as LSTM	NLP, vision, audio, most large models

When not to use it

- Large-scale NLP: transformers win on quality and trainability (parallel over time).
- Tabular time series with good lag features: gradient-boosted trees are often better and simpler (12.10).

Common mistakes

- Forgetting `batch_first=True` and feeding (batch, time, features) to a layer expecting (time, batch, features).
- `out[:, -1]` on **padded** batches — the last position may be padding. Pack (`nn.utils.rnn.pack_padded_sequence`) or index each sequence's real final step.
- Not clipping gradients.
- Shuffling time series across the train/test boundary (**leakage**: future information inflates scores).

Real-world examples

LSTMs powered Google voice-search recognition (2015), Google Neural Machine Translation (2016) and handwriting recognition before transformers; still common on-device, in streaming, and in industrial forecasting and anomaly detection.

In the interview

> **30-second answer:** "An LSTM adds a cell state updated additively — c_t = f ⊙ c_{t−1} + i ⊙ c̃ — controlled by sigmoid forget, input and output gates. The cell path isn't repeatedly squashed or multiplied by the recurrent matrix, so gradients survive many steps. A GRU does the same with update and reset gates and no separate cell — lighter. Both are sequential, which is why transformers replaced them for large-scale NLP."

- *Q: Forget gate, and why bias 1?* A: It scales the old cell state; bias 1 gives f ≈ 0.73 so memory persists by default early on.
- *Q: LSTM vs GRU?* A: GRU is simpler (3 vs 4 blocks), faster; accuracy usually similar — try both.
- *Q: Why can't you parallelise an RNN over time?* A: h_t depends on h_{t−1}.

TL;DR

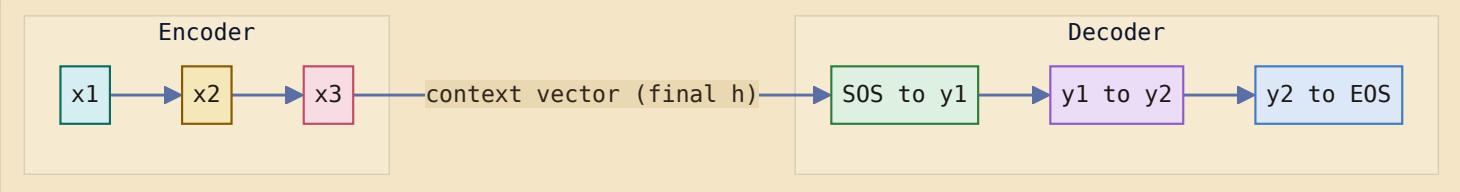
LSTM = cell state + forget/input/output gates, additive memory → long-range dependencies. GRU = update/reset gates, lighter. Clip gradients; forget-bias 1.

12.5 Sequence-to-sequence (encoder-decoder) models

What it is

Seq2seq (Sutskever, Vinyals & Le; Cho et al.; both 2014) maps an input sequence to an output of *different* length:

1. **Encoder**: an RNN reads the input, ending in a fixed-size hidden state — the **context vector**.
2. **Decoder**: another RNN initialised with it generates output tokens one at a time, each step fed the previous token, until end-of-sequence.



How it works

- **Teacher forcing** (training): the decoder input at step t is the **true** previous token (target shifted right, starting with SOS), not its own prediction — fast, stable. Cross-entropy loss at every output step.
- **Decoding** (inference) feeds back the model's own predictions:
  - **Greedy**: most likely token each step.
  - **Beam search**: keep the k most probable partial sequences, extend, keep the best k. Finds higher-probability outputs; standard in translation. k = 1 is greedy.

    - **Sampling** (temperature, top-k, top-p): for creative generation (AI Handbook, Chapter 4).
- **Exposure bias**: trained on perfect prefixes, tested on its own possibly wrong ones — errors compound. Scheduled sampling (gradually feeding its own predictions in training) mitigates.

The bottleneck problem

The whole input is squeezed into one fixed-size vector; early information is lost and quality drops sharply with sentence length. Sutskever et al. found **reversing the source** helped (shorter distance between first source and first target words) — a hint that short paths matter. The real fix is attention.

In the interview

> **30-second answer:** "An encoder RNN reads the input into a context vector; a decoder RNN generates the output token by token from it. Training uses teacher forcing; inference uses greedy or beam search on the model's own outputs. The weakness is the fixed-size bottleneck, which attention removes."

TL;DR

Encoder → context vector → decoder; teacher forcing to train, greedy/beam to decode; the bottleneck motivates attention.

12.6 The attention mechanism

What it is

Attention (Bahdanau, Cho & Bengio, 2014) lets the decoder, at every output step, **look back at all encoder states** and take a relevance-weighted average — a fresh, targeted context vector per step instead of one summary.

How it works — queries, keys and values

A soft dictionary lookup:

- **Query (q)**: what I'm looking for (the decoder's current state).
- **Keys (k)**: a label per findable item (one per encoder position).
- **Values (v)**: the content returned (often the same encoder states).

1. **Score** each key against the query. Dot-product (Luong, 2015): q·k. Additive (Bahdanau): $v^T \tanh(W_1 q + W_2 k)$. Transformers: **scaled dot-product** q·k / √d_k.
2. **Softmax** scores into positive weights summing to 1.
3. **Output** = Σ $\text{weight}_i \cdot v_i$.

Matrix form: **Attention(Q, K, V) = softmax(Q K^T / √d_k) V**, d_k = key length.

Worked example with small numbers

Three tokens ("the", "cat", "sat"), d_k = 2 (in a real model Q, K, V = token vectors × learned W_Q, W_K, W_V; here picked directly). Q = K = [[1, 0], [0, 1], [1, 1]], V = [[1, 2], [3, 4], [5, 6]].

Query 1 (q = [1, 0]): dot products (1, 0, 1); ÷√2 → (0.707, 0, 0.707); softmax: e^0.707 = 2.028, e^0 = 1, sum 5.056 → (0.401, 0.198, 0.401); output = 0.401·[1, 2] + 0.198·[3, 4] + 0.401·[5, 6] = [3.0, 4.0].

All three queries (`code/attention_by_hand.py`):

```
scores = Q K^T / sqrt(2):
 [[0.707 0.    0.707]
 [0.    0.707 0.707]
 [0.707 0.707 1.414]]
attention weights (softmax per row):
 [[0.401 0.198 0.401]
 [0.198 0.401 0.401]
 [0.248 0.248 0.503]]
output = weights @ V:
 [[3.    4.   ]
 [3.407 4.407]
 [3.51  4.51 ]]
matches torch.nn.functional.scaled_dot_product_attention: True
```

```python
def attention(Q, K, V, mask=None):
    d_k = Q.shape[-1]
    scores = Q @ K.T / np.sqrt(d_k)                 # similarity of every query with every key
    if mask is not None:
        scores = np.where(mask, scores, -np.inf)    # hide disallowed positions
    weights = softmax(scores)                       # each row sums to 1
    return weights @ V, scores, weights             # weighted average of values
```

Why divide by √d_k? For random unit-variance vectors, the dot product's standard deviation grows like √d_k. Large scores push softmax into a nearly one-hot, **saturated** regime where score changes barely move the output and gradients are tiny. Scaling keeps scores near unit variance:

```
d_k=  4: std of q.k =   1.99   std after / sqrt(d_k) = 1.00
d_k= 64: std of q.k =   8.05   std after / sqrt(d_k) = 1.01
d_k=512: std of q.k =  22.44   std after / sqrt(d_k) = 0.99
```

Attention fixes seq2seq — runnable demo

`code/seq2seq_attention.py` trains two GRU encoder-decoders (1,500 steps, teacher forcing) to **reverse a 12-digit sequence**; one decoder sees only the final encoder state, the other also attends over all encoder states (query = decoder state, keys = values = encoder states).

```
def forward(self, src, tgt_in):
    enc_out, h = self.encoder(self.src_emb(src))               # enc_out: (B, L, H) - one vector per input token
    dec_out, _ = self.decoder(self.tgt_emb(tgt_in), h)         # decoder starts from the encoder's summary h
    if not self.use_attention:
        return self.out(dec_out), None
    scores = dec_out @ enc_out.transpose(1, 2) / H ** 0.5      # (B, L_tgt, L_src): query=decoder, keys=encoder
    weights = scores.softmax(-1)
    context = weights @ enc_out                                # weighted sum of encoder states
    return self.out(torch.cat([dec_out, context], -1)), weights
```

```
no attention  : token accuracy 0.669, whole-sequence accuracy 0.156
   source [7, 7, 5, 1, 0, 0, 0, 3, 6, 9, 0, 0]
   output [0, 0, 9, 6, 0, 3, 0, 3, 1, 5, 0, 7]
with attention: token accuracy 0.991, whole-sequence accuracy 0.980
   source [7, 7, 5, 1, 0, 0, 0, 3, 6, 9, 0, 0]
   output [0, 0, 9, 6, 3, 0, 0, 0, 1, 5, 7, 7]
   where each output step looks (argmax of attention over source positions): [11, 11, 10, 9, 8, 7, 6, 6, 4, 3, 2, 1]
```

Without attention the first few outputs (recent inputs) are right, then garbled — the bottleneck. With attention 98% of sequences are perfect, and the pattern is interpretable: the peak moves from source position 11 toward 0, a reversed diagonal — the model **learned to read backwards**. It isn't always exactly on the copied digit: identical neighbours (the 0s at 10–11 and 4–6) are interchangeable, and in the last steps it sits one position right (looks at 4, 3, 2, 1 while copying 3, 2, 1, 0) — fine, because a left-to-right GRU's state at j + 1 has just read x_j.

In the interview

30-second answer: "Attention scores a query against every key, softmaxes the scores into weights, and returns the weighted average of the values: softmax($QK^T/\sqrt{d_k}$)V. In seq2seq the decoder state is the query and encoder states are keys and values, so each output step gets its own context. Dividing by $\sqrt{d_k}$ keeps scores at unit variance so softmax doesn't saturate."

- *Q: Additive vs dot-product attention?* A: Additive scores with a small MLP; dot-product is a matrix multiply — faster and, scaled, as good.

TL;DR

Attention = softmax($QK^T/\sqrt{d_k}$)·V — a learned, **differentiable** weighted lookup. It removes the seq2seq bottleneck and gives interpretable alignments.

12.7 Self-attention and the transformer (high level)

What it is

In **self-attention**, Q, K and V come from the **same** sequence (three learned projections): every token attends to every token of its own sentence ("…because **it** was too tired" — "it" draws from "animal"). The **transformer** (Vaswani et al., 2017, "Attention Is All You Need") drops recurrence and stacks self-attention with position-wise **feed-forward** layers.

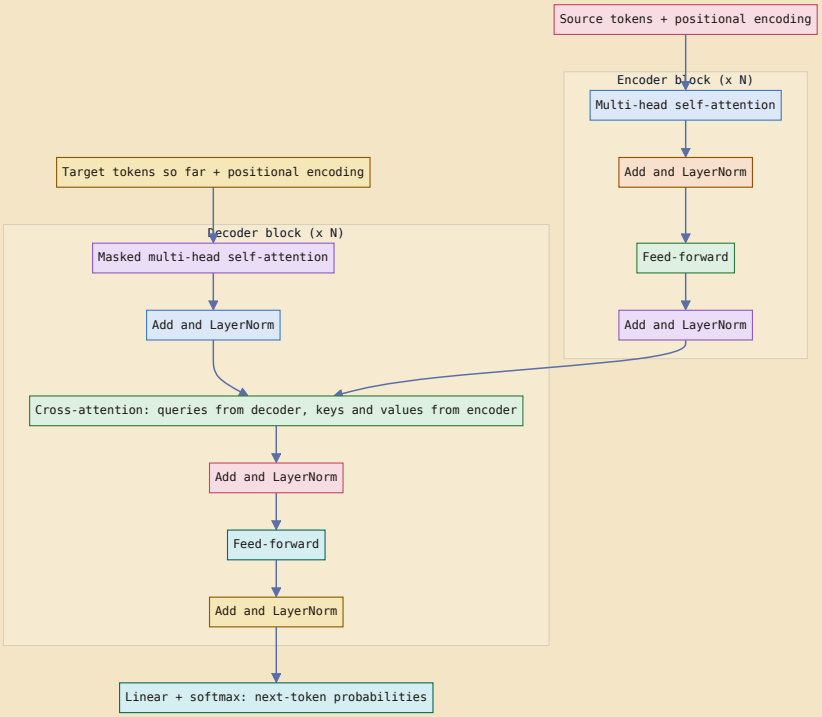
Why it matters

- **Parallel** over positions — trains efficiently on GPUs over huge data.
- **Short paths**: any two positions are one step apart — no vanishing through time.
- **Scales**: BERT, GPT, T5, Claude, Gemini, LLaMA, Whisper (speech), ViT (Chapter 11).

How it works — the building blocks

1. **Token embeddings** of size d_model (512 originally).
2. **Positional encoding**: self-attention is **permutation-equivariant** (shuffle inputs → outputs shuffle the same way), so without positions "dog bites man" = "man bites dog". Original: fixed sinusoids PE(pos, 2i) = sin(pos / 10000^(2i/d)), PE(pos, 2i+1) = cos(pos / 10000^(2i/d)); later learned positions or **RoPE** (rotate query/key vectors by position).
3. **Multi-head attention**: h heads in parallel, each projecting to d_model/h dims with its own W_Q, W_K, W_V; outputs concatenated and projected. Each head captures different relations in its own subspace, at the same total cost as one full-width head.
4. **Feed-forward network**: 2-layer MLP per position (typically 4× wider inside).
5. **Residual connections + layer normalization** (each token's vector rescaled to zero mean, unit variance) around each sub-layer (the ResNet skip idea, Chapter 11).
6. Stack N blocks (original: 6 encoder + 6 decoder, 8 heads).

Each piece is worked through with code in the AI Handbook, Chapter 3 (§3.3 positions, §3.5 multi-head, §3.6 the block, §3.7 causal masking, §3.8 the three families).



- **Encoder**: bidirectional — each token sees the whole input.
- **Decoder**: **masked (causal)** self-attention — token i attends only to ≤ i; needed because training processes all positions at once and must not peek at the tokens being predicted — plus **cross-attention** (queries from decoder, keys/values from encoder: the 12.6 seq2seq attention).
- Families: **encoder-only** (BERT — understanding), **decoder-only** (GPT, Claude, LLaMA — generation), **encoder-decoder** (original, T5, Whisper — translation, summarisation, speech-to-text).

Causal masking on the worked example (upper triangle → −∞ before softmax):

```
causal (decoder) weights:
 [[1.    0.    0.   ]
 [0.33  0.67  0.   ]
 [0.248 0.248 0.503]]
causal output matches torch (is_causal=True): True
MultiheadAttention: input (2, 5, 16) -> output (2, 5, 16) | averaged attention weights (2, 5, 5)
positional encoding rows 0 and 1 (first 4 dims): [0. 1. 0. 1.] [0.841 0.54  0.682 0.732]
```

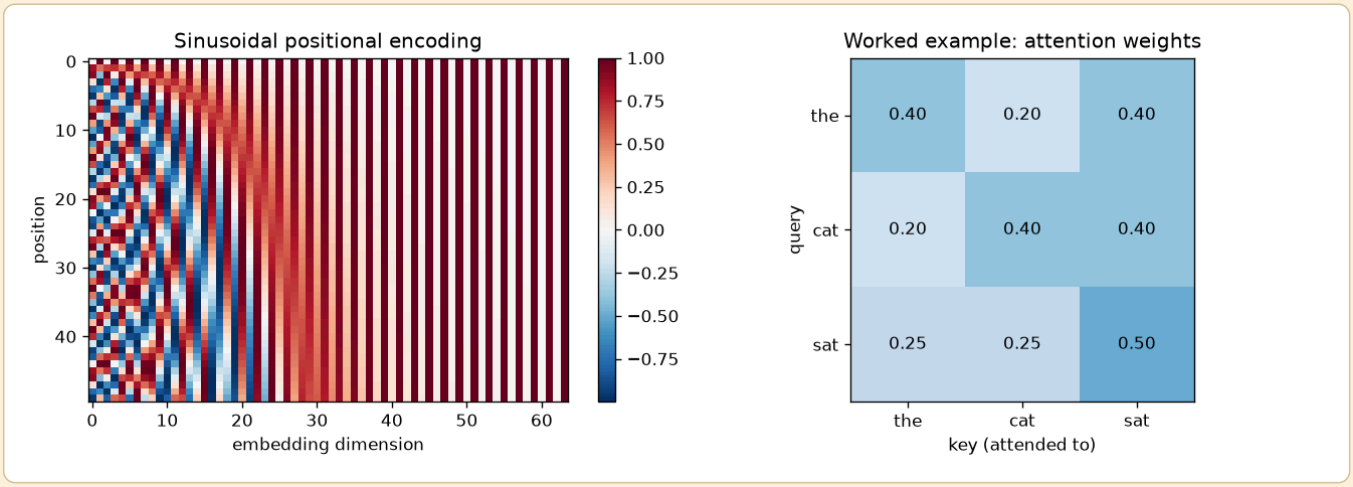


Figure — Left: sinusoidal encodings, 50 positions × 64 dims; low dims vary fast, high ones slowly. Right: the worked example's attention weights.

Trade-offs

	RNN/LSTM	Transformer
Compute per layer (big-O)	O(n · d²)	O(n² · d) for attention — quadratic in length n
Parallel over sequence	No	Yes
Path length between distant tokens	O(n)	O(1)
Memory at inference	Constant state	Key/value cache grows with context

Long contexts are expensive (n^2): FlashAttention computes exact attention faster with less memory; sparse/linear attention and **state-space models** (recurrent-style, fixed-size state; e.g. Mamba) trade off differently.

Where next: AI Handbook Chapter 3 builds the transformer step by step; Chapter 4 covers training objectives, scaling, fine-tuning and **RLHF** (reinforcement learning from human feedback). This book stops at ML foundations.

In the interview

> **30-second answer:** "Self-attention lets each token take a weighted average of all tokens' values, weights from softmax of query-key dot products scaled by √d. A transformer stacks multi-head self-attention and position-wise feed-forward layers with residuals and layer norm, adding positional encodings because attention is order-agnostic. Encoders attend bidirectionally; decoders use causal masks and cross-attention. It replaced RNNs because it's parallel and connects any two positions in one step; cost is quadratic in length."

- *Q: Complexity in sequence length n?* A: $O(n^2)$ time and memory for the score matrix.
- *Q: GPT? BERT?* A: Decoder-only; encoder-only.

TL;DR

Embeddings + positional encoding + N × (multi-head self-attention + feed-forward, each with residual + LayerNorm). Encoder bidirectional; decoder causal + cross-attention. Parallel, short paths, $O(n^2)$.

12.8 Text preprocessing and tokenization

What it is

Tokenization splits text into **tokens** and maps each to an integer id from a **vocabulary**; an embedding layer then maps ids to vectors.

How it works — a classic pipeline

1. **Normalise**: lowercase (sometimes), Unicode normalisation, replace URLs/numbers with placeholders, strip markup (e.g. HTML tags).
2. **Tokenize**.
3. **Build a vocabulary** from training data (frequency cut-off) plus special tokens `<pad>`, `<unk>` (out-of-vocabulary), and `<sos>`/`<eos>` for seq2seq.
4. **Encode** tokens → ids; unknowns → `<unk>`.
5. **Pad/truncate** to a fixed length for batching; mask the padding.

Classic NLP also used **stop-word removal** ("the", "a") and **stemming/lemmatization** ("running" → "run"). These help **bag-of-words** models (word counts, order ignored) such as **TF-IDF** (counts down-weighted for words common in every document) + logistic regression, but usually **not** neural models — and "not" is a stop word you must keep for sentiment.

```python
def normalize(text):
    text = text.lower()
    text = re.sub(r"https?://\S+", "<url>", text)          # replace URLs with a placeholder
    return re.findall(r"<url>|[a-z0-9]+(?:'[a-z]+)?|[.,!?]", text)

def encode(tokens, stoi, max_len):
    ids = [stoi.get(t, stoi["<unk>"]) for t in tokens][:max_len]   # unknown words -> <unk>, truncate
    return ids + [stoi["<pad>"]] * (max_len - len(ids))            # pad to a fixed length
```

Output (`code/text_and_embeddings.py`):

```
'The cat sat on the mat!'                          -> ['the', 'cat', 'sat', 'on', 'the', 'mat', '!']
"Dogs don't like cats, see https://example.com"    -> ['dogs', "don't", 'like', 'cats', ',', 'see', '<url>']
'The mat is red.'                                  -> ['the', 'mat', 'is', 'red', '.']
vocab size 18 | encode('the dog sat') = [17, 1, 15, 0, 0, 0]
characters of 'mat': ['m', 'a', 't'] | bytes of 'café' (UTF-8): [99, 97, 102, 195, 169]
```

"dog" was never seen (only "dogs") → id 1 = `<unk>`: the weakness of word-level vocabularies.

Word vs character vs subword tokens

Level	Vocabulary	Sequence length	Unknown words	Used by
Word	Huge (100k+), still misses rare words and typos	Short	`<unk>` problem	Classic NLP, word2vec
Character / byte	Tiny (≈100 chars, 256 bytes)	Very long	None	Some specialised models (ByT5)
Subword (BPE, WordPiece, Unigram)	Medium (≈30k–200k)	Medium	Rare words split into known pieces	Essentially all modern LLMs

Byte-Pair Encoding (BPE) (for NLP: Sennrich et al., 2016): start from characters; repeatedly merge the **most frequent adjacent pair** in the training corpus into a new symbol; stop after a set number of merges (= vocabulary size). Frequent words become single tokens; rare words split into pieces ("lowest" → "low" + "est").

```
BPE merges learned: ['lo', 'low', 'er', 'er</w>', 'low</w>', 'es']
words as subword tokens: ['low er</w>', 'low es t </w>', 'low</w>', 'n e w er</w>', 'n e w es t </w>', 'w i d er</w>', 'w i d es t </w>']
```

(`</w>` = end of word.) After six merges "low" is one token and "er", "es" are emerging. GPT-2 onward use **byte-level BPE**: start from the 256 byte values 0–255 (UTF-8 encodes each character as 1–4 bytes), so any string — any language, emoji, code — encodes with no `<unk>`. BERT uses **WordPiece**; T5 and many multilingual models **SentencePiece** (Unigram or BPE). Rule of thumb: one token ≈ ¾ of an English word (≈4 characters) — why API prices and **context windows** (max tokens a model reads at once) are quoted in tokens.

Common mistakes

- Building the vocabulary (or fitting a tokenizer) on validation/test text.
- A different tokenizer at inference than training — with a pretrained model always load its own.
- Removing stop words/negations for neural sentiment models.
- Forgetting attention/padding masks, so the model "reads" padding.

TL;DR

Normalise → tokenize → vocabulary with `<pad>`/`<unk>` → ids → pad. Modern models use subword tokenizers (BPE/WordPiece/SentencePiece) — no out-of-vocabulary problem.

12.9 Word embeddings

What it is

A **word embedding** is a dense vector (100–300 numbers) such that words used in similar contexts get similar vectors. It replaces **one-hot encoding** (a 50,000-long vector with one 1) — huge, sparse, and every word equally different ("cat" as far from "kitten" as from "carburettor").

Distributional hypothesis: "you shall know a word by the company it keeps" (J. R. Firth, 1957).

How word2vec works (Mikolov et al., 2013)

Shallow networks trained on raw text:

- **Skip-gram**: from the centre word, predict each word in a window around it.
- **CBOW** (continuous bag of words): from the surrounding words, predict the centre.

Skip-gram with **negative sampling**:

1. Slide a window (±2 words); every (centre, context) pair is a positive.
2. Per positive, draw k random "negative" words (frequency^0.75 distribution).
3. Each word has an input and a context vector; train σ(v_centre · u_context) → 1 for real pairs, → 0 for negatives (binary cross-entropy). This replaces an expensive full-vocabulary softmax with k+1 cheap binary classifications.
4. Keep the input vectors.

Words sharing contexts get similar updates and converge. Side effect on large corpora: vec("king") − vec("man") + vec("woman") ≈ vec("queen").

```
class SkipGram(nn.Module):
    def __init__(self, n_words, dim=16):
        super().__init__()
        self.inp = nn.Embedding(n_words, dim)      # the word vectors we keep
        self.ctx = nn.Embedding(n_words, dim)      # separate vectors for words used as context
    def forward(self, center, context, negatives):
        c = self.inp(center)                                          # (B, d)
        pos = (c * self.ctx(context)).sum(-1)                         # score for the real pair
        neg = torch.bmm(self.ctx(negatives), c.unsqueeze(-1)).squeeze(-1)  # scores for random pairs
        return -(nn.functional.logsigmoid(pos) + nn.functional.logsigmoid(-neg).sum(-1)).mean()
```

On a synthetic 3,000-sentence corpus where royal, animal and fruit nouns each appear with their own context words:

```
word2vec: 3000 sentences, vocab 31, final batch loss 1.709
  nearest to 'king': [('kingdom', 0.78), ('throne', 0.6), ('queen', 0.59)]
  nearest to 'dog': [('chases', 0.7), ('pet', 0.68), ('cat', 0.55)]
  nearest to 'mango': [('apple', 0.72), ('cherry', 0.71), ('banana', 0.7)]
  mean cosine: same-group nouns 0.50  vs  different-group nouns 0.20
```

(**Cosine similarity**: 1 = same direction, 0 = unrelated.) Never told the groups, the model puts "king" near "queen", "dog" near "cat", fruits together, purely from co-occurrence. Topical words ("kingdom") also rank high — embeddings capture relatedness as well as similarity.

Other embeddings

Method	Idea	Note
GloVe (Stanford, 2014)	Factorise global word co-occurrence statistics	Similar quality to word2vec
fastText (Facebook, 2016)	Word vector = sum of character n-gram vectors	Handles rare/misspelled/unseen words
Contextual embeddings (ELMo 2018, BERT 2018, all transformers)	Vector depends on the sentence — "bank" (river) ≠ "bank" (money)	word2vec gives one vector per word regardless of sense
Sentence/document embeddings	One vector per text	Semantic search and RAG (AI Handbook, Chapter 7)

Every PyTorch NLP model starts with `nn.Embedding(vocab_size, dim)` — e.g. `nn.Embedding(10000, 128)` is a 10,000 × 128 trainable table, one row per id — learned with the model, optionally initialised from pretrained vectors.

Real-world examples

Embeddings of words, products, users and songs drive recommendation (Airbnb's listing embeddings learned word2vec-style from browsing sessions, KDD 2018), search ranking, and **vector databases** (fast nearest-vector search) for **retrieval-augmented generation** (fetching documents for an LLM to answer from).

In the interview

> **30-second answer:** "Embeddings map discrete tokens to dense vectors where geometry reflects meaning. word2vec learns them by predicting context words from a centre word (skip-gram) or vice versa (CBOW), with negative sampling instead of a full softmax; similar contexts → similar vectors. Static embeddings give one vector per word; transformers produce contextual embeddings."

- *Q: Limitation of word2vec?* A: One vector per word regardless of sense; none for unseen words (fastText helps).

TL;DR

Learned dense vectors; similar contexts → similar vectors (skip-gram/CBOW + negative sampling). Modern models learn contextual embeddings.

12.10 Time-series forecasting — an introduction

What it is

Forecasting predicts future values of a time-indexed series (sales, energy demand, server load). Observations are **ordered and dependent**, and the future may not resemble the past.

- **Trend**: long-term rise/fall. **Seasonality**: repeating patterns (weekly, yearly). **Noise**: the unpredictable rest.
- **Lag**: a past value; lag_7 at day t is the value at t − 7.
- **Horizon**: how far ahead (one-step vs multi-step).
- **Stationarity**: mean and variance don't change over time — assumed by many classical methods; **differencing** (y_t − y_{t−1}) removes trends.

How it works — forecasting as supervised learning

1. Each row is a time t; features use **only past information**: lags, rolling means (shifted so the current value doesn't leak), calendar features.
2. Target = value at t (or t + h).
3. Split **chronologically**. **Cross-validation** (scoring on several train/validation splits) must use expanding/rolling windows (`sklearn.model_selection.TimeSeriesSplit`), never random K-fold.
4. Always compare with **baselines**: naive (last value) and seasonal naive (value one season ago).

`code/time_series.py` — a synthetic daily series with upward trend, weekly cycle and noise (std 2):

```
def lag_features(s, lags=(1, 2, 3, 7, 14)):
    df = pd.DataFrame({"y": s})
    for L in lags:
        df[f"lag_{L}"] = s.shift(L)                      # value L days earlier (only past info!)
    df["rolling_mean_7"] = s.shift(1).rolling(7).mean()   # shift first so today's value doesn't leak
    df["dayofweek"] = s.index.dayofweek
    return df.dropna()
```

Output (**MAE** = mean absolute error, in the series' units):

```
feature table (first 3 rows):
                y  lag_1  lag_2  lag_3  lag_7  lag_14  rolling_mean_7  dayofweek
2024-01-15   98.9   93.0   86.8   96.8  102.6   100.3            99.8          0
2024-01-16  107.9   98.9   93.0   86.8  107.2   107.7            99.3          1
2024-01-17  110.3  107.9   98.9   93.0  108.1   111.2            99.4          2

one-step-ahead MAE on the last 60 days (noise std = 2.0):
  naive (yesterday)                          5.94
  seasonal naive (same weekday last week)    2.20
  linear AR on lag features                  1.80
  gradient boosting on lag features          2.77
  gradient boosting on differences           1.90
train target range 91-154, test range 134-160
```

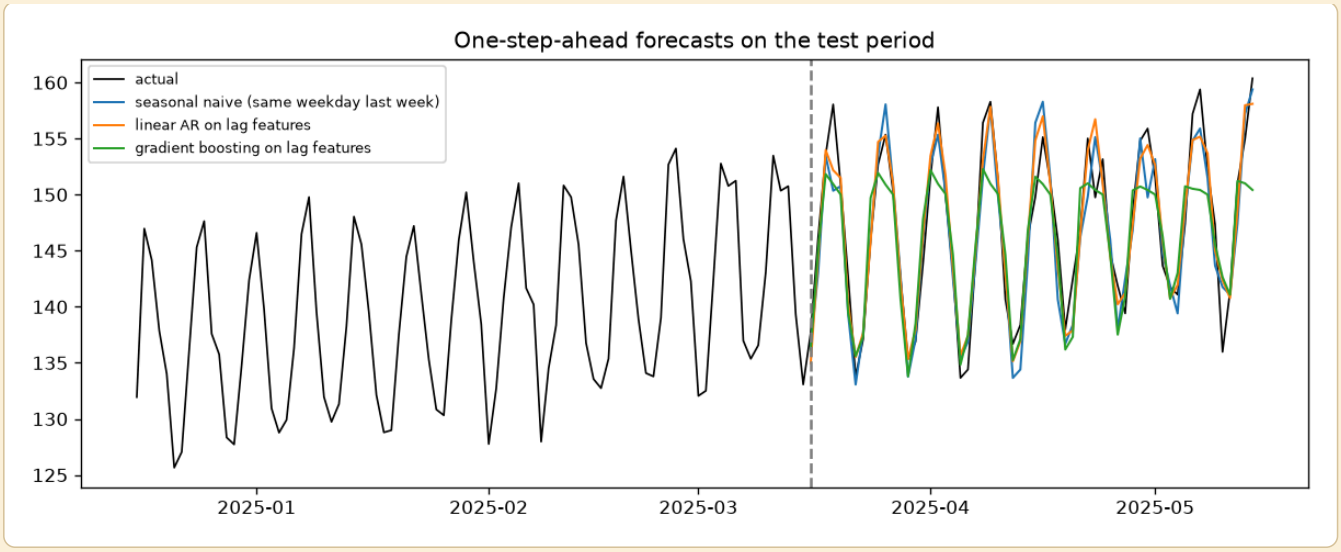


Figure — Last 150 days with one-step-ahead forecasts over the 60-day test period (dashed line = start of test).

- **Seasonal naive** is already strong (2.20); naive is poor — it ignores the weekly cycle.
- **Linear autoregression** (value as a weighted sum of its own past values) wins (1.80, near the noise floor ≈1.6 = MAE of pure N(0, 2) noise) — the data is linear.
- **Gradient boosting on raw levels loses to the baseline** (2.77): trees can't **extrapolate** — test values (to 160) exceed most of the training range and a tree only outputs values it saw. Modelling **differences** fixes it (1.90). A classic trap.

ARIMA vs ML — brief

Classical families: **ARIMA** and **exponential smoothing** (weighted averages of past values, weights fading with age). GBM = gradient-boosting machine.

	ARIMA / exponential smoothing	ML with lag features (GBM, linear)	Deep learning (LSTM, transformers, N-BEATS…)
Model	ARIMA(p, d, q): **AR** — regression on p own lags; **I** — difference d times for stationarity; **MA** — regression on q past forecast errors. SARIMA adds seasonal terms. ETS/Holt-Winters smooth level, trend, season	Any regressor on engineered lag/calendar features	Learn features from raw windows
Series handled	One at a time (univariate)	Many series in one **global model**	Many series, global
Extra features (price, promotions, weather)	Limited (ARIMAX)	Easy	Easy
Data needed	Small (dozens–hundreds of points)	Medium	Large
Uncertainty	Built-in prediction intervals	Needs quantile loss or conformal methods	Needs extra work
Tools	`statsmodels` (ARIMA, ETS), `pmdarima`, Prophet	scikit-learn, LightGBM, XGBoost	PyTorch, `darts`, `neuralforecast`

Practice: start with seasonal-naive and ETS/ARIMA baselines; for many related series with extra features, a global gradient-boosting model on lags is usually strongest (LightGBM-based approaches dominated the M5 retail competition, 2020); deep models pay off with lots of data (M4, 2018, was won by an exponential-smoothing + RNN hybrid). Pretrained forecasting **foundation models** (Amazon's Chronos, Google's TimesFM) are emerging.

Common mistakes

- Random splits or K-fold on time series (future leaks into training).
- Rolling features that include the current value.
- Skipping baselines — fancy models losing to seasonal naive is common.
- Trees on trending levels — difference or detrend first.
- Evaluating one-step-ahead when the business needs 30 days ahead (multi-step errors compound).

In the interview

30-second answer: "I frame forecasting as supervised learning with only past information as features — lags, shifted rolling stats, calendar effects — and split chronologically with rolling-origin validation (train up to a date, test the next period, move forward). I beat naive and seasonal-naive baselines first. ARIMA models a single series with autoregressive, differencing and moving-average terms and gives intervals; LightGBM handles many series and external features but can't extrapolate trends unless I model differences. Deep models help with lots of data."

- *Q: What does the I in ARIMA do?* A: Differencing to remove trends, making the series closer to stationary.

TL;DR

Past-only features, chronological splits, strong baselines. ARIMA for single series; GBMs on lags for many series (model differences for trend); deep models with lots of data.

Chapter TL;DR

- **RNN**: h_t = tanh(W_xh x_t + W_hh h_{t−1} + b); train with **BPTT**; gradients through time **vanish/explode** (clip for explosions).
- **LSTM**: c_t = f ⊙ c_{t−1} + i ⊙ č — additive gradient path. **GRU**: update/reset gates, lighter. Toy task: RNN failed at 15 steps; LSTM/GRU 100%.
- **Seq2seq**: encoder → context → decoder; teacher forcing; greedy/beam search; fixed-size bottleneck.
- **Attention**: softmax($QK^T/\sqrt{d_k}$)V; reversal task went from 16% to 98% perfect sequences.
- **Transformer**: self-attention + feed-forward with residuals and LayerNorm; positional encodings; encoder bidirectional, decoder causal + cross-attention; parallel, $O(n^2)$.
- **Text**: normalise → subword tokenize → ids → embeddings; **word2vec** (skip-gram/CBOW + negative sampling); transformers give contextual embeddings.
- **Time series**: lag features, chronological splits, seasonal-naive baselines; ARIMA, GBMs on lags (watch extrapolation), deep models with lots of data.

Chapter 13 — Recommender Systems and Embeddings

When Netflix fills a row with shows, YouTube picks the next video or Amazon shows "customers who bought this also bought", a **recommender system** predicts which items a user most likely wants from a catalogue too large to browse. This chapter goes from a ratings matrix and a similarity formula, through matrix factorisation and learned **embeddings**, to the production **two-tower + ranking funnel** — and how to evaluate it honestly.

What you'll learn: framing and feedback types; similarity measures; content-based vs collaborative filtering; matrix factorisation (SGD, ALS, implicit ALS — implemented); embeddings and item2vec (implemented); two-tower retrieval and its serving/versioning; the funnel with MMR re-ranking; ANN search (IVF implemented, HNSW, PQ); cold start, popularity bias, feedback loops; offline metrics and A/B tests — and how Netflix, YouTube, Spotify and Amazon do it.

All code is in `code/` (numpy, scikit-learn, PyTorch; no downloads — `code/data.py` generates a small dataset shaped like **MovieLens**, the classic GroupLens movie-ratings set). Outputs are real.

13.1 The recommendation problem

What it is

Users U, items I, and a log of **interactions** (ratings, clicks, plays, purchases). For a user u, produce a short **ranked list** of unseen items u is likely to value.

Why it matters

- **Choice overload**: millions of items, ~ten fit on a phone screen.
- **Business value**: Gomez-Uribe & Hunt (2015) wrote that Netflix's recommender influences the choice behind about 80% of hours streamed, most of the rest coming from search.
- **Two-sided value**: helps creators and sellers in the **long tail** (the many niche items) find an audience.

How to frame it

Framing	Question	Typical loss / metric	Example
Rating prediction (regression)	"How many stars would u give i?"	Squared error / RMSE	The Netflix Prize (2006–2009)
Top-K ranking (the real product)	"Which K items should u see, in what order?"	Ranking losses / precision@K, recall@K, NDCG (§13.11)	Home feeds, "Up next", playlists

Modern systems optimise **ranking**: users see only the top, and most feedback is implicit. Framing questions an interviewer expects:

1. **What is a good outcome?** Clicks are cheap and reward clickbait; watch time, purchase or "save" is closer to value.
2. **What signals exist?** Ratings, views, **dwell time**, skips, purchases, searches, item **metadata** (genre, author, price), user profile, context (device, time, location).
3. **Scale and latency** (a home page must be ready well under a second — System Design Fundamentals). 1,000 items can be scored exhaustively; 100 million cannot — hence the funnel (§13.8).
4. **Freshness** (news stales in hours, movies last years) and **constraints** (diversity, creator fairness, safety, "don't show what they already bought").

The user–item matrix

Rows = users, columns = items; cell (u, i) holds the interaction or is empty.

```
          Movie1  Movie2  Movie3  Movie4
  Ana        5       4       .       1
  Ben        4       5       1       .
  Cy         1       .       5       4
```

It is **huge** (10^9 users × 10^8 items) and **extremely sparse** (ours is ~9% filled; real ones often < 0.1%). Recommendation = **filling in missing cells** well enough that each row's top-scoring empty cells are items the user would enjoy.

`code/data.py`: 600 users, 400 movies, 6 genres. Users have hidden genre tastes; movies have 1–2 genres and hidden quality; popular movies are seen far more (a long tail).

```
affinity = taste @ item_genres.T / item_genres.sum(1) + quality     # hidden "true" liking
stars = clip(round(3 + 1.2 * affinity + noise), 1, 5)               # what a rating would be
p_seen = popularity[None, :] * exp(1.5 * affinity)                  # people watch popular + liked
observed = rng.random(p_seen.shape) < p_seen                         # only these cells are known
```

```
users=600 items=400 ratings=21216
density = 8.840%
top 10% of items receive 38% of all ratings
star histogram: {1: 47, 2: 624, 3: 3054, 4: 6335, 5: 11156}
```

Stars skew high because people mostly watch what they expect to like — **selection bias** (data **missing not at random**): observed ratings aren't a random sample of opinions, in real data too.

TL;DR

Fill in a huge, sparse user–item matrix; show each user the top-scoring empty cells. Frame as **top-K ranking** on a well-chosen outcome; state scale, latency, freshness and constraints first.

13.2 Explicit vs implicit feedback

What it is

- **Explicit**: the user says so — stars, thumbs, reviews.
- **Implicit**: inferred from behaviour — clicks, plays, watch time, purchases, add-to-cart, skips, dwell.

Why it matters

Implicit data is **plentiful**, explicit **scarce**. Netflix replaced stars with thumbs in 2017, saying far more members would give a thumb. Most production recommenders learn mainly from implicit signals — which changes the maths.

	Explicit	Implicit
Volume	Low	Very high
Negative signal	Yes (1 star = dislike)	Mostly **no**: "not watched" may mean "never saw it"
Noise	Rating scale differs by person	Accidental clicks, background plays
Strength	Clear preference	Graded: watched 90% vs 5%, played 30 times
Modelling	Regress on observed cells only	All cells: observed = positive, unobserved = weak negative, with **confidence** weights

How it works — behaviour → training data

1. Define the **positive event** (watched > 50%, purchased, saved).
2. Optionally map intensity to **confidence** $c = 1 + \alpha \cdot$(play count).
3. **Sample negatives**: random unobserved items, or — much stronger — **impressions without clicks**.
4. Correct **position bias** (top slots get clicks for being on top): log position, or randomise a small traffic slice.

Common mistakes

- Treating every unobserved cell as a full-weight dislike.
- Optimising clicks when the goal is satisfaction.
- Ignoring the negative implicit signals you have: early skips, "not interested", short dwell.

TL;DR

Explicit = scarce, real negatives. Implicit = abundant, positives only, unobserved ≠ negative — use confidence weights and sampled negatives.

13.3 Similarity measures

Measure	Formula	Range	Use when
Dot product	$a \cdot b = \Sigma\, a_k b_k$	$(-\infty, \infty)$	Vector length carries meaning (e.g. popularity)
Cosine	$a \cdot b \,/\, (\|a\|\|b\|)$	$[-1, 1]$	Direction matters, length doesn't (text, rating vectors)
Adjusted cosine / Pearson	cosine after subtracting each user's mean	$[-1, 1]$	Ratings, where some users rate everything high
Jaccard	$\|A \cap B\| \,/\, \|A \cup B\|$	$[0, 1]$	Sets / binary data ("bought", "followed")
Euclidean distance	$\|a - b\|$ (smaller = closer)	$[0, \infty)$	Dense embeddings; for unit vectors $\|a-b\|^2 = 2 - 2\cdot\cos$, so rankings match cosine

Worked numbers

From the §13.1 matrix (0 = not rated):

- cos(Ana, Ben) = (5·4 + 4·5 + 0·1 + 1·0) / (√42 · √42) = 40/42 = **0.952** — similar.
- cos(Ana, Cy) = (5·1 + 1·4) / 42 = **0.214** — different.
- Jaccard on movies *liked* (≥ 4): Ana {1, 2} vs Ben {1, 2} → **1.0**; Ana vs Cy {3, 4} → **0**.

Why subtract the mean? A generous rater gives 4–5 to everything, a harsh one 2–3; raw cosine sees them as different even if they order movies identically. E.g. [5, 4, 3] vs [3, 2, 1]: raw cosine ≈ 0.98; mean-centred both become [1, 0, −1] → **1.0**.

Dot product vs cosine: dot product keeps length, so a broadly popular item with a long vector scores higher for everyone — a signal or a bias, depending on the goal.

Common mistakes

- Raw cosine where 0 means "missing" — zeros drag similarity down; mean-centre on observed entries only.
- Trusting similarities from 2 co-rated items — **shrink** toward 0 when overlap is small, or require minimum overlap.
- Training with dot product but serving with cosine (or vice versa).

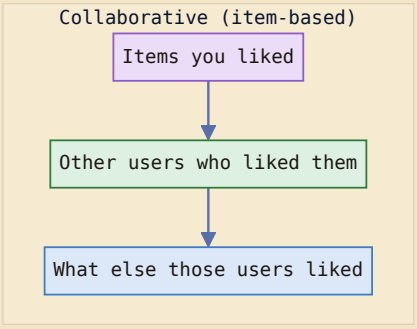
TL;DR

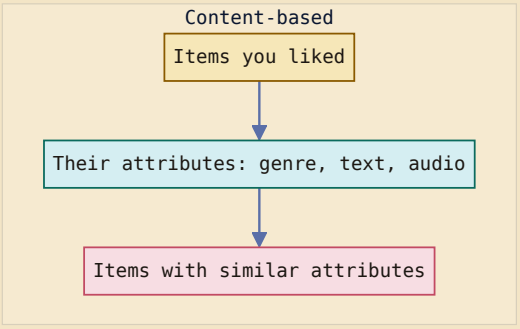
Cosine for direction, dot for magnitude, adjusted cosine for ratings, Jaccard for sets. Mean-centre and shrink small-overlap similarities.

13.4 Content-based vs collaborative filtering

What it is

- **Content-based filtering**: recommend items with **similar attributes** to what the user liked (genre, actors, text, audio, image).
- **Collaborative filtering (CF)**: ignore attributes, use **other users' behaviour** — "people who liked what you liked also liked X".
  - **User-based CF**: find similar users, recommend what they liked.
  - **Item-based CF**: for each item u liked, find items the same users liked. Made famous by Amazon.
- **Hybrid**: blend both, or feed both signals into one model.





How item-based CF predicts a rating

1. Item–item similarity S(i, j) (adjusted cosine over users who rated both).
2. Take the k items most similar to i **that u rated**.
3. r̂(u, i) = mean(u) + Σ_j S(i, j)·(r(u, j) − mean(u)) / Σ_j |S(i, j)| (absolute value keeps the denominator positive; our code keeps only positive neighbours).

Item-based usually beats user-based: items have more ratings each, item similarities are more stable, and they can be precomputed **offline** (batch job) rather than **online** (while the user waits). User–user similarity over 100M users is O(U²).

Code — `code/similarity_cf.py`

```
# collaborative: item-item adjusted cosine on mean-centred ratings
Rc = np.where(mask, R - user_mean[:, None], 0.0)
norms = np.linalg.norm(Rc, axis=0) + 1e-12
S_cf = (Rc.T @ Rc) / np.outer(norms, norms)
# content-based: item-item cosine on genre vectors
Gn = G / np.linalg.norm(G, axis=1, keepdims=True)
S_cb = Gn @ Gn.T
# ... predict each held-out rating with the k=20 most similar items the user rated
```

```
== Rating prediction on held-out ratings (RMSE, lower is better) ==
predict global mean      : 0.833
predict user's mean      : 0.811
content-based item-kNN   : 0.6872
collaborative item-kNN   : 0.6820
hybrid (average of both) : 0.6711

Movie 51 (Drama) - 5 nearest neighbours
  collaborative: ['40:Drama', '341:Drama', '321:Drama', '325:Drama', '270:Drama']
  content-based: ['109:Drama', '40:Drama', '52:Drama', '56:Drama', '341:Drama']
```

- Both beat the baselines clearly ("item-kNN" = predict from the k nearest items); the **hybrid** beats both — they capture partly different information.
- **CF discovered "Drama" by itself** from who-rated-what, never seeing a genre label.
- Here genre drives taste, so content does well. On real data CF usually wins for items with history (behaviour captures quality and mood metadata misses); content wins for new items.

Trade-offs

	Content-based	Collaborative filtering
Needs	Item attributes	Many user interactions
New item (no history)	✅ Works immediately	❌ Cold start
New user	⚠️ Needs a few likes	❌ Cold start
Serendipity (pleasant surprise)	Low — "more of the same" (**filter bubble**)	High — cross-genre finds
Captures quality/taste beyond metadata	❌	✅
Explainability	"Because it's sci-fi like X"	"People like you also liked X"
Popularity bias	Low	High (popular items co-occur with everything)

When not to use it

- Pure CF when the catalogue turns over daily (news, listings) — most items are always cold.
- Pure content when metadata is poor or quality varies wildly within a genre.

Common mistakes

- Re-recommending already-consumed items.
- User–user similarity at huge scale — go item-based, or embeddings + ANN.
- A random split for a "predict what they do next" product — use a **time-based** split (§13.11).

Real-world examples

- **Amazon** "Customers who bought this also bought": item-to-item CF (Linden, Smith & York, *IEEE Internet Computing*, 2003) — item–item similarities precomputed offline so the online step is a cheap lookup.
- **Pandora**'s Music Genome Project: content-based, analysts tagging songs on hundreds of attributes.
- Most modern systems are **hybrids**.

In the interview

> **30-second answer:** "Content-based matches item attributes to what the user liked — handles new items but over-specialises. Collaborative filtering uses co-behaviour — item-based CF is the scalable classic — and finds surprising, quality-aware matches but has cold start and popularity bias. Production combines both, often in an embedding model whose item side uses ID and content features."

- *Why item-based over user-based?* More interactions per item, more stable, precomputable offline.
- *Scaling CF?* Precompute top-N neighbours per item, or embeddings + ANN index (§13.9).
- *Rich metadata, sparse interactions?* Start content-heavy; blend in CF as data grows.

TL;DR

Content = similar attributes; collaborative = similar people's behaviour. CF finds structure (even genres) from behaviour but needs history; content covers cold items. Hybrids win.

13.5 Matrix factorisation

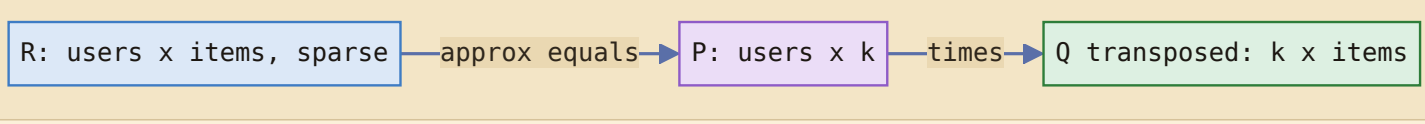
What it is

Matrix factorisation (MF) describes each user and item by a short vector of k hidden **latent factors** (k ≈ 10–200):

$\hat{r}(u, i) = \mu + b_u + b_i + p_u \cdot q_i$

- μ = global mean; b_u = user bias (a generous rater); b_i = item bias (a universally loved film).
- p_u, $q_i \in \mathbb{R}^k$ = user and item factor vectors.

So R (users × items) ≈ **P (users × k) · Qᵀ (k × items)**. Each person gets "taste coordinates" and each item "style coordinates" on a few hidden axes that training discovers.



Why it matters

- Star of the **Netflix Prize** (Simon Funk's 2006 SGD "SVD" post; Koren, Bell & Volinsky, 2009).
- **Generalises** CF: two users with no co-rated movie can still be similar if their vectors align.
- p_u, q_i are **embeddings** — the bridge to two-towers (§13.7); storage drops from U·I to (U + I)·k.

How it works — the objective

Squared error on **observed** ratings only, plus L2 regularisation:

$L = \Sigma_{(u,i)\ \text{observed}} [\ (r - \hat{r})^2 + \lambda(\|p_u\|^2 + \|q_i\|^2 + b_u^2 + b_i^2)\ ]$

(Penalty inside the sum as in Koren et al. 2009 — what the per-rating SGD updates minimise; the factor 2 from differentiating is absorbed into the learning rate η.)

This is **not classical SVD**: SVD needs a full matrix, and zero-filling tells the model "everyone hates everything unseen". Two optimisers:

1. SGD — one rating at a time:

1. $\text{err} = r - \hat{r}(u, i)$
2. $b_u \leftarrow b_u + \eta(\text{err} - \lambda b_u)$; $b_i \leftarrow b_i + \eta(\text{err} - \lambda b_i)$
3. $p_u \leftarrow p_u + \eta(\text{err} \cdot q_i - \lambda p_u)$; $q_i \leftarrow q_i + \eta(\text{err} \cdot p_u - \lambda q_i)$ (use the old p_u)

2. Alternating least squares (ALS) — the objective isn't **convex** in P and Q jointly, but with one frozen it's a **ridge regression** with a closed-form solution:

1. Freeze Q. For each user: $p_u = (Q_u^T Q_u + \lambda I)^{-1} Q_u^T r_u$ (Q_u = rows of Q for items u rated; r_u = u's ratings on them, minus μ in our bias-free version).
2. Freeze P; solve each q_i the same way.
3. Repeat until error stops falling (10–20 rounds).

Each solve is independent, so ALS **parallelises** trivially (Spark MLlib ships it).

	SGD	ALS
Update	One rating at a time	Whole vector solved exactly
Parallelism	Harder (conflicting updates)	Embarrassingly parallel
Tuning	Learning rate, epochs	Few knobs, few iterations
Implicit data (all cells)	Needs negative sampling	Efficient (Hu et al. 2008)
Extra features, custom losses	Flexible	Less flexible

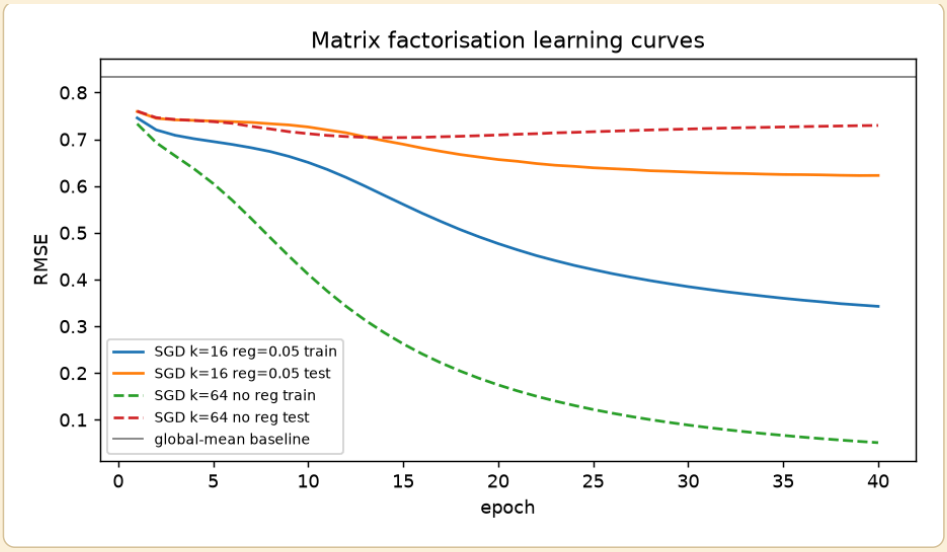
Code — `code/matrix_factorization.py`

```python
def mf_sgd(train, test, n_users, n_items, k=16, lr=0.02, reg=0.05, epochs=40, seed=0):
    ...
    for _ in range(epochs):
        for idx in rng.permutation(len(tr)):          # one rating at a time
            u, i, r = tu[idx], ti[idx], tr[idx]
            err = r - (mu + bu[u] + bi[i] + P[u] @ Q[i])
            bu[u] += lr * (err - reg * bu[u])
            bi[i] += lr * (err - reg * bi[i])
            pu = P[u].copy()
            P[u] += lr * (err * Q[i] - reg * P[u])
            Q[i] += lr * (err * pu - reg * Q[i])

def mf_als(train, test, n_users, n_items, k=16, reg=5.0, iters=10, seed=0):
    ...
        for u, idx in enumerate(by_user):              # fix Q, ridge-solve each user
            Qi = Q[ti[idx]]
            P[u] = np.linalg.solve(Qi.T @ Qi + I_k, Qi.T @ (tr[idx] - mu))
        # ... then the same loop over items with P fixed
```

```
baseline (global mean) test RMSE: 0.833
SGD epoch  1: train RMSE 0.746  test RMSE 0.759
SGD epoch  5: train RMSE 0.695  test RMSE 0.739
SGD epoch 10: train RMSE 0.650  test RMSE 0.726
SGD epoch 20: train RMSE 0.477  test RMSE 0.657
SGD epoch 40: train RMSE 0.343  test RMSE 0.623
SGD k=64, no regularisation, epoch 40: train 0.051  test 0.730
ALS iter   1: train RMSE 0.769  test RMSE 0.818
ALS iter   2: train RMSE 0.576  test RMSE 0.724
ALS iter   5: train RMSE 0.496  test RMSE 0.677
ALS iter  10: train RMSE 0.488  test RMSE 0.675

Top-5 for user 0: [98, 325, 236, 40, 314] predicted [5.16 5.14 5.11 4.98 4.93]
true (hidden) stars of those movies: [5. 5. 5. 5. 4.]
```



- Regularised SGD reaches **0.623** — better than item-kNN (0.682) and the mean (0.833).
- ALS converges in ~5 iterations (exact solves); ends slightly worse because this version has no biases.
- **Overfitting** (dashed lines): 64 factors without regularisation drive train error to 0.05 while test climbs to 0.73. With many parameters and sparse data, λ isn't optional.
- User 0's top-5 truly rate 5, 5, 5, 5, 4.

Implicit-feedback MF (weighted ALS)

Hu, Koren & Volinsky (2008) cover **every** cell:

- preference p_{ui} = 1 if u interacted with i, else 0;
- confidence c_{ui} = 1 + α·(count) — observed cells matter a lot, unobserved a little;
- minimise Σ over **all** (u, i) of $c_{ui}(p_{ui} - x_u \cdot y_i)^2 + \lambda(\ldots)$.

Solved with ALS. Speed trick: $Y^T C_u Y = Y^T Y + Y^T(C_u - I)Y$ — $Y^T Y$ is shared by all users and the second term touches only u's few items. `code/implicit_eval.py` uses the dense form; libraries: `implicit`, Spark MLlib `ALS(implicitPrefs=True)`.

Trade-offs / when not to use it

- MF sees only IDs: cold items have no vector, and context (time, device) is hard to add — use feature-based models (**factorisation machines**, which learn pairwise interactions between any features, and two-towers).
- A dot product is a limited interaction function, yet tuned MF stays a strong baseline: Rendle et al. (2020) found a tuned dot product beat MLP-based "neural collaborative filtering".

Common mistakes

- Zero-filling and running `np.linalg.svd` — teaches "unseen = hated".
- No regularisation, or tuning k on the test set (sweep k on validation; larger k needs more λ).
- Dropping biases — much of the signal is "this user rates high" and "this movie is loved".

Real-world examples

- **Netflix Prize**: *BellKor's Pragmatic Chaos* won in 2009 with the required 10% RMSE improvement over Cinematch, using a large blend of factor and neighbourhood models. Netflix wrote in 2012 that it productionised two components from an earlier progress prize (SVD-style factorisation and restricted Boltzmann machines) but not the final blend — the extra accuracy didn't justify the engineering cost.
- **Spotify** used implicit-feedback MF on listening data as one pillar of Discover Weekly.

In the interview

> **30-second answer:** "MF gives every user and item a k-dimensional vector and predicts affinity as a dot product plus biases, trained on observed entries with L2 — by SGD, or by ALS, which becomes many small parallel ridge regressions. For implicit data, weighted ALS with confidence weights. Weak spots: cold start and no context, which feature-based two-towers fix."

- *Why not plain SVD?* It needs a complete matrix; imputing zeros biases toward "dislike".

TL;DR

Users and items as short vectors; affinity = dot product + biases. Train on observed cells with L2 by SGD (flexible) or ALS (parallel, great for implicit). Strong baseline; fails on cold start.

13.6 Embeddings

What it is

An **embedding** is a short, dense, learned vector (tens to a few hundred numbers) representing a discrete thing — user ID, item ID, word, category. MF's p_u and q_i are embeddings.

The problem it solves

Raw IDs carry no meaning (`item_5521` isn't closer to `item_5522` than to `item_9`). **One-hot** vectors are enormous for 100M items and every pair is equally distant. Embeddings are small and training places them so **distance reflects the training objective**: items serving the same users end up near each other. Mechanically an embedding layer is a lookup table (one row per ID); multiplying a one-hot vector by it selects a row. "Close" means close *for that objective* — co-watched items may still differ in price, language or quality.

Four ways to train them

Route	Idea	Best when
Matrix factorisation	Factor the interaction matrix (§13.5)	Only interactions; strong baseline
Two-tower (contrastive)	Real (user, item) pairs must outscore sampled negatives (§13.7)	Large-scale retrieval with features
Graph embeddings	Random walks or graph neural networks over users, items, follows, co-purchases	Relationships are the signal (Pinterest's PinSage)
Pretrained content encoders	Text/image/audio model embeds item content	Brand-new items; text/image-heavy catalogues

Text embeddings from pretrained language models (semantic search, RAG) are in the AI book (embeddings, vector databases and RAG); the geometry is the same.

word2vec → item2vec

word2vec (Mikolov et al., 2013) skip-gram with negative sampling — centre word predicts window context, σ(centre · context) → 1 for real pairs and → 0 for k frequency^0.75-sampled negatives — is taught in §12.9. **item2vec** (Barkan & Koenigstein, 2016) treats a session, playlist or basket as a "sentence" and its items as "words", so co-consumed items get nearby vectors — no labels needed. Airbnb (listing embeddings from search click sessions, Grbovic & Cheng, KDD 2018) and Spotify (tracks co-occurring in playlists) describe using it; case study 18.9 builds a similar-listings carousel on it.

`code/item2vec.py` implements it in numpy:

```
pos = sig(np.sum(vc * W_out[o], 1))                # centre . true context  -> want 1
ngs = sig(np.einsum("bd,bkd->bk", vc, W_out[negs]))  # centre . 5 negatives   -> want 0
g_pos = (pos - 1)[:, None]                         # gradient of -log(pos)
g_neg = ngs[:, :, None]                            # gradient of -log(1 - ngs)
grad_c = g_pos * W_out[o] + np.sum(g_neg * W_out[negs], 1)
np.add.at(W_out, o, -lr * g_pos * vc)              # np.add.at handles repeated ids
np.add.at(W_out, negs, -lr * g_neg * vc[:, None, :])
np.add.at(W_in, c, -lr * grad_c)
```

```
epoch  1  mean loss per pair 2.884
epoch 10  mean loss per pair 2.581
top-5 neighbours sharing the main genre: 79% (random would give ~17%)
item 51 (Drama) neighbours: ['113:Drama', '236:Drama', '281:Action', '321:Drama', '146:Drama']
```

Seeing only lists of item IDs, 79% of nearest neighbours share the item's genre (chance 17%) — **meaningful geometry learned from behaviour**.

When not to use embeddings

- **Small catalogues** where exact scoring already fits the budget.
- **Numeric inputs** (price, distance, account age) — already meaningful; scale and use directly.
- **Decisions that must be explained** — embedding dimensions have no human meaning; credit and fraud keep interpretable features alongside.

Common mistakes

- Comparing vectors from two different trainings — each run has its own coordinate system (§13.7).
- An embedding per rare ID (noisy) — use a minimum count, **hashing** (many IDs into a fixed number of shared buckets), or content features.
- Assuming "near" means "similar in every way".

TL;DR

Short learned vectors giving IDs a geometry where near = related (for the objective). Learn via MF, two-tower, graphs or content encoders; item2vec = word2vec on sessions.

13.7 Two-tower retrieval

What it is

A **two-tower model** (*dual encoder*) has two separate networks:

- **user (query) tower**: user features — ID, recent clicks, search terms, device, hour → vector;
- **item (candidate) tower**: item features — ID, category, text/image content → vector of the **same length** (64–256).

Score = **dot product** (or cosine) of the two.

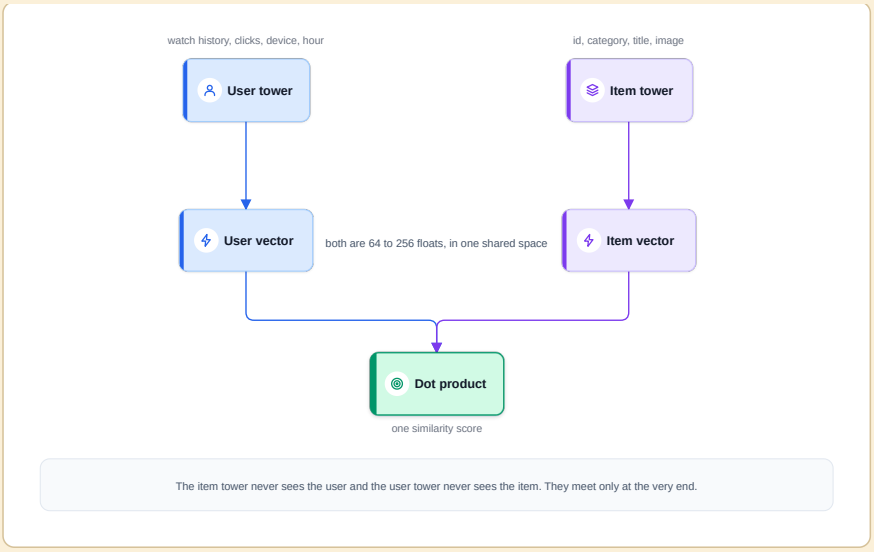


Figure — The towers run independently and meet only at the final dot product (course diagram).

Why it matters — the key property

The towers **never see each other's inputs**. So the item tower embeds the **entire catalogue ahead of time** into a nearest-neighbour index; per request only the user vector is computed and the index returns its nearest items — 100M items searched in milliseconds.

A **cross model** reads user and item features together and learns interactions ("this user × this item's price") — more accurate, but nothing precomputes; it would run per item per request. Hence two-towers do **retrieval**, cross models **ranking** (§13.8).

How it is trained

Logs hold positive pairs (user, video watched), (user, product bought).

1. Take a batch of B positive pairs; encode → B user and B item vectors.
2. Score all users against all items: a B × B dot-product matrix divided by a **temperature** τ (0.05–0.1; small τ sharpens the softmax).
3. Row j's correct "class" is column j; the other B − 1 items are negatives; softmax cross-entropy. This is **in-batch negative sampling** — negatives for free.
4. **logQ correction** (Yi et al., 2019): a hit item appears as a negative in batch after batch and gets pushed down too hard, so subtract log(item's sampling probability) from its logit.
5. Optionally a few **hard negatives** (close but wrong, e.g. shown-not-clicked) — useful, but too many or mislabelled ones hurt. Add them last.

Code — `code/two_tower.py`

```
class Tower(nn.Module):
    def __init__(self, n_ids, n_feats, id_dropout=0.0):
        super().__init__()
        self.id_emb = nn.Embedding(n_ids, 32)
        self.mlp = nn.Sequential(nn.Linear(32 + n_feats, 64), nn.ReLU(), nn.Linear(64, DIM))
        self.id_dropout = id_dropout   # randomly hide the id so the tower also learns from content

    def forward(self, ids, feats, use_id=True):
        e = self.id_emb(ids)
        if not use_id:
            e = torch.zeros_like(e)
        elif self.training and self.id_dropout:
            e = e * (torch.rand(len(ids), 1) > self.id_dropout).float()
        return F.normalize(self.mlp(torch.cat([e, feats], 1)), dim=1)

# training step
uv = user_tower(u, U_feat[u])                # user tower: id + genre profile of history
iv = item_tower(i, G[i])                     # item tower: id + genre multi-hot
logits = uv @ iv.T / temperature             # (B, B) in-batch scores
logits = logits - log_q[i][None, :]          # logQ popularity correction
loss = F.cross_entropy(logits, torch.arange(len(idx)))
```

40 items are held out entirely as brand-new items, scored with the item tower's ID part zeroed (content only).

```
epoch  1  in-batch softmax loss 6.637
epoch  5  in-batch softmax loss 6.022
epoch 15  in-batch softmax loss 5.798

warm items  recall@50: two-tower 0.649 | popularity 0.510
cold items  hit@10 among the 40 new items: 0.586 (random = 0.250)
```

- Known items: recall@50 0.65 vs 0.51 for "most popular".
- **Never-seen items**: content alone puts the right new item in the top 10 of 40 for 59% of users (random 25%) — two-towers soften **item cold start**. `id_dropout` is what makes the content-only vector meaningful.
- (Batch 512 → uniform guessing loss ln 512 ≈ 6.24, but logQ shifts the logits, so read the trend and judge by recall.)

How it serves

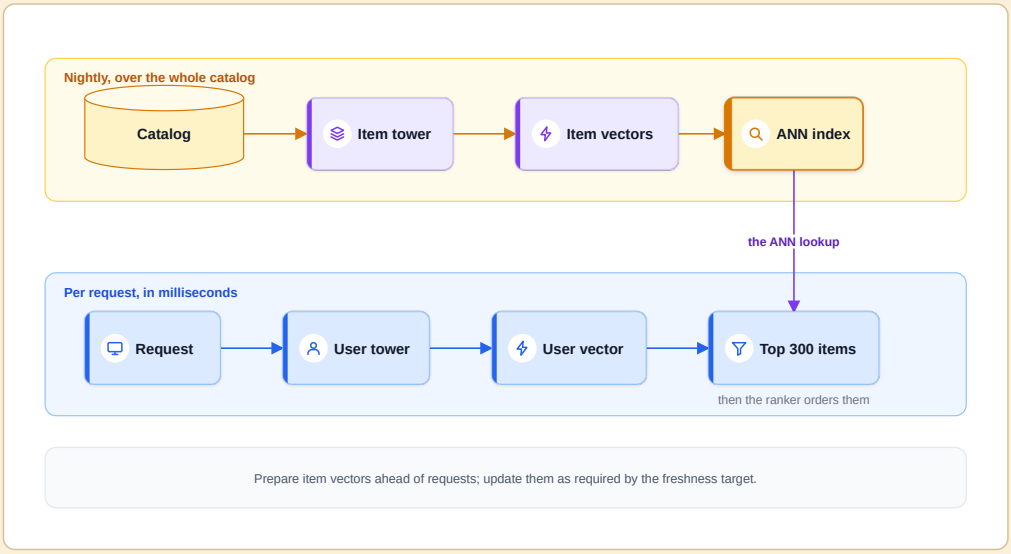


Figure — Offline item embedding and index build vs the online per-request path (course diagram).

1. **Offline (e.g. nightly):** embed the catalogue, rebuild the **ANN index** (approximate nearest neighbour, §13.9).
2. **Per request:** run the user tower (or reuse a recently cached vector).
3. **Lookup:** a few hundred nearest items in milliseconds — the **candidates** for the ranker.

Freshness differs by tower. Item vectors refresh as often as catalogue change and measured retrieval quality demand (nightly is an example). The user side must be fresher — three cooking videos just watched should yield cooking candidates *now* — so the user tower reads **real-time features** from an online **feature store** at request time.

The versioning trap

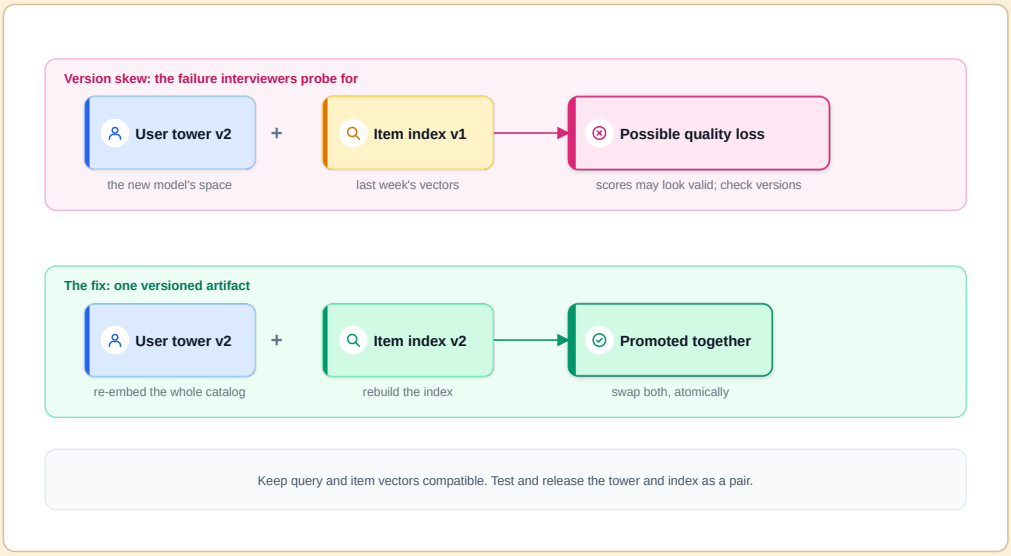


Figure — Version skew: a new user tower querying an index built by an old item tower (course diagram).

An embedding means something only **inside its model's space**; retraining rotates and rescales it. Upgrading is a coordinated migration: (1) re-embed the catalogue with the **new** item tower, (2) build a **new** index, (3) switch new user tower and new index **together, atomically** (never half-switched) as a versioned pair in the **model registry**, keeping the old pair for **rollback**.

Version skew (new tower on old index, or the reverse) is dangerous because nothing crashes — scores are valid numbers, latency is normal, only quality silently drops. Defences: version tags on requests and indexes, routing to compatible pairs, offline validation of the pair, retrieval-quality monitoring during rollout. Budget re-embedding compute (100M items is not free).

Trade-offs

	Two-tower	Cross model (ranker)
User–item feature interactions	Only via final dot product	Full (any feature crosses)
Precompute items	✅	❌
Items scored per request	Whole catalogue (via ANN)	A few hundred
Role	Retrieval / candidate generation	Ranking

When not to use it

Small catalogues (score everything exactly), or when co-occurrence is good enough and training + indexing + versioning machinery isn't affordable.

Common mistakes

- The same item twice in a batch becomes its own false negative — deduplicate or mask.
- No popularity correction → hits pushed down (or, with uniform random negatives, pushed up).
- Mixing tower and index versions; leaking the future into user features (events after the label).

Real-world examples

- **YouTube**: Covington, Adams & Sargin (2016) — candidate generation as a huge softmax over videos, served by nearest-neighbour search, then a ranking network; Yi et al. (2019) — two-tower with streaming logQ.
- **Pinterest, LinkedIn, Airbnb** and many e-commerce sites describe embedding-based retrieval.

In the interview

30-second answer ("How do you find candidates among 100M items?"): "A two-tower model trained with in-batch negatives and logQ correction. Item vectors go offline into an ANN index; per request I compute a fresh user vector and pull a few hundred nearest items for the ranker. I'd watch retrieval recall and refresh cadence, and ship tower and index as a tested, versioned pair."

- *What keeps the sides consistent?* Tower + index as one versioned **artifact**, switched atomically, with a rollback pair.
- *Can retrieval surface a zero-interaction item?* Yes, via content features; add a "new arrivals" slot or exploration traffic.
- *Why not one network over user and item?* Can't precompute — use it as the ranker on the few hundred retrieved.

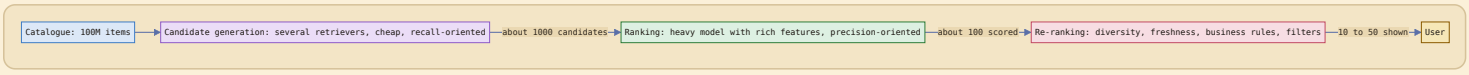
TL;DR

Two towers encode users and items independently; they meet only at a dot product, so items go offline into an ANN index and only the user tower runs per request. In-batch negatives + logQ; release tower and index together.

13.8 The funnel: candidate generation → ranking → re-ranking

What it is

Large recommenders are **multi-stage funnels**; each stage sees fewer items and can afford a heavier model.



Stage	Input → output	Model	Optimises
Candidate generation (retrieval)	$10^8 \to 10^3$	Several merged sources: two-tower + ANN, item-kNN, **co-visitation** (items viewed in the same sessions), trending, followed creators — each covers a different need, merging raises recall	**Recall**: don't lose good items
Ranking	$10^3 \to$ ordered 10^2	GBDTs or deep nets (Wide & Deep, DCN, DLRM) on user × item × context features; predicts P(click), watch time, P(purchase)… combined into one score	**Precision** at the top
Re-ranking	$10^2 \to 10$–50	Rules + algorithms (MMR, quotas)	Diversity, freshness, fairness, policy, de-duplication

Some systems add a **pre-ranking** ("light ranking") stage when candidate counts are very large. No single model is both cheap enough for 100M items and rich enough for fine feature crosses — the funnel spends compute where it matters. Retrieval sets a **ceiling**: an item not retrieved can never be ranked.

Maximal Marginal Relevance (MMR)

A ranker scores items independently, so the top 10 are often near-duplicates (ten episodes of one show) — diversity therefore belongs in re-ranking. **MMR** (Carbonell & Goldstein, 1998) re-ranks greedily:

next item = argmax over remaining j of $[\ \lambda \cdot \text{relevance}(j) - (1-\lambda) \cdot \max \text{similarity}(j, \text{already chosen})\ ]$

$\lambda = 1$ is pure relevance; smaller λ trades relevance for variety.

Code — `code/funnel_rerank.py`

Stage 1: item-kNN retrieves 100 candidates. Stage 2: a logistic-regression ranker trained on half the users scores them with richer features (kNN score, implicit-ALS score, popularity, genre match, average stars). Stage 3: MMR ($\lambda = 0.7$) on the ranker's top 30.

```
def mmr(cands, relevance, sim, lam=0.7, k=K):
    chosen, pool = [], list(range(len(cands)))
    while len(chosen) < k:
        best = max(pool, key=lambda j: lam * relevance[j]
                   - (1 - lam) * max((sim[cands[j], cands[c]] for c in chosen), default=0))
        chosen.append(best); pool.remove(best)
    return [cands[j] for j in chosen]
```

```
stage 1 candidate recall@100 = 0.787  (ceiling for later stages)
stage 1 only (kNN order)           recall@10 0.375  NDCG@10 0.398  diversity 0.728
stage 2 ranker (logistic regr.)    recall@10 0.393  NDCG@10 0.409  diversity 0.524
stage 3 re-rank (MMR, lambda=0.7)  recall@10 0.375  NDCG@10 0.391  diversity 0.683
ranker weights: {'knn_score': 0.18, 'als_score': 0.63, 'log_popularity': 0.36, 'genre_match': 0.62, 'avg_star': 0.07}
```

- Retrieval keeps 79% of held-out items in 100 candidates — the most any later stage can recover.
- The ranker lifts NDCG (0.398 → 0.409) by combining signals, leaning on ALS score and genre match.
- But it makes lists much **less diverse** (0.728 → 0.524), piling onto the favourite genre. MMR restores diversity (0.683) at a small offline cost; online, users often reward variety — an A/B test decides.

Common mistakes

- Training the ranker only on what the *old* system showed (exposure bias) — log impressions and some randomised exposure.
- Evaluating the ranker but not retrieval recall.
- A single objective (clicks) when the business wants long-term satisfaction — use a **multi-task** ranker (click, watch time, like, "not interested") with tuned weights.

In the interview

> **30-second answer:** "No single model is both cheap enough for the whole catalogue and rich enough for fine feature crosses, so: several high-recall retrievers to ~a thousand candidates, a heavy ranker trained on logged impressions, and a re-ranker for diversity, freshness and policy. I track retrieval recall separately — it caps everything after it."

TL;DR

Cheap high-recall retrieval ($10^8 \to 10^3$) → heavy precise ranking ($\to 10^2$) → re-ranking for diversity/rules ($\to 10$). Retrieval caps ranking; MMR buys variety for a little relevance.

13.9 Approximate nearest neighbour (ANN) search

What it is

k-nearest-neighbour search returns the k vectors most similar to a query q. **Exact** (brute-force) search is O(N·d) per query. **ANN** returns *almost* the same top-k while scanning a small fraction. 10^8 items × 128 dims ≈ $1.3 \cdot 10^{10}$ multiply-adds per query is far too slow at thousands of QPS; ANN trades a little **recall** (fraction of the true top-k found) for 10–1000× speed-ups.

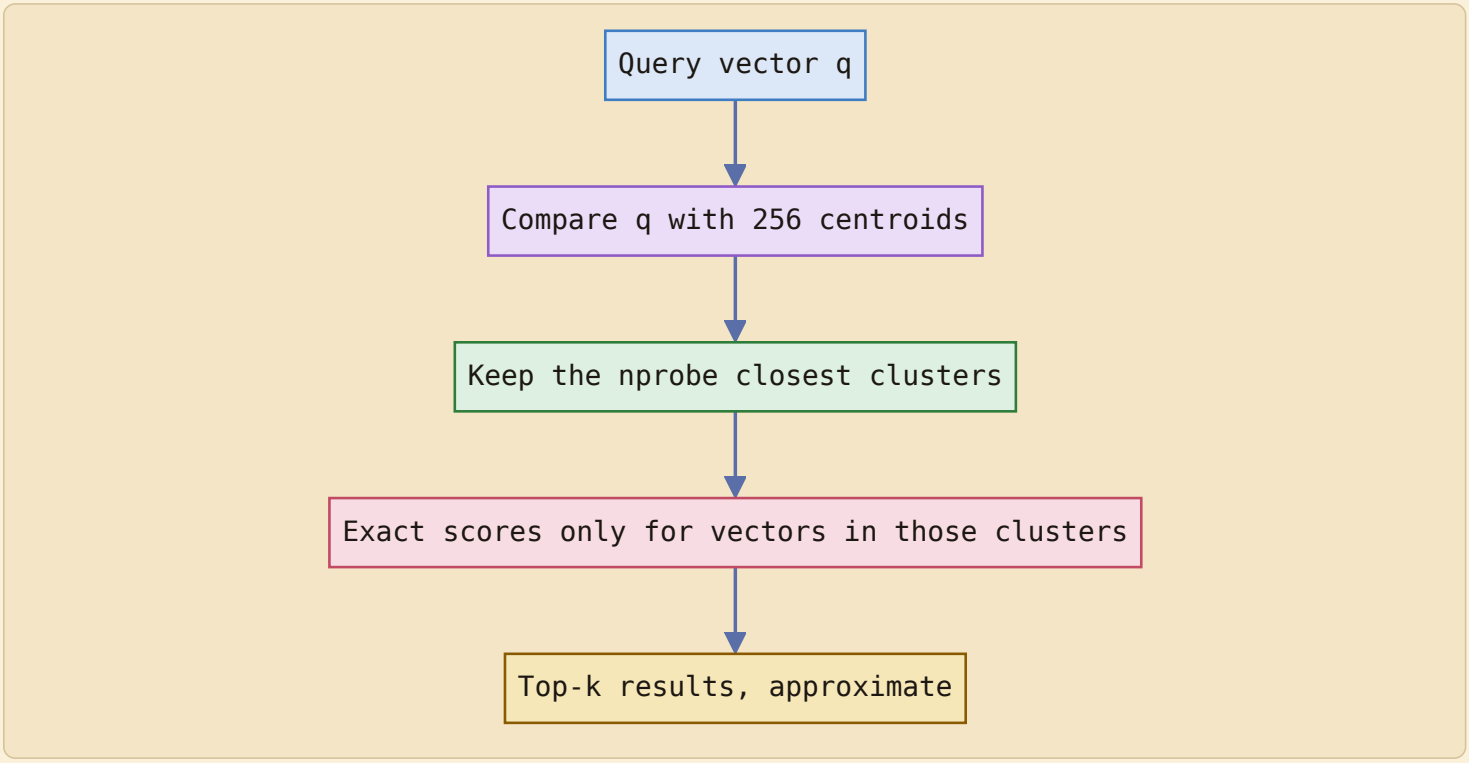
How the main families work

1. IVF — inverted file index (clustering). Offline: **k-means** the vectors into `nlist` centroids; store each vector in its nearest centroid's list. Query: find the `nprobe` nearest centroids, scan only those lists exactly. Larger `nprobe` → higher recall, slower.

2. HNSW — Hierarchical Navigable Small World graphs (Malkov & Yashunin, 2016/2018). Each vector links to ~M close neighbours; upper layers hold few nodes (long-range "express" hops), the bottom holds all. Query: start at the top, greedily hop closer to q, drop a layer when stuck; at the bottom keep a candidate list of size `ef_search`. Knobs M, `ef_construction`, `ef_search`. Excellent recall/latency and incremental inserts; more memory than IVF.

3. Product quantisation (PQ) compresses vectors (128 floats = 512 bytes → 16 bytes) by splitting into sub-vectors, each replaced by its nearest sub-centroid's ID; distances come from small lookup tables. Often combined: **IVF-PQ**.

4. Tree forests (Spotify's Annoy): random **hyperplane** splits build several trees; search a few.

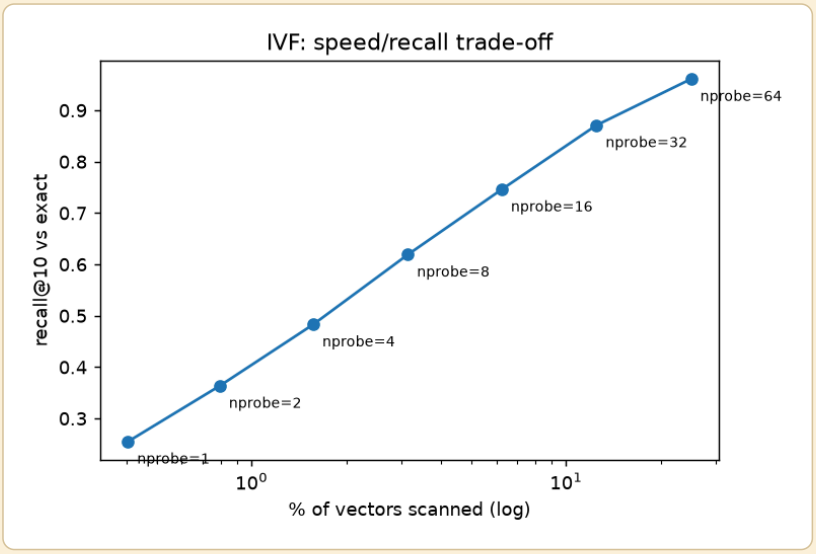


Code — `code/ann_demo.py` (brute force vs IVF)

```
exact = np.argsort(-(queries @ X.T), axis=1)[:, :K]          # brute force: score every vector
C, _ = kmeans(X[rng.choice(N, 20_000, replace=False)], 256)  # train centroids on a sample
assign = np.argmax(X @ C.T, axis=1)
lists = [np.flatnonzero(assign == c) for c in range(256)]    # the "inverted file"
for q in queries:
    probe = np.argsort(-(C @ q))[:nprobe]                    # closest clusters
    cand = np.concatenate([lists[c] for c in probe])
    top = cand[np.argsort(-(X[cand] @ q))[:K]]
```

100,000 unit vectors, 32 dims (timings machine-dependent):

```
brute force: scans 100% of 100,000 vectors, 6.13 ms/query, recall@10 = 1.000
IVF build (k-means, 256 lists): 0.1 s
IVF nprobe= 1: scans  0.41%, 0.06 ms/query, recall@10 = 0.255
IVF nprobe= 2: scans  0.79%, 0.08 ms/query, recall@10 = 0.363
IVF nprobe= 4: scans  1.57%, 0.14 ms/query, recall@10 = 0.483
IVF nprobe= 8: scans  3.14%, 0.27 ms/query, recall@10 = 0.619
IVF nprobe=16: scans  6.25%, 0.56 ms/query, recall@10 = 0.746
IVF nprobe=32: scans 12.49%, 1.21 ms/query, recall@10 = 0.871
IVF nprobe=64: scans 24.96%, 2.56 ms/query, recall@10 = 0.961
```



Scanning 12% gives 87% recall at ~5× speed; 25% gives 96%. (Deliberately hard, overlapping clusters; real embeddings reach high recall at smaller `nprobe`, and production libraries are far faster than this numpy loop.)

Trade-offs

Method	Recall/speed	Memory	Build	Updates
Brute force	Exact, O(N)	Vectors only	None	Trivial
IVF	Good, tune `nprobe`	Low	k-means (fast)	Add to lists; re-train centroids occasionally
IVF-PQ	Good at huge scale	Very low (compressed)	Slower	As IVF
HNSW	Excellent	Higher (graph links)	Slowest	Incremental inserts; deletes awkward

Libraries: **FAISS** (Meta), **ScaNN** (Google), **hnswlib**, **Annoy/Voyager** (Spotify); **vector databases**: Milvus, Weaviate, Qdrant, pgvector, OpenSearch.

When not to use it

Up to ~a million vectors, batched brute-force matrix multiply (especially on GPU) may meet the latency budget — exact and simple. Measure first.

Common mistakes

- A cosine index fed un-normalised vectors (or vice versa).
- Reporting latency without recall@k against exact search.
- Filtering (country, availability) *after* ANN and ending with too few results — use filtered search or over-retrieve.

TL;DR

Exact k-NN is O(N); ANN (IVF clustering, HNSW graphs, PQ compression) scans a fraction for a small recall loss. Always report recall with latency.

13.10 Cold start

Cold start = recommending with little or no history: **new user**, **new item** (CF/MF have no vector), **new system**.

Case	Strategies
New user	Popular/trending by region or device; short onboarding ("pick 3 genres"); context (location, time, referrer); fast updates from first clicks (session-based models)
New item	Content features in the item tower (59% vs 25% random in `two_tower.py`); creator/brand signals; **exploration** — a small controlled traffic share and a "new arrivals" slot
New system	Rules + popularity + content similarity, then collect data

Exploration is essential: if only proven items are shown, new ones never get to prove themselves. **Multi-armed bandits** (ε-greedy, UCB, Thompson sampling — Chapter 14) balance exploiting proven items with exploring newcomers.

Common mistakes

Letting cold items sink forever because their score rests on zero data; long onboarding questionnaires few finish.

TL;DR

New users: popularity + onboarding + context + fast session updates. New items: content features + controlled exploration. "No data" must never mean "never shown".

13.11 Evaluation

What it is

Offline (held-out history — fast, cheap) then **online** (A/B tests — the final judge).

Offline accuracy metrics (top-K)

For one user with ranked `recs` and held-out `relevant` items:

Metric	Formula	Meaning
Precision@k	(# relevant in top k) / k	Quality of what we showed
Recall@k	(# relevant in top k) / (# relevant)	How much of the good stuff we found
Hit rate@k	1 if any relevant in top k	At least one right? (common with leave-one-out)
AP@k (average precision)	(Σ precision@i over ranks i holding a hit) / min(#relevant, k)	Rewards early hits
MAP@k	mean AP@k over users	
NDCG@k	DCG / ideal DCG, DCG = Σ_i rel_i / $\log_2(i + 1)$ (variant gain: $2^{rel_i} - 1$)	Position-discounted, normalised to [0, 1]; supports graded relevance
MRR	mean of 1 / (rank of first hit)	Only the first hit matters (search-like)
RMSE / MAE (root mean squared / mean absolute error)	error on predicted ratings	Rating-prediction framing only

Worked example (`implicit_eval.py`): recs = A B C D E, relevant = {A, C, F}, k = 5.

- Hits at ranks 1 and 3. Precision@5 = **0.400**; Recall@5 = **0.667**; Hit@5 = **1**.
- AP@5 = (1/1 + 2/3) / 3 = **0.556**.
- DCG = $1/\log_2 2 + 1/\log_2 4$ = 1.5; ideal = 1 + 0.631 + 0.5 = 2.131; NDCG@5 = **0.704**.
- (Single hit at rank 4: NDCG@5 = $1/\log_2 5 \approx 0.431$; MRR = 0.25.)

```
def average_precision_at_k(recs, relevant, k=K):
    hits, total = 0, 0.0
    for rank, item in enumerate(recs[:k], start=1):
        if item in relevant:
            hits += 1
            total += hits / rank          # precision at each position where we hit
    return total / min(len(relevant), k)

def ndcg_at_k(recs, relevant, k=K):
    dcg = sum(1 / np.log2(rank + 1) for rank, it in enumerate(recs[:k], 1) if it in relevant)
    ideal = sum(1 / np.log2(rank + 1) for rank in range(1, min(len(relevant), k) + 1))
    return dcg / ideal
```

Beyond-accuracy metrics

- **Catalogue coverage**: fraction of items shown to *anyone*.
- **Intra-list diversity**: 1 − mean pairwise similarity within a list (here genre cosine).
- **Novelty**: mean popularity of recommendations (lower = more novel); **serendipity** = relevant *and* unexpected.
- **Fairness / exposure**: how impressions spread across creators or sellers.

Code — three models on implicit data (`code/implicit_eval.py`)

Interactions binarised to "watched"; 5 random watched items per user hidden as **test**, ranked among all unseen items. ALS settings chosen on a separate **validation** split (5 more items per user), never on test.

```
ALS chosen on validation split: alpha=10, lambda=50 (validation NDCG@10 0.331)

model             P@10   R@10   MAP@10 NDCG@10 HR@10  cover  divers  avgPop
Popularity        0.107  0.215  0.104  0.197  0.708  0.07   0.741   219
Item-kNN (cosine) 0.184  0.367  0.256  0.388  0.882  0.34   0.733   186
Implicit ALS      0.195  0.390  0.282  0.416  0.893  0.42   0.591   154
(average item popularity in the catalogue = 46 interactions)
```

- **Popularity** is a strong baseline (hit rate 71%) — always include it — but covers 7% of the catalogue with items ~5× more popular than average.
- **Implicit ALS** wins every accuracy metric, covers 42%, recommends less popular items — but lists are less genre-diverse. Accuracy, coverage, novelty and diversity pull apart; choose deliberately.
- The sweep ($\alpha \in \{5, 10, 20\}$, $\lambda \in \{10, 50, 100\}$, $k = 16$) picked $\alpha = 10$, $\lambda = 50$; refit on all training data, scored once on test.

Offline evaluation pitfalls

- **Random splits leak the future** — use a **time-based split**.
- **Sampled metrics** (true item vs 100 random negatives) inflate scores and can flip which model wins (Krichene & Rendle, 2020) — rank against the full catalogue when possible.
- **Logs are biased by the old system**. **Inverse propensity scoring (IPS)** reweights each example by 1 / P(old system showed it) — unbiased, but higher variance.

Online evaluation

- **A/B test**: random user split; business metrics (watch time, **retention**, purchases) plus **guardrails** (latency, complaints, diversity); run long enough for **novelty effects** to fade.
- **Interleaving**: merge two rankers' lists and see whose items get clicked — needs far less traffic; Netflix uses it to pre-screen ideas.
- Offline metrics are a filter, not the verdict — logs reflect the old policy and ignore presentation and long-term effects.

In the interview

30-second answer: "Offline: time-based split, recall@k for retrieval and NDCG@k or MAP@k for ranking over the full catalogue, plus coverage, diversity and novelty, versus a popularity baseline. Then an A/B test on the real objective, such as retention, with latency and diversity guardrails."

- *Precision@k vs recall@k?* Quality of what's shown vs how much of the relevant set was found; retrieval cares about recall, the final list about precision/NDCG.
- *Why NDCG over precision?* Rewards higher placement; supports graded relevance.
- *Model A: higher NDCG, 5% coverage; B: slightly lower NDCG, 40% coverage?* Depends on goals; usually test B online — low coverage starves the long tail and feeds the popularity loop.

TL;DR

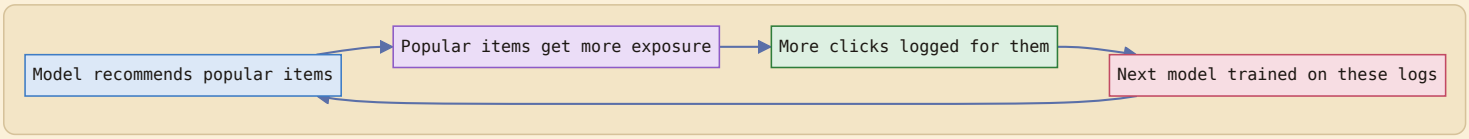
Offline: precision/recall/hit@k, MAP, NDCG on a time-based split over the full catalogue, plus coverage/diversity/novelty and a popularity baseline. Online: A/B tests or interleaving on the true objective.

13.12 Popularity bias and feedback loops

What it is

Popularity bias: models over-recommend popular items — from the data (more interactions → more updates and co-occurrences) and the model (dot products favour long vectors).

Feedback loop: popular items shown → more clicks → next model rates them higher → shown more. Long-tail items never collect data to prove themselves → less diversity, filter bubbles, struggling niche creators.



How to counter it

1. **Measure**: average popularity of recommendations, coverage, Gini coefficient of exposure (0 = even, 1 = all to one item).
2. **Correct in training**: logQ (§13.7), down-weighting popular positives, IPS.
3. **Explore**: a little bandit traffic for new and long-tail items.
4. **Re-rank**: MMR, novelty boosts, exposure quotas for new creators.
5. **Log impressions** to tell "not shown" from "shown and ignored".

Common mistakes

- Celebrating an offline win that comes only from recommending more popular items (test sets are popularity-biased too).
- Removing popularity entirely — it's real information; aim for calibration, not elimination.

TL;DR

Popular items get more data, so models over-recommend them and the loop reinforces itself. Measure, correct in training (logQ, IPS), explore, re-rank.

13.13 Real-world systems

Amazon's item-to-item CF is in §13.4.

Company	Publicly described approach	Ideas to remember
Netflix	Netflix Prize (2006–2009, $1M, won by BellKor's Pragmatic Chaos); personalised rows and ranking; artwork chosen per member with contextual bandits; recommender influences ~80% of hours streamed (2015)	Moved from star prediction to ranking on implicit signals; heavy A/B testing plus interleaving
YouTube	Deep candidate generation + ranking net predicting expected watch time (Covington et al., 2016); two-tower with streaming logQ (Yi et al., 2019); multi-task ranking (Zhao et al., 2019)	Two-stage funnel; watch time over clicks; "example age" feature for freshness
Spotify	Discover Weekly (2015) blends CF on listening and playlists, NLP on text about music, and audio models; bandits on home shelves; open-sourced Annoy and Voyager	Playlists as co-occurrence data; skips and saves as implicit feedback
Pinterest	PinSage (Ying et al., 2018): graph convolutional network over billions of pins and boards	The graph-embedding route

---

Chapter TL;DR

- **Frame** as top-K ranking on a meaningful outcome; implicit feedback dominates; unobserved ≠ negative.
- **Content-based** covers new items; **collaborative** finds quality and surprise but needs history; hybrids win.
- **MF** ($\hat{r} = \mu + b_u + b_i + p_u \cdot q_i$; SGD or ALS; weighted ALS for implicit) is the strong baseline; its vectors are **embeddings**.
- **Two-tower** retrieval: items precomputed into an ANN index (IVF, HNSW, PQ); only the user tower runs per request; ship tower and index as one version.
- **Funnel**: retrieval (recall ceiling) → ranking (precision) → re-ranking (diversity, rules).
- **Evaluate** on a time-based split over the full catalogue against a popularity baseline, then A/B test. Cold start → content + exploration; counter popularity feedback loops.

Chapter 14 — Reinforcement Learning

Supervised learning learns from an answer key; unsupervised learning finds structure without one. **Reinforcement learning (RL)** is the third way: an agent learns *what to do* by acting, seeing what happens and collecting rewards — no answer key, just a score that often arrives late. It's how AlphaGo beat a world champion at Go, how robots learn to walk in simulation, how some recommenders decide what to try next, and how chat models such as ChatGPT and Claude were tuned to be more helpful (RLHF, §14.10).

What you'll learn: agent/environment vocabulary; MDPs, returns and discounting; V, Q and the Bellman equations; bandits (ε-greedy, UCB, Thompson — implemented); dynamic programming (policy/value iteration on a **gridworld** — a toy world of grid cells — and a FrozenLake-style lake); Monte Carlo vs TD; Q-learning and SARSA from scratch in numpy (no **Gym**/Gymnasium — the standard library of RL test environments — needed); DQN (PyTorch); policy gradients, actor-critic, PPO; RLHF; real-world uses and pitfalls.

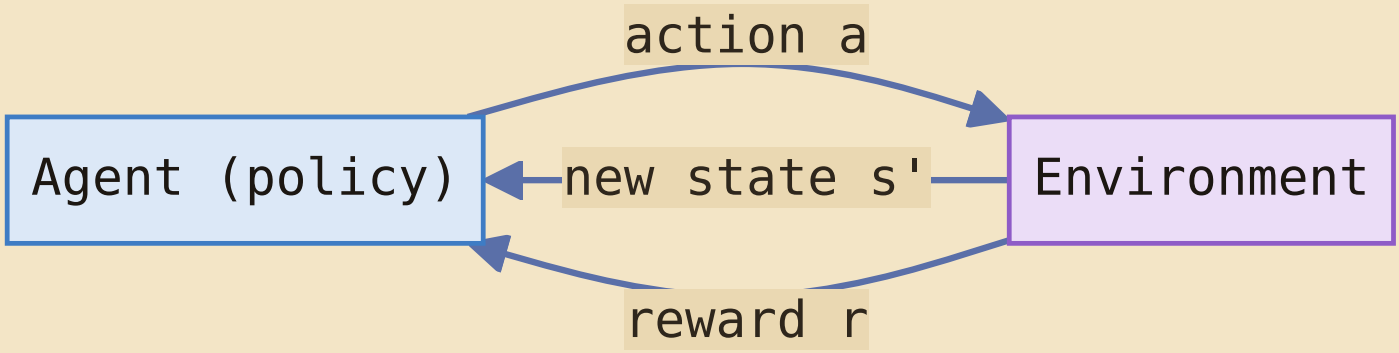
All code is in `code/` (numpy, matplotlib, PyTorch). `code/gridworld.py` is a tiny environment for this chapter (lake, treasure grid, cliff walk); its action numbering (0 = left, 1 = down, 2 = right, 3 = up) matches Gymnasium. Outputs are real.

14.1 What reinforcement learning is

What it is

An **agent** repeatedly (1) observes the **state** of its **environment**, (2) picks an **action**, (3) receives a **reward** (positive = good, negative = penalty) and a new state. Its goal is **maximum total reward over time**, not the next reward. The learned rule "in this situation, do that" is a **policy**.

Term	Meaning	Treasure-hunt example
Agent	The learner and decision-maker	A robot, a game player, a program
Environment	Everything the agent interacts with; reacts to actions	The house, the maze, the board
State (s)	Description of the current situation	The robot's position
Action (a)	A choice available to the agent	Up / down / left / right
Reward (r)	Immediate numeric feedback	+10 treasure, −5 trap, −1 per step
Episode	One run from start to end (goal, failure, time limit)	One attempt at the maze
Policy (π)	Rule for choosing actions in each state	"In the hallway, go left"
Goal	Maximise total (discounted) reward	Reach the treasure by the best route



Intuition: like learning to ride a bicycle — nobody hands you the right movements; **act → observe → get feedback → shift future choices** toward what paid off. A cookie-seeking robot rewarded for getting closer, or a tic-tac-toe player whose rare wins tilt it toward winning moves, are the same loop.

How RL differs from other learning

	Supervised	Unsupervised	Reinforcement
Data	Inputs with labels	Inputs only	Experience generated by the agent's own actions
Feedback	"The right answer was X"	None	"That was worth +3" — how good, not what was right
Timing	Immediate	—	Often **delayed** (the win comes 40 moves later)
Data distribution	Fixed dataset	Fixed dataset	**Depends on the policy**
Key challenge	Generalisation	Finding structure	Exploration and **credit assignment** (which earlier action earned the reward?)

Why it matters

RL fits **sequential decisions under uncertainty** where actions change what happens next: games, robot control, resource allocation (cooling, bidding, inventory), dialogue, sequential recommendation. It can discover strategies humans never wrote down (AlphaGo's "move 37").

When not to use it

- Labelled examples of the right action exist → supervised learning (**behaviour cloning**: copy the expert) is simpler and far more data-efficient.
- Decisions don't affect future states → a **contextual bandit** (§14.4) suffices.
- No simulator and real trials are expensive or dangerous (medicine, finance) → pure **online RL** (learning by acting live) is risky; consider **offline RL** (learning only from logged data) or stay supervised.

TL;DR

An agent learns a policy by trial and error to maximise long-run reward. Feedback is a score (not the answer), often delayed, and the agent's behaviour decides its data.

14.2 Markov decision processes, returns and discounting

What it is

A **Markov decision process (MDP)** is the tuple (S, A, P, R, γ):

- **S** states, **A** actions.
- **P(s' | s, a)**: **transition probability** of landing in s' (deterministic worlds put 1 on one outcome; a slippery lake spreads it).
- **R(s, a, s')**: reward for that transition.
- **γ** ($0 \le \gamma \le 1$): the **discount factor**.

Markov property: next state and reward depend only on the current state and action, not the history. If the state omits something important (a car's speed, not just position), it isn't Markov — add the information or stack recent observations (DQN stacks 4 Atari frames).

Return and discount

The **return** from time t: $G_t = r_{t+1} + \gamma \cdot r_{t+2} + \gamma^2 \cdot r_{t+3} + \ldots = \Sigma_k \, \gamma^k \cdot r_{t+k+1}$

Why discount? Keeps sums finite for never-ending tasks; a reward now is worth more than later (like interest) and the future is uncertain. $\gamma = 0$ → only the next reward counts; $\gamma \to 1$ → far-sighted. With $\gamma = 0.9$ a reward 10 steps away counts $0.9^{10} \approx 0.35$; rough **horizon** = $1/(1-\gamma)$ (10 steps for 0.9, 100 for 0.99). E.g. $\gamma = 0.5$, rewards 1, 1, 1 → G = 1.75.

Key identity: $\mathbf{G_t = r_{t+1} + \gamma \cdot G_{t+1}}$ — everything in §14.3 builds on it.

Episodic tasks end; **continuing** tasks (thermostat, server fleet) never do and need $\gamma < 1$ or an average-reward formulation.

Worked example — the treasure grid

`code/treasure_grid.py`:

```
S.-..      S = start, G = treasure (+10, ends the episode)
..-..      - = trap (-5, you keep going)
...-.      every other move costs -1
.-...      (walking into the edge = you stay put, still -1)
....G
```

```python
def play(env, actions, gamma=0.9):
    s = env.reset()
    rewards = []
    for a in actions:
        s, r, done, _ = env.step(a)
        rewards.append(r)
        if done:
            break
    G = sum(gamma ** t * r for t, r in enumerate(rewards))   # discounted return
    return rewards, sum(rewards), G, done
```

```
A: right along the top, then down    rewards=[-1, -5, -1, -1, -1, -1, -1, 10]  total=-1  discounted G(gamma=0.9)=-4.03  reached=True
B: down the left edge, then right    rewards=[-1, -1, -1, -1, -1, -1, -1, 10]  total=+3  discounted G(gamma=0.9)=-0.43  reached=True
C: zig-zag through the middle        rewards=[-1, -1, -1, -1, -1, -1, -1, 10]  total=+3  discounted G(gamma=0.9)=-0.43  reached=True

random tries that reached the treasure within 30 moves: 307 / 2000
best of 2,000 random tries: total=-1  moves=RDDRRDRD
```

Path B by hand: $G = -(1 + 0.9 + \ldots + 0.9^6) + 10 \cdot 0.9^7 = -(1 - 0.9^7)/(1 - 0.9) + 10 \cdot 0.478 = -5.22 + 4.78 = \mathbf{-0.43}$.

- A hits a trap; B and C tie on the best total +3 (8 moves is the minimum). Discounting keeps the ranking, but +10 eight steps away is worth only 4.78.
- **Blind trial and error is weak**: 2,000 random attempts never found a +3 route; only 15% reached the treasure. The agent must *learn from* trials — what value functions and Q-learning do.

TL;DR

MDP = states, actions, transitions, rewards, γ. $G_t = r_{t+1} + \gamma G_{t+1}$; γ trades short- for long-term reward. Random exploration alone rarely finds the best plan.

14.3 Policies, value functions and the Bellman equations

What it is

- **Policy** π: **deterministic** π(s) = a, or **stochastic** π(a | s).
- **State-value function** Vπ(s) = **expected** return from s following π — "how good is it to be here?"
- **Action-value function** Qπ(s, a) = expected return taking a in s, then following π — "how good is doing *this* here?" (V averages over the policy's first action; Q fixes it.)
- **Optimal** V*(s), Q*(s, a): best achievable by any policy. Knowing Q*, act greedily: $\pi(s) = \text{argmax}_a\ Q(s, a)$**.

Why Q matters: choosing from V needs the transition model (one-step look-ahead); choosing from Q just compares numbers. Hence **model-free** methods (learning without P and R — Q-learning, DQN) learn Q.

The Bellman equations

Substitute $G_t = r_{t+1} + \gamma G_{t+1}$ into the definitions:

Equation	Formula	Used by
Bellman expectation (for a given π)	$V^\pi(s) = \Sigma_a\ \pi(a\|s)\ \Sigma_{s'}\ P(s'\|s,a)\ [R(s,a,s') + \gamma V^\pi(s')]$	Policy evaluation
Bellman optimality for V	$V^*(s) = \max_a\ \Sigma_{s'}\ P(s'\|s,a)\ [R(s,a,s') + \gamma V^*(s')]$	Value iteration
Bellman optimality for Q	$Q^*(s,a) = \Sigma_{s'}\ P(s'\|s,a)\ [R(s,a,s') + \gamma\ \max_{a'}\ Q^*(s',a')]$	Q-learning, DQN

(V of a terminal state is 0.) In words: *value here = reward now + discounted value of where you land* (averaged over outcomes; optimal version picks the best action).

Worked numbers (treasure grid, γ = 0.9, deterministic): cell above G → V* = **10**; next up −1 + 0.9·10 = **8.0**; then **6.2**, **4.58**, …; the start: **−0.43** — exactly path B's return.

TL;DR

Policy = behaviour; V = how good a state is; Q = how good an action in a state is. Bellman: value now = reward now + γ × value next. Greedy on Q* is optimal.

14.4 Exploration vs exploitation — multi-armed bandits

What it is

The **exploration–exploitation dilemma**: **exploit** the best-known option or **explore** others that might be better? Always exploiting can lock in a mediocre choice; always exploring wastes reward.

The **multi-armed bandit** isolates it: k slot machines with unknown payoff probabilities and a single state — RL without the sequential part. Real versions: which ad banner, headline or thumbnail to show; which new item gets traffic.

Regret = reward from always pulling the best arm minus what you got. Good strategies make regret grow **logarithmically** (doubling time adds a fixed amount).

The strategies

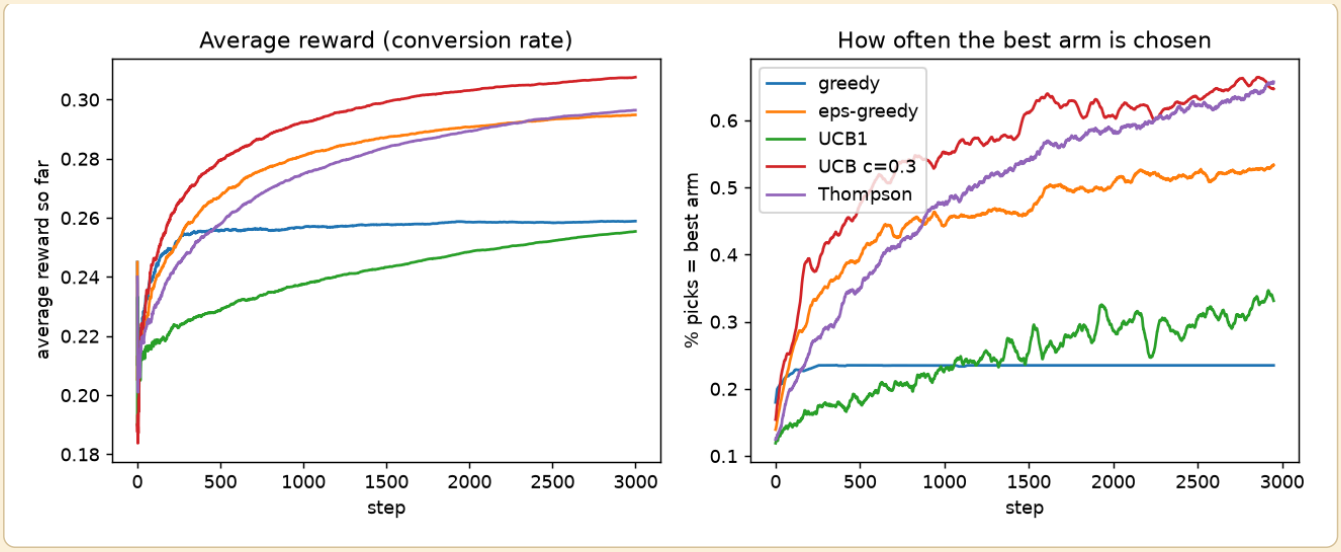
1. **Greedy**: try each arm once, then always the best average. Gets stuck on an arm that got lucky early — the error is never corrected.
2. **ε-greedy**: with probability ε (e.g. 0.1) a random arm, else the best. Simple, robust, but explores uselessly at rate ε forever (decay ε).
3. **UCB (Upper Confidence Bound)** — *optimism in the face of uncertainty*: $\text{argmax}_a\ [\ Q(a) + c\cdot\sqrt{(\ln t\ /\ N_a)}\ ]$, N_a = pulls of arm a. Rarely tried arms get a big bonus that shrinks like $1/\sqrt{N_a}$. UCB1 uses c = √2 (theory, rewards in [0, 1]).
4. **Thompson sampling** — *probability matching*: keep a belief (**posterior**) over each arm's payoff — for 0/1 rewards a **Beta(wins + 1, losses + 1)** distribution over rates in [0, 1], narrowing with evidence — draw one sample per arm, pull the highest. Each arm is chosen with the probability that it's the best.

Code — `code/bandits.py`

10 arms with **conversion rates** (chance a shown option leads to a click/purchase) from 5–35%, 3,000 pulls, averaged over 200 problems.

```
if policy == "eps-greedy":
    a = int(rng.integers(N_ARMS)) if rng.random() < eps else int(np.argmax(q))
elif policy.startswith("UCB"):
    c = np.sqrt(2) if policy == "UCB1" else 0.3
    a = t if t < N_ARMS else int(np.argmax(q + c * np.sqrt(np.log(t + 1) / n)))
elif policy == "Thompson":
    a = int(np.argmax(rng.beta(wins, losses)))           # sample a belief, act on it
r = float(rng.random() < true_p[a])                      # conversion (1) or not (0)
n[a] += 1
q[a] += (r - q[a]) / n[a]                                # incremental mean
```

```
policy      avg reward  % optimal arm (last 500)  total regret
greedy      0.2589       23.5%                197.0
eps-greedy  0.2948       52.4%                 89.1
UCB1        0.2554       31.5%                207.4
UCB c=0.3   0.3076       64.9%                 50.9
Thompson    0.2964       63.4%                 84.4
```



- **Greedy** locks in early: best arm in only 23.5% of problems.
- **ε-greedy** and **Thompson** learn steadily; Thompson picks the best arm most often among untuned methods, with no ε to tune.
- **UCB1**'s textbook c = √2 explores *too much* at this horizon (bonus large vs rewards ~0.2–0.3). c = 0.3 gives the lowest regret. Theoretical constants guarantee long-run behaviour, not best finite-horizon performance — tune on simulations of your traffic.

Contextual bandits

Choose the arm using **context** (user features, time, device), e.g. **LinUCB** (a linear model per arm plus a UCB bonus). Yahoo! used LinUCB for front-page news (Li et al., 2010); Netflix describes contextual bandits for choosing artwork per member. Arguably RL's most widely deployed industrial form — simple, relatively safe, easy to measure.

Common mistakes

- A fixed ε forever — decay it, or use Thompson/UCB.
- Ignoring **non-stationarity** (click rates drift) — sliding window or constant step size instead of an all-time mean.
- Declaring a winner from a handful of pulls.

In the interview

> **30-second answer:** "Exploration vs exploitation is using the best-known option vs gathering information about others. Bandits isolate it. ε-greedy explores randomly at a fixed rate; UCB adds an optimism bonus that shrinks with tries; Thompson samples from a posterior over each arm's reward and acts greedily on the sample. In production I'd use a contextual bandit — Thompson or LinUCB — for headline or thumbnail selection."

- *Bandit vs A/B test?* A/B splits traffic evenly until the end; a bandit shifts traffic toward winners during the test — less regret, less clean statistics.

TL;DR

Explore to learn, exploit to earn: ε-greedy, UCB (optimism bonus), Thompson (posterior sampling). Contextual bandits are RL's biggest industrial success.

14.5 Dynamic programming — planning when the model is known

What it is

If you **know** P and R, compute optimal values without trial and error by repeatedly applying the Bellman equations — **dynamic programming (DP)** (Bellman, 1950s).

- **Policy evaluation**: from V = 0, repeatedly replace V(s) with the Bellman *expectation* right-hand side until stable.
- **Policy improvement**: make π greedy w.r.t. V (one-step look-ahead with the model).
- **Policy iteration**: alternate evaluation and improvement until the policy stops changing; few rounds.
- **Value iteration**: apply the Bellman *optimality* (max) update sweep after sweep, then read off the greedy policy.

Code — `code/value_iteration.py`

```python
def q_from_v(env, V, s, gamma):
    """One-step look-ahead: Q(s,a) = sum_s' P(s'|s,a) * [r + gamma * V(s') * (not done)]."""
    return np.array([sum(p * (r + gamma * V[s2] * (not done)) for p, s2, r, done in env.transitions(s, a))
                     for a in range(env.nA)])

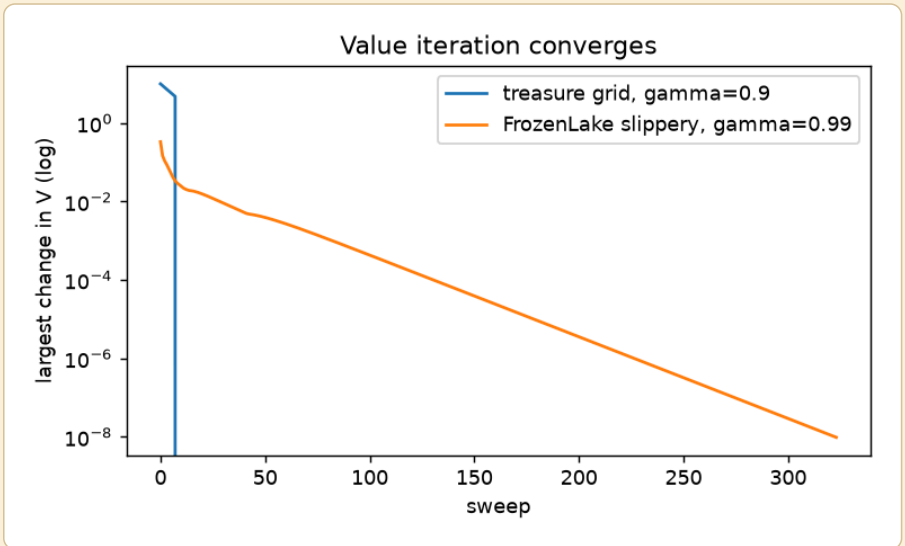
def value_iteration(env, gamma, tol=1e-8):
    V, deltas = np.zeros(env.nS), []
    while True:
        delta = 0.0
        for s in range(env.nS):
            if env.is_terminal(s):
                continue
            best = q_from_v(env, V, s, gamma).max()      # Bellman OPTIMALITY backup
            delta = max(delta, abs(best - V[s]))
            V[s] = best
        deltas.append(delta)
        if delta < tol:
            break
    policy = np.array([np.argmax(q_from_v(env, V, s, gamma)) for s in range(env.nS)])
    return V, policy, deltas
```

FrozenLake (a classic toy): cross a frozen lake from S to G without falling into holes H. **Slippery**: you move the chosen way only 1/3 of the time and slide to each side 1/3. Reward 1 only at G.

```
SFFF      F = frozen (safe), H = hole (episode ends, reward 0)
FHFH
FFFH
HFFG
```

```
== Treasure grid (deterministic), gamma = 0.9 ==
value iteration converged in 9 sweeps
V*:
 -0.43   0.63   1.81   3.12   4.58
  0.63   1.81   3.12   4.58   6.20
  1.81   3.12   4.58   6.20   8.00
  3.12   4.58   6.20   8.00  10.00
  4.58   6.20   8.00  10.00   0.00
greedy policy:
vv>vv
vvv>v
v>vvv
vvvvv
>>>>G

== FrozenLake 4x4, slippery (2/3 of moves slide sideways), gamma = 0.99 ==
value iteration converged in 324 sweeps
V*:
  0.54   0.50   0.47   0.46
  0.56   0.00   0.36   0.00
  0.59   0.64   0.62   0.00
  0.00   0.74   0.86   0.00
policy:
<^^^
<H<H
^v<H
H>vG
policy iteration: 7 improvement rounds, same policy as VI: True
simulated success rate of this policy: 73.8%
a random fixed policy succeeds:         2.9%
```



- Treasure grid: values rise smoothly toward G (§14.3's numbers); the greedy policy avoids every trap.

- Lake: the optimal policy looks **strange** — from the start it presses *left*, into the wall. Slips go only sideways, so pushing against the wall means staying put or sliding down — never into a hole; top-row cells press *up* for the same reason. Optimal under uncertainty = **avoiding risk**, not the shortest path: ~74% success within 100 steps vs 3% random.
- 324 sweeps on the lake (γ = 0.99, slow propagation) vs 9 on the grid; policy iteration needed just 7 rounds.

Trade-offs / when not to use it

DP needs the **full model** and sweeps **every state** — fine for thousands of states, impossible for Go (≈10^{170} positions) or **continuous** states. It's what model-free methods approximate from samples.

TL;DR

Known model → value iteration (repeat the optimality backup) or policy iteration (evaluate, act greedily, repeat) give exact optimal policies. Great for small, fully known problems.

14.6 Learning from experience: Monte Carlo vs temporal difference

What it is

Usually P and R are unknown; **model-free** methods estimate values from sampled experience. To estimate V_π:

	Monte Carlo (MC)	**Temporal difference, TD(0)**
Update target	Actual return G_t at episode end	$r_{t+1} + \gamma V(s_{t+1})$ — one real reward + current estimate
Update rule	$V(s) \leftarrow V(s) + \alpha[G_t - V(s)]$	$V(s) \leftarrow V(s) + \alpha[r_{t+1} + \gamma V(s_{t+1}) - V(s)]$
When it can update	After the episode	Every step (works for never-ending tasks)
Bias (systematic error)	Unbiased (real return)	Biased (uses its own estimate — **bootstrapping**)
Variance (scatter of estimates)	High (sum of many random steps)	Low (one random step)
Usually	Slower, simpler	Faster in practice

$\delta = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$ is the **TD error** — "how much better or worse did this step go than expected?" — central to RL (and paralleled by dopamine signalling, Schultz, Dayan & Montague, 1997). α = step size. Example: V(s) = 0.5, r = 0, V(s') = 0.6, γ = 1, α = 0.1 → δ = 0.1, V(s) = **0.51**.

Commute analogy: MC updates the trip estimate on arrival; TD updates at each landmark ("at the bridge 5 minutes late → adjust now").

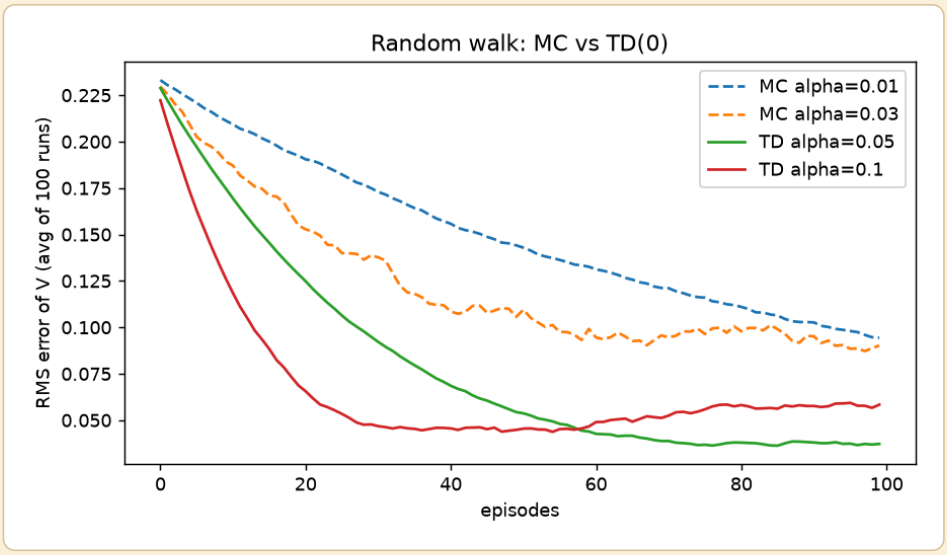
Code — `code/mc_vs_td.py` (5-state random walk)

States A–E; start at C; step left/right at random; right end +1, left end 0. True values 1/6 … 5/6.

```
if method == "MC":        # wait until the end, then move each V(s) toward the actual return G
    for s in states:
        V[s] += alpha * (G - V[s])
else:                     # TD(0): after every step, move V(s) toward r + gamma * V(s')
    for t, s in enumerate(states):
        target = gamma * V[states[t + 1]] if t + 1 < len(states) else final_r
        V[s] += alpha * (target - V[s])
```

(Target written with the zero intermediate reward; every-visit, constant-α MC.)

```
MC alpha=0.01  RMS error after 10 eps 0.211, after 100 eps 0.094
MC alpha=0.03  RMS error after 10 eps 0.189, after 100 eps 0.090
TD alpha=0.05  RMS error after 10 eps 0.175, after 100 eps 0.037
TD alpha=0.1   RMS error after 10 eps 0.126, after 100 eps 0.058
TD estimate after 300 episodes: [0.22 0.41 0.51 0.67 0.84]  true: [0.17 0.33 0.5  0.67 0.83]
```



TD learns faster (lower **RMS error** vs true values), as in Sutton & Barto. Step-size trade-off: larger α learns faster early but leaves more noise (TD α = 0.1 plateaus above 0.05); constant α keeps jittering around the truth.

n-step returns and TD(λ) blend the two: n real rewards before bootstrapping (n = 1 TD, n = ∞ MC), or average all n-step targets with weights $(1 - \lambda)\lambda^{n-1}$ (λ = 0 → TD(0), λ = 1 → MC). Deep RL uses this (e.g. **GAE**, generalised advantage estimation, in PPO).

TL;DR

MC → actual return (unbiased, noisy, waits for the end). TD → reward + γ·V(next) (bootstraps, lower variance, every step). The TD error δ drives most of RL.

14.7 Q-learning and SARSA — tabular control

What it is

To **act** without a model, learn **Q(s, a)** with TD updates while behaving ε-greedily.

Q-learning (Watkins, 1989) — *off-policy*: $Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)]$

SARSA (Rummery & Niranjan, 1994) — *on-policy*, named after (s, a, r, s', a'): $Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \cdot Q(s', a') - Q(s, a)]$, a' = the action actually taken next.

Off-policy: learn about one policy (greedy — the max) while following another (ε-greedy). **On-policy**: learn the value of the policy actually followed, exploration included.

How it works (Q-learning loop)

1. Q = 0. 2. Each episode, s ← start. 3. Choose a ε-greedily. 4. Take a; observe r, s'. 5. Update toward $r + \gamma \cdot \max_{a'} Q(s', a')$ (0 instead of the max if s' is terminal). 6. s ← s' until the episode ends. 7. Decay ε over episodes.

Tabular Q-learning (one table entry per state–action pair) **converges** to Q* if every pair is visited infinitely often and step sizes decay appropriately.

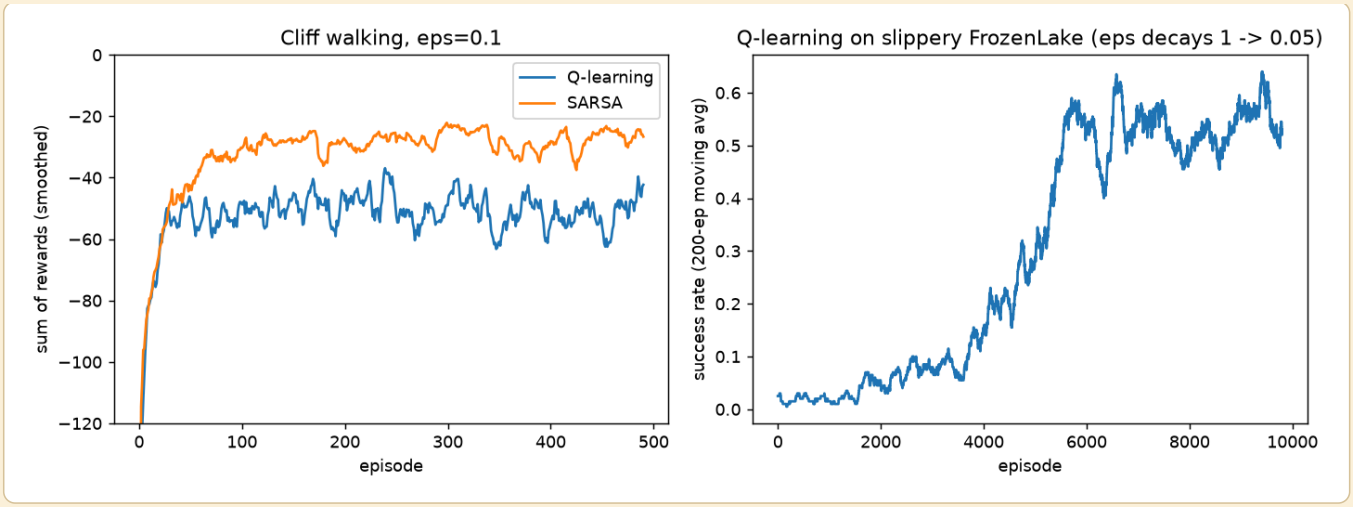
Code — `code/q_learning_sarsa.py`

```python
def train(env, method, episodes, alpha, gamma, eps_schedule, seed=0):
    rng = np.random.default_rng(seed)
    Q = np.zeros((env.nS, env.nA))
    returns = []
    for ep in range(episodes):
        eps = eps_schedule(ep)
        s = env.reset()
        a = epsilon_greedy(Q, s, eps, rng)
        total, done, trunc = 0.0, False, False
        while not (done or trunc):
            s2, r, done, trunc = env.step(a)
            a2 = epsilon_greedy(Q, s2, eps, rng)
            if method == "Q-learning":           # off-policy: bootstrap from the BEST next action
                target = r + gamma * Q[s2].max() * (not done)
            else:                                # SARSA, on-policy: from the action we WILL take
                target = r + gamma * Q[s2, a2] * (not done)
            Q[s, a] += alpha * (target - Q[s, a])
            s, a, total = s2, a2, total + r
        returns.append(total)
    return Q, np.array(returns)
```

Part 1 — Cliff walking (Sutton & Barto, Example 6.6): S to G along the bottom of a 4×12 grid; cells between (C) are a cliff: −100 and back to S. Each step −1; ε fixed at 0.1. **Part 2 — slippery FrozenLake**, ε decaying 1 → 0.05 over 6,000 of 10,000 episodes.

```
Q-learning avg return per episode (last 100, while exploring eps=0.1):   -51.7
greedy path learned:
>^>>>>>>>>>v
>>v>>v>>>>vv
>>>>>>>>>>v
^CCCCCCCCCCG
SARSA      avg return per episode (last 100, while exploring eps=0.1):   -27.2
greedy path learned:
>>>>>>>>>>v
^>>^^^^^>^vv
^^^^^^>>>^>v
^CCCCCCCCCCG

FrozenLake Q-learning policy:
<^^^
<H<H
^v<H
H>vG
greedy success rate: Q-learning 73.4% | value-iteration optimum 74.0%
```



- **Q-learning** learns the *optimal* path along the cliff edge (row 3), but while exploring a random step sometimes drops it off — worse training return (−51.7).
- **SARSA**'s values include its own random slips, so it prefers the **safe path** along the top — longer, but more reward while exploring (−27.2).
- Which is right? If exploration continues in deployment (a robot that slips), SARSA's caution helps; if acting greedily after training, Q-learning's policy is better.
- On FrozenLake, Q-learning — not knowing the slip probabilities — **rediscovers value iteration's exact policy** (73–74%), but needed thousands of episodes for what DP did from the model: **sample inefficiency**.

Trade-offs

	Q-learning	SARSA
Type	Off-policy (learns greedy policy)	On-policy (learns behaviour policy)
Target	max over next actions	the chosen next action
During training	Riskier (optimistic about its own future)	Safer
Learns from others' data / replay	Yes — basis of DQN, replay buffers, offline RL	Not directly
Known issue	**Maximisation bias** (max over noisy values overestimates; fix: Double Q-learning — one estimator selects, another evaluates)	Converges to optimal only as ε → 0

Common mistakes

- Bootstrapping from a terminal state (use 0 — the `(not done)` factor). A **time-limit** truncation isn't a real terminal; ideally keep bootstrapping there.
- Never decaying ε, or decaying so fast the agent never finds the goal (with **sparse rewards**).
- Breaking ties by always taking action 0 when Q is all zeros — the agent walks into a wall forever. Break ties randomly.
- Tabular methods on large state spaces — use function approximation (§14.8).

In the interview

> **30-second answer:** "Q-learning is off-policy TD: it updates Q(s,a) toward r + γ·max Q(s',·), learning the optimal greedy policy while exploring ε-greedily. SARSA is on-policy: it bootstraps from the action it actually takes next, so it learns the value of its exploratory behaviour and is safer — as cliff walking shows."

TL;DR

Q-learning: off-policy, max target → optimal policy. SARSA: on-policy, target Q(s', a') → learns what it does, safer. Both are TD control with ε-greedy exploration.

14.8 Deep reinforcement learning — DQN

What it is

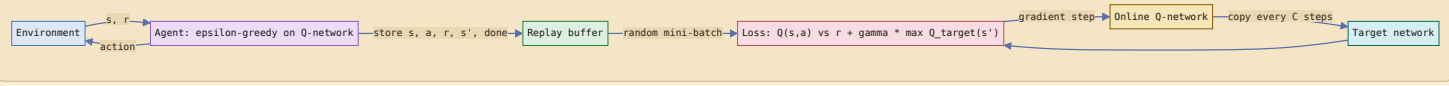
A Q-table needs one entry per state–action pair; Atari screens or robot joint angles have far too many states. **Function approximation** replaces the table with Q(s, a; θ) — a neural network taking the state and outputting one Q-value per action, generalising across similar states.

DQN (Deep Q-Network) — DeepMind, Mnih et al., *Nature* 2015 — played 49 Atari 2600 games from raw pixels and score with one architecture and **hyperparameter** set, reaching human level on many. Naive Q-learning + neural net is unstable; DQN added:

1. **Experience replay**: store transitions (s, a, r, s', done) in a large buffer; train on **random mini-batches** from it. Breaks the correlation between consecutive steps (which violates the **i.i.d.** assumption of SGD) and reuses each experience many times.

2. **Target network**: compute the TD target r + γ·max$_{a'}$ Q(s', a'; θ⁻) with a **frozen copy** θ⁻, copied from the **online network** only every C steps — otherwise each update moves the target it's chasing (oscillation or divergence).

Plus ε-greedy, the **Huber loss** (less sensitive to large errors), and frame stacking for motion.



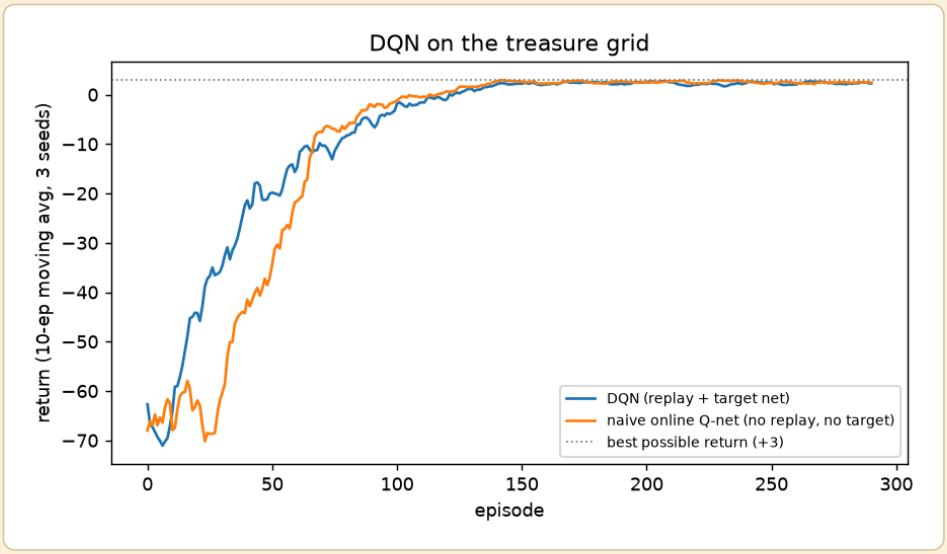
The **deadly triad** (Sutton & Barto): function approximation + bootstrapping + off-policy learning can **diverge**. DQN's tricks mitigate, not guarantee.

Code — `code/dqn.py` (DQN on the treasure grid)

State **one-hot** encoded (25 zeros with one 1); a 64-unit MLP outputs 4 Q-values.

```
buffer = deque(maxlen=5000)                                # experience replay memory
...
buffer.append((s, a, r, s2, done))
batch = random.sample(buffer, min(64, len(buffer))) if use_replay else [buffer[-1]]
...
with torch.no_grad():                                      # TD target from the frozen network
    nxt = (target_net if use_target else q_net)(S2).max(1).values
    y = R + gamma * nxt * (1 - D)
q = q_net(S).gather(1, A[:, None]).squeeze(1)
loss = nn.functional.smooth_l1_loss(q, y)                  # Huber loss
opt.zero_grad(); loss.backward(); opt.step()
if use_target and steps % 200 == 0:
    target_net.load_state_dict(q_net.state_dict())         # periodic hard copy
```

```
DQN (replay + target net)                 mean return, last 50 episodes: +2.31
naive online Q-net (no replay, no target)  mean return, last 50 episodes: +2.42
greedy policy of the DQN (seed 0):
vv>>v
vv>>v
>>vvv
vvvvv
>>>>G
```



- DQN learns a trap-free 8-move route (+3; last-50 average lower because ε = 0.05 exploration continues).
- With replay it learns noticeably **faster** (each experience reused).
- On this 25-state problem the naive version also gets there. Honest takeaway: replay and target nets matter at scale — large nets, pixel inputs, long training — where naive versions oscillate or diverge.

Beyond DQN

- **Double DQN**: select the next action with the online net, evaluate it with the target net → less overestimation.
- **Dueling DQN**: separate streams for V(s) and the **advantage** A(s, a) (how much better a is than average in s).
- **Prioritised replay**: sample large-TD-error transitions more often.
- **Rainbow** (2017): combines these and more.
- DQN handles **discrete** actions only (a max over infinitely many continuous actions is impractical); continuous control (robot **torques**) uses actor-critic methods — DDPG, TD3, SAC.

In the interview

30-second answer: "DQN replaces the Q-table with a neural network trained toward r + γ·max Q(s', ·). Experience replay, sampled at random, breaks correlations and reuses data; a target network, refreshed every C steps, stops the target moving. Discrete actions only; continuous control uses actor-critic methods like SAC or TD3."

TL;DR

Neural Q-function stabilised by experience replay (decorrelate, reuse) and a target network (freeze the target). Discrete actions; beware the deadly triad.

14.9 Policy gradients, REINFORCE and actor-critic

What it is

Policy-gradient methods learn the policy **directly**: a parameterised $\pi(a \mid s; \theta)$ (a softmax over action scores, or a net outputting a **Gaussian** over torques), improved by **gradient ascent** on expected return $J(\theta)$.

Why?

- Natural for **continuous** and huge action spaces (no max over actions).
- Can learn **stochastic** policies (rock-paper-scissors; **partial observability**, where the agent can't see the full state).
- Small parameter changes → smooth policy changes (vs a greedy argmax that can flip).

The policy-gradient theorem, intuitively

$\nabla\theta\ J(\theta) = E[\ \nabla\theta \log \pi(a \mid s; \theta) \cdot (\text{return that followed})\ ]$

Make actions that led to high returns more likely, low returns less likely, in proportion to the outcome. $\nabla \log \pi(a|s)$ is the direction raising the taken action's probability; for a softmax with scores $\theta[s, \cdot]$ it is (one-hot(a) − $\pi(\cdot|s)$) — e.g. $\pi = [0.7, 0.2, 0.1]$, action 0 → $[0.3, -0.2, -0.1]$.

REINFORCE (Williams, 1992)

1. Run an episode; record (s, a, r) each step.
2. Compute each return G_t.
3. $\theta \leftarrow \theta + \alpha \cdot G_t \cdot \nabla\theta \log \pi(a_t \mid s_t; \theta)$. (The textbook γ^t factor is usually dropped, as in our code.)

Problem: variance. G_t depends on everything afterwards, so the gradient is very noisy. If all returns are negative (−1 per step), every action taken is pushed down and learning can lock into bad behaviour.

Fix: a baseline. Subtract b(s), usually a learned V(s): the **advantage** $A_t = G_t - V(s_t)$ ("better or worse than usual from here?"). It doesn't change the expected gradient, only cuts variance.

Actor-critic

Use the **TD error** $\delta = r + \gamma V(s') - V(s)$ as the advantage:

- **actor** (policy π) chooses actions, updated by $\delta \cdot \nabla \log \pi$;
- **critic** (value function V) judges them, updated by TD. Every-step updates, lower variance (some bias). Variants: **A2C/A3C**, **PPO**, **SAC**.

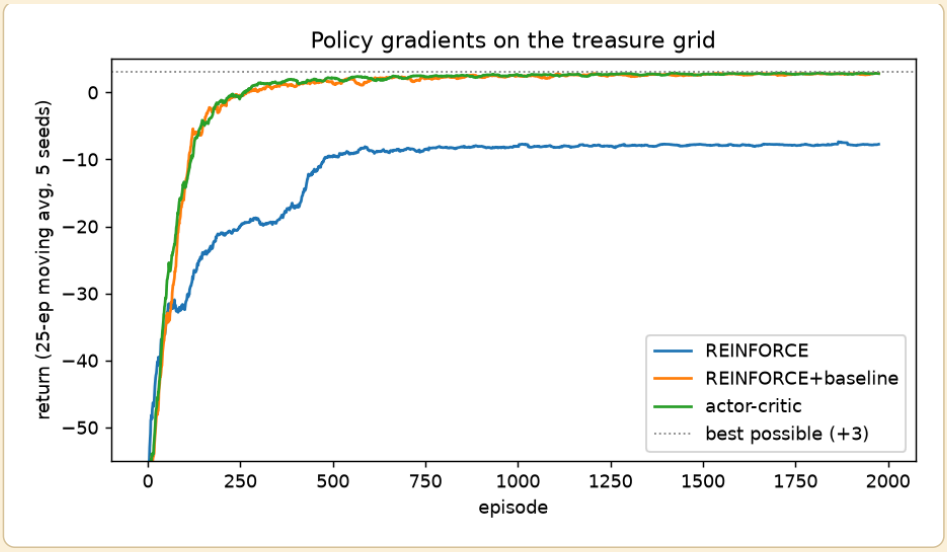
PPO (Proximal Policy Optimization, Schulman et al., 2017) is the default workhorse: an actor-critic that **clips** how far each update moves the policy from the data-collecting one. With ratio $\rho = \pi_new(a|s) / \pi_old(a|s)$ and advantage A it maximises $\min(\rho \cdot A, \text{clip}(\rho, 1 - \varepsilon, 1 + \varepsilon) \cdot A)$, $\varepsilon \approx 0.2$ — pushing ρ outside the band earns nothing, preventing destructive steps. It was the RL algorithm in InstructGPT (2022), the RLHF recipe behind early ChatGPT. Cost: on-policy, so old data can't be reused much — more samples than replay-based methods.

Code — `code/policy_gradient.py` (tabular softmax policy, treasure grid)

```
p = softmax(theta[s])
a = rng.choice(4, p=p)
s2, r, done, trunc = env.step(a)
if method == "actor-critic":          # update every step using the TD error
    delta = r + GAMMA * V[s2] * (not done) - V[s]
    V[s] += lr_v * delta              # critic
    grad = -p; grad[a] += 1           # d log pi(a|s) / d theta[s]
    theta[s] += lr * delta * grad     # actor
...
# REINFORCE (after the episode), going backwards to accumulate the return
G = r + GAMMA * G
adv = G if method == "REINFORCE" else G - V[s]   # baseline version subtracts V(s)
theta[s] += lr * adv * grad
```

5 seeds, 2,000 episodes each:

```
REINFORCE           mean return: episodes 1-100 -40.96 | last 100 -7.80 | std across seeds (last 100) 21.10
REINFORCE+baseline  mean return: episodes 1-100 -42.48 | last 100 +2.75 | std across seeds (last 100) 0.08
actor-critic        mean return: episodes 1-100 -39.86 | last 100 +2.80 | std across seeds (last 100) 0.02
```



- **Plain REINFORCE** is unreliable: four seeds learn a good route, one gets stuck never reaching the treasure (−50/episode) — mean −7.8, spread 21.1. The variance problem in action.
- **Baseline** fixes it — every seed near the optimal +3.
- **Actor-critic** learns as fast with the smallest spread (lower-variance TD error).

Value-based vs policy-based

	Value-based (Q-learning, DQN)	Policy-based (REINFORCE, PPO)	Actor-critic
Learns	Q(s, a)	π(a | s)	Both
Action spaces	Discrete	Discrete or continuous	Both
Sample efficiency	Higher (off-policy replay)	Lower (on-policy)	Middle
Stability	Can diverge (deadly triad)	High variance	Better balanced
Examples	DQN (Atari)	REINFORCE	PPO (RLHF, games, robotics), SAC (robotics)

In the interview

30-second answer: "Policy gradients adjust the policy's parameters so actions followed by high returns become more likely: gradient = average of ∇log π(a|s) times the return. REINFORCE uses the full return and is noisy; subtracting a baseline V(s) gives the advantage and cuts variance without bias; actor-critic uses the TD error and updates every step. PPO clips the probability ratio, making it stable enough to be the default."

TL;DR

Push up log-probabilities of actions in proportion to outcome. REINFORCE noisy → baseline (advantage = G − V) → actor-critic (TD error) → PPO (clipped, default).

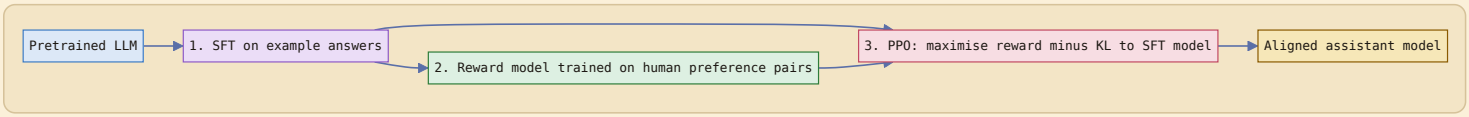
14.10 RLHF — the bridge to large language models

What it is

Reinforcement Learning from Human Feedback (RLHF) tunes a pretrained language model toward responses people prefer (helpful, honest, harmless). Environment = prompt; action = the generated response (a sequence of **token** choices); reward = a model trained on human preferences.

How it works (InstructGPT recipe, Ouyang et al., 2022)

1. **Supervised fine-tuning (SFT)** on human-written example answers.
2. **Reward model (RM)**: people compare two or more responses to a prompt (comparisons are easier and more consistent than absolute scores); train the RM to score preferred ones higher — pairwise loss maximising log σ(score_preferred − score_rejected).
3. **RL with PPO**: generate, score with the RM, update the LLM to increase reward **minus a KL penalty** (KL divergence = how different two distributions are) keeping it close to the SFT model. Without it the policy drifts into gibberish or RM exploits — reward hacking (§14.12).



Variants you'll hear about

- **RLAIF / Constitutional AI** (Anthropic, Bai et al., 2022): an AI model guided by written principles produces the preference labels, reducing human labelling of harmful content.

- **DPO** (Direct Preference Optimization, Rafailov et al., 2023): removes the separate reward model and RL loop — optimises the policy directly on preference pairs with a classification-style loss. Simpler; popular for open models.
- **RL with verifiable rewards** for reasoning: reward = an automatic check (maths answer correct, unit tests pass). DeepSeek's **GRPO** (DeepSeekMath, 2024; used for DeepSeek-R1, 2025) is a PPO variant that samples a group of answers per prompt and scores each relative to the group (reward minus group mean, divided by group std) instead of a learned critic.

Why it is still "RL"

Outputs change the training data (on-policy sampling), the reward is a scalar judgement not a correct answer, and credit is assigned across many tokens. All RL pitfalls apply: reward hacking (verbose or flattering answers scoring well with the RM), instability, cost.

> **TL;DR**
>
> SFT → reward model from preference comparisons → PPO with a KL leash. DPO removes the RL loop; RLAIF replaces human labels with AI feedback; verifiable-reward RL powers reasoning models. Full LLM pipeline: AI book's large language models chapter.

14.11 Real-world RL

Domain	Example (publicly described)	What kind of RL
Games	**AlphaGo** (DeepMind) beat Lee Sedol 4–1 in March 2016: policy and value networks trained first on expert games, then improved by **self-play** RL, combined with **Monte Carlo tree search** (planning by simulating many future move sequences). **AlphaGo Zero** (2017): self-play alone; **AlphaZero** (2018): chess, shogi and Go with one algorithm; **MuZero** (Nature, 2020): same without being told the rules, learning its own model of the dynamics. Also DQN on Atari (2015), OpenAI Five (Dota 2, 2019), AlphaStar (StarCraft II, 2019)	Self-play, deep RL + planning
Robotics	OpenAI's Dactyl hand solved a Rubik's Cube (2019) with a policy trained entirely in simulation using automatic **domain randomisation** (randomised physics and visuals so it copes with the real hand); RL did the finger control, the face-turn sequence came from Kociemba's solver. Legged locomotion is now commonly trained **sim-to-real**	Deep RL (PPO/SAC) in simulation
Recommenders (Chapter 13)	Yahoo! news with LinUCB (2010); Netflix artwork with contextual bandits; Spotify home shelves with bandits; RL for long-term engagement (Google's REINFORCE-based YouTube recommender, Chen et al., 2019)	Contextual bandits (most common), off-policy RL
Ads & pricing	Bandits for ad/creative selection; **real-time bidding** (a per-slot auction run in milliseconds) and **budget pacing** (spreading spend over the day) as sequential decisions	Bandits, constrained RL
Operations	DeepMind reported (2016) neural-network recommendations cut Google data-centre cooling energy by up to 40%; a 2018 follow-up controlled cooling directly	Control / RL-style optimisation
LLMs	RLHF / RLAIF for ChatGPT, Claude and others; verifiable-reward RL for reasoning models	PPO, DPO, GRPO

The honest pattern: RL shines with a **simulator or cheap trials** (games, simulated robots) or **one-step** decisions (bandits). Full sequential RL in live products is rarer — exploration costs real money or user trust.

14.12 Pitfalls

Reward hacking (specification gaming)

The agent maximises **the reward you wrote**, not the goal you meant. `code/reward_hacking.py` adds a "+3 bonus" tile beside the start to encourage exploration:

```
S+-..
..-..
...-.
.-...
....G
```

```
bonus tile worth +3: first 12 moves RUUUUUUUUUUU reward +36 reached treasure: False   V(start) = 30.00
bonus tile worth +0: first 12 moves RDDRDDRR     reward +4 reached treasure: True   V(start) = 0.57
```

The optimal policy steps onto the bonus and **walks into the wall forever**: bumping the edge keeps it on the tile, +3 every step — worth $3/(1-0.9) = 30$, far more than a one-off +10. A bug in the reward, not the learner. Real cases: OpenAI's CoastRunners boat (2016) circling to hit respawning targets instead of finishing; chat models learning longer or more flattering answers score higher with the RM.

Defences: reward the outcome, not a **proxy** (and only once, not re-collectable); penalise side effects; constrain the policy (RLHF's KL penalty); watch actual behaviour, not just the reward curve; **red-team** the reward.

Other pitfalls

Pitfall	What it means	Mitigations
Sample inefficiency	Thousands of episodes for a 16-cell lake; hundreds of millions of Atari frames in some deep-RL variants	Simulators, **model-based RL** (learn an environment model and plan), replay, offline data, **imitation learning** from demonstrations to start
Sparse rewards	Only reward is at the far goal; random exploration never finds it	**Reward shaping** (intermediate rewards — carefully), **curriculum** (easy tasks first), **curiosity bonuses** (reward for new states), demonstrations
Credit assignment	Which of 200 moves earned the win?	Discounting, TD learning, advantage estimation (GAE)
Instability & seed sensitivity	Results swing between runs (plain REINFORCE's std 21)	Report many seeds with confidence intervals; target nets, clipping (PPO), normalisation
Sim-to-real gap	Policies overfit simulator quirks	Domain randomisation, **system identification** (measuring real physics), fine-tune on real data
Safe exploration	Random actions break robots or lose money	Constrained RL, safety layers, offline RL, human oversight
Distribution shift (offline RL)	The policy wants actions the logs never tried	Conservative methods (CQL, IQL); stay close to the data policy
Non-stationarity	Users, markets, opponents change	Continual learning, sliding windows, monitoring
Evaluation is hard	Can't A/B test every idea; off-policy evaluation is noisy	**IPS** (reweight logged data by how likely the old policy was to act that way) / **doubly robust** estimators, **shadow mode** (new policy runs silently alongside), staged rollouts

In the interview

30-second answer ("When would you use RL, and what goes wrong?"): "When decisions are sequential and change future states, and I can simulate or safely experiment — otherwise supervised learning or a contextual bandit is simpler. Typical problems: reward hacking, sample inefficiency, sparse rewards, seed instability, sim-to-real gap. I'd start with a bandit or offline evaluation, design and audit the reward carefully, and roll out in stages with **guardrails**."

- *Bandit or full RL for a recommender?* Contextual bandit for the immediate response; full RL only if you can model long-term effects (retention) and evaluate off-policy credibly.
- *Evaluating before launch?* Simulation; off-policy evaluation (IPS, doubly robust) on logs with known **propensities** (probability the logging policy chose each action); shadow mode; small **canary** traffic.

Chapter TL;DR

- **RL**: state → action → reward + next state; maximise discounted return; feedback evaluative and delayed; behaviour shapes data.
- **MDP** (S, A, P, R, γ); $G_t = r_{t+1} + \gamma G_{t+1}$; γ sets the horizon.
- **V, Q, Bellman**: value = reward now + γ × value next; greedy on Q* is optimal.
- **Bandits**: ε-greedy, UCB, Thompson; contextual bandits are RL's biggest industrial use; tune exploration for your horizon.
- **DP** (model known): value/policy iteration — solved the slippery lake exactly (≈74%, "hug the wall").
- **MC vs TD**: actual return (unbiased, noisy) vs bootstrapped target (lower variance, every step).
- **Q-learning** (off-policy, optimal, risky while exploring) vs **SARSA** (on-policy, safer cliff path).
- **DQN**: neural Q + replay + target network; discrete actions.
- **Policy gradients**: REINFORCE → baseline → actor-critic → PPO.
- **RLHF**: SFT → reward model → PPO with KL leash; DPO, RLAIF, verifiable-reward RL (GRPO).
- **Pitfalls**: reward hacking, sample inefficiency, sparse rewards, seed instability, sim-to-real, unsafe exploration.

Chapter 15 — ML System Design: The Interview Framework

"Design YouTube's recommendations." "Design a fraud detector." "Add AI to our support desk." These now sit beside "Design Twitter" in system design loops, and are asked of ordinary **backend engineers**, not only ML specialists. Everything about load balancers, caches, queues and databases still applies (System Design Fundamentals). The new part: one box **learns from data** — its output can be wrong while the request succeeds, its quality can decay while the code is unchanged, and each call may cost real money.

This chapter gives a repeatable 45-minute method: the five question forms and role expectations; four classic assumptions that break; three question families (predictive, generative, agentic); a seven-step framework (frame → success metrics → data → baseline → serving → keep-it-working → failures and cost); turning a **business goal** into an **ML objective**; scoping a vague question (capstone); the mistakes that sink answers. Chapter 16 goes deep on each production block (training pipelines, feature stores, serving, evaluation, monitoring, A/B tests, **MLOps** — practices and tooling for running ML in production).

Code is in `code/`; outputs are real.

15.1 ML design vs classic system design

What it is

An **ML (or AI) system design question** asks for a product in which at least one component is a **model** — a function learned from data rather than written as rules — mapping an input (watch history, a transaction, a question) to an output (score, label, number, text).

Term	Meaning	Example
Training	Fitting or updating the model from data; usually a background job separate from user traffic	Nightly job learning from yesterday's orders
Inference	Running the trained model on an input	Scoring one transaction at checkout
Online inference	Inference inside a live request; must meet its latency limit	Fraud score before a payment is approved
Batch inference	Inference over many inputs in a scheduled job; results stored	Tonight's recommendation lists for every active user

A model is like a new hire who learned from past examples: you still need an office (infrastructure), checks on their work (evaluation), a way to notice decline (monitoring), a budget (cost) and a plan for mistakes (fallbacks). Model internals help, but the round mostly grades the **whole system** — candidates who spent weeks on attention and loss derivations often freeze at "where does the training data come from?" or "what if the model times out?".

The five forms of the question

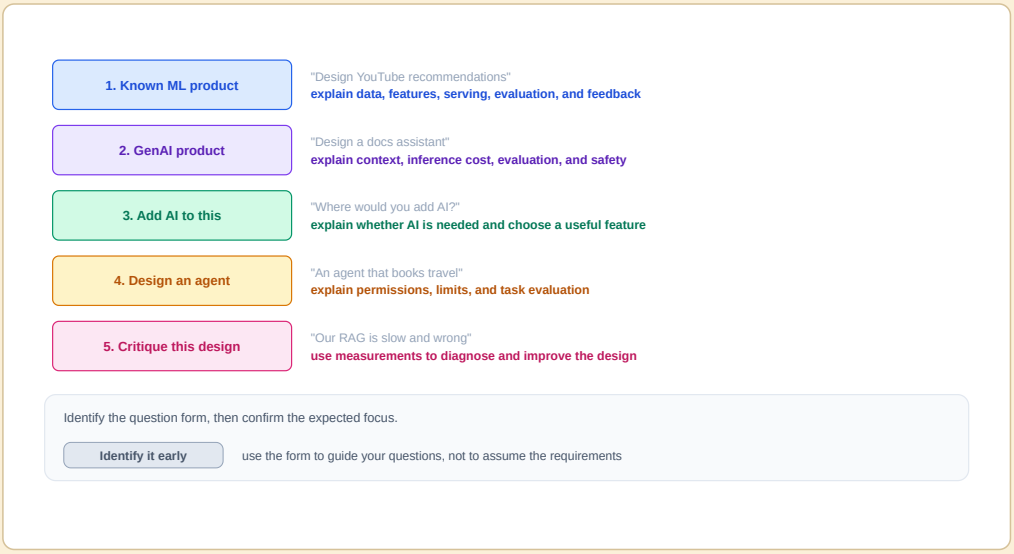


Figure — Five shapes of AI design prompts (course diagram)

#	Form	Example prompts	What to emphasise
1	**Known ML product**	"Design YouTube recommendations", "Design search autocomplete"	Data, features, training, serving, evaluation, feedback loop
2	**GenAI product**	"Design ChatGPT", "Design a docs Q&A assistant"	Retrieval, context assembly (building the text sent to the model), inference cost, output evaluation, safety
3	**Add AI to an existing system**	"Our store runs fine — which part should get AI?"	Whether AI is needed at all, one useful feature, simplest viable design
4	**Design an agent**	"An agent that books travel"	Tool permissions, approvals, step/cost limits, stop conditions, task-success evaluation
5	**Critique / scale an AI design**	"Our RAG system is slow and wrong — fix it"	Diagnose with measurements, logs and failed examples *before* changing anything

Terms: a **feature** is one input to the model (watch history, hour); the **feature pipeline** computes and delivers them; the **feedback loop** collects outcomes to evaluate and improve. **GenAI** = generative AI. **LLM** = large language model. **RAG** = retrieval-augmented generation (put relevant fetched documents in the model's input — grounds answers, doesn't guarantee correctness). **Fine-tuning** = extra training of an existing model for a narrower task.

Who gets asked what

Depth varies by role and company — **ask**.

Role	Forms to prepare	Depth expected
Software engineer (senior+)	2, 3, sometimes 1	Architecture, serving, cost, failure handling; model inputs/outputs/limits
ML engineer	1, 2, 5	Above + model choice, features, training pipelines, evaluation metrics
AI / ML platform engineer	2, 4, 5	Inference infrastructure: GPUs, batching, caching, latency under load
Engineering manager / staff+	3, 5	Build vs buy, cost limits, ownership, risk, gradual rollout

First-minute question: *"Would you like me to go deep on model development, or on the system that serves and maintains the model?"*

What interviewers grade (and what they don't)

Graded	Usually secondary
Requirements discipline — users, latency, quality measure, out of scope, numeric assumptions	Deriving the transformer or backprop by hand
A complete loop, not just the request path — outcomes come back and improve the system	Naming this month's newest model
Cost and latency realism — assume, estimate, optimise	Framework trivia without saying why a component is needed
Failure honesty — a model can be confidently wrong with HTTP 200; detection, **blast-radius** limits (how much one failure can damage), fallbacks	

How it works — four assumptions that break when a model enters

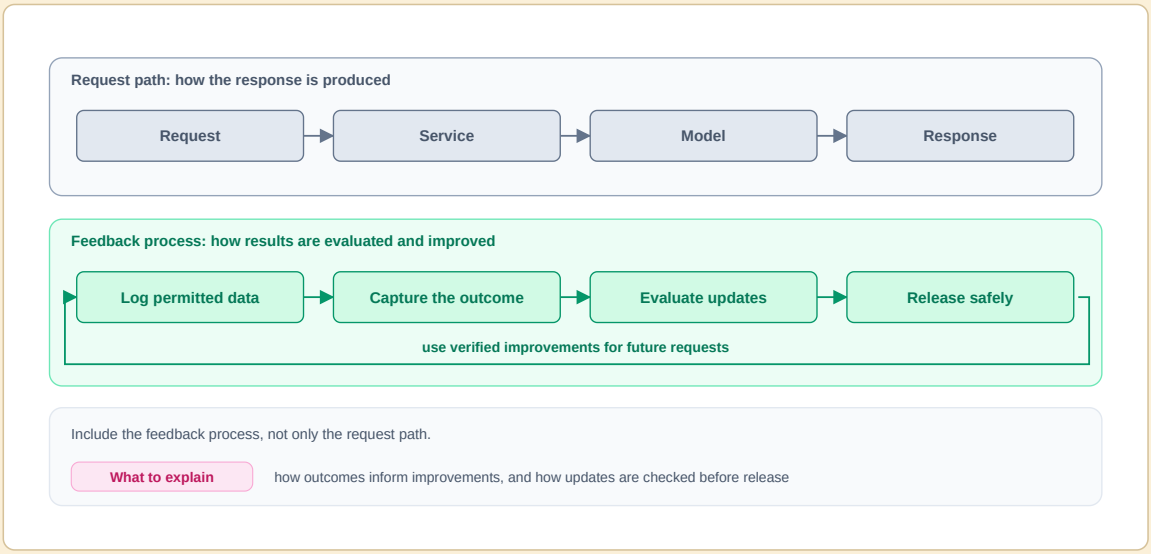


Figure — Serving path on top, improvement loop underneath (course diagram)

Contrasts, not absolutes — but where incomplete AI designs fall down.

#	Classic assumption	What changes with a model	Design response
1	**Correctness is binary** (a payment is right or a bug)	Quality is a spectrum: a recommendation can render fine and be irrelevant	**Quality metrics** (CTR — clicks ÷ impressions, resolution rate, suggestion-acceptance) *alongside* latency and availability **SLOs** (service-level objectives); log predictions and join them with later outcomes
2	**The code is the behaviour**	Behaviour depends on data. **Drift** — change in input patterns or the input→outcome relationship — degrades an unchanged model (new fraud tactics, changed tastes)	Feedback process: collect outcomes, investigate quality changes, evaluate candidate vs current, release gradually with rollback. Retrain on evidence, not on every alert
3	**Compute is cheap**	A model call can cost orders of magnitude more than a DB read, depending on model size, input/output length, hardware and utilisation	Put a number on it: 10M requests/day × $0.01 = $100,000/day ≈ $3M/month (illustrative). Then caching (when permissions/freshness allow), routing easy requests to a smaller model (if it meets the quality bar), batch precompute
4	**Failures are loud** (500s, timeouts)	A wrong answer arrives as `200 OK`: invented citation, missed fraud, harmful text	Treat user text and retrieved text as possibly hostile (**prompt injection** — untrusted text overriding instructions — can't be fully filtered; enforce permissions outside the model), check outputs, human approval for high-impact actions, fallbacks (unpersonalised results, hand-off to a human)

For every design ask: **"Who is hurt by a wrong output, and what limits the damage?"**

Build, host, or call an API?

Option	Pros	Cons / what you still own
Hosted API (pay per use)	Fastest launch, no GPU fleet	Data-handling terms, rate limits, provider availability, per-call cost at scale; you still build integration, evaluation, monitoring. Poor fit under strict **data-residency**/sensitivity rules, very high steady volume, or latency the provider can't meet
Self-host open-weights model (weights downloadable under a licence)	Control over data, deployment, latency; can be cheaper at high, steady utilisation	Capacity, security, upgrades, on-call, often GPUs; savings not guaranteed
Train / fine-tune your own	Needed when your interaction data is the product (ranking, fraud, recommendations); fine-tuning can help a narrow generative task	Data preparation, training infra, ongoing maintenance

Options overlap (fine-tune, then managed hosting). Compare on quality, data sensitivity, volume, latency, total cost and team skills; start simplest and say what evidence would make you switch.

Trade-offs

Going ML at all is a trade-off. Google's *Rules of ML*: **Rule #1: don't be afraid to launch without ML** — a **heuristic** (hand-written rule of thumb) may suffice to start, gives a baseline, and produces the data you'll learn from. **Rule #3: choose ML over a complex heuristic** — once rules become a tangle, a model is easier to maintain.

Common mistakes

- Answering an SWE design question like a research talk (15 minutes on fine-tuning, nothing on serving or evaluation).
- Drawing only the request path — no logging, outcome join, retraining or evaluation loop.
- Assuming model calls are free or cache-fast.
- Treating thumbs up/down as trustworthy training labels.
- Claiming a fixed traffic share can go to a small model without evidence.

Real-world examples

Netflix, YouTube, TikTok feeds (form 1); **GitHub Copilot**-style assistants measured by suggestion acceptance and **retention** (form 2); **Stripe Radar**, a fraud model inside an existing payments flow (form 3 in spirit); support agents issuing refunds through limited tools (form 4).

In the interview

> **30-second answer** — *"What changes compared with an ordinary backend service?" → "Distributed-systems fundamentals still hold. I add quality metrics beside latency and uptime SLOs; a loop that notices when quality slips and tests any new version before a staged rollout; a budget for per-call latency and spend; and safeguards for bad outputs — validation, fallbacks, human review where stakes warrant."*

- **"What do you mean by quality?"** → Feed: CTR plus a longer-term signal (watch time, return visits). Fraud: **precision** (share of flagged that is fraud) and **recall** (share of fraud flagged) at the chosen threshold. Support: correct resolution rate plus **CSAT** (customer-satisfaction score) and repeat-contact rate.
- **"Would you fine-tune?"** → Not first. Check whether an existing model with good instructions (plus retrieval for private data) meets quality and cost on an evaluation set; fine-tune only for a measured gap simpler changes don't close.
- **"Assume the model exists."** → Spend the time on integration, serving, cost, evaluation and feedback.

TL;DR

Classic design still applies; a model adds **quality measurement, drift, cost, and silent failure**. Five forms; confirm depth. Graded: requirements, full loop, cost/latency realism, failure honesty. Build vs host vs API is requirements-driven — start simplest.

15.2 The three question families

What it is

Classify what the system mainly **produces**:

Family	The model produces…	Typical questions	Focus of your answer	Classic failure
1. Predictive	A score, number or label the system acts on	Recommendations, feed/search ranking, ads CTR, fraud/spam, ETA, churn	Data & features, candidate generation + ranking (cheap shortlist, then careful ordering), offline & online metrics, feedback loop	Picking a model ("an LLM ranks the feed") without data, latency or cost reasoning
2. Generative	Content: answer, summary, code, image	ChatGPT-style assistant, docs bot, code assistant, meeting summaries	Retrieval & context assembly, inference latency/cost (streaming, caching, routing), output evaluation, safety	Training a foundation model from scratch; ignoring cost; no quality measurement
3. Agentic	Which **action** to take next (call a tool, read result, decide again)	Refund-issuing support agent, travel booking, coding agent opening PRs	Tool interface & permissions, step/time/cost limits, stop conditions, approval for risky actions, task-level evaluation	Unlimited tools and no stop rule — or human approval for every harmless step

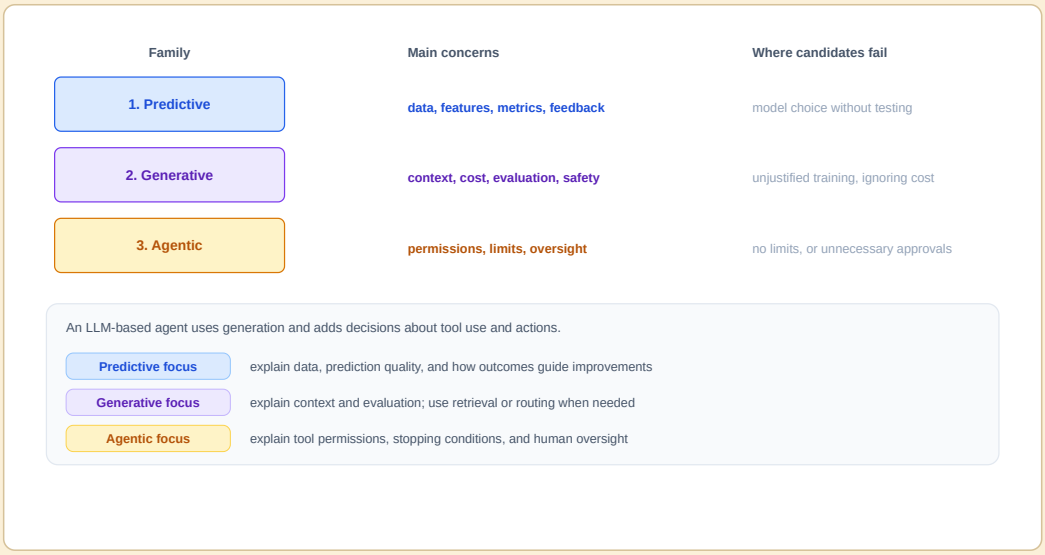


Figure — Predictive, generative and agentic compared (course diagram)

Terms: **label** = the target to predict. **Pretrained** = trained before your task. **Foundation model** = large model trained on broad data, adaptable to many tasks. **Token** = the text chunk an LLM reads/writes (word, fragment, punctuation). **Hallucination** = false or unsupported generated content. **Streaming** = showing output as it's generated. **Model routing** = picking which model handles a request by difficulty/cost.

"Design an AI shopping experience" and "Design fraud detection" need different designs, metrics and risks — classifying early avoids 30 minutes on the wrong system.

How it works — the output test

1. Does the system mainly output a **prediction, content, or an action**?
2. If vague, state your read and confirm.
3. Mixed products → split into subsystems (a streaming service ranks the homepage — predictive — and writes "why you might like this" blurbs — generative; a support platform routes tickets, drafts replies, issues refunds via an authorised tool — all three).
4. Name each subsystem and its interfaces, and ask which to go deep on — usually the riskiest.

This book focuses mostly on **family 1**; the AI Handbook covers LLM serving, RAG and agents. The framework works for all three.

Common mistakes

- An LLM for everything, e.g. one call per item to rank a 50M-item catalog (ignores personalisation data, evaluation, latency, cost). Sensible LLM roles are narrower — offline item enrichment, reordering a short list of finalists (~top 50) — and even then benchmark against a learned ranker on quality, speed and spend.
- Multi-agent designs when a fixed workflow would do (known steps → hard-code them).

In the interview

30-second answer to an unclear prompt: *"First, what does the system output? A ranking or score puts my time on data, features and evaluation. Generated text shifts it to context, per-call spend and answer quality. Actions put tools, permissions and limits at the centre. My read is this is mostly ranking items per user — shall I go that way?"*

- **"It's all three."** → List subsystems and interfaces, go deep on the riskiest ("wrong refunds cost money"), let the interviewer redirect.
- *"Summarise tickets and auto-close duplicates" — family?* → Generative plus a small agentic part (closing) needing permission and review rules.

TL;DR

Predictions → data/features/metrics. Content → context/cost/quality/safety. Actions → permissions/limits/oversight. Mixed: split, connect, pick a focus.

15.3 The seven-step framework

What it is

A fixed order of decisions so requirements and data drive the architecture. It's the classic structure (requirements → estimates → API → high-level design → deep dive → wrap-up) with two ML concerns made explicit: **define quality early** and **show how the system keeps working after launch**.

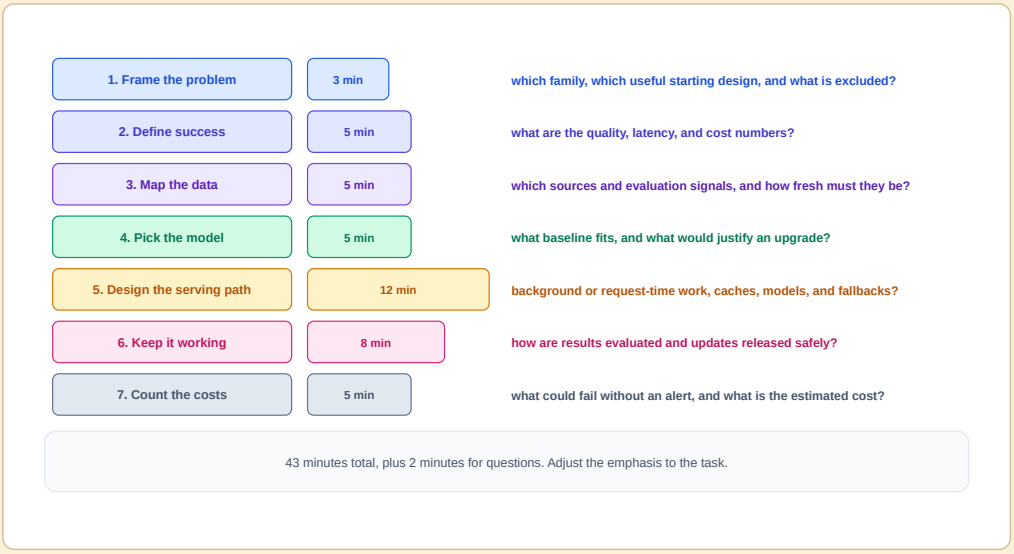
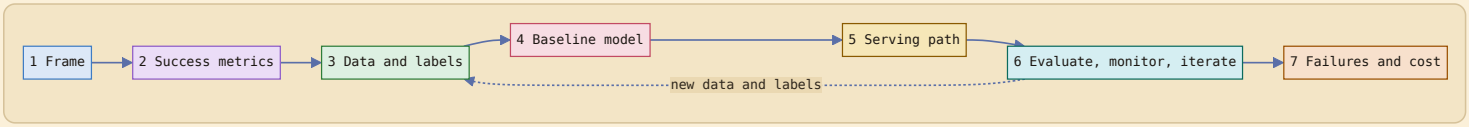


Figure — Seven steps with their guiding questions and minutes (course diagram)

Step	Question it answers	Minutes (45-min round)
1. Frame the problem	What family, what known system does it resemble, what's in/out of scope?	3
2. Define success	Quality, latency, scale and cost targets?	5
3. Map the data	Sources, label origin, required freshness?	5
4. Pick the model	Simplest baseline, and what would justify upgrading?	5
5. Design the serving path	Offline vs per request, caches, cheaper paths, failure handling?	12
6. Keep it working	How are outcomes logged, quality measured, changes evaluated and rolled out?	8
7. Count failures and cost	What breaks, what controls exist, cost vs budget?	5
Buffer	Questions	2

The same structure is in Aminian & Xu's *ML System Design Interview* (clarify requirements → frame as ML task → data preparation → model development → evaluation → deployment/serving → monitoring) and Chip Huyen's *Designing ML Systems* lifecycle (scoping → data engineering → model development → deployment → monitoring and continual learning → business analysis). Any wording; cover every step. Shift time toward data/models for predictive, inference/context for generative, permissions for agents.



Step 1 — Frame the problem (and frame it as an ML task)

1. **Name the family** (output test).
2. **Name the known system** it resembles ("a recommender where items are podcasts") and say which requirements differ — don't paste a memorised design.
3. **State scope**: "English text support first; voice and other languages later. OK?"
4. For predictive problems, **frame as an ML task** (Aminian & Xu's step 2):
   - **Input and output** (input = user, video, context; output = P(user watches ≥ 30 s)).
   - **ML category**: binary/multiclass classification, regression, ranking (learning-to-rank), retrieval, clustering, or generation.
   - **ML objective**: what the model optimises, chosen because it moves the business metric.

Business metric → ML objective (worked examples)

Business goal / metric	ML task	ML objective (what the model predicts)	Why not the obvious one
Increase time spent on video platform	Ranking	P(watch ≥ X seconds) or expected watch time	Raw clicks reward clickbait (watch-time objectives risk short-clip bias — guard with surveys, retention)
Reduce fraud losses without annoying customers	Binary classification	P(fraud) → thresholds set by expected cost	Accuracy is meaningless at 0.2% fraud
Accurate delivery promise at checkout	Regression	Minutes from order to delivery	Business also needs a *promise-kept rate*, handled by a display buffer, not the model
Improve marketplace search revenue	Ranking	P(purchase | query, item), possibly × price	Relevance-only ranking may ignore **conversion** (view → purchase)
Reduce subscription churn	Binary classification	P(cancel within 30 days)	Label arrives 30 days late — shapes monitoring

Rules of ML: **Rule #12 — don't overthink which objective you directly optimise** at first; **Rule #13 — pick a simple, observable and attributable metric** for the first objective (a click is attributable to the item shown; "user happiness" is not).

Step 2 — Define success

Numbers; state assumptions when not given.

1. **Quality** — a *business/online metric* plus a correlated *offline ML metric*:
   - "Users find videos they enjoy" → online: watch time, CTR, with **guardrails** (metrics that must not worsen: satisfaction surveys, policy-violating content); offline: NDCG / recall@k (Chapter 13) on **held-out** interactions.
   - "Helpful support bot" → correct resolution rate, CSAT, repeat-contact rate.
2. **Latency** — a feed in a few hundred ms; a chatbot has a **time-to-first-token** target *and* a full-answer target; a batch job has a completion deadline and freshness target. Latency decides what must be precomputed.
3. **Scale and cost** — users, requests/day, **peak QPS** (System Design Fundamentals), cost per request or per *completed task*. A free feature needs tiny per-request cost; a support bot can cost more if it really resolves tickets. A concrete budget, never "near zero".

Rules of ML **#2: design and implement metrics first** — log what you'll need before building.

Step 3 — Map the data

Sources: interaction logs, catalog/content, user profiles (where permitted), documents for generation. Then:

1. **Label source** — often late or imperfect:
   - Fraud: confirmed investigations and **chargebacks** (disputed card payments reversed) — weeks late, and not every chargeback is fraud.
   - Feeds: clicks — instant but only a **proxy** for satisfaction.
   - Support: reviewed conversations, verified resolutions; a thumbs-down is feedback, not a correctness label.
   - **Self-supervised learning** (labels made from the data itself, e.g. next word) and pretrained models reduce hand-labelling, but you still need labelled *evaluation* data.
2. **Freshness** — daily for long-term tastes; minutes or seconds for trending content and fraud. Decides batch vs **streaming** (continuous event processing) vs request-time computation.

Name **feature groups** (entity, recent-behaviour aggregates over time windows, request context, cross features such as user × category — §16.4) and flag **leakage** (features unavailable at prediction time) and **privacy** limits.

Step 4 — Choose a model, beginning with a baseline

A **baseline** is the least elaborate approach meeting basic requirements: it tests the pipeline end to end and is the yardstick for everything later (*Rules of ML #4: keep the first model simple and get the infrastructure right*) — and sometimes it's good enough.

Family	Baseline	Upgrade path	Trigger to upgrade
Predictive	Popularity / rules / per-entity averages; logistic regression	GBDT on tabular features → deep models (two-tower retrieval, multitask rankers — Chapter 13)	Baseline misses the offline target and the gain is worth the complexity
Generative	Existing model + clear instructions; add retrieval for private/fresh data	Better retrieval, model routing, fine-tuning	Evaluation shows a failure prompts/retrieval don't fix
Agentic	Fixed workflow if steps are known; else one agent with a few narrow tools	More tools, then multiple agents	Separation clearly helps and justifies coordination cost

Always state the condition: *"If the baseline misses the target, I'll compare a GBDT on the same held-out weeks."*

Code — baseline first (`code/baseline_first.py`)

Synthetic delivery-time task: naive baseline → heuristic → GBDT, then model error → the business metric "orders delivered within the promised time" via a display buffer.

```python
# 1) naive baseline: everyone gets the global average
rows.append(report("global mean", yte, np.full(len(yte), ytr.mean())))

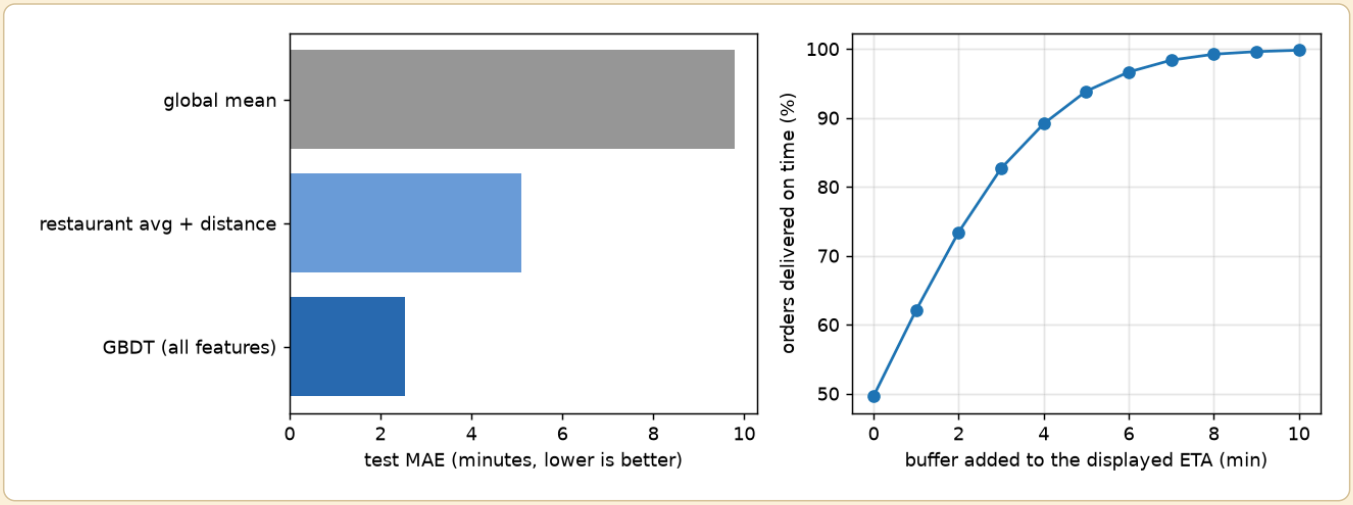
# 2) heuristic: per-restaurant average + a distance term (shippable in a week)
rest_avg = {r: ytr[Xtr[:, 0] == r].mean() for r in np.unique(Xtr[:, 0])}
resid = ytr - np.array([rest_avg[r] for r in Xtr[:, 0]])
dist_fit = LinearRegression().fit(Xtr[:, [1]], resid)
heur = np.array([rest_avg.get(r, ytr.mean()) for r in Xte[:, 0]]) + dist_fit.predict(Xte[:, [1]])

# 3) learned model: gradient-boosted trees on all features (restaurant id as categorical)
gbdt = HistGradientBoostingRegressor(random_state=0, categorical_features=[0]).fit(Xtr, ytr)
pred = gbdt.predict(Xte)

# business metric: % of orders that arrive within (prediction + buffer)
kept = [(yte <= pred + b).mean() * 100 for b in buffers]
```

```
model                          MAE  p90 abs err
global mean                   9.80        19.92
restaurant avg + distance     5.11        10.45
GBDT (all features)           2.54         5.31

buffer (min) -> promise kept % / avg promised minutes   [GBDT]
  + 0 min ->  49.6%  /  37.8
  + 3 min ->  82.7%  /  40.8
  + 5 min ->  93.9%  /  42.8
  + 8 min ->  99.2%  /  45.8
```



- (MAE in minutes; **p90 abs err** = error size 90% of predictions stay within.) Every number is judged **against a baseline**: heuristic halves naive error, GBDT halves it again (synthetic data — real gains are usually smaller).
- An unbiased regressor's errors are roughly symmetric, so it's late **half the time** (49.6% kept). The business metric depends on a **product decision** (the buffer): +5 min keeps ~94% but makes the ETA look slower. Keep that knob outside the model so product owns it.

Step 5 — Design the serving path

Draw the architecture and trace one request. Whatever the shape, answer four questions:

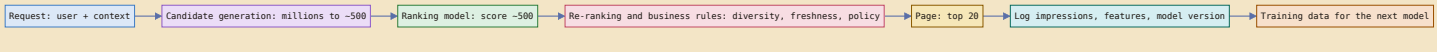


Figure — Step 5 checklist: offline/online, caches, model choice, failure (course diagram)

1. **Offline or online?** What can be precomputed vs needs request-time data (§16.5)?
2. **Where are the caches?** Latency/cost saved, freshness, **invalidation**, and **permission** rules so no user sees another's cached result.
3. **Which requests can take a cheaper path** (smaller model, cached answer, rule)?

4. **What if the model is slow, down or rate-limited?** Timeouts, bounded retries (only when safe), a fallback (previous model, cached score, heuristic, unpersonalised list) or a clear failure; for batch, deadlines and recovery.

A typical predictive design for a large catalog is a **funnel**:



Step 6 — Keep it working (evaluation, monitoring, iteration)

Show the feedback process **in the diagram**:

1. **Log** inputs, outputs, features used and model version (privacy controls, retention limits).
2. **Capture outcomes**: clicks, ratings, escalations, confirmed fraud, actual delivery time.
3. **Measure** step-2 metrics on live outcomes and human-reviewed samples.
4. **Improve** data, features, retrieval, prompts or model.
5. **Release safely**: offline evaluation → **shadow deployment** (new version scores copied traffic, outputs hidden, side effects disabled) and/or **A/B test** → gradual ramp with explicit stop/**rollback** conditions.

	Offline evaluation	**Online evaluation**
Data	Held-out historical data (time-based split)	Live users
Metrics	ML metrics: PR-AUC, NDCG, recall@k, MAE, **calibration** (predicted probabilities match real frequencies)	Business metrics: CTR, watch time, revenue, resolution rate + guardrails
Cost / risk	Cheap, no user risk	Real traffic; needs time and **statistical power**
Blind spots	Proxy labels, feedback loops, serving-time degradation	Slow, noisy, needs good experiment design
Role	Filter candidates	Decide the launch

(Details §16.7, §16.9.) Monitoring in one line: system health → input feature distributions → prediction distributions → outcome quality (as labels arrive), sliced per segment. Retrain on a schedule justified by measured decay or after an investigation — never auto-promote because an alert fired.

Step 7 — Failure modes and the bill

Failure list specific to your system — unsupported answers, drifting data, unsafe actions, feedback loops reinforcing bad behaviour (clicks-only → clickbait) — each with a control (thresholds, quality monitors, content checks, permission limits, a **kill switch** to turn the feature off) and what the control *cannot* guarantee.

Cost ≈ requests × share hitting each model × calls per request × cost per call, plus retrieval, storage, retries and human escalation.

Code — napkin math (`code/napkin_math.py`)

```python
def qps(per_day, peak_factor=3.0):
    avg = per_day / 86_400
    return avg, avg * peak_factor

def model_cost_per_day(requests_per_day, share, calls_per_request, cost_per_call):
    return requests_per_day * share * calls_per_request * cost_per_call
```

```
10M req/day -> avg 116 QPS, peak ~347 QPS (3x peak factor assumed)
all requests x $0.01        -> $100,000/day, $3,000,000/30-day month
20% to large model x $0.01  -> $20,000/day (excludes small model, retrieval, retries)
support: 40,000 conversations/day x $0.10 = $4,000/day model cost
  compare per resolved issue, not per call: human ticket assumed $4.00
  resolve 30%: expected cost/ticket $2.90 vs $4.00 human-only
  resolve 50%: expected cost/ticket $2.10 vs $4.00 human-only
  resolve 70%: expected cost/ticket $1.30 vs $4.00 human-only
precompute 25M users x 500 recs = 12.5B rows; active-only (5M) = 2.5B rows
```

The habit is the **unit**: cost *per resolved issue* (AI cost + human cost for the unresolved share), not one model call vs one human ticket. All prices are illustrative assumptions.

> Short on time? Steps 6–7 in two sentences: *"Every prediction is matched to its eventual outcome; when quality moves I dig in, and any new version passes offline tests, then shadow or **canary** (a small slice of real traffic), before a staged ramp — with cost per request on the dashboard. The two risks I'd watch hardest are X and Y."*

Trade-offs of the framework

Benefit	**Cost**
Decisions follow requirements and data	Feels slow if the interviewer wants the diagram — state assumptions in one sentence each and jump to step 5
You don't forget evaluation/monitoring/cost	Following it rigidly when steered elsewhere looks robotic — adapt order and depth

When not to use it rigidly

- "Skip to the architecture": state quality and latency as assumptions, draw, revisit requirements when they change a choice.

- "You have 10 minutes": scope, main architecture, biggest risk, cost — one sentence each on quality target, data, evaluation.

In the interview

30-second opener: *"Plan: agree the task and boundaries; put numbers on quality, latency and spend; find the data and label source. Then a baseline and the serving path. I'll close with evaluation, safe rollout, failures, and cost. To start — this reads like a ranking problem…"*

- **"Why a baseline if GBDT will win?"** → Validates pipeline and logging, gives a number to beat, sometimes good enough (Rules #1, #4).
- **"What's your ML objective?"** → Name the target, how it moves the business metric, the proxy risk and the guardrail that catches it.
- **"How will you know it works after launch?"** → Four monitoring layers, outcome join, per-segment quality, drift alerts → investigation, gated retraining.

TL;DR

Frame (3) → Success (5) → Data (5) → Baseline (5) → Serving (12) → Keep it working (8) → Failures & cost (5). Observable ML objective, beat a baseline, show the loop, price it.

15.4 Capstone — scoping a vague AI question

The exercise

Hypothetical prompt (numbers are assumptions):

> "We're a food-delivery company: ~25 million monthly users, 500,000 orders a day. Leadership wants AI to improve customer experience. Design something."

Feature, metric and family are unspecified. The first ten minutes turn this into an agreed, measurable problem — not an architecture.

Step A — Generate options by family, then choose

Family	Options
Predictive	ETA prediction; restaurant/dish recommendations; fraud on refund claims; predictions feeding courier assignment (assignment itself also needs optimisation/rules)
Generative	Support assistant for order status and refund policy; AI-written dish descriptions; review summaries
Agentic	"Dinner for four under $60" assistant that places an order after checks and confirmation

Choose on **impact × feasibility × risk** (feasibility = can the team build and run it with available data, time, people):

- **ETA** — touches every order; delivered orders give labels (check completeness).
- **Support assistant** — savings show directly in support spend; order records and policy documents likely exist; risk limited by restricted actions and escalation.
- **Ordering agent** — spends users' money; needs payment authorisation, confirmation, cancellation. Feasible, more controls.

No universally right pick — state your preference under these assumptions and what would change your mind.

Step B — Framework opening (support assistant chosen)

1. **Quality**: primary = *correct resolution without escalation*; plus CSAT and repeat contacts (closed ≠ fixed). Human ticket assumed $4. Target: resolve ≥ 50% of eligible **tier-1** tickets (order status, coupons, simple cancellations) with CSAT no worse than humans. Explaining the cancellation policy differs from *cancelling* — actions need their own authorisation and rule checks.
2. **Latency**: stream; p95 (95% of requests meet it) time-to-first-token ≤ 1 s, p95 completion ≤ 5 s for short answers; tool calls may get separate targets.
3. **Scale and cost**: 500,000 orders × 8% contact rate = **40,000 conversations/day** (average; peaks and repeats separately). 10 calls × $0.01 → $0.10/conversation → **$4,000/day**. Compare *total cost per correctly resolved issue* (retrieval, infrastructure, retries, escalations included).
4. **Out of scope**: voice, non-English. Refunds above an assumed $10 go to a human; smaller ones still need identity, eligibility and duplicate-payment checks enforced **outside** the model.

Step C — Classify and preview the shape

"At its core this generates answers grounded in order data the user may see and retrieved policy text. A refund tool adds an agentic slice, so reading data and moving money get separate safeguards. Deep dives: retrieval accuracy, answer grading, cost per conversation, and backend checks guarding refunds ≤ $10."

Sample opening (paraphrased dialogue)

- **Interviewer:** "Use AI to improve customer experience."
- **You:** "Three candidates: ETA prediction, a support assistant, or an ordering agent — the agent needs payment and confirmation controls. I'd start with support: savings are measurable against current support costs and we likely have order data and policies. Support, or ETA?"
- **Interviewer:** "Support works."
- **You:** "Target: correctly resolve at least half of eligible routine tickets, CSAT at least human-level, no rise in repeat contacts. 500k orders × 8% ≈ 40k conversations a day. p95 first output within 1 s, completion within 5 s. Ten one-cent calls is ten cents a conversation before retrieval and infrastructure, so we track total spend per issue actually resolved. V1 skips voice and non-English. Refunds above $10 go to a person; smaller ones still pass authorisation and policy checks. So: answer generation over orders and policies plus a refund action; deep dive on retrieval, grading, spend and guarding the action — agreed?"

Now you (10 minutes each, out loud)

Cover options by family, one choice with reasons, quality/latency/scale/cost targets with assumptions flagged, scope, family and key decisions.

1. "Our music-streaming app wants listeners to discover more. Where does AI help?"
2. "Bank leadership wants something AI-powered in the mobile app." (Weigh the damage of a wrong output: personal money advice is far riskier than explaining a purchase's category; forecasts and fraud warnings still need watching. No family is automatically safe — say what it may do, must not do, and when a human reviews.)
3. "Candidates and jobs on our job board are poorly matched. Fix it with AI."

TL;DR

Vague prompt → options by family → choose on impact, data, feasibility, risk → quality/latency/scale/cost numbers in comparable units → confirm scope, family, key decisions.

15.5 Common mistakes (the whole interview)

Mistake	Better
"I'd use a transformer" in minute 2	Frame, success and data first; baseline before deep models
Optimising a proxy with no guardrail (clicks → clickbait)	Pair the objective with guardrails (satisfaction, retention, policy violations)
"We'll have labels"	Where from, how late, how noisy, who pays
Random train/test split on time-dependent data	Time-based split; **point-in-time-correct** features (only data available at that moment) (Rules of ML #33)
Accuracy on imbalanced data	Precision/recall at a threshold, PR-AUC; threshold owned by product/risk
No fallback	Timeouts + fallback prediction agreed with product and risk owners
Only the request path	Logging → outcome join → evaluation → gated retraining → staged rollout
Training features computed differently from serving	One feature definition; log serving features and train on them (Rules of ML #29, #32)
"GPU cluster" for a small tabular model	Benchmark on CPU first
Auto-retraining on every drift alert	Alert → investigate (often a broken pipeline) → validated retrain → gates
Hand-waving cost	Requests × share × calls × price, per completed task
Ignoring the interviewer's redirect	The framework is a checklist, not a script

Chapter TL;DR

- ML design = classic system design **plus** quality metrics, a data/label story, baseline-first modelling, a feedback loop, cost realism.
- Identify the **form** and the **family** (prediction, content, action) in the first minute; confirm depth.
- Seven steps in ~43 minutes: frame (3), success (5), data (5), model (5), serving (12), keep-it-working (8), failures and cost (5).
- Business metric → observable ML objective; product knobs (buffers, thresholds) outside the model.
- Numbers mean something only next to a baseline; every design needs a fallback; cost goes in the same unit as benefit.
- Next: Chapter 16 builds each production piece and assembles an ETA platform.

Chapter 16 — ML in Production: Pipelines, Serving, Evaluation, Monitoring and MLOps

Fitting a model in a **notebook** (Jupyter) is a small slice of the job. The rest is machinery: trustworthy labels, identical features for training and live traffic, deciding when predictions are computed, fast serving with a fallback, the metric that proves one model beats another, alarms for quiet decay, and safe releases. This chapter builds each piece for **predictive** models (scores, labels, numbers), then assembles an ETA (delivery-time) platform.

What you'll learn: the five pieces and the offline/online divide; training vs serving and skew (leakage demo); labels (natural, behavioural, human, LLM-assisted), imbalance and recalibration, negatives, splits, privacy; features and feature stores; batch/online/streaming inference; serving (budgets, batching, caching, rollout, hardware, compression, fallbacks); production offline metrics; monitoring and drift (PSI, KS), retraining, feedback loops; A/B tests (power, peeking, guardrails, interleaving, switchbacks, bandits); MLOps (tracking, registry, validation, **CI/CD/CT**, shadow, canary); the ETA capstone.

Embeddings and two-tower retrieval are in Chapter 13. Code is in `code/`; outputs are real.

16.1 The five pieces and the offline/online divide

What it is

Recommenders, fraud detectors and ETA predictors all decompose into:

Piece	Job
1. **Training pipeline**	Builds a dataset, fits and scores a model, on a schedule or trigger
2. **Feature layer**	Code plus storage turning raw events into model inputs
3. **Inference path**	Everything between a request and its score
4. **Serving platform**	Machines, routing and autoscaling behind live traffic
5. **Monitoring loop**	Signals that reveal and help diagnose degradation

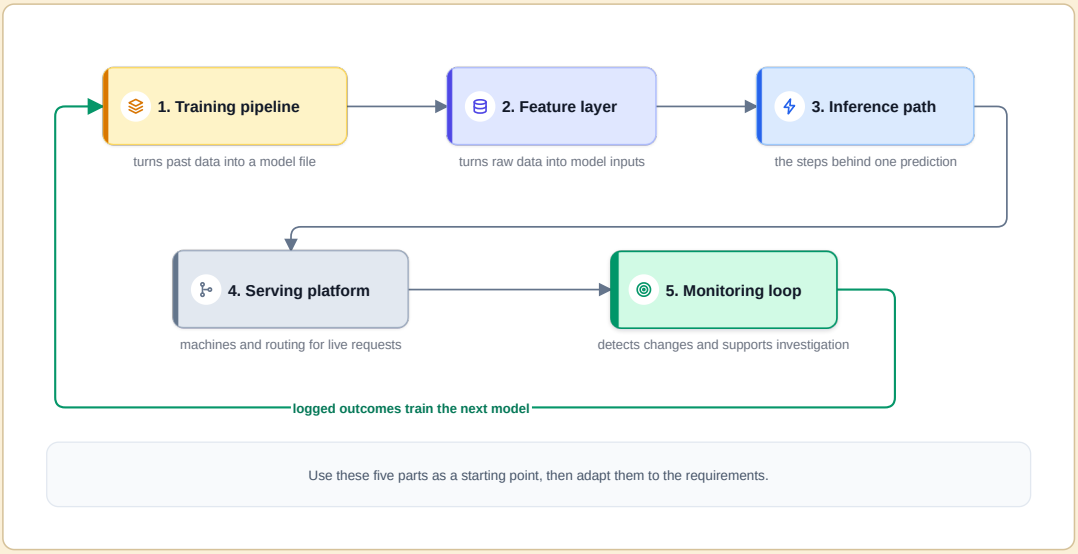


Figure — Five building blocks of a predictive ML system (course diagram)

Vocabulary: a **feature** is a model input derived from raw data (order events are raw; "orders this restaurant took in the past half hour" is a feature). A **label** is the outcome predicted — delivery minutes, a click — possibly imperfect (a chargeback isn't always fraud), and it doesn't exist at prediction time. A **training example** = features + label.

How it works — one prediction, start to finish

ETA at checkout: (1) checkout calls the prediction service; (2) it gathers features from storage and the request, (3) arranged exactly as the model expects; (4) the model returns, say, 34 minutes; (5) business rules add a buffer and it's shown; (6) features, prediction, model version and order id are logged. (7) ~40 minutes later the food arrives — the **label** is known; (8) overnight, training joins logged features with actual times; (9) a fresh candidate replaces the current model only if it wins. Steps 1–6 live inside a **latency budget**; 7–9 run hours or days later — that gap is the thread of this chapter.

The offline/online divide

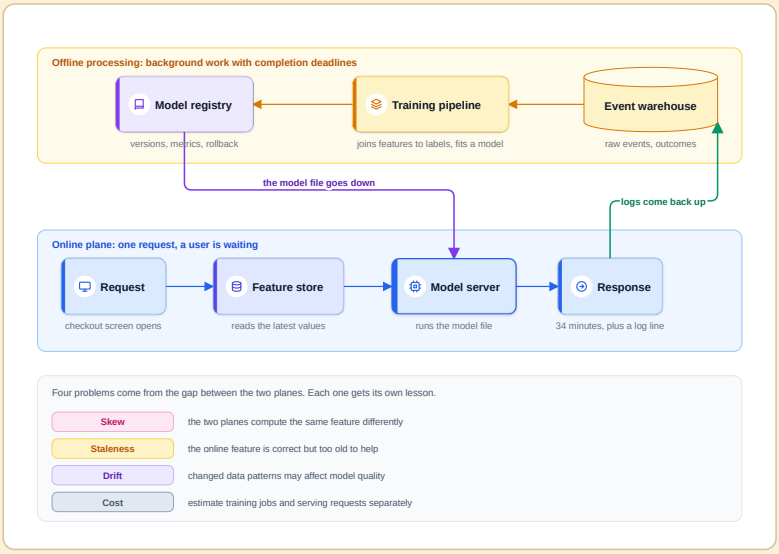


Figure — Offline and online planes: model goes down, logs come up (course diagram)

- **Offline plane**: background jobs over history (training, batch inference) — **throughput** and deadlines (System Design Fundamentals).
- **Online plane**: live requests under a response-time limit.
- The **model artifact** (saved trained model) crosses offline → online; **prediction logs + outcomes** cross back. **Schemas**, feature definitions and config are shared.

Problem	Meaning	Section
Skew	A feature computed differently in training and serving	16.2
Staleness	Correct but old (hourly average when the kitchen got busy 10 min ago)	16.4
Drift	Data or input→outcome relationships change; quality falls while the service is "up"	16.8
Cost	Training cost (job size × frequency) and serving cost (traffic × hardware × utilisation) — separate, both estimated	16.5, 16.6

TL;DR

Five pieces; two planes; model goes down, logs come back up; skew, staleness, drift and cost live at the boundary.

16.2 Training vs serving

What it is

Two systems sharing artifacts: **training** consumes history and emits a model; **serving** loads it and answers live traffic. Draw them as separate groups from the start.

	Training	Serving
Runs	Scheduled (nightly/weekly) or triggered	Continuously, per request
Optimises for	Throughput over large datasets	Latency and availability
Hardware	Big GPU or high-memory clusters; cheap **spot machines** (reclaimable spare cloud capacity) work if the job checkpoints and resumes	Always-on CPUs or small GPUs, sized for the **SLO**
When it breaks	New model is late; previous one stays live (urgency depends on staleness)	Users see errors, on-call is paged — a safe fallback is required
Data	Months/years of history	This request's inputs, now

How it works — the training pipeline

A scheduled workflow (Airflow, Kubeflow Pipelines, Dagster, cloud equivalents), four stages:

1. **Data preparation** — read raw events from the **warehouse** (cleaned tables) or **lake** (raw files); pair feature values **as of the prediction moment** with the later label. Test hard: one wrong **join** quietly poisons everything downstream.
2. **Training** — for tabular data a **GBDT** is the strong default; a modest neural net the other common choice. Too big for one machine → **data parallelism** (examples split across workers, each with the full model) or **model parallelism** (the model split across devices).
3. **Evaluation** — score on **held-out** data; compare with production on quality, latency, cost and safety. Equally good but cheaper is a valid release.
4. **Registration** — store in a **model registry** with version, metrics, stage and **lineage** (data and code that produced it). **Rollback** needs compatible features, dependencies and config too — not just the old file.

Propose a cadence with a reason ("daily — labels arrive within the hour, demand shifts weekly") and validate it against label availability, measured decay and compute cost.

How it works — the serving path

Request → features → model → prediction returned and logged. Budget the **entire** path; with small models, feature fetching often outweighs inference. Service hygiene: **timeouts**, a **fallback** score (default or cached), horizontal scaling, versioned releases.

State one product decision: **fail closed** (block when unsure/unavailable — safer for high-risk payments) or **fail open** (allow — better experience)? For card fraud often open for small amounts with rules as backstop, closed/review for high-risk — and say who decides.

Training–serving skew

Skew: serving inputs differ from what the model learned from. Quality drops while error logs stay empty.

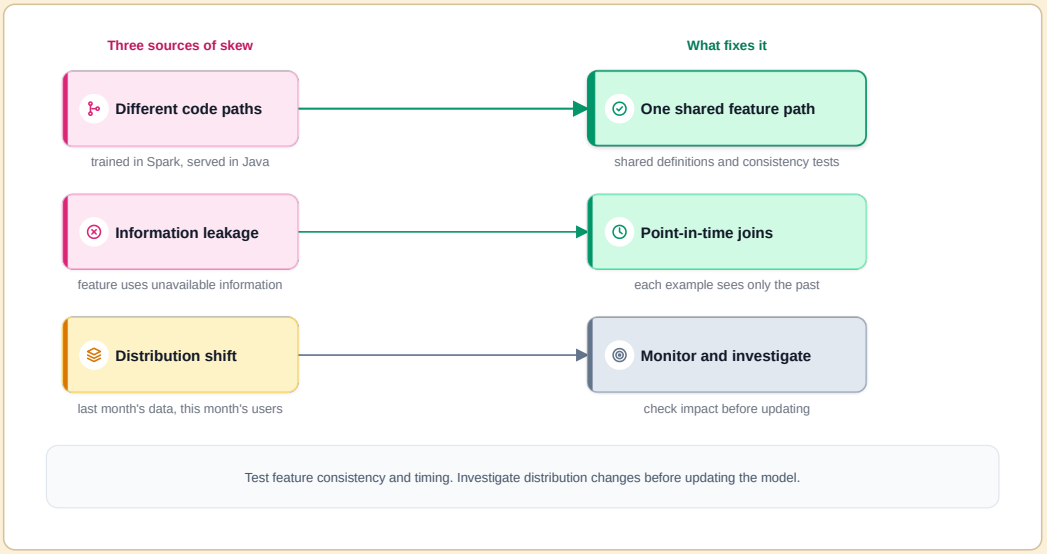


Figure — Skew: causes and remedies (course diagram)

Source	Example	Fix
Different code paths	Spark/Python feature for training rewritten in Java for serving, with a different null default, rounding or time zone	Shared, tested feature definitions; consistency tests comparing both paths
Information leakage	"Today's average delivery time for this restaurant" built from the full day, so it contains the order being predicted	**Point-in-time-correct joins**: each example sees only data that existed at its timestamp
Distribution shift	Trained last month, serving this month	Not a bug, same symptom — monitoring and retraining (16.8)

Rules of ML: **#29** — log serving-time features and train on those logs; **#32** — reuse code between training and serving; **#37** — measure training/serving skew. Logs can still be incomplete or mis-joined, but this removes the biggest source.

Code — leakage vs point-in-time join (`code/point_in_time_join.py`)

`pandas.merge_asof`: for each order, the running mean of same-restaurant, same-day orders **finished before** it was placed.

```python
def pit_feature(df):
    done = df[["rest", "day", "done_h", "minutes"]].sort_values("done_h").copy()
    done["cum_sum"] = done.groupby(["rest", "day"])["minutes"].cumsum()
    done["cum_n"] = done.groupby(["rest", "day"]).cumcount() + 1
    left = df[["rest", "day", "placed_h"]].reset_index().sort_values("placed_h")
    j = pd.merge_asof(left, done[["rest", "day", "done_h", "cum_sum", "cum_n"]],
                      left_on="placed_h", right_on="done_h", by=["rest", "day"],
                      direction="backward", allow_exact_matches=False)   # strictly before
    j = j.set_index("index").sort_index()
    return (j["cum_sum"] / j["cum_n"]).to_numpy()     # NaN = nothing finished yet today

df["avg_today_LEAKY"] = df.groupby(["rest", "day"])["minutes"].transform("mean")  # whole day!
df["avg_today_PIT"] = pit_feature(df)
# ... temporal split (last 12 days = test), fit one GBDT per feature version
```

```
   placed_h  rest  minutes  avg_today_LEAKY  avg_today_PIT
0      10.0    20     31.3             34.1            NaN
1      10.0    31     36.6             44.5            NaN
...
PIT feature missing (no finished orders yet today): 15.0%

setup                                    MAE (min)
distance only (no daily feature)         5.88
LEAKY model, offline test (looks amazing)    3.60
LEAKY model, fed real serving features       4.91
PIT model, offline = what serving will see   4.67
```

The leaky model wins offline (3.60) because its feature partly contains the label; fed serving-time values it's *worse* (4.91) than the point-in-time model (4.67), whose offline number honestly forecasts production. The honest feature is **missing** 15% of the time (first orders of the day) — the model must learn to handle that.

Trade-offs

Choice	Benefit	Cost
Frequent retraining	Fresher model	Compute, evaluation load, release risk
Train on logged serving features	Removes most skew	Log volume, privacy review, no logs yet for new features
Separate training/serving teams	Specialisation	Handoff skew — needs shared definitions

Common mistakes

- Random split of time-ordered data.
- Rewriting feature logic for serving "because it's faster in Java".
- No fallback: checkout hangs when the model service is slow.

In the interview

> **30-second answer** ("walk me through the ML architecture"): *"Offline, a scheduled job assembles examples with features frozen at prediction time, fits a candidate, compares it with what's live, and registers it with lineage. Online, a **stateless** service (no memory between requests) computes features from the same definitions, calls the model within a time budget with timeout and fallback, and logs features, output and version. Artifacts travel to serving; logs and outcomes travel back."*

- **"Why not train continuously?"** → Freshness value vs label delay, compute and evaluation burden; online learning needs guards (16.8).
- **"First model with no logs?"** → Historical data if any; else a rule-based baseline that starts collecting data — with selection bias (outcomes only for what the rules allowed).

TL;DR

Four-stage pipeline ending in a registry; serving with budget, timeout, fallback, logging; kill skew with shared definitions, point-in-time joins and training on logged serving features.

16.3 Where training data comes from

What it is

A supervised model is only as good as the pipeline attaching outcomes to feature rows. Expect probes on labels' source, delay and cost.

Source	Examples	Cost	Delay	Main weakness
Natural labels (observed outcomes)	Delivery duration, chargeback, cancellation	Logging + validation	Minutes to weeks	Late, incomplete; a chargeback is evidence, not proof
Behavioural labels	Clicks, skips, saves, watch time	Logging + processing	Often seconds	Indirect (click ≠ satisfaction); only covers what the system showed
Human annotation	Relevance grades, moderation decisions	Roughly cents to a few dollars each	Hours to days	Budget caps volume; reviewers need quality control

Chip Huyen calls the prediction-to-label gap the **feedback loop length** — short for ETAs and clicks, long for fraud and churn. Delay doesn't stall training — only "mature" examples (old enough to have a label) are used — but outcome monitoring lags by the same amount.

Human labelling is a system, not a task

- **Versioned guidelines** — record the guideline version in lineage; redefining "spam" breaks metric comparisons.
- **Inter-annotator agreement** — several reviewers label the same items (e.g. **Cohen's κ** for two raters, agreement corrected for chance). Low agreement usually means an unclear guideline, hard items or missing context; high agreement ≠ correctness.
- **Gold questions** — known-answer items seeded into the queue track each reviewer's accuracy.
- **Active learning** — spend labels where they're worth most: least-certain items, disagreements, new product areas.
- **Reuse existing queues** — fraud analysts already decide; record context and uncertainty, validate and version before training.
- **Weak supervision** (Snorkel-style **labelling functions** — heuristics that vote) gives many noisy labels cheaply; audit them the same way.

LLM-assisted labelling

Given the guideline, an LLM proposes labels fast. In production it's **tiered**: the LLM labels eligible items, humans check a random sample plus low-confidence or contested ones, and accuracy is measured against human-reviewed sets **per important subgroup**. The catch: LLM errors are **correlated** — one blind spot repeated a million times (dialects, specialist topics, adversarial content) and invisible from inside. Keep human audit slices, re-measure whenever prompt, model or policy changes, check privacy rules before sending data.

Class imbalance

At 0.2% fraud (1 in 500), "always legit" scores 99.8% accuracy, and compute goes on easy negatives.

- **Downsample negatives**: keep all positives and a fraction w (say 10%) of negatives → data ~10× smaller, positive share 0.2% → ~2%.
- **Scores are now inflated** (odds × 1/w). If scores are used as probabilities (dollar thresholds, auctions), **recalibrate**: $p = w \cdot q / (w \cdot q + 1 - q)$, q = downsampled score (e.g. w = 0.1, q = 0.05 → p ≈ 0.0052) — or fit **Platt** (sigmoid) / **isotonic** (step-wise increasing) calibration on held-out data at the real **base rate**. (Rules of ML **#30**: importance-weight sampled data, don't drop it arbitrarily.)
- **Reweighting** alternative: keep all data, weight positives more in the loss. More compute, no resampling.
- Evaluate with **precision and recall**, not accuracy (16.7).

Code — downsample, recalibrate, check (`code/imbalance_calibration.py`)

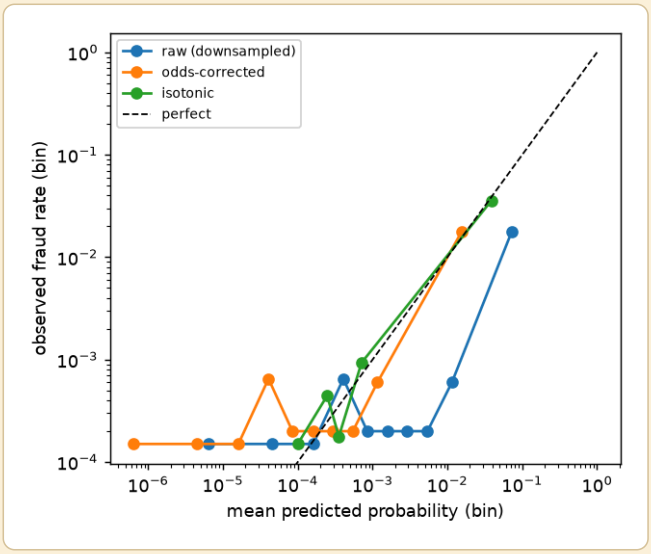
```
w = 0.10
keep = (ytr == 1) | (rng.random(len(ytr)) < w)              # all fraud, 10% of legit
model = LogisticRegression(max_iter=1000).fit(Xtr[keep], ytr[keep])
q = model.predict_proba(Xte)[:, 1]
p_corrected = w * q / (w * q + 1 - q)                       # undo the 1/w odds inflation

# or learn the mapping on held-out data with the true base rate (scikit-learn >= 1.6)
cal_platt = CalibratedClassifierCV(FrozenEstimator(model), method="sigmoid").fit(Xcal, ycal)
cal_iso = CalibratedClassifierCV(FrozenEstimator(model), method="isotonic").fit(Xcal, ycal)
```

```
base rate: 0.200%
'always legit' accuracy: 99.80%  (catches 0 fraud)

downsampled train: 30,732 rows (was 300,000), positive share 1.94%

scores                        mean  PR-AUC     Brier  logloss      ECE
(true rate)                 0.0020
raw (downsampled)           0.0095   0.653  0.001610   0.0119  0.00758
odds-corrected (w=0.1)      0.0018   0.653  0.001093   0.0063  0.00044
Platt on held-out           0.0021   0.653  0.001016   0.0062  0.00038
isotonic on held-out        0.0021   0.624  0.001020   0.0061  0.00020
-> ranking (PR-AUC) is identical for raw and corrected: calibration is a separate property
```



Raw scores average 0.95% vs a true 0.2% (≈5× too high); the formula and Platt restore ~0.2% and nearly halve **log-loss**. **Monotonic** fixes leave **PR-AUC** unchanged (ranking ≠ calibration); isotonic's small drop comes from ties in its step function. (`FrozenEstimator` replaced `cv="prefit"` in scikit-learn 1.6.) **ECE** (expected calibration error) = weighted mean gap between score and observed rate per bin; **Brier** = mean squared error of probabilities.

Manufacturing negatives (retrieval)

Logs show clicks, never what a user ignored, so retrieval models (Chapter 13) manufacture negatives: **random** (easy, teaches little about close calls); **in-batch** (other users' positives — cheap, but popular items are over-represented, so apply a **popularity/sampling-bias correction** or the model learns to demote popularity itself); **hard** (shown-not-clicked, near neighbours — sharpen the boundary, but some are false negatives). Mix and tune the ratio.

Split hygiene and the leakage checklist



Figure — Time-based vs random split (course diagram)

- **Temporal splits** (nearly always): train weeks 1–10, validate 11, test 12; a random split lets the model train on week 12 and "predict" week 3 (Rules of ML **#33**: trained through Jan 5 → test from Jan 6). Generalising to new users needs **entity-based splits** (all of a user's rows in one split).
- **Point-in-time features** in every example (16.2).
- Leakage checklist: (1) aggregates whose window includes the label moment ("merchant fraud rate" including this transaction); (2) **label-adjacent fields** filled only because the outcome happened (refund flag, case-resolution code); (3) near-duplicates across splits (decide whether the same user may appear in both, per what you're testing).

Privacy, in one paragraph

Minimise **PII** (personally identifiable information) at ingestion: collect only what's needed, restrict access, set retention. Hashed IDs are still linkable. Define how deletion requests reach raw data, derived datasets, embeddings and models (versioned data, deletion tracking, scheduled rebuilds — involve privacy/legal). Fraud verdicts and moderation decisions are sensitive too.

In the interview

> **30-second answer** ("where does your training data come from?"): *"Outcomes, behaviour and human judgments, each with a cost and delay. I validate labels, join them to point-in-time features, split by time, downsample and recalibrate for imbalance, and audit LLM labels against human slices."*

- **"Fraud outcomes take six weeks. What do you train on today?"** → Matured rows (window stops six weeks back) plus provisional signals — rule hits, analyst verdicts — for recent traffic. Outcome monitoring lags six weeks; score-distribution monitors cover the gap.
- **"Annotators disagree 30% of the time."** → A guideline bug first: cluster disagreements, add examples, re-version, re-measure. For truly ambiguous items, collect several judgments and train on the majority or the distribution.

TL;DR

Outcomes (slow), behaviour (fast, biased), humans (costly, need QA). Downsample then recalibrate. Split by time, join point-in-time, dedupe, minimise PII.

16.4 Features and feature stores

What it is

A **feature store** computes, stores and serves features consistently for training (history) and serving (latest values). For each feature say what it is, its source, required freshness and cost.

How it works — four feature groups (ETA example)

Group	Meaning	ETA examples	Typical compute path
Entity	Rarely-changing attributes of restaurant, courier, customer	Typical prep time, courier vehicle, building type	Batch (nightly)
Behavioural aggregates	Counts/averages over a recent window	Orders last 30 min, courier speed today, dish median prep over 7 days	Streaming for short windows, batch for long
Contextual	The request moment	Time, weekday, weather, distance, big match on TV	Request time
Cross	Two entities together	This courier at this restaurant; this customer in this cuisine	Batch or streaming

A second gap-finding checklist by **signal source**: the item's content, the actor, their recent behaviour, graph links between entities, the request's circumstances.

Turning raw signals into model inputs

Kind of input	Usual encoding	Example
Huge ID vocabularies	Learned **embedding tables** (Chapter 13); for unbounded ID sets the **hashing trick** maps each ID to one of a fixed number of buckets	user, item, merchant IDs
Numbers	Bucketing, normalisation, log transform for **heavy tails** (a few enormous values)	order counts, km, prices
Short category lists	One-hot or tiny embeddings	device type, country
Interacting pairs	Hand-built crosses, or a deep model that discovers them	user × category

Trees don't need scaling; linear models and neural nets do.

Freshness: batch path + streaming path

A half-hour order count refreshed nightly tells the model nothing, so the feature layer may have two arms:

- **Batch path** — Spark/SQL jobs compute slow features for every entity; easy to **backfill** (compute past values).
- **Streaming path** — Flink or Spark Structured Streaming reading **Kafka** (a durable, multi-consumer event log) keeps windowed aggregates current within seconds; more operational work (state, late events, backfills).

This is the **lambda architecture** (Nathan Marz); streaming-only is **kappa**. Run both only when freshness requires it — Rules of ML **#8: know the freshness requirements of your system**.

What a feature store actually is

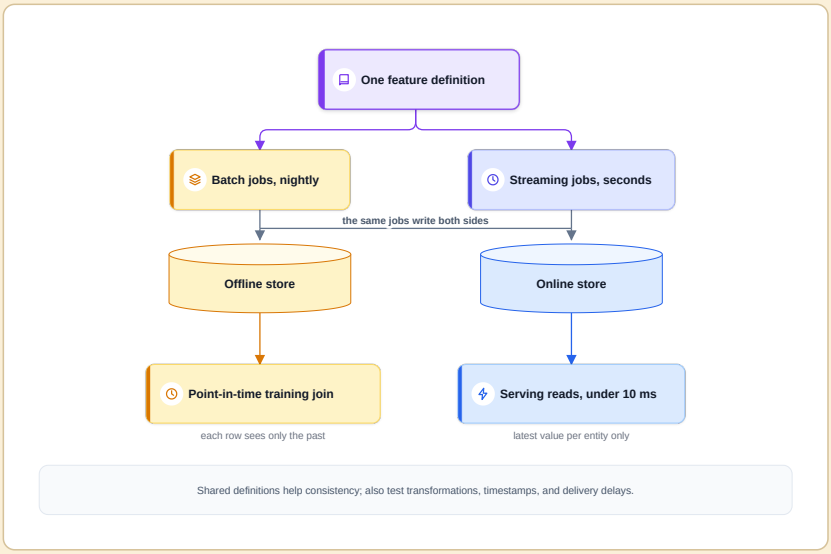


Figure — Feature store: shared definitions, offline history, online latest (course diagram)

Side	Storage	Holds	Used for
Offline store	Warehouse / lake (Parquet, BigQuery, Snowflake…)	Timestamped history of feature values	Training sets via **point-in-time joins** — a row from 6:02 pm on 3 March gets values as of then
Online store	Low-latency **key-value store** (Redis, DynamoDB, Cassandra, Bigtable)	Only the **current** value per feature per entity	Serving: "40 features for restaurant 812, courier 3407, this route — in < 10 ms"

One set of definitions feeds both, and the store doubles as a **registry** so teams reuse rather than re-derive (re-derivation recreates skew). Still test transforms, timestamps, late data and empty values. Products: **Feast** (open source), **Tecton**, **Databricks**, **SageMaker** and **Vertex AI** feature stores; Uber's **Michelangelo** popularised the pattern.

Request-time features

Some values exist only at request time (route distance, current cart). The trap: an expensive call (routing service, third-party API) with no latency plan. If it eats 80 ms of a 100 ms budget: (1) **cache** a recent answer; (2) **approximate** (straight-line vs road distance); (3) **default** a "missing" value on timeout. Train and evaluate with those degraded inputs so the fallback is rehearsed and its quality cost known. Budget line: *"15 ms to assemble features — one batched lookup and two computed inline; anything late takes the default the model met in training."*

Trade-offs

	Batch features	Streaming features	Request-time features
Freshness	Hours	Seconds	Instant
Cost/complexity	Low	High (state, late events, backfills)	Latency on the critical path
Backfill for training	Easy	Harder	Must be logged at serving

Common mistakes

- 7-day aggregates computed at request time (millions of events per call).
- Silent staleness: the stream falls behind with no alarm. Put **feature freshness** on the main dashboard with its own alert; consider feeding "feature age" to the model.
- A second team re-implements a feature → skew.

In the interview

> **30-second answer**: *"Entity features in nightly batch, short-window aggregates in streaming, request context inline under a budget. A feature store gives point-in-time history for training and latest values for serving from one definition, and I test that both paths agree."*

- **"Why not compute everything per request?"** → Too slow and costly: precompute what tolerates it, stream what must be current, request-time only for what can't be known sooner.
- **"The streaming job lags?"** → Freshness alert, trained-in defaults, feature age as input.

TL;DR

Four groups; batch for slow, streaming for fresh, inline for request-only; feature store = offline history + online latest; budget request-time features.

16.5 Batch vs online vs streaming inference

What it is

Batch inference precomputes predictions on a schedule into a key-value store (a request becomes a lookup). **Online inference** computes per request. **Streaming inference** scores events arriving through a queue.

	Batch	Online
When computed	On schedule, for all (or active) entities	Per request
Request-time work	KV lookup, ~1 ms	Feature fetch + model, ~10–100 ms
Infra	Scheduled job + KV store	Always-on capacity, autoscaling
Freshness	As old as the last run	As fresh as the inputs
Can use request context?	No	Yes (query, cart, route)
Cost shape	Batch size × reuse × hardware	Scales with traffic, peaks at busy hours

How it works — the deciding question

Does the prediction need information that exists only at request time? Then weigh freshness and reuse.

- ETA: yes (route, cart, live queue) → online.
- Fraud: the transaction *is* the input → online (or streaming).
- "Videos you might like" shelf: mostly no → batch (last night's top-500 is fine).
- Search ranking: query is request-time, per-item quality scores precomputable → hybrid.

Waste: 25M users × 500 recommendations = 12.5B rows, mostly unread if 80% don't visit this week. Precompute for **active users only** (5M → 2.5B rows), keep **fewer items per user** and reorder cheaply online, or precompute per **segment**.

The hybrid: precompute-then-rerank

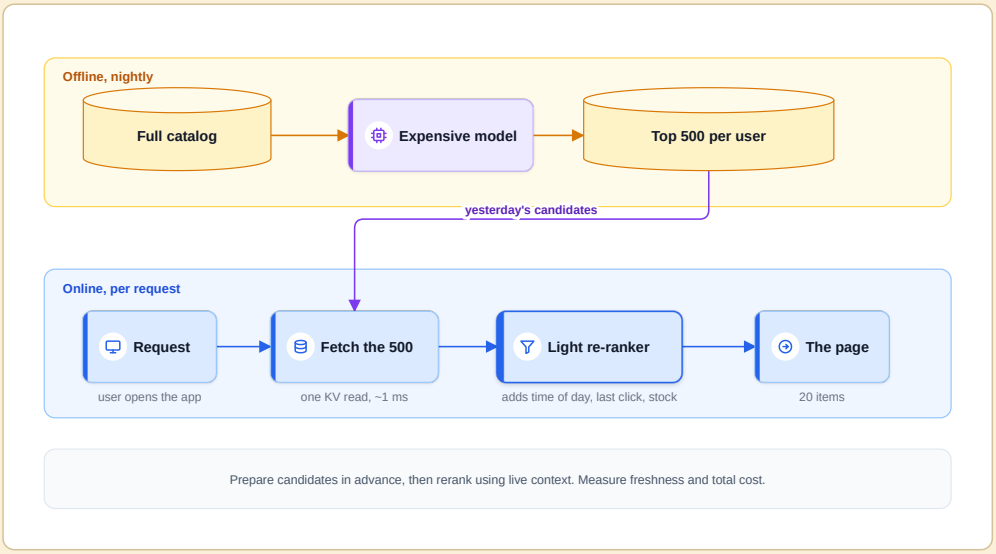


Figure — Nightly candidates, live rerank (course diagram)

Offline, an expensive model picks ~500 candidates per user; online, a small model reorders them using the moment (time, last click, stock). New items need an online retrieval path. Degrades gracefully: yesterday's candidates if the nightly job fails (alert on staleness, rerun — an online failure would be an immediate incident), unreranked candidates if the reranker fails.

Streaming inference

Events trigger predictions (each uploaded video scored for policy, each payment in the pipeline): online inference behind a queue, no synchronous caller. Handle queue delay, retries, **duplicate events** (**idempotent** scoring — twice has the same effect as once) and **backpressure** (slowing producers when consumers can't keep up) — see System Design Fundamentals.

Trade-offs / when not to use

- Batch: wrong when inputs exist only at request time or staleness hurts (fraud, ETA).
- Online: wasteful when predictions are stable and heavily reused.
- Streaming: wrong when a caller waits **synchronously** for the answer.

In the interview

> **30-second answer**: *"Does it need request-time inputs, how fresh must it be, how often is a stored prediction reused? Stable and reused → batch; request-dependent → online; event-driven async → streaming. For feeds, precompute candidates for active users and rerank online."*

- **"How fresh is fresh enough?"** → Measure: plot CTR against stored-candidate age; the decay rate sets the rerun cadence.

TL;DR

Batch = lookup, stale, cheap per request; online = fresh, request context, pay at peak; streaming = event-triggered async. Precompute-then-rerank is the default for large catalogs.

16.6 Model serving basics

What it is

A typical predictive model server is **stateless**: it loads the model at startup and turns feature vectors into scores, so it scales like any web tier — **replicas** behind a **load balancer**, autoscaling on CPU or queue length, **rolling deploys**, **health checks** (System Design Fundamentals). ML-specific parts: artifact source, latency budget, safe rollout, hardware.

The latency budget, itemised (illustrative small-model numbers)

Stage	Rough cost	Comment
Model inference	1–10 ms	Trees and small tabular models are quick; deep nets need tuning
Online-store lookup	2–10 ms	One batched read; frequently the slowest step (the usual small-model bottleneck)
Features computed inline	0–15 ms	Each with its own allowance and a default if late
Routing, auth, parsing	1–5 ms	Usual web-service overhead
Writing the log, sending the reply	1–2 ms	Logging asynchronous so it doesn't block

Profile before optimising:

- **Batching** (dynamic/micro-batching) — group requests for a few ms for better hardware use, especially GPUs; trades a small wait for throughput, so test **p99** (latency 99% of requests beat).
- **Feature caching** — keep the most-requested entities' vectors (busy restaurants, trending videos) in process memory so most traffic never hits the online store.

Shipping a new model safely

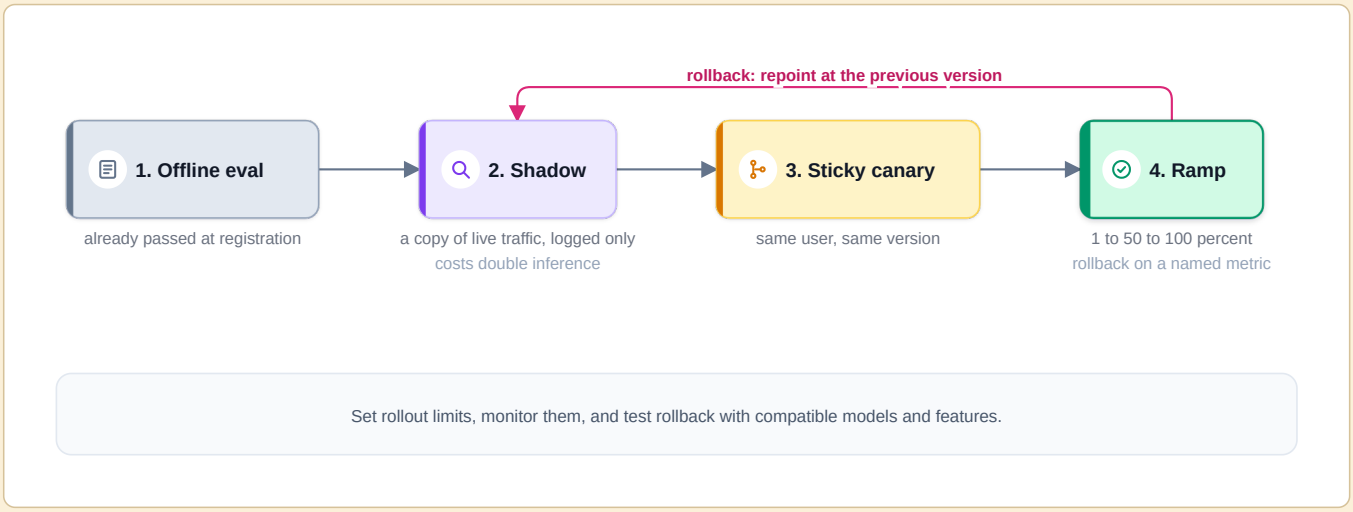


Figure — Rollout stages: offline eval, shadow, canary, ramp (course diagram)

1. **Offline evaluation** — passed (that's how it got registered).
2. **Shadow deployment** — candidate scores a copy of live traffic; outputs logged, never used. Catches serving bugs, latency, big behaviour differences; isolate capacity, disable side effects.
3. **Canary / A/B** — a small traffic share gets real outputs from the new model, usually **sticky per user** (sessions, orgs, regions or time windows in other cases).
4. **Ramp up with a pre-agreed abort rule** — "revert if session CTR falls 2% or p99 exceeds 150 ms". Rollback restores a compatible model *and* features and config; rehearse it and know how long it takes.

Hardware

Trees and small nets serve well on CPU — say *"I'd benchmark the GBDT on CPU replicas first"*, not "GPU cluster" by reflex; GPUs pay off for large neural models with batching. Runtimes worth naming: ONNX Runtime, NVIDIA Triton, TensorFlow Serving.

Shrinking the model

Technique	Idea	Watch out
Distillation	A small *student* mimics a large *teacher*	Measure the quality loss
Quantisation	Fewer bits per weight/activation (int8 vs float32)	Needs runtime support; check accuracy
Pruning	Drop weights that contribute little	Speedups need sparse-aware runtimes

Compare with simply changing hardware, counting engineering effort and quality loss. (Central for LLMs — AI Handbook.)

Degradation is part of the spec

Agree the **fallback prediction** with product and risk owners: previous version kept warm, cached prediction, heuristic (average ETA; rules-only fraud), or product default (unpersonalised shelf). For fraud it may be review or decline, not "allow". Name the owner and how long degraded mode may last before paging.

Model-in-service vs model-as-a-service

Embedding the model in the app saves a network hop but couples deploys and scaling. A standalone model service costs a few ms but gives independent rollout, sharing across teams and right-sized hardware — the default at scale.

Common mistakes

- Forgetting timeouts and autoscaling once a model is in the diagram.
- No model version logged per prediction — A/B results can't be attributed.
- Rollback = "redeploy the old file" after the feature schema changed.

In the interview

> **30-second answer**: *"A stateless, versioned model service loading from the registry, scaled like a web tier. I itemise the latency budget and profile before adding batching or caches. Releases: offline eval → shadow → sticky canary → ramp with a named rollback metric. Fallback: previous version or a heuristic agreed with product."*

- **"A/B two models cheaply?"** → One service with both versions loaded, sticky per-user router, version logged per prediction; shadow first, since experiments cost weeks of traffic.

TL;DR

Stateless versioned service; itemised budget; batching and hot caches; shadow → canary → ramp; CPU first; distill/quantise/prune; explicit fallback.

16.7 Offline evaluation: choosing the metric

What it is

Offline metrics are the *first gate* (held-out data, before users see anything); the online experiment (16.9) is the second. Definitions — precision/recall, ROC-AUC vs PR-AUC, MAE/RMSE, recall@k/NDCG/MRR, calibration — are in Chapter 6 (§6.2 regression, §6.3 classification and thresholds, §6.4 ranking, §6.5 calibration). This section adds only the production twist.

Problem	Report	Production twist
Rare-event classification (fraud, abuse)	P/R at the shipping threshold, PR-AUC beside the base rate	Thresholds are business policy; often two (approve / review / block)
Retrieval → ranking	recall@k for candidate generation, NDCG@k or MRR for ordering	Each funnel stage its own metric; click labels are position-biased
Regression (ETA, demand)	MAE or RMSE plus p90 error	Break down per **cohort** — new cities or segments hide inside a good average
Any score used as a number	Calibration (reliability diagram, ECE)	Recheck after downsampling/re-weighting (16.3) and population shifts

Production checklist

1. **Eval set looks like traffic**: time-based split, real base rate, no leaked features (16.2, §4.7–4.8).

2. **Report the operating point, not only an area**: "92% precision at 60% recall at threshold 0.05", plus the review workload (alerts/day).
3. **The cost owner picks the threshold**: product/risk price blocked good customers vs missed fraud; ML supplies the PR curve and expected costs.
4. **Slice**: country, device, new vs returning, new vs mature market. Ship only if no important slice regresses.
5. **Beat two baselines**: naive (majority class, popularity, "same hour last week") and current production. A 6-minute MAE impresses against a naive 8, not a naive 6.2.
6. **Ranking label source**: judged sets (human graders following guidelines) plus behavioural labels with position-bias correction.

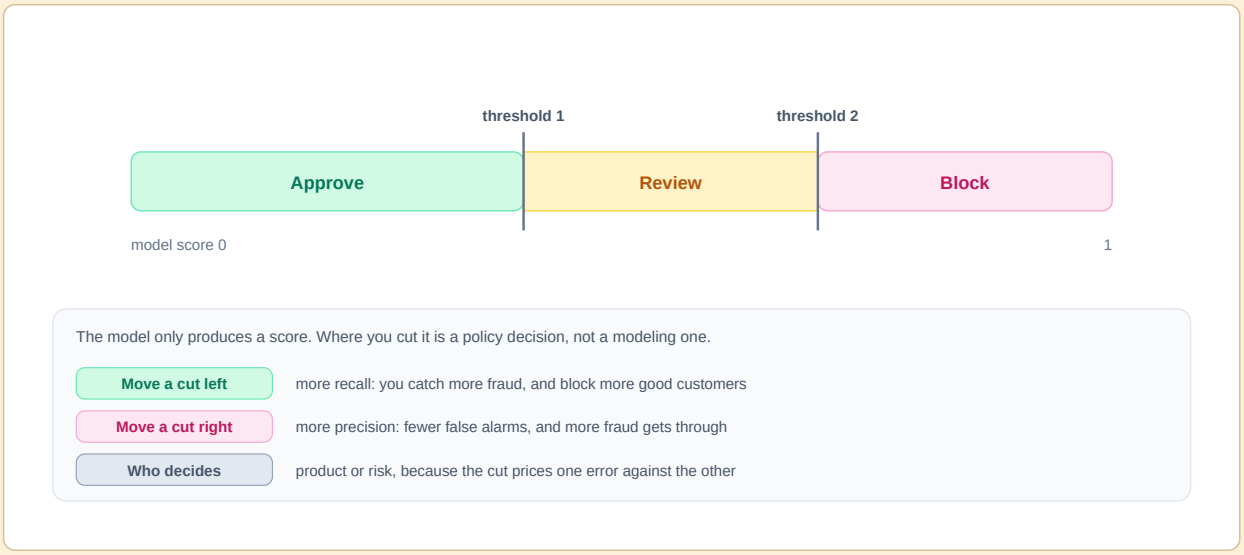


Figure — Approve / review / block bands from two cut-offs (course diagram)

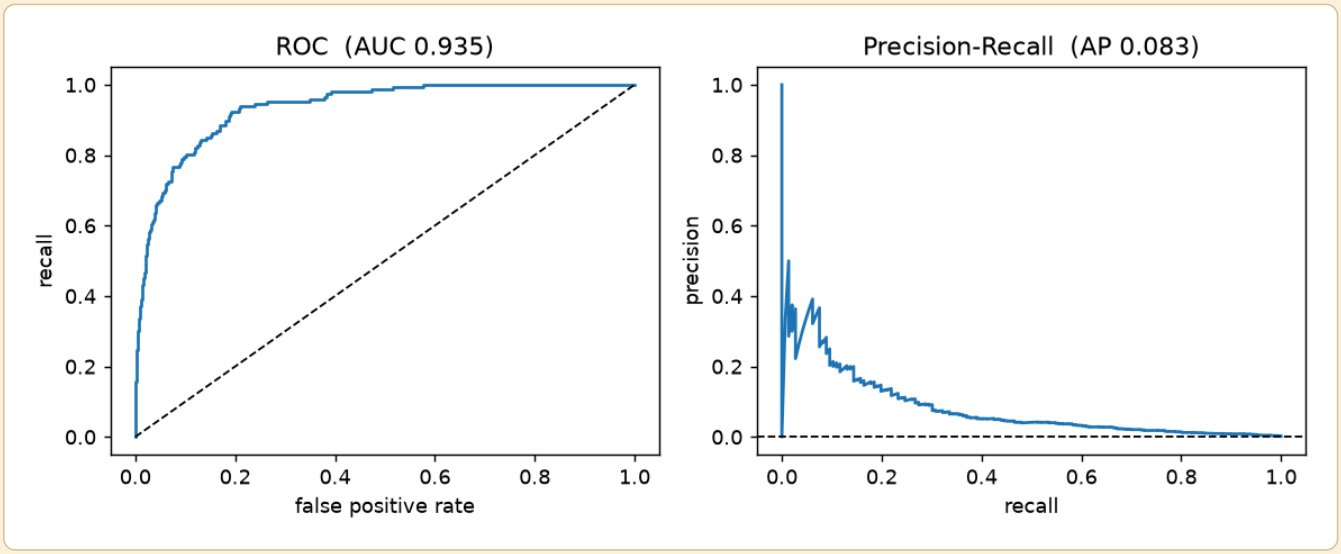
Code — one production-sized run per family (`code/offline_metrics.py`)

Logistic regression on synthetic transactions with 0.2% fraud: threshold sweep, two-threshold bands, a by-hand NDCG checked against scikit-learn, ETA errors split by city cohort.

Output (abridged):

```
== classification (rare positives) ==
test rows 80,000, positives 146 (base rate 0.18%)
ROC-AUC 0.935   PR-AUC (average precision) 0.083   PR-AUC of a random model = base rate = 0.002
threshold  flagged   TP     FP precision  recall     FPR
    0.005     5776  106   5670      0.02    0.73  0.0710
    0.050      462   42    420      0.09    0.29  0.0053
  approve :  77081 txns, fraud rate in band 0.07%
  review  :   2457 txns, fraud rate in band 1.91%
  block   :    462 txns, fraud rate in band 9.09%

== regression (ETA minutes) ==
overall  MAE  3.71  RMSE  5.27  p90|err|  7.41
mature   MAE  3.17  RMSE  3.99  p90|err|  6.55  share 93%
new      MAE 10.93  RMSE 13.69  p90|err| 22.66  share 7%
naive baseline (predict the mean) MAE  9.46
```



(TP = fraud caught; FP = good transactions flagged; FPR = false-positive rate.) A 0.935 ROC-AUC passes a dashboard review, but catching 73% of fraud means 5,670 false alarms for 106 catches — the analyst queue, not the AUC, is what the business feels. Two bands put 9% fraud in "block" and 1.9% in "review", so humans look at 2,457 cases instead of 80,000. Overall ETA error 3.7 min hides 10.9 min in new cities.

In the interview

> **30-second answer**: *"I pick the offline metric from the task, but in production I report it at the shipping threshold with the workload it creates, sliced by cohort, against a naive baseline and the current model. The cost owner chooses the threshold. Offline wins only earn an A/B test."*

- **"Offline metric up, A/B flat?"** → The offline metric is a proxy: weak label (click vs watch time), test set no longer like traffic, or serving-time degradation (skew). Check experiment power and logging first.

TL;DR

Definitions in §6.2–6.5. Production: traffic-like eval set, operating point + workload, owner-chosen thresholds, cohort slices, naive and production baselines — then an A/B test.

16.8 Monitoring, drift and retraining

What it is

Deployed models lose quality as the world changes — fraudsters adapt, a pandemic reshapes delivery times, new slang fools a toxicity classifier. Decay rates differ; measure them.

Drift type	What changes	Example	Also called (Huyen)
Data drift	Inputs P(X)	Orders shift from lunch to dinner	Covariate shift
Label / prior shift	Outcomes P(Y)	Fraud rate doubles in a holiday	Label shift
Concept drift	The relationship P(Y | X)	Same features, different outcomes — new fraud tactics, a restaurant changes kitchens	Concept drift

Data drift may not hurt accuracy; concept drift always needs new labels. Monitoring must also tell drift from plain **bugs** (a broken upstream pipeline looks like drift).

How it works — the four-layer monitoring stack

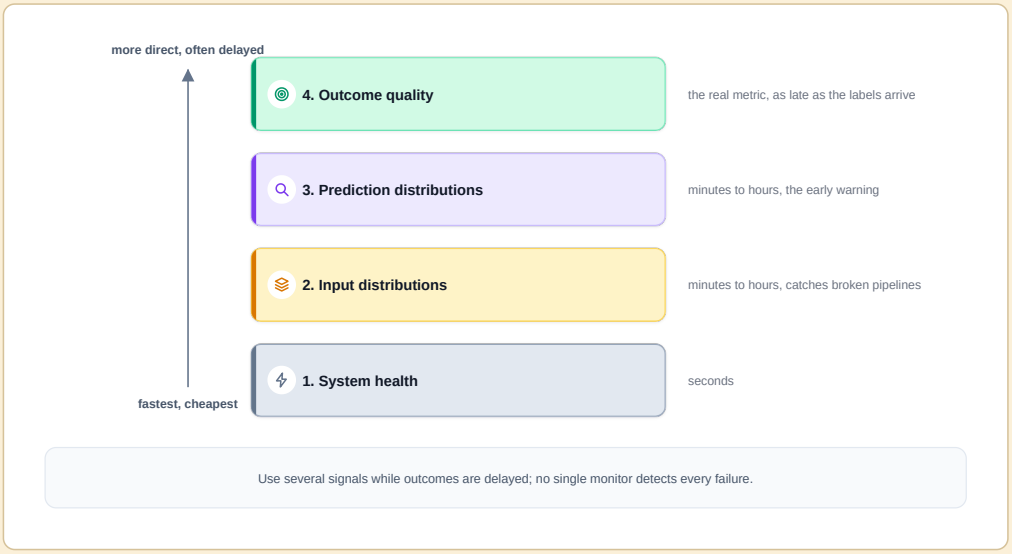


Figure — Monitoring layers from fast/cheap to slow/direct (course diagram)

Layer	How quickly it reacts	Signals tracked	Typical findings
1. Service health	Seconds	Traffic, latency, error rate, feature-store value age	Outages; pipelines that stopped updating
2. Inputs	Minutes to hours	Each feature vs its training reference — null share, means, PSI or KS	Data drift; a producer sending garbage
3. Outputs	Minutes to hours	Score histogram; share of positive decisions	Model behaving differently, visible before labels exist
4. Outcomes	When labels land — seconds for clicks, ~an hour for ETAs, weeks for chargebacks	CTR, realised ETA error, recall on confirmed fraud	Concept drift and genuine quality loss

Layers 2–3 exist because layer 4 is late; some attacks move neither inputs nor scores, so combine signals. Rules of ML **#9: detect problems before exporting models**; **#10: watch for silent failures** (a joined table that stopped updating). *Reliable Machine Learning* (Chen, Murphy et al., O'Reilly 2022) treats model quality as SLOs with incident playbooks.

Drift statistics

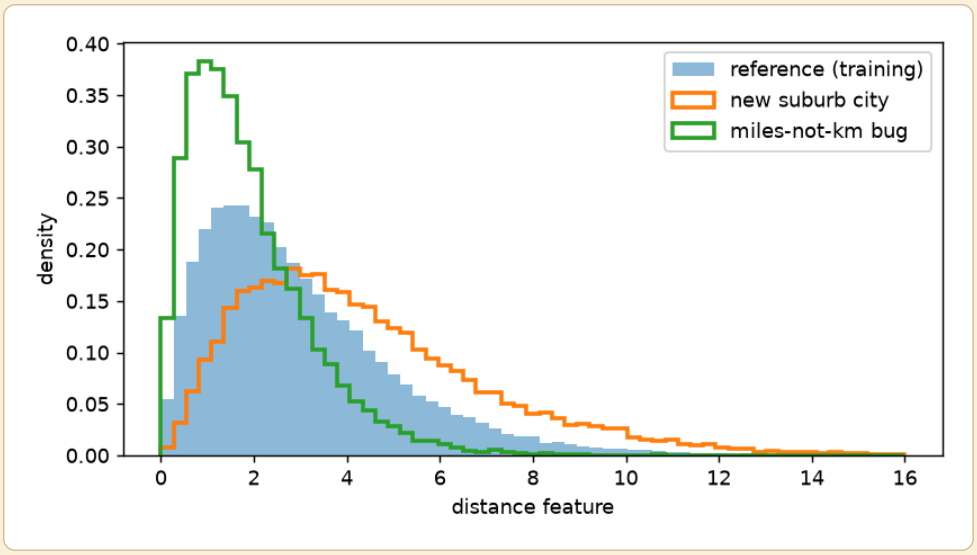
- **PSI (Population Stability Index)**: bin the reference distribution into **quantiles**; PSI = Σ (cur% − ref%) · ln(cur% / ref%). Credit-scoring convention: < 0.1 stable, 0.1–0.25 watch, > 0.25 investigate.
- **Two-sample Kolmogorov–Smirnov (KS) test**: D = max gap between the two empirical **CDFs**, plus a **p-value**. With large samples trivial shifts become "significant" — look at D (the **effect size**).
- **Null rates and schema checks** — PSI on non-null values can't see a producer that started sending 30% nulls.
- Categorical features: **chi-square test** or PSI over categories. Tools: Evidently, WhyLabs, NannyML, SageMaker Model Monitor, Vertex AI Model Monitoring, TFDV.

Code — PSI and KS (`code/drift_psi_ks.py`)

```python
def psi(reference, current, bins=10, eps=1e-6):
    ref, cur = reference[~np.isnan(reference)], current[~np.isnan(current)]
    edges = np.quantile(ref, np.linspace(0, 1, bins + 1))
    edges[0], edges[-1] = -np.inf, np.inf            # catch values outside the training range
    r = np.histogram(ref, edges)[0] / len(ref)
    c = np.histogram(cur, edges)[0] / len(cur)
    r, c = np.clip(r, eps, None), np.clip(c, eps, None)
    return float(np.sum((c - r) * np.log(c / r)))

ks = ks_2samp(reference, current)                    # scipy.stats
```

```
same distribution (new sample)     PSI  0.001  KS D 0.007  p  4.9e-01  nulls  0.0%  -> stable
tiny shift (+3%) -- big n          PSI  0.002  KS D 0.018  p  2.5e-11  nulls  0.0%  -> stable
new suburb city (longer trips)     PSI  0.361  KS D 0.241  p  0.0e+00  nulls  0.0%  -> INVESTIGATE
upstream bug: miles not km         PSI  0.426  KS D 0.251  p  0.0e+00  nulls  0.0%  -> INVESTIGATE
30% nulls (broken producer)        PSI  0.000  KS D 0.006  p  7.5e-01  nulls 30.2%  -> stable
```



(1) A 3% shift on large samples gives p ≈ 10^{-11} but PSI 0.002 — don't page on p-values. (2) A new city and a unit bug look the **same** — alerts start investigations (PSI 0.4 ≠ "retrain"), they don't prove drift. (3) PSI/KS missed the 30% nulls — check null rates separately.

Retraining: cadence, trigger, gates

Retraining is one possible response — first fix broken data, policy or serving bugs.

1. **Scheduled** — simple, right for gradual drift; set cadence from measured decay (a weekly model down 2% AUC by day 7 → twice a week).
2. **Triggered** — monitor fires → investigation → candidate run if validated data exists. Never auto-promote because an alert fired.
3. **Online / continual learning** — incremental updates (Huyen: **stateless retraining** from scratch vs **stateful training** from the last checkpoint). Suits fast-changing tasks; needs bounded updates, validation, rollback — a bad stream corrupts an online learner fast.

Every candidate passes the same gates: offline vs production (overall **and per segment**) → shadow → canary, approved by a named owner or automated limits. The history window should cover **seasonality** yet drop stale patterns — treat it as a **hyperparameter**, weighting recent data in fast-drift domains.

Feedback loops that damage your data

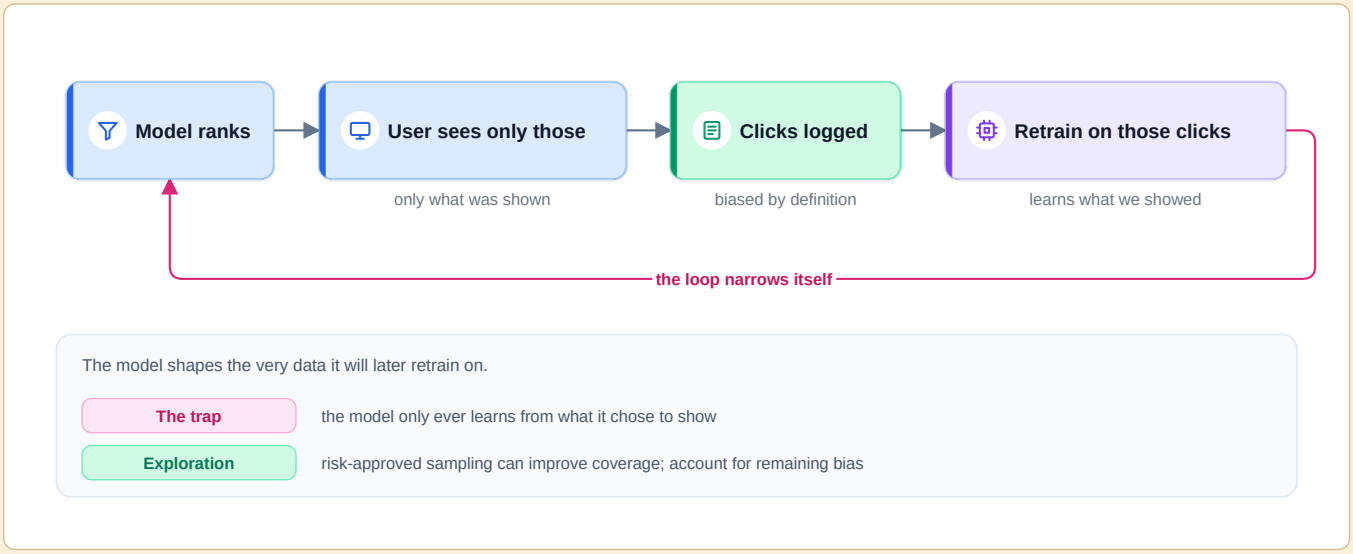


Figure — How a ranker narrows its own training data (course diagram)

The model shapes the data it later learns from — Huyen's **degenerate feedback loop**. A feed shows what it predicts you'll like, you can click only what was shown, and the retrained model learns "what we showed gets clicked" (popularity bias, filter bubbles). In fraud, blocked transactions never produce an outcome (investigator evidence gives partial labels — record source and uncertainty).

Mitigations:

- **Controlled exploration** — in low-risk recommenders, a small approved traffic share gets randomised candidates, with **selection probabilities logged** for later **inverse propensity weighting** (unbiased estimates); bandits (Chapter 14) formalise this.
- Fraud: never let suspicious payments through merely to learn outcomes; use risk-approved reviews, simulations, or small strictly limited experiments.
- Positional features: log position at training, fix it to a constant at serving (Rules of ML **#36**).
- Exploration improves coverage but doesn't remove all bias; measure its benefit and user cost.

In the interview

30-second answer: *"Four layers — service health, inputs, outputs, real outcomes; labels are late, so the first three warn early. An alert starts an investigation; often it's a broken pipeline, not the model. Retraining runs on a cadence set by measured decay or after an investigation, and every candidate clears offline, shadow and canary gates. A small, risk-approved exploration budget guards against the model skewing its own labels."*

- **"Quality dropped overnight — debug it."** → Bottom up: pipeline freshness and feature nulls (most likely, fastest) → prediction histogram → segment the drop (one country? app version?) → only then the model.

TL;DR

Data/label/concept drift; four monitoring layers; PSI/KS + null checks; gated retraining by schedule or after investigation; beware models that shape their own data.

16.9 Online experiments and A/B testing

What it is

Offline asks whether the new model predicts held-out labels better; production asks whether it makes the **product** better. An **A/B test** randomly splits units between control (A, current) and treatment (B, new) and compares a business metric.

Why offline numbers aren't enough

The label is a proxy (a model can win on click-AUC while lowering watch time); **deploying changes the behaviour it's judged on** (a new ranker changes what people see and click — a frozen test set can't reflect that); **serving isn't scoring** (timeouts, stale caches, fallbacks never seen offline).

How it works — two gates, and shadow in between

Offline gate (predefined quality, cost, safety, per-subgroup criteria) → **shadow mode** when failures are expensive (identical live requests, outputs hidden) → **online gate**: A/B on the **business** metric (some products can't A/B safely or lack traffic).

A/B mechanics

1. **Randomise by unit** — usually users, via a **hash** of user id + experiment name for **sticky** assignment; sessions, orgs, regions or time windows when users affect each other (**interference**) or effects carry over.
2. **One primary metric, chosen in advance** (avoids p-hacking) — the rest are guardrails or diagnostics.
3. **Size for power** — fix the **minimum detectable effect** (MDE), significance α (false-positive rate, typically 0.05) and **power** (chance of detecting a true effect of that size, typically 0.8). Per-arm sample size for two proportions: $n = [z_{1-\alpha/2} \cdot \sqrt{(2 \cdot \bar{p}(1-\bar{p}))} +$

$z_1\text{-}\beta \cdot \sqrt{p_1(1-p_1) + p_2(1-p_2)}]^2 / (p_2 - p_1)^2$ ($z_1\text{-}\alpha/_2 \approx 1.96$, $z_1\text{-}\beta \approx 0.84$ from the standard normal; p_1, p_2 the arms' rates, $\bar{p}$ their mean.) Rare events, small effects and noisy metrics need big samples.

4. **Duration** — from sample size, outcome delay and whole weekly cycles; watch **novelty effects** and seasonality.
5. **Don't peek** — stopping at the first daily $p < 0.05$ inflates false positives. Fix an end date or use a **sequential test** (alpha-spending, always-valid methods). **CUPED** (pre-experiment behaviour as a **covariate**) cuts variance and sample size.

Code — sample size, z-test, peeking (`code/ab_test.py`)

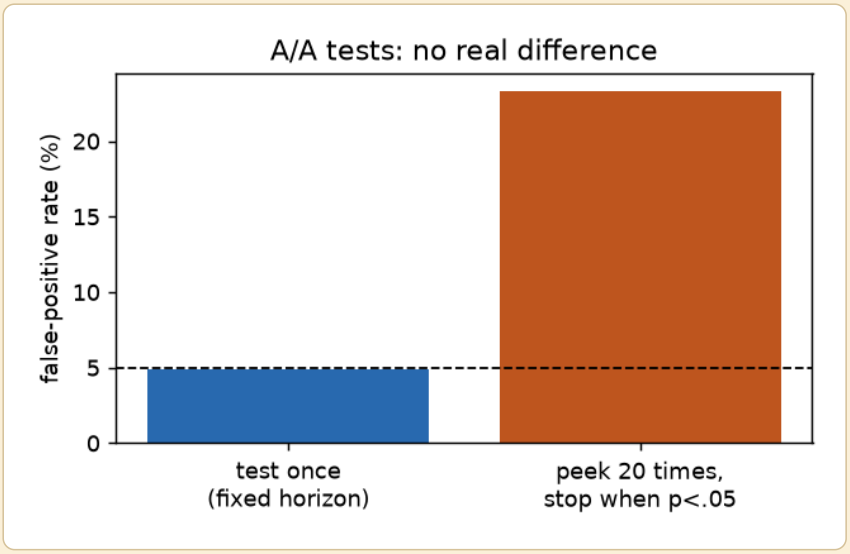
```python
def assign(user_id, experiment, treatment_share=0.5):
    h = int(hashlib.sha256(f"{experiment}:{user_id}".encode()).hexdigest(), 16)
    return "B" if (h % 10_000) / 10_000 < treatment_share else "A"      # sticky

def two_prop_ztest(x_a, n_a, x_b, n_b):
    pa, pb = x_a / n_a, x_b / n_b
    pooled = (x_a + x_b) / (n_a + n_b)
    se0 = np.sqrt(pooled * (1 - pooled) * (1 / n_a + 1 / n_b))        # SE under H0
    z = (pb - pa) / se0
    p = 2 * (1 - norm.cdf(abs(z)))
    se = np.sqrt(pa * (1 - pa) / n_a + pb * (1 - pb) / n_b)           # SE for the CI
    return pa, pb, z, p, (pb - pa - 1.96 * se, pb - pa + 1.96 * se)
# ... sample_size_per_arm(), and 2,000 simulated A/A tests with 20 peeks each
```

```
sticky assignment: ['A', 'B', 'A', 'A', 'A', 'A', 'B', 'B'] | again: True
baseline CTR 5%, detect +1% relative lift (alpha .05, power .8): 2,996,698 users per arm
baseline CTR 5%, detect +2% relative lift (alpha .05, power .8): 752,703 users per arm
baseline CTR 5%, detect +5% relative lift (alpha .05, power .8): 122,124 users per arm

A: 19,906/400,000 = 4.9765%   B: 20,575/400,000 = 5.1437%
diff +0.1673% (relative +3.4%), z = 3.41, p = 0.0006, 95% CI [+0.0712%, +0.2633%]

A/A tests, alpha 0.05: false-positive rate testing once at the end = 4.9%; stopping at the first significant of 20 peeks = 23.3%
```



Halving the lift roughly **quadruples** the sample ($n \propto 1/\delta^2$): a 1% relative CTR lift needs ~3M users per arm. The finished test is significant (the 95% **CI** excludes zero). In an **A/A test** both groups get the same version, so any "win" is a false positive; peeking 20 times turns 5% into ~23%.

Guardrails and the metric hierarchy

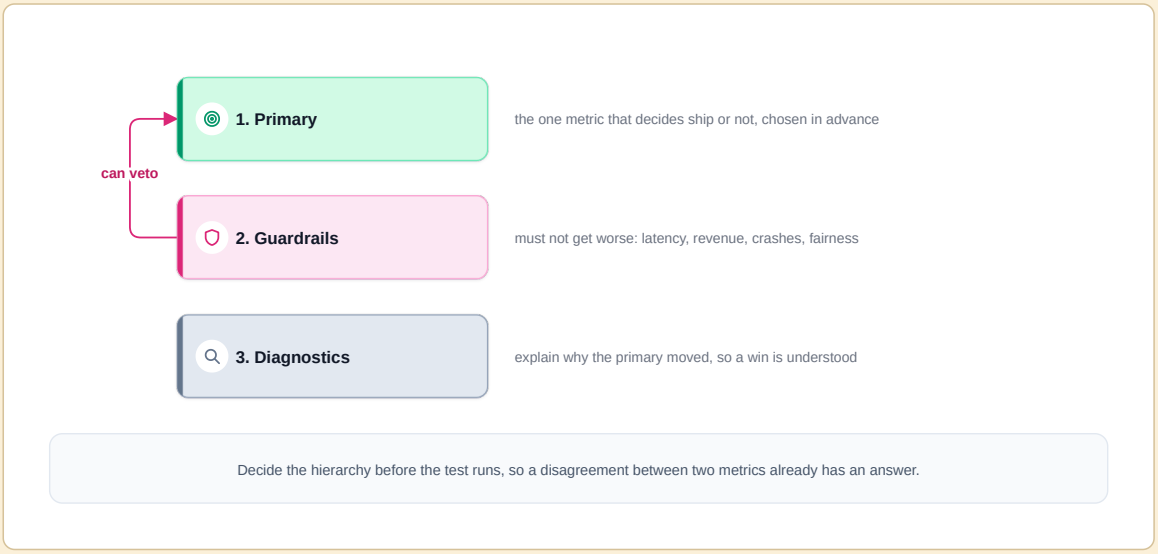


Figure — Metric tiers: primary, guardrails, diagnostics (course diagram)

1. **Primary** — decides ship/no-ship (e.g. watch time per session).
2. **Guardrails** — must not degrade: latency, crash rate, unsubscribes, revenue, fairness across segments. A primary win doesn't override a breached guardrail.
3. **Diagnostics** — explain *why* the primary moved.

Fixing the hierarchy beforehand settles disagreements in advance. Also check **sample ratio mismatch** (a 50/50 split arriving as 52/48 = broken assignment or logging).

Beyond the simple A/B

Variant	Use when	How
Interleaving	Ranking, low traffic	Merge two rankers' results into one list per user (e.g. team-draft); see whose items get clicked. Much more sensitive, but measures preference, not long-term impact
Switchback	Marketplaces with network effects (dispatch, surge pricing)	Flip the whole region between variants over time windows, because treating some riders affects the shared drivers
Multi-armed bandits	Ongoing selection, reward during the test	Shift traffic toward the winner as evidence accrues (Thompson, UCB); analysis differs from fixed splits
Holdback	Durable long-term effect	Keep a small slice on the old experience for months

Real-world examples

Microsoft (Bing/ExP), Netflix, Booking.com, LinkedIn and Airbnb run thousands of experiments a year; Netflix and Airbnb have published on interleaving, DoorDash and Lyft on switchbacks; CUPED originated at Microsoft (Deng et al., 2013).

In the interview

> **30-second answer**: *"Offline filters candidates; shadow catches serving problems; then an A/B test with a predeclared unit, primary metric and guardrails, sample size from MDE and power, whole weekly cycles, and a fixed-horizon or sequential stopping rule — then a gradual rollout with rollback conditions."*

- **"Very little traffic?"** → More sensitive metrics, interleaving, longer runs, variance reduction, bolder changes — and admit some rare-event effects can't be A/B tested; rely on shadow plus careful ramps.
- **"Improved for a month, then reverted."** → Novelty, seasonality, population change or data issues; run past the novelty window, keep a long-term holdback.

TL;DR

Offline filters, online decides: right unit, one primary metric, vetoing guardrails, powered sample, whole weeks, no peeking; interleaving, switchbacks, bandits, holdbacks for special cases.

16.10 MLOps: the glue that makes it repeatable

What it is

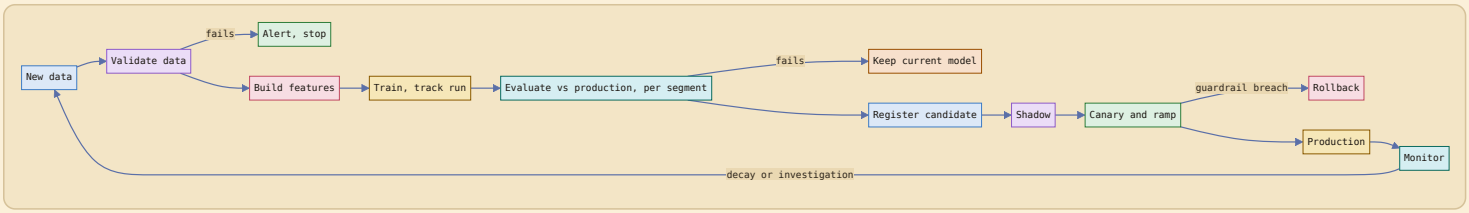
MLOps applies **DevOps** practices — version control, automated testing, **CI/CD** (continuous integration: test every change automatically; continuous delivery/deployment: release it automatically), monitoring — to ML, where the "code" includes **data** and **models**. Google's maturity levels:

Level	What's automated	Typical symptom
0 — Manual	Nothing: notebooks, hand-exported models, rare releases	"Only Alice can retrain it"
1 — ML pipeline automation	Training runs automatically on new data (**continuous training**, CT), with data & model validation and a registry	Models refresh; pipeline code still deployed by hand
2 — CI/CD pipeline automation	Pipeline code itself built, tested, deployed automatically	Commit to production safely

The components

Component	Job	Examples
Experiment tracking	Log each run's code version, params, **data hash** (fingerprint of the exact data), metrics, artifacts — reproducible (same code + data → same model) and comparable	MLflow Tracking, Weights & Biases, Neptune
Data & model versioning	Snapshot datasets and link them to models (lineage)	DVC, lakeFS, Delta Lake time travel
Data validation	Schema, ranges, null rates, distributions **before** training and at serving; fail the pipeline on bad data	TFDV, Great Expectations, Pandera
Model registry	Versions, stages (candidate/staging/production/archived), metrics, approvals, rollback target	MLflow Model Registry, SageMaker, Vertex AI
Orchestration	**DAG** (steps with dependencies, no loops): ingest → validate → features → train → evaluate → register	Airflow, Kubeflow Pipelines, Dagster, Metaflow, TFX
CI for ML	Feature-code unit tests, training–serving consistency tests, a "trains and beats baseline" **smoke test**, validation gate	GitHub Actions + pytest, etc.
CD for ML	Package model + serving code, deploy via shadow → canary → ramp, auto-rollback on guardrails	KServe, Seldon, BentoML, SageMaker endpoints
Monitoring	16.8's four layers, feeding CT	Evidently, Arize, WhyLabs, Prometheus/Grafana

Rules of ML **#5: test the infrastructure independently from the ML** — push a trivial model through and confirm identical offline and online predictions before trusting any accuracy number. Google's "ML Test Score" paper (Breck et al., 2017) is a checklist for this.



Code — a toy end-to-end loop (`code/mlops_pipeline.py`)

Data validation, tracked runs with data hashes, a staged registry, a per-segment promotion gate, shadow comparison, a canary with automatic rollback, and a skew check — ~170 lines of plain Python.

```
def gate(cand, prod, tolerance=0.01):
    c, p = cand["metrics"], prod["metrics"]
    if c["mae"] >= p["mae"]:
        return False, f"overall MAE {c['mae']:.2f} not better than {p['mae']:.2f}"
    for k in ("mae_mature", "mae_new_city"):                    # no segment may regress
        if c[k] > p[k] * (1 + tolerance):
            return False, f"segment {k} regressed {p[k]:.2f} -> {c[k]:.2f}"
    return True, f"MAE {p['mae']:.2f} -> {c['mae']:.2f}, no segment regressed"

def canary(cand_model, prod_model, live, ylive, cand_features=None):
    for share in (0.01, 0.05, 0.25, 0.50):
        # ... compare canary MAE with production MAE on the traffic slice
        if mae_c > mae_p * 1.05:
            return False            # ROLLBACK
    return True
```

```
1) validation of a clean batch: OK
   validation of a broken batch: ['distance_km: 99.9% out of range [0.0, 80.0]', 'hour: 2.0% nulls']

2-3) train + track + register; v1 (ridge) bootstraps production
   v1 ridge            MAE 2.94 (mature 2.84, new city 3.79)
   v2 gbdt             MAE 2.43 (mature 2.43, new city 2.40)
   v3 gbdt_no_new_city MAE 2.58 (mature 2.43, new city 3.85)
   tracked run files: ['run_1.json', 'run_2.json', 'run_3.json']

4) promotion gate vs production (v1)
   v2 gbdt             -> PASS: MAE 2.94 -> 2.43, no segment regressed
   v3 gbdt_no_new_city -> FAIL: segment mae_new_city regressed 3.79 -> 3.85

5) shadow v2: 20,000 live requests, mean |prod - cand| = 1.49 min, max = 27.3 min (logged, never served)

6) canary v2, first attempt (serving container has the UTC-hour bug):
   canary   1%: MAE 2.95 vs prod 2.93 -> continue ramp
   canary   5%: MAE 3.47 vs prod 2.92 -> ROLLBACK (guardrail: >5% worse)
7) skew check on logged features: 'hour' differs from the training pipeline on 100% of rows -> fix the serving code path
   canary v2, after the fix:
   canary   1%: MAE 2.20 vs prod 2.93 -> continue ramp
   canary   5%: MAE 2.48 vs prod 2.93 -> continue ramp
   canary  25%: MAE 2.42 vs prod 2.93 -> continue ramp
   canary  50%: MAE 2.46 vs prod 2.92 -> continue ramp
   registry: [(1, 'ridge', 'archived'), (2, 'gbdt', 'production'), (3, 'gbdt_no_new_city', 'candidate')]
```

Validation stops a km→m unit bug; v3 beats v1 on average but is blocked for regressing on new cities; the canary catches a **serving-side** bug (hour in UTC inside the serving **container**) invisible offline — 1% was too noisy, 5% caught it; comparing logged serving features with training finds the cause; after the fix v2 ramps and is promoted.

Common mistakes

- Metrics tracked without the data version → unreproducible models.
- A registry without stages or owners → nobody knows what's live.
- Auto-retrain and auto-promote without gates.
- CI that never checks training and serving produce the same features.

In the interview

30-second answer: *"Every run tracked with code, data hash, params and metrics; data validated before training; candidates registered and promoted only through an evaluation gate, shadow and auto-rollback canary. The pipeline retrains on schedule or trigger (CT), and its code ships through CI/CD with training–serving consistency tests."*

TL;DR

Track runs, version data and models, validate, staged registry, orchestrate, CI/CD/CT with gates, shadow → canary → rollback, monitor, loop.

16.11 Capstone — an ML platform for ETA prediction

The exercise

> "Show a delivery-time estimate at checkout and keep it accurate as the business grows. 500K orders/day, peak 30 orders/s at dinner. The estimate appears as soon as the customer opens checkout."

(Numbers illustrative.) Try it with the Chapter 15 framework first.

Steps 2–4: success, data, baseline (first ~12 minutes)

- **Metrics**: MAE vs real delivery time, **p90 absolute error** (customers remember the 25-minute promise that took 70), and **promise-kept rate** — share arriving inside the window shown. The business may trade a slightly longer displayed time for promises that almost never break.
- **Latency**: assume < 100 ms. 30 *orders*/s ≠ checkout views/s — estimate views, abandoned sessions and retries separately.
- **Labels**: each completed order reveals its time within ~an hour — excellent. Validate timestamps, cancellations, missing events, the checkout-view vs order-time gap.
- **Sources**: order event stream, courier location pings, restaurant accept/finish times, past weather.
- **Features**: entity (restaurant median prep, courier vehicle); streaming (restaurant queue now, courier supply in zone, prep-time drift last hour); request context (route distance, cart size, time, rain).
- **Baseline**: per-restaurant average + distance term — ships fast, tests logging and evaluation. **Upgrade**: GBDT on all features, benchmarked on CPU.

Step 5: architecture (minutes 12–30)

Online inference: nobody knows the kitchen queue or route until checkout.

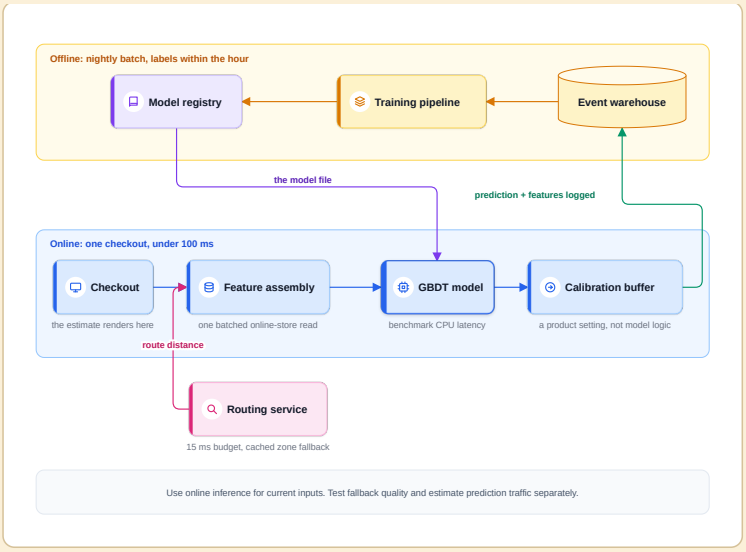
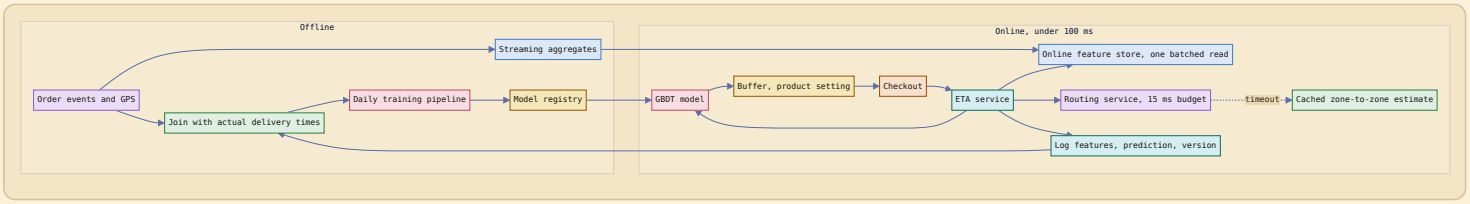


Figure — ETA platform end to end (course diagram)



Say out loud:

1. **Routing is where latency goes wrong** — an external dependency on every checkout. Budget 15 ms with a cached zone-to-zone fallback; train/evaluate with that approximation so degradation is measured.
2. **Log predictions with the exact features used** — kills skew; join outcomes on completion, validating timestamps, ID matches, duplicates, late events.
3. **A buffer layer after the model** — displayed ETA = prediction + buffer. Bigger buffer keeps more promises but makes the restaurant look slower than rivals; it's a **product** setting (Chapter 15's buffer curve).

Steps 6–7: loop, failures, cost (minutes 30–40)

- **Monitoring**: freshness & nulls → input distributions (a new city skews distances) → prediction distribution (a rising mean promised time is business news) → MAE on completed orders by prediction hour, **per city, restaurant type, weather**.
- **Retraining**: daily to start, checked against data maturity, measured decay and cost; evaluate **per segment** before promotion — a better average can hide a worse Manhattan dinner rush.
- **Failures and fallbacks** (rehearsed, never improvised):

Failure	Fallback
Routing service down	Cached zone estimates + wider buffer
Feature store stale	Trained-in defaults; freshness monitor fires
Model service down	Warm, versioned per-restaurant heuristic table; never block checkout — show the fallback or "estimate unavailable"

- **Cost**: count prediction requests separately from orders; benchmark CPU, replicas, storage, stream processing, routing calls. For a small GBDT, features and routing may cost more than inference — the opposite of LLM systems.

What each level sounds like

- **Mid**: right components with prompting (GBDT, feature store, daily retrain, dashboard) but gaps between them — features not logged, routing unbudgeted, MAE only global.
- **Senior**: unprompted label timing and validation, features logged with predictions, routing budget + trained-in fallback, per-segment promotion via shadow and canary.
- **Staff**: separates model from promise (business owns the buffer), starts with the baseline to prove logging and evaluation, compares inference vs pipeline vs routing cost, builds a platform for future models.

Follow-ups to drill

- **p99 creeps past 100 ms; profiling blames feature reads** → one batched read, in-process cache for hot restaurants, defaults for the slowest aggregates, drop features costing more ms than they add accuracy.
- **ETAs on the restaurant list (100× checkout traffic)** → routing load breaks latency and cost budgets. Serve browsing from batch estimates per zone and restaurant; keep the live path for checkout, where the figure becomes a commitment.
- **New city, zero history** → distance baseline + **priors** (starting estimates) from the most similar city; monitor as its own segment; labels arrive within hours, so retrains fold it in within days.
- **A storm doubles delivery times tonight** → input drift (weather), then prediction drift, then hourly MAE fire; training data may hold no storm this severe. Fastest lever: widen the buffer (product decision) while a triggered retrain absorbs tonight's labels.

Same structure, different problem (practise 40 minutes each, out loud)

- **Delivery date on a retailer's product page** → labels take days, monitoring slows, buffer matters more; batch per zone × warehouse, online only at checkout.
- **Freight ETA across a country** → multi-day horizons, sparse GPS, thousands (not millions) of predictions; more label quality, per-segment accuracy, dispatcher overrides.
- **Subscription churn** → "churned" is known 30 days later; reshapes training windows, monitoring and retraining.
- **Ride-hailing price estimate** → surge makes freshness paramount; seconds-level aggregates; routing on every render. For each: when labels arrive, what happens with no prediction, which component dominates cost.

TL;DR

Per-request inference (live queue, route); labels within the hour; routing budgeted, cached, its absence rehearsed in training; features logged with predictions; buffer owned by product; per-segment monitoring and promotion; features and external calls outweigh small-model inference cost.

Chapter TL;DR

- **Five pieces, two planes**; models flow to serving, logs flow back.
- **Skew**: one set of feature definitions, as-of joins, train on logged serving features (Rules of ML #29, #32, #37).
- **Labels** have a source, cost and delay; downsample then recalibrate ($p = wq / (wq + 1 - q)$); split by time.
- **Features**: entity / aggregates / context / cross; batch, streaming, request-time; feature store = offline history + online latest.
- **Inference**: request-time inputs → online; stable and reused → batch; events → streaming; big catalogs → precompute-then-rerank.
- **Serving**: stateless, budgeted, CPU first, shadow → canary → ramp, explicit fallback.
- **Offline metrics**: operating point + workload, owner-chosen thresholds, cohorts, calibration, always vs baselines.
- **Monitoring**: health → inputs → predictions → outcomes; investigate before retraining; watch feedback loops.
- **A/B tests**: predeclared unit and metrics, powered sample, whole weeks, no peeking.
- **MLOps**: tracking, versioning, validation, registry, CI/CD/CT, gated promotion — every time.

Chapter 17 — ML Case Studies I: Recommendations, Feed Ranking, Search

Chapter 15 gave the seven-step framework and Chapter 16 the production pieces. This chapter and the next apply both to the questions interviewers actually ask — recommendations, feeds, search, ads, fraud, moderation, "people you may know" — each teaching a different lesson about data, serving and evaluation. Learn them and a new question ("design job matching") becomes a matter of saying which parts transfer and which requirement is new.

This chapter covers the four **retrieval-and-ranking** designs: video recommendations (multi-source retrieval, two-tower + ANN, cold start, position bias, the proxy trap), social feed ranking (social candidates, multi-task heads, a product-owned value formula, integrity layer), e-commerce search (query understanding, hybrid lexical + semantic retrieval, L1/L2 rankers, NDCG, IPW) and text-to-video search (indexing speech with **ASR** and pictures with **CLIP**-style frame embeddings, fusing channels, segment vectors for moments) — each with a small runnable toy on synthetic data. Chapter 18 covers ads, fraud, harmful content, people-you-may-know, music, visual search, privacy blurring, events and similar listings, ending with a thirteen-way comparison.

Code is in `code/`; outputs are real.

17.0 The case-study template

What it is

One fixed order of questions for every design in Chapters 17–18 — the Chapter 15.3 framework split so the ML parts get their own slot:

#	Section	The question it answers	Ch. 15 step
1	Clarify requirements	Product surface, scale, latency, out of scope?	1–2
2	Frame as an ML task	Input, output, ML category, objective?	1
3	Data and labels	Which logs and catalogs, where labels come from, how biased/late?	3
4	Features	Which groups, how fresh, what leakage risk?	3
5	Model: baseline → advanced	Simplest thing that works, and what's worth upgrading to?	4
6	Offline metrics	How do we pick models before exposing users?	2, 6
7	Online metrics and A/B	What decides a launch, what may never get worse (guardrails)?	2, 6
8	Serving architecture	Offline vs per request; draw the request path	5
9	Scale numbers	QPS, candidates scored per second, memory — arithmetic shown	5, 7
10	Monitoring and failure modes	What breaks silently; fallbacks?	6–7
11	Interview follow-ups	Questions you'll get, with answers	—

Each case also has a **toy**, **common mistakes** and **same design, other questions**; the chapter-end comparison table is the one-row summary of each case. Generic material is referenced, not retaught: A/B mechanics (sticky randomisation, primary metric, guardrails, novelty, interleaving, holdbacks) in Chapter 16.9; drift and monitoring in 16.8; shadow → canary → ramp in 16.6; embeddings, two-tower, ANN and ranking metrics in Chapter 13.

The shared shape: a funnel (for retrieval and ranking designs)

Too many items to score carefully per request, so a **funnel** of stages, each with fewer items and more compute per item:

1. **Retrieve** (**candidate generation**): cheap methods pick a few hundred from millions. Graded on **recall** — anything missing is lost for good, since later stages only reorder.
2. **Rank**: a richer model scores and sorts them. Graded on **top-of-list precision** — the few slots people actually look at.
3. **Re-rank / apply rules**: freshness, variety, policy, "no two ads in a row".

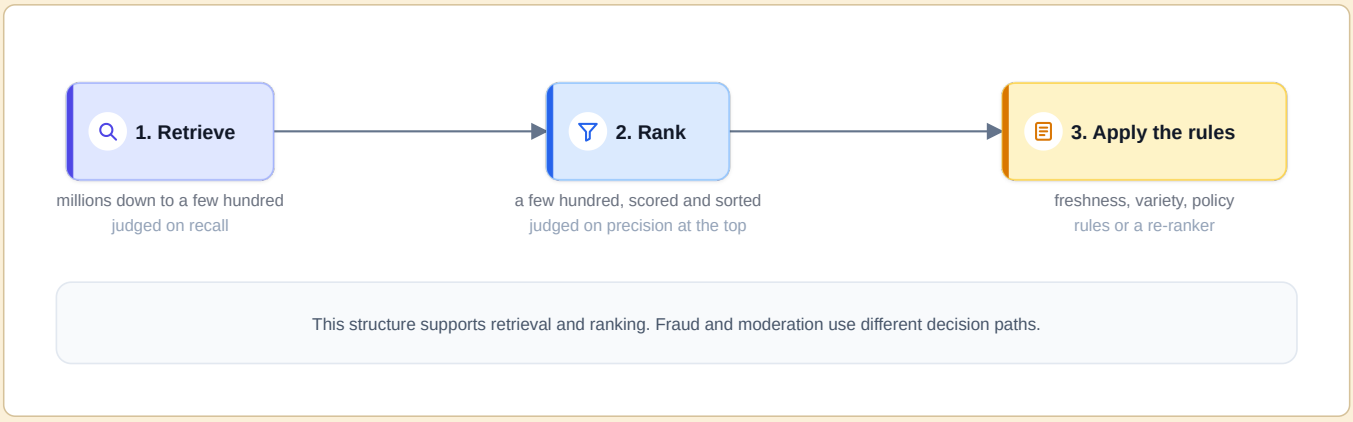


Figure — Retrieve, rank, apply rules: the funnel shared by recommendations and search (course diagram)

Not every design is a funnel: **fraud** scores one transaction at a time; **harmful content** uses a **cascade** (cheap checks on everything, expensive checks — bigger models, humans — only on what they flag); **privacy blurring** is a once-per-image batch detection pipeline (Chapter 18).

What each case study adds

Case study	New idea it teaches	Chapter
Recommendations	Multi-source retrieval, ranking, rule layer, cold start	17.1
Feed ranking	Many goals at once (like, comment, hide, report) → value formula	17.2
Search ranking	A query as input; clicks biased by position	17.3
Text-to-video search	No reliable text: index speech and pictures, fuse channels	17.4
Ad click prediction	Probabilities feed an auction, so **calibration** matters	18.1
Fraud detection	An adaptive adversary; asymmetric per-transaction costs	18.2
Harmful content	Platform scale, high stakes both ways, human review	18.3
People you may know	Graph-based candidates, two-sided acceptance	18.4
Music discovery	Sequences, sessions, exploration vs familiarity	18.5
Visual search	Image embeddings and nearest-neighbour search at scale	18.6
Privacy blurring	A miss is a privacy breach: recall per person, fail closed	18.7
Event recommendation	Every item new, perishable, tied to place and time	18.8
Similar listings	Item-to-item similarity from browsing sessions	18.9

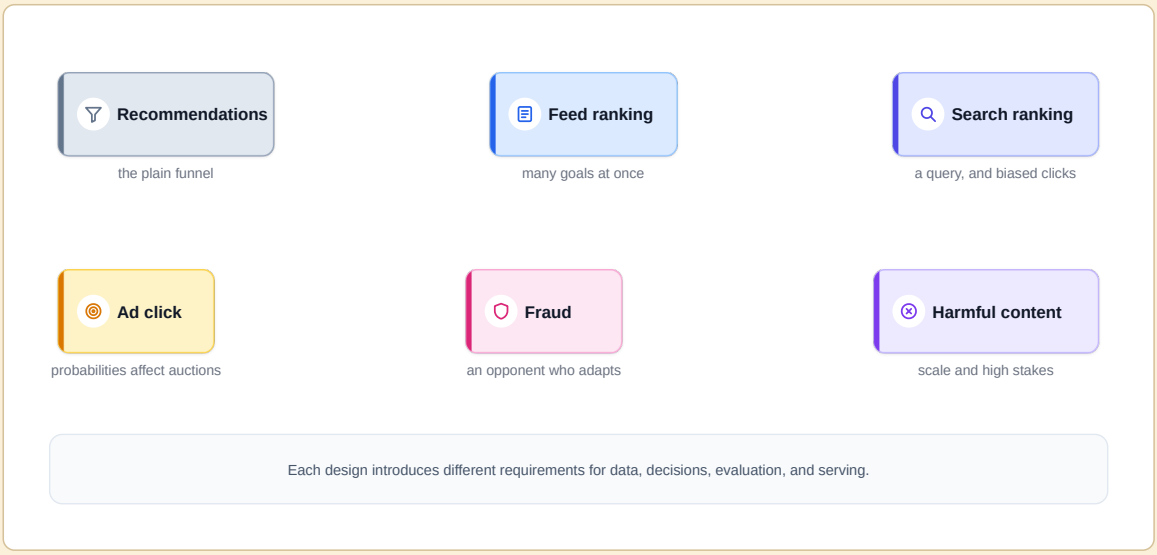


Figure — The main focus of each of the six core designs (course diagram)

Diagnosing a bad recommendation: if better items were never pulled in, it's a **retrieval failure**; if pulled in but ranked too low, a **ranking failure** (logs or experiments tell which).

17.1 Case study: video recommendations

> "Design the homepage recommendations for a video platform (think YouTube)."

1. Clarify requirements

Question	Assumption used below
Which surface?	Homepage feed only (search, "up next", notifications out of scope)
Catalog size?	50 million videos, thousands uploaded per hour
Users?	100 million daily active users (DAU)
Latency?	Whole page in about 200 ms
Page size?	About 20 videos
Business goal?	People come back and watch more

Running a careful model on 50M videos per request would blow the 200 ms and cost budgets by orders of magnitude — so say early: *"This is a prediction problem with a ranking-funnel shape."*

2. Frame it as an ML task

- **Input**: user (history, profile), context (time, device, location), candidate video.
- **Output**: a score per (user, video) — P(click) and expected watch time, or P(watch ≥ 30 s).

- **Category**: retrieval + **learning-to-rank**.
- **Objective**: predicted engagement per **impression** (one item shown once). Engagement arrives in minutes; the true goal (daily return rate, long-term **retention**) takes weeks.

Proxy trap: clicks alone reward misleading thumbnails; watch time alone rewards needlessly long videos. Combine several predicted outcomes and check the long-term metric in experiments.

3. Data and labels

1. **Interaction logs** — the label source. Log every impression with user and item IDs, timestamp, position, model version, ranking features; join outcomes (click, watch seconds, like, "not interested") within an **observation window** (e.g. 24 h). Respect privacy and retention rules.
2. **Catalog content** — title, tags, description, channel, audio/visual embeddings.
3. **Signup signals** — language, country, declared interests, device.

An engaged impression is a positive; a skipped one only a *weak* negative (maybe never noticed). Name two flaws unprompted:

- **Exposure bias** — outcomes exist only for items shown; the model may generalise to unshown items, but they're never measured.
- **Position bias** — slot 1 gets more clicks than slot 15 for the same item; clicks mix relevance with placement (fixes in section 10 and 17.3).

Content and signup data exist for what logs can't cover: brand-new users and videos.

4. Features

Group	Examples	Freshness
User	Watch history (IDs + embeddings), subscriptions, language, country	Nightly + in-session updates
Short-term user state	Last 10 videos this session, topics skipped	Seconds (streaming)
Item	Channel, topic, length, upload age, embedding, CTR and watch-rate aggregates	Stable fields nightly; counts every few minutes
User × item (cross)	Watched this channel before? Item embedding vs recent watches	Request time
Context	Device, hour, weekday, network speed	Request time

Leakage: any feature using data from after the impression (total watch count including the watch being predicted) — use point-in-time-correct joins (Chapter 16).

5. Model: baseline → advanced

Baseline: "most popular in your region + newest from your subscriptions" — ships in a week, tests logging, gives a number to beat.

Candidate generation — several cheap sources in parallel, merged:

1. **Two-tower embeddings** (collaborative filtering) — the main source; user and item towers trained so a user's vector is near engaged items and away from sampled negatives; relevance = dot product; items precomputed, only the user vector computed per request (§13.7).
2. **Co-occurrence** (viewers of X went on to watch Y) — item-to-item lists precomputed offline, looked up from recent watches. Cheap, explainable, surprisingly strong (Amazon's item-to-item CF, Linden, Smith & York, 2003).
3. **Non-personalised** — trending, fresh uploads, popular in region; serve new users and keep the page fresh.

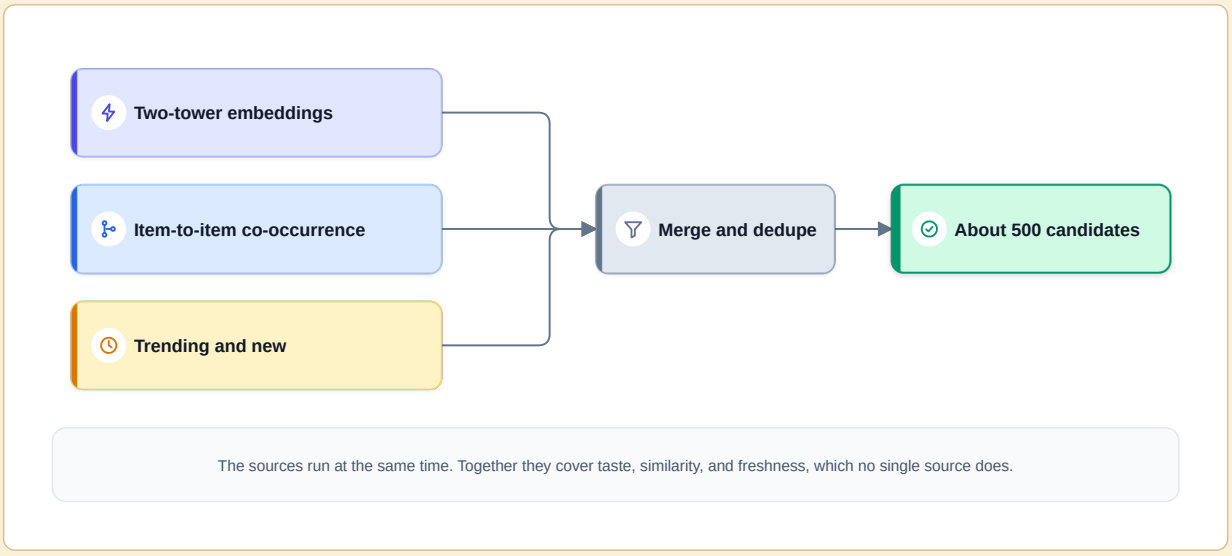


Figure — Three cheap candidate sources running at the same time (course diagram)

Ranker — ~500 candidates, so rich features and a stronger model: **GBDT** first, then a deep **multi-task** network predicting click, watch time, like. Most modelling effort goes here. YouTube's published systems follow this split: candidate-generation and ranking networks (Covington et al., 2016), later a multi-task ranker with a small "shallow tower" modelling position bias (Zhao et al., 2019).

6. Offline metrics

- **Retrieval**: **recall@K** — of videos watched in a later held-out week, what share appear among the K candidates?
- **Ranking**: **AUC** or a ranking loss; **NDCG@k** (worked example in 17.3) with graded outcomes.
- Always a **time-based split** (train weeks 1–4, test week 5). Offline metrics select candidates for an online test; they don't decide launches.

7. Online metrics and A/B testing

- **Primary**: watch time per user, daily return rate over a long enough window.
- **Guardrails**: "not interested"/dislike rate, policy-violating impressions, latency, creator-side diversity.
- User-level A/B (16.9), long enough to see return behaviour, not just first-week **novelty**.

8. Serving architecture

1. **Candidate generation: 50M → ~500.** Item vectors precomputed into an **ANN** index (e.g. **HNSW**, Malkov & Yashunin, 2016). Per request: fetch the user vector, pull the nearest few hundred in a few ms, merge with co-occurrence and trending. ANN trades a little recall for huge speed — measure that loss and pick index settings (memory, update speed, latency) from benchmarks.
2. **Ranking: ~500 → ~50.** All candidates in one **batch** model call; treat 20–40 ms as illustrative and verify under concurrent load.
3. **Re-ranking: ~50 → ~20 on screen.** Cap slots per channel, lift recent uploads, policy demotions, drop just-seen items, editor-pinned slots. Mandatory rules live **outside** the model (policy changes need no retraining); re-check fast-changing restrictions just before display.

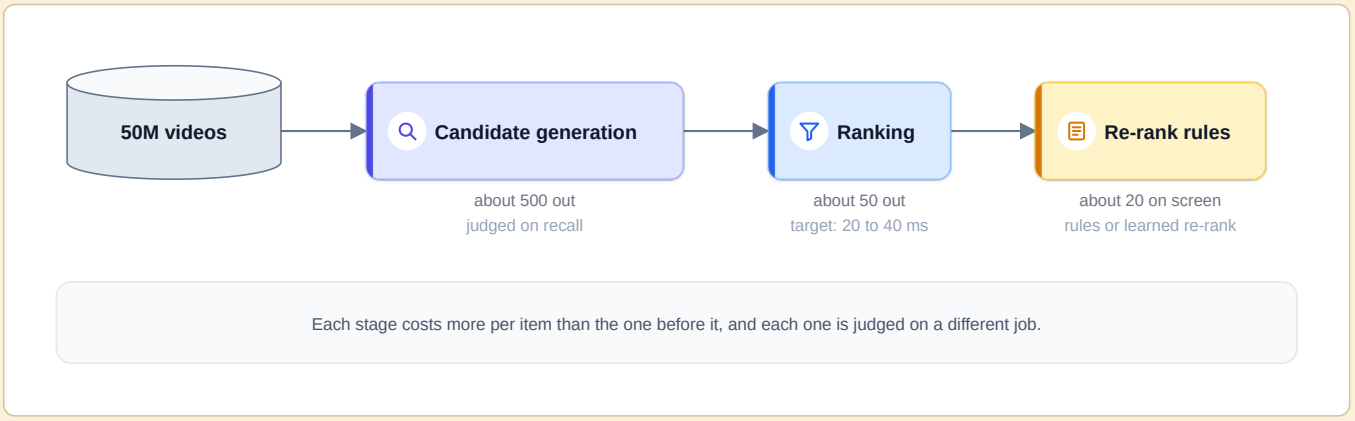


Figure — Example candidate counts and latency targets for the recommendation pipeline (course diagram)



Cold start (raise both kinds yourself):

- **New user**: popular overall and in region, signup signals (language, interests, platform), and fast first-session learning — weight early clicks heavily, update session state in seconds.
- **New video**: **content features** (title, tags, audio/visual embeddings) so it's retrieved by resembling established videos, plus a small controlled **exploration boost** — otherwise established videos soak up all exposure and new ones never collect data.

9. Scale numbers

Quantity	Arithmetic	Result
Homepage QPS	100M DAU × 5 loads/day ÷ 86,400 s (assumed 5 loads)	≈ 5,800 avg; ~17K at a 3× peak
Items scored by ranker	17K QPS × 500	≈ 8.7M scores/s at peak
Raw item vectors	50M × 128 dims × 4 bytes	25.6 GB (before index structure, IDs, metadata, replicas)

Heavy work (embedding and index builds, co-occurrence lists) moves offline so each request is cheap; incremental updates skip unchanged items.

10. Monitoring and failure modes

Update loops at three speeds (validate each against measured freshness needs): ranker retrained daily; embeddings and ANN index rebuilt nightly with incremental inserts for uploads; short-term user state updated in-session via streaming.

Training traps:

- **Position bias** → model *examination* separately from relevance (a position tower used only in training), or **inverse-propensity weighting (IPW)**: weight each example by 1 / P(user examined that slot); validate propensities and cap large weights. Feeding slot number as a feature in training and fixing it at serving does **not** reliably remove the bias. (Toy in 17.3.)
- **Proxy trap** → multiple objectives + long-term guardrails.

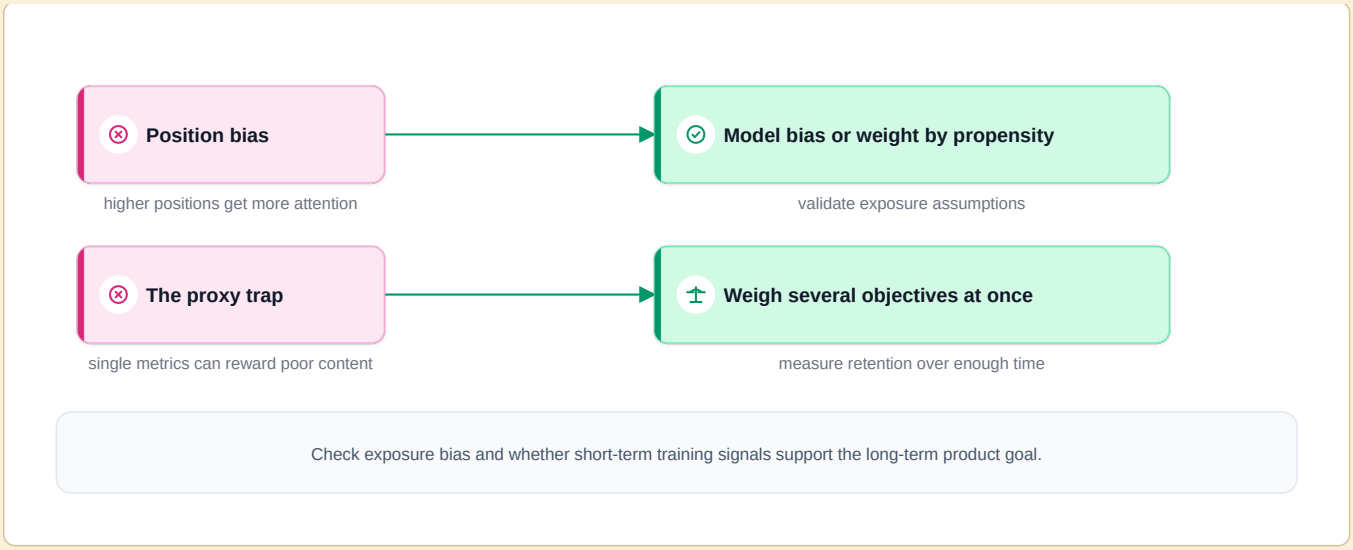


Figure — The two training traps, and the fix for each (course diagram)

Failure	Fallback	Monitor
Nightly index build fails	Last validated index up to an age limit; still enforce current availability and policy filters	Index age (severity rises with staleness)
Ranker times out	Candidate order or trending list, same required filters	Fallback rate, latency
Diversity collapse (feedback loop)	Re-rank diversity rules, exploration	Watch time by category narrowing over time

11. Interview follow-ups

- **"Why not one end-to-end model?"** → Any model good enough to order items costs too much to run 50M times per page; stages fix that, each with its own yardstick (recall for the shortlist, top precision for the order).
- **"Recall looks fine but users seem bored."** → Check re-ranking and the objective per segment: diversity may have collapsed into one taste cluster, or the proxy over-rewards short-term clicks. A shrinking weekly category mix is the warning sign.
- **"Real-time personalisation?"** → Session events stream into short-term features within seconds and nudge the user vector (cheap version: recency-weighted average of the last few videos' vectors); full retraining stays daily.
- **"Add a new candidate source?"** → Measure its **marginal** value (watched items only it finds), then **shadow**, then a small experiment slice (the toy computes this).

Toy — multi-source retrieval and marginal value (`code/rec_funnel_toy.py`)

Synthetic users and videos in a hidden 8-dimensional "taste space"; each user watches 35 videos sampled by taste + popularity; the last 5 held out. Three sources each return K = 100; then their union.

```
def two_tower(train, k, dim=16, epochs=15, lr=0.05, reg=1e-4, n_neg=5):
    """Matrix factorisation = the simplest 'two-tower': user tower and item tower are embedding tables."""
    U = 0.1 * rng.normal(size=(N_USERS, dim))
    V = 0.1 * rng.normal(size=(N_ITEMS, dim))
    # ... for each (user, watched item): 1 positive + 5 random negatives, logistic loss, SGD
            scores = V[items] @ U[u]
            g = 1 / (1 + np.exp(-scores)) - labels           # logistic-loss gradient
    # ...
        s = V @ U[u]                                          # in production: ANN search, not a full scan
        s[row] = -np.inf                                      # never re-recommend watched items

union = [np.unique(np.concatenate(x)) for x in zip(pop, co, tt)]
# marginal value: share of held-out hits that ONLY this source found
```

```
2000 users, 3000 videos, 30 past watches each, last 5 held out

candidate source          size  recall@K
popularity                 100     0.129
co-occurrence              100     0.177
two-tower (MF)             100     0.203
union of the three         223     0.306
hits found ONLY by popularity      : 0.022 of held-out watches
hits found ONLY by co-occurrence   : 0.043 of held-out watches
hits found ONLY by two-tower (MF)  : 0.096 of held-out watches
```

- ("MF" = matrix factorisation, the simplest two-tower.) Learned embeddings beat popularity and co-occurrence, but **the union** (≈223 unique) reaches clearly higher recall — sources overlap only partly, which is why real systems run several.
- **Marginal value** judges a source: popularity finds 12.9% of held-out watches but only 2.2% nobody else finds — yet it's the only source that works for brand-new users, a benefit this offline test can't see. (Synthetic numbers.)

Common mistakes

Weak	Strong
Opens by naming an architecture	Opens with 50M items vs 200 ms, which forces the funnel
Treats clicks as ground truth	Raises position bias unprompted, offers a validated correction
Hides diversity and policy inside the model	Separate rule layer product can change without retraining
Waits to be asked about cold start	Covers both kinds, with different fixes

Same design, other questions

- **"Customers also bought"** — item-to-item co-occurrence carries retrieval; target = purchase (rarer, far less ambiguous). Funnel, cold start and bias unchanged.
- **Podcast discovery** — hour-long episodes make completion signals late and weak, so audio/transcript embeddings carry more of retrieval and new-item handling becomes the main path.

17.2 Case study: social feed ranking

> "Design the ranking for Instagram's (or LinkedIn's) home feed."

1. Clarify requirements

Say in one sentence that the funnel carries over, then spend time on what a feed changes:

1. **The candidate pool is social** — mostly from who you follow, not a 50M catalog.
2. **"Good" is plural** — several positive and negative reactions weighed together.
3. **Integrity moves inside ranking** — engagement alone rewards bait and misinformation.

Question	Assumption used below
Users	500M DAU
Candidates per session	A few hundred to a few thousand posts (graph-limited), plus suggested posts
Latency	< 200 ms per ranking call
Calls per session	Several — infinite scroll re-ranks (assume 3)
Freshness	New eligible posts rankable within seconds

2. Frame it as an ML task

- **Input**: viewer, post (author, content, age, engagement so far), viewer–author relationship, context.
- **Output**: a *vector* per post: P(like), P(comment), P(reshare), E[dwell time], P(profile visit), and negatives P(hide), P("see less"), P(unfollow), P(report).
- **Category**: **multi-task** classification/regression feeding a ranking.
- **Objective**: a **value formula** combining the predictions — the formula, not any one probability, is optimised.

3. Data and labels

- **Retrieval = the social graph**: (a) **fan-in** — recent posts by followed accounts; (b) **engagement spillover** — posts your connections reacted to, from outside your follow list; (c) a slice of **out-of-network suggestions** via two-tower + ANN (17.1).
- **Labels = the event stream**: each (impression, event kind) is a row with the features the ranker saw. Log positives (like, comment, reshare, dwell) *and* negatives (hide, report).
- **Negative events are valuable but not ground truth**: a hide is far clearer than a scroll-past but context-dependent; a report needs validation, not automatic proof.
- **Survey labels**: ask a sample "was this post worth your time?" and train a satisfaction head. Check **response bias** (responders differ from non-responders), sample size and group differences.

4. Features

Group	Examples
Viewer	Interests, past engagement rates by content type, language, session depth
Author	Follower count, posting frequency, historical engagement and hide rates, integrity history
Viewer × author	Interaction history (DMs, comments, profile visits), closeness, recency
Post content	Type (photo/video/text/link), text and image embeddings, topic, integrity scores
Post engagement (streaming)	Likes/comments/hides in the last 5 min, 1 h — sparse for new posts
Session	Posts already seen (dedupe), in-session reactions

A brand-new post has no reactions, so the ranker relies on author, content and likely audience until first-hour engagement streams in.

5. Model: baseline → advanced

- **Baseline**: reverse-chronological fan-in, or one logistic regression for P(like) — the "Following" tab many apps still offer.
- **Multi-task learning (MTL)**: one network with a **shared body** (reads viewer, post, context once) and a small **head** per event. Rare events (reports, reshares) borrow statistical strength from common ones (likes). **MMoE** (multi-gate mixture of experts) helps when tasks conflict.
- **Value formula** (the product's definition of "good"):

```
score = w1·P(like) + w2·P(comment) + w3·P(reshare) + w4·E[dwell]  −  w5·P(hide) − w6·P(report)
```

Harm terms are *subtracted*. Rules:

- Product, safety and ML choose weights together — they encode policy.
- **Normalise units** — dwell is in seconds, other heads in [0, 1].
- Heads must be **calibrated** (predicted 0.1 ≈ 10% happen), else the sum is meaningless and an over-predicting head silently gains weight.
- Weights live **outside** the model: many policy changes are a formula change + A/B test, no retraining (some still need new labels or features).

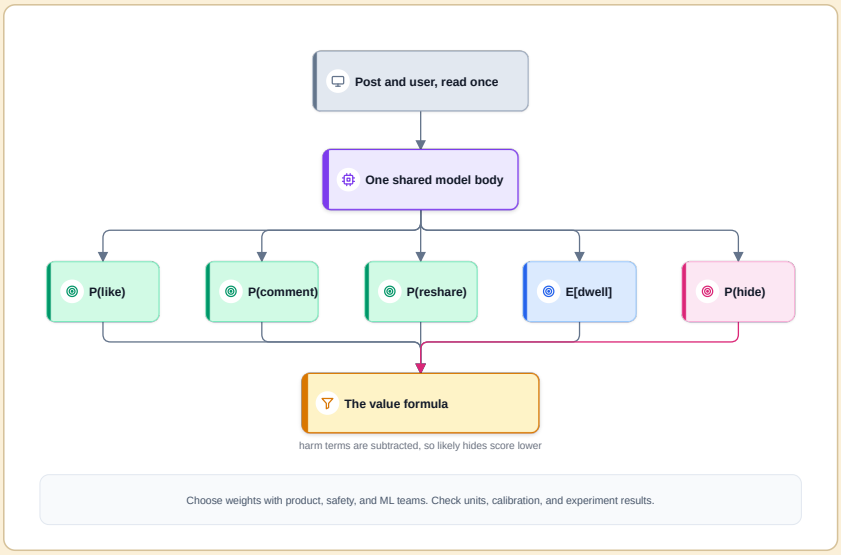


Figure — One shared body, one head per event, one value formula (course diagram)

Real-world: Facebook's 2018 "meaningful social interactions" change re-weighted toward comments and reshares among friends; Twitter's open-sourced 2023 "heavy ranker" predicted about ten engagement probabilities combined with fixed weights — e.g. +0.5 for a like, −74 for negative feedback ("show less", mute, block), −369 for a report.

6. Offline metrics

- Per-head **AUC** and **log loss**; **calibration** per head (reliability curve: predicted vs actual rate per score bucket), since the formula sums them.
- Ranking-level: offline replay of the formula — expected likes/comments/hides in the top-k per weight setting (the toy does this).
- Slice by viewer segment, content type, new vs established authors.

7. Online metrics and A/B testing

- **Primary** (decides launch): sessions per user per day, or share of "worth your time" survey answers.
- **Guardrails** (can veto alone, limits fixed in advance — 16.9): hide rate, report rate, integrity-violating impressions, creator distribution.
- Weight changes are A/B tested on the same primary + guardrails; urgent safety changes may use a faster process.

8. Serving architecture

Getting posts into candidate lists — the classic news-feed question:

- **Push (fan-out on write)**: each post written into every follower's inbox. Fast reads; a celebrity with 100M followers = 100M writes.
- **Pull (fan-out on read)**: gather followed accounts' posts at open. Cheap writes; hot accounts get huge read load.
- **Hybrid**: push for normal accounts, pull for mega-accounts. Mention it, offer to go deeper, return to the ML if declined.

Two-pass ranking: a **light ranker** (a small **distilled** model imitating the heavy one) trims ~2,000 → ~500; the **heavy multi-task ranker** scores those; the value formula orders them; then the **integrity layer**:

- demote clickbait, engagement bait, misinformation and borderline content via probabilistic classifiers;
- **per-author caps**;
- **diversity** interleaving across friends, groups, suggested content;
- durable **per-user suppression** after explicit "see less".

The classifiers feeding it are a separate **adversarial** ML system (attackers try to evade it; cf. 18.2, 18.3).

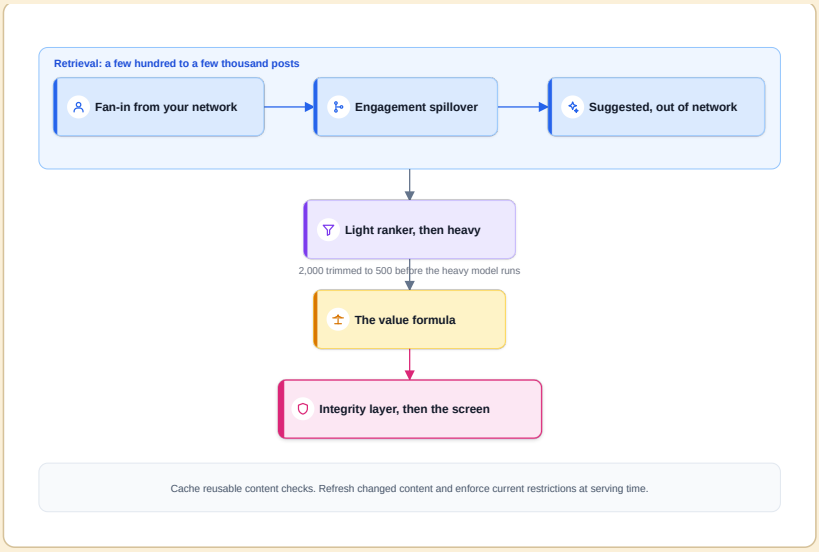
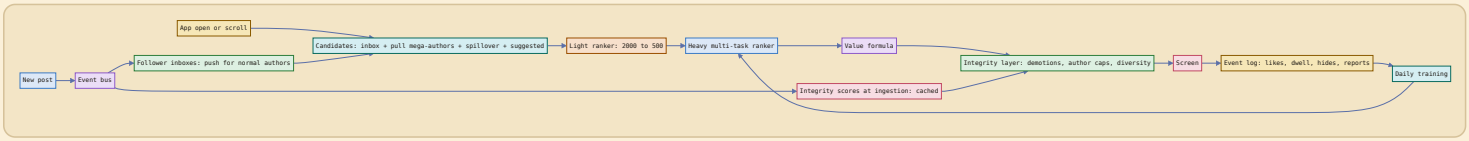


Figure — The feed serving path, from fan-in to the screen (course diagram)



9. Scale numbers

Quantity	Arithmetic	Result
Ranking calls, average	500M DAU × 5 sessions × 3 calls ÷ 86,400	≈ 86,800 /s
Peak (assume 2×)	86,800 × 2	≈ 174,000 /s
Posts scored, average	86,800 × 500	≈ 43.4M scores/s (provision for peak)

Affordable thanks to the two-pass funnel (light ranker absorbs most volume), **micro-batching**, and **item-side feature caching** (one viral post is scored in millions of feeds). Post-only integrity scores are computed at upload and cached (redone on edit or policy change); viewer-dependent checks run per request.

10. Monitoring and failure modes

Three loop speeds (set from data availability, measured quality and cost): session state in seconds, engagement aggregates in minutes, model retrained daily. Candidate freshness comes from streaming, not a nightly build.

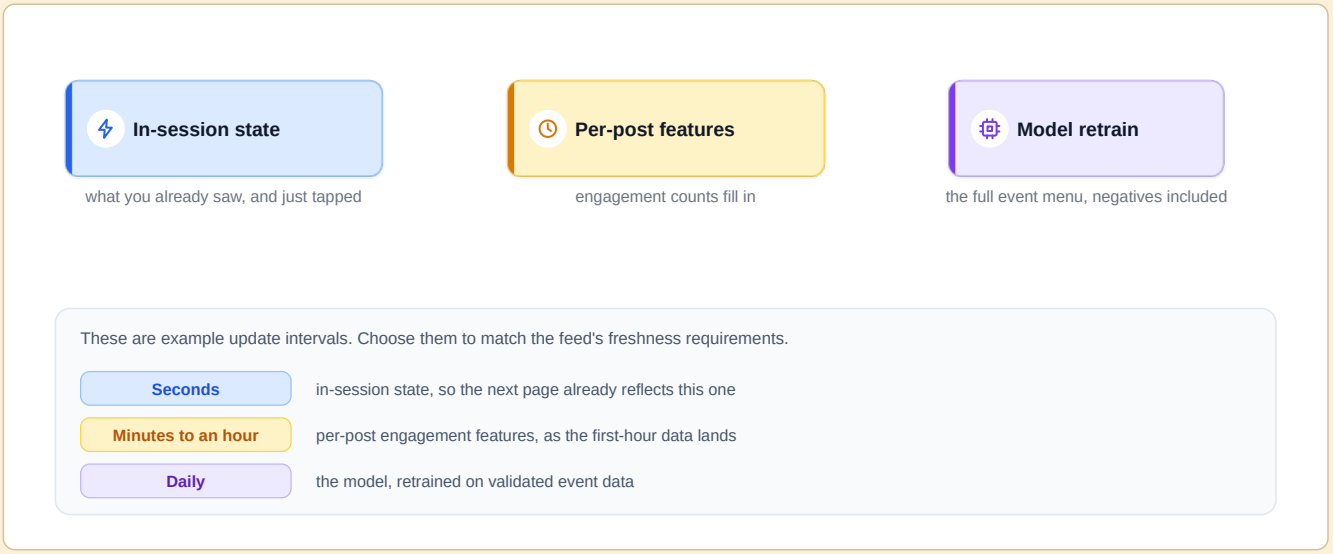


Figure — Three loops running at three different speeds (course diagram)

Failure	Response
Heavy ranker times out at peak	Serve the light ranker's order; with no ranking at all, followed posts newest-first. **Tag degraded sessions** so their engagement (produced by a fallback ranking) doesn't pollute training. Privacy, eligibility and safety checks stay on in every path — if they can't run, suppress.
Feedback loop (silent)	Feed shows what it predicts, people react to what's shown, model retrains on that — amplifying low-quality engagement. Monitor satisfaction surveys and quality metrics; keep guardrails and the integrity layer.
Calibration drift in one head	The formula silently re-weights; monitor predicted vs actual per head.
Recency mis-weighting	Breaking news must surface in seconds; an old personal update may still matter. Let the model learn recency per content type.

11. Interview follow-ups

- **"Defend ranking over chronological."** → Most users follow enough accounts that newest-first buries the few posts they'd value; test that ranking improves interaction *and* satisfaction. Concede less transparency and possible bubbles; offer a newest-first tab and "why am I seeing this".
- **"A post went up seconds ago — how is it ranked?"** → Streaming adds it to candidate pools at once; with no reactions the model uses author history, content embedding and audience **priors** (starting estimates from similar posts); a modest freshness boost buys exploration data.
- **"Who sets the weights?"** → Offline sweeps propose, A/B tests judge on primary + guardrails, product leadership signs off (weights are policy). No retraining while heads still predict the right events.

Toy — multi-task heads + value formula (`code/feed_multitask_toy.py`)

Synthetic impressions with 6 features; some posts are "bait" attracting likes *and* hides. A PyTorch shared body with three heads (like, comment, hide), trained with one **binary cross-entropy** per head, summed. 500 simulated users get 200 candidates each; compare the top 10 under three rankings.

```
class MultiTask(nn.Module):
    def __init__(self):
        super().__init__()
        self.body = nn.Sequential(nn.Linear(N_FEAT, 32), nn.ReLU(), nn.Linear(32, 16), nn.ReLU())
        self.heads = nn.ModuleList([nn.Linear(16, 1) for _ in range(3)])   # like, comment, hide

    def forward(self, x):
        h = self.body(x)                                  # shared representation, computed once
        return torch.cat([head(h) for head in self.heads], 1)   # 3 logits

# training: loss = sum(BCEWithLogits(logits[:, k], y[:, k]) for k in range(3))
formulas = {"P(like) only": np.array([1.0, 0.0, 0.0]),
            "gentle 1/2/-3": np.array([1.0, 2.0, -3.0]),
            "strong 1/4/-10": np.array([1.0, 4.0, -10.0])}
top = np.argsort(-(ph @ w))[:10]          # ph = predicted [P(like), P(comment), P(hide)]
```

```
event rates in logs  like=0.259 comment=0.077 hide=0.060
head like     AUC=0.818  mean pred=0.244 vs actual=0.252
head comment  AUC=0.782  mean pred=0.071 vs actual=0.082
head hide     AUC=0.868  mean pred=0.057 vs actual=0.061

expected events per user in the top 10 (true probabilities):
ranking           likes  comments  hides
P(like) only       8.16      1.08   2.34
gentle 1/2/-3      5.37      2.79   0.33
strong 1/4/-10     3.51      3.18   0.17
```

- P(like) alone maximises likes but fills the top 10 with bait: 2.34 expected hides per user.
- The **gentle** formula cuts hides ~86% (2.34 → 0.33) and more than doubles comments, losing about a third of likes; **strong** goes further. Which is "right" is a **product decision** settled by A/B — why weights sit outside the model.
- Mean predictions are close to actual rates per head — the calibration check to run before summing.

Common mistakes

Weak	Strong
Rebuilds the recommendation funnel for ten minutes	One sentence "it transfers", then feed-specific parts
Ranks on one P(click)	Multi-task heads + product-owned value formula
Drops hides and reports from training as noise	Trains on negative feedback, weights it by evidence, durable suppressions
Integrity only in the training objective	Separate post-formula integrity layer policy teams can tune

Same design, other questions

- **Notification ranking** — push-or-hold per candidate, users tolerate few pings; predicted annoyance (P(turns notifications off)) gets a big negative weight; the rule layer is mostly a frequency cap.
- **"Following" tab beside "For You"** — pure fan-in, near-pure recency; log both tabs identically so one training and experimentation pipeline serves both.

17.3 Case study: e-commerce search ranking

> "Design product search for an online store (think Amazon)."

1. Clarify requirements

The funnel again, reshaped by the **query**: recommendations *infer* intent from history; a query *states* it — often incompletely ("apple", "case"). Three jobs in order: **understand the query → retrieve against it → rank with it**. Say early: *"recommendation funnel plus a query-understanding stage at the front."*

Question	Assumption used below
Catalog	100M products
Traffic	10K queries/s at peak
Latency	~300 ms end to end: a few ms for query understanding, the biggest slice for the heavy ranker
Scope	Results page; autocomplete out of scope unless asked
Business	Search drives purchases directly — revenue matters as well as relevance

2. Frame it as an ML task

- **Input**: query, user, context (locale, device), candidate product.
- **Output**: relevance/purchase score per (query, product), e.g. P(purchase | query, product) or graded relevance.
- **Category**: **learning-to-rank**. Loss styles: **pointwise** (each item's label independently), **pairwise** (which of two is better — RankNet), **listwise** (optimise a whole-list metric like NDCG). **LambdaMART** — GBDTs on pairs weighted by how much swapping them changes NDCG — sits between the last two and is the classic industrial default.
- **Supporting tasks** in query understanding: spelling correction (sequence-to-sequence rewriting, or noisy-channel models of how typos corrupt words), query classification, attribute extraction (tagging spans as brand, colour, size…).

3. Data and labels

1. **Catalog**: titles, descriptions, attributes, category tree, price, ratings, stock.
2. **Search logs**: query, results with positions, clicks, add-to-carts, purchases, reformulations.
3. **Human relevance judgments**: raters grade (query, product) from *irrelevant* to *highly relevant* — expensive but unbiased; for evaluation and training heads.
4. **The query itself**: frequencies are roughly **Zipf-like** — a few very common head queries, an enormous long tail — which matters for caching.

Graded judgments are the clean labels; behaviour (click < add-to-cart < purchase) is cheap, plentiful, biased. **Position bias** is strongest here: training on raw clicks teaches the new ranker to copy the old one.

4. Features

The defining ingredient is **query–item features**:

Group	Examples
Query–item match	BM25 per field (title, brand, description), embedding similarity, attribute agreement (query "red", item red), category match
Item quality	Rating, review count, **sales velocity**, return rate, in stock, shipping speed
Query	Length, class (product / brand / help), head vs tail, predicted category
Personalisation	Brand and price-range history, past purchases (capped influence)
Context	Locale, device, time, season

5. Model: baseline → advanced

Query understanding (first, a few ms, cached for repeated queries):

1. **Normalise and spell-correct**: "iphon 15 cas" → "iphone 15 case", trained on **reformulation logs** (users retyping a query).
2. **Expand with synonyms**: "sofa" ↔ "couch", "running shoes" ↔ "sneakers" — hand lists plus **co-click mining** (queries leading to clicks on the same products), increasingly embeddings.
3. **Classify intent and extract structure**: "red nike shoes size 10" → brand=nike, colour=red, size=10, category=shoes. **Reliable, explicit** constraints become **hard filters**; uncertain ones become ranking features (or prompt clarification).

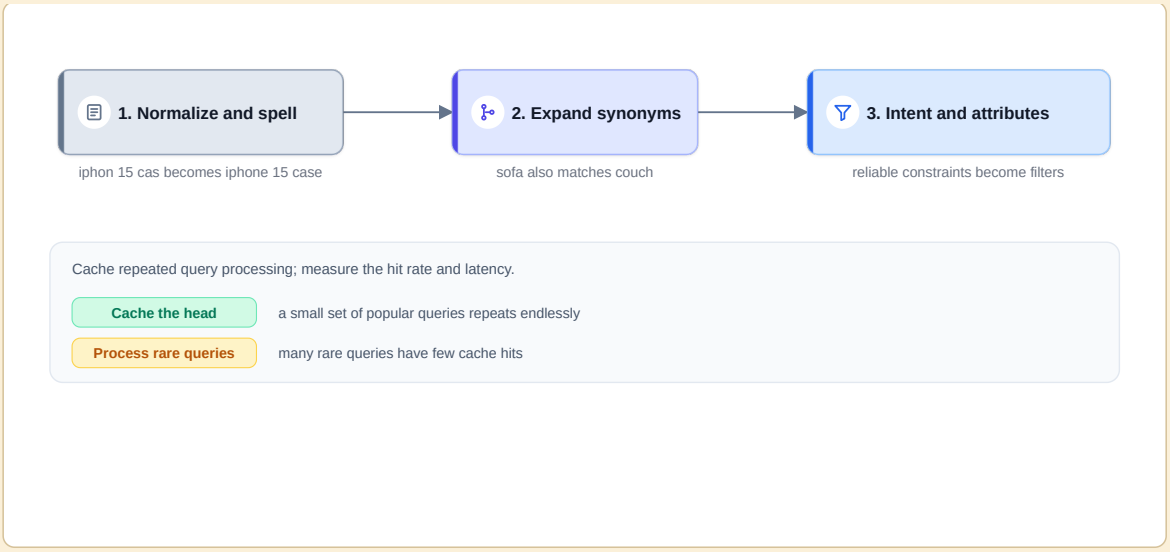


Figure — The three jobs of the query-understanding stage (course diagram)

Retrieval — two engines in parallel:

- **Lexical**: an **inverted index** (for each word, the products containing it) scored with **BM25** — rewards query words in the document, more for rare words (**IDF**, inverse document frequency), with diminishing returns for repeats and a length correction. Shines on literal hits (part numbers, brands, "X200-B"); fast, explainable. Blind spot: **vocabulary mismatch** ("something warm for the snow" misses a "thermal winter jacket").
- **Semantic**: a two-tower model trained on (query, clicked product) pairs + ANN. Captures meaning across wordings; unreliable on exact codes and can confidently return a plausible wrong item.

Ranking — two levels: a light **L1** (small GBDT on cheap features) trims the union; a heavy **L2** (LambdaMART-style GBDT or deep model with query–item features) orders the final list. Optionally a **cross-encoder** (a transformer reading query and product text *together* — slower, sharper) re-ranks the top ~20 if latency allows.

Baseline: BM25 + popularity + in-stock filter; everything above is the upgrade path.

6. Offline metrics

NDCG@k rewards relevant results near the top:

1. DCG@k = Σ over positions i = 1..k of (2^rel_i − 1) / $\log_2(i + 1)$
2. NDCG@k = DCG@k / IDCG@k (IDCG = DCG of the perfect ordering) → 0 to 1.

Worked (k = 3): relevances [3, 0, 2]: DCG = 7/1 + 0/1.585 + 3/2 = 8.5. Ideal [3, 2, 0]: IDCG = 7 + 3/1.585 = 8.89. NDCG ≈ 0.956; swapping the 0 and 2 reaches 1.0. (Relevances [0, 3] at k = 2: DCG = 7/1.585 = 4.42, IDCG = 7, NDCG ≈ 0.63.)

Also **MRR** for **navigational** queries (seeking one specific product or page), and retrieval recall. Compute on human-graded sets; behaviour labels only after bias correction.

7. Online metrics and A/B testing

- **Primary**: session success rate (search session ending in add-to-cart or purchase), revenue per search.
- **Also**: add-to-cart rate, result CTR.
- **Dissatisfaction**: **reformulation**, **abandonment**, **zero-results** rates — interpret with care (a reformulation can be exploration).
- **Guardrails**: relevance of sponsored slots, latency, zero-results by query segment.
- **Interleaving** (16.9) compares rankers more sensitively.

8. Serving architecture

Run both retrieval engines **in parallel** and take the **union** (a few hundred) — not redundancy: each catches what the other misses. The same **hybrid retrieval** (BM25 + vectors) reappears in RAG in the AI book. Each index needs a defined **update path** (both can update incrementally, implementation-dependent) — measure how long a new or changed product takes to become searchable.

The **business layer** matters most here: out-of-stock or unshippable items removed outright; margin/strategy boosts afterwards; **paid placements** only in marked slots above a minimum **relevance floor**. Money vs relevance is a decision product leadership owns, protected by guardrails.

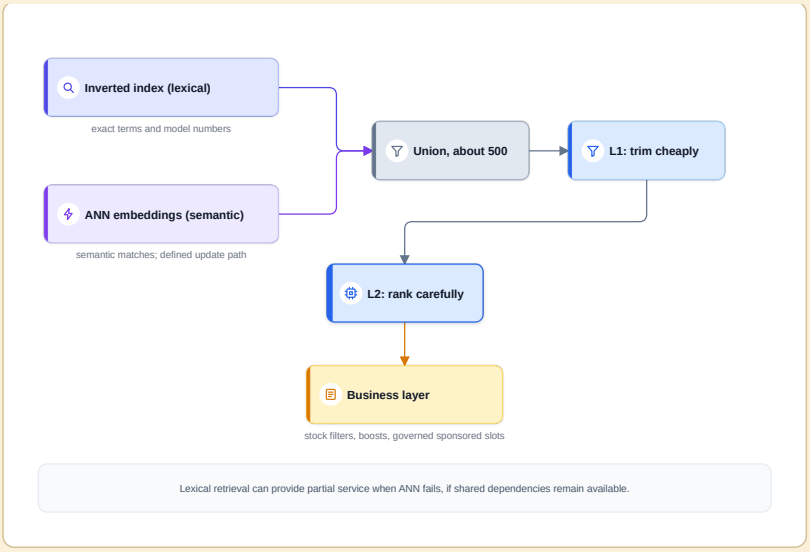
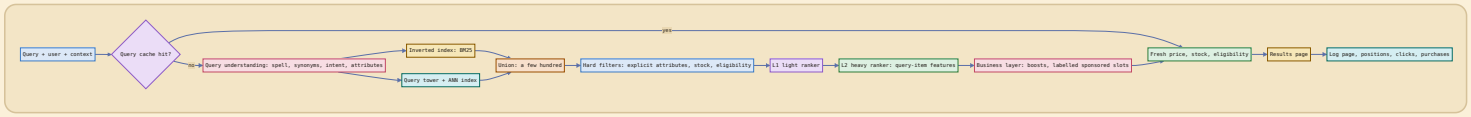


Figure — Two retrieval engines, one union, then two rankers (course diagram)



Caching done right: cache the **ranked ID list** for head queries — locale, permissions and personalisation in the **cache key** — while volatile fields (**price, stock, delivery eligibility**) are looked up live at render and checked again at checkout. Inventory events **invalidate** the filter layer. A cache holding yesterday's availability sells products you can't ship — a correctness bug while the service "works".

9. Scale numbers

Quantity	Arithmetic	Result
Peak QPS	given	10,000 /s
Cache misses (assume 40% — *measure it*)	10,000 × 0.4	4,000 /s pay full price
L2 scores	4,000 × 200 candidates after L1	800K scores/s
Per-item LLM ranking	4,000 × 200 LLM calls	800K LLM calls/s — not viable; benchmark before any LLM on the request path

The head is cheap (cached); the long tail pays for understanding, two retrievals and two rankers. LLMs help **offline**: training labels, product-text enrichment, query rewriting.

10. Monitoring and failure modes

Training loop: log every results page with features and positions → retrain on **position-corrected clicks** and **within-session preference pairs** (a purchased result ranked below a skipped one) → gate on offline NDCG and online session success. Pairs help but aren't automatically unbiased: a skip doesn't prove the user saw the item.

Position-bias fixes:

- Model examination separately from relevance (click = *examined* slot **and** found item relevant).
- **IPW**: weight each click by 1 / P(examined at its position). Estimate propensities with randomisation (swapping adjacent results for a small traffic share) or from multiple rankers' logs; validate and cap large weights. (Joachims, Swaminathan & Schnabel, 2017 — the standard unbiased learning-to-rank reference.)

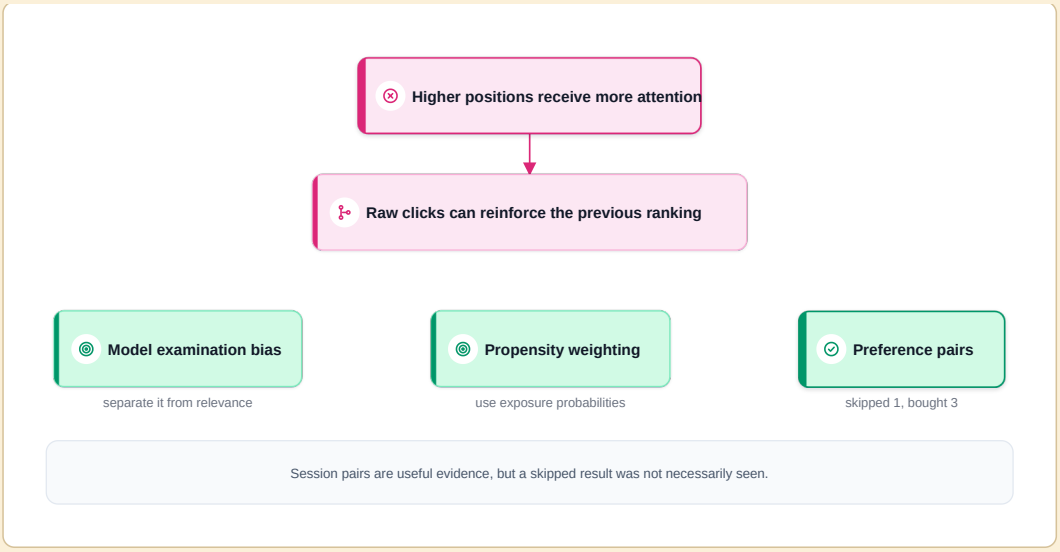


Figure — Position affects click labels; each correction needs validated assumptions (course diagram)

Monitor by query segment: reformulation and zero-results reveal what aggregate revenue hides (a broken synonym list for one category).

Failure	Fallback / effect
Embedding service / ANN index down	Lexical still serves exact and head queries (if not on the failed infrastructure); long-tail natural-language queries degrade — reformulation and zero-results are the outage dashboard
Query understanding down	Search the raw query
Stale cache	Wrong availability → refresh volatile fields at render, validate at checkout

11. Interview follow-ups

- **"A search returns nothing."** → Cause first: no inventory or vocabulary mismatch? Then fallbacks — relax *uncertain inferred* filters (never explicit ones or eligibility), spelling variants, semantic-only retry, category browsing — each logged as a health metric.
- **"How far would you personalise?"** → Personal signals reorder results that already match; never outrank what the user explicitly typed. Capped personal features in L2; check segments in experiments.
- **"Why not an LLM ranking everything?"** → At 10K QPS per-item calls blow latency and cost; LLMs offline, at most a small cross-encoder on the top 20.

Toy — position bias and IPW (`code/search_position_bias_toy.py`)

Each synthetic query has 20 products with 3 features: x0, x1 drive true relevance (0–3); x2 is a "promoted" score the **old ranker** used, with **no effect** on relevance. Users click only if they **examine** the slot (P = 1/position) **and** are attracted (likelier with relevance). A linear scorer is trained three ways; NDCG@10 vs true relevance on fresh queries.

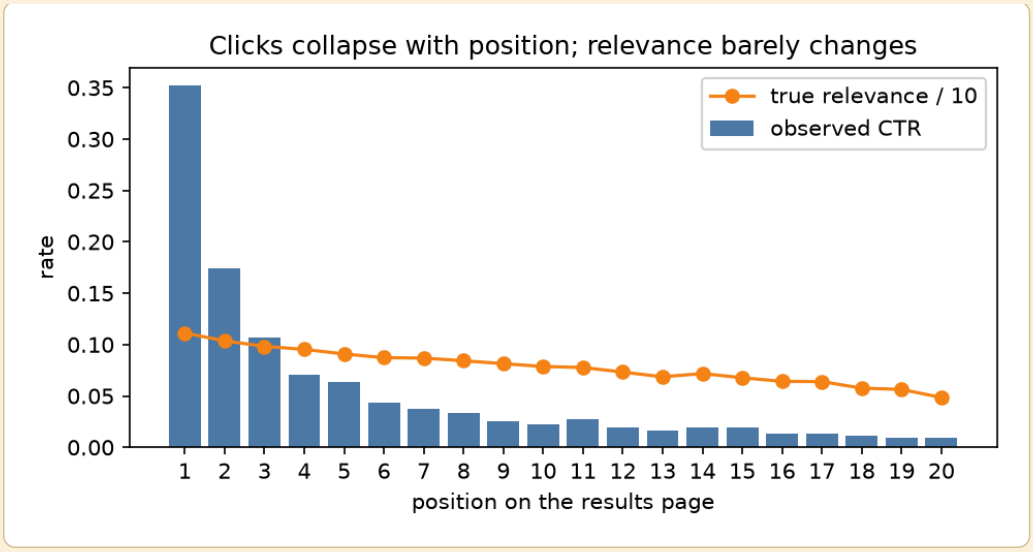
```
PROPENSITY = 1.0 / POS                                # P(examined at rank k) = 1/k

def fit_linear(x, target, weight_pos, weight_neg, epochs=300):
    """Weighted logistic loss: w_pos*log(s) + w_neg*log(1-s), full-batch Adam."""
    # ...
        loss = -(wp * logsigmoid(z) + wn * logsigmoid(-z)).mean()

ipw = clicks / np.clip(prop, 0.1, None)                  # clip: cap weights at 10x
models = {
    "naive clicks": fit_linear(xs, clicks, clicks, 1 - clicks),
    "IPW clicks":   fit_linear(xs, clicks, ipw, 1 - ipw),   # unbiased pointwise loss (weights may be < 0)
    "oracle (true rel)": fit_linear(xs, None, (rs >= 2).astype(float), (rs < 2).astype(float)),
}
```

```
CTR by position (1..10): [0.352 0.174 0.107 0.071 0.064 0.044 0.038 0.033 0.026 0.022]
mean true relevance by position (1..10): [1.11 1.04 0.98 0.95 0.91 0.87 0.87 0.84 0.81 0.79]

model                 w_x0   w_x1   w_x2  NDCG@10
naive clicks          1.03   0.50   0.82    0.820
IPW clicks            1.28   0.79   0.22    0.948
oracle (true rel)     3.72   2.48   0.01    0.956
old ranker                                   0.514
saved img/search_ctr_by_position.png
```



- CTR falls 16× from position 1 to 10 while true relevance barely moves (1.11 → 0.79) — clicks mostly measure **where** an item was shown.
- **Naive** gives the irrelevant x2 a large weight (0.82) — it copied the old ranker. **IPW** shrinks it to 0.22 and reaches NDCG 0.948, near the **oracle** (0.956) trained on labels we'd never have.
- IPW trails the oracle partly because weights are clipped at 10× — a little bias for much lower **variance** — and its guarantee holds only if propensities are right.
- Why it works: with click c and propensity p, c/p has expected value P(attracted), so the loss equals, on average, the loss on true attraction. The negative weight 1 − c/p for clicks below slot 1 is fine in a custom PyTorch loss but rejected by libraries forbidding negative weights.

Common mistakes

Weak	Strong
Jumps straight to embedding retrieval	Keyword and vector retrieval side by side, naming each one's blind spot
Treats the query as an opaque string	A stage for spelling, synonyms, intent, attributes; reliable constraints as filters, uncertain as features
Trains on raw clicks	Names position bias; validated correction; session pairs as imperfect evidence
Ignores sponsored results	Revenue vs relevance as a governed trade-off with a relevance floor

Same design, other questions

- **App-store search** — small catalog, extreme head skew, exact title/brand matching dominates; success = install-and-retain.
- **Job search** — **two-sided** relevance (seeker fits job *and* employer wants seeker); qualifications as hard filters; postings expire, so freshness matters.

17.4 Case study: text-to-video search

> "Users type a query such as 'how to fold a fitted sheet' or 'goal from last night's match'. Design the search system that returns videos (think YouTube search)."

1. Clarify requirements

Recognition sentence: **"The search funnel of 17.3, but the documents are videos, and most of what a video contains isn't written down."** Titles may be empty, foreign or clickbait; content lives in **frames** and **audio**. The new work is **indexing**: turn pictures and speech into something a text query can match. Query understanding, hybrid retrieval, L1/L2 and position bias carry over.

Question	Assumption used below
Corpus	1B searchable videos; ~1M new uploads/day, average 10 minutes
Traffic	20K queries/s at peak
Latency	~300 ms for the results page
In scope	Text query → ranked videos; "jump to the moment" timestamp when one scene of a long video matches
Out of scope	Query by image or clip (18.6), autocomplete, ads
Freshness	New uploads searchable by title within minutes, by content within hours

Business goal: a **satisfied** watch — a click, 5 seconds of viewing and a return to results is a failure.

2. Frame it as an ML task

- **Input**: query, user and context (language, region, device), candidate video with all its evidence.

- **Output**: relevance per (query, video), then a ranking score that also predicts **satisfied watch** (P(watch ≥ 30 s or ≥ 50% of a short video | shown for this query)).
- **Category**: **cross-modal retrieval** (text query, picture + sound + text item) then learning-to-rank.
- **Upload-time supporting tasks**: **ASR** (automatic speech recognition: audio → timestamped transcript), **visual embedding** (frames → vectors in the text space), optionally **OCR** (optical character recognition of on-screen text — slide titles, scoreboards).

3. Data and labels

1. **Metadata**: title, description, tags, channel, language, upload time, duration, creator chapters.
2. **Derived content**: timestamped ASR transcript, OCR text, sampled frames (one per second, or one per **shot** — a continuous run from one camera cut).
3. **Search logs**: query, positions, clicks, **watch time after the click**, quick returns to results (**"pogo-sticking"**), reformulations.
4. **Human relevance ratings** on sampled pairs — the unbiased evaluation set, split by query type.

Labels:

- **Ranker**: satisfied watches = positives; short click + quick return = strong negatives; skipped = weak negatives. Position bias as in 17.3.
- **Text–video encoder**: (query, satisfyingly watched video) pairs are best; before logs, **weak pairs**: (title, video) and (transcript sentence, frames at that timestamp). Research datasets do the same: MSR-VTT pairs clips with human captions (Xu et al., 2016); HowTo100M uses narration of 1.2M instructional videos as free, noisy captions (Miech et al., 2019).

4. Features

Group	Examples
Query–text match	BM25 on title, description, tags, transcript, OCR (one score per field); text-embedding similarity
Query–visual match	Query vs video-level visual vector; **best-matching segment** score and timestamp
Channel availability	Has transcript? Title empty/very short? Speech language vs query language
Video quality	Satisfied-watch rate for similar queries, likes vs dislikes, completion, channel authority, upload age
Query	Class (navigational; informational "how to…"; fresh: news, sport), language, head vs tail
Personalisation	Subscribed channel, languages watched (capped)

Availability features matter: a ranker can't tell a bad title match from a missing title unless told it's missing (see toy).

5. Model: baseline → advanced

Baseline: BM25 over title + description + tags × popularity — fails exactly on thin or misleading text.

Upgrade 1 — index what is said. ASR on every upload; index the timestamped transcript as another field. Whisper (Radford et al., 2022, trained on 680K hours of weakly labelled audio) makes speech searchable across many languages. Big gain for tutorials and talks; nothing for music or silent footage.

Upgrade 2 — index what is shown. A **shared text–image embedding space**, CLIP-style (Radford et al., 2021: contrastive training on image–caption pairs so a caption and its picture land close — as in 18.6). Image tower encodes frames, text tower the query; similarity = dot product. From frames to a video:

- **Mean-pool** all frame vectors → one ANN entry per video; cheap, but a short scene is diluted.
- **Segment vectors** (per shot or few seconds) find moments inside long videos, at many times the index size.
- **Temporal models** (a small transformer over frames) learn which frames matter. CLIP4Clip (Luo et al., 2022): parameter-free mean-pooling is strong on small datasets; learned temporal models need more data. Fine-tune the towers contrastively (**InfoNCE**: pick the true pair out of many — 18.6) on (query, satisfied-watch) pairs once logs exist.

Fusion. Each channel (lexical, text embedding, visual embedding) returns its top few hundred. Merge with:

- **Reciprocal rank fusion** (RRF; Cormack, Clarke & Büttcher, 2009): score = Σ over channels of 1 / (k + rank), k ≈ 60 — no training, and sidesteps channels' raw scores being on different scales.
- **A learned ranker** (L1 → L2) taking every channel's score *and* the availability features. The real gain: it learns a missing transcript isn't evidence against a video.

Ranking objective: multi-task P(click) × P(satisfied watch | click) (the 17.2 value-formula idea) plus a freshness term for fresh queries.

6. Offline metrics

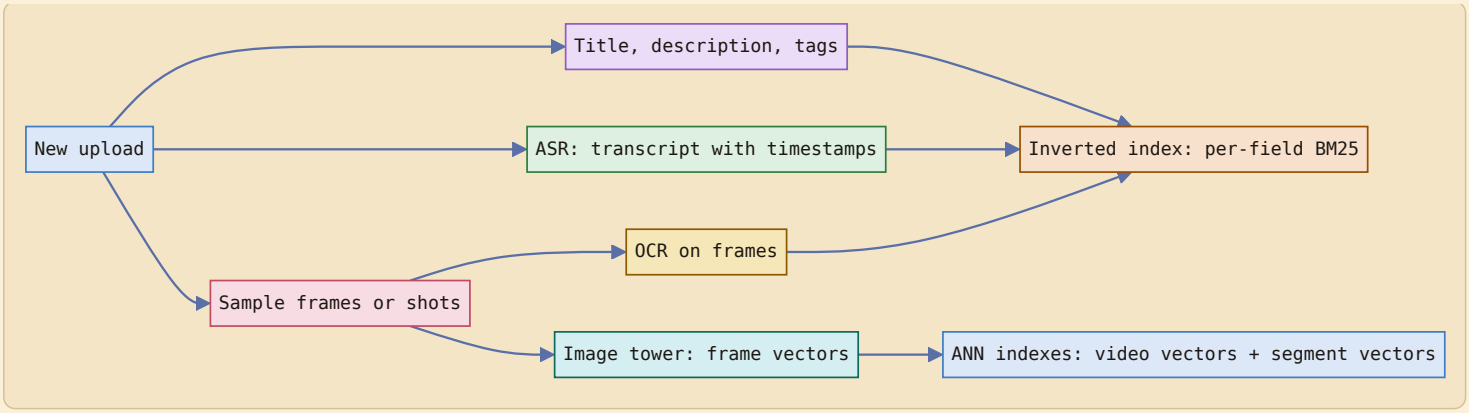
- **Retrieval**: recall@K **per channel and after fusion**; **marginal recall** per channel (17.1's test).
- **Ranking**: NDCG@10 on graded ratings; MRR for navigational queries.
- **Slices**: query class, language, no-title / no-speech videos, long videos with moment matches, fresh uploads.
- **Moment accuracy**: is the returned timestamp within a few seconds of the right scene?

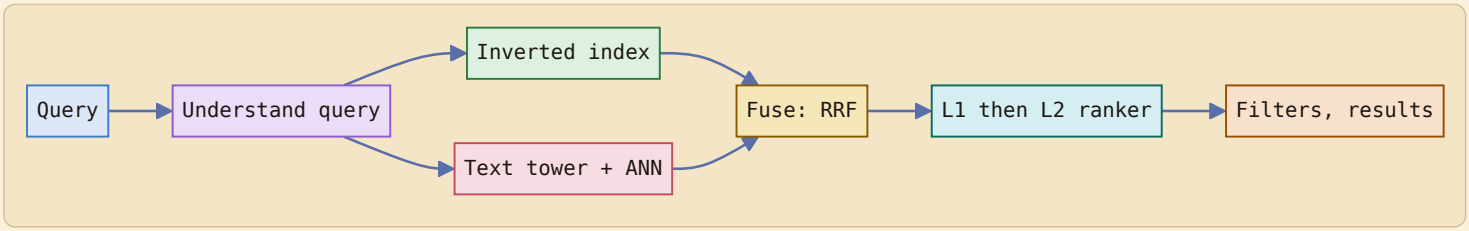
7. Online metrics and A/B testing

- **Primary**: search success rate (searches ending in a satisfied watch); **time to first satisfied watch**.
- **Dissatisfaction**: quick returns, reformulations, abandonment.
- **Guardrails**: harmful/misleading videos (18.3), latency, creator diversity, freshness on fresh queries. Interleaving (16.9) needs far fewer users.

8. Serving architecture

Two pipelines: **indexing** (once per upload) and **query** (per search).





Query understanding = spelling, language, class, freshness intent (17.3). ANN covers video- and segment-level vectors; fusion keeps ~1,000; L2 predicts click × satisfied watch; policy, safety and region filters run last; results carry a moment timestamp when a segment matched.

Staged freshness: title/description indexed within minutes; ASR and visual vectors in minutes to hours — meanwhile the ranker sees "transcript: not yet available", not "no match". Fresh queries also search a small **real-time index** of the last day's uploads.

Caching: head queries' ranked ID lists (language and region in the key; short **TTLs** for fresh queries); policy status (taken down, age-restricted, region-blocked) checked at render.

9. Scale numbers

Quantity	Arithmetic	Result
Video-level vectors	1B × 512 dims × 2 bytes (float16)	1 TB raw → **shard** across machines; with PQ (product quantisation, 18.6) at 64 B/vector, 64 GB
Segment vectors	long videos only: 100M × 30 segments × 64 B (PQ)	192 GB — 3× the video-level index, so index selectively
Frame encoding	1M uploads/day × 600 s ÷ 2 s per frame	300M frames/day ≈ 3,500/s — a steady GPU fleet
ASR	1M × 10 min	10M audio-minutes/day
Query side	20K QPS × (1 text-tower pass + 3 index lookups)	cheap vs indexing; L2 on 20K × 300 = 6M scores/s

The expensive half is **indexing**, not querying — the reverse of 17.3.

10. Monitoring and failure modes

- **Indexing lag**: upload → title-, transcript-, visually-searchable; alert on each.
- **Coverage**: share with transcript, with visual vectors, per language — a broken ASR language model silently makes a whole language unsearchable by speech.
- **Encoder upgrades**: full re-embed and **atomic swap** of model + index (18.6 rule); weeks of GPU time for 1B videos.
- **Keyword stuffing**: creators pack popular words into titles/tags; content channels plus satisfied-watch labels defend.

Failure	Fallback / effect
Visual ANN index down	Text channels still serve; visually-only-findable videos vanish — watch success on the "no title / no speech" slice
ASR backlog	Serve without transcripts; ranker treats them as missing, not no-match
Stale cache on a taken-down video	Render-time policy check drops it

11. Interview follow-ups

- **"Why not just titles and descriptions?"** → Many videos have empty, foreign or misleading titles; what a video shows and says is the reliable evidence. Show per-channel recall and marginal recall.
- **"Jump to the right moment?"** → Segment vectors and transcript timestamps; return the best segment's start; segments only for long videos.
- **"Breaking-news queries?"** → Classify as fresh, search the real-time index in parallel, freshness feature in the ranker; rely on title/ASR first since visual embedding may lag.

Toy — three evidence channels, fusion and moments (`code/video_search_toy.py`)

Each synthetic video has a hidden topic vector; assume a CLIP-style encoder maps text and frames into one space. Channels: a **title** vector (31% empty, 7% clickbait), **16 frame** vectors (very noisy, always present), a **transcript** vector (only 44% have speech). Queries are noisy views of a topic.

```
s_rrf = sum(1.0 / (K + 1 + ranks(s)) for s in (s_title, s_visual, s_trans))      # K = 60

def feats(i):   # learned fusion: channel scores gated by availability + availability flags
    has_t, has_s = (~empty).astype(float), speech.astype(float)
    return np.stack([s_title[i] * has_t, s_visual[i], np.where(speech, s_trans[i], 0) * has_s,
                     has_t, has_s], axis=1)
lr = LogisticRegression(max_iter=1000).fit(X_train, y_train)    # target vs hard + random negatives

seg = unit(frames.reshape(N, F // 2, 2, D).mean(axis=2))         # 8 segments of 2 frames
seg_scores = np.einsum("qd,nsd->qns", mq, seg).max(axis=2)       # best segment per video
```

```
5,000 videos; 31% empty titles, 7% clickbait titles, 44% with speech

main-topic queries, recall@10 on 500 test queries   (all | target title empty, n=132)
  title/tags only           0.672 | 0.000
  visual (mean of frames)   0.652 | 0.606
  transcript only           0.462 | 0.424
  average of raw scores     0.714 | 0.288
  reciprocal rank fusion    0.740 | 0.379
  learned fusion (LR)       0.896 | 0.886
  learned weights [title, visual, transcript, has_title, has_speech]: [ 9.7  8.  10.8 -5.7 -6. ]

'moment' queries (scene lasts 2 of 16 frames), recall@10
  mean-pool whole video       0.138
  max over 2-frame segments   0.978
  index size: 1 vector/video = 5,000; 8 segments/video = 40,000
```

- **Title-only search never finds an empty-title video** (0.000); the always-present visual channel still works there (0.606).
- **Untrained fusion hurts the empty-title slice**: averaging raw scores (0.288) and RRF (0.379) let a missing title drag a video below visual-only (0.606).
- **Learned fusion with availability flags** fixes it: 0.896 overall, 0.886 on empty titles. Negative weights on `has_title` / `has_speech` say "an available channel raises the bar; a missing one isn't evidence against."
- **Moments**: a 2-of-16-frame scene is diluted by mean-pooling (0.138); best of 8 segments finds it (0.978) — at 8× the index. Toy scenes are clean, so real gains are smaller; direction and cost hold.

Common mistakes

Weak	Strong
Indexes titles and descriptions only	Indexes what's said (ASR), shown (visual embeddings), written on screen (OCR), each a channel
Treats a missing channel as a low score	Availability features + learned fusion; "not yet indexed" ≠ "no match"
One vector per video for everything	Video vectors for topics, segment vectors (selectively) for moments
Optimises clicks	Satisfied watch; quick returns are strong negatives

Same design, other questions

- **Podcast or meeting search** — visual channel replaced by speaker identification; ASR quality is everything; timestamps become the product.
- **Stock-footage search** — no speech, weak titles: the visual channel carries the design; buyers want moments, so segment vectors are core.

Chapter comparison (17.1–17.4)

	Recommendations	Feed ranking	Search ranking	Text-to-video search
Task type	Retrieval + learning-to-rank	Multi-task ranking	Query understanding + learning-to-rank	Cross-modal retrieval + learning-to-rank
Candidate source	Two-tower ANN, co-occurrence, trending	Social fan-in, spillover, suggested	Inverted index (BM25) + embedding ANN	Inverted index over title/ASR/OCR + text–visual ANN (video and segment level)
Key features	Watch history, item embeddings, user×item similarity	Viewer–author closeness, post content, live engagement	Query–item match (BM25, embedding sim, attributes), item quality	Per-channel match scores, availability, satisfied-watch rate, freshness
Model	GBDT → deep multi-objective ranker	Shared-body multi-task net + value formula	L1 GBDT → L2 LambdaMART / deep; cross-encoder top-k	BM25 → + ASR → + CLIP-style encoders; RRF → learned fusion
Offline metric	Recall@K, AUC	Per-head AUC, calibration, formula replay	NDCG@k, MRR	Recall@K per channel, NDCG@10, moment accuracy
Online metric	Watch time, return rate	Sessions/user or worthwhileness + hide/report guardrails	Session success, revenue per search, reformulation	Searches ending in a satisfied watch, quick returns
Hardest challenge	Cold start + position bias	Many goals without rewarding bait	Biased clicks + revenue vs relevance	Indexing cost; missing or misleading text

The full thirteen-way table is at the end of Chapter 18.

Chapter TL;DR

- One template: requirements → ML framing → data/labels → features → baseline → advanced → offline → online → serving → scale → monitoring → follow-ups.
- Retrieval-and-ranking problems share a **funnel**: cheap high-recall retrieval, expensive high-precision ranking, rules outside the model.
- In all four, clicks are a biased proxy: log positions and features, correct for bias, let long-term online metrics with guardrails decide.

Chapter 18 — ML Case Studies II: Ads, Fraud, Moderation, PYMK, Music, Visual Search, Blurring, Events, Similar Listings

Chapter 17 worked through four retrieval-and-ranking designs. This chapter adds nine, each bringing one new pressure: money riding on every probability (**ads** — calibration, down-sampling correction, delayed conversions, pacing), an adaptive opponent (**fraud** — imbalance, velocity and graph features, rules + model, two cost thresholds, censored labels), platform-scale harm in both directions (**moderation** — risk cascade, action ladder, prevalence of violating views, reviewer capacity, reach-aware re-scoring), a two-sided graph problem (**people you may know** — friends-of-friends, Adamic-Adar, P(send) × P(accept), privacy that fails closed), two serving modes for one product (**music** — weekly batch vs online radio, audio embeddings, session context), an index that changes every minute (**visual search** — shared image-text space, contrastive training, IVF/HNSW, encoder upgrades), a detector whose misses are privacy breaches (**street-imagery blurring** — recall per person, padding, audits, fail closed), an all-new perishable catalogue (**events** — content, place, time, social features, point-in-time features) and similarity from browsing sessions (**similar listings** — skip-gram, booked-listing context, same-market negatives, cold-start averaging). Each follows the **eleven-part template** of 17.0 and ends with a runnable toy on synthetic data; a comparison table of all thirteen case studies closes the chapter. A/B and monitoring mechanics are not repeated — see Chapter 16 (16.8 monitoring, 16.9 experiments).

Code is in `code/`; every output shown is real.

18.1 Case study: ad click prediction

> "Design the system that picks which ad to show and predicts whether it will be clicked."

1. Clarify requirements

Ranking again, plus auctions (advertisers bid per slot; the system picks winner and price), budgets and billing. The key difference: **the model's output is part of a financial transaction**. In a simplified cost-per-click (**CPC**) setup ads rank by **expected value per impression = bid × pCTR** (predicted click-through rate). A wrong probability changes who wins and what the platform earns.

Question	Assumption used below
Traffic	100K ad requests/s at peak
Latency	50–100 ms for the whole auction (eligibility → retrieval → scoring → auction)
Pricing	CPC and cost-per-conversion campaigns both exist
Auction	State one (e.g. quality-weighted second-price); real auctions add **reserves** (minimum prices)
Freshness	Campaigns start, pause and change budgets all day

2. Frame it as an ML task

- **Input**: user and context (page, device, time), ad and creative, advertiser history.
- **Output**: **pCTR** = P(click | impression) and, for conversion campaigns, **pCVR** = P(conversion | click) — the purchase or sign-up the advertiser wants.
- **Category**: binary classification whose output must be a **calibrated probability**, not just a good ordering; minimise **log loss**, which rewards accurate probabilities and punishes confident wrong ones hardest.

Calibration is what this question is really about: of many impressions given pCTR = 2%, about 2% should be clicked. **AUC** checks only ordering, so a model can have excellent AUC with every probability 3× too high — tolerable in a feed, not in an auction multiplying it by money.

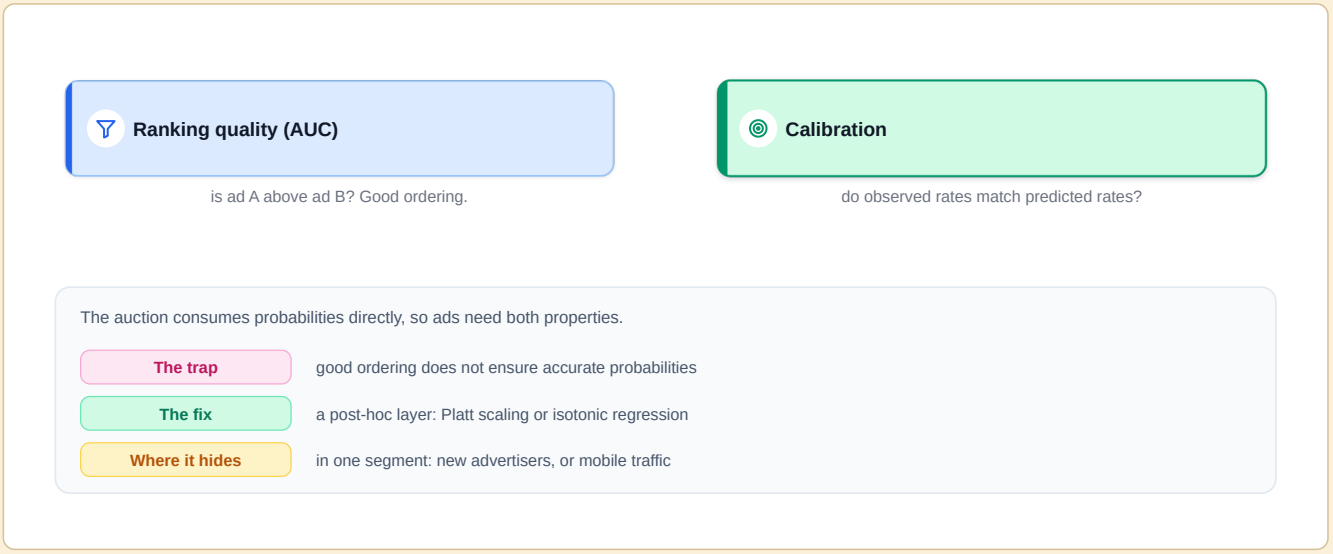


Figure — Good ranking and accurate probabilities are different properties (course diagram)

3. Data and labels

Two clocks:

- **Clicks** arrive in seconds, but a *non-click* is final only after an **observation window** — else late clicks become negatives.
- **Conversions** arrive hours to days later, some never. A recent click without conversion is **unresolved**, not negative; treating it as negative biases pCVR down (**delayed feedback**).

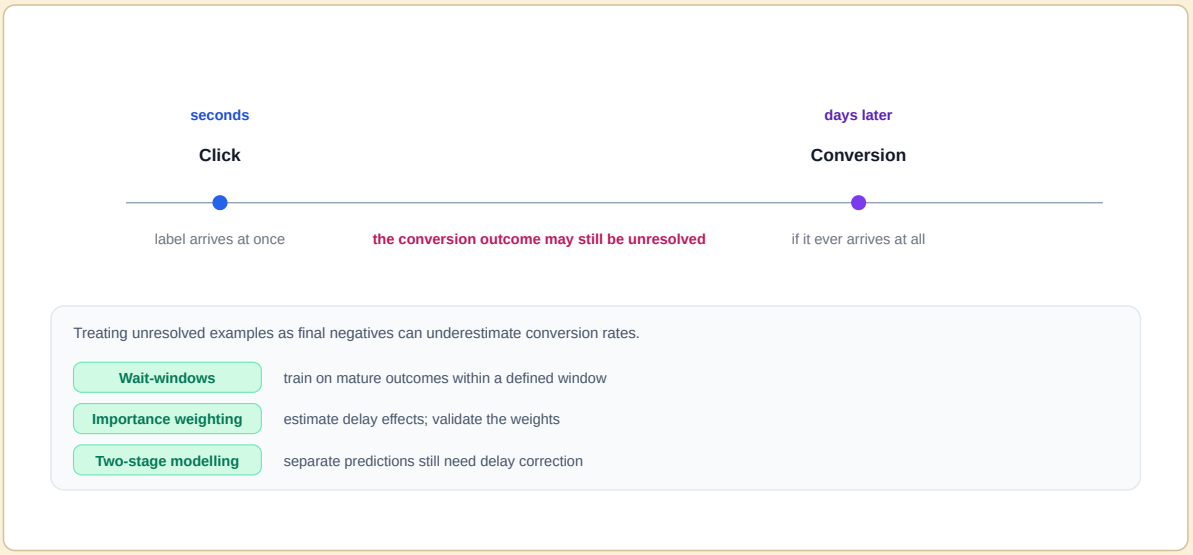


Figure — A click is instant; the conversion it may cause is not (course diagram)

Also **eligibility data** (targeting, remaining budget, policy approval) for a hard pre-model filter, and **invalid-traffic** signals (bots, click farms) removed before billing *and* training — with later credits and label fixes for what's found late.

4. Features

Group	Examples
User / context	Interest segments, recent activity, device, OS, placement, page topic, time, country
Ad / creative	Format, text and image embeddings, landing-page category
Advertiser history	Historical CTR by placement/device, account age, policy record
Cross features	User segment × ad category, placement × format (hand-made for logistic regression, learned by deep models)
Counters	Impressions/clicks per ad in the last hour/day (streaming)

Features are sparse (millions of IDs): **embedding tables** or **feature hashing** (IDs into a fixed number of buckets), all cached for the 50–100 ms budget.

5. Model: baseline → advanced

- **Baseline**: logistic regression on hashed and crossed features — cheap, usually well calibrated under log loss; the historical workhorse (Google search ads: online LR with **FTRL-Proximal**, an update rule keeping most weights exactly zero; McMahan et al., 2013).
- **Next**: boosted-tree outputs as features for a logistic model (Facebook, He et al., 2014), then deep CTR models that learn feature crosses — **factorisation machines** (Rendle, 2010; every feature pair through small learned vectors), Wide & Deep (Cheng et al., 2016, Google Play), DeepFM, DCN.
- **Multi-head**: shared body with pCTR and pCVR heads (17.2's pattern). Alibaba's **ESMM** (Ma et al., 2018) trains on *all* impressions by predicting pCTCVR = pCTR × pCVR, so the conversion head isn't learned only from the biased "clicked" subset.
- **Calibration layer** (part of the model): map raw scores to probabilities on held-out data with **Platt scaling** (one-feature logistic regression on the score — 2 parameters, little data) or **isotonic regression** (non-decreasing step function — flexible, needs more data, can overfit). If you **down-sample negatives** keeping fraction w, correct analytically: $q = p / (p + (1 - p)/w)$ (He et al., 2014) — e.g. $w = 0.05$, $p = 0.5 \to q \approx 0.048$.

6. Offline metrics

- **Log loss** and **normalised entropy** (log loss ÷ that of always predicting the average CTR; Facebook's metric) — both calibration-sensitive.
- **AUC** for ordering.
- **Calibration**: predicted/observed ratio, reliability curves, **ECE** (bucket-size-weighted mean |mean prediction − observed rate|). Break down **per segment** (advertiser size, slot, device, market): a model can be calibrated on average while over-predicting for new advertisers and under-predicting on mobile.

7. Online metrics and A/B testing

- **Platform**: revenue per thousand impressions (**RPM**), CTR. **Advertiser**: cost per click/conversion, return on ad spend — losing advertisers leave. **User guardrails**: ad load, ad hide/report rate, page latency.
- Ads-specific trap: **budget interference** — both arms spend the same campaign budgets, so one can starve the other; split budgets per arm.

8. Serving architecture

1. **Eligibility filter first** (targeting, budget, policy) — serving outside them breaks the advertiser contract, far worse than a poor match.

2. **Retrieval** by relevance (embedding or rule-based).
3. **Scoring**: calibrated pCTR/pCVR in one batched call.
4. **Auction**: rank by expected value (bid × pCTR, plus quality terms and reserves), price by the chosen rule. In a one-slot **second-price** (Vickrey) auction the winner pays the second-highest bid, making truthful bidding optimal; multi-slot **generalised second-price (GSP)** does *not* guarantee that (Edelman, Ostrovsky & Schwarz, 2007); Google Ad Manager moved to **first-price** in 2019.
5. **Pacing**: a control loop compares spend rate with plan and throttles or boosts participation — else budgets burn out by mid-morning.

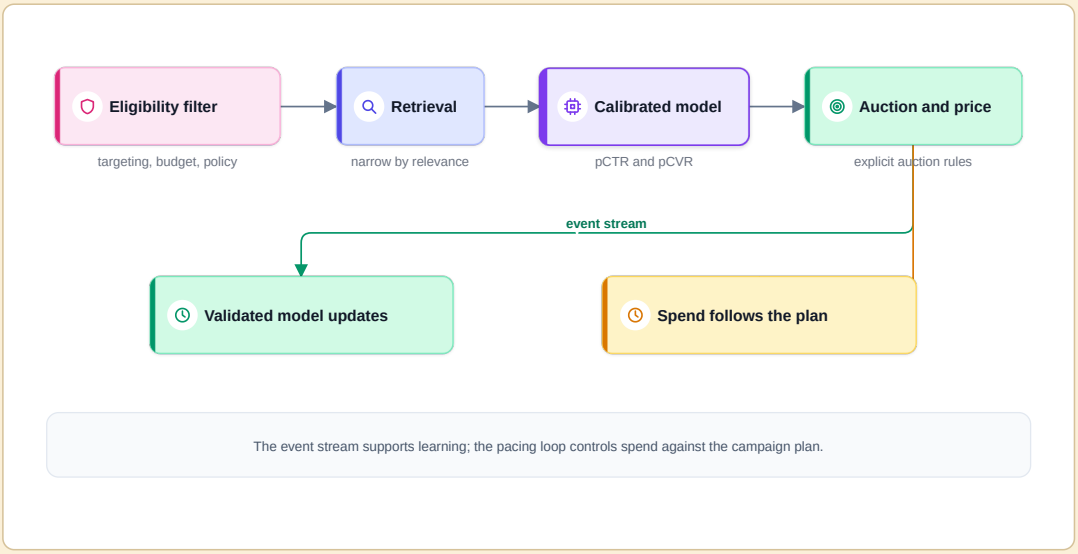
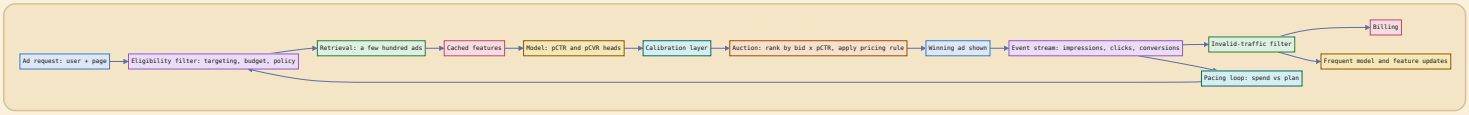


Figure — An example auction path with a 50–100 ms target (course diagram)



9. Scale numbers

Quantity	Arithmetic	Result
Scores per second at peak	100K requests × 300 candidates	30M scores/s
Cost sensitivity	30M/s × 86,400 s	~2.6 trillion scores/day at peak rate — a microsecond per score matters
Outage cost	revenue/day ÷ 86,400	Priced in dollars per second

10. Monitoring and failure modes

- **Update loop**: among the fastest in ML — incremental updates every minutes to hours, each validated with a tested rollback.
- **Cold start is permanent**: new campaigns start from **priors** (advertiser, category, creative) plus a small capped **exploration** budget.
- **Delayed conversions**: train on **mature** examples inside an **attribution window** (period after a click in which a conversion is credited to it), or model the delay (Chapelle, 2014).

Failure	Response
pCTR service degraded	Warm fallbacks: last-known-good model → segment-average CTR tables; eligibility and budget checks never skipped
Calibration drift in one segment	Per-segment alerts; recalibrate; investigate the feature pipeline
Invalid-traffic spike	Filter before billing and training; credits and label fixes later
Budget over-delivery	Pacing plus hard budget accounting

11. Interview follow-ups

- **"Second-price or first-price?"** → Incentives: truthfulness holds only for single-slot second-price, not GSP; calibration matters in every variant.
- **"An advertiser says they're overcharged."** → Predicted vs realised CTR for their segment, then auction logs, invalid-click records, billing — recalibration fixes predictions, billing errors need separate correction.
- **"Conversions are scarce — how does pCVR learn?"** → Factor into click × conversion-given-click (ESMM-style) so each head uses the densest labels; correct for delay; priors for thin advertisers.

Toy — calibration, down-sampling and the auction (`code/ads_calibration_toy.py`)

Synthetic impressions, ~4% CTR; a GBDT trained on all clicks and 10% of non-clicks. Correct the probabilities, check calibration per segment, run a two-advertiser auction.

```
keep = (ytr == 1) | (rng.random(len(ytr)) < w)          # all clicks, 10% of non-clicks
model = HistGradientBoostingClassifier(max_iter=200, random_state=0).fit(Xtr[keep], ytr[keep])
raw = model.predict_proba(Xte)[:, 1]
corrected = raw / (raw + (1 - raw) / w)                  # analytic down-sampling correction
platt = LogisticRegression().fit(np.log(raw_cal / (1 - raw_cal)).reshape(-1, 1), ycal)   # Platt on logit
iso = IsotonicRegression(out_of_bounds="clip").fit(raw_cal, ycal)                        # isotonic
```

```
true CTR in test traffic: 0.0397

pCTR source               AUC  mean pCTR  log loss     ECE
raw (down-sampled)      0.788     0.2368    0.3217  0.1971
analytic correction     0.788     0.0404    0.1438  0.0016
Platt scaling           0.788     0.0402    0.1438  0.0017
isotonic                0.787     0.0403    0.1444  0.0020

segment check (skewed model)   overall pred/actual = 0.0403/0.0397
  mobile   predicted 0.0601  observed 0.0460  ratio 1.31
  desktop  predicted 0.0203  observed 0.0334  ratio 0.61
auction, calibrated pCTR      : A wins 49.8% of mobile / 48.9% of desktop, expected value $1,064
auction, A inflated on mobile : A wins 75.9% of mobile / 48.9% of desktop, expected value $1,040
```

- The raw model has the **same AUC** (0.788) yet predicts 23.7% CTR for 4% traffic — AUC can't see it; log loss and ECE can.
- The formula and both calibrators restore ≈4% and cut ECE ~100×. A model can match the overall rate (0.0403 vs 0.0397) while 31% high on mobile and 39% low on desktop — only **per-segment** checks reveal it.
- Inflating one advertiser's mobile pCTR lifts its mobile win share 50% → 76% and lowers expected value ($1,040 vs $1,064): impressions go to the ad less likely to be clicked.

Common mistakes

Weak	Strong
Optimises AUC and stops	Calibration layer plus per-segment calibration monitoring
Starts with relevance retrieval	Eligibility (targeting, budget, policy) is the first, hard filter
Diagram ends at the winning ad	Delayed conversions, invalid-traffic filtering, pacing, warm fallbacks

Same design, other questions

- **Sponsored search ads** — must clear a query-relevance bar before bid × pCTR applies (17.3's retrieval). **Native feed ads** — a quality floor no bid can buy past, and an ad-load cap on slots shared with organic posts.

18.2 Case study: payment fraud detection

> "Design fraud detection for a payments platform."

1. Clarify requirements

No funnel: a stream of transactions, each needing a decision while the payment waits. The new ingredient is an **adversary** — fraudsters adapt to whatever you deploy, so some patterns die within days.

Question	Assumption used below
Volume	10K transactions/s at peak
Fraud rate	Below 1% (the toy uses ~0.2%)
Latency	~50 ms inside the authorisation call, ~10 ms of it for the model
Actions	Approve, **step-up authentication** (extra identity check: 3-D Secure, one-time passcode), review, block
Costs	Missed fraud → loss + **chargeback** fees; false decline → lost sale and trust; review → analyst time

Class imbalance: at 0.2% fraud, "approve everything" is 99.8% accurate and catches nothing — accuracy is off the table from minute one.

2. Frame it as an ML task

- **Input**: transaction (amount, merchant, channel), card/account, device, recent activity, graph links.
- **Output**: risk score ≈ P(fraud); binary classification on heavily imbalanced data.
- **Decision** — *separate from the score*: an **action policy** maps score (plus amount, review capacity, mandatory rules) to **approve** (below the low threshold — the vast majority), **extra checks** between thresholds (step-up, or human review via a **pending/delayed-capture** state — approved now, money taken later — since humans can't answer in 50 ms), or **decline** (above the high threshold).

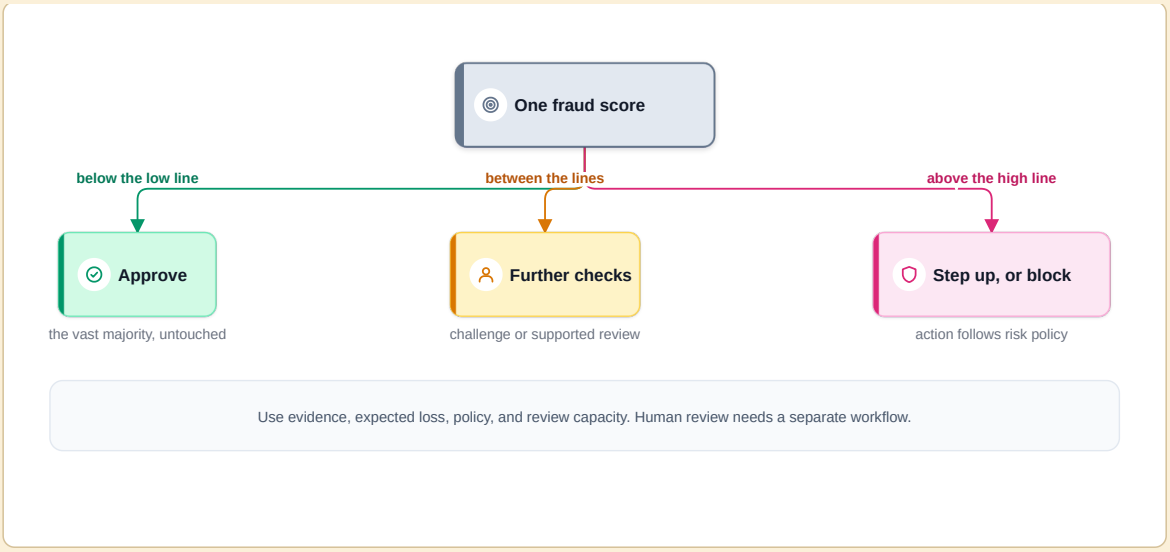


Figure — The decision is three-way, set by two thresholds (course diagram)

Thresholds trade precision against recall; set them from **expected cost**, review capacity and policy.

3. Data and labels

Name all three label problems:

1. **Delayed labels.** Chargebacks and reports arrive weeks later (assume 2–8); not every chargeback is fraud, not every fraud reported. Fresh recall *doesn't exist yet*; early warning comes from score distributions and rule-hit rates.
2. **Censored labels.** A blocked payment never produces an outcome — holes exactly where the model sees most risk. Partial remedies: analyst investigations, verification results, appeals — record each label's source. **Letting likely fraud through to learn is off the table** (any deliberate test needs risk-owner sign-off and a loss cap).
3. **Imbalance.** Class weights or down-sampling (then recalibrate — 18.1); report **PR-AUC**, precision/recall at operating thresholds, or best, **expected dollar loss**.

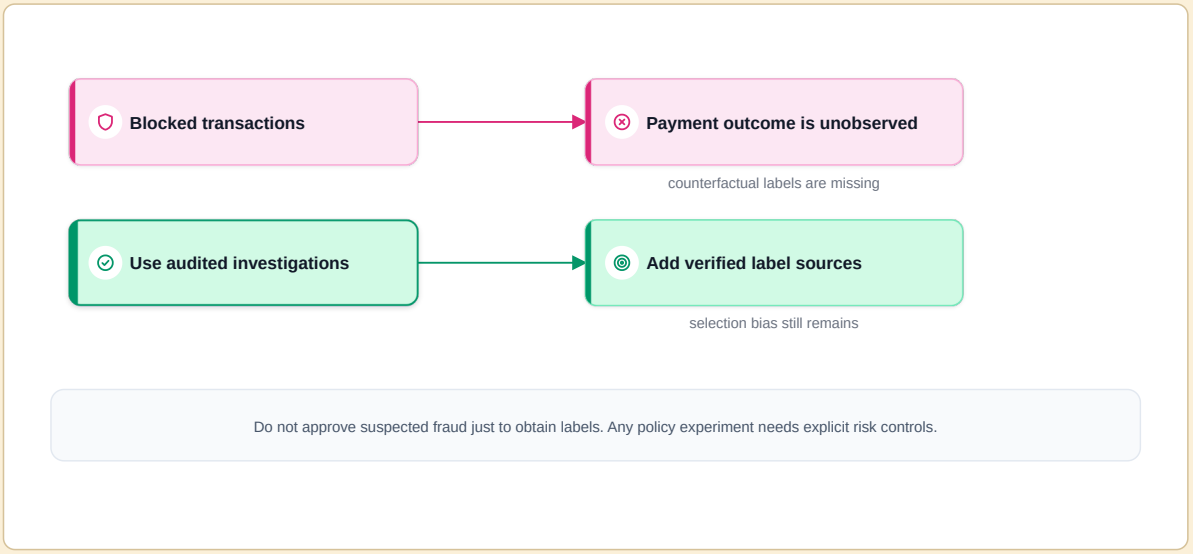


Figure — Blocked payments have missing outcomes; audited evidence helps but doesn't remove all bias (course diagram)

4. Features

Dominated by **velocity** (how much activity, how fast) and **graph** signals:

Group	Examples	Freshness
Velocity	Transactions on this card in the last 10 min / 1 h; distinct merchants today; amount vs this card's usual	Seconds (streaming)
"Impossible travel"	Distance between consecutive in-person transactions ÷ time between them	Request time
Device / session	New device? Device seen with other cards? Emulator, proxy, typing cadence	Request time
Graph	Device linked to a confirmed-stolen card; many accounts sharing one address or IP	Minutes
Merchant / account	Merchant fraud rate (from *past* windows only — leakage risk), account age	Hourly–daily

Fraud rings show up as dense clusters in the card–device–address graph before any single transaction looks odd. Shared devices also occur innocently (families, offices) — a graph link is evidence, not proof.

5. Model: baseline → advanced

Rules and model together, on purpose. A **rules engine** gives what a learned score can't: **hard constraints** (sanctions checks, verified blocklists aren't probabilities), **speed** against a new attack (a rule ships in minutes; a model has no training data for today's scheme), and **explainability** for regulators and dispute teams. The **model** catches combinations of weak signals.

- **Baseline**: rules + logistic regression on velocity features.
- **Standard**: **GBDT** on tabular features plus graph features or a **graph neural network** for rings; optionally an anomaly score (**isolation forest** — flags points random splits isolate unusually fast) for never-seen patterns.
- **Rule hygiene**: each rule gets an owner, a precision/recall dashboard and an **expiry date**, or **rule sprawl** builds up.

Real-world: Stripe Radar combines ML risk scores with merchant-written rules; in Europe, **PSD2**'s strong customer authentication makes 3-D Secure step-up a standard middle action.

6. Offline metrics

- **PR-AUC** — meaningful under heavy imbalance; with 500 legitimate per fraud, a small false-positive *rate* is still a large *number* of false alarms, which ROC-AUC hides.
- **Recall at a fixed false-positive rate**, or **precision at a fixed review capacity**.
- **Expected cost** of the whole policy (missed fraud + false declines + review), on a **time-based** split with **mature** labels only — a **backtest**.

7. Online metrics and A/B testing

- **Business**: fraud loss in **basis points** of volume (1 bp = 0.01%), chargeback rate, false-decline rate, review-queue size and SLA.
- You can't A/B a fraud model like a feed (deliberately letting fraud through is unacceptable). Rollout: backtest → **shadow scoring** (compare distributions and reviewer agreement) → **canary** under approved risk limits → wait for mature labels → full rollout (16.6).

8. Serving architecture

~50 ms end to end, set by the payment network: **rules** → **feature assembly** (online store + streaming velocity) → **model** (~10 ms) → **policy**. Labels return as three streams at different speeds: analyst findings, verification results, late chargebacks.

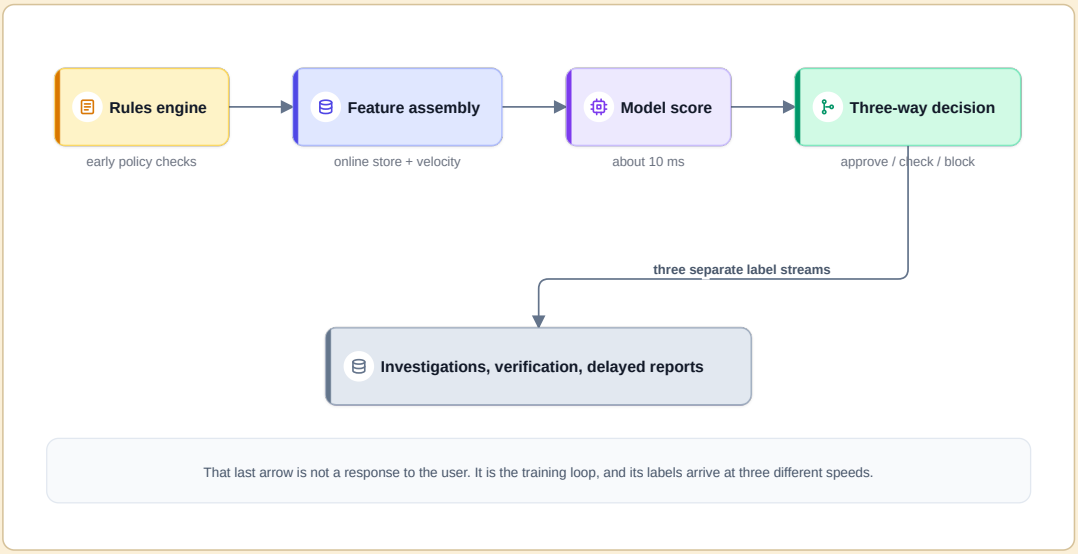
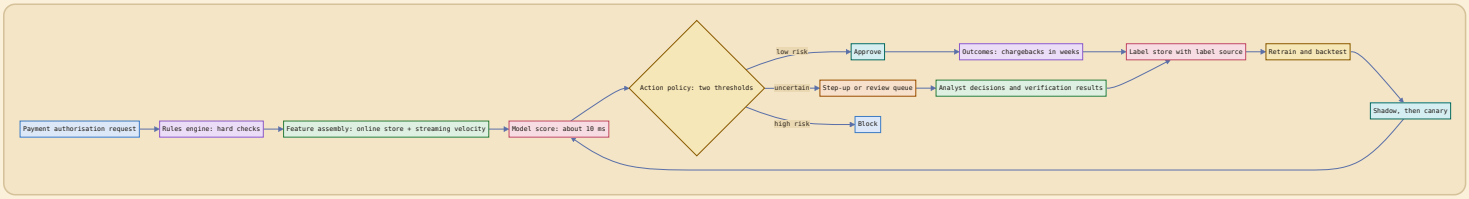


Figure — An example 50 ms automated decision path, with labels arriving separately (course diagram)



9. Scale numbers

Quantity	Arithmetic	Result
Fraud events	10K/s × 0.2%	~20 fraudulent attempts/s at peak
Review load	0.5% in the middle band: 10K × 0.005 × 3,600	180K reviews per peak hour — impossible for humans, so step-up handles most of the band and review is kept for high-value cases
Velocity state	200M active cards × 4 counters × 8 bytes	≈ 6.4 GB in memory (plus keys, replicas), updated per transaction

10. Monitoring and failure modes

- **Early warning** while labels are weeks late: score distributions, rule-hit rates, approval rate — per merchant, country, channel.
- **Adversarial drift**: bursts of small "test" transactions on new cards. A **sudden drop** in flagged rate is possible evasion or a broken feature, not good news.
- **Timeout policy**: neither global **fail open** nor global **fail closed** — decide by risk in advance: small amounts from long-standing accounts pass, large or risky ones are challenged or held.

Failure	First check
False-decline rate doubles overnight	Data feeds before the model: a stalled velocity counter moves every score. One segment spiking → a rule or feature; everything spiking → upstream
Fraud volume ×10, analysts ×1	Narrow the review band to high-value cases, lean on step-up; never weaken mandatory controls

11. Interview follow-ups

- **"Once the model is strong, can the rules go?"** → No: speed during an attack, hard constraints, readable logic. Manage rules with metrics and expiry dates.
- **"How do you catch a fraud ring?"** → Graph features, **community detection** (finding tightly connected groups), tools to expand from one confirmed case to its neighbours — checking for innocent shared infrastructure.

Toy — velocity features and two thresholds (`code/fraud_thresholds_toy.py`)

One synthetic week of card transactions (~0.23% fraud); stolen cards used in quick bursts. Train days 1–5, test days 6–7 (time split), then pick approve/review/block thresholds by expected dollar cost under a review-capacity limit.

```
def velocity(card, t, window=10.0):
    """# of earlier transactions on the same card within `window` minutes (a streaming aggregate)."""
    # ... keep, per card, the timestamps still inside the window

def cost(lo, hi):
    block, review = p >= hi, (p >= lo) & (p < hi)
    missed = (yt == 1) & ~block & ~review                       # approved fraud: lose the amount
    c = amt[missed].sum() + COST_FALSE_DECLINE * ((yt == 0) & block).sum() + COST_REVIEW * review.sum()
    c += amt[(yt == 1) & review].sum() * 0.1                    # reviewers miss ~10% of what they see
```

```
360,769 transactions, fraud rate 0.230%
'approve everything' accuracy = 99.770%  (catches 0 fraud)

amount+foreign+hour    ROC-AUC 0.838   PR-AUC 0.054
+ velocity (10 min)    ROC-AUC 0.849   PR-AUC 0.408

no model (approve all): cost $33,052
single threshold 0.5   lo=0.50 hi=0.50: cost $17,258  blocked 163  reviewed 0  fraud caught 50%
best two thresholds    lo=0.02 hi=0.70: cost $15,554  blocked 158  reviewed 293  fraud caught 57%
```

- **Accuracy is useless**: 99.77% for a policy that catches nothing.
- **ROC-AUC barely moves** (0.838 → 0.849) with velocity, but **PR-AUC jumps ~7.6×** (0.054 → 0.408) — ROC-AUC hides false alarms under imbalance.
- A **review band** (within 300-review capacity) catches more fraud at lower cost than one threshold. (In practice pick thresholds on a separate validation window.)

Common mistakes

Weak	Strong
Binary classifier, one threshold	Three-way decision, two cost-based thresholds, step-up in the middle
"Retire the rules once the model is good"	Rules for speed, constraints, explainability; model for complex patterns
Quotes accuracy at a 0.2% base rate	PR-AUC, precision/recall at operating points, expected dollar loss

Same design, other questions

- **Account takeover at login** — the unit is a session; device fingerprint, location velocity and typing rhythm dominate; step-up is the main action. **Promo/referral abuse** — audit labels in days (faster loop); graph features dominate because abuse comes from rings.

18.3 Case study: harmful content detection (and running it at platform scale)

> "Design a system to detect harmful content — hate speech, harassment, spam, graphic violence, misinformation." Scale-up version: trust and safety for a short-video app receiving ~200M new items daily (clips, sound, captions, comments), with a rulebook from CSAM (zero tolerance, legally mandated handling) to grey-zone health misinformation.

1. Clarify requirements

Fraud's adaptive adversary and unequal error costs, plus:

1. **Multimodal input** — a harmless photo plus a hateful caption is harmful though neither half is: several models (per modality or harm) plus a policy layer combining scores.
2. **Costs both ways, not only money**: a **false negative** exposes people to harm; a **false positive** silences a legitimate user. Thresholds become an explicit policy conversation.
3. **Huge scale, low base rates** varying by harm type and surface.

Question	Assumption used below
Volume	200M uploads/day ≈ 2,300/s average (tens of thousands/s at peak across all item types)
Violation rate	Below 1% per category
Latency	Livestream near-real-time (rolling few-second windows); uploads a second or two; back-catalogue re-scans offline
Scope	Public posts; ads and private messages have their own pipelines and privacy rules
Jurisdiction	One policy engine keyed by **harm category and country** — legal lines differ

Highest-harm category first. CSAM (child sexual abuse material) needs a dedicated, tightly governed process: **hash matching** (comparing a short file fingerprint) against approved databases of known material (Microsoft's **PhotoDNA**; Meta open-sourced **PDQ** and **TMK+PDQF** in 2019), restricted-access review, evidence preservation, legally required reporting. In the US, providers must report apparent CSAM to **NCMEC**'s CyberTipline once they know of it (18 U.S.C. § 2258A); that statute doesn't itself require proactive scanning, and duties differ by country — name who owns policy and escalation. Hashes find *known* material only.

2. Frame it as an ML task

- **Input**: the item (frames, audio, text), context (account, where posted, replies), distribution (views, share velocity).
- **Output**: per-category risk P(violates policy c). **Category**: multi-label classification run as a **cascade** (cheap models on everything, expensive ones only on what earlier stages flag), then a policy decision — an **action ladder**, not remove/allow (section 8).

The metric that matters: prevalence of violating views = views landing on violating content ÷ all views, estimated by human-labelling a random sample of *views* — harm scales with exposure, so each miss counts by its reach. Report **per category with confidence intervals** (Meta's Community Standards Enforcement Report gives violating views per 10,000). **Opposite-direction guardrails**: **false-removal rate** (audits) and **appeal-overturn rate**; plus **time to action** and **views before action**.

3. Data and labels

1. **Rare, expensive labels** — reviewer time is the bottleneck. **Active learning** sends the most useful items (uncertain, diverse, high reach) to reviewers; a *separate* random sample measures prevalence, since active-learning samples are biased by design.
2. **Three label feeds**: reviewer decisions (best, scarcest); **appeals** (false-positive evidence but self-selected — add random audits); **user reports** (huge, noisy, manipulable — weight by **reporter reputation**, the historical precision of that reporter; evidence, never verdicts).
3. **Adversarial evasion**: **leetspeak** ("fr33"), **homoglyphs** (Cyrillic "a" for Latin "a"), **text inside images** (needs **OCR**), **splitting** the attack into the comment under an innocent post, **re-encoding / cropping / mirroring** to break hashes, **coded language** rotating faster than training cycles.
4. **Consistency and context**: guidelines plus **inter-rater agreement** checks; the same image can be news or gore.

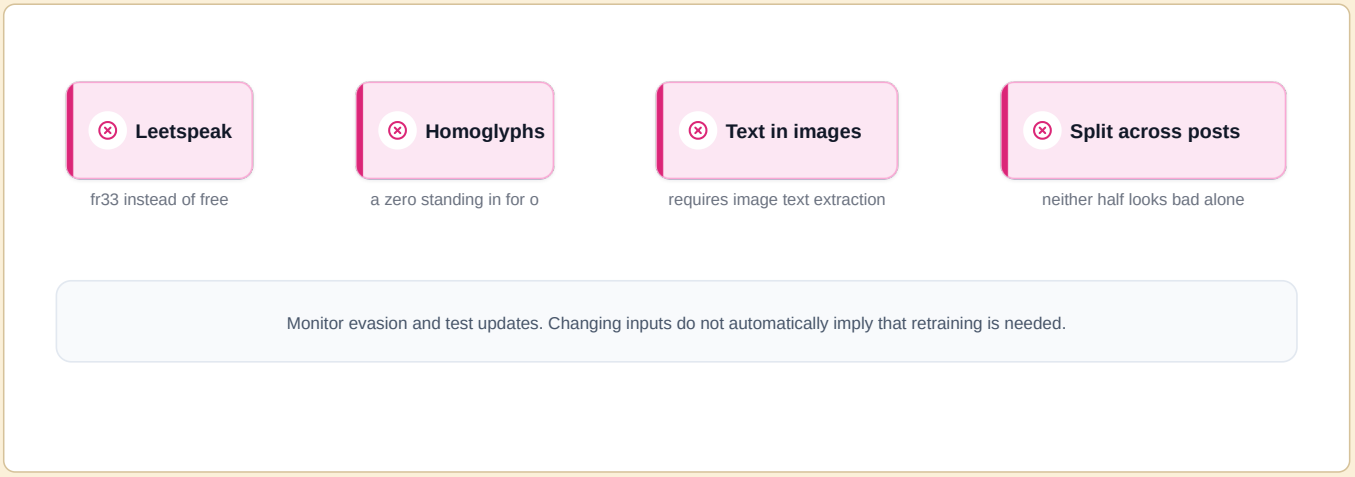


Figure — Four forms of adversarial evasion (course diagram)

4. Features

Group	Examples
Content	Text embeddings (after normalising leetspeak/homoglyphs), image and frame embeddings, OCR text, **ASR** transcript, audio embeddings
Hashes	Exact hash, **perceptual hash** (stays similar after small edits) of keyframes, audio fingerprint
Account / context	Account age, prior strikes, posting rate; surface (comment, post, live), thread replied to
Distribution	Views, **share velocity**, audience
Reports	Count weighted by reporter reputation, report-graph clustering

5. Model: baseline → advanced (one model per stage)

1. **Hashes and high-confidence rules** — exact hashes catch identical files; perceptual hashes tolerate some edits (test against crops and mirroring; audio fingerprints survive a mirrored video). Hash-sharing programmes exist for some categories (GIFCT's database for terrorist content).
2. **Fast classifiers on everything** — small per-modality models (a distilled text transformer, a small image CNN) whose only job is triage.
3. **Heavy multimodal models on the flagged slice** — large image-text models and LLM-based classifiers for nuanced text and memes.

4. **Human review** — borderline and appealed cases; also the label source.

Baseline for a new category: rules + hashes + report routing; a fast classifier as labels accumulate; heavy models last. **Upgrade** by distilling the heavy model into the fast one.

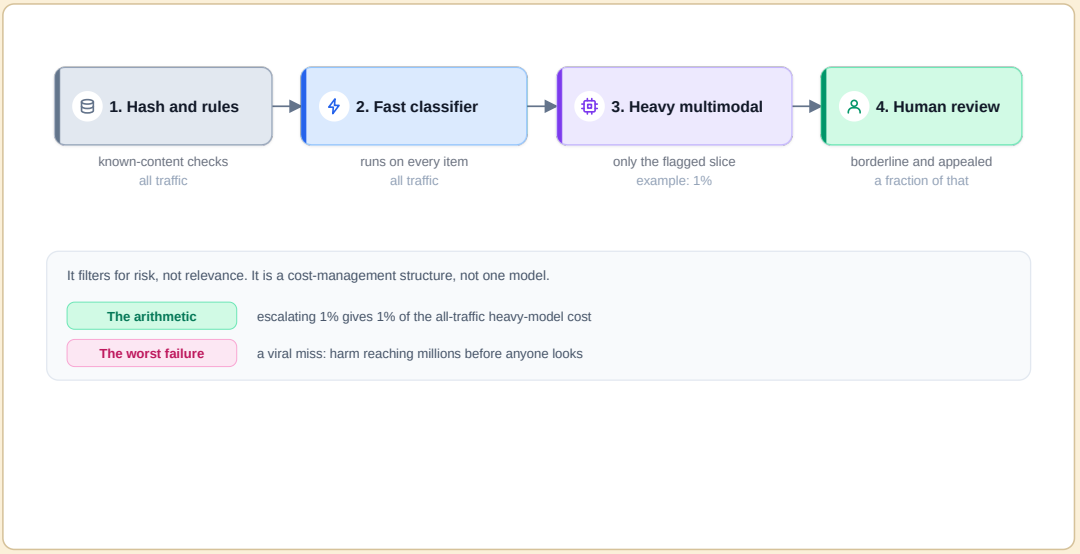


Figure — The risk funnel spends compute in proportion to risk (course diagram)

6. Offline metrics

- Per-category **precision and recall at the operating thresholds** on a **fully reviewed** set, sliced by language, region, content type.
- **Recall of the whole cascade**, not of each stage: an item stage 2 wrongly passes never reaches stage 3.
- **Hash recall under mutation**: known items transformed by current evasion tricks.

7. Online metrics and A/B testing

Metrics from section 2, plus creator churn and queue age. You can't ethically randomise *leaving harm up*: backtest → shadow scoring → **independent adjudication** of disagreements (agreement with the old system isn't proof) → canary → sampled prevalence.

8. Serving architecture

The action ladder — policy as callable actions, per category and region:

Rung	When	Notes
Auto-remove	Confidently violating	Tuned so false removals stay rare; user notified, can appeal
Human review	Uncertain cases, appeals	Prioritised queue
Reduce / age-gate / warn	Borderline or allowed-but-restricted	Stays up but isn't amplified (the feed integrity layer, 17.2), hidden from minors, or behind a warning
Auto-allow	Clearly fine — the vast majority	

Thresholds are owned by Trust & Safety and Legal (not the model team), set from measured error rates, severity, reviewer capacity and law.

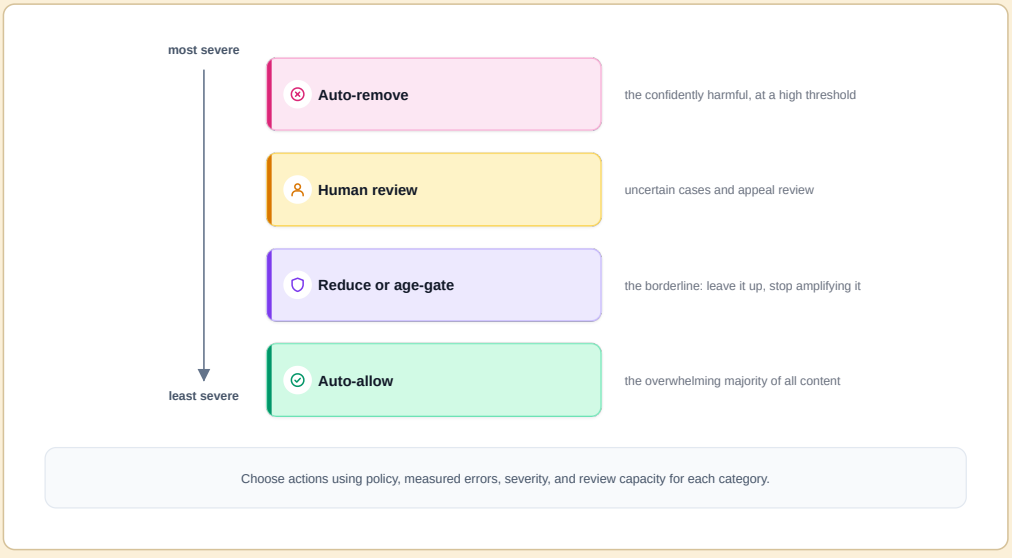


Figure — The action ladder, not a binary (course diagram)

Review-queue priority = f(severity, expected reach, uncertainty, deadline), with capacity reserved for random audits and appeals.

Reach-aware re-scoring against the **viral miss** (harm reaching millions before anyone looks): crossing each view tier (10K, 100K, 1M) triggers a re-check by a stronger model; affordable because few items climb the tiers. Share velocity also pushes fast spreaders to deeper

checks and the head of the queue.

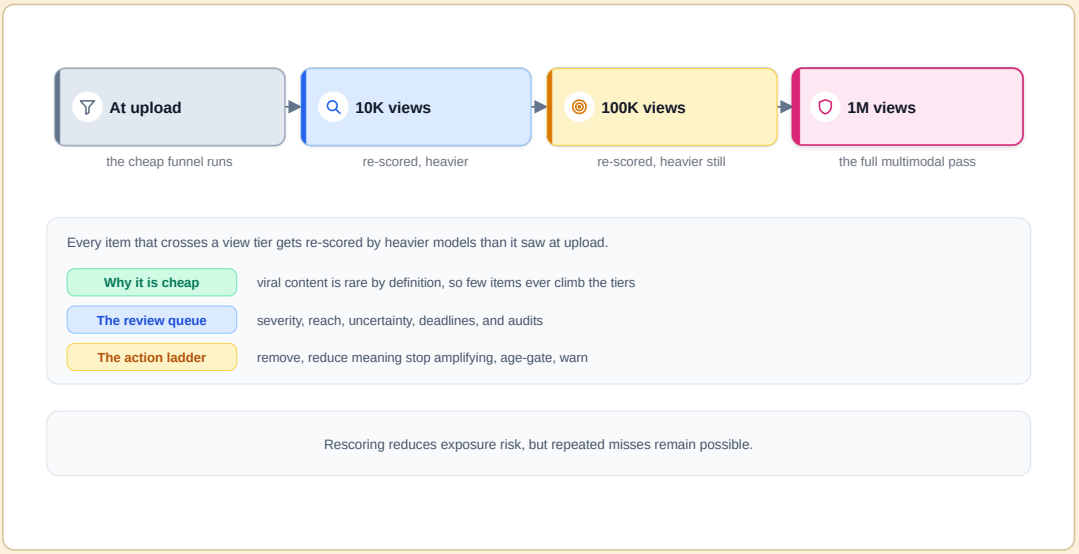
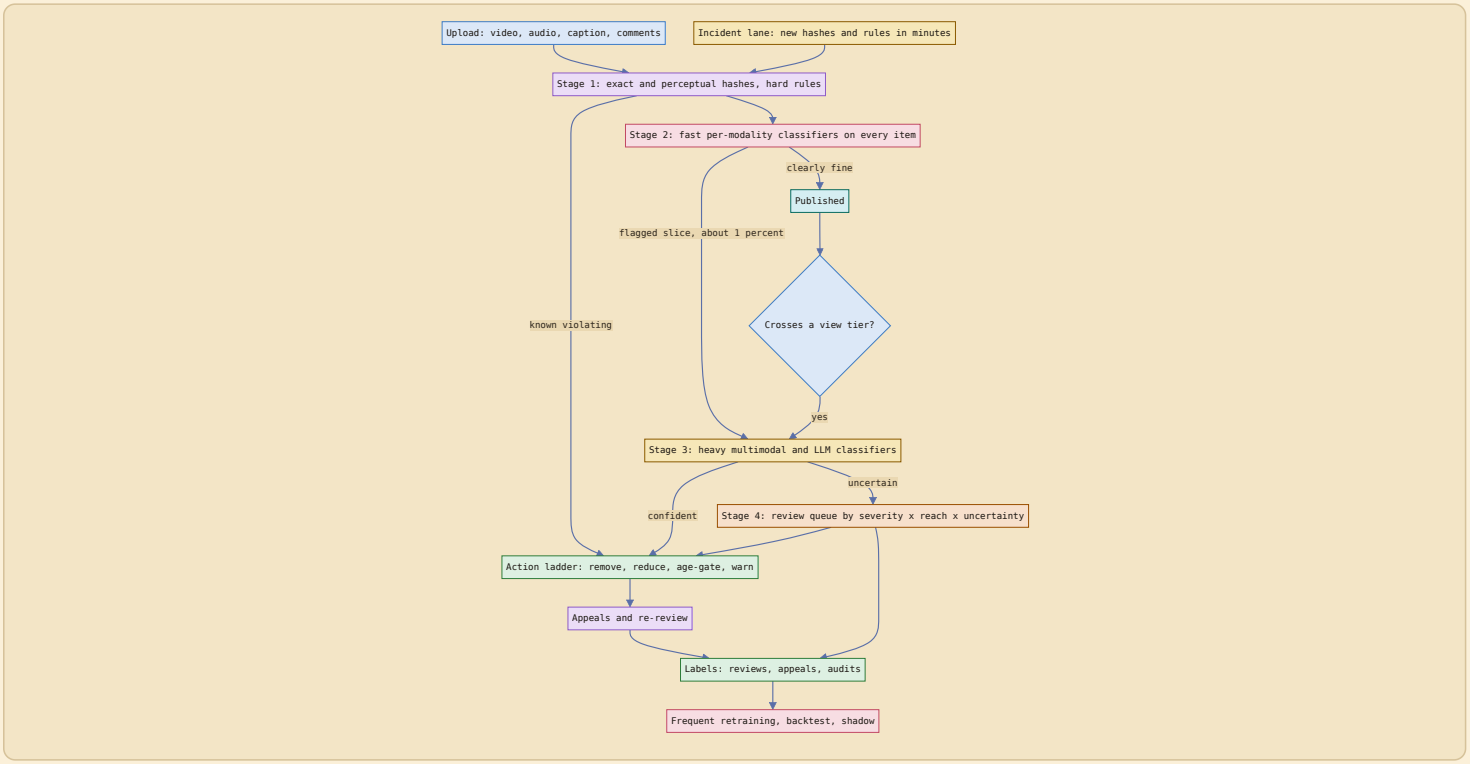


Figure — Reach-aware re-scoring: scrutiny grows with the audience (course diagram)

Incident lane: in a crisis (a live-streamed attack being re-uploaded) new fingerprints and rules go live in minutes — no retrain is that fast.



9. Scale numbers

Quantity	Arithmetic	Result
Fast stage	200M/day ÷ 86,400	≈ 2,300 items/s average → small models, strict budgets
Heavy stage at 1% flag rate	200M × 0.01	2M/day ≈ 23/s average (plus media length, decoding, peaks, replicas)
Cost of the dial	+0.1 percentage point flag rate	+200,000 heavy inferences/day
Human review at 0.1%	200M × 0.001	200,000 decisions/day
Reviewers	200,000 ÷ ~300 decisions per reviewer-day	≈ 670 reviewer-days per day → with shifts and leave, many hundreds of staff

The flag rate is the GPU bill; the review threshold is a **headcount** setting.

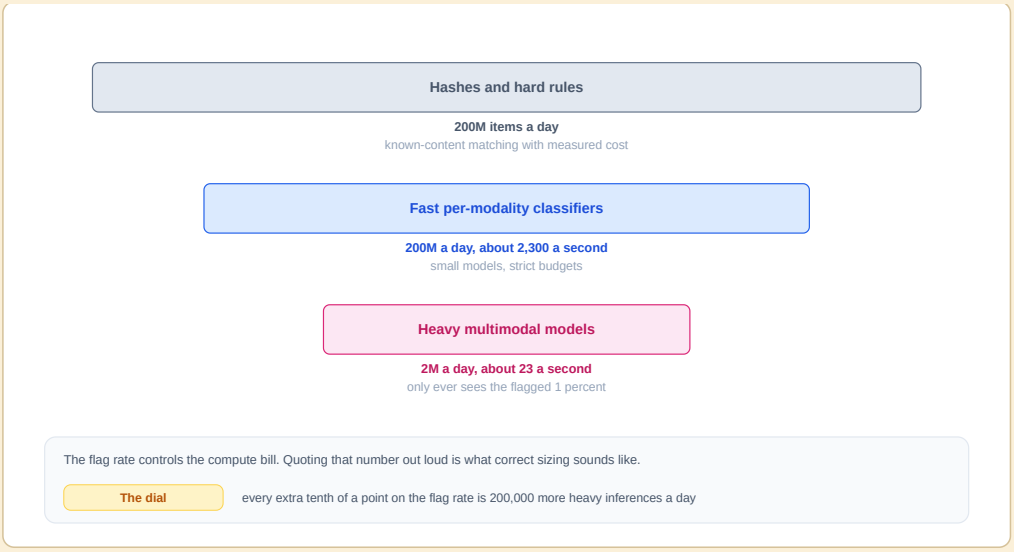


Figure — The risk funnel, with the volume said out loud at every stage (course diagram)

10. Monitoring and failure modes

- A sudden **drop** in flag rate means possible evasion, not improvement.
- **Policy changes trigger a back-catalogue re-scan** — yesterday's allowed content can violate today's rule.
- **Outage policy per category**: never treat unscored content as low-risk; high-risk surfaces hold (fail closed), low-risk ones may publish (fail open).
- **Moderator well-being is designed capacity**: exposure limits per shift, rotation off graphic queues, blurred-first media, attrition in the headcount plan.

Failure	Loud or quiet?	Defence
Viral miss	Loud	Re-check at view tiers; for the worst harms escalate generously (accept extra false alarms)
Over-removal	Quiet (overturns, creator churn)	Audited false-removal and overturn rates beside prevalence
Evasion wave	Quiet at first	Flag-rate drop alerts, near-duplicate clustering, fast rules

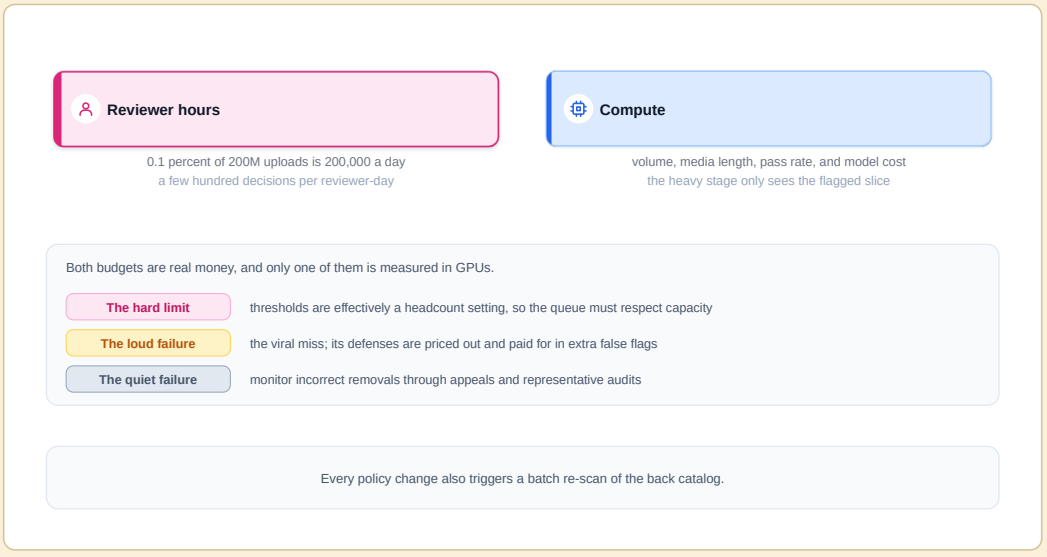


Figure — Two budgets, and only one of them is measured in GPUs (course diagram)

11. Interview follow-ups

- **"A brigade mass-reports one creator overnight."** → Reports raise priority but can't remove; reputation weighting and report-graph clustering (same target, correlated timing, shared origin) expose the campaign — a fraud-ring problem in moderation clothing.
- **"A government bans a new category within a week."** → Rules, hashes and report routing on day one, classifier in weeks; the region-keyed policy engine scopes it to one jurisdiction.

Toy — the cascade dial and prevalence sampling (`code/moderation_cascade_toy.py`)

One million uploads, 0.5% violating, heavy-tailed views. The fast classifier scores everything; the heavy model (100× the cost) scores the flagged slice; removal needs a high heavy score. Vary the flag rate, add view-tier re-scoring, estimate prevalence by sampling views. (Simplification: removals happen before any views.)

```python
def run(bad, views, fast, heavy, flag_rate, viral_tier=None):
    flagged = fast >= np.quantile(fast, 1 - flag_rate)
    if viral_tier is not None:
        flagged |= views >= viral_tier                          # reach-aware re-scoring
    removed = flagged & (heavy > 2.5)
    compute = N * 1 + flagged.sum() * HEAVY_COST                # fast on all + heavy on flagged
    viol_views = views[bad & ~removed].sum() / views[~removed].sum() # prevalence of violating views
```

```
violating uploads 0.49%, violating share of views before action 0.45%; top 1% of items get 62% of views

policy                            heavy runs  compute*  recall  viol. views false rem.
flag top 0.5%                          5,000     0.015   30.4%       0.345%         24
flag top 1.0%                         10,000     0.020   41.3%       0.288%         49
flag top 2.0%                         20,000     0.030   52.4%       0.172%        103
flag top 5.0%                         50,000     0.060   67.6%       0.136%        281
flag top 1.0% + re-score >=10K views     12,903     0.023   41.5%       0.136%         67
* compute relative to running the heavy model on every item

true prevalence of violating views: 0.136%
  sample  5,000 labelled views -> estimate 0.120%  (95% CI 0.024% to 0.216%)
  sample 50,000 labelled views -> estimate 0.152%  (95% CI 0.118% to 0.186%)
```

- The cascade costs 1.5–6% of "heavy model on everything"; higher flag rates buy recall with diminishing returns.
- **Reach-aware re-scoring** matches the 5% flag rate's violating-view prevalence (0.136%) at well under half the compute (0.023 vs 0.060) and a quarter of the false removals, because 1% of items get 62% of views. Upload recall barely moves (41.5%) — the view metric is the one tracking harm.
- Measuring 0.1% prevalence is hard: 5,000 labelled views give a CI of 0.02–0.22%, 50,000 narrow it to 0.12–0.19% — metric precision costs reviewer hours.

Common mistakes

Weak	Strong
One big classifier over everything	Cascade: expensive compute only on the slice that needs it
Remove vs allow	Full action ladder, including reduce and age-gate
Thresholds read off a PR curve	Thresholds as policy, per category and region, informed by measured errors

Same design, other questions

- **Marketplace counterfeits** — photos, text and price that don't fit (a luxury bag at a tenth of retail); actions hit listings and sellers. **DM spam** — private content, so triage uses sending behaviour and reports; **rate limits** before removal.

18.4 Case study: People You May Know (PYMK)

> A LinkedIn-style network with 900M members wants a connection-suggestion module, shown on the home page and during onboarding. Design it.

1. Clarify requirements

Question	Assumption used below
Members	900M registered (not daily active)
Surfaces	Home-page module; post-sign-up onboarding
Latency	~200 ms for the module; heavy work precomputed
Privacy	Suggestions can reveal sensitive relationships or invite unwanted contact — privacy controls are core requirements

Success has three layers, because each simple metric is a trap: invites sent ignores whether the recipient wants it; acceptance rate alone rewards suggesting obvious acquaintances; the real goal is **network value** — connections that go on to interact.

Layer	Metric
Primary (online)	Accepted invitations per suggestion impression
Guardrails (can veto)	Invitation-ignore rate; "I don't know this person" reports (weighted heavily in training, like hides in 17.2)
Long-term (weeks later)	Interaction between newly connected pairs

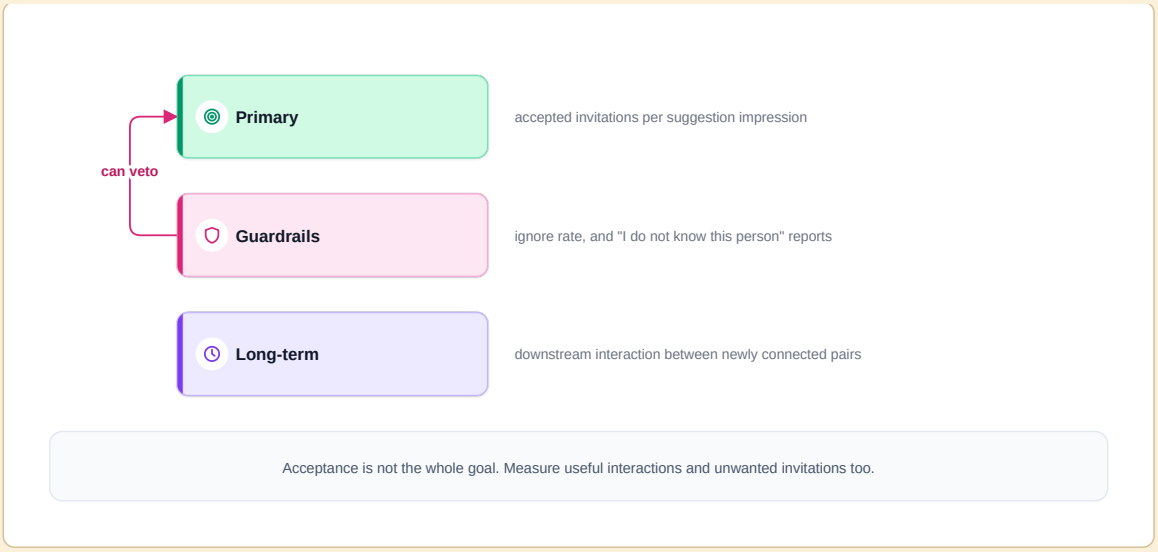


Figure — Three layers of metric, because acceptance alone is a trap (course diagram)

2. Frame it as an ML task

- **Input**: viewer, candidate, the graph around them, context (new vs tenured member).
- **Output**: **P(send)** = P(invite sent | shown) — does the *viewer* want it? — and **P(accept | sent)** — does the *candidate*?
- **Ranking score** = P(send) × P(accept | sent) = probability of an accepted invitation per impression; both sides must want it.
- **Category**: **link prediction** (which graph edges will form) + two-sided ranking.

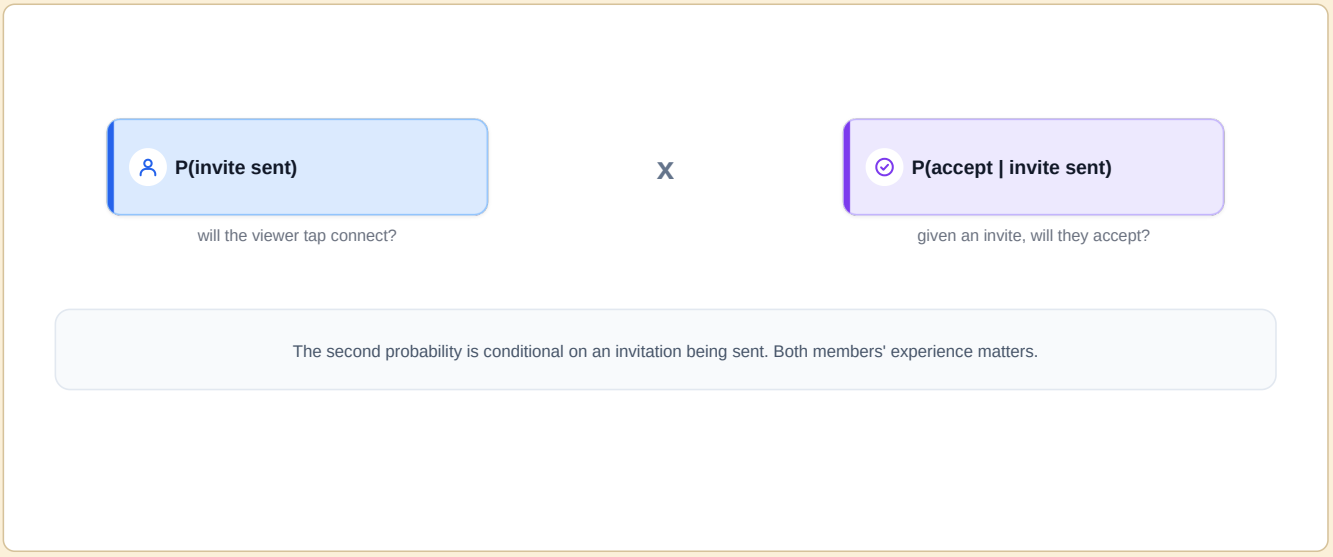


Figure — The score multiplies the viewer's intent by the recipient's willingness (course diagram)

3. Data and labels

- **Graph**: connections, follows, past invites (sent, accepted, ignored), dismissals, blocks. **Profiles**: employer, school, groups, skills, location, with dates (overlap in time matters).
- **Imported contacts** (only with permission): minimise collection and retention, support opt-out and deletion; hashing identifiers does *not* anonymise them.
- **Labels**: invite-sent arrives in-session; acceptance takes days. A pending invite is *unresolved*, not rejected — use an observation window (18.1's delayed-label idea).

4. Features

Group	Examples
Graph	Mutual-connection count and strength; **Adamic-Adar** = Σ over mutual friends of 1 / log(that friend's degree) — a shared friend with few connections counts more than one who knows everyone; graph-embedding similarity
Affinity	Same employer/school/group with overlapping dates, similar skills, mutual profile views
Behavioural	Viewer's historic invite-acceptance rate; candidate's accept rate and inbox load
Context	New vs tenured; surface; times this pair was already shown

5. Model: baseline → advanced

Candidate generation — four sources, unioned and deduped (17.1's multi-source retrieval, people as the catalog):

1. **Friends-of-friends (FoF) / triangle closing** — A–B and B–C connected → C is a candidate for A. The main source, precomputed and **bounded**: expand only top neighbours by interaction, sample very-high-degree accounts' lists, cap candidates per member.

2. **Shared affiliations** — same company/school/group with overlapping dates; catches the new coworker with no mutuals.
3. **Imported contacts** — permission-based matches.
4. **Embedding similarity** — two-tower member vectors from profile and activity, or graph embeddings (**node2vec**-style random walks, or graph neural networks), via an ANN index.

Hard filters at retrieval (re-checked before display): already connected, blocked, dismissed, opted out. **Impression discounting** lowers candidates shown many times without action — softer than a permanent dismissal.

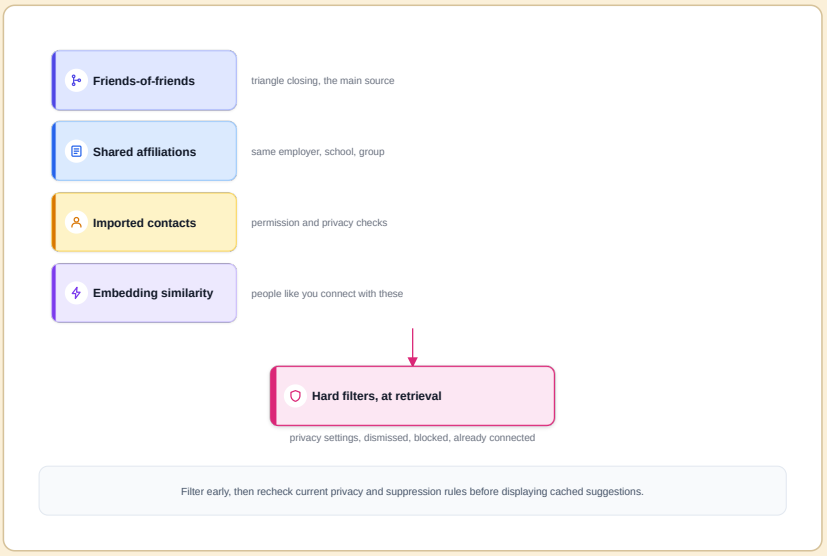


Figure — Four retrieval sources, then hard filters before ranking (course diagram)

Ranker: baseline = sort FoF by mutual count or Adamic-Adar; advanced = GBDT or a two-head network predicting P(send) and P(accept | sent), multiplied.

6. Offline metrics

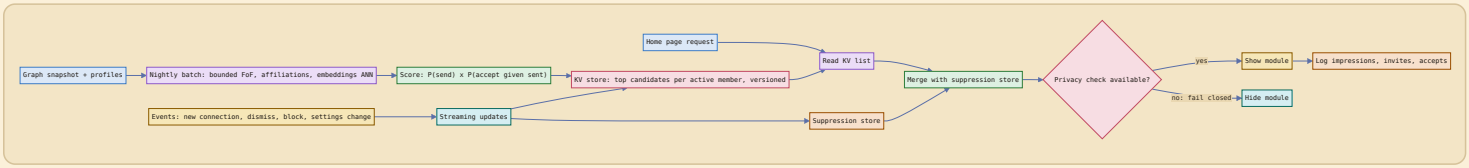
- **Recall@K of future edges**: hide connections formed after a cutoff; would retrieval have produced them?
- **AUC / log loss** per head, plus calibration — the heads are multiplied.
- Diversity: share of suggestions to the top-1% most-connected members; share crossing community boundaries.

7. Online metrics and A/B testing

- The three metric layers, on **both sides** (inviter and recipient).
- **Network interference**: a treated viewer invites a control member, so user-level randomisation leaks. Use **graph-cluster randomisation** (whole communities to one arm) or model the interference.
- The interaction metric takes weeks — keep a long-running **holdback** (16.9).

8. Serving architecture

- **Nightly batch**: bounded FoF, affiliations, scoring → top candidates per active member in a **KV store**, with **idempotent, versioned** writes so a retry can't overwrite a newer run (System Design Fundamentals).
- **Streaming path**: new connections, dismissals, blocks update a **suppression store** within minutes (seconds for dismissals).
- **Request time**: read the list, merge suppressions, re-check current privacy settings, render.
- **New members**: contacts and affiliations carry the first session, refreshed within minutes as first connections form.



9. Scale numbers

Quantity	Arithmetic	Result
Batch scope	Active in last 30 days only (assume 300M) instead of 900M	~⅓ of the graph work; dormant members refreshed weekly
FoF blow-up	500 connections, each with 500 → up to 250,000 two-hop paths	why expansion is capped to top neighbours
KV storage	300M members × 100 candidates × ~16 bytes	≈ 480 GB before replication

The dominant cost is usually **graph computation**, not inference.

10. Monitoring and failure modes

Graph-shaped feedback loops: **rich-get-richer** (well-connected members sit in more triangles, get suggested more, grow more connected — cap exposure per candidate) and **cluster amplification** (triangle closing reinforces silos — monitor cross-cluster suggestions).

Failure	Response
Nightly graph job fails	Serve last validated candidates within a freshness limit, still enforcing *current* privacy, blocks, dismissals; alert on age
Privacy filter unavailable	**Fail closed**: render nothing rather than unverified suggestions
Dismissed person reappears an hour later	Design bug: dismissals must go through the streamed suppression store merged at render time

11. Interview follow-ups

- **"FoF for 900M members without melting the cluster?"** → Top neighbours only, 30-day-active members nightly, sampled celebrity neighbour lists, capped outputs — never the full two-hop closure.
- **"Legal asks you to prove opted-out members never appear."** → Auditable filters at retrieval *and* before display, fail-closed on timeouts, prompt cache invalidation, filter-version logging, tests for stale caches and race conditions.

Toy — FoF retrieval, Adamic-Adar, two-sided ranking (`code/pymk_toy.py`)

A 4,000-member clustered graph (networkx `powerlaw_cluster_graph`: a few hubs, many small clusters), 10% of edges hidden as "future". Part 1 ranks FoF candidates three ways; part 2 simulates viewers who like inviting popular members who rarely accept.

```
def fof_candidates(G, u):
    """Two-hop neighbours with their mutual-connection lists (bounded in production: top neighbours only)."""
    mutual = {}
    for f in G[u]:
        for c in G[f]:
            if c != u and not G.has_edge(u, c):
                mutual.setdefault(c, []).append(f)
    return mutual

scores = {
    "random FoF": {c: random.random() for c in cands},
    "common neighbours": {c: len(m) for c, m in mutual.items()},
    "Adamic-Adar": {c: sum(1 / math.log(deg[f]) for f in m if deg[f] > 1) for c, m in mutual.items()},
}
# part 2: rank the top 10 by p_send alone vs p_send[c] * p_acc[c]
```

```
4000 members, 15977 connections, 1597 hidden as future; max degree 264, median 5
avg FoF candidates per member 155; FoF retrieval covers 65.0% of future connections

random FoF           recall@10 = 0.080
common neighbours    recall@10 = 0.216
Adamic-Adar          recall@10 = 0.287

ranking (top 10 per member) invites sent  accepted accept rate  to top-1% members
P(send) only                       4.44      0.80       17.9%              65.4%
P(send) x P(accept)                2.48      1.35       54.6%               1.5%
```

- FoF covers 65% of future connections — a third come from elsewhere, hence four sources. Within FoF, common neighbours is 2.7× random, and **Adamic-Adar** adds another third.
- Ranking by P(send) alone sends 65% of suggestions to the top 1% most-connected members, who accept only 18%. The **two-sided score** sends fewer invites but yields 69% more accepted connections (1.35 vs 0.80 per member). (Behaviour simulated; a real system learns both heads.)

Common mistakes

Weak	Strong
Optimises invites sent or acceptance alone	P(send) × P(accept | sent), ignore/report guardrails, long-term interaction
Privacy filtering only at display	Filters at retrieval *and* before display; fails closed
Dismissals wait for the nightly batch	Streamed suppression merged at render time

Same design, other questions

- **Groups to join** — one side is an aggregate, so predict join × stay-and-participate; big groups dominate triangles, so a diversity cap is mandatory. **CRM contact suggestions** — shared accounts matter more than mutuals; the second side is "will this contact reply".

18.5 Case study: music discovery (Spotify-style)

> "Design discovery for a music streaming service with 400 million users and 100 million tracks: personalised weekly playlists like Discover Weekly, plus a 'radio' that plays endlessly from any seed song."

1. Clarify requirements

Recognition sentence: **"The recommendation funnel used twice — once in batch, once online."**

- **Batch**: Discover Weekly is one precomputed playlist per user per week (Spotify, 2015; refreshed every Monday) — a deadline and a cache.
- **Online**: radio picks the next track from the current session, which needs per-request context.

Question	Assumption used below
Users / tracks	400M users (≈ 250M weekly active), 100M tracks, many uploads daily (pick a rate for capacity planning)
Discover Weekly	Batch with a deadline (ready before Monday) plus a serving cache
Radio	Next-track decision in ~100–200 ms using session context
Out of scope	Search, social features, licensing economics

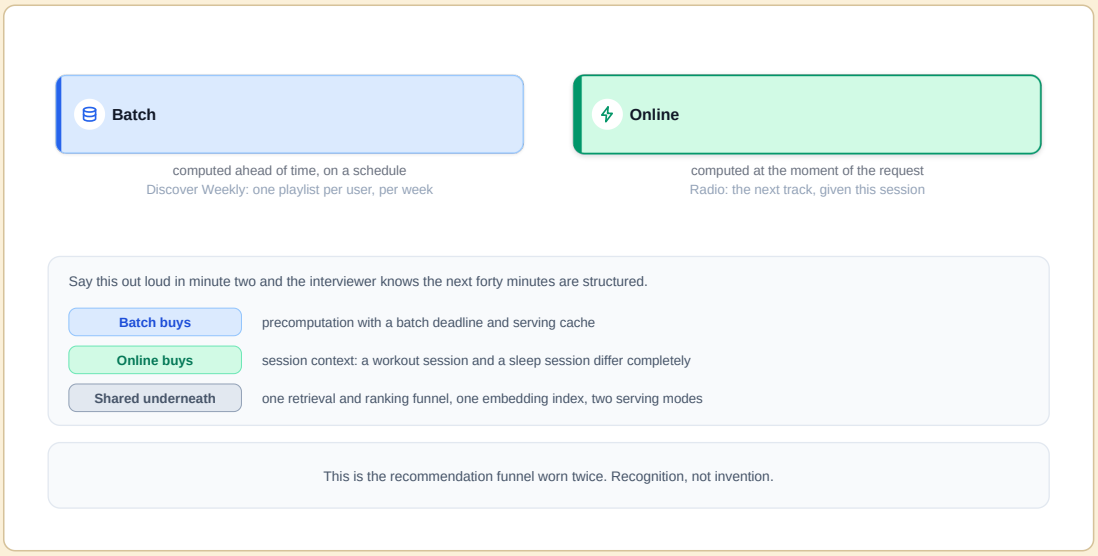


Figure — The same machinery, used as batch and as online (course diagram)

2. Frame it as an ML task

- **Discover Weekly**: retrieval + ranking of tracks the user *hasn't heard* but will like — recommendation with a novelty constraint.
- **Radio**: **sequential recommendation** — predict the next track from seed and session.
- **Labels**: listens past **30 seconds**, saves, playlist adds, artist follows = positive; an **early skip** = fast but noisy negative (interruptions, repetition, activity change).
- **Launch metric** (Discover Weekly): tracks **saved** per eligible user. **Guardrail**: **diversity and artist fairness** — discovery that keeps serving the same hits fails even when engagement looks fine.

3. Data and labels

Plays, skips (with position in the track), saves, session sequences; catalogue metadata (artist, genre, release date, explicit flag, market availability); **the audio itself**; billions of **user-made playlists** (tracks on the same playlists go together). Labels arrive fast (a skip in seconds).

4. Features

Three signal families — and what each does for **cold start**:

Signal	What it captures	New track with zero plays?
Collaborative (listening co-occurrence, matrix factorisation)	Who listens to what together	Useless until plays accumulate
Audio content embeddings (model over raw audio)	Tempo, **timbre** (texture of the sound), mood, instrumentation	Works from day zero
Text / metadata embeddings (tags, playlist titles, editorial, web text)	Genre, scene, cultural context	Works from day zero

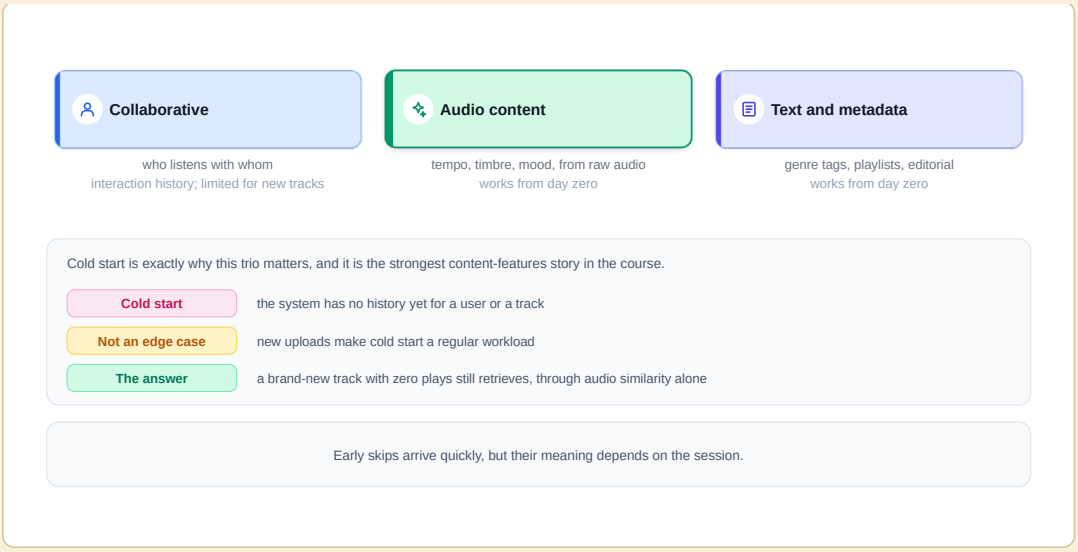


Figure — Three signal sources, and what each does at cold start (course diagram)

Radio session features: seed, last N tracks, skips this session, time, device. **Session context is first-class** — the same user wants different music at the gym and at bedtime.

5. Model: baseline → advanced

1. **Baseline**: popularity within a **cohort** (similar users, e.g. new listeners in one country who like a genre).
2. **Collaborative embeddings**: implicit-feedback MF (implicit ALS — Hu, Koren & Volinsky, 2008) or a two-tower model (Chapter 13).
3. **Content encoders**: a CNN or audio transformer over **spectrograms** (images of how sound frequencies change over time; van den Oord, Dieleman & Schrauwen, 2013, showed CNNs can predict CF vectors from audio); a text model for metadata. Map content embeddings into the collaborative space so new tracks are retrieved like established ones.
4. **Sequence model for radio**: item-to-item transitions → recurrent **GRU4Rec** (Hidasi et al., 2016) → self-attention **SASRec** (Kang & McAuley, 2018).
5. **Ranker**: GBDT or neural, with all three families plus session context.

6. Offline metrics

- **Recall@K / Hit@K** on held-out listens (Hit@K = true next item in the top K); **MRR** for radio's next track.
- **Coverage and diversity**: share of catalogue ever recommended, artist concentration (**Gini coefficient** of exposure: 0 = even, 1 = one artist gets everything), novelty.
- **Cold-start slice**: tracks released after the training cutoff.

7. Online metrics and A/B testing

- **Discover Weekly**: saves per user, 30-second-listen rate, return next week. **Radio**: skip rate, session length. **Longitudinal truth**: repeat listening of discovered tracks a month later.
- **Exploration**: a small randomised slice gives less-exposed tracks a chance — **artist-ecosystem policy** as much as ML. If it costs skip rate, bound it and judge it on a long-term holdback rather than let the short-term metric veto it.

8. Serving architecture

1. **The weekly batch runs only on weekly-active users** — scoring all 400M when ~250M are active wastes over a third of compute. Output → KV store, served from cache.
2. **Radio prefetches the next 2–3 tracks**, hiding ranking latency; a skip **re-scores the queue immediately**; a backup queue covers skip storms.
3. **Rules outside the model**: artist spacing, explicit-content filters, market licensing.

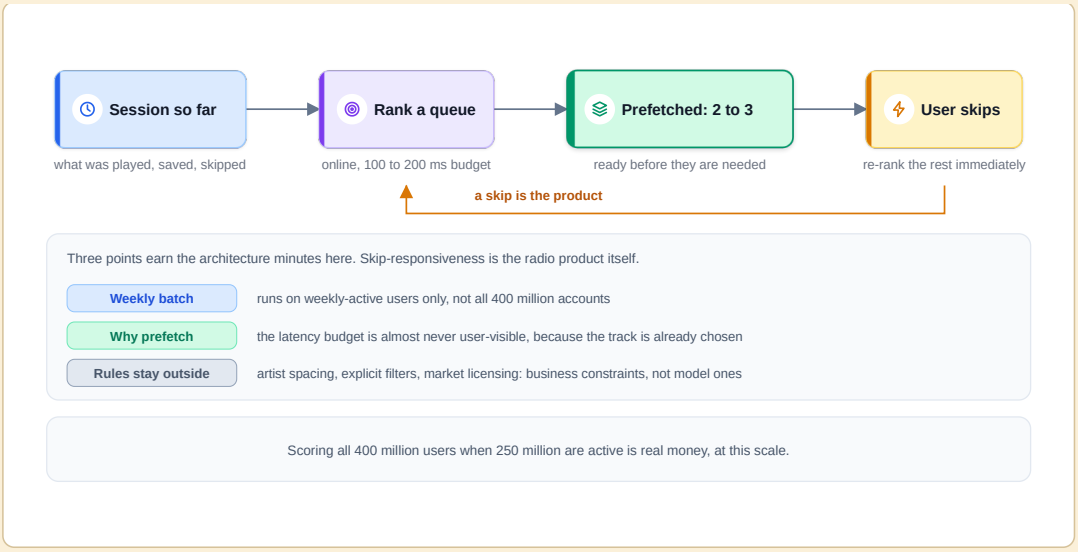
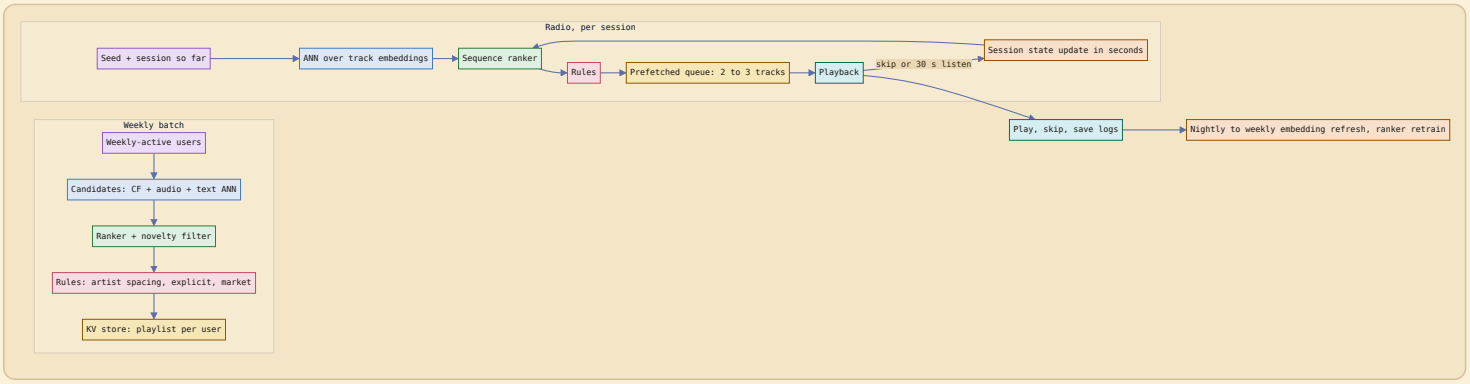


Figure — Radio prefetches the queue, so a skip re-ranks instantly (course diagram)



9. Scale numbers

Quantity	Arithmetic	Result
Weekly batch	250M users × 30 tracks	7.5B playlist entries/week — a deadline-bound compute job, not a latency problem
Waste avoided	150M inactive users not scored	~37.5% of the batch
Track embeddings	100M × 128 dims × 4 bytes	51.2 GB raw per embedding space (before index overhead and replicas)
Radio QPS	e.g. 20M concurrent listeners ÷ ~200 s per track	≈ 100K next-track decisions/s

10. Monitoring and failure modes

- **Loops**: skips update session state in seconds; embeddings refresh nightly to weekly; ranker training uses **position-bias-corrected** logs (17.3), or it copies its own previous ordering.
- **Funnel observability**: log per track, per stage — retrieved → ranked where → shown → played. If a label claims its artists are buried, show which stage dropped them.

Failure	Fallback
Weekly batch fails	Last week's playlists keep serving; alert on staleness
ANN index or ranker down during radio	Cached queue → genre-station heuristic. **Never silence**
Embedding refresh out of sync	Version index + vectors as a pair, promote atomically, check versions per request

11. Interview follow-ups

- **"Power users are bored by midweek."** → Keep the weekly ritual; add a daily shelf re-ranked on this week's listens from the same precomputed candidates — cheap freshness.
- **"Radio for a brand-new user?"** → Audio neighbours of the seed carry the session, market priors fill the ranker, the first skips teach fast — radio is the *best* cold-start surface.

Toy — session context and audio embeddings for the next track (`code/music_session_toy.py`)

2,000 tracks, each with a genre and a mood (workout/sleep/focus) encoded in a synthetic "audio embedding". Users like two genres, each session has one mood, 20% of tracks released after training. Predict a session's 4th track from its first 3 (Hit@10).

```
elif method == "session transitions":
    score = trans[prefix[-1]] + 1e-3 * plays          # item-to-item "played B right after A"
elif method == "session audio embedding":
    score = audio @ audio[prefix].mean(0)             # nearest to the session's average sound
elif method == "audio + user taste":
    score = audio @ audio[prefix].mean(0) + 0.3 * (user_genre[u][genre] > 0)
```

```
method                       Hit@10  Hit@10 on NEW tracks
popularity                    0.004                 0.000
user taste (no session)       0.020                 0.000
session transitions           0.039                 0.000
session audio embedding       0.080                 0.107
audio + user taste            0.081                 0.107
ceiling (true pool, random)   0.092                 0.073
(205 of 1000 test targets are tracks released after training)
```

- **Ignoring the session** scores 0.020 (right genres, usually wrong mood). **Session transitions** double that but, like every play-count method, score **zero on new tracks**.
- **Session audio embeddings** reach 0.080, near the ceiling of knowing the exact eligible pool (0.092), and alone find new tracks (0.107) — cold start in miniature. (Numbers are low because each pool holds ~130 equally likely tracks.)

Common mistakes

Weak	Strong
One serving mode for both products	Batch weekly playlist, online sequential radio — said in minute two
Collaborative filtering only	Audio + text embeddings so new tracks are retrievable from day zero
Engagement only	Diversity/artist-fairness guardrail and a defended exploration slice

Same design, other questions

- **Podcasts** — same funnel; transcript and speech embeddings, completion curves as labels, a higher precision bar (each pick costs an hour of trust). **News "For You"** — recency is everything, so overnight precomputation is nearly worthless: streaming signals and per-request ranking.

18.6 Case study: visual search for a second-hand marketplace

> A used-goods marketplace has about 100M active listings, each disappearing when it sells, so the catalogue turns over constantly. Shoppers snap a picture of something they like — a vintage lamp, a friend's coat — and see comparable items on sale. Second request: let people type a description to find things among listings they've saved.

1. Clarify requirements

Recognition sentence: **"The search funnel of 17.3, except the query is a photo."** The front stage becomes an **encoder** and retrieval runs in an embedding space, not an inverted index (which can search captions but not visual similarity). The hard part is **the index**: 100M listings where every sale is a deletion. The second request is a clue: if photos and text share **one embedding space**, a single index answers both.

Question	Assumption used below
Listings	100M live, millions new per day with several photos each
Latency	~300 ms photo-to-results — say whether upload and compression count
In scope	Picture-to-listing lookup; typed lookup within a user's *saved* items
Out of scope	Exact-product recognition, AR try-on, anything generative
Signature failure	Showing items already sold

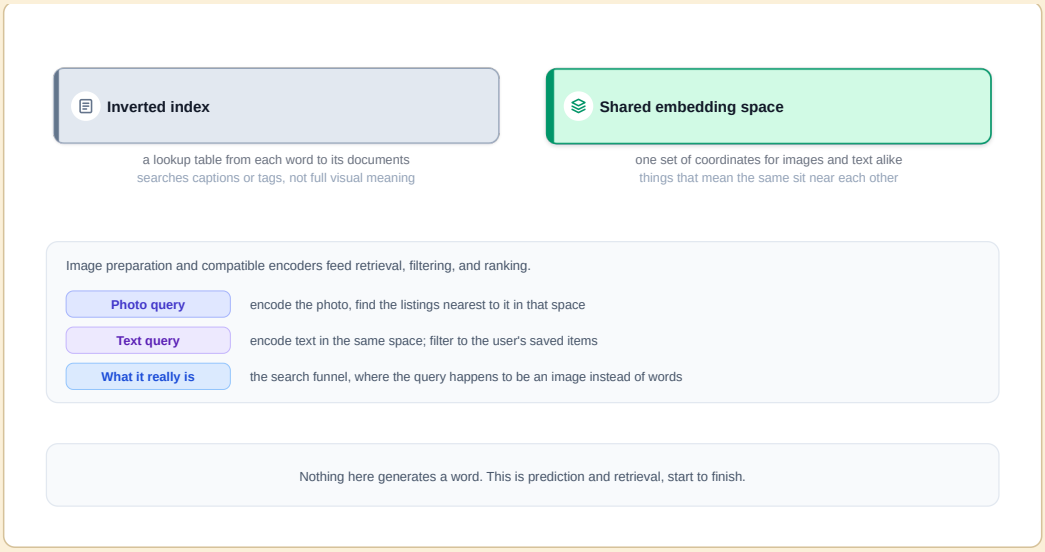


Figure — Image-text embeddings support visual retrieval beyond caption matching (course diagram)

2. Frame it as an ML task

- **Input**: query photo (or text); candidate listings (photos, title, attributes).
- **Output**: similarity in the shared space, then a purchase-likelihood ranking score.
- **Category**: **representation (metric) learning** — an embedding where distance = similarity — trained with a **contrastive loss** (pull matching pairs together, push others apart), plus nearest-neighbour retrieval and ranking.

3. Data and labels

- **Photos**: a few amateur phone shots per listing; **query photos are often worse** — include them in training and evaluation. **Text and attributes**: title, description, brand, size, condition, price, category.
- **Labels in two phases** (at launch there are no behaviour logs — the **bootstrap problem**):
  1. **Human-judged pairs**: raters mark (query photo, listing) as close match, partial or miss; scored with NDCG. The judged set becomes the permanent **regression suite** re-run on every version.
  2. **Behaviour** later: clicks, then **contact-seller** events, then transactions — position-bias corrected on the results grid (17.3).
- **Free positive pairs**: different photos of the same single-item listing (screen out bundles and stock images).

4. Features

Stage	Features
Retrieval	Query image (or text) embedding vs listing image embeddings
Filters	Category, price range, location/shipping, live status (not sold)
Ranker	Embedding similarity, **price proximity**, condition, seller rating, listing freshness, distance/shipping cost

Retrieval finds lookalikes; the ranker decides what's worth buying.

5. Model: baseline → advanced

- **Baseline**: a **pretrained image-text model**, no fine-tuning — CLIP-style (OpenAI's CLIP, Radford et al., 2021, contrastively trained on ~400M image-caption pairs). Ship it against the judged set in week one.
- **Upgrade — fine-tune on marketplace pairs** with **InfoNCE** (CLIP's contrastive loss: a softmax over similarities that should pick the true partner out of the batch):
  - positives: two photos of the same listing; photo ↔ its title/description;
  - **in-batch negatives**: other batch items — cheap, but near-duplicates become false negatives;
  - **category-hard negatives**: a *different sofa* rather than a toaster — sharpens the distinctions buyers make.
- **Light ranker** (GBDT) on structured features.
- **Query preprocessing**: a light object detector crops the dominant item (plus tap-to-select). **Composed queries** ("this jacket, but in black"): fuse image and text scores, route colour/size to filters, evaluate them explicitly.

6. Offline metrics

- **NDCG@k and recall@k** on the judged set, **per category** — an embedding can excel on sofas and quietly fail on sneakers.
- **ANN recall vs exact search** at the chosen settings. Include poor query photos, or offline numbers flatter the model.

7. Online metrics and A/B testing

- **Primary**: contact-seller rate per search. **Guardrails**: sold-item impressions (target ≈ 0), zero-results rate, latency, offensive-adjacency reports.
- **Testing an encoder needing a full re-embed**: judged-suite gate → index one category with it → **interleave** for a sliver of users → only then re-embed everything.

8. Serving architecture

Request path: upload → crop → image encoder (e.g. a 20 ms GPU budget — benchmark it) → **ANN search** with filters → light ranker → **live-status hydration** (filling in whether it has sold, just before display) → results.

ANN index families (Chapter 13.9): **IVF** — k-means buckets, a query scans the `nprobe` nearest (higher = better recall, more latency), often with **product quantisation (PQ)** compressing each vector to a few bytes (FAISS; Johnson, Douze & Jégou, 2017); **HNSW** — a layered proximity graph walked coarse to fine.

Filtered ANN: a very selective post-filter can leave too few results — fall back to more candidates, partitioned indexes, or exact search on small subsets.

The index path is the heart of this design:

- **Embed at creation**, streamed into the index within minutes — a listing unfindable for a day may already be sold.
- **Tombstone on sale** (mark deleted) and **check live status at render time** — index updates alone can't guarantee no sold items.
- **Version model and index together, promote atomically**: a new encoder against an old index returns nonsense even if dimensions match, so upgrades mean a planned, budgeted **re-embed of the corpus**, not a deploy.

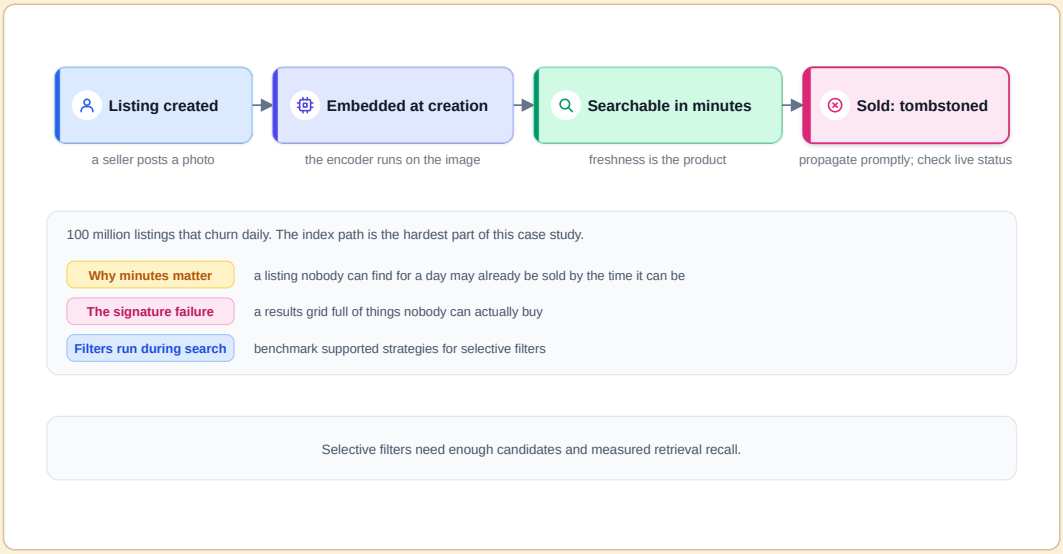


Figure — Index churn: every sale is a deletion (course diagram)

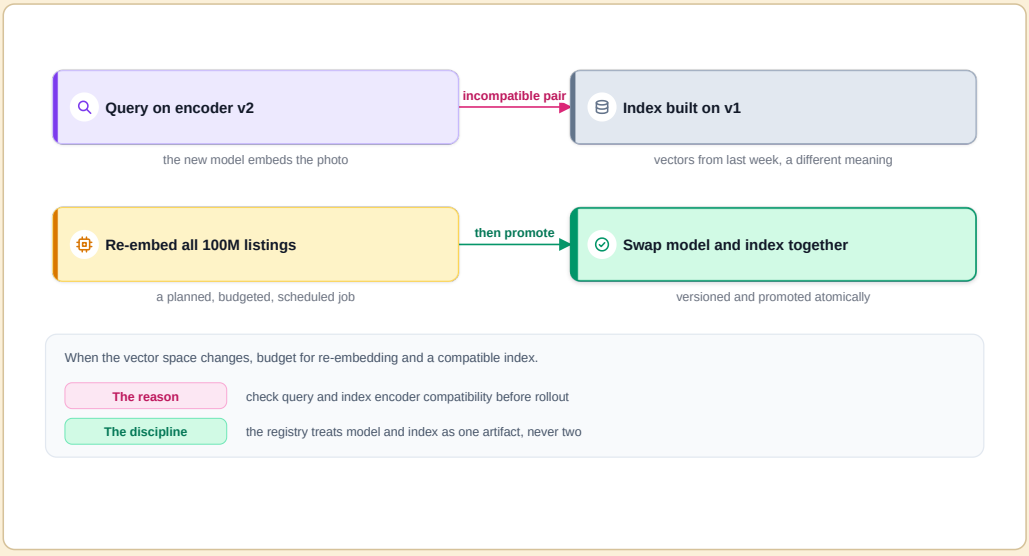
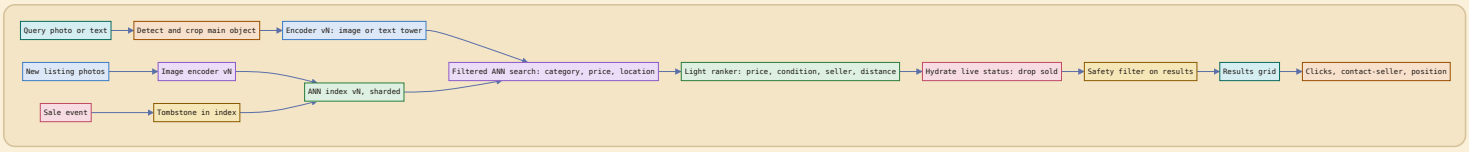


Figure — An incompatible encoder upgrade requires a compatible replacement index (course diagram)



9. Scale numbers

Quantity	Arithmetic	Result
Index memory	100M × 512 dims × 4 bytes (float32)	204.8 GB raw, before graph overhead, metadata, replicas; more if several photos per listing are indexed
With PQ compression	e.g. 64 bytes per vector	6.4 GB — at some recall cost
Encoding load	e.g. 5M new listings/day × 4 photos	20M encodings/day ≈ 230/s — possibly more than query traffic
Re-embed on upgrade	100M listings × several photos	a multi-day batch plus temporary dual-index capacity

Shard the index; a lost shard means reduced recall, backed up by text search or category browse — degrade, don't error.

10. Monitoring and failure modes

Every encoder version replays the growing judged suite; online, watch contact-seller rate **per category**. **Seasonal drift** (coat season → swimwear) means budgeted periodic retrains and index rebuilds.

Failure	Defence
Model–index **version skew**	Atomic promotion + per-query version check
Tombstone lag (sold items shown)	Render-time live-status hydration — 17.3's stale-cache rule applied to "sold"
Offensive adjacency (similar-looking but inappropriate result)	Safety filter on results, using 18.3's machinery
Counterfeits via stock photos	Near-duplicate detection against catalogue imagery at listing time

11. Interview follow-ups

- **"Why not predict a category and search inside it?"** → Classifiers break on items straddling categories (a futon: couch or bed?) while embeddings degrade gently; use the taxonomy as a filter and feature, not the retrieval.
- **"Results look right but nobody contacts sellers."** → Relevance ≠ desirability: overpriced, worn, distant or zero-history lookalikes. That's the ranker's job — measure on contact-seller rate, not clicks.

Toy — contrastive image-text space + IVF search (`code/visual_search_toy.py`)

Synthetic "photos" are noisy 64-d views of a hidden item vector plus nuisance directions (lighting, background); "descriptions" are noisy 24-d views. Two small encoders are trained with InfoNCE on 30K listings and tested on 5K *new* ones with a noisier "buyer photo" as query; then IVF over 100K listings.

```
def info_nce(a, b, temp=0.07):
    a, b = nn.functional.normalize(a, dim=1), nn.functional.normalize(b, dim=1)
    logits = a @ b.T / temp                                        # every other row = an in-batch negative
    target = torch.arange(len(a))
    return (nn.functional.cross_entropy(logits, target) + nn.functional.cross_entropy(logits.T, target)) / 2

loss = info_nce(f_img(p1[b]), f_img(p2[b])) + info_nce(f_img(p1[b]), f_txt(d[b]))   # photo-photo + photo-text

# IVF: k-means buckets (plain numpy Lloyd's, 10 iterations), scan only the nprobe nearest buckets
centers, labels = kmeans(idx_vec, 256)
buckets = np.argsort(-(centers @ q))[:nprobe]
cand = np.concatenate([lists[c] for c in buckets])
```

Output (timings vary run to run):

```
recall@10 among 5,000 listings
  raw features, photo -> listing     : 0.070
  trained encoder, photo -> listing  : 0.527
  trained encoder, text  -> listing  : 0.381   (same index, text query)

IVF over 100,000 vectors, 256 buckets (exact search: 6.99 ms/query)
  nprobe= 1: scans  0.4% of index, recall vs exact 0.270, 0.05 ms/query
  nprobe= 4: scans  1.6% of index, recall vs exact 0.596, 0.15 ms/query
  nprobe=16: scans  6.3% of index, recall vs exact 0.916, 0.62 ms/query

without a live-status check, 4.9% of returned results are already sold -> filter tombstones
```

- Raw features barely work (0.070) — lighting and clutter dominate distance; contrastive training learns to ignore them (7.5× better on unseen listings). The **same index** answers **text queries** (0.381), so the saved-items request comes almost free.
- **IVF** trades recall for speed: scanning 6.3% recovers 92% of the exact top-10 in about a tenth of the time (a Python loop; FAISS is far faster).
- ~5% of results are already sold — render-time hydration is non-negotiable.

Common mistakes

Weak	Strong
Treats the index as a static corpus	Embed at creation, tombstone on sale, hydrate live status at render
Swaps in a new encoder like a normal deploy	Versioned model+index pairs, atomic promotion, budgeted re-embed
Stops at "nearest neighbours"	A ranker with price, condition, seller, distance — lookalikes aren't purchases

Same design, other questions

- **On-device photo search** — distilled, quantised encoder and a small on-phone index; battery is the cost, nothing leaves the device. **Video moment search** — chunk videos into timestamped shots; the corpus multiplies while churn nearly disappears.

18.7 Case study: privacy blurring in street imagery

> A mapping service drives camera cars through cities and publishes 360° street-level panoramas. Before any image is published, every face and every licence plate in it must be blurred. Design the system.

1. Clarify requirements

Recognition sentence: **"Object detection where one error is a privacy breach and the other a blurred poster — recall first, and the unit that matters is the person, not the box."** It's also **batch**: nothing is served per request, so latency means capture-to-publication time.

Question	Assumption used below
Volume	5M new panoramas/day, each ~32 megapixels (8,192 × 4,096)
Objects	Faces (any angle, including tiny far ones) and licence plates (many countries' formats)
Deadline	Published within a few days of capture; nothing unblurred ever leaves the pipeline
In scope	Detection, blurring, quality audits, a viewer "report a problem" path
Out of scope	Recognising *who* someone is — the system must never identify people, only locate faces
Legal	Privacy law (e.g. the EU's **GDPR**) treats faces and plates as personal data; retention of unblurred originals is a legal decision, not the ML team's

The two errors are not symmetric:

- **Miss** (false negative) → a recognisable face or readable plate is published: harm to a real person, legal exposure, press.
- **False alarm** → a billboard face, a statue, a street-sign number blurred: mildly ugly; blurring shop signs and house numbers does reduce map usefulness — a real but smaller cost.

Say it: *"maximise recall per person; spend precision only where it hurts the product."* Google's published Street View system (Frome et al., 2009) framed it this way and reported automatically blurring about 89% of faces and 94–96% of plates on its evaluation sets at the time — why a manual **report path** is a permanent part of the design.

2. Frame it as an ML task

- **Input**: a panorama (plus neighbouring panoramas along the drive, camera **pose** — position and direction — GPS).
- **Output**: regions to blur, each from a detected box with class (face / plate) and confidence.
- **Category**: **object detection** (11.7): boxes + classes + confidences, then **NMS** to remove duplicates.
- **Objective**: at the chosen threshold, maximise **object-level recall** (every view of every person covered) subject to a budget on falsely blurred area.

3. Data and labels

- **Labelled panoramas**: boxes on faces and plates, including tiny, blurry, side-on, partly hidden and reflected ones. Guidelines matter: is a poster face a face? (Usually yes — blurring it is cheap.) Write down the minimum identifiable size.
- **Hard-case mining** (**active learning**): errors live in rare cases (night, rain, crowds, motorbike and foreign plates); send low-confidence and disagreement regions to labellers.
- **Public datasets** for pre-training: WIDER FACE (Yang et al., 2016) — large variation in scale, pose and **occlusion**, many tiny faces; plate datasets per region (e.g. CCPD, Xu et al., 2018, Chinese plates).
- **The training data is itself sensitive**: annotators see unblurred faces. Restrict and log access; delete labelled originals on schedule.

Labels from users: every "please blur this" report is a missed detection with a location — the most valuable label; confirmed reports return to training as hard positives.

4. Features

An image model — no feature table; the choices are about **what the model sees**:

Choice	Why
Tiling the panorama into overlapping perspective views (e.g. cube faces), not the raw **equirectangular** (flat, stretched world-map-style) image	Panoramas stretch objects near top and bottom; detectors are trained on normal photos
Multiple scales: normal resolution plus 2× upscaled tiles near the horizon	Far faces are 10–20 px tall — too small for default detector inputs
Overlap between tiles	An object cut by a tile edge is missed in both tiles
Neighbouring panoramas (another shot 5–10 m later)	The same person appears in 2–4 consecutive panoramas; one view is often easier
Geometry priors from camera height and horizon	A "face" in the sky or a "plate" 3 m above the road is probably a false alarm

5. Model: baseline → advanced

Baseline: off-the-shelf face and plate detectors on tiles, threshold 0.5, blur every box. The audit set shows where it misses (small, side-on, dark).

Upgrade 1 — one fine-tuned detector for both classes. A **one-stage detector** (boxes in a single pass, no region-proposal step: YOLO family, Redmon et al., 2016; or **RetinaNet**, Lin et al., 2017, whose **focal loss** down-weights the millions of easy background locations so training focuses on hard examples) fine-tuned on in-domain panoramas with tiling and multi-scale training. Transformer detectors such as DETR (Carion et al., 2020) predict a box set without NMS — worth benchmarking, but one-stage CNNs remain usual for high-resolution, high-throughput batch work.

Upgrade 2 — a cascade (18.3's idea) for recall at a sane false-alarm rate.

1. Stage 1 at a **very low threshold** (high recall, many false alarms).
2. Stage 2: a small classifier on each candidate crop *with more surrounding context* (face on a person, on a poster, or a pattern?) removes most stage-1 false alarms. Frome et al. (2009) used this shape: sensitive detectors, then a second model combining detector scores with extra features.
3. Anything stage 2 is unsure about goes to **human review** before publication.

Upgrade 3 — multi-view fusion. Track objects across consecutive panoramas using camera motion: a confident detection is projected into neighbouring views and blurred there even if the detector missed it. The unit of success becomes the *person*.

Blurring: pad each box (localisation is imperfect — see toy), use an information-destroying method (strong Gaussian blur or solid fill; light pixelation of text can sometimes be reversed), apply to the published copy only.

6. Offline metrics

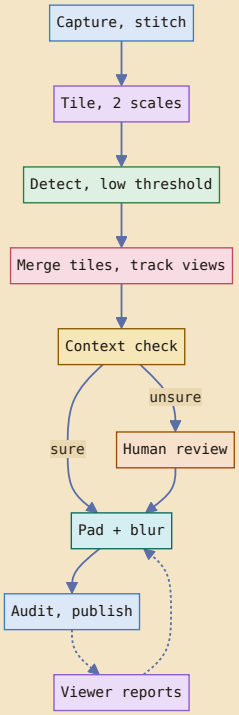
- **Recall by size bucket, class, region, lighting** at the operating threshold — overall recall hides the tiny-object failure.
- **Object-level recall**: share of physical people/plates covered in *every* view they appear in.
- **Pixel coverage**: share of each true object's area inside a blur; an object **leaks** if, say, > 5% is uncovered. Found-but-misplaced still leaks.
- **False-alarm cost**: false blurs per panorama, share of area wrongly blurred; separately, blurred **text** (signs, house numbers) — the useful content.
- **mAP** (11.7) still compares detectors but isn't the launch metric — it averages over thresholds nobody uses.

7. Online metrics and A/B testing

No A/B test on privacy — you can't publish a less-safe variant to half the users. Instead:

- **Audits**: humans review a weekly random sample of published panoramas → **missed faces and plates per 1,000 panoramas** with confidence intervals — the headline number (18.3's prevalence-by-sampling).
- **Report rate** per million panorama views, and **time to fix** confirmed reports.
- **Product guardrails**: complaints about unreadable signs; share of panoramas routed to human review (headcount).
- New models are compared on the frozen audit set and in **shadow** on new captures.

8. Serving architecture



Panoramas are cut into overlapping tiles at two scales; stage 1 runs batched on GPUs; boxes map back to panorama coordinates, merge with NMS across tiles and the seam, and propagate to neighbours by tracking; the context classifier confirms or sends to review; blurring is padded and irreversible; a weekly sample is audited. Confirmed reports are blurred by hand within hours and become hard positives, as do audit misses.

Say out loud:

- **The seam**: a 360° panorama wraps around; a face cut by the left/right edge must be merged across it.
- **Fail closed**: if detection fails or times out, the panorama is **not published** (the opposite of most serving designs).
- **Irreversibility and originals**: the published copy is destructively blurred; retention of originals is legal policy — the pipeline must make deletion possible.

9. Scale numbers

Quantity	Arithmetic	Result
Tiles	5M panoramas × 24 tiles (1,024², incl. overlap and a 2× horizon band)	120M tile inferences/day ≈ 1,400/s
GPU time	120M × 10 ms per tile (batched; benchmark it)	1.2M GPU-seconds/day ≈ 14 GPUs busy all day; ~40 with headroom for re-processing and peaks
Stage 2	~2 low-confidence candidates per panorama × 5M	10M small crops/day — cheap
Human review	0.5% of panoramas × 5M × 20 s each	25K reviews/day ≈ 140 reviewer-hours/day (~18 full-time reviewers)
Re-processing	a better model on a 10B-panorama archive at the same rate	~5.4 years on 14 GPUs → a separate, much larger budget; prioritise by views

The hidden cost: a better model helps new captures immediately, but the archive improves only if you pay to re-process it.

10. Monitoring and failure modes

- **Drift**: new countries (plate formats), camera hardware, winter clothing, night drives → audited recall **per country and camera model**; report spikes from one city are an early alarm.
- **Model upgrades** can raise mAP while lowering tiny-face recall: gate releases on per-size-bucket recall.

Failure	Defence
Detector misses tiny/side-on faces	Multi-scale tiling, low stage-1 threshold, multi-view propagation, audits by size bucket
Found but misplaced box leaks part of a face	Padding (toy: 20% padding cuts leaks from ~80% of objects to ~6%)
Face cut by a tile edge or the 360° seam	Overlapping tiles, merge across the seam
Pipeline error on a panorama	Fail closed: do not publish
Over-blurring signs and numbers	Stage 2 context classifier; measure blurred text separately

11. Interview follow-ups

- **"Can you guarantee 100% of faces are blurred?"** → No detector reaches 100%. Layer defences — high-recall detector, multi-view tracking, padding, human review, audits, a fast report-and-fix path — and state the audited miss rate honestly.
- **"Why not blur every whole person with segmentation?"** → Possible (11.7), and some operators do; it costs more of the image and compute, while faces and plates are the identifying parts. A policy choice to name, not a default.
- **"Should the model recognise people who asked to be removed?"** → No — that needs face *recognition*, far more sensitive. Handle requests by location (this house, this spot), not identity.

Toy — the recall decisions around a detector (`code/blur_recall_toy.py`)

No images: we simulate detector outputs to study the decisions around them. 13,333 physical objects each appear in 3 consecutive frames at different sizes; confidence rises with size; background patches give low-confidence false alarms; predicted box corners are off by ~8% of object size.

```
score = sig(2.2 * (np.log(h) - np.log(14)) + rng.normal(0, 1.2, h.shape))   # true objects: bigger = easier
det = score >= t
all_views = det.all(axis=1)                                    # a person is safe only if blurred in EVERY view
tracked = det | (det.any(axis=1, keepdims=True) & (rng.random(det.shape) < 0.9))   # propagate between views
bx = pred + np.stack([-pad * w_, -pad * h_, pad * w_, pad * h_], 1)                # pad the blur box
```

```
13,333 physical objects x 3 views; 21% of views are under 16 px tall

threshold  recall/view  recall <16px  FP per frame  % frame blurred by FP
      0.7        0.670         0.137          0.02                0.003%
      0.5        0.796         0.321          0.09                0.015%
      0.3        0.889         0.556          0.31                0.047%
      0.1        0.971         0.868          1.19                0.188%
     0.05        0.990         0.953          1.94                0.307%

threshold 0.1: per-view recall 0.971
  objects blurred in ALL 3 views, per-frame detection only : 0.929
  ... with box propagation between neighbouring views       : 0.992
  objects never detected in any view (tracking cannot help) : 0.001

box padding vs leaked pixels (detected objects only)
 padding    objects with >5% of pixels unblurred   blur area vs true box
      0%                                 79.8%                  0.99x
     10%                                 27.1%                  1.43x
     20%                                  6.4%                  1.94x
     30%                                  1.6%                  2.54x
```

- **The default 0.5 threshold is badly wrong**: it misses 20% of views and 68% of tiny ones. At 0.05: 99% (95% of tiny) for ~2 false blurs per frame covering 0.3% of the image — cheap. Recall-first thresholds are right when misses are the expensive error.
- **Per-view recall overstates safety**: 97.1% per view becomes 92.9% of people blurred in all 3 views. Propagating boxes lifts it to 99.2%; only people missed in *every* view (0.1%) remain.
- **Found is not covered**: with unpadded boxes, 80% of detected objects leak > 5% of their pixels. 20% padding cuts that to 6.4% at about twice the blurred area — why padding is part of the blur step.

Common mistakes

Weak	Strong
"Train a detector, use threshold 0.5"	Recall-first threshold from miss vs false-blur cost; cascade to recover precision
Reports overall recall or mAP	Recall by size, country, lighting; object-level across views; pixel coverage
Treats each image independently	Tracks people across neighbouring panoramas, propagates boxes
Blurs the exact box	Pads boxes, irreversible blur, handles tile edges and the seam
Fails open on pipeline errors	Fails closed: nothing unverified is published

Same design, other questions

- **Redacting documents or screenshots** (card numbers, names, addresses) — text-region detection + classification; same recall-first threshold, padding, fail-closed.
- **Blurring bystanders in body-camera or dash-cam video** — tracking over dozens of frames; per-person, all-frames recall matters even more.

18.8 Case study: event recommendation

> An events platform (think Eventbrite or Meetup) lists concerts, workshops, meetups and classes created by organisers. Design the "events for you" recommendations on the home page and in a weekly email.

1. Clarify requirements

Recognition sentence: **"The recommendation funnel of 17.1, except every item is new, perishable and tied to a place and time."** An event sells for a few weeks, happens once, is gone — almost never enough history for collaborative filtering on its ID. Score events by **what they are, where and when, and who else is going**.

Question	Assumption used below
Users	50M registered, 10M monthly active; many occasional (a few events a year)
Catalogue	~1M upcoming events; ~50K new per day; most live 2–6 weeks
Surfaces	Home page (online, ~200 ms) and a weekly email (batch)
Actions	Register (free RSVP or paid ticket); later, attend (check-in)
Constraints	Future, not sold out, reachable (distance, or online)
Business	Registrations, ticket revenue, and organisers' success — zero registrations loses the organiser

A **two-sided marketplace** (users and organisers, like PYMK's two sides): showing only already-popular events pleases attendees and starves new organisers.

2. Frame it as an ML task

- **Input**: user (location, history, friends), context (time, device, surface), candidate event.
- **Output**: P(register | shown), optionally P(attend | registered); score = their product for free events (no-shows waste capacity) or P(register) × price for revenue-weighted surfaces.
- **Category**: **pointwise learning-to-rank** on a **content- and context-based** model; item-ID embeddings are nearly useless given short lives.
- **Hard constraints first**: time window, distance or online, capacity, age restrictions — filters, not features.

3. Data and labels

1. **Events**: title, description, category, organiser, venue, start time, price, capacity and **remaining capacity**, online/in-person.
2. **Organisers**: past events, registration and attendance rates, ratings, followers.
3. **Users**: home (or current, with permission) location, past registrations and check-ins, categories browsed, organisers followed, sign-up interests.
4. **Social graph** (if present, or imported with consent): who's registered for what — shown to others only if they allow it.
5. **Impression logs**: event, position, surface, then registration within an observation window.

Labels:

- Registration within, say, 7 days of an impression = positive; many register days later (calendars), so use a **delayed-label window** (18.1), not same-session clicks.
- **Attendance** is known only after the event — weeks late — and only where check-in exists: a second head or sample weight, not the only label.
- **Point-in-time correctness is critical**: "registrations so far" and "friends going" must be computed **as of impression time**, not at training time when the event has filled up — the most common leakage bug here (offline AUC jumps suspiciously, online gains vanish).

4. Features

Group	Examples
User–event match	Topic affinity (category history; description text-embedding similarity to past events), price vs usual spend
Place	**Distance** or travel time to venue; same city/neighbourhood; online flag
Time	Days until event, weekday/weekend and hour vs past attendance pattern, clash with already-registered events
Social	Friends registered (count, closeness), organiser followed by user or friends
Event quality	Organiser's past attendance rate and rating, description completeness, photos
Momentum	Registrations in last 24 h, **remaining capacity**, sell-out velocity (as of impression)
Context	Surface (home vs email), device, time of day

5. Model: baseline → advanced

Baseline: rules — upcoming events within 20 km, in attended categories, sorted by registrations so far. Ships in days, a fair number to beat. Weakness: "registrations so far" is zero for every new event, so already-popular events decide the ranking (see toy).

Main model — GBDT (e.g. XGBoost, Chen & Guestrin, 2016) handles distances, counts, prices and categorical fields without heavy engineering, and generalises to never-seen events because it scores **features of the event**, not its ID. Research on event-based social networks did the same: Macedo, Marinho & Santos (2015) combined content, social, location and time signals with learning-to-rank on Meetup data because CF struggles when every event is new.

Candidate generation for 1M events:

- **Geo + time index**: events within the user's radius (a **geohash** or **S2 cell** lookup — cells of a hierarchical map grid) in the next N weeks, plus online events — 1M → a few thousand.
- **Two-tower retrieval** (13.7) with a **content-only** event tower (category, text embedding, organiser, price) — a new event has a vector the moment it's created.
- **Social source**: events friends registered for; followed organisers.
- **Exploration source**: a small quota of new events from low-history organisers so they can collect data at all.

Advanced ranker: deep model with the description embedding and multi-task heads (register, attend) — only when GBDT stops improving; the features, not the model, carry this design.

Re-ranking: spread categories and dates (not five pub quizzes the same night), an organiser-exposure floor, drop already-registered events, never show sold-out ones.

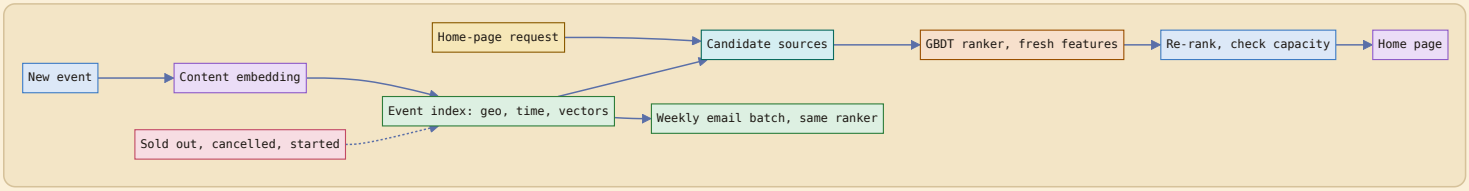
6. Offline metrics

- **Time-based split by event creation date**: train on events from weeks 1–4, test on events **created in week 5** — a random split puts the same event in train and test and flatters ID memorisation.
- **AUC / log loss** of P(register); **hit@k / NDCG@k** per user on events actually shown.
- **Slices**: new users, new organisers, zero-registration-at-impression events, online vs in-person, small towns vs big cities.
- **Calibration** if the weekly email uses a threshold ("send only if ≥ 3 events score above p").

7. Online metrics and A/B testing

- **Primary**: registrations per active user per week; ticket revenue for paid events.
- **Quality**: **attendance rate** of recommended registrations, post-event ratings.
- **Marketplace guardrails**: share of events with ≥ 1 registration, organiser retention, concentration on the top 1% of events.
- **Email guardrails**: unsubscribe and spam-report rates.
- Randomise **by user** but check effects by city — an experiment working only in the three largest cities isn't a win everywhere.

8. Serving architecture



Candidate sources = geo + time filter, content-only two-tower ANN, friends' events, followed organisers, exploration quota. Impressions, registrations and check-ins are logged for training.

- **Event lifecycle drives the index**: embed and index **at creation** (minutes, not nightly — a new event matters most in its first days); remove on sell-out, cancellation or start. Remaining capacity is **hydrated at render time** (18.6's rule).
- **Streaming features**: registrations in the last hour and remaining capacity come from a stream processor — they change fastest exactly when demand peaks.
- **Email** runs the same ranker in batch over active users weekly — one model, two serving modes (18.5).

9. Scale numbers

Quantity	Arithmetic	Result
Home-page QPS	10M MAU × ~10 visits/month ÷ (30 × 86,400 s)	≈ 40/s average; ~150/s at a weekend-evening peak
Ranker load	150 QPS × 500 candidates	75K scores/s — small for a GBDT
Event vectors	1M × 128 dims × 4 B	512 MB — fits in memory per region, no sharding
Weekly email	10M users × 500 candidates	5B scores per week in batch — a few hours on a modest cluster
New-event embedding	50K/day	< 1/s — trivial

Small by 17.1's standards: the difficulty is **correctness** (freshness, leakage, capacity), not throughput.

10. Monitoring and failure modes

- **Seasonality**: holidays, summer festivals, the week before a long weekend — compare with the same week last year, not last week.
- **New-city launches**: no local history — category popularity from similar cities plus the exploration quota.
- **Feature freshness**: a lagging capacity stream recommends sold-out events; alert on stream lag.

Failure	Fallback / effect
Ranker down	Rules baseline (nearby + category + date), still capacity-filtered
Two-tower / ANN down	Geo + time filter + followed organisers; quality drops most for sparse-history users
Capacity stream stale	Render-time check against the ticketing service; never sell a sold-out seat
Leakage in training ("registrations so far" computed late)	Point-in-time feature store (16.4)

11. Interview follow-ups

- **"Why not matrix factorisation or item-ID embeddings?"** → Each event lives a few weeks with few registrations — too little history per ID. Model users and events through transferable **features**: category, organiser, place, time, price, text.
- **"A brand-new user opens the app."** → Location plus sign-up interests; popular nearby events this week; learn from first clicks in-session.
- **"Show 'your friend is going'?"** → Only if the friend's privacy setting allows; then it's among the strongest signals (toy: 7.6% vs 1.7% register rate), but present on few impressions.
- **"Online events?"** → Distance is meaningless: a separate flag, time zone instead of distance, their own candidate source so they neither crowd out nor get buried by local events.

Toy — every item is cold (`code/event_rec_toy.py`)

A synthetic city of 4,000 users with home locations, topic tastes and price sensitivity; friends live nearby. Each week brings 300 **new** events. True P(register) rises with topic match, falls with distance and price, rises with friends registered and host quality. Train on four weeks of impressions, test on week 5's events (none in training).

```python
def true_p(u, ev, j, n_friends):
    dist = np.linalg.norm(u_loc[u] - ev["loc"][j], axis=-1)
    z = (-3.0 + 6.0 * u_topic[u, ev["topic"][j]] - 0.18 * dist + 1.2 * np.log1p(n_friends)
         - 0.03 * u_price[u] * ev["price"][j] + 0.5 * ev["host"][j])
    return sig(z), dist

for drop in [None, "distance_km", "friends_registered", "user_topic_affinity"]:     # ablations
    keep = [i for i, c in enumerate(COLS) if c != drop]
    m = HistGradientBoostingClassifier(max_iter=200, random_state=0).fit(Xtr[:, keep], ytr)
```

```
train: 240,000 impressions on 1200 past events, register rate 2.0%
test : 60,000 impressions on 300 NEW events (0 overlap with training IDs)

4.3% of test impressions have >=1 friend already registered (register rate 7.6% vs 1.7% without)

model                              AUC   hit@3  AUC, friend-going slice
random / item-ID CF (all cold)     0.496   0.204
popularity (early registrations)   0.679   0.430
GBDT, all features                 0.894   0.797                   0.845
GBDT without distance_km           0.837   0.714                   0.791
GBDT without friends_registered    0.892   0.785                   0.844
GBDT without user_topic_affinity   0.810   0.650                   0.676
```

- **Item-ID CF has nothing to work with**: every test event is new → AUC ≈ 0.5.
- **Popularity needs a warm-up**: it works only because early registrations accumulate first; at posting time it's zero for everyone.
- **Feature-based GBDT generalises to unseen events** (AUC 0.894, hit@3 0.797) — it scores *what, where, when, who*, not *which ID*.
- **Ablations** (retrain with one group removed) **price each feature group**: topic affinity costs most (→ 0.810), distance next (→ 0.837). The social feature is powerful where it exists (~4.5× the register rate) but present on 4% of impressions, so it barely moves overall AUC — measure such features on their own slice.

Common mistakes

Weak	Strong
Item-ID CF or MF	Content- and context-based model scoring events it has never seen
Ignores location and time	Distance, travel time, date fit as core features; time window and reachability as filters
"Registrations so far" computed at training time	Point-in-time features as of the impression; leakage checked
Ranks by popularity	Exploration quota and organiser-exposure floor for a healthy marketplace
Counts registrations only	Attendance, ratings and organiser success as quality and guardrail metrics

Same design, other questions

- **Restaurant or class booking** — same place-and-time constraints; longer-lived items, so CF helps more.
- **Job recommendations** — perishable postings, location and qualifications as filters, two-sided.
- **Live-stream recommendations** — happening *now*: momentum (viewers joining per minute) dominates and the index churns every few minutes.

18.9 Case study: similar listings from session embeddings

> A vacation-rental marketplace (think Airbnb) shows a "similar listings" carousel on every listing page. A guest looking at a flat in Lisbon for 12–15 June should see good alternatives. Design it.

1. Clarify requirements

Recognition sentence: **"Item-to-item retrieval: the anchor is the listing on screen, and 'similar' should mean *what guests consider as alternatives*, not just matching attributes."** Two listings can share price, size and neighbourhood yet attract different guests (design loft vs family flat); different attributes can still be substitutes. Browsing sessions record those judgements for free — learn from them (the item2vec idea of 13.6).

Question	Assumption used below
Listings	5M active over ~50K **markets** (destinations: a city or region)
Traffic	50M listing-page views/day
Latency	~100 ms for the carousel (loads alongside the page)
Hard constraints	Same destination; available for the dates; fits the guest count
Goal	Bookings — ideally the guest books *something*, even if not the anchor
Out of scope	Main search ranking (a sibling system)

A carousel that steals clicks from a listing the guest would have booked anyway is no win: the launch metric is **site-wide bookings**, not carousel clicks.

2. Frame it as an ML task

- **Input**: anchor listing; guest context (dates, guests, recent session clicks).

- **Output**: a ranked list of ~12 listings.
- **Category**: **representation learning** (distance = "considered together") + nearest-neighbour **retrieval** + a small **ranker**.
- **Embedding objective**: listings clicked close together in a session should be close; booked listings pull the session's clicks toward them.

3. Data and labels

1. **Click sessions**: listing pages opened during one search. Clean them: only clicks with meaningful **dwell time** (e.g. ≥ 30 s, so mis-taps don't count); split sessions at 30+ minute gaps.
2. **Bookings**: which listing each session ended in (a minority book).
3. **Listing attributes**: location, type (entire home, private room), price, capacity, bedrooms, amenities, photos, review score, host responsiveness.
4. **Availability calendar**: free nights — changes constantly.
5. **Carousel impression logs** (once it exists): shown, clicked, booked, position.

No labelled "similar pairs": the embedding learns **self-supervised** from session co-occurrence (like word2vec from neighbouring words); the later ranker learns from carousel impressions → clicks → bookings.

4. Features

Stage	Features
Retrieval	Cosine similarity of anchor and candidate listing embeddings
Filters	Same market, available for the dates, capacity ≥ guests, not the anchor, not hidden by host
Ranker	Embedding similarity, **similarity to the guest's session** (mean of recently clicked listings' vectors), distance to anchor, price ratio to anchor, type/capacity match, review score and count, host response rate, instant-book, listing age

Retrieval defines "alternatives"; the ranker decides which are worth booking for this guest and these dates.

5. Model: baseline → advanced

Baseline — attribute nearest neighbours: same market, type, capacity ≥ guests, price within ±30%, sort by distance to anchor. Explainable, zero training, enough to launch and start logging; misses intangible taste — style, vibe, "feels like a hotel" vs "feels like home".

Upgrade 1 — session-based listing embeddings (skip-gram with negative sampling). Sessions are sentences, listings are words: for each clicked listing predict those clicked just before and after, contrasted with random **negatives** (Mikolov et al., 2013; item2vec, Barkan & Koenigstein, 2016; §12.9). Grbovic & Cheng (2018) did this at Airbnb for similar listings and search personalisation, with two adaptations:

- **Booked listing as global context**: in sessions ending in a booking, every clicked listing also predicts the booked one — pulling embeddings toward what guests *book*, not just view.
- **Negatives from the same market**: sessions rarely leave one destination, so random negatives are nearly always from other markets and the model can minimise loss by learning *"Lisbon vs Paris"* and little else. Same-market negatives force distinctions *inside* the market — all the carousel needs, since market is a hard filter anyway (see toy).

Cold start — new listings have no clicks. Average the vectors of a few nearby existing listings of the same type and price bucket (Grbovic & Cheng's approach). Longer term, train a **content encoder** (photos, text, attributes → vector) to predict the session embedding, so every new listing has a vector at publication.

Upgrade 2 — a ranker on top. GBDT on the ranker features, trained on carousel impressions with **booking** as main label (clicks auxiliary). Dates, price and quality enter here — a similar listing at double the price or with 3-star reviews shouldn't lead.

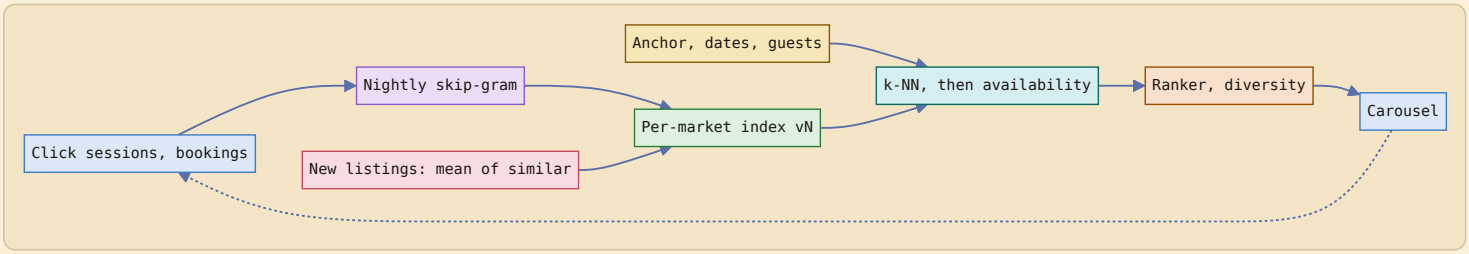
6. Offline metrics

- **Next-click hit rate@K** on held-out sessions: from a clicked listing, is the next click among the K most similar (after serving's filters)?
- **Booking hit rate@K**: from the session's first click, is the eventually booked listing in the K most similar? Closer to the goal.
- **Sanity checks**: neighbours should share the market (unfiltered), usually the type and similar price; plot a 2-D projection per city. Embeddings that pass metrics but fail the eye test usually have a data bug.
- **Slices**: big vs small markets, cold-start listings, luxury and budget ends.

7. Online metrics and A/B testing

- **Primary**: **site-wide bookings per visitor** (captures **cannibalisation** — the carousel stealing bookings the anchor would have got).
- **Secondary**: carousel CTR, carousel-attributed bookings.
- **Guardrails**: anchor-listing bookings (a big drop = distraction), host exposure concentration, latency, empty-carousel rate.
- Randomise by **guest**; check marketplace-wide effects (hosts losing exposure) by market too.

8. Serving architecture



Skip-gram with booked-listing context and same-market negatives; k-NN fetches ~200 neighbours in the anchor's market; the calendar service removes unavailable or too-small ones; a GBDT ranker (similarity, session similarity, price ratio, quality) plus a host-diversity rule picks ~12.

- **Per-market indexes**: search never crosses a market, so one small index per market; most hold hundreds to a few thousand listings — exact search suffices; only the largest need ANN (13.9).
- **Availability is the expensive filter**: fetch 200, check the calendar, rank. In peak season many neighbours are booked — fetch more when the filter empties the list (18.6's filtered-ANN issue).
- **Versioning**: a retrained skip-gram yields a *new, unrelated* (possibly rotated) space. Swap vectors and index together atomically (18.6), or **warm-start** each night's training from the previous vectors to keep neighbours stable.
- **Precompute for popular anchors**: nightly top-200 neighbour lists, cached; only availability filtering and ranking per request.

9. Scale numbers

Quantity	Arithmetic	Result
Carousel QPS	50M page views/day ÷ 86,400	≈ 580/s average, ~1,700/s at a 3× peak
Vectors	5M × 32 dims × 4 B	640 MB — the whole catalogue fits in memory on each server
Largest market	e.g. 60K listings × 32 dims	~2M multiply-adds per exact k-NN query — well under a millisecond
Training pairs	e.g. 300M sessions/month × 8 clicks × 4 context listings	~10B pairs/month — a few hours per nightly run on a small cluster, or incremental training
Availability checks	1,700/s × 200 candidates	340K listing-date lookups/s → cache calendars; precompute "available next 30 days" **bitmaps** (one bit per night)

32 dimensions suffice: the space only separates listings *within* a market.

10. Monitoring and failure modes

- **Neighbour stability**: overlap of each listing's top-10 between yesterday's and today's model; a sudden drop = training bug or rotated space served with an old index.
- **Cold-start share**: fraction of carousel slots filled by averaged (not learned) vectors, and their booking rate.
- **Seasonality**: summer vs winter sessions differ (beach houses vs ski chalets); train on a recent window; watch per-market hit rate.

Failure	Fallback / effect
Embedding job fails	Keep yesterday's vectors and index (both, as a pair)
Calendar service slow	Precomputed availability bitmaps; confirm at booking
Empty carousel after filters (peak season)	Widen: more neighbours, then attribute nearest neighbours, then nearby markets
Model–index version skew	Atomic promotion; per-request version check

11. Interview follow-ups

- **"Why not just attributes?"** → They miss taste; sessions capture what guests treat as substitutes — style, neighbourhood feel, kind of trip — with no labelling. Attributes remain filters, ranker features and the cold-start fallback.
- **"Why same-market negatives if you filter by market anyway?"** → The filter is exactly why: with random negatives the model spends capacity separating markets (which the filter already does) instead of ordering listings within one.
- **"Personalise the carousel?"** → Add the guest's **in-session** vector (mean of recent clicks) as a ranker feature (Grbovic & Cheng used similar in-session features for search ranking); long-term preferences (always books whole homes) from booking history.
- **"Relation to search ranking?"** → Same embeddings, different anchor (query + dates vs anchor listing): share the embedding pipeline, keep separate rankers.

Toy — skip-gram listing vectors, market negatives, cold start (`code/listing_embeddings_toy.py`)

20 synthetic cities × 100 listings with hidden attributes (price tier, style, size). A session stays in one city and clicks 8 listings near the guest's hidden taste; 30% end in a booking. Skip-gram (window 2, 5 negatives) three ways, evaluated on held-out sessions with the same-city filter serving would apply.

```
n = torch.randint(0, N, (len(b), neg))                          # catalogue-wide negatives
if same_city_neg:                                               # half the negatives from a's own city
    local = city_t[a][:, None] * PER + torch.randint(0, PER, (len(b), neg // 2 + 1))
    n = torch.cat([n[:, : neg - local.shape[1]], local], 1)
loss = (-logsigmoid((va * emb_out(p)).sum(1)).mean()               # pull real co-clicks together
        - logsigmoid(-(va[:, None] * emb_out(n)).sum(2)).mean())   # push negatives apart

# cold start: a new listing = mean vector of the 3 closest known listings in its city by attributes
near = cand[np.argsort(((obs[cand] - obs[i]) ** 2).sum(1))[:3]]
E[i] = E[near].mean(0)
```

```
2000 listings in 20 cities; 20,000 training sessions, 2,000 held out

                                          top-10 in          after same-city filter
variant                                   same city next-click HR@10  booking HR@10
random vectors                                0.054            0.100          0.075
a) skip-gram, catalogue-wide negatives        1.000            0.337          0.310
b) + same-city negatives                      0.957            0.380          0.387
c) + booked listing as global context         0.969            0.378          0.383

cold start: 200 listings treated as new (no clicks); next-click HR@10 starting from them
  random vector                         0.089
  mean of 3 similar nearby listings     0.326
  (vector learned from real clicks      0.353)
```

- **Sessions alone learn geography perfectly**: with catalogue-wide negatives 100% of top-10 neighbours share the city — nobody told the model about cities.
- **Same-city negatives trade that for better within-city order**: next-click 0.337 → 0.380, booking 0.310 → 0.387 after the filter. A few neighbours now come from other cities (95.7% same city) — irrelevant, since serving filters by market.
- **Booked-listing context adds nothing here** (0.378 vs 0.380) — honestly, because in this toy the booked listing is a random clicked one with no extra signal. In real logs bookings differ from clicks (guests click attractive photos, book good value), which is why the technique exists; test it on your data.
- **Cold start by averaging** recovers most of the quality: 0.326 vs 0.353 learned, far above random (0.089).

Common mistakes

Weak	Strong
Similarity by attributes only	"Considered together" from sessions; attributes as filters, features, cold-start fallback
Random negatives from the whole catalogue	Same-market negatives so the space orders listings inside a market
Ignores new listings	Averaged neighbour vectors on day one, a content encoder later
Optimises carousel clicks	Site-wide bookings at launch; anchor bookings as cannibalisation guardrail
Forgets dates	Per-request availability filter; fetch more neighbours in peak season
Retrains embeddings like a normal deploy	Version vectors and index together; monitor neighbour stability

Same design, other questions

- **"Customers also viewed"** — same session embeddings; categories play markets, stock replaces availability.
- **Next-song or next-video "up next"** — sessions become sequences to *predict* (18.5's radio), not just co-occurrence to embed.
- **Hotel search personalisation** — listing vectors as ranker features, compared with the guest's in-session vector.

Comparison across all thirteen case studies (Chapters 17–18)

Case study	Task type	Key features	Model (baseline → advanced)	Main metric (offline / online)	Hardest challenge
Video recommendations (17.1)	Retrieval + learning-to-rank	Watch history, item embeddings, user×item similarity, session state	Popularity → two-tower ANN + co-occurrence → GBDT / deep multi-objective ranker	Recall@K, AUC / watch time, return rate	Cold start and position bias; clickbait proxy trap
Feed ranking (17.2)	Multi-task ranking over a social candidate pool	Viewer–author closeness, content embeddings, live engagement counts, integrity scores	Chronological / P(like) LR → shared-body multi-task net + value formula	Per-head AUC + calibration / sessions per user or worthwhileness, with hide/report guardrails	Many goals without rewarding bait; ~87K ranking calls/s
Search ranking (17.3)	Query understanding + learning-to-rank	Query–item match (BM25, embedding similarity, attributes), item quality, capped personalisation	BM25 + popularity → hybrid retrieval, L1 GBDT → L2 LambdaMART/deep, cross-encoder on top-k	NDCG@k / session success, revenue per search, reformulation	Position-biased clicks; revenue vs relevance
Text-to-video search (17.4)	Cross-modal retrieval + learning-to-rank	Per-channel match (title, ASR, OCR, visual), channel availability, satisfied-watch rate, freshness	BM25 on metadata → + ASR → + CLIP-style text–frame embeddings (video + segment) → learned fusion ranker	Recall@K per channel, NDCG@10 / searches ending in a satisfied watch	Indexing cost; missing or misleading text; moments inside long videos
Ad click prediction (18.1)	Binary classification with calibrated probabilities feeding an auction	Sparse user/context/ad IDs, crosses, advertiser history, streaming counters	LR → GBDT+LR / deep CTR, pCTR+pCVR heads, calibration layer	Log loss, calibration by segment / RPM, advertiser ROI	Calibration per segment; delayed conversions; 100K QPS in 50–100 ms
Fraud detection (18.2)	Imbalanced binary classification + cost-based action policy	Velocity counters, impossible travel, device and graph links	Rules + LR → GBDT + graph features (GNN for rings)	PR-AUC, expected $ loss / fraud bps, false-decline rate	Adaptive adversary; weeks-late and censored labels
Harmful content (18.3)	Multi-label multimodal classification in a cost cascade	Text/image/audio/OCR embeddings, hashes, account history, share velocity, weighted reports	Hashes + rules → fast per-modality classifiers → heavy multimodal/LLM → humans	Per-category precision/recall / prevalence of violating views, false-removal & overturn rates	Harm both ways at huge scale; reviewer capacity; evasion
People You May Know (18.4)	Link prediction + two-sided ranking	Mutual count, Adamic-Adar, shared affiliations, accept history, graph embeddings	Mutual-count sort → two-head P(send)×P(accept) over FoF+affiliation+contacts+ANN candidates	Recall of future edges / accepted invites per impression, "don't know" reports	Two-sided objective; privacy fail-closed; graph-scale batch; rich-get-richer
Music discovery (18.5)	Batch + online sequential recommendation	Collaborative vectors, audio embeddings, metadata/playlist text, session context	Cohort popularity → MF/two-tower + audio encoder → sequence model (GRU4Rec/SASRec) for radio	Recall/Hit@K, diversity / saves, skip rate, long-term repeat listening	New-track cold start; skip-responsive radio; diversity and artist fairness
Visual search (18.6)	Representation learning + ANN retrieval + ranking	Image and text embeddings, price proximity, condition, seller quality, live status	Pretrained CLIP-style → contrastive fine-tune with hard negatives + light ranker	Judged NDCG per category / contact-seller rate, sold-item impressions	Index churn (sold = deleted), encoder upgrades needing re-embeds
Privacy blurring (18.7)	Object detection (batch), recall-first	Tiled multi-scale panoramas, neighbouring views, geometry priors	Off-the-shelf detectors → fine-tuned one-stage detector at low threshold → context classifier + multi-view tracking + human review	Recall by size/region, object-level recall, pixel coverage / audited misses per 1,000 panoramas, report rate	Misses are privacy breaches; tiny faces; per-person recall across views; no A/B tests
Event recommendation (18.8)	Content- and context-based pointwise ranking	Topic affinity, distance, date fit, friends going, organiser quality, momentum (as of impression)	Nearby-popular rules → GBDT → content two-tower retrieval + deep multi-task (register, attend)	AUC, hit@k on newly created events / registrations per user, attendance, events with ≥1 registration	Every item cold and perishable; point-in-time leakage; two-sided marketplace
Similar listings (18.9)	Item-to-item representation learning + ranking	Session-embedding similarity, in-session vector, price ratio, distance, availability, quality	Attribute k-NN → skip-gram on click sessions (booked context, same-market negatives) → GBDT ranker on bookings	Next-click / booking hit rate / site-wide bookings, anchor cannibalisation	Learning 'substitutes' without labels; cold-start listings; availability filtering

Patterns that repeat — learn once, reuse everywhere:

- **Funnels and cascades**: cheap high-recall stage → expensive precise stage (17.1–17.4, 18.3, 18.5–18.9).

- **Rules outside the model**: re-rank rules, integrity layers, eligibility filters, fraud rules, action ladders, availability and capacity filters — policy changes without retraining.
- **Biased and late labels**: position bias (search, recs, visual grids), delayed conversions (ads, PYMK acceptance, event registrations), weeks-late chargebacks and attendance, censored blocks, selective appeals.
- **Two-sided or multi-goal objectives**: value formulas (feeds), bid × pCTR (ads), P(send) × P(accept) (PYMK), organiser exposure (events), costs of both error types (fraud, moderation, blurring).
- **Fail-safe defaults decided in advance**: yesterday's index, light-ranker order, segment CTR tables, risk-based timeout policy, privacy fail-closed (PYMK, blurring), genre station.
- **Perishable items and render-time checks**: sold listings, sold-out events, unavailable dates, taken-down videos — filter in the index *and* re-check when the page is built (17.4, 18.6, 18.8, 18.9).

Chapter TL;DR

Across all thirteen: name the shape, tie the ML objective to the business goal, be honest about labels, keep policy outside the model, decide failure behaviour in advance. One-line reminders — **ads**: the probability is priced, calibrate per segment; **fraud**: rules + GBDT, two cost thresholds, late censored labels; **moderation**: flag rate = compute, review threshold = headcount, prevalence of *views*; **PYMK**: P(send) × P(accept), privacy fails closed; **music**: one funnel, two serving modes, audio for new tracks; **visual search**: perishable index, version model and index together; **blurring**: recall per person and pixel, fail closed; **events**: features not IDs, point-in-time; **similar listings**: skip-gram on sessions with same-market negatives, launch on site-wide bookings.

Chapter 19 — Responsible ML: Fairness, Explainability, Privacy and Security

Every chapter so far asked "is the model accurate?". This one asks what decides whether a model *should ship*: Is it fair to every group it touches? Can we explain its decisions? Does it leak the private data it learned from? Can someone fool, poison or steal it? Which laws apply? A credit model that quietly rejects qualified applicants from one neighbourhood, or a medical model that reveals who was in its training set, is *broken* however good its AUC — and these failures have measurable symptoms and concrete fixes.

What you'll learn: where bias enters; fairness definitions, why they can't all hold, and measuring them in code; mitigation (reweighing, fairness penalties, group thresholds); explainability (permutation importance, partial dependence, exact SHAP, LIME); privacy (why "anonymised" often isn't, k-anonymity, differential privacy, federated learning); security (FGSM, poisoning, extraction, inversion, membership inference, generative-AI misuse); model cards, datasheets, regulation (**GDPR**, the EU AI Act, US rules), real incidents and a pre-launch checklist.

All code is in `code/` (one script per section), figures in `img/`. Every output is real (Python 3.14, scikit-learn 1.9, PyTorch 2.14, NumPy 2.5). The optional libraries `shap`, `lime` and `fairlearn` are not installed in the book environment, so their snippets are marked **(not run)**; each has a runnable from-scratch equivalent.

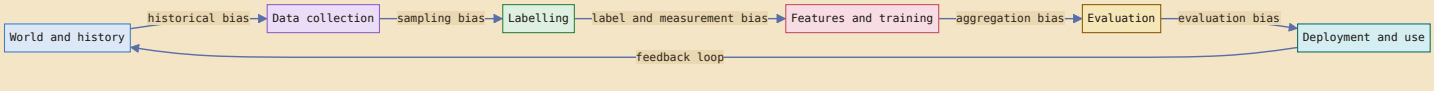


Figure — bias can enter at every stage of the ML lifecycle, and deployment feeds back into the next round of data.

19.1 Where bias comes from

What it is

Here **bias** means a *systematic* difference in how a model treats or performs for different groups that the task doesn't justify (not the statistical bias of the bias–variance trade-off, Chapter 6). A **protected** or **sensitive attribute** is a characteristic anti-discrimination law or ethics says shouldn't drive decisions — race, sex, age, religion, disability. A **group** shares a value of it. A model learns only by watching past decisions: if they were unfair, or it watched only one kind of customer, it reproduces that — at scale, with an air of objectivity.

Why it matters

1. **Data encodes past decisions.** Hiring, lending and arrest records carry the inequalities of their time: a résumé screener trained on "who we hired" learns "who we used to prefer".
2. **Some groups are thin in the data**, so the training loss barely notices their errors. *Gender Shades* (Buolamwini & Gebru, 2018): commercial gender classifiers erred on up to 34.7% of darker-skinned women vs at most 0.8% of lighter-skinned men.
3. **Stereotyped associations get learned and amplified.** Web-trained word embeddings place "nurse" near "she" and "programmer" near "he" (Bolukbasi et al., 2016); job recommendations, ad targeting and autocomplete built on them inherit it.

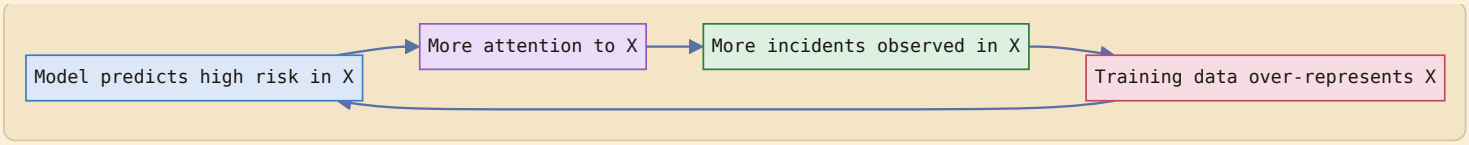
Harm (denied loans, misidentification, worse service) tends to land on the already disadvantaged; for the organisation it means legal exposure and lost trust.

How it works — a taxonomy of sources

(Suresh & Guttag's framework.)

Source	What goes wrong	Example
Historical bias	The world was unequal; even perfect data reproduces it	Past hiring favoured men, so "good hire" labels skew male
Sampling / representation bias	Groups under-sampled or missing	Face datasets dominated by lighter skin; a voice assistant trained mostly on one accent
Measurement bias	A feature or **proxy** (stand-in for something unmeasurable) is measured differently, or means different things, across groups	"Prior arrests" reflects policing intensity, not just behaviour
Label bias	The target is a flawed proxy or recorded unevenly	Health *cost* as a stand-in for health *need* (Obermeyer et al., 2019): less was spent on Black patients at the same illness level, so the algorithm under-rated their need
Aggregation bias	One model for groups whose feature–outcome relationship differs	A single HbA1c (blood-sugar test) threshold across ethnicities with different typical values
Evaluation bias	Benchmark doesn't represent users; only an overall metric reported	95% overall accuracy hiding 70% on a minority group
Deployment bias	Used for a purpose or population it wasn't built for	A risk score designed to target help used to set punishment
Feedback loops	The model's decisions shape the next training data	Predictive policing (below)

Feedback loops. Predictive policing is the textbook case (Lum & Isaac, 2016, simulated on Oakland drug-crime data): the model flags district X partly because X was patrolled more; more officers go there, more incidents are *recorded*, the next training set shows even more crime in X. Nothing checks whether X really has more crime — the data reflects where you looked. The same pattern appears in lending (never-approved people never show they'd repay — the **selective labels** problem) and recommenders (never-shown items get no clicks). Retraining more often just reinforces the loop; you need exploration or external ground truth.



Representative data is the first defence: coverage across ages, accents, skin tones, dialects, devices and regions.

Common mistakes

- "The algorithm is just maths, so it's neutral" — the data carries the bias.
- Checking only overall accuracy; bias shows per group.
- "More data fixes it" — more of the same skewed data makes the model *more confident* in the skew.
- Ignoring the label: if the target is biased (arrests, past hires, cost), no model choice rescues it.

In the interview

30-second answer: "Bias usually comes from data, not the algorithm: past decisions, thin coverage, proxy labels or features, feedback loops. I audit per-group coverage and metrics, question whether the label measures what we care about, and watch for loops after launch."

- *Hiring model trained on who managers promoted — which bias?* → Historical and label bias.
- *Speech model poor for Scottish accents?* → Sampling/representation bias (and evaluation bias if nobody measured per accent).
- *Breaking a feedback loop?* → A small randomised exploration slice, log what you *didn't* show or approve, monitor whether inputs drift toward the model's own predictions.

TL;DR

Bias enters at every stage, from history to feedback loops. Representative data and per-group evaluation come first; no algorithm fixes a biased label.

19.2 Fairness definitions and the impossibility trade-off

What it is

A **fairness criterion** is a precise statistical condition on predictions across groups. Notation: **A** group, **Y** true outcome (1 = good, e.g. repays), **Ŷ** decision (1 = approve), **S** score. TPR, FPR, FNR = 1 − TPR and PPV are the Chapter 6 confusion-matrix rates computed per group. Three families (Barocas, Hardt & Narayanan, *Fairness and Machine Learning*):

- **Independence** — the decision doesn't depend on the group.
- **Separation** — errors don't depend on the group, given the true outcome.
- **Sufficiency** — a given score means the same in every group.

Criterion	Family	Condition (equal across groups)	Plain meaning
Demographic (statistical) parity	Independence	P(Ŷ=1 | A=a)	Same approval rate in every group
Disparate impact ratio	Independence	P(Ŷ=1 | B) / P(Ŷ=1 | A) ≥ 0.8	The US "four-fifths rule" of thumb (1978 Uniform Guidelines on employee selection); e.g. 35% vs 50% → 0.70, failing
Equal opportunity	Separation (half)	TPR = P(Ŷ=1 | Y=1, A=a)	Qualified people approved at the same rate
Equalized odds	Separation	TPR *and* FPR = P(Ŷ=1 | Y=0, A=a)	Same hit rate and same false-alarm rate
Predictive parity	Sufficiency (at the threshold)	PPV = P(Y=1 | Ŷ=1, A=a)	An approval is equally reliable in each group
Calibration within groups	Sufficiency	P(Y=1 | S=s, A=a) = s for all s	A score of 0.7 means 70%, whatever the group

Also by name:

- **Individual fairness** (Dwork et al., 2012): similar individuals get similar predictions — hard because "similar" must be defined.
- **Counterfactual fairness** (Kusner et al., 2017): the prediction wouldn't change if the person's group were different, with everything causally downstream adjusted — needs a causal model.

Why it matters

"Make it fair" isn't actionable; "equalise TPR within 2 points" is. Picking a definition is a **values decision** tied to the harm:

- Hiring shortlist where **base rates** (true positive share per group) are themselves products of discrimination → **demographic parity** may be justified.
- Loans, medical screening, where missing a qualified person is the key harm → **equal opportunity**.
- Criminal justice, where false accusations and missed risks both matter → **equalized odds**.
- Risk scores read by humans (doctor, judge) → **calibration**.

How it works — the impossibility result

With **different base rates**, you generally can't have calibration *and* equal FPR *and* equal FNR unless the model is perfect (Kleinberg, Mullainathan & Raghavan, 2016; Chouldechova, 2017 — prompted by COMPAS, §19.10). Chouldechova's identity, for base rate p:

FPR = p / (1 − p) · (1 − PPV) / PPV · (1 − FNR)

If PPV and FNR are equal across groups but p differs, FPR *must* differ.

`code/impossibility.py` builds scores **perfectly calibrated by construction** (outcome drawn with probability = score) for two groups with base rates 0.6 and 0.4:

```
groups = {"A": rng.beta(6, 4, n), "B": rng.beta(4, 6, n)}   # mean score 0.6 vs 0.4
data = {g: (s, (rng.random(n) < s).astype(int)) for g, s in groups.items()}
# ... one shared threshold 0.5, then group thresholds chosen so FPR = 0.20 in both
```

```
group  base_rate  calib(0.7-0.8 bin)  FPR    FNR    PPV   selection
  A      0.601      0.746->0.748     0.624  0.171  0.667  0.747
  B      0.399      0.737->0.728     0.171  0.623  0.594  0.253

Group thresholds chosen so FPR = 0.20 in both groups:
  A: threshold=0.673  FPR=0.200  FNR=0.574  PPV=0.762
  B: threshold=0.483  FPR=0.200  FNR=0.582  PPV=0.582
```

Both groups are calibrated (scores near 0.74 come true ~74% of the time), yet one shared threshold gives FPR 0.62 for A and 0.17 for B. Group thresholds equalise FPR (and here FNR), but PPV splits, 0.76 vs 0.58 — an approval "means" different things per group. Identity check for A at 0.5: 0.601/0.399 × 0.333/0.667 × 0.829 ≈ 1.506 × 0.499 × 0.829 ≈ **0.623**, matching the measured 0.624.

Trade-offs

If you enforce…	You typically give up…
Demographic parity	Calibration and accuracy when base rates truly differ; may approve less-qualified people in one group
Equalized odds	Calibration / predictive parity; usually needs group-specific thresholds
Calibration	Equal error rates when base rates differ
Any constraint	Some overall accuracy (often small, §19.4)

When not to use a given metric

- Demographic parity when base-rate differences are real and legitimate (disease prevalence by age).
- Equal opportunity when labels are biased — you'd equalise against a distorted target.
- The 4/5 rule as a legal **safe harbour** — it's only a screening heuristic.

Common mistakes

- One fairness number without saying which definition.
- Comparing tiny groups — 10-point gaps can be noise; report confidence intervals.
- Forgetting **intersectional** groups (older women), where gaps are often largest.

In the interview

> **30-second answer:** "Criteria fall into independence (demographic parity), separation (equal opportunity, equalized odds) and sufficiency (predictive parity, calibration). With different base rates, calibration and equal error rates can't both hold (Kleinberg; Chouldechova), so I pick the criterion matching the harm, document why, and report the rest."

- *Loan approvals?* → Usually equal opportunity plus calibration checks; confirm with legal and domain experts.
- *Is fairness hopeless, then?* → No; trade-offs must be chosen explicitly, and a more accurate model shrinks all gaps.

TL;DR

Parity = equal approval rates; equal opportunity = equal TPR; equalized odds = equal TPR and FPR; calibration = scores mean the same everywhere. With unequal base rates they conflict: choose by harm, report the rest.

19.3 Measuring fairness in code

What it is

A **fairness audit** computes confusion-matrix rates per group and compares them, making gaps visible and trackable like accuracy. It needs the true label, the decision (and score) and the group — even if the model never sees the group.

How it works

1. Hold out a test set including the sensitive attribute (collected with consent for auditing).
2. Per group: n, base rate, selection rate, TPR, FPR, PPV, accuracy.
3. Gaps: demographic-parity difference, disparate-impact ratio, equal-opportunity difference, equalized-odds difference (larger of the TPR and FPR gaps), predictive-parity difference.
4. Per-group calibration, and **proxies**: how well do other features predict the group?

Dataset. `code/credit_data.py` generates 20,000 synthetic applicants whose true repayment depends *only on finances*, with two injected biases: group B has lower average income (historical inequality), and 20% of B applicants who repaid are *recorded* as defaults (label bias). `zip_region` correlates strongly with group — a proxy.

```
       income_k  zip_region  y_true      y
group
0        58.048       0.145   0.534  0.534
1        45.310       0.796   0.425  0.340
```

Core of `code/fairness_metrics.py`:

```python
def group_report(y, y_hat, group):
    rows = []
    for g in np.unique(group):
        m = group == g
        yt, yp = y[m], y_hat[m]
        tp = np.sum((yt == 1) & (yp == 1)); fp = np.sum((yt == 0) & (yp == 1))
        fn = np.sum((yt == 1) & (yp == 0)); tn = np.sum((yt == 0) & (yp == 0))
        rows.append({"group": "AB"[g], "n": m.sum(), "base_rate": yt.mean(),
                     "selection_rate": yp.mean(), "TPR": tp / (tp + fn),
                     "FPR": fp / (fp + tn), "PPV": tp / (tp + fp),
                     "accuracy": (tp + tn) / m.sum()})
    return pd.DataFrame(rows).set_index("group")

def fairness_summary(r):
    a, b = r.loc["A"], r.loc["B"]
    return {"demographic_parity_diff": a.selection_rate - b.selection_rate,
            "disparate_impact_ratio": b.selection_rate / a.selection_rate,
            "equal_opportunity_diff": a.TPR - b.TPR,
            "equalized_odds_diff": max(abs(a.TPR - b.TPR), abs(a.FPR - b.FPR)),
            "predictive_parity_diff": a.PPV - b.PPV}

# the model is "group-blind": FEATURES excludes `group`
model = make_pipeline(StandardScaler(), LogisticRegression()).fit(train[FEATURES], train.y)
```

```
Group report vs RECORDED labels (what an auditor can see):
         n  base_rate  selection_rate    TPR    FPR    PPV  accuracy
group
A     4245      0.538           0.516  0.709  0.292  0.739     0.709
B     1755      0.349           0.281  0.436  0.198  0.542     0.675
  demographic_parity_diff  +0.235
  disparate_impact_ratio   +0.544
  equal_opportunity_diff   +0.273
  equalized_odds_diff      +0.273
  predictive_parity_diff   +0.197

Same predictions vs the hidden TRUE labels:
       base_rate  selection_rate    TPR    FPR
group
A          0.538           0.516  0.709  0.292
B          0.435           0.281  0.439  0.159

AUC for predicting group from the 'group-blind' features: 0.886
calibration A: (0.13->0.13) (0.30->0.33) (0.50->0.53) (0.70->0.73) (0.89->0.89)
calibration B: (0.12->0.14) (0.30->0.30) (0.50->0.45) (0.69->0.59) (0.87->0.66)
```

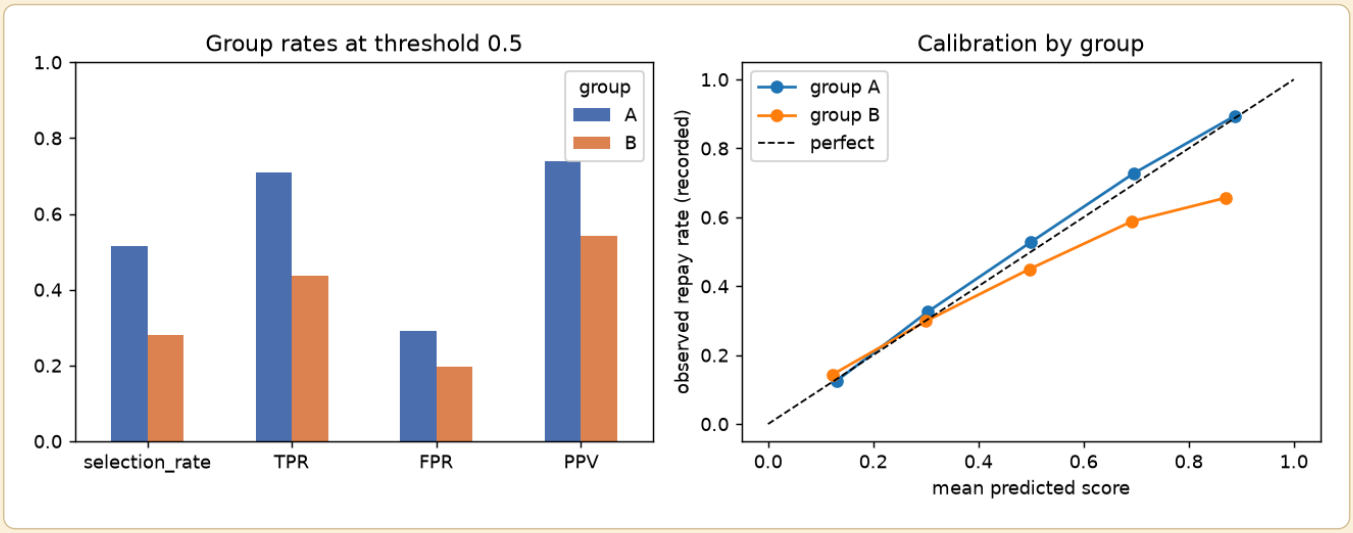


Figure — left: selection rate, TPR, FPR and PPV per group at threshold 0.5. Right: calibration curves; group B's sags below the diagonal at high scores.

1. **Every criterion fails.** B approved 28% vs 52% (ratio 0.54, far below 0.8); qualified B applicants approved 44% vs 71%.
2. **"Group-blind" didn't work.** The features predict `group` with AUC 0.886, mostly via `zip_region`. Dropping the sensitive column (**fairness through unawareness**) doesn't remove its influence — and stops you measuring it.
3. **Label bias is invisible to the auditor**: recorded B base rate 0.349, truth 0.435. Only knowing *how labels were produced* reveals artefactual "defaults".

4. **Calibration differs**: a 0.89 score comes true 89% of the time for A, 66% for B — a human reading it is misled for one group.

Fairlearn's `MetricFrame` **(not run — `fairlearn` not installed)**:

```
from fairlearn.metrics import MetricFrame, selection_rate, true_positive_rate, false_positive_rate
mf = MetricFrame(metrics={"sel": selection_rate, "tpr": true_positive_rate, "fpr": false_positive_rate},
                 y_true=y_test, y_pred=y_pred, sensitive_features=group_test)
print(mf.by_group); print(mf.difference())
```

Trade-offs

Choice	Pro	Con
Collect the sensitive attribute	Lets you measure and fix bias (store separately, strict access)	Privacy risk; restricted in some jurisdictions; needs consent and access controls
Don't collect it	Simpler legally	Bias still happens through proxies — you just can't see it
Infer it (e.g. **BISG** — Bayesian Improved Surname Geocoding, surname + geography, used by US regulators)	Audits without collection	Error-prone; auditing use only

Common mistakes

- Auditing on training data instead of held-out data.
- Gaps without group sizes and confidence intervals.
- Treating recorded labels as ground truth when the labelling process is suspect.
- Checking once before launch — drift changes gaps.

Real-world examples

Fairlearn (from Microsoft) and IBM's AIF360 are open-source fairness toolkits; Google's What-If Tool visualises per-group metrics. US lenders run fair-lending analyses on underwriting models for compliance.

In the interview

> **30-second answer:** "I slice held-out data by group, compute selection rate, TPR, FPR, PPV and calibration per group and the gaps, and predict the sensitive attribute from the features to find proxies. I report group sizes and confidence intervals and keep tracking in production."

- *Finding proxies?* → Predict the group from features (AUC), check feature–group correlations, inspect importances.

TL;DR

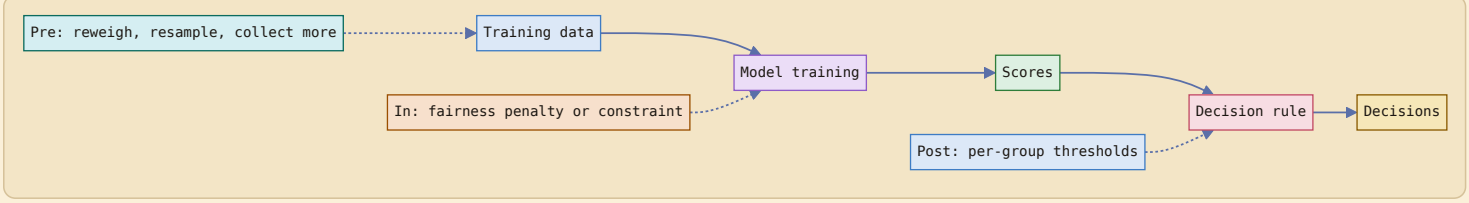
Slice metrics by group, compute gaps and per-group calibration, test for proxies, never trust "we removed the sensitive column".

19.4 Mitigating bias: before, during and after training

What it is

Bias mitigation changes the data, objective or decision rule so a chosen criterion improves at acceptable accuracy cost:

Stage	Idea	Techniques
Pre-processing (fix the data)	Change what the model learns from	Collect more data for thin groups, re-sample, **reweighing** (Kamiran & Calders, 2012), relabel suspect labels, learn fair representations, synthetic augmentation for rare groups
In-processing (fix the training)	Add fairness to the objective	**Fairness penalties / constraints** (Zafar et al., 2017), **reductions** (Agarwal et al., 2018; Fairlearn's `ExponentiatedGradient` — the constraint becomes a series of reweighted ordinary training runs), **adversarial debiasing** (Zhang et al., 2018; a second network tries to guess the group from the output and the model learns to defeat it)
Post-processing (fix the decisions)	Keep the model, change score → decision	**Group-specific thresholds** (Hardt, Price & Srebro, 2016), **reject-option classification** (flip uncertain decisions near the threshold toward the disadvantaged group)



Measurement without mitigation only documents harm. Post-processing needs no retraining (good for vendor models); pre-processing leaves training code untouched; in-processing usually gives the best accuracy–fairness trade-off but needs custom training.

How it works

Reweighing. Weight each (group, label) cell so label and group are independent in the weighted data:

$w(g, y) = P(A=g) \cdot P(Y=y) / P(A=g, Y=y)$

Cells rarer than independence predicts (group B with a good outcome) get weights > 1 (e.g. P(B)=0.3, P(Y=1)=0.5, P(B,Y=1)=0.1 → 1.5); train with `sample_weight=w`.

Fairness penalty. Loss = logistic loss + λ·(mean score of A − mean score of B)²; larger λ → more parity, less accuracy.

Threshold adjustment. Keep scores; fit a threshold per group on a *validation* set (never the test set — that overfits the adjustment to evaluation data):

1. Fix A's threshold (0.5).
2. Measure A's target rate on validation — TPR (equal opportunity) or selection rate (demographic parity).
3. For B, pick the threshold at the matching **quantile** of its scores (among its true positives for TPR, among all members for selection rate).
4. Apply both on test and re-audit.

From `code/mitigation.py`:

```
def reweighing_weights(y, g):
    w = np.empty(len(y))
    for gv in (0, 1):
        for yv in (0, 1):
            m = (g == gv) & (y == yv)
            w[m] = (np.mean(g == gv) * np.mean(y == yv)) / np.mean(m)
    return w

def fair_logreg(X, y, g, lam, epochs=300, seed=0):
    # ... torch Linear layer + Adam
        p = torch.sigmoid(logits)
        gap = p[gA].mean() - p[gB].mean()
        loss = F.binary_cross_entropy_with_logits(logits, y_t) + lam * gap ** 2

def threshold_for_rate(score, target, positives_only_mask=None):
    s = score if positives_only_mask is None else score[positives_only_mask]
    return np.quantile(s, 1 - target)           # the top `target` fraction clears it

# equal opportunity: B's threshold makes TPR_B (validation) = TPR_A
tpr_A = np.mean(s_va[(gva == 0) & (yva == 1)] >= 0.5)
tB = threshold_for_rate(s_va[gva == 1], tpr_A, yva[gva == 1] == 1)
y_eo = np.where(gte == 0, s_te >= 0.5, s_te >= tB).astype(int)
```

Output (`acc` vs recorded labels, `acc_true` vs hidden true labels; `EO_diff` = TPR gap):

```
reweighing weights  (A,y=0) (A,y=1) (B,y=0) (B,y=1): [1.123, 0.892, 0.791, 1.413]
equal-opportunity thresholds: A=0.500  B=0.356
demographic-parity thresholds: A=0.500  B=0.340
                          acc  acc_true  sel_A  sel_B  DP_diff  EO_diff  EO_diff_true
method
baseline (t=0.5)          0.704     0.701  0.532  0.269    0.263    0.290         0.291
pre: reweighing           0.703     0.704  0.501  0.349    0.153    0.155         0.159
in: DP penalty (lam=5)    0.690     0.695  0.472  0.420    0.053    0.055         0.059
post: equal opportunity   0.694     0.702  0.532  0.481    0.051    0.041         0.044
post: demographic parity  0.690     0.699  0.532  0.511    0.021    0.017         0.024
```

1. **Reweighing** roughly halves both gaps with no accuracy loss (B positives weighted 1.41).
2. **The penalty** closes the parity gap to 5 points for ~1.4 points of recorded accuracy.
3. **Threshold adjustment** closes the TPR gap 29 → 4 points by lowering B's threshold to 0.356; accuracy on *true* labels is unchanged (0.701 → 0.702) because part of the "unfairness" was label bias.
4. The accuracy cost is often **smaller than expected**, but data-dependent — measure it.

Fairlearn equivalents **(not run — `fairlearn` not installed)**:

```
from fairlearn.postprocessing import ThresholdOptimizer
from fairlearn.reductions import ExponentiatedGradient, EqualizedOdds
post = ThresholdOptimizer(estimator=base_model, constraints="true_positive_rate_parity",
                          prefit=True, predict_method="predict_proba")
post.fit(X_val, y_val, sensitive_features=g_val)
y_hat = post.predict(X_test, sensitive_features=g_test, random_state=0)

inproc = ExponentiatedGradient(LogisticRegression(), constraints=EqualizedOdds())
inproc.fit(X_train, y_train, sensitive_features=g_train)
```

Process controls matter as much as algorithms: audit who's in the data (per group and intersection) before training; collect more examples for thin groups (synthetic data only as a supplement); give humans authority to review and override high-stakes decisions; document data choices for outside audit.

Trade-offs

Method	Pros	Cons
Reweighing / resampling	Simple, model-agnostic, no group needed at prediction time	Limited control over which criterion improves
Fairness penalty / constraints	Directly targets a criterion; good trade-offs	Custom training; hyperparameter λ; can be unstable
Group thresholds	No retraining; precise; works on vendor models	Uses the group at decision time — **may be unlawful disparate treatment** in some settings (e.g. US employment law)
Better data / labels	Fixes the cause	Slow and costly

When not to use it

Don't bolt on post-processing when the real issue is a broken label (fix the label); don't deploy group thresholds without legal review.

Common mistakes

- Fitting thresholds on the test set.
- Tiny groups: noisy thresholds — pool data or use Bayesian estimates (shrinking a small group toward the population).
- Declaring victory on one metric while another worsens.

Real-world examples

LinkedIn described re-ranking for gender-representative candidate results in LinkedIn Recruiter. Fairlearn's `ThresholdOptimizer` implements Hardt et al.; AIF360 implements reweighing, adversarial debiasing and reject-option classification.

In the interview

> **30-second answer:** "Pre-processing reweighs or improves the data; in-processing adds a fairness penalty or constraint; post-processing sets per-group thresholds on validation data. Post-processing is cheap and precise but uses the attribute at decision time, with legal implications. Afterwards I re-audit every metric and the accuracy cost."

- *Which is best?* → Fix data and labels first; in-processing usually gives the best trade-off; post-processing when you can't retrain (e.g. a vendor model).
- *Accuracy cost?* → Measure it; often a point or two, especially with biased labels.

TL;DR

Pre fixes data (reweighing), in fixes the loss (penalty), post fixes the decision (group thresholds). Measure the cost, re-audit every criterion.

19.5 Explainability

What it is

Explainability (**interpretability**) is saying *why* a model made a prediction in terms a human can check. **Global** explanations describe overall behaviour (which features matter, how); **local** ones explain a single prediction (why *this* applicant was declined) — a recipe book vs why tonight's dish was salty. **Intrinsically interpretable** models are readable directly (linear models, small trees, rule lists, **GAMs** — generalised additive models); **post-hoc** methods explain a **black box** from outside.

Why it matters

- **Debugging**: spot leakage, shortcuts (spurious cues) and proxies.
- **Trust**: doctors, underwriters, judges need reasons.
- **Law**: US lenders must state principal reasons for denying credit (**ECOA/Regulation B** adverse-action notices); GDPR requires meaningful information about the logic of solely automated significant decisions (§19.9).
- **Recourse**: telling a person what would change the decision.

How it works

1. Permutation importance (global). Score the model on held-out data (e.g. ROC-AUC); shuffle one column (breaking its link to the target) and re-score; importance = the drop, averaged over shuffles. Unlike tree impurity importance (Chapter 7) it is model-agnostic and uses held-out data (training-set importance rewards overfitting).

2. Partial dependence (PDP) and ICE. $PD(x_s) = (1/n)\ \Sigma_i\ f(x_s, x_c^{(i)})$: set feature s to a value for *every* row, average predictions, repeat over a grid. **ICE** (individual conditional expectation) curves show each row separately; the PDP is their average. ICE reveals interactions the average hides.

3. SHAP (local, adds up to global). Shapley values (game theory, Shapley, 1953): features are "players" sharing the "payout" $f(x) - E[f]$; feature i gets its average marginal contribution over all orders of adding features:

$\varphi_i = \Sigma$ over subsets S not containing i of $|S|!(M-|S|-1)!/M! \cdot [v(S \cup \{i\}) - v(S)]$

$v(S)$ = expected prediction when only features in S are known (others drawn from background data); the weight is the fraction of the $M!$ orderings in which exactly S precedes i. Properties: **efficiency** (φ's sum to $f(x) - E[f]$ — e.g. baseline 0.40 + 0.10 − 0.05 + 0.20 = 0.65), **symmetry**, **dummy** (a feature that never changes the output gets 0), **additivity**. Exact computation needs 2^M subsets, so the SHAP library (Lundberg & Lee, 2017) approximates: **KernelSHAP** (any model, sampling), **TreeSHAP** (exact and fast for tree ensembles).

4. LIME (local). Ribeiro et al., 2016: perturb the input around x, weight the black box's predictions by closeness to x, fit a sparse linear model whose coefficients are the explanation. Fast, but shifts with **kernel width** (how fast closeness weight falls off) and seed.

5. Counterfactual explanations (local). "Approved if your debt ratio were 0.25 instead of 0.40" — the smallest *actionable* change that flips the decision.

`code/explainability.py` trains a histogram gradient-boosting model on the credit data (without `group`):

```
pi = permutation_importance(model, Xte, yte, scoring="roc_auc", n_repeats=10, random_state=0)
PartialDependenceDisplay.from_estimator(model, Xte, [0, 1, 4], feature_names=FEATURES,
                                        kind="both", subsample=300, random_state=0)

def exact_shapley(f, x, background):
    M = len(x)
    def v(S):                                   # expected output with features S fixed to x
        Z = background.copy(); Z[:, list(S)] = x[list(S)]
        return f(Z).mean()
    phi = np.zeros(M)
    for i in range(M):
        others = [j for j in range(M) if j != i]
        for size in range(M):
            for S in combinations(others, size):
                w = factorial(size) * factorial(M - size - 1) / factorial(M)
                phi[i] += w * (v(S + (i,)) - v(S))
    return phi, v(())

def lime_like(f, x, X_ref, n=5000, width=0.75, seed=0):
    # ... Gaussian samples around x, proximity weights, weighted Ridge on standardised offsets
```

```
test accuracy: 0.705

Permutation importance (drop in test ROC-AUC when a column is shuffled):
  income_k        0.1472 +/- 0.0044
  debt_ratio      0.0596 +/- 0.0028
  prior_defaults  0.0573 +/- 0.0035
  years_employed  0.0132 +/- 0.0019
  zip_region      0.0019 +/- 0.0010

Applicant: {'income_k': 70.9, 'debt_ratio': 0.27, 'years_employed': 10.0, 'prior_defaults': 1.0, 'zip_region': 0.0}
model score f(x) = 0.816   baseline E[f] = 0.471
Shapley values (push towards approval if +):
  income_k        +0.178
  years_employed  +0.120
  debt_ratio      +0.111
  prior_defaults  -0.090
  zip_region      +0.027
check: baseline + sum(phi) = 0.816  (equals f(x))

LIME-style local slopes (score change per +1 sd):
  income_k        +0.195
  debt_ratio      -0.089
  prior_defaults  -0.069
  years_employed  +0.055
  zip_region      -0.016
```

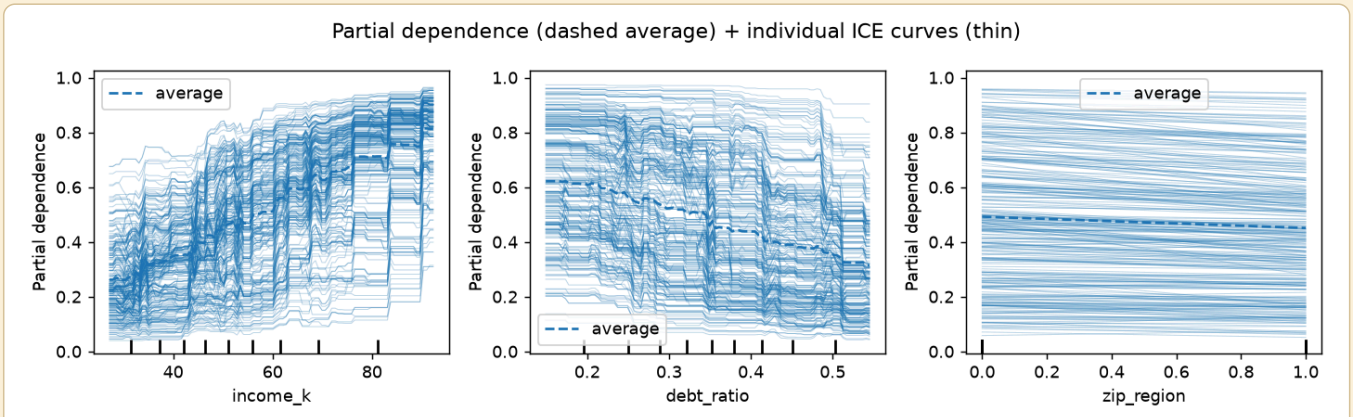


Figure — PDP (dashed) and ICE curves (thin): approval probability rises with income, falls with debt ratio, dips slightly for zip region 1.

1. Income dominates globally. `zip_region` has small permutation importance yet is a strong group proxy (§19.3) — **low importance doesn't mean "fair"**; much group signal flows through income.
2. Shapley values add up exactly: 0.471 + 0.178 + 0.120 + 0.111 − 0.090 + 0.027 = 0.816 (efficiency).
3. SHAP and LIME frame things differently but consistently: Shapley says living in zip 0 helped vs an average applicant (+0.027); LIME's slope says moving *toward* zip 1 lowers the score. (Gaussian perturbation of a 0/1 column is crude; real LIME samples categoricals properly.)

SHAP library version **(not run — `shap` not installed)**:

```
import shap
explainer = shap.TreeExplainer(model)          # exact, fast for tree ensembles
sv = explainer(X_test)                         # Explanation object, one row per sample
shap.plots.waterfall(sv[3])                    # local: one applicant
shap.plots.beeswarm(sv)                        # global: distribution of impacts per feature
```

Trade-offs

Method	Scope	Pros	Cons
Interpretable model	Both	Faithful by construction	May lose accuracy on complex data
Permutation importance	Global	Model-agnostic, held-out data	Misleading with correlated features (unrealistic shuffled rows; importance split between twins, each looking unimportant)
PDP / ICE	Global / local	Shows the shape of an effect	Assumes feature independence; averages hide interactions (use ICE)
SHAP	Local → global	Principled, additive, consistent	Costly for non-tree models; depends on background data; still correlational
LIME	Local	Fast, any model	Unstable; depends on kernel and sampling — check across seeds
Counterfactuals	Local	Actionable recourse	Many possible answers; must respect feasibility

When not to use it

- Don't present post-hoc explanations as *causal* — they explain the model, not the world.
- For high-stakes decisions consider an interpretable model first (Rudin, 2019).
- Don't show raw SHAP plots to customers; translate into plain-language reasons.

Common mistakes

- Importance on training data; ignoring correlated features; one LIME run as truth.
- Explaining the wrong output (**log-odds** — log of p/(1−p), a logistic model's raw output — vs probability).

Real-world examples

Lenders produce "reason codes" for adverse-action notices; Vertex AI, SageMaker Clarify and Azure ML offer SHAP-style attributions; TreeSHAP is built into XGBoost and LightGBM (`pred_contribs=True` / `pred_contrib=True`).

In the interview

> **30-second answer:** "Global — permutation importance on held-out data, PDP and ICE — describes the whole model. Local explains one prediction: SHAP splits it into additive contributions with game-theory guarantees; LIME fits a local linear **surrogate**. Both are correlational, so I use them for debugging and reason codes, not causal claims."

- *SHAP vs LIME?* → SHAP has axioms and consistent results; LIME is faster, less stable. TreeSHAP is exact for trees.
- *Explaining a decline to a customer?* → Top negative SHAP contributors mapped to plain-language reason codes, plus a counterfactual if possible.
- *PDP vs waterfall plot?* → PDP global; waterfall local.

TL;DR

Global: permutation importance, PDP. Local: SHAP (sums to f(x) − E[f]), LIME, counterfactuals. Explanations describe the model, not causality.

19.6 Privacy

What it is

Privacy in ML = controlling what personal information is collected, how it's used, and what the trained model can leak. **PII** (personally identifiable information) identifies a person directly (name, email, national ID) or in combination; **quasi-identifiers** (ZIP code, birth date, sex) are harmless alone but identifying together; **sensitive attributes** (health, finances, religion, biometrics) do great harm if revealed.

Why it matters

1. **Consent and ownership** — did people agree to their data training a model? If company data goes to a third-party AI service, is it used to train the vendor's models, and where can it flow?
2. **Legal duties** — GDPR, California's **CCPA/CPRA** and similar laws require transparency, a lawful basis, and rights to access, delete or opt out.
3. **Harms** — identity theft, harassment, discrimination; fines and lost trust.
4. **Models memorise** — a trained model can reveal training records (§19.7); "we only ship the model" is no guarantee.

How it works

Anonymisation pitfalls. Removing names is **pseudonymisation**, not anonymisation; GDPR still treats it as personal data (hashed user IDs included). Famous failures:

- Latanya Sweeney estimated 87% of Americans are likely unique on 5-digit ZIP + birth date + sex (1990 census), and re-identified the Massachusetts governor's hospital record by linking "anonymised" health data with a public voter roll.
- AOL's "anonymised" 2006 search logs let journalists identify users; Netflix Prize ratings were re-identified via public IMDb reviews (Narayanan & Shmatikov, 2008).
- Strava's 2018 global heatmap exposed military bases from soldiers' runs: aggregates leak too.

k-anonymity (Sweeney, 2002): every quasi-identifier combination is shared by ≥ k records, via **generalisation** (ZIP 02138 → 021**, age 29 → "≤35") and **suppression** (dropping outliers). Weaknesses led to **l-diversity** (each group has ≥ l distinct sensitive values) and **t-closeness** (each group's sensitive-value distribution close to the overall one).

Differential privacy (DP) (Dwork et al., 2006) is a guarantee about an *algorithm*: its output distribution barely changes whether or not any one person's record is in the data. M is ε-DP if for all datasets D, D' differing in one record and every output set O:

P[M(D) ∈ O] ≤ e^ε · P[M(D') ∈ O]

Small ε = strong privacy (ε = 1: any outcome at most e ≈ 2.72× likelier with you in the data).

The **Laplace mechanism** for a numeric query f:

1. **Sensitivity** Δf = the most one person can change f: a count has Δf = 1; a mean of values clipped to [L, U] over n people has Δf = (U − L)/n (e.g. [0, 100] → 100/n).
2. Add Laplace(0, b) noise with b = Δf/ε (count at ε = 0.5 → b = 2).
3. Release f(D) + noise; expected absolute error b, standard deviation √2·b.

DP properties: **composition** (k queries at ε each cost up to kε — the **privacy budget**; ten at 0.2 = 2.0) and **post-processing** (anything computed from a DP output stays DP).

`code/privacy_dp.py`:

```
def laplace_count(true_count, eps, rng):
    return true_count + rng.laplace(0.0, 1.0 / eps)           # sensitivity of a count = 1

def laplace_mean(x, lo, hi, eps, rng):
    x = np.clip(x, lo, hi)                                     # clipping bounds the sensitivity
    sens = (hi - lo) / len(x)
    return x.mean() + rng.laplace(0.0, sens / eps)

gen = df.assign(zip=df.zip.str[:3] + "**",
                age=pd.cut(df.age, [0, 35, 120], labels=["<=35", ">35"]).astype(str))
```

```
raw table: k = 1
generalised (zip prefix + age band): k = 4
zip    age   sex
021**  <=35  F                    [flu]
       >35   M      [cancer, flu, heart]

exact: count(all)=54  count(all except Bob)=53  -> Bob's status leaked: 1
DP (eps=0.5 each): 52.8 vs 48.8 -> difference +4.0 (noise sd ~ 2.8 per query)

true count(hiv) = 54,  true mean income = 61.69k
eps     noisy count (3 draws)        mean |err| count   noisy mean income
0.01       96.8,   122.2,  -155.1         96.54          62.01k
0.1        58.2,    57.6,    60.9          9.83          62.20k
1.0        54.4,    54.3,    54.6          1.06          61.56k
10.0       53.9,    54.0,    53.9          0.10          61.68k
```

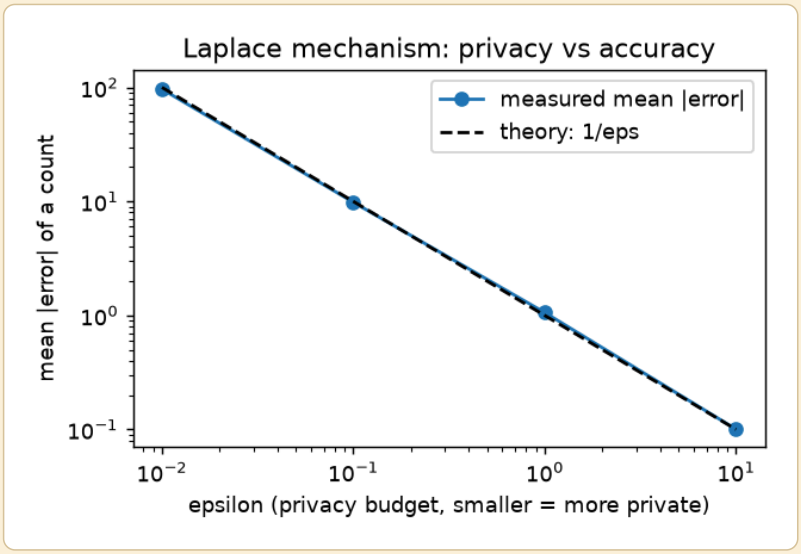
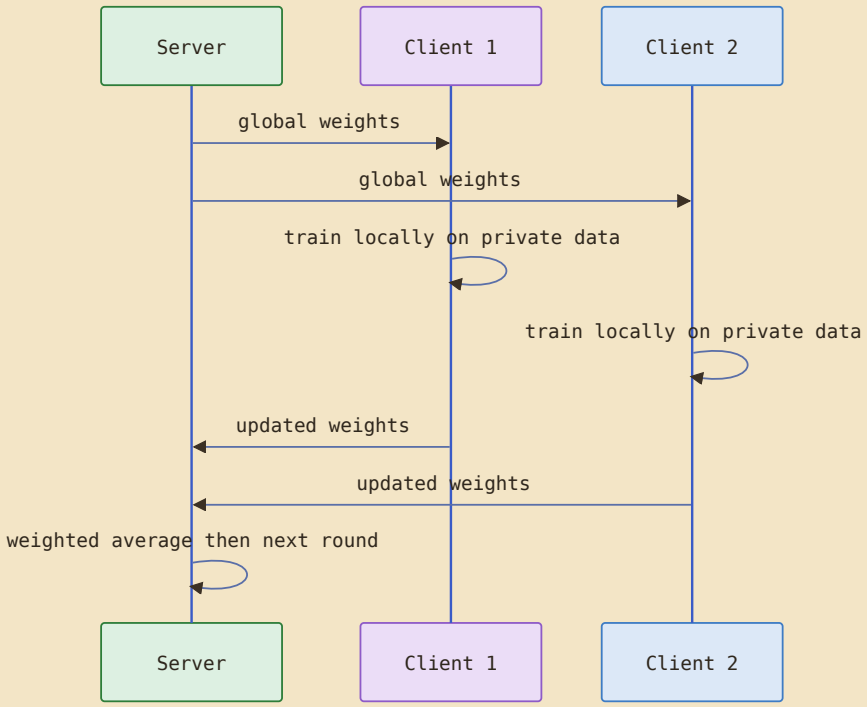


Figure — the Laplace mechanism's mean absolute error on a count follows 1/ε: ten times more privacy costs ten times more error.

1. Generalisation lifts k from 1 to 4, but everyone in "021**, ≤35, F" has flu — knowing a woman of that profile is in the table reveals her diagnosis: a **homogeneity attack**, which l-diversity guards against.
2. Two exact counts ("all patients", "all except Bob") reveal Bob's HIV status: a **differencing attack**. With DP noise the difference (+4.0) says nothing reliable.
3. The mean-income query stays accurate even at ε = 0.01 because its sensitivity is tiny (300/10,000 = 0.03). **DP is cheap for large aggregates and expensive for small groups** — the central trade-off.

DP in training. DP-SGD (Abadi et al., 2016) clips each example's gradient and adds Gaussian noise before each update (Opacus for PyTorch, TensorFlow Privacy). Costs accuracy, especially on small datasets and rare classes.

Federated learning (FL) (McMahan et al., 2017) keeps raw data on devices or at institutions and moves the *model*: the server sends global weights, each client trains a few epochs locally and returns updated weights, the server averages them weighted by client data size (**FedAvg**), repeat.



`code/federated_avg.py` (three "hospitals" of different sizes and non-identical data):

```
for rnd in range(1, 31):
    updates, sizes = [], []
    for Xc, yc in clients:                          # runs ON each client
        updates.append(local_train(w_global.copy(), Xc, yc))
        sizes.append(len(yc))
    w_global = np.average(updates, axis=0, weights=sizes)  # server: weighted average
```

```
round  1: global test accuracy = 0.786
round  5: global test accuracy = 0.815
round 10: global test accuracy = 0.823
round 30: global test accuracy = 0.826

centralised (all data pooled, 150 epochs): 0.824
client 0 alone (n=2500):           0.810
client 1 alone (n=1500):           0.767
client 2 alone (n=1000):           0.775
```

The federated model matches pooling (0.826 vs 0.824) and beats every client alone — no raw row leaves a client. Caveat: updates can leak (**gradient inversion** rebuilds training examples from shared updates), so production FL adds **secure aggregation** (server sees only the sum of updates) and often DP noise.

Trade-offs

Technique	Protects against	Cost / limits
Data minimisation, access control, encryption	Breaches, misuse	Discipline; doesn't stop model leakage
Pseudonymisation	Casual identification	Easily reversed by linkage
k-anonymity (+ l-diversity, t-closeness)	Linkage on quasi-identifiers	Loses detail; weak against background knowledge; no formal guarantee
Differential privacy	Any attacker, any side information (formal bound)	Noise reduces accuracy; budget accounting; hard for small groups
Federated learning	Raw data centralisation	Communication cost, **non-IID** client data, updates can leak — add secure aggregation/DP
Synthetic data	Sharing raw records	Can memorise real records unless generated with DP

When not to use it

- DP on tiny datasets where noise swamps signal — restrict access instead.
- Claiming "anonymous" because names were removed.
- FL when data can legally and safely be centralised — real engineering complexity.

Common mistakes

- Forgetting composition: many queries at ε = 1 add up to a weak guarantee.
- Not clipping inputs — unbounded sensitivity makes the "DP" fake.

- Raw PII in feature stores, debug logs or LLM prompts.
- Keeping training data forever "just in case" (violates GDPR storage limitation).

Real-world examples

The US Census Bureau protected 2020 census results with DP ("TopDown" algorithm). Apple uses **local DP** (noise added on-device before sending) to learn popular emoji and new words; Google's RAPPOR (2014) collected Chrome statistics with local DP. Google's Gboard trains next-word prediction with FL; hospitals form FL consortia for medical imaging.

In the interview

> **30-second answer:** "Removing names isn't anonymisation — quasi-identifiers link back to people. k-anonymity resists linkage without a formal guarantee. Differential privacy bounds any one person's influence: Laplace noise of scale sensitivity/ε, with ε budgeted across queries; DP-SGD applies it to training. Federated learning keeps data local and averages updates, with secure aggregation and DP because updates can leak."

- *A 5-anonymous table where a group shares one diagnosis?* → No l-diversity: homogeneity attack.

TL;DR

Pseudonymised ≠ anonymous. k-anonymity resists linkage but not background knowledge; DP adds noise of scale Δf/ε for a provable bound; FL moves the model, not the data.

19.7 Security: attacks on ML models

What it is

Adversarial ML studies attackers who target the model itself. A **threat model** states what the attacker wants and can do: **white-box** (knows weights and code) or **black-box** (only queries and answers); at **training time** (the data) or **inference time** (the inputs).

Attack	When	Goal
Adversarial examples (evasion)	Inference	Misclassify a crafted input (white-box via gradients; black-box via queries or a substitute model)
Data poisoning	Training	Degrade the model or plant a **backdoor** trigger
Model extraction (stealing)	Inference, via queries	Copy a paid or proprietary model
Model inversion	Inference, via queries	Reconstruct sensitive features or typical training inputs
Membership inference	Inference, via queries	Learn whether a record was in the training set
Training-data extraction	Inference, via queries	Make a generative model emit memorised training text or images

Models guard money and safety (fraud filters, malware detection, moderation, driving perception), so each attack maps to real harm: fraud slipping through, a stolen model, proof that a named person was in a cancer study.

How it works

1. Adversarial examples and FGSM. Szegedy et al. (2013) found tiny targeted perturbations flip neural-network predictions; Goodfellow, Shlens & Szegedy (2015) explained it by near-linear behaviour in high dimensions — many tiny aligned changes add up to a large logit change — and introduced the **Fast Gradient Sign Method**:

$$x_{adv} = \text{clip}(x + \varepsilon \cdot \text{sign}(\nabla_x L(\theta, x, y)), 0, 1)$$

1. Gradient of the loss w.r.t. the *input* pixels (not the weights).
2. Its sign: +1/−1 per pixel — the loss-increasing direction.
3. Move every pixel by ε (an **L∞ budget**: cap on the largest change to any pixel) and clip to the valid range.

Their famous image: a panda at 57.7% confidence becomes "gibbon" at 99.3% after an imperceptible perturbation. **PGD** (Madry et al., 2018) repeats FGSM-style steps — the standard strong attack. The main defence is **adversarial training**: include adversarial examples in each batch (cost: slower training, sometimes lower clean accuracy, robustness only within the trained budget).

`code/fgsm_attack.py` (an MLP on sklearn's 8×8 digits, pixels in [0, 1]):

```
def fgsm(model, x, y, eps):
    x = x.clone().requires_grad_(True)
    loss = nn.functional.cross_entropy(model(x), y)
    grad, = torch.autograd.grad(loss, x)
    return (x + eps * grad.sign()).clamp(0, 1).detach()

# adversarial training: each batch = clean examples + their FGSM versions
if adversarial:
    xb = torch.cat([xb, fgsm(model, xb, yb, eps)]); yb = torch.cat([yb, yb])
```

```
eps    plain-model acc   adversarially-trained acc
0.0    0.976             0.989
0.05   0.796             0.946
0.1    0.365             0.826
0.2    0.061             0.298
0.3    0.017             0.019

example: true=1  clean pred=1 (1.00)  adversarial pred=8 (1.00)
```

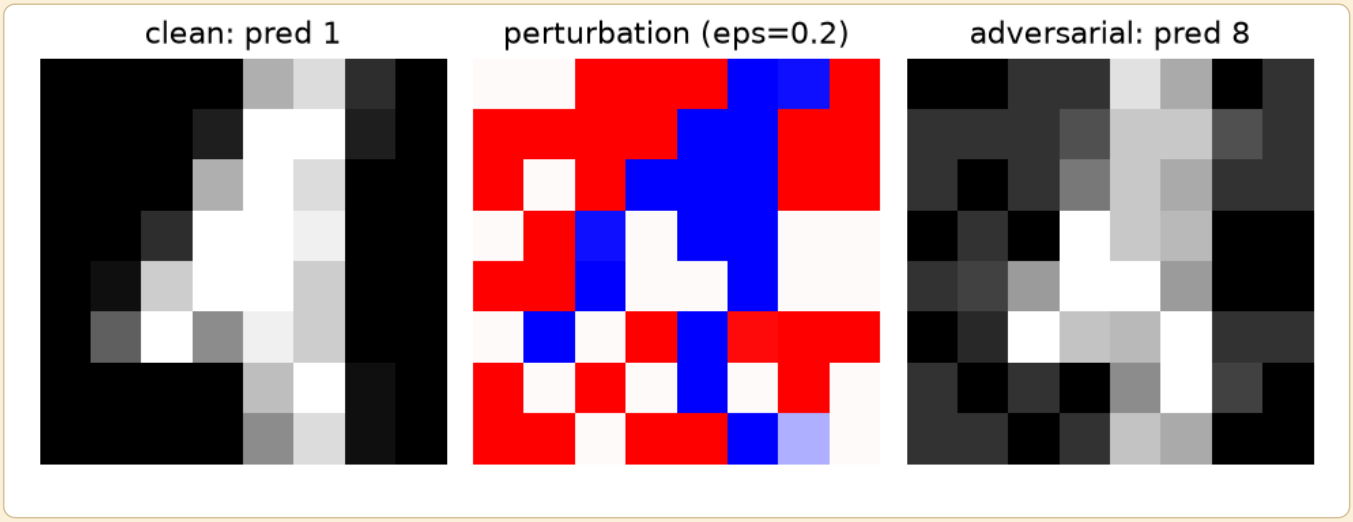


Figure — a "1" (left), the FGSM perturbation with ε = 0.2 (middle, red = brighter, blue = darker), and the result classified as "8" with full confidence (right).

A per-pixel budget of 0.1 cuts the plain model from 97.6% to 36.5%; adversarial training at ε = 0.1 keeps 82.6% but gives little protection at larger budgets it never saw. (On 8×8 images ε = 0.2 is visible; on high-resolution photos far smaller budgets suffice and are invisible.)

Physical-world versions: Eykholt et al. (2018) made stop signs misread with sticker patterns; adversarial patches and glasses frames fool face recognition.

2. Data poisoning. Label flipping changes labels; a **backdoor** (e.g. BadNets, Gu et al., 2017) adds a trigger pattern with a chosen label, so the model behaves normally until it sees the trigger. Risk grows with scraped web data, user feedback, crowd-sourced labels. Microsoft's Tay chatbot (2016) was withdrawn within a day after users deliberately taught it offensive content.

3. Model extraction. Tramèr et al. (2016): prediction APIs (System Design Fundamentals) can be copied by querying them and training a surrogate on the answers; the copy also enables white-box attacks on the original.

4. Model inversion. Fredrikson et al. (2015) reconstructed recognisable face images from a face-recognition model's confidence scores.

5. Membership inference. Shokri et al. (2017): models are more confident on training examples, so high confidence on the true label suggests membership; overfitting drives it. Carlini et al. (2021) extracted verbatim memorised training text — including personal contact details — from GPT-2 (OpenAI, 2019).

`code/attacks_demo.py`:

```python
# poisoning: random vs targeted (class 1 relabelled as 0)
y_tgt[rng.choice(ones, int(frac * len(ones)), replace=False)] = 0
# extraction: query the victim, train a copy on its answers
answers = victim.predict(Q); stolen = DecisionTreeClassifier(max_depth=10).fit(Q, answers)
# membership inference: confidence on the true label, members vs non-members
conf_in = model.predict_proba(X_in)[np.arange(len(y_in)), y_in]
auc = roc_auc_score(np.r_[np.ones(len(conf_in)), np.zeros(len(conf_out))], np.r_[conf_in, conf_out])
```

```
1) label-flip poisoning (logistic regression, test accuracy / recall of class 1)
   poisoned  0%:  random flips 0.843/0.838   targeted 1->0 0.843/0.838
   poisoned 10%:  random flips 0.846/0.835   targeted 1->0 0.841/0.778
   poisoned 20%:  random flips 0.843/0.854   targeted 1->0 0.812/0.689
   poisoned 30%:  random flips 0.840/0.835   targeted 1->0 0.769/0.574

2) model extraction - victim test accuracy 0.870
     100 queries: agreement with victim - random queries 0.659, look-alike queries 0.820
    1000 queries: agreement with victim - random queries 0.720, look-alike queries 0.892
    5000 queries: agreement with victim - random queries 0.845, look-alike queries 0.933

3) membership inference (attack = 'high confidence on the true label means member')
   overfit forest (no depth limit)  train acc 1.000  test acc 0.902  attack AUC 0.842
   regularised forest (depth 4)     train acc 0.896  test acc 0.851  attack AUC 0.556
```

Random label noise barely hurts, but **targeted** flipping of 10–30% of positives drags recall 0.84 → 0.57 — a fraud filter can be taught to miss exactly the attacker's class. **Extraction** reaches 93% agreement with 5,000 realistic queries. **Membership inference** is strong (AUC 0.84) against an overfit model and near chance (0.56) after regularisation (a depth limit): generalisation is also a privacy defence.

6. Generative-AI misuse. Deepfakes (fake video, audio or images impersonating real people for fraud, blackmail or non-consensual imagery) and cheap **misinformation at scale**. Responses: detection classifiers (an arms race), **provenance** standards such as C2PA "Content Credentials", **watermarking** (e.g. Google DeepMind's SynthID), platform policies, labelling duties such as the EU AI Act's (§19.9), media literacy.

Defences at a glance

Threat	Defences
Evasion	Adversarial training (PGD), input preprocessing (weak alone), **certified robustness** (randomised smoothing), ensembles, human review for high stakes
Poisoning / backdoors	Data provenance and access control, outlier and label-noise detection, robust training, dataset versioning, backdoor scanning
Extraction	Rate limits, authentication, labels instead of full probabilities, query-pattern monitoring, model watermarking
Inversion / membership	Regularisation, DP-SGD, limiting confidence outputs, deduplicating training data
Generative misuse	Provenance and watermarking, usage policies, abuse monitoring, **red-teaming**

Common mistakes

- Testing robustness only against weak attacks (FGSM) — use PGD and adaptive attacks.
- Relying on **gradient masking** defences that merely hide gradients; routinely broken.
- Returning full-precision probability vectors from public APIs when a label would do.
- Training on unvetted scraped or user-feedback data with no provenance.

Real-world examples

Spam and malware detection are adversarial by nature. MITRE ATLAS catalogues real attacks on ML systems; the **OWASP** Top 10 for LLM Applications lists data/model poisoning and sensitive-information disclosure among its risks.

In the interview

> **30-second answer:** "Evasion uses adversarial examples — FGSM moves each input feature by ε along the sign of the loss gradient — countered by adversarial training. Poisoning corrupts training data or plants backdoors; defend with provenance and anomaly checks. Through a query API attackers can extract, invert or probe for membership; rate limits, label-only outputs, regularisation and DP help."

TL;DR

Adversarial training vs evasion; provenance vs poisoning; rate limits and label-only outputs vs extraction; regularisation and DP vs membership inference; provenance and policy vs generative misuse.

19.8 Documentation: model cards and datasheets

What it is

- A **model card** (Mitchell et al., 2019): a short document shipped with a model — purpose, evaluation *broken down by group and conditions*, limits.
- A **datasheet for datasets** (Gebru et al., 2018; *Communications of the ACM*, 2021): how a dataset was created, what's in it, how it should (not) be used — modelled on electronics datasheets.

Most harms come from use outside a model's intended conditions; versioned documentation makes intended use, known gaps and results visible to downstream teams, auditors and regulators (the EU AI Act requires technical documentation for high-risk systems).

How it works

Model card sections (Mitchell et al.):

Section	Our credit model
Model details, intended use	Logistic regression v1.3, owner, date; pre-screening loan applications with human review, *not* employment or insurance
Factors, metrics	Groups: sex, age band, region and intersections; AUC, per-group TPR/FPR, disparate impact, calibration
Training and evaluation data	Sources and time range, known label issues; held-out 2024 applications (n = 6,000)
Quantitative analyses	Per-group metrics with confidence intervals
Ethics, caveats	Proxy features (zip region), mitigation applied, residual gaps; re-audit quarterly, not for other populations

Datasheet sections: *motivation* (why, by whom, funded by whom), *composition* (contents, gaps, sensitive data, identifiability), *collection process* (how, consent), *preprocessing and labelling*, *uses*, *distribution*, *maintenance*.

Common mistakes

- Writing it once for launch — keep it in the repo, generate metric tables in the training pipeline, version it with each release.
- Only aggregate metrics — the point is disaggregated results.
- Omitting out-of-scope uses.

Real-world examples

Hugging Face model cards (a README on every model page) and dataset cards, Google's Model Card Toolkit, and system cards published by AI labs for large language models.

In the interview

30-second answer: "A model card documents intended and out-of-scope uses, training and evaluation data, per-group metrics and limitations; a datasheet documents a dataset's motivation, composition, collection, labelling, uses and maintenance. I generate the metric sections in the pipeline and version them with the model."

TL;DR

Model cards and datasheets make intended use, data provenance and per-group performance explicit and versioned.

19.9 Regulation overview

An engineer's orientation, not legal advice. Laws change — check official texts and your legal team.

What it is and why it matters

Laws govern the **data** ML uses (privacy law) and the **systems** it produces (AI and anti-discrimination law); engineering artefacts — logs, human review, documentation, explanations — are often what compliance actually requires.

GDPR (EU General Data Protection Regulation, applies since 25 May 2018)

- Covers **personal data** of people in the EU, including processing by non-EU companies serving them. Truly anonymous data is out of scope; pseudonymised data is not.
- **Principles**: purpose limitation, **data minimisation**, accuracy, storage limitation, accountability; every use needs a **lawful basis** (consent, contract, legitimate interests…), with stricter rules for "special categories" (health, biometrics, ethnicity).
- **Rights**: access, rectification, **erasure** ("right to be forgotten"), restriction, portability, objection.
- **Article 22**: the right not to be subject to decisions based *solely* on automated processing with legal or similarly significant effects; the exceptions (contract, law, explicit consent) need safeguards — human intervention, the chance to give one's view and contest. Articles 13–15 require "meaningful information about the logic involved"; whether that's a full "right to explanation" is debated.
- High-risk processing needs a **DPIA** (data protection impact assessment); fines up to €20 million or 4% of worldwide annual turnover, whichever is higher.

EU AI Act (Regulation (EU) 2024/1689)

In force since 1 August 2024, obligations phased in; classifies AI systems by **risk**:

Tier	Examples	Obligations
Unacceptable (prohibited)	Social scoring; harmful manipulation or exploitation of vulnerabilities; untargeted scraping of face images; emotion recognition at work or school; biometric inference of sensitive traits; crime prediction purely from profiling; real-time remote biometric identification in public by police (narrow exceptions)	Banned (applied from 2 February 2025)
High risk	Safety components of regulated products (Annex I), and Annex III areas: biometrics, critical infrastructure, education, **employment** (e.g. hiring screeners), access to essential services including **credit scoring**, law enforcement, migration, justice	Risk management, data governance (including bias checks), documentation, logging, **human oversight**, accuracy and robustness, conformity assessment, registration
Transparency (limited) risk	Chatbots, AI-generated or manipulated content (deepfakes), emotion recognition	Tell people they're interacting with AI; label synthetic content
Minimal risk	Spam filters, game AI, most business ML	No new obligations (voluntary codes)

Providers of **general-purpose AI models** (e.g. LLMs) have documentation and copyright-policy duties from 2 August 2025, with extra duties for models posing systemic risk. Top fines: €35 million or 7% of worldwide turnover for prohibited practices.

Timeline update (2026). The "Digital Omnibus on AI" (Regulation (EU) 2026/1744, in force since 27 July 2026) postponed high-risk obligations to **2 December 2027** for stand-alone Annex III systems and **2 August 2028** for AI in Annex I products; added prohibitions on AI generating non-consensual intimate imagery or child sexual abuse material (from 2 December 2026); and gave generative systems placed on the market before 2 August 2026 until 2 December 2026 to meet the Article 50 duty to mark output as AI-generated. Check the consolidated text for current dates.

Other rules worth knowing

- **CCPA/CPRA (California)**: rights to know, delete, correct and opt out of the sale or sharing of personal information.
- **US anti-discrimination law**: disparate *treatment* (using a protected attribute) and disparate *impact* (a neutral rule with unequal effect and no business necessity); under ECOA/Regulation B lenders must give specific denial reasons, and the **CFPB** says complex algorithms are no excuse (Circular 2022-03).
- **NYC Local Law 144** (enforced from July 2023): automated employment decision tools for NYC jobs need an annual independent bias audit, published results and candidate notice.
- **Sector rules**: HIPAA (US health data); model-risk-management guidance for banks (US SR 11-7).
- **Voluntary frameworks**: NIST AI Risk Management Framework (2023), ISO/IEC 42001 (AI management systems, 2023).

In the interview

30-second answer: "GDPR governs personal data: lawful basis, minimisation, rights such as erasure, and Article 22 safeguards for solely automated significant decisions. The EU AI Act is risk-based: some uses are banned; high-risk systems such as credit scoring and hiring need risk management, data governance, documentation, logging and human oversight; chatbots and deepfakes carry transparency duties. In the US, fair-lending and employment law (adverse-action reasons, disparate impact, NYC's bias audits) shape design."

TL;DR

GDPR governs personal data (lawful basis, minimisation, rights, Art. 22); the EU AI Act sorts systems into risk tiers, credit and hiring high-risk; US fair-lending and employment law add their own duties.

19.10 Real-world incidents

COMPAS recidivism scores (2016)

- **What happened**: ProPublica analysed COMPAS recidivism-risk scores (by Northpointe, now Equivant) for defendants in Broward County, Florida, against two-year re-arrest. Among those who did **not** reoffend, Black defendants were labelled higher risk about twice as often as white defendants (44.9% vs 23.5%); among those who **did**, white defendants were more often labelled lower risk (47.7% vs 28.0%) — i.e. unequal error rates (FPR/FNR).
- **Response**: Northpointe argued the scores were equally predictive for both groups — a given score meant similar reoffence rates (calibration/predictive parity).
- **Lesson**: *both sides were right about their own metric*; with different base rates calibration and equal error rates can't both hold — this debate triggered the impossibility proofs (§19.2). And "re-arrest" is a proxy label shaped by policing (measurement bias).

Amazon's experimental résumé-screening tool (reported 2018)

- **What happened**: Reuters reported that from 2014 Amazon built a tool rating candidates, trained on ten years of mostly male résumés. It penalised the word "women's" ("women's chess club captain") and downgraded graduates of two all-women's colleges. Engineers removed those terms but couldn't rule out other proxies; the team was disbanded; Amazon said recruiters never used it to evaluate candidates.
- **Lesson**: historical label bias and proxies — removing explicit terms doesn't remove the pattern.

Apple Card credit limits (2019–2021)

- **What happened**: In November 2019 David Heinemeier Hansson complained his credit limit was 20× his wife's despite shared finances; Steve Wozniak reported similar. New York's Department of Financial Services (**NYDFS**) investigated the issuer, Goldman Sachs.
- **Outcome**: In March 2021 NYDFS found **no fair-lending violation** (individual credit histories explained the differences) but criticised the lack of transparency and poor customer service.
- **Lesson**: even a lawful model loses trust if it can't explain decisions; spouses with shared finances can still have very different individual credit files.

Gender Shades and the Obermeyer health-risk algorithm (§19.1) complete the set interviewers most often cite.

TL;DR

COMPAS = the impossibility result in real life; Amazon = historical bias and proxies; Apple Card = explainability and trust.

19.11 Responsible-ML checklist

Use before launch and at every major retrain.

Problem framing

- [ ] Is ML appropriate? Who could be harmed? High-risk under the EU AI Act or similar?
- [ ] Does the label measure what we care about? Which fairness criterion fits the harm (agreed with legal/domain experts)?

Data

- [ ] Datasheet: source, lawful basis, collection, labelling, known gaps; coverage audit per group and intersection.
- [ ] PII minimised and access-controlled; sensitive attributes stored separately for auditing; provenance and poisoning checks.

Model and evaluation

- [ ] Per-group metrics with confidence intervals; proxy check; mitigation chosen, cost measured, all criteria re-audited.
- [ ] Explanations reviewed for leakage and proxies; reason codes where decisions affect people.
- [ ] Robustness tests where attackers exist; membership-inference risk; DP/FL where needed.

Deployment and monitoring

- [ ] Model card with intended and out-of-scope uses; human oversight and an appeal path.
- [ ] API hardening (authentication, rate limits, minimal outputs).
- [ ] Per-group monitoring, drift and feedback-loop checks; incident and rollback plan; scheduled re-audits.

19.12 Interview Q&A

Cross-cutting questions that combine the sections above:

1. **Your model is 94% accurate. Is it fair?** → Unknown until you slice by group: selection rate, TPR, FPR, PPV and calibration per group, the gaps, and a proxy check. Accuracy hides group disparities.
2. **Explain SHAP to a product manager.** → It splits one prediction into per-feature contributions that add up from the average prediction to this one: "income added +0.18, a prior default took away −0.09".
3. **Is hashing user IDs anonymisation?** → No — pseudonymisation; linkage attacks re-identify people and GDPR still applies.

4. **Someone is scraping our pricing model's API. Risks and defences?** → Model extraction, which then enables white-box attacks. Rate limits, authentication, label-only responses, query-pattern monitoring, contractual terms.
5. **Design a fair hiring screener.** → Question whether to automate at all; define the label carefully (job performance, not past hiring decisions); audit coverage; choose a criterion (often selection-rate parity plus equal TPR) with legal input; mitigate and re-audit; give candidates reasons; keep human review; run bias audits (NYC LL 144 if applicable) and meet EU AI Act high-risk duties; monitor per-group outcomes after launch.

Chapter TL;DR

- **Bias** enters via history, sampling, measurement, labels, aggregation, evaluation, deployment and feedback loops; a biased label can't be fixed by a better model.
- **Fairness criteria** (parity, equal opportunity, equalized odds, predictive parity, calibration) conflict when base rates differ — choose by harm, audit per group, test for proxies.
- **Mitigate** before (reweighing, better data), during (penalties, constraints) or after (group thresholds); measure the cost, re-audit.
- **Explain** globally (permutation importance, PDP/ICE) and locally (SHAP, LIME, counterfactuals) — never as causal claims.
- **Privacy**: pseudonymised ≠ anonymous; k-anonymity + l-diversity; DP (noise $\Delta f/\varepsilon$, budgets compose), DP-SGD, FedAvg with secure aggregation.
- **Security**: evasion (FGSM/PGD, adversarial training), poisoning, extraction, inversion, membership inference, generative misuse.
- **Document and comply**: model cards, datasheets, GDPR Art. 22, EU AI Act risk tiers, US fair-lending and hiring-audit rules; COMPAS, Amazon, Apple Card.

Chapter 20 — Cheat Sheets and Glossary

This last chapter squeezes Chapters 1–19 into pages you can scan in a minute or two: one **cheat sheet** per topic, **decision tables** for "which algorithm / which metric for this problem?", the **50 interview questions** that come up most often with short answers, and an alphabetical **glossary** of 911 terms. Nothing here is new. Every line points back to where the idea is taught properly, written like **§6.3** (= Chapter 6, section 6.3); if a line doesn't make sense, follow the reference. Every term and abbreviation used in the book also has a one-line plain-English definition in the glossary (§20.18) — look there first when a word on a sheet is new to you.

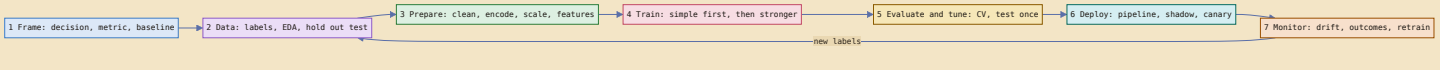
Every formula or number on a sheet that can be computed was recomputed by `code/cheatsheet_checks.py` (it also asserts the library defaults quoted here, for scikit-learn 1.9, PyTorch 2.x and NumPy 2.x):

```
OK  sheet_20_2_math
OK  sheet_20_3_api
OK  sheet_20_4_data_prep
OK  sheet_20_5_algorithms
OK  sheet_20_6_losses
OK  sheet_20_7_metrics
OK  sheet_20_8_validation
OK  sheet_20_9_deep_learning
OK  sheet_20_10_cnn_rnn_attention
OK  sheet_20_11_recommenders
OK  sheet_20_12_rl
OK  sheet_20_13_system_design
OK  sheet_20_14_production
OK  sheet_20_15_responsible
all 14 cheat-sheet checks passed
```

Chapter links: 1 · 2 · 3 · 4 · 5 · 6 · 7 · 8 · 9 · 10 · 11 · 12 · 13 · 14 · 15 · 16 · 17 · 18 · 19

Notation used throughout: x = feature vector, y = true label, ŷ = prediction, w = weights, b = bias, η = learning rate, σ = sigmoid, n = number of examples, d = number of features, k = number of neighbours / clusters / results (the context says which).

20.1 The ML workflow (§1.9, §4.10, §6.11, §15.3, §16.1)



Step	Do	Don't	Where
1. Frame	Name the decision the prediction drives, the task type (classification / regression / ranking / clustering), the metric tied to business value, and the **baseline** (current rule, human, "predict the mean")	Start by picking a model	§1.9, §15.3
2. Data	Find where labels come from and how late they arrive; set the test split aside **before** deep EDA (exploratory data analysis); check legality and privacy	Explore the test set; assume labels "exist"	§4.1, §4.2
3. Prepare	Dedupe, impossible values → NaN (the missing-value marker), impute (fill gaps), encode, scale — all inside one `Pipeline`	Fit anything on the full dataset (leakage)	§4.3–4.6, §4.10
4. Train	Linear/logistic model first, then a random forest and gradient boosting; deep nets for images/text/audio	Jump to the fanciest model	§5.1, §8.4
5. Evaluate	Cross-validate on train, tune on validation folds, report the test score **once**, next to the baseline, per segment	Tune on the test set; quote accuracy on imbalanced data	§6.1, §6.6, §6.9
6. Deploy	Ship preprocessing + model as one artifact (a single saved file); offline gate → shadow → canary → ramp, with a fallback	Re-implement features in another language (skew)	§16.2, §16.6
7. Monitor	Health → inputs → predictions → outcomes; investigate alerts; retrain through the same gates	Auto-promote on a drift alert	§16.8, §16.10

Diagnosing a model from two numbers (§1.10, §6.7):

Train error	Validation error	Diagnosis	First fixes
High	High (close)	Underfitting (high bias)	More capacity, better features, less regularisation, train longer
Low	Much higher	Overfitting (high variance)	More data, more regularisation, simpler model, early stopping, bagging
Low	Low (close)	Good fit	Test once, ship
Suspiciously perfect	Suspiciously perfect	Probably **leakage**	Check feature timestamps, splits, duplicates, top importances (§4.8)

Supervision types (§1.4): supervised (labels) · unsupervised (no labels: clusters, dimensions, anomalies) · semi-supervised (few labels + many unlabelled) · self-supervised (labels made from the data, e.g. next token) · reinforcement (rewards from actions). **Classification** predicts a category, **regression** a number (§1.5). **Batch** learning retrains on everything periodically; **online** learning updates with `partial_fit` (§1.7). **Instance-based** (k-NN) keeps the data; **model-based** keeps parameters (§1.8).

20.2 Math essentials (Ch 2)

Linear algebra (§2.1–2.3)

Idea	Formula	Worked value
Dot product	$a \cdot b = \Sigma\, a_i b_i = \|a\|\|b\|\cos\theta$	$[1,2,3]\cdot[4,5,6] = \mathbf{32}$
L2 / L1 / L∞ norm	$\sqrt{\Sigma a_i^2}$ · $\Sigma\|a_i\|$ · $\max\|a_i\|$	$\|[1,2,3]\|_2 = \sqrt{14} \approx 3.74$
Cosine similarity	$a \cdot b$ / ($\|a\|\|b\|$)	32 / (3.74·8.77) ≈ **0.975**
Linear model	$\hat{y} = w \cdot x + b$; batch: $\hat{y} = Xw + b$	shapes (n×d)(d,) → (n,)
Matrix product	(m×n)(n×p) → m×p; entry = row·column; **AB ≠ BA**	(100×20)(20×5) → 100×5
2×2 determinant / inverse	det = ad − bc; $A^{-1} = (1/\det)[[d, -b], [-c, a]]$	[[1,2],[3,4]]: det −2, inverse [[−2,1],[1.5,−0.5]]
Solve Ax = c	`np.linalg.solve(A, c)` (never `inv`)	$x_1+2x_2=5$, $3x_1+4x_2=11 \to x = [1, 2]$
Eigenvector	$Av = \lambda v$	[[2,1],[1,2]]: λ = 3 on [1,1], λ = 1 on [1,−1]
SVD	$M = U\Sigma V^T$ (rotate, stretch, rotate)	top-k singular values = best rank-k approximation
Normal equation	$\theta = (X^TX)^{-1}X^Ty$ (use `lstsq`)	Ridge: $(X^TX + \alpha I)^{-1}X^Ty$

Calculus and optimisation (§2.4–2.5)

Idea	Formula
Power / chain rule	$(x^n)' = nx^{n-1}$ · $(f(g(x)))' = f'(g(x))\cdot g'(x)$
Sigmoid and derivative	$\sigma(z) = 1/(1 + e^{-z})$; $\sigma' = \sigma(1 - \sigma)$; $\sigma(0) = 0.5$, $\sigma'(0) = 0.25$ (its maximum)
Gradient	∇f = vector of partial derivatives; points **uphill**
Gradient descent	$w \leftarrow w - \eta\nabla L(w)$. On $f = (w-3)^2$, $\eta = 0.1$: 0 → 0.6 → 1.08 (each step keeps 80% of the gap)
MSE gradient	$\nabla = (2/n)X^T(Xw - y)$
Log-loss gradient	$\nabla = (1/n)X^T(\sigma(Xw) - y)$ — "features × (prediction − truth)"
Numerical gradient check	$f'(x) \approx (f(x+h) - f(x-h)) / 2h$
Convergence (MSE)	GD converges if $\eta < 2/\lambda_{max}$ of the Hessian (§5.4)

Probability and statistics (§2.6–2.11)

Idea	Formula / rule	Worked value
Bernoulli / binomial / normal	mean p, var p(1−p) · mean np, var np(1−p) · mean μ, var σ^2	P(3 of 10, p = 0.3) = C(10,3)·0.3^3·0.7^7 ≈ **0.2668**
68–95–99.7 rule	±1σ, ±2σ, ±3σ of a normal	P(z > 2) ≈ 2.3%
Expectation rules	E[aX+b] = aE[X]+b; Var[aX+b] = a^2Var[X]; independent: Var[X+Y] = Var X + Var Y	Var[3X] = 9·Var[X]
Conditional probability	$P(A\|B) = P(A \cap B)/P(B)$	
Bayes	$P(A\|B) = P(B\|A)P(A)/P(B)$ — posterior ∝ likelihood × prior	1% disease, 90% sensitivity, 5% FPR → P(sick|+) ≈ **0.154**; a 2nd positive → 0.766
z-score	$z = (x - \mu)/\sigma$	130 with μ 100, σ 15 → 2
Standard error (CLT, central limit theorem)	$\sigma/\sqrt{n}$; 4× the data halves it	σ 20, n 100 → 2
Population vs sample std	NumPy `ddof=0` (÷n), pandas `ddof=1` (÷(n−1))	
Hypothesis test	p-value = P(data this extreme | H_0); reject if p < α; Type I = α, power = 1 − β	p is **not** P(H_0 true)
Confidence interval	estimate ± critical value × SE (1.96 for 95%, large n)	bootstrap: resample, take 2.5th–97.5th percentiles
Multiple testing	Bonferroni: use α/m	20 tests at 0.05 → expect 1 false hit
Correlation	Pearson r (linear), Spearman (ranks, monotonic); r = 0 ≠ independent	$y = x^2$ can give r = 0

Information theory (§2.12)

Idea	Formula	Worked value (bits)
Entropy	$H(p) = -\Sigma\, p \log p$	fair coin 1; 90/10 coin 0.469; p = [0.7, 0.2, 0.1] → 1.157
Cross-entropy	$H(p, q) = -\Sigma\, p \log q \geq H(p)$	q = [0.5, 0.3, 0.2] → 1.280
KL divergence	$KL(p\|q) = H(p,q) - H(p) \geq 0$, **not symmetric**	0.123
Log loss (one-hot p)	$-(1/n)\ \Sigma$ [y ln p + (1−y) ln(1−p)]	labels [1,0,1,1], p [0.9,0.2,0.6,0.99] → **0.2123** (nats)

20.3 NumPy, pandas and scikit-learn API (Ch 3, §4.10)

NumPy (§3.2)

Task	Code	Remember
Create	`np.array`, `zeros`, `ones`, `eye`, `arange`, `linspace(0, 1, 5)`	one dtype per array; `float32` = 4 bytes, `float64` = 8
Inspect	`a.shape`, `a.dtype`, `a.ndim`, `a.size`, `a.nbytes`	shape `(n,)` ≠ `(n, 1)`
Reshape	`a.reshape(3, 4)`, `a.reshape(-1, 1)`, `a[:, None]`, `a.T`	scikit-learn wants 2-D `X`
Index	`M[0]`, `M[:2, 1:3]` → **view**; `M[M > 5]`, `M[[0, 2]]` → **copy**	`.copy()` when you need independence
Reduce	`M.sum(axis=0)` (one per column), `axis=1` (one per row), `keepdims=True`	the named axis disappears
Broadcast	align shapes from the right; equal or 1 → OK	(3,1)+(1,4) → (3,4); (3,2)+(3,) → error
Matrix product	`A @ B` (not `*`, which is element-wise)	
Standardise columns	`(X - X.mean(axis=0)) / X.std(axis=0)`	fit statistics on training data only
Random	`rng = np.random.default_rng(42)`; `rng.normal`, `rng.integers`, `rng.permutation`	seed once, draw everything from `rng`
Floats	`np.isclose(0.1 + 0.2, 0.3)`	`int8` 127 + 1 wraps to −128

pandas (§3.3)

Task	Code
Load / first look	`pd.read_csv(path)`; `df.head()`, `df.info()`, `df.dtypes`, `df.describe()`, `df.isnull().sum()`, `df["c"].value_counts()`
Fix types	`pd.to_numeric(s, errors="coerce")`, `pd.to_datetime(s)`, `s.astype("category")`
Select	`df["c"]` (Series), `df[["a", "b"]]` (DataFrame), `df.loc[rows, cols]` (labels, **end inclusive**), `df.iloc[0:2]` (positions, end exclusive)
Filter	`df[(df.a > 1) & (df.b == "x")]` — `&`, `|`, `~` with parentheses
Missing values	`dropna()`, `fillna(v)`, `df.groupby(g)["x"].transform("median")`, `df["x_missing"] = df.x.isna()`
Group	`df.groupby("g").agg(n=("x", "size"), mean_x=("x", "mean"))` (one row per group); `transform` keeps the original length
Join / stack	`a.merge(b, on="k", how="left", validate="many_to_one")`; `pd.concat([a, b])`
Vectorise	column arithmetic, `np.where`, `pd.cut`, `.str`, `.dt` — avoid row-wise `apply`
pandas 3	copy-on-write: assign (`df["c"] = df["c"].fillna(0)`), don't chain in-place calls

scikit-learn estimator API (§3.5, §4.10)

Convention	Meaning
`Model(hyper=...)`	hyperparameters go in the constructor
`fit(X, y)` → `self`	learns; learned attributes end in `_` (`coef_`, `mean_`, `classes_`)
`transform` / `fit_transform`	`fit_transform` on **train only**; `transform` on validation/test/production
`predict`, `predict_proba`, `decision_function`, `score`	`score` = accuracy (classifiers), R^2 (regressors)
`Pipeline` / `make_pipeline`	preprocessing + model = one estimator; refit per CV fold (no leakage), one artifact to save
`ColumnTransformer`	different preprocessing per column group; default `remainder="drop"`
`step__param`	address a parameter inside a pipeline, e.g. `prep__num__impute__strategy`, `model__C`
`set_output(transform="pandas")`	named DataFrame outputs for debugging
`joblib.dump(pipe, "m.joblib")`	save the whole pipeline; only load files you trust

```
pre = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median", add_indicator=True), StandardScaler()), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
])
pipe = Pipeline([("pre", pre), ("model", LogisticRegression(max_iter=1000))])
search = GridSearchCV(pipe, {"model__C": [0.01, 0.1, 1, 10]}, cv=StratifiedKFold(5), scoring="average_precision")
search.fit(X_train, y_train)                # CV refits the preprocessing inside every fold
print(search.best_params_, search.score(X_test, y_test))   # test set used once
```

Defaults worth knowing (checked by the script): `LogisticRegression` C = 1.0 (L2), solver `lbfgs` · `RandomForestClassifier` `max_features="sqrt"` (regressor: 1.0) · `HistGradientBoosting*` learning_rate 0.1, `max_leaf_nodes` 31, `min_samples_leaf` 20 · `SVC` RBF kernel, C = 1, `gamma="scale"` · `KNeighbors*` k = 5, Minkowski p = 2 · `KMeans` `init="k-means++"`.

Module	Tools you'll use constantly
`model_selection`	`train_test_split`, `cross_val_score`, `cross_validate`, `KFold`, `StratifiedKFold`, `GroupKFold`, `TimeSeriesSplit`, `GridSearchCV`, `RandomizedSearchCV`
`preprocessing`	`StandardScaler`, `MinMaxScaler`, `RobustScaler`, `OneHotEncoder`, `OrdinalEncoder`, `TargetEncoder`, `PolynomialFeatures`, `KBinsDiscretizer`, `FunctionTransformer`
`impute`	`SimpleImputer`, `KNNImputer`, `IterativeImputer`
`metrics`	`accuracy_score`, `precision_score`, `recall_score`, `f1_score`, `roc_auc_score`, `average_precision_score`, `log_loss`, `brier_score_loss`, `mean_absolute_error`, `mean_squared_error`, `r2_score`, `ndcg_score`, `confusion_matrix`, `classification_report`
`calibration`	`CalibratedClassifierCV`, `calibration_curve`
`inspection`	`permutation_importance`, `PartialDependenceDisplay`

Frameworks (§3.7): scikit-learn (and XGBoost / LightGBM / CatBoost) for tables; PyTorch or TensorFlow/Keras for images, text, audio, GPUs and custom networks. Jupyter: seed first, Restart & Run All before sharing, move stable code into `.py` modules (§3.6).

20.4 Data preparation and feature engineering (Ch 4)

Labels (§4.1, §16.3)

Source	Speed	Weakness	Handle with
Natural outcomes (delivery time, chargeback)	Minutes → weeks	Late, incomplete, outcome ≠ truth	Train on mature rows; proxy labels for fresh data
Behaviour / implicit (clicks, watch time)	Seconds	Exposure and position bias	Log impressions and positions; correct bias (IPW, inverse propensity weighting)
Human annotation	Hours → days	Cost, reviewer quality	Versioned guidelines, Cohen's κ = (p_o − p_e)/(1 − p_e), gold questions, active learning
Weak supervision (labelling functions + label model)	Instant	Noisy, correlated rules	Snorkel-style label model; audit
LLM-assisted	Seconds	Correlated, systematic errors	Humans review samples, hard cases and per-subgroup slices

κ example: agreement 0.8, chance 0.5 → κ = **0.6**. Retrieval negatives: random (easy), in-batch (cheap, popularity-biased → logQ correction, which subtracts each item's log sampling probability), hard (sharp, some false negatives).

EDA checklist (§4.2): shape & types → missingness (and is it related to the target?) → distributions (skew, sentinels like 999 / 9,999,999, cardinality) → target balance → feature–target and feature–feature relationships (**Spearman** for skewed data) → duplicates, impossible values, leaky IDs. Do detailed EDA on the training split.

Cleaning (§4.3)

Problem	Default fix
Exact duplicates	`drop_duplicates()`, then check the entity key is unique; dedupe **across** splits
Impossible values (age 999)	Set to NaN — they are errors, not outliers
Outliers	IQR (interquartile range) fences (Q1 − 1.5·IQR, Q3 + 1.5·IQR), |z| > 3, or log first; cap at training-set percentiles; never blindly delete (fraud *is* the outlier)
Missing: MCAR / MAR / MNAR (missing completely at random / at random / not at random)	Median (numbers) or `"missing"` (categories) **+ missing indicator**; KNN or iterative imputation when other columns predict it; HistGradientBoosting/XGBoost/LightGBM handle NaN natively
Judging an imputer	Hide known values, measure error, and compare downstream CV scores

Encoding (§4.4)

Encoder	Use for	Risk
One-hot (`handle_unknown="ignore"`)	Nominal, low–moderate cardinality	Width; dummy trap for unregularised OLS (`drop="first"`)
Ordinal	Ordered categories; fine for trees	Fake order for linear models / k-NN
Target encoding (`TargetEncoder`)	High cardinality	**Leakage** unless cross-fitted and smoothed
Hashing (`FeatureHasher`)	Unbounded IDs, streaming	Collisions, not reversible
Embeddings	IDs in deep models / recommenders	Needs data per ID; versioning

Scaling (§4.5)

Scaler	Formula	Use when
Standard	(x − mean)/std	Default for linear models, SVMs, neural nets, PCA, k-NN, k-means
Min-max	(x − min)/(max − min)	Bounded inputs (pixels ÷ 255), no big outliers
Robust	(x − median)/IQR	Outliers you keep: [10,11,11,12,12,13,100] → 100 ↦ (100 − 12)/1.5 = 58.67
Log / power	log1p(x), `PowerTransformer`	Heavy right tails (income, counts) — scaling alone never fixes skew
None	—	Trees and tree ensembles

Feature creation (§4.6)

Technique	Example
Interactions / polynomials	price × quantity; `PolynomialFeatures(2)` on 10 features → 65 columns
Binning	age buckets let a linear model fit a U-shape
Log transform	`log1p(income)`
Dates and cycles	day of week, is_weekend, "days since signup"; hour h → (sin 2πh/24, cos 2πh/24): hour 6 → (1, 0)
Text	Bag-of-words, TF-IDF: idf(t) = ln((1+n)/(1+df)) + 1, rows L2-normalised (n = 4, df = 2 → 1.511)
Aggregates and ratios	orders in last 30 min, tickets per month of tenure — must be **point-in-time correct**
Production groups	**entity** (batch), **behavioural aggregates** (streaming), **context** (request time), **cross** (pairs) — §4.6, §16.4

Splits (§4.7): random (i.i.d. data) · **stratified** (classification, keeps class ratios) · **time-based** (anything predicting the future) · **group** (all rows of one user/patient together, to generalise to new entities). Millions of rows → 98/1/1 is fine; a few thousand → cross-validation.

Leakage checklist (§4.8, §16.3): (1) would I know this value at prediction time? (2) split first, then fit everything inside a `Pipeline`; (3) time/group splits when deployment needs them; (4) dedupe across splits; (5) aggregates whose window includes the label moment; (6) label-adjacent fields (refund flag, resolution code); (7) distrust any single dominant feature or a score far above published baselines.

Imbalance (§4.9)

Technique	Effect on probabilities
Better metrics (precision, recall, PR-AUC)	—
Threshold moving	None — often all you need
Class weights (`class_weight="balanced"`)	Inflated → recalibrate
Negative downsampling (keep fraction w)	Inflated → **p_true = p·w / (p·w + 1 − p)**; w = 0.1, p = 0.5 → 0.091
Oversampling / SMOTE (a + u·(b − a) between minority neighbours)	Inflated; resample the **training** data only, after splitting

---

20.5 Every algorithm in one table (Ch 5, 7–14)

n = rows, d = features, k = neighbours/clusters, B = trees, M = boosting rounds, T = sequence length, h = hidden width. Complexities are rough orders of magnitude for intuition, not exact bounds (big-O notation: O(nd) means cost grows in proportion to n × d).

Algorithm	What it does	Use when	Key hyperparameters	Pros	Cons	Cost (train / predict)	§
Linear regression (OLS)	$\hat{y} = w \cdot x + b$ minimising MSE	Roughly linear trends, baselines, explaining effects	none (fit_intercept)	Fast, interpretable, extrapolates a trend	Misses curves; outliers; multicollinearity	$O(nd^2 + d^3)$ solver or $O(nd)$/epoch GD / $O(d)$	5.2–5.3
Ridge / Lasso / ElasticNet	OLS + L2 / L1 / both penalty	Many or correlated features; want sparsity (L1)	α (and `l1_ratio`)	Less overfitting; Lasso selects features	Must scale; α tuned by CV	As OLS / $O(d)$	5.7
Polynomial regression	Linear model on powers/products	Smooth curves, few features	degree	Same solvers	Feature explosion; wild at the edges	As OLS on expanded d	5.6
Logistic regression	$p = \sigma(w \cdot x + b)$, log-loss	Binary baseline, calibrated probabilities, sparse text	C (= 1/strength), class_weight	Convex, fast, odds ratios, roughly calibrated	Linear boundary	$O(nd)$ per iteration / $O(d)$	5.8
Softmax regression	K linear scores → softmax	Mutually exclusive classes	C	Coherent probabilities	Linear boundaries	$O(ndK)$ / $O(dK)$	5.9
k-NN	Vote/average of k nearest points	Small low-dimensional data, similarity search	k, metric, weights	No training, any boundary	Slow prediction, needs scaling, curse of dimensionality	$O(1)$ / $O(nd)$ brute force	8.1
Linear SVM	Max-margin hyperplane, hinge loss	High-dimensional sparse text	C	Fast, strong on text	No probabilities	$\sim O(nd)$ / $O(d)$	8.2
Kernel SVM (RBF)	Max margin in an implicit feature space	Small/medium data with curved boundaries	C, γ (tune jointly, log scale)	Convex, flexible, compact (support vectors)	Slow beyond tens of thousands of rows; scaling; no native probabilities	$\sim O(n^2 d)$ to $O(n^3 d)$ / $O(n_{SV} \cdot d)$	8.2
SVR	Fit inside an ε-tube	Small-data regression with noise	C, ε, γ	Ignores small errors	As kernel SVM	As SVM	8.2
Naive Bayes	Bayes + "features independent given class"	Text, tiny data, streaming baselines	α (smoothing), variant (Multinomial/Bernoulli/Gaussian)	Fastest to train, few data needed	Over-confident probabilities, no interactions	$O(nd)$ / $O(dK)$	8.3
Decision tree (CART)	Greedy binary splits maximising impurity decrease	Readable rules, quick look	max_depth, min_samples_leaf, ccp_alpha	Interpretable, no scaling, non-linear	High variance, axis-aligned, no extrapolation	$\sim O(nd \log n)$ / $O(\text{depth})$	7.1
Random forest	Bagged deep trees + random feature subset per split	Strong default with little tuning	n_estimators, max_features, max_depth / min_samples_leaf	Robust, parallel, free OOB score, more trees never hurt	Bigger/slower; less accurate than tuned boosting; no extrapolation	B × tree / $O(B \cdot \text{depth})$	7.3
AdaBoost	Re-weight misclassified samples; weighted vote of stumps	Clean data, simple baseline	n_estimators, learning_rate	Simple	Sensitive to noise/outliers (exponential loss)	M × stump / $O(M)$	7.4
Gradient boosting (sklearn HGB, XGBoost, LightGBM, CatBoost)	Sequential shallow trees fitted to the negative gradient	**Best default for tabular data**	learning_rate, n_estimators (+ early stopping), depth/leaves, min leaf size, subsample, L1/L2	Top tabular accuracy, native NaN/categoricals	Overfits with too many rounds; sequential	M × histogram tree ≈ $O(M \cdot nd)$ / $O(M \cdot \text{depth})$	7.4, 7.7
Voting / stacking	Combine different model families (stacking: meta-model on out-of-fold predictions)	Diverse models with different errors	base models, meta-model, cv	Reduces risk	More latency and upkeep; not guaranteed to win	Sum of models	7.5
k-means	Assign to nearest centroid, move centroids, repeat	Round, similar-sized clusters	k, n_init (k-means++)	Fast, scales (MiniBatch)	Must pick k; spherical assumption; outliers	$O(nkd)$ per iteration	9.1

Algorithm	What it does	Use when	Key hyperparameters	Pros	Cons	Cost (train / predict)	§
Agglomerative	Merge closest clusters bottom-up (dendrogram)	Small data, nested structure	linkage, n_clusters or cut height	No k up front, interpretable tree	$O(n^2)$ memory	$\geq O(n^2)$	9.2
DBSCAN / HDBSCAN	Clusters = dense regions; noise = −1	Odd shapes, outliers, unknown k	eps, min_samples (HDBSCAN: min_cluster_size)	Any shape, labels noise	One global density (DBSCAN); eps hard in high-d	~O(n log n) with an index	9.3
Gaussian mixture (EM)	Soft assignment to k Gaussians	Elliptical/overlapping clusters, need probabilities	k (by BIC), covariance_type	Soft, elliptical	Local optima; k to choose	$O(nkd^2)$ per EM step (full cov.)	9.4
PCA	Project onto top eigenvectors of the covariance	Compression, denoising, pre-step, quick 2-D look	n_components (e.g. 0.95 variance)	Fast, deterministic, `transform`	Linear only; scale first	full SVD ~O(nd·min(n,d))	9.5
t-SNE / UMAP	Non-linear 2-D maps preserving neighbours	Visualisation only	perplexity / n_neighbors, min_dist	Reveals clusters	Sizes and gaps meaningless; seed-dependent	t-SNE ~O(n log n) (Barnes-Hut)	9.6
Isolation Forest	Anomalies isolate in few random splits	Multivariate anomaly default	n_estimators, max_samples (256), contamination	Fast, no scaling	Misses anomalies visible only in combinations	~linear	9.7
LOF / One-Class SVM	Local density vs neighbours / boundary around normal data	Mixed densities / novelty on clean data	k / ν, γ	Local / flexible	$O(n^2)$-ish / slow on big n	—	9.7
Apriori / FP-Growth	Frequent itemsets → rules ranked by lift	Market-basket analysis	min_support, min_confidence	Readable rules	Many trivial rules	Grows with candidates	9.8
MLP (neural network)	Layers of h = f(Wx + b), trained by backprop	Large data; building block of deep nets	depth, width, lr, batch size, dropout, weight decay	Learns features, very flexible	Data-hungry, many knobs; usually loses to GBDT on small tables	O(params) per example per step	10.1
CNN	Shared convolution kernels + pooling	Images, spatial grids	depth, channels, kernel size, augmentation	Few parameters, translation equivariance	Needs data or a pretrained backbone	$O(k^2 \cdot C_{in} \cdot C_{out} \cdot H \cdot W)$ per layer	11.1–11.2
RNN / LSTM / GRU	Hidden state carried step by step	Streaming/small sequence models, time series	hidden size, layers, clipping	Constant memory per step	Sequential (no parallelism over time)	$O(T \cdot h^2)$	12.2–12.4
Transformer	Self-attention + feed-forward blocks	Text, speech, vision at scale	layers, d_model, heads, context length	Parallel, O(1) path between tokens	$O(T^2 \cdot d)$ attention cost	$O(T^2 d + T d^2)$ per layer	12.7
Matrix factorisation (SGD/ALS)	$\hat{r} = \mu + b_u + b_i + p_u \cdot q_i$	Collaborative filtering baseline	k factors, λ, (α for implicit)	Strong, compact embeddings	Cold start; no context	SGD $O(\|R\| \cdot k)$/epoch	13.5
Two-tower	User and item encoders meeting at a dot product	Retrieval over millions of items	embedding dim, temperature, negatives	Items precomputed → ANN	Weak feature crosses; versioning	Per request: user tower + ANN	13.7
Multi-armed bandits (ε-greedy, UCB, Thompson)	Explore/exploit over k arms	Headlines, thumbnails, new-item traffic	ε, c, priors	Less regret than A/B	Non-stationarity handling	O(k) per step	14.4
Q-learning / SARSA	TD control on a Q-table	Small discrete MDPs	α, γ, ε schedule	Model-free, simple	Table must fit; sample-hungry	O(1) per update	14.7
DQN	Neural Q-function + replay + target network	Discrete actions, large state spaces	replay size, target sync C, ε	Scales to pixels	Unstable (deadly triad)	Network forward/backward	14.8
Policy gradient / actor-critic / PPO	Gradient ascent on expected return	Continuous actions, RLHF	lr, clip ε, entropy bonus, GAE λ	Stochastic and continuous policies	High variance, on-policy	Network forward/backward	14.9
ARIMA / ETS	AR + differencing + MA / smoothing	Single short time series	(p, d, q), seasonality	Built-in intervals, little data	One series at a time, few extra features	Small	12.10

20.6 Loss functions (Ch 2, 5, 8, 10, 13, 14)

Loss	Formula (one example)	Task / model	Notes	§
MSE (squared error)	$(\hat{y} - y)^2$	Regression, linear regression, neural nets	Punishes big errors; optimal constant = **mean**; MLE under Gaussian noise	5.2, 10.3
MAE (L1)	$\lvert\hat{y} - y\rvert$	Robust regression	Optimal constant = **median**	6.2, 10.3
Huber	$\frac{1}{2}e^2$ if $\lvert e\rvert \le \delta$, else $\delta(\lvert e\rvert - \frac{1}{2}\delta)$	Regression with outliers; DQN	Quadratic near 0, linear far away	10.3, 14.8
Quantile (pinball)	$\max(\tau \cdot e, (\tau - 1) \cdot e)$, $e = y - \hat{y}$	Prediction intervals, p90 forecasts	$\tau = 0.9$ → 90th percentile	12.10
Log loss / binary cross-entropy	$-[y \ln p + (1-y) \ln(1-p)]$	Logistic regression, sigmoid outputs, multi-label	Convex for LR; gradient at logit = $p - y$; PyTorch: `BCEWithLogitsLoss` (logits in)	5.8, 10.3
Categorical cross-entropy	$-\ln p_{true}$	Softmax classifiers, language models	Uniform guess = ln K (ln 10 = 2.303); gradient = p − one_hot; PyTorch `CrossEntropyLoss` takes logits + class indices	5.9, 10.3
Hinge	$\max(0, 1 - y \cdot f(x))$, $y \in \{-1, +1\}$	SVM	Zero for points beyond the margin	8.2
ε-insensitive	$\max(0, \lvert y - f(x)\rvert - \varepsilon)$	SVR	Errors inside the tube are free	8.2
Exponential	$e^{-y \cdot f(x)}$	AdaBoost	Explains its outlier sensitivity	7.4
Regularised squared error	$\Sigma(r - f)^2 + \lambda(\lVert p\rVert^2 + \lVert q\rVert^2 + b^2)$ over observed cells	Matrix factorisation	Implicit version weights every cell by confidence	13.5
Sampled softmax / InfoNCE	$-\ln [e^{s^+/\tau} / \Sigma e^{s/\tau}]$ over in-batch items (logits − log Q for popularity)	Two-tower retrieval, CLIP-style contrastive learning	Temperature $\tau \approx 0.05$–0.1	13.7, 18.6
Skip-gram negative sampling	$-\ln \sigma(v \cdot u^+) - \Sigma \ln \sigma(-v \cdot u^-)$	word2vec, item2vec	Negatives ∝ frequency^0.75	12.9, 13.6
Pairwise / listwise ranking	pairwise: which of two items is better (RankNet/LambdaMART); listwise: optimise NDCG-like list metrics	Learning-to-rank	Pointwise = predict each label independently	17.3
Regularisation terms	L2: $\lambda\Sigma w^2$ · L1: $\lambda\Sigma\lvert w\rvert$	Any	L1 → exact zeros; L2 → shrinkage; AdamW = decoupled L2	5.7, 10.7
TD / Q-learning target	$(r + \gamma \max_{a'} Q(s',a'; \theta^-) - Q(s,a; \theta))^2$ (Huber in DQN)	Value-based RL	θ^- = frozen target network	14.8
Policy gradient	$-\log \pi(a\lvert s) \cdot A_t$	REINFORCE, actor-critic	A = G − V(s) or the TD error	14.9
Reward model (preferences)	$-\ln \sigma(s_{preferred} - s_{rejected})$	RLHF	PPO then maximises reward − $\beta \cdot KL(\pi \,\|\, \pi_{SFT})$	14.10
Fairness penalty	loss + $\lambda \cdot$(mean score A − mean score B)^2	In-processing bias mitigation	Larger λ → more parity, less accuracy	19.4

Rule of thumb: pick the output layer and the loss together — identity + MSE/MAE/Huber (regression), sigmoid + BCE (binary, multi-label), softmax + cross-entropy (one label out of K) (§10.2). Never train on accuracy: its gradient is zero almost everywhere.

20.7 Metrics: formula and when to use (Ch 6, 9, 11, 13, 16)

Regression (§6.2, §16.7) — worked on y = (30, 25, 40, 35, 50), ŷ = (28, 27, 35, 35, 60):

Metric	Formula	Value	Use when
MAE	$(1/n)\Sigma\lvert e\rvert$	3.8	Cost grows linearly with error; label outliers shouldn't dominate
MSE / RMSE	$(1/n)\Sigma e^2$ / √MSE	26.6 / 5.16	Big misses are disproportionately bad (a 40-min miss adds 16× what a 10-min miss adds to MSE)
R^2	$1 - \Sigma e^2/\Sigma(y - \bar{y})^2$	0.641	Share of variance explained; 0 = predicting the mean, < 0 = worse
MAPE	$(100/n)\Sigma\lvert e/y\rvert$	—	Relative error; breaks near y = 0
p90 absolute error	90th percentile of $\lvert e\rvert$	—	Promises made on the tail (ETAs)
Per-cohort error	Any of the above per segment	—	Always: averages hide failing segments

Classification (§6.3, §6.5) — worked on 1,000 transactions, 100 fraud, 80 flagged, 60 correct (TP 60, FP 20, FN 40, TN 880):

Metric	Formula	Value	Use when
Accuracy	(TP + TN)/all	0.94 ("flag nothing" = 0.90)	Balanced classes, equal costs only
Precision (PPV)	TP/(TP + FP)	0.75	False alarms are expensive
Recall (sensitivity, TPR)	TP/(TP + FN)	0.60	Misses are expensive
F1 / Fβ	2PR/(P + R) / (1+β²)PR/(β²P + R)	0.667	One number, imbalanced data; F2 favours recall
Specificity / FPR	TN/(TN + FP) / FP/(FP + TN)	0.978 / 0.022	Screening; ROC axes
ROC-AUC	P(random positive scored above random negative)	positives (0.9, 0.7, 0.4) vs negatives (0.8, 0.3, 0.2) → 7/9 = 0.778	Threshold-free ranking quality, moderate imbalance
PR-AUC (average precision)	Area under precision–recall curve; no-skill = base rate	—	**Rare positives** (fraud, ads, disease)
Log loss	−mean[y ln p + (1−y) ln(1−p)]	—	Probabilities feed decisions
Brier score	mean((p − y)²)	y (1,0,1,1), p (0.9,0.2,0.6,0.3) → 0.175; p ≡ 0.5 → 0.25	Calibration quality (proper scoring rule)
ECE	Weighted gap between mean score and observed rate per bin	—	Calibration monitoring (ads)
Macro / weighted averages	Mean of per-class metrics (unweighted / by support)	—	Multiclass; macro exposes weak minority classes

Choosing a threshold (§6.3): with calibrated p, flag when **p > C_FP / (C_FP + C_FN)** (review $5, missed fraud $100 → p > 0.048). Fraud and moderation use **two thresholds**: approve / review or step-up / block.

Ranking and retrieval (§6.4, §13.11, §17.3)

Metric	Formula	Use when
Precision@k	relevant in top k ÷ k	Fixed number of slots
Recall@k	relevant in top k ÷ all relevant	Candidate generation (did the good items survive?)
Hit rate@k	1 if any relevant item in top k	Leave-one-out recommender tests
AP / MAP	mean of precision@i at each relevant rank ÷ #relevant; MAP = mean over queries	Binary relevance, whole list
MRR	mean of 1/rank of the first relevant result	One good answer fast (Q&A, navigation)
NDCG@k	DCG/IDCG, DCG = Σ rel_i / $\log_2(i + 1)$ (exponential variant: (2^rel − 1))	Graded relevance, position matters
Coverage, diversity, novelty	share of catalogue recommended; 1 − mean pairwise similarity; mean popularity	Beyond-accuracy health of a recommender

Worked: relevances [3, 0, 2, 1, 0, 0], one relevant grade-2 document not retrieved → DCG@5 = 4.431, NDCG@5 = **0.778**; `sklearn.metrics.ndcg_score` builds the ideal only from items you pass it and reports 0.930 — always say which NDCG. With exponential gain, [3, 0, 2] → NDCG@3 = 0.956. Recs A–E with relevant {A, C, F}: P@5 0.4, R@5 0.667, AP@5 0.556, NDCG@5 0.704.

Other families

Area	Metrics	§
Clustering (no labels)	Silhouette s = (b − a)/max(a, b) (higher), Davies–Bouldin (lower), Calinski–Harabasz (higher), inertia/BIC (compare k only) — all favour convex clusters	9.1, 9.9
Clustering (some labels)	Adjusted Rand Index (0 random, 1 perfect), NMI	9.9
Dimensionality reduction	Explained variance, reconstruction error, trustworthiness	9.5, 9.9
Anomaly detection	Precision@k, average precision on a reviewed sample	9.7
Detection / segmentation	IoU = intersection/union (worked 0.5625); mAP over IoU thresholds (VOC 0.5; COCO 0.5–0.95); Dice for masks	11.7
Association rules	support, confidence = supp(A∪B)/supp(A), **lift** = confidence/supp(B) (> 1 positive)	9.8
Online / business	CTR, watch time, retention, revenue per search, resolution rate, promise-kept rate + guardrails	6.1, 16.9
Drift	PSI = Σ(cur − ref)·ln(cur/ref): < 0.1 stable, 0.1–0.25 watch, > 0.25 investigate; KS statistic D; null rates	16.8
Fairness	Selection rate, TPR, FPR, PPV and calibration per group; disparate-impact ratio (four-fifths rule)	19.2–19.3

20.8 Validation and tuning (Ch 4, 6)

Splitter	Folds	Use for
`train_test_split(stratify=y)`	One hold-out	Final test set
`KFold(5, shuffle=True)`	Random equal folds	Regression, i.i.d. rows
`StratifiedKFold`	Same class mix in every fold	Classification (default in `cross_val_score` for classifiers)
`GroupKFold` / `StratifiedGroupKFold`	One entity per fold	Users, patients, devices
`TimeSeriesSplit(gap=…)`	Train on past, validate on the next block	Anything time-ordered; gap mimics label delay
`RepeatedStratifiedKFold` / `LeaveOneOut`	Repeats / k = n	Small data / tiny data only
Nested CV	Outer loop around the whole tuning procedure	Unbiased estimate of "tune then train"

Report CV as **mean ± std**; differences under about one std are not clearly real (§6.6). Expected test error = **bias² + variance + irreducible noise**; averaging B models with variance σ^2 and correlation ρ gives variance $\rho\sigma^2 + (1-\rho)\sigma^2/B$ — bagging shrinks the second term, random feature subsets shrink ρ (§6.7, §7.2).

Tool	Answers	Read it
Learning curve (score vs training size)	Would more data help?	Curves converge low → more capacity; big persistent gap → more data / regularisation
Validation curve (score vs one hyperparameter)	Where is the sweet spot?	Pick the peak of the validation curve

Tuning strategies (§6.9)

Method	Budget control	Best for
Grid search	Product of grid sizes (4 × 5 values × 5 folds = 100 fits)	1–2 hyperparameters, cheap models
Random search	`n_iter`	Default for 3+ hyperparameters (explores each dimension more finely)
Successive halving / Hyperband	Resource per round	Many candidates, scalable resource (rows, epochs)
Bayesian optimisation (Optuna TPE, Gaussian process + expected improvement)	Trials	Expensive trials (boosting on big data, deep nets)

Recipe: defaults + baseline → tune the few that matter (boosting: learning rate + tree size; linear/SVM: C, γ; trees: depth/leaf size) → log-uniform ranges → widen a range if the best value sits on its edge → refit on all training data → test once. **Early stopping** (§6.10): monitor validation loss, stop after `patience` rounds without improvement, restore the best weights; never early-stop on the test set.

Probability calibration (§6.5): reliability diagram + Brier/log-loss on held-out data; fix with **Platt scaling** (sigmoid on the score, small data) or **isotonic regression** (step function, roughly ≥ 1,000 examples) via `CalibratedClassifierCV`; recheck when the base rate shifts. ROC-AUC is unchanged by any strictly increasing map.

20.9 Deep learning: layers, activations, optimizers, regularisation, debugging (Ch 10)

Building blocks (§10.1, §10.3, §10.4)

Piece	Formula / rule
Neuron	$z = w \cdot x + b$, $a = f(z)$
Dense layer	$h = f(Wx + b)$; batch: $H = f(XW^\top + b)$; params = inputs × outputs + outputs (256 → 128: 32,896)
Why nonlinearity	Without f, $W_2(W_1x + b_1) + b_2$ is still one linear map; one perceptron can't learn XOR, one hidden layer can
Forward → loss → backward → update	$\theta \leftarrow \theta - \eta \cdot \partial L/\partial\theta$; backprop = chain rule from the loss backwards, ≈ 2–3× a forward pass
Output gradient	sigmoid + BCE and softmax + CE: $\partial L/\partial \text{logits} = p - y$
Weight gradient	(layer input)$^\top$ @ (gradient at the layer's pre-activation)
Gradient check	$(L(\theta + \varepsilon) - L(\theta - \varepsilon))/2\varepsilon \approx$ analytic gradient

Activations (§10.2)

Name	Formula	Range	Use
Sigmoid	$1/(1 + e^{-z})$; $\sigma' \le 0.25$	$(0, 1)$	Binary / multi-label output; LSTM gates
Tanh	$(e^z - e^{-z})/(e^z + e^{-z})$; max slope 1	$(-1, 1)$	RNN hidden states
ReLU	$\max(0, z)$	$[0, \infty)$	**Default hidden layer**; can "die"
Leaky ReLU / PReLU / ELU	z or αz ($\alpha \approx 0.01$) / α learned / $\alpha(e^z - 1)$	—	When many ReLUs die
GELU / SiLU (Swish)	$z \cdot \Phi(z)$ / $z \cdot \sigma(z)$	—	Transformers (BERT, GPT), EfficientNet, SwiGLU in LLMs
Softmax	$e^{z_i} / \Sigma e^{z_j}$ (subtract max first)	sums to 1	One label out of K; (2, 1, 0.1) → (0.659, 0.242, 0.099)
Identity	z	$\mathbb{R}$	Regression output

Optimizers (§10.5) — g = mini-batch gradient

Optimizer	Update	Default / note
SGD	$\theta \leftarrow \theta - \eta g$	Set lr explicitly
Momentum	$v \leftarrow \mu v + g$; $\theta \leftarrow \theta - \eta v$	$\mu = 0.9 \approx$ averaging ~10 steps; Nesterov = look-ahead
AdaGrad	$s \leftarrow s + g^2$; $\theta \leftarrow \theta - \eta g/(\sqrt{s} + \varepsilon)$	Sparse features; step shrinks forever
RMSProp	$s \leftarrow \rho s + (1 - \rho)g^2$; $\theta \leftarrow \theta - \eta g/(\sqrt{s} + \varepsilon)$	PyTorch lr 0.01, α (ρ) 0.99
Adam	$m \leftarrow \beta_1 m + (1-\beta_1)g$; $s \leftarrow \beta_2 s + (1-\beta_2)g^2$; $\hat{m} = m/(1-\beta_1^t)$, $\hat{s} = s/(1-\beta_2^t)$; $\theta \leftarrow \theta - \eta\hat{m}/(\sqrt{\hat{s}} + \varepsilon)$	lr 1e-3, $\beta = (0.9, 0.999)$, ε = 1e-8
AdamW	Adam + $\theta \leftarrow \theta - \eta\lambda\theta$ (decoupled)	weight_decay 0.01; the correct "Adam with weight decay"

Schedules: step decay (÷10 at milestones), exponential, reduce-on-plateau, **cosine annealing**, **linear warmup + cosine decay** (transformers/LLMs), one-cycle. LR finder: grow lr exponentially per batch, pick ≈ one-tenth of the lr at the loss minimum.

Initialisation and gradient flow (§10.6)

Scheme	Weight variance	For
Xavier / Glorot	$2/(\text{fan_in} + \text{fan_out})$	tanh, sigmoid, linear
He / Kaiming	$2/\text{fan_in}$ (fan_in 256 → 0.0078, std 0.088)	ReLU family
LeCun	$1/\text{fan_in}$	SELU, linear

Never initialise all weights equal (symmetry never breaks). Vanishing/exploding gradients (ten sigmoid layers scale gradients by $\le 0.25^{10} \approx 9.5 \times 10^{-7}$) → ReLU, He/Xavier init, BatchNorm/LayerNorm, residual connections, gradient clipping (`clip_grad_norm_(params, 1.0)`), gates (LSTM/GRU).

Regularisation (§10.7)

Technique	How	Watch out
Dropout (p)	Zero units at random in training; survivors × 1/(1 − p) (inverted dropout)	Call `model.eval()` for inference
Weight decay	$\lambda\Sigma w^2$ or AdamW's decoupled shrinkage (1e-4…1e-2 SGD; 0.01–0.1 AdamW)	Often skip biases and norm parameters
BatchNorm	$\hat{x} = (x - \mu_{batch})/\sqrt{\sigma^2_{batch} + \varepsilon}$, $y = \gamma\hat{x} + \beta$; running stats at eval	Small batches; LayerNorm in transformers/RNNs, GroupNorm for small-batch vision
Early stopping	Stop after `patience` epochs without validation improvement; restore best	Report on a separate test set
Data augmentation	Label-preserving transforms (flips, crops, colour jitter, noise, Mixup/CutMix)	Training data only; don't flip a 6 into a 9
More data / smaller model	—	Often the most effective of all

Debugging checklist (§10.8)

1. Look at a few raw examples and labels **after** the data pipeline.
2. Initial loss ≈ chance: ln K for K classes (ln 10 = 2.303). Much higher → bad init/scale, wrong labels or loss.
3. **Overfit one mini-batch** to ~0 loss. Can't → a bug (labels, loss, no gradient flow, missing `optimizer.step()`).
4. LR range test; start with Adam 1e-3 (or SGD + momentum at the suggested lr).
5. Train with a validation set; plot both curves; normalise inputs with **training** statistics.
6. Only then add regularisation and tune; keep a simple baseline (logistic regression / GBDT) for "what good means".

Curve looks like	Likely cause	Try
Flat from step 0, near ln K	lr far too small; bug (frozen weights, shuffled labels, no step)	Overfit one batch; check the data pipeline
Rising, NaN, spikes	lr too high, exploding gradients, log(0)	Lower lr, clip gradients, logit-based losses
Train ↓, validation ↓ then ↑	Overfitting	Regularise, early stop, more data
Both high and close	Underfitting	Bigger model, train longer, higher lr, better features
Validation better than train	Dropout/augmentation only in training	Usually fine — check the split
Jagged	Small batch or high lr	Larger batch, lower lr

PyTorch loop (§10.9): `model.train()` → for each batch: `logits = model(xb)`; `loss = loss_fn(logits, yb)`; `opt.zero_grad()`; `loss.backward()`; `opt.step()` (then `scheduler.step()`). Evaluate under `model.eval()` + `torch.no_grad()`. Save `state_dict()`. `CrossEntropyLoss` wants `long` class indices; `BCEWithLogitsLoss` wants float targets shaped like the logits. Batch size 32–256 to start; imbalance → `weight=` / `pos_weight=` or `WeightedRandomSampler`.

20.10 CNN, RNN and attention: shapes and formulas (Ch 11, 12)

Convolutional networks (§11.1–11.2)

Quantity	Formula	Example
Output size	out = ⌊(n + 2p − k)/s⌋ + 1	n 5, k 3: s 1 p 0 → 3; s 2 → 2; p 1 → 5 ("same"); 224, k 7, s 2, p 3 → 112
Conv weights	k·k·C_in·C_out + C_out	`Conv2d(64, 128, 3)` → 73,856; `Conv2d(3, 16, 3)` → 448 vs 49,168 for a dense layer on 32×32×3
Tensor layout	PyTorch N×C×H×W; TensorFlow/NumPy images N×H×W×C	`permute(0, 3, 1, 2)`
Pooling	2×2 max, stride 2 halves H and W; no parameters; global average pooling before the head	
Receptive field	each k×k layer adds (k − 1) × product of earlier strides	stacked 3×3 (stride 1): 3, 5, 7 — three 3×3 = one 7×7 with $27C^2$ vs $49C^2$ weights
1×1 conv	mixes channels only	bottlenecks (Inception, ResNet)
Depthwise-separable	per-channel spatial conv + 1×1	≈ 8–9× less compute for 3×3 (MobileNet)
Residual block	y = ReLU(x + F(x))	identity gradient highway (ResNet)
ViT patches	(H/P)·(W/P) tokens	224×224, 16×16 → 196

Architecture timeline (§11.4): LeNet-5 (1998, conv-pool) → AlexNet (2012, ReLU + dropout + GPUs, top-5 15.3%) → VGG (2014, stacked 3×3) → Inception/GoogLeNet (2014, parallel branches, 1×1, GAP) → **ResNet** (2015, skip connections, 3.57% top-5 ensemble) → MobileNet (2017, depthwise-separable) → EfficientNet (2019, compound scaling of depth/width/resolution) → ViT (2020, patches + transformer) → ConvNeXt (2022).

Transfer learning (§11.5): small data similar to ImageNet → freeze backbone, train a new head; more or different data → fine-tune (all or top blocks) with a ~10× smaller lr; preprocess exactly like the pretraining data. **Augmentation (§11.6):** `RandomResizedCrop`, `RandomHorizontalFlip`, `ColorJitter`, `RandomErasing`, Mixup/CutMix, RandAugment — training only.

Vision tasks (§11.7): classification (label) · detection (box + class; IoU, **NMS**: keep the best box, drop overlaps above an IoU threshold, repeat; two-stage Faster R-CNN vs one-stage YOLO grid; mAP) · semantic segmentation (class per pixel; **U-Net** encoder–decoder with concatenated skip connections) · instance segmentation (mask per object; Mask R-CNN, SAM).

Recurrent networks (§12.2–12.5)

Model	Equations	Notes
RNN	$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b)$	Shared weights over time; BPTT; truncated BPTT for long sequences
LSTM	$f_t, i_t, o_t = \sigma(W[h_{t-1}, x_t] + b)$; $\check{c}_t = \tanh(\ldots)$; $\mathbf{c_t = f_t \odot c_{t-1} + i_t \odot \check{c}_t}$; $h_t = o_t \odot \tanh(c_t)$	Additive cell path fixes vanishing gradients; forget-gate bias 1; 4 weight blocks
GRU	$z_t, r_t = \sigma(\ldots)$; $\tilde{h}_t = \tanh(W[r_t \odot h_{t-1}, x_t])$; $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$	3 weight blocks, similar accuracy
Seq2seq	Encoder → context → decoder; teacher forcing in training; greedy / **beam search** / sampling at inference	Fixed-size bottleneck → attention

Worked LSTM unit: c = 2.0, f 0.9, i 0.2, č 0.5 → c_new = 1.9; o 0.7 → h = 0.7·tanh(1.9) = 0.669. PyTorch `nn.LSTM(input, hidden, num_layers, batch_first=True)` takes (batch, time, features) and returns (output, (h_n, c_n)). Parameter ratio RNN : GRU : LSTM = 1 : 3 : 4.

Attention and transformers (§12.6–12.7)

Idea	Formula / fact
Scaled dot-product attention	**Attention(Q, K, V) = softmax(QKᵀ/√d_k)·V** — score, softmax, weighted sum
Why √d_k	Dot products of random unit-variance vectors have std √d_k; scaling keeps softmax out of saturation
Worked example	Q = K = [[1,0],[0,1],[1,1]], V = [[1,2],[3,4],[5,6]] → query 1 weights (0.401, 0.198, 0.401), output [3.0, 4.0]
Self- vs cross-attention	Q, K, V from the same sequence / Q from decoder, K and V from encoder
Causal mask	Upper triangle set to −∞ before softmax (decoders)
Multi-head	h heads of size d_model/h, concatenated and projected
Positional encoding	PE(pos, 2i) = sin(pos/10000^(2i/d)), PE(pos, 2i+1) = cos(…); or learned / RoPE
Block	Multi-head attention and a position-wise FFN (≈ 4× wider), each wrapped in a residual connection + LayerNorm; original: 6 + 6 layers, d_model 512, 8 heads
Families	Encoder-only (BERT), decoder-only (GPT, Claude, LLaMA), encoder–decoder (T5, Whisper)
Cost	Attention O(T²·d) vs RNN O(T·d²); path length O(1) vs O(T); KV cache grows with context

Text (§12.8–12.9): normalise → tokenize → vocabulary (+ `<pad>`, `<unk>`, `<sos>`, `<eos>`) → ids → pad + mask. Subword tokenizers (BPE: merge the most frequent adjacent pair repeatedly; WordPiece; SentencePiece) have no OOV problem; ≈ 0.75 English words per token. word2vec: skip-gram (centre → context) or CBOW (context → centre) with negative sampling; GloVe, fastText (character n-grams), contextual embeddings (BERT).

Time series (§12.10): past-only features (lags, **shifted** rolling means, calendar); chronological splits (`TimeSeriesSplit`); beat naive and **seasonal-naive** baselines; ARIMA(p, d, q) = autoregression + differencing + moving average; trees can't extrapolate a trend → model differences.

20.11 Recommenders (Ch 13, 17.1)

Topic	Key facts
Framing	Fill a huge, sparse user–item matrix; optimise **top-K ranking** on an outcome closer to value than clicks (watch time, saves, purchases)
Feedback	Explicit (stars, real negatives, scarce) vs implicit (clicks/plays, abundant, unobserved ≠ negative → confidence c = 1 + α·count and sampled negatives)
Similarity	Cosine (direction), dot product (includes magnitude/popularity), adjusted cosine (subtract each user's mean), Jaccard |A∩B|/|A∪B| for sets; for unit vectors ‖a−b‖² = 2 − 2cos
Content-based vs CF	Attributes (handles new items, over-specialises) vs other users' behaviour (quality and surprise; cold start, popularity bias); hybrids win
Item-based CF	r̂(u,i) = mean(u) + Σⱼ S(i,j)(r(u,j) − mean(u)) / Σⱼ|S(i,j)|; precompute item–item neighbours (Amazon, 2003)
Matrix factorisation	r̂ = μ + bᵤ + bᵢ + pᵤ·qᵢ; train on **observed** cells with L2; SGD: err = r − r̂, pᵤ ← pᵤ + η(err·qᵢ − λpᵤ); ALS: fix one side, solve ridge per user/item (parallel); implicit ALS covers all cells with confidence weights
Embeddings	Short learned vectors; routes: MF, two-tower, graph (PinSage), pretrained content encoders; item2vec = word2vec on sessions; "near" only means near for the training objective
Two-tower	User tower and item tower meet only at a dot product → item vectors precomputed into an ANN index, user tower per request; in-batch softmax with temperature, **logQ correction**, a few hard negatives; ship tower + index as **one version**
Funnel	Retrieval 10^8 → 10^3 (recall) → ranking 10^3 → 10^2 (precision/NDCG, rich crosses; GBDT, DLRM, Wide & Deep, DCN) → re-ranking → 10–50 (diversity, freshness, policy)
MMR	next = argmax [λ·rel(j) − (1 − λ)·maxₛ sim(j, s)]
ANN	IVF (k-means lists; `nprobe` ↑ = recall ↑), HNSW (layered graph; M, ef_search), PQ (compressed codes; IVF-PQ), trees (Annoy); always report recall vs exact search with latency
Cold start	New user: popularity, onboarding, context, fast session updates; new item: content features + controlled exploration (bandits)
Evaluation	Time-based split, full-catalogue ranking (sampled metrics mislead); P/R/hit rate@k, MAP, NDCG, coverage, diversity, novelty; **popularity baseline**; then A/B or interleaving
Bias	Popularity bias + feedback loops → measure (coverage, Gini of exposure), correct (logQ, IPS), explore, re-rank; log impressions and positions

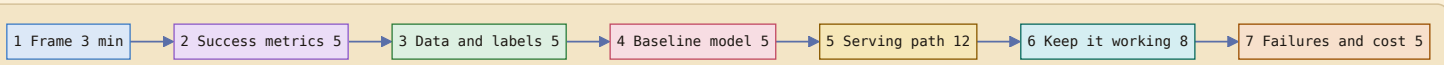
---

20.12 Reinforcement-learning update rules (Ch 14)

Idea	Formula	Notes / worked value
MDP	(S, A, P(s'|s,a), R, γ); Markov: next state depends only on current state and action	§14.2
Return	$G_t = \Sigma_k\ \gamma^k\ r_{t+k+1} = r_{t+1} + \gamma G_{t+1}$	γ = 0.9: horizon ≈ 1/(1−γ) = 10; $0.9^{10} \approx 0.35$
Value functions	V^π(s) = E[G | s]; Q^π(s,a) = E[G | s, a]; π*(s) = argmax_a Q*(s,a)	Q needs no model to act
Bellman expectation	V^π(s) = Σ_a π(a|s) $\Sigma_{s'}$ P(s'|s,a)[R + γV^π(s')]	Policy evaluation
Bellman optimality	V*(s) = $\max_a$ $\Sigma_{s'}$ P[R + γV*(s')]; Q*(s,a) = $\Sigma_{s'}$ P[R + γ $\max_{a'}$ Q*(s',a')]	Treasure grid: 10, 8, 6.2, 4.58, …, −0.43 at the start
ε-greedy	random arm with probability ε, else best	Decay ε
UCB1	argmax_a [Q(a) + c√(ln t / N_a)], c = √2 in theory	Tune c for your horizon
Thompson sampling	Sample θ_a ~ Beta(wins + 1, losses + 1), pull argmax	Probability matching
Value iteration	V(s) ← $\max_a$ Σ P[R + γV(s')] until stable	Needs the model (DP)
Policy iteration	Evaluate π, make π greedy on V, repeat	Few rounds
Monte Carlo	V(s) ← V(s) + α[G_t − V(s)]	Unbiased, high variance, end of episode
TD(0)	V(s) ← V(s) + α[r + γV(s') − V(s)]; **δ = r + γV(s') − V(s)**	Bootstraps, lower variance, every step
Q-learning (off-policy)	Q(s,a) ← Q(s,a) + α[r + γ $\max_{a'}$ Q(s',a') − Q(s,a)]	Learns the greedy policy; cliff edge path
SARSA (on-policy)	Q(s,a) ← Q(s,a) + α[r + γQ(s',a') − Q(s,a)], a' = next action taken	Learns what it does; safer path
DQN	Minimise Huber(r + γ $\max_{a'}$ Q(s',a'; θ^-) − Q(s,a; θ)) on replay mini-batches	Experience replay + target network; Double DQN: select with online, evaluate with target
REINFORCE	θ ← θ + α·G_t·∇log π(a_t|s_t)	High variance → baseline: $A_t = G_t - V(s_t)$
Actor-critic	actor: θ ← θ + α·δ·∇log π; critic: TD learning on V	PPO = clipped actor-critic update
RLHF	SFT → reward model (−ln σ(s_w − s_l)) → PPO on reward − β·KL(π ‖ π_SFT)	DPO skips the RL loop; GRPO uses group-mean baselines

Pitfalls (§14.12): reward hacking (a re-collectable +3 per step is worth 3/(1 − 0.9) = 30), sample inefficiency, sparse rewards, credit assignment, seed instability, sim-to-real gap, unsafe exploration, the **deadly triad** (function approximation + bootstrapping + off-policy).

20.13 ML system design: the 7-step framework and napkin math (Ch 15, 17, 18)



Step	Say out loud	§
1. Frame	Family (predictive / generative / agentic — does it output a score, content or an action?); the known system it resembles; scope; ML task: input, output, category, **ML objective**	15.2, 15.3
2. Success	Online business metric + offline ML metric + guardrails; latency (p95/p99); scale (QPS, peak); cost per request or per completed task	15.3
3. Data	Sources; **label source and delay**; freshness (batch / streaming / request-time); feature groups; leakage and privacy	15.3, 16.3, 16.4
4. Model	Baseline (popularity, rules, LR) → GBDT → deep (two-tower, multi-task); name the trigger to upgrade	15.3
5. Serving	What's offline vs per request; caches (with permissions); cheaper paths; timeouts and fallbacks; funnel if the catalogue is big	15.3, 16.5, 16.6
6. Keep it working	Log inputs/outputs/versions → join outcomes → monitor 4 layers → gated retraining → shadow → canary → ramp	15.3, 16.8, 16.9
7. Failures and cost	Specific failure list with controls; cost = requests × share × calls × price, in the same unit as the benefit	15.3

Business goal → ML objective (§15.3): time spent → P(watch ≥ X s) or expected watch time (not raw clicks) · fraud losses → P(fraud) + cost-based thresholds · delivery promise → minutes to delivery + a product-owned buffer · search revenue → P(purchase | query, item) · churn → P(cancel within 30 days) (label is 30 days late).

Four assumptions that break with a model (§15.1): correctness is a spectrum (need quality metrics) · behaviour depends on data (drift) · compute is not cheap (model calls cost money) · failures are silent (a wrong answer is HTTP 200).

Napkin numbers (illustrative, from the chapters)

Quantity	Arithmetic	Result	§
QPS (queries per second)	requests/day ÷ 86,400; peak ≈ 2–3×	10 M/day → 116 avg, ~347 peak (3×)	15.3
Homepage recs	100 M DAU (daily active users) × 5 loads ÷ 86,400	≈ 5,800 avg, ~17 K peak (3×); × 500 candidates ≈ 8.7 M scores/s	17.1
Feed ranking	500 M DAU × 5 sessions × 3 calls ÷ 86,400	≈ 86.8 K/s; × 500 posts ≈ 43 M scores/s	17.2
Ads scoring	100 K requests/s × 300 candidates	30 M scores/s in a 50–100 ms auction	18.1
Vector memory	N × dims × 4 bytes (float32)	50 M × 128 → 25.6 GB; 100 M × 512 → 204.8 GB; PQ to 64 B → 6.4 GB	17.1, 18.6
Batch precompute	users × items per user	25 M × 500 = 12.5 B rows; active 5 M only → 2.5 B	16.5
LLM cost	10 M req/day × $0.01	$100 K/day ≈ $3 M/month	15.1
Support bot	500 K orders × 8% contacts × 10 calls × $0.01	40 K conversations, $4 K/day; compare **per resolved issue**	15.4
Moderation cascade	200 M uploads/day, 1% flagged to heavy models, 0.1% to humans	2 M heavy/day (≈ 23/s), 200 K human decisions/day (~670 reviewer-days)	18.3
A/B sample size	$n \propto 1/\delta^2$	5% CTR, +1% relative lift → ~3.0 M users per arm; half the lift → ~4× users	16.9
Latency budget (small model)	inference 1–10 ms, online-store lookup 2–10 ms, inline features 0–15 ms, routing/auth 1–5 ms, logging 1–2 ms	batch KV lookup ~1 ms vs online 10–100 ms	16.5, 16.6
Fraud	10 K tx/s × 0.2% fraud; ~50 ms authorisation, ~10 ms for the model	~20 fraud attempts/s	18.2
Video indexing	1 M uploads/day × 600 s ÷ 2 s per sampled frame	300 M frame encodings/day ≈ 3,500/s — indexing, not querying, dominates	17.4
Blurring batch	5 M panoramas × 24 tiles × 10 ms	1.2 M GPU-s/day ≈ 14 GPUs busy all day	18.7

Thirteen case studies: the one-row-per-design comparison (task type, features, model, metrics, hardest challenge) is the table at the end of Chapter 18.

20.14 Production and MLOps checklist (Ch 16)

Architecture

- [ ] Five pieces named: training pipeline, feature layer, inference path, serving platform, monitoring loop; offline plane vs online plane (§16.1).
- [ ] Training pipeline: data preparation with **point-in-time-correct joins** → training → evaluation vs production → model registry with lineage (§16.2).
- [ ] One feature definition for training and serving; serving features logged and reused for training (Rules of ML — Google's published list of ML-engineering rules — #29, #32, #37) (§16.2).
- [ ] Feature store: offline history for training, online latest values for serving; batch vs streaming vs request-time features with freshness alerts (§16.4).
- [ ] Inference mode chosen by the deciding question "does it need request-time information?": batch, online, streaming, or precompute-then-rerank (§16.5).

Serving

- [ ] Stateless, versioned model service; itemised latency budget; batched feature reads; hot-entity caches (§16.6; caches, load balancers and timeouts are explained in System Design Fundamentals).
- [ ] Timeouts and an agreed **fallback** (previous model, cached score, heuristic); fail-open vs fail-closed decided per risk (§16.2, §16.6).
- [ ] CPU benchmark before GPUs; distillation / quantisation / pruning if needed (§16.6).
- [ ] Model version logged with every prediction (§16.6).

Evaluation and release

- [ ] Offline metric matches the problem (PR-AUC + threshold, recall@k + NDCG, MAE/RMSE + p90 + cohorts), calibration if scores are used as probabilities, naive and production baselines (§16.7).
- [ ] Release ladder: offline gate (overall **and per segment**) → shadow → sticky canary → ramp with a pre-agreed abort rule; rehearsed rollback of model + features + config (§16.6).
- [ ] A/B test: randomisation unit, one primary metric, guardrails, power (MDE — minimum detectable effect, α, $1 - \beta$), whole weeks, no peeking (or a sequential test), sample-ratio-mismatch check (did each arm get the share of users it should?); interleaving / switchback / holdback when appropriate (§16.9).

Monitoring and retraining

- [ ] Four layers: service health → input distributions (PSI, KS, **null rates**) → prediction distribution → outcomes as labels arrive (§16.8).
- [ ] Data drift vs label shift vs concept drift vs plain bugs; alerts start investigations (§16.8).

- [ ] Retraining cadence set by measured decay and label maturity; triggered retrains go through the same gates; never auto-promote (§16.8).
- [ ] Feedback loops: log impressions and selection probabilities; small risk-approved exploration (§16.8).

MLOps (§16.10)

- [ ] Experiment tracking (code version, params, **data hash**, metrics, artifacts) — MLflow, W&B.
- [ ] Data and model versioning (DVC, lakeFS, Delta), data validation that fails the pipeline (TFDV, Great Expectations, Pandera).
- [ ] Registry with stages and owners; orchestration (Airflow, Kubeflow, Dagster); CI with feature unit tests and training–serving consistency tests; CD with shadow/canary/auto-rollback; continuous training (MLOps level 1 → 2).
- [ ] Test the infrastructure independently from the ML: push a trivial model through, confirm identical offline and online predictions (Rules of ML #5).
- [ ] Labels: source, delay, cost; downsample-then-recalibrate; temporal splits; PII minimised, deletion path defined (§16.3).

20.15 Responsible-ML checklist (Ch 19)

Run the pre-launch checklist in §19.11 (framing → data → model and evaluation → deployment and monitoring). Specifics to remember: pick the criterion by harm (demographic parity, equal opportunity, equalized odds, calibration) with legal input; audit per group **and intersection** with CIs, disparate-impact ratio < 0.8 as a warning, proxy test (predict the group from features); reweighing $w(g, y) = P(g)P(y)/P(g, y)$, fairness penalty, or group thresholds fitted on validation data, then re-audit; SHAP values add up to $f(x) - E[f]$, never causal; human oversight and appeal (GDPR Art. 22); authentication, rate limits, label-only outputs.

Key formulas: disparate impact = $P(\hat{Y}=1 \mid B)/P(\hat{Y}=1 \mid A)$ (0.35/0.50 = 0.70) · Chouldechova: $FPR = p/(1-p)\cdot(1-PPV)/PPV\cdot(1-FNR)$ · ε-DP: $P[M(D) \in O] \le e^{\varepsilon}\cdot P[M(D') \in O]$; Laplace noise scale $\Delta f/\varepsilon$ (count, ε 0.5 → 2); k queries at ε each cost $k\varepsilon$ · FGSM: $x_{adv} = \text{clip}(x + \varepsilon\cdot\text{sign}(\nabla_x L), 0, 1)$ · FedAvg = data-size-weighted average of client weights.

20.16 Decision tables: which algorithm, which metric

Which algorithm for which problem

Problem	Start with (baseline)	Then try	Avoid / notes	§
Tabular classification or regression, < ~1 M rows	Logistic / linear regression; majority-class or mean baseline	Random forest → **gradient boosting** (LightGBM, XGBoost, CatBoost, `HistGradientBoosting`)	Deep nets usually lose on small tables; k-NN beyond ~20–50 features	7.6, 8.4
Many categorical features	CatBoost or `HistGradientBoosting` with native categoricals	Target encoding (cross-fitted) + GBDT	One-hot of 100 K levels for a linear model	4.4, 7.4
High-dimensional sparse text	TF-IDF + logistic regression / linear SVM; Naive Bayes for tiny data	Fine-tuned transformer if data and budget allow	RBF SVM on sparse text	4.6, 8.2, 8.3
Images	Fine-tune a pretrained CNN/ViT (ResNet-50, EfficientNet) with augmentation	Bigger backbone, more data	Training from scratch on a small dataset	11.5
Detection / segmentation	YOLO-style one-stage (real time) or Faster R-CNN; U-Net for masks	Transformer detectors (DETR)	Judging detectors by accuracy (use mAP)	11.7
Sequences / text generation	Transformer (pretrained)	LSTM/GRU for small, streaming or on-device models	Simple RNN for long dependencies	12.4, 12.7
Time-series forecast	Naive and seasonal-naive; ETS / ARIMA for one series	Global GBDT on lag features (model differences); deep models with lots of data	Random splits; trees on trending levels	12.10
Clustering, round groups	k-means (k-means++, scaled, silhouette/elbow for k)	GMM (ellipses, soft), agglomerative (hierarchy)	k-means on moons/rings	9.1–9.4
Clustering, odd shapes / noise	DBSCAN / HDBSCAN	—	One eps for mixed densities (use HDBSCAN)	9.3
Compression / pre-step	PCA (standardise first; ~95% variance)	Kernel PCA, TruncatedSVD for sparse	PCA for visualising non-linear structure	9.5
2-D visualisation	PCA first, then t-SNE / UMAP	UMAP for big data or new points	Reading cluster sizes/gaps in t-SNE	9.6
Anomaly detection, unlabelled	Robust z / IQR per feature; Isolation Forest multivariate	LOF (mixed densities), One-Class SVM (clean novelty data), autoencoders	Plain z-scores on tiny or skewed samples	9.7
Rare-event classification (fraud)	Rules + logistic regression → GBDT with velocity and graph features	Graph neural nets for rings, anomaly scores	Accuracy; single threshold	18.2
Recommendations	Popularity + item-based CF / co-occurrence	Implicit MF (ALS) → two-tower retrieval + GBDT/deep ranker	Pure CF for always-new catalogues	13.4–13.8
Search ranking	BM25 + popularity	Hybrid lexical + semantic retrieval, LambdaMART / deep L2, cross-encoder top-k	Training on raw position-biased clicks	17.3
Choosing among a few options with live feedback	A/B test	Bandits (Thompson, UCB, contextual LinUCB)	Fixed ε forever	14.4
Sequential decisions with a simulator	Value iteration (known model) / Q-learning (small)	DQN (discrete), PPO/SAC (continuous)	Full online RL on live money or people	14.5–14.9
Must explain every decision	Linear/logistic model, small tree, rule list, GAM	GBDT + SHAP reason codes	Deep black box without oversight	19.5

Which metric for which problem

Situation	Primary metric	Also report	Don't	§
Balanced classes, equal costs	Accuracy + confusion matrix	F1, log loss	—	6.3
False positives costly (spam hiding real mail)	Precision at a recall floor	PR curve	Quote precision without threshold and base rate	6.3
Misses costly (screening, fraud)	Recall at a precision floor	Expected cost	—	6.3
Rare positives (≤ ~1%)	PR-AUC / average precision + P/R at the operating threshold	Base rate, expected $ loss	Accuracy; ROC-AUC alone	6.3, 16.7
Comparing rankers, moderate imbalance	ROC-AUC	PR-AUC	—	6.3
Probabilities used in arithmetic (auctions, expected cost)	Log loss, Brier, ECE per segment	Reliability diagram	Trust AUC for calibration	6.5, 18.1
Multiclass with weak minority classes	Macro-F1 / per-class P, R	`classification_report`	Micro-average only	6.3
Regression, errors cost linearly	MAE	p90 error, per cohort	—	6.2
Regression, big misses very costly	RMSE	MAE, p90, per cohort	R^2 alone	6.2
Retrieval / candidate generation	Recall@k	Coverage	NDCG at this stage	6.4, 13.8
Ranked list with graded relevance	NDCG@k (state the gain and ideal)	MAP, MRR	Sampled-negative metrics	6.4, 17.3
One right answer fast	MRR	Hit rate@k	—	6.4
Recommender health	NDCG/MAP + coverage, diversity, novelty	Popularity baseline	Accuracy alone	13.11
Clustering without labels	Silhouette / DB / CH to compare settings	Stability, downstream value	Trust them on non-convex shapes	9.9
Object detection	mAP at IoU thresholds	Per-class AP	Accuracy	11.7
Moderation	Prevalence of violating views per category	Wrongful removals; share of appeals won	Items removed	18.3
Launch decision	Online business metric in an A/B test	Guardrails (latency, complaints, fairness)	Offline metric alone	16.9
Fairness	The criterion that matches the harm (equal opportunity, equalized odds, parity, calibration)	All others + group sizes and CIs	One unnamed "fairness" number	19.2

20.17 Top 50 interview questions

Crisp answers; each points to the full treatment.

Foundations

1. **What's the difference between AI, ML and deep learning?** → AI is the goal of intelligent behaviour (including rules and search); ML learns behaviour from data; deep learning is ML with many-layer neural networks that learn their own features. (§1.1)
2. **Supervised vs unsupervised vs self-supervised vs reinforcement?** → Labels / no labels / labels generated from the data (next token) / rewards from actions in an environment. (§1.4)
3. **Parameters vs hyperparameters?** → Parameters are learned in training (weights); hyperparameters are set before it (learning rate, depth, C) and tuned on validation data. (§1.6)
4. **Why three data splits?** → Train fits parameters, validation chooses hyperparameters and models, the test set is used once so its score stays unbiased. (§1.6, §4.7)
5. **What is data leakage? Give examples.** → Information unavailable at prediction time reaching training or evaluation: post-outcome features, scaling or feature selection fitted before splitting, random splits of time series, duplicates across splits. Prevent with point-in-time features, split-first pipelines and time/group splits. (§1.10, §4.8)
6. **Explain the bias–variance trade-off.** → Test error = bias² + variance + noise. Simple models underfit (high bias), flexible ones overfit (high variance); total error is U-shaped in complexity. (§6.7)
7. **How do you detect and fix overfitting?** → Train score much better than validation; fix with more data, regularisation, a simpler model, early stopping, bagging or augmentation. (§1.10, §6.7)
8. **Explain gradient descent and the learning rate.** → Repeat $w \leftarrow w - \eta \nabla L$. Too small is slow, too large diverges; mini-batches are the practical default; scale features for a rounder loss surface. (§2.5, §5.4)
9. **Explain Bayes' theorem with an example.** → Posterior ∝ likelihood × prior. A 90%-sensitive, 5%-false-positive test for a 1%-prevalence disease gives only ~15% P(sick | positive) — the base-rate effect behind low precision on rare events. (§2.7)
10. **What is a p-value — and what isn't it?** → P(data at least this extreme | H_0). It is not P(H_0 is true); significance ≠ importance; correct for multiple tests and don't peek. (§2.11)
11. **Correlation vs causation?** → Correlation can come from confounders, reverse causation or chance; causal claims need randomised experiments. Models find correlations — fine for prediction, risky for decisions. (§2.9)
12. **What is cross-entropy and why is it the classification loss?** → −ln of the probability given to the true class; it's the maximum-likelihood loss, convex for logistic regression, and punishes confident mistakes hardest, with gradient $p - y$ at the logits. (§2.12, §10.3)

Data

13. **How do you handle missing values?** → Ask why they're missing (MCAR/MAR/MNAR); median or "missing" category plus a missing indicator as a default; KNN/iterative imputation when other columns predict it; fit imputers inside the pipeline; measure by masking known values. (§4.3)
14. **One-hot vs target encoding vs hashing?** → One-hot for small nominal sets; target encoding for high cardinality (cross-fitted and smoothed to avoid leakage); hashing for unbounded IDs (collisions); embeddings in deep models. (§4.4)
15. **Which models need feature scaling?** → Anything using distances, gradients or penalties: k-NN, k-means, SVM, linear/logistic with regularisation, neural nets, PCA. Trees don't. Fit the scaler on training data only. (§4.5)
16. **How do you deal with class imbalance?** → Change the metric (precision/recall, PR-AUC); move the threshold; class weights or downsampling (then recalibrate: $p \cdot w/(p \cdot w + 1 - p)$); SMOTE sometimes; resample training data only. (§4.9)
17. **Where do labels come from, and what goes wrong?** → Outcomes (late), behaviour (biased by exposure and position), human annotation (costly, needs agreement checks), weak supervision and LLMs (correlated errors). (§4.1, §16.3)

Classic models

18. **Linear vs logistic regression?** → Linear predicts a number by minimising MSE; logistic predicts a probability through a sigmoid by minimising log loss; its boundary is $w \cdot x + b = 0$ and e^w is an odds ratio. (§5.2, §5.8)
19. **Normal equation vs gradient descent?** → Exact $\theta = (X^TX)^{-1}X^Ty$, $O(d^3)$, no tuning — fine for small d; GD scales to huge n and d and any differentiable loss but needs a learning rate and scaled features. (§5.3)
20. **Ridge vs Lasso?** → L2 shrinks all weights and shares weight among correlated features; L1 sets some weights exactly to zero (feature selection). ElasticNet mixes both. More regularisation → more bias, less variance. (§5.7)
21. **How does a decision tree choose a split?** → It tries every feature and threshold and keeps the split with the largest impurity decrease (Gini or entropy; variance for regression); greedy, then prune by depth/leaf size or ccp_alpha. (§7.1)
22. **Bagging vs boosting?** → Bagging trains deep trees in parallel on bootstrap samples and averages (cuts variance); boosting adds shallow trees sequentially, each fitting the current errors (cuts bias). (§7.2–7.4)
23. **Random forest vs gradient boosting?** → RF: robust, little tuning, doesn't overfit by adding trees, free OOB score. GBDT: usually the most accurate on tables but needs a small learning rate and early stopping. (§7.3, §7.4)
24. **Why can feature importance mislead?** → Impurity importance is computed on training data, favours high-cardinality features and splits credit among correlated ones; use permutation importance on held-out data or SHAP. (§7.3, §19.5)
25. **Explain SVMs and the kernel trick.** → Maximise the margin $2/\|w\|$ (C trades width for violations); the dual needs only dot products, so a kernel $K(x, z) = \varphi(x) \cdot \varphi(z)$ gives non-linear boundaries without building φ. Tune C and γ together. (§8.2)
26. **Why is Naive Bayes "naive", and why does it still work?** → It assumes features are independent given the class; classification only needs the argmax right, so it works well for text, but its probabilities are over-confident. (§8.3)
27. **How does k-NN work and when does it fail?** → Vote/average of the k closest scaled points; prediction is O(nd) and distances concentrate in high dimensions (curse of dimensionality). (§8.1)
28. **How does k-means work and how do you choose k?** → Assign to nearest centroid, recompute means, repeat (local optimum; k-means++ and restarts). Choose k by silhouette, elbow and domain sense; it assumes round, similar clusters. (§9.1)
29. **What does PCA do?** → Rotates standardised data onto orthogonal directions of maximum variance (eigenvectors of the covariance / SVD); keep enough components for e.g. 95% variance. Linear and unsupervised. (§9.5)

Evaluation

30. **Precision vs recall — which do you optimise?** → Depends on error costs: precision when false alarms hurt, recall when misses hurt; set the threshold where $p > C_{FP}/(C_{FP} + C_{FN})$ with calibrated scores, or use two thresholds with a review band. (§6.3)
31. **ROC-AUC vs PR-AUC?** → ROC-AUC = P(a positive outranks a negative), insensitive to base rate; with rare positives, FPR's huge denominator hides false alarms, so PR-AUC (no-skill = base rate) is the honest metric. (§6.3, §16.7)
32. **What is calibration and when does it matter?** → Predicted probabilities matching observed frequencies — separate from ranking. Needed when scores feed arithmetic (auctions, expected cost, human-read risk scores). Fix with Platt or isotonic on held-out data. (§6.5, §18.1)
33. **Explain k-fold cross-validation and its variants.** → k train/validate rounds averaged (mean ± std); stratified for classes, group folds for entities, forward-in-time splits with a gap for temporal data; nested CV for an unbiased tuned estimate. (§6.6)
34. **Grid vs random search vs Bayesian optimisation?** → Grid for 1–2 parameters; random search explores each dimension better on the same budget; Bayesian (Optuna) models score vs hyperparameters for expensive trials; log scales; test once. (§6.9)
35. **Explain NDCG.** → DCG sums graded relevance discounted by $\log_2(\text{rank} + 1)$; dividing by the ideal DCG normalises to [0, 1]. Rewards putting the most relevant items at the top. (§6.4, §17.3)

Deep learning

36. **How does backpropagation work?** → The chain rule applied from the loss backwards through the computational graph, reusing intermediate values, giving every gradient for about the cost of an extra forward pass; an optimizer then updates the weights. (§10.4)
37. **Why do we need non-linear activations, and which ones?** → Without them any stack is one linear map. ReLU for hidden layers (GELU/SiLU in transformers); sigmoid/softmax/identity at the output by task. (§10.2)
38. **Vanishing and exploding gradients — causes and fixes?** → Gradients are products of per-layer (or per-time-step) factors. Fix with ReLU, He/Xavier initialisation, normalisation, residual connections, gating (LSTM/GRU) and gradient clipping. (§10.6, §12.3)
39. **SGD vs momentum vs Adam vs AdamW?** → Momentum accumulates velocity; RMSProp/Adam scale each parameter's step by its recent gradient size (Adam adds bias correction); AdamW decouples weight decay. Adam(W) is the default for transformers; SGD + momentum stays strong for CNNs. (§10.5)
40. **What do dropout and batch normalisation do?** → Dropout zeroes random units in training so the net can't rely on co-adapted features (off at eval); BatchNorm normalises activations per mini-batch, allowing higher learning rates and adding mild regularisation (running stats at eval). (§10.7)
41. **Why CNNs for images?** → Weight sharing and local connectivity give far fewer parameters and translation equivariance; stacked 3×3 convs with downsampling grow the receptive field; ResNet skip connections make very deep nets trainable. (§11.1, §11.4)
42. **What is transfer learning?** → Start from a model pretrained on a big dataset, replace the head, freeze then fine-tune with a small learning rate; biggest gains when labels are scarce. (§11.5)

43. **How does an LSTM fix the RNN's problems?** → An additive cell state $c_t = f \odot c_{t-1} + i \odot \tilde{c}_t$ controlled by sigmoid gates lets information and gradients flow across many steps. (§12.4)
44. **Explain attention and the transformer.** → softmax($QK^T/\sqrt{d_k}$)V is a learned weighted lookup; transformers stack multi-head self-attention and feed-forward layers with residuals and LayerNorm plus positional encodings — parallel over the sequence, O(1) path between tokens, $O(n^2)$ cost. (§12.6, §12.7)

Recommenders, RL and systems

45. **Content-based vs collaborative filtering, and cold start?** → Attributes vs co-behaviour; CF finds quality and surprise but fails on new users/items. Cold start: popularity, onboarding, content features and controlled exploration. (§13.4, §13.10)
46. **How do you retrieve candidates from 100 M items in milliseconds?** → Two-tower model: item vectors precomputed into an ANN index (HNSW/IVF), user tower per request; in-batch negatives with logQ correction; tower and index shipped as one version; then a ranker. (§13.7–13.9)
47. **Q-learning vs SARSA? Exploration vs exploitation?** → Q-learning bootstraps from the max next action (off-policy, learns the optimal policy); SARSA from the action actually taken (on-policy, safer while exploring). Exploration: ε-greedy, UCB, Thompson sampling. (§14.4, §14.7)
48. **Walk me through designing an ML system.** → Frame → success metrics → data and labels → baseline → serving path → keep-it-working loop → failures and cost, translating the business goal into an observable ML objective and keeping product knobs (thresholds, buffers) outside the model. (§15.3)
49. **How do you monitor a model in production, and when do you retrain?** → Health → input drift (PSI/KS/nulls) → prediction drift → outcomes; alerts trigger investigation (often a broken pipeline); retrain on measured decay through offline, shadow and canary gates. (§16.8)
50. **Your model is 94% accurate — is it fair?** → Unknown until you slice by group: selection rate, TPR, FPR, PPV and calibration per group, proxy checks; criteria conflict when base rates differ, so choose by harm and mitigate (reweighing, penalties, thresholds). (§19.2–19.4, §19.12)

20.18 Glossary (A–Z)

911 terms. Each entry: **term** — plain meaning — the section that teaches it (the link opens that chapter).

Term	Meaning	Where
1×1 convolution	Convolution mixing channels at each pixel; used to shrink or expand channel count	11.1
A2C / A3C / SAC / DDPG / TD3	Common deep actor-critic algorithms (synchronous, asynchronous, entropy-regularised, deterministic for continuous actions)	14.9
Ablation	Removing one component or feature group and retraining to measure its contribution	18.8
A/B test	Randomised experiment splitting units between control and treatment to measure a business metric	16.9
Accuracy	Share of predictions that are correct, (TP + TN)/all; misleading on imbalanced data	6.3
Acquisition function	Rule in Bayesian optimisation deciding which setting to try next (e.g. expected improvement)	6.9
Action ladder	Graded moderation responses (remove, review, reduce reach/age-gate/warn, allow) set per category and region	18.3
Activation function	Non-linear function applied to a neuron's pre-activation (ReLU, sigmoid, tanh, GELU)	10.2
Active learning	Routing the examples with the most model uncertainty (or annotator disagreement) to human labellers first	4.1
Actor-critic	Policy-gradient method where a critic's TD error δ serves as the advantage for the actor's update	14.9
AdaBoost	Boosting by re-weighting misclassified samples; each stump gets a vote α = ½ ln((1 − ε)/ε)	7.4
AdaGrad	Optimizer dividing each step by the root of the summed squared past gradients	10.5
Adam	Optimizer combining momentum and RMSProp-style per-parameter step sizes, with bias correction	10.5
Adamic-Adar	Link-prediction score summing 1/log(degree) over shared neighbours, so rare mutual friends count more	18.4
AdamW	Adam with decoupled weight decay applied directly to the weights	10.5
Adjusted cosine	Cosine similarity after subtracting each user's mean rating	13.3
Adjusted Rand Index (ARI)	Agreement between two clusterings corrected for chance: about 0 random, 1 perfect	9.9
Advantage	How much better an action is than the state's average, A = Q − V	14.8
Adversarial drift	Data changing because attackers adapt to your defences	18.2
Adversarial example	Input perturbed slightly, on purpose, so that a model misclassifies it	19.7
Adversarial training	Including adversarial examples in each training batch to gain robustness	19.7
Agent / environment	The learner that acts / the world it acts in and gets rewards from	14.1
Agentic AI	Systems that plan and take multi-step actions toward a goal using tools	1.1
Agglomerative clustering	Bottom-up clustering that repeatedly merges the two closest clusters	9.2
Aggregation / evaluation / deployment bias	One model for groups that differ / an unrepresentative benchmark / use outside the intended setting	19.1
AI (artificial intelligence)	The broad goal of making machines behave intelligently, by rules, search or learning; ML is one part of it	1.1
AlphaGo / AlphaZero	DeepMind systems combining deep RL and tree search that beat top players at Go, chess and shogi	14.11
ALS (alternating least squares)	Matrix-factorisation solver that fixes one side and solves a ridge regression for the other, alternately	13.5
Anchor boxes	Preset box shapes a detector adjusts instead of predicting boxes from scratch	11.7
ANN (approximate nearest neighbour)	Search that returns almost the exact nearest vectors while scanning only a fraction of them	13.9
Anomaly detection	Finding rare points that differ markedly from the majority, usually without labels	9.7
Anonymisation	Irreversibly removing the ability to identify people; removing names is not enough	19.6
API (application programming interface)	Defined way for one program to call another, e.g. sending a request to a model service and reading the answer	1.1
Apriori	Frequent-itemset algorithm that prunes candidates because every subset of a frequent set must be frequent	9.8
ARIMA	Time-series model: autoregression on p lags, d differences, moving average on q past errors	12.10
Article 22 (GDPR)	Right not to be subject to solely automated decisions with legal or similarly significant effects	19.9
Artifact (ML)	Saved output of a training run, such as a model file, index or preprocessing pipeline	13.7
Atomic swap (promotion)	Switching model and index versions together in one step so no request sees a mix	13.7
Attention	Weighted average of value vectors with weights softmax(q·k/√d_k) from query–key similarity	12.6
Attribution window	Period after a click during which a conversion is credited to it	18.1
AUC (ROC-AUC)	Probability a random positive is scored above a random negative; 0.5 random, 1 perfect	6.3
Autoencoder	Network trained to reconstruct its input through a narrow middle layer; large reconstruction error suggests an anomaly	9.7

Term	Meaning	Where
Autograd / automatic differentiation	Software computing exact derivatives by applying the chain rule to recorded tensor operations; PyTorch runs backprop on `.backward()`	10.4
Automatic speech recognition (ASR)	Converting speech audio into a (timestamped) transcript, e.g. to make videos searchable	17.4
Average pooling	Downsampling by averaging each window	11.2
Average precision (AP)	Area under the precision–recall curve, the usual PR-AUC; no-skill value equals the base rate	6.3
Backbone	Pretrained feature-extracting part of a network, reused under new heads	11.2
Backdoor (trojan)	Hidden trigger planted through poisoned training data that makes the model misbehave on command	19.7
Backfill	Computing a new feature or table for past dates	16.4
Backpressure	Slowing producers when consumers can't keep up, so queues don't overflow	16.5
Backpropagation (backward pass)	Chain rule applied from the loss backwards to get every parameter's gradient efficiently	10.4
Backpropagation through time (BPTT)	Backprop through an RNN unrolled over the sequence; gradients summed over time steps	12.2
Backtest	Replaying a model or policy on past data whose outcomes are already known	18.2
Bagging	Training copies of a model on bootstrap samples and averaging them to cut variance	7.3
Bag-of-words	Text representation counting each word's occurrences and ignoring order	4.6
Bandit (multi-armed)	One-state RL problem of choosing among arms with unknown payoffs; isolates exploration vs exploitation	14.4
Baseline	Score of a trivial predictor (majority class, mean, last value, popularity) every model must beat	6.1
Base rate	How often the positive class occurs in the population; drives precision and PR-AUC	6.3
Base-rate fallacy	Ignoring how rare something is, e.g. trusting a positive test for a rare disease too much	2.7
Basis point (bp)	One hundredth of a percent (0.01%)	18.2
Batch inference	Predictions precomputed on a schedule and stored so a request becomes a lookup	16.5
Batch learning	Training on the full dataset at once and redeploying a frozen model periodically	1.7
Batch normalisation (BatchNorm, BN)	Layer normalising activations over the mini-batch, then scaling and shifting with learned γ and β	10.7
Bayesian optimisation	Tuning with a surrogate model of score vs hyperparameters and an acquisition function	6.9
Bayes' theorem	$P(A\|B) = P(B\|A)\cdot P(A)/P(B)$: posterior ∝ likelihood × prior	2.7
Beam search	Decoding that keeps the k most probable partial sequences at each step	12.5
Bellman equation	Value now = expected reward now + γ × value of the next state	14.3
Bernoulli distribution	Single yes/no trial that succeeds with probability p	2.6
BERT / GPT	Landmark encoder-only (understanding) and decoder-only (generation) transformer families	12.7
Bias (fairness)	Systematic, unjustified difference in how a model treats or performs for groups	19.1
Bias mitigation (pre / in / post-processing)	Fixing the data / the training objective / the decision rule to improve a fairness criterion	19.4
Bias (statistical)	Error from wrong assumptions; distance of the average model from the truth	6.7
Bias–variance trade-off	Simple models miss patterns (bias), flexible ones chase noise (variance); the best model balances the two	1.10
BIC	Bayesian Information Criterion: likelihood penalised by parameter count, lower is better (choose k in GMMs)	9.4
Bidirectional RNN	Two RNNs reading the sequence forwards and backwards, outputs combined	12.2
Binary classification	Predicting one of two classes (yes/no, fraud/legit)	5.8
Binary cross-entropy (BCE)	Log loss for 0/1 targets with sigmoid outputs	10.3
Binomial distribution	Number of successes in n independent yes/no trials	2.6
BM25	Lexical relevance score rewarding rare query terms, with diminishing returns and length normalisation	17.3
Boosting	Building an ensemble sequentially, each weak model correcting the current errors	7.4
Bootstrap confidence interval	Interval from resampling the data with replacement many times and taking percentiles of the statistic	2.11
Bootstrapping (RL)	Updating an estimate using other estimates (e.g. V(s')) instead of waiting for the full return	14.6
Bootstrap sample	n rows drawn with replacement from n rows; about 36.8% of rows are left out	7.3
BPE (byte-pair encoding)	Subword tokenizer that repeatedly merges the most frequent adjacent symbol pair	12.8
Brier score	Mean squared error of predicted probabilities; a proper scoring rule for calibration	6.5
Broadcasting	NumPy rule for combining arrays of different shapes by stretching size-1 dimensions	3.2
Budget interference	A/B arms competing for the same campaign budgets and distorting each other's results	18.1

Term	Meaning	Where
Budget pacing	Spreading an ad budget over the day by throttling or boosting bids	14.11
C2PA	Industry standard for attaching tamper-evident provenance ('Content Credentials') to media	19.7
Cache	Fast store of recently computed results so repeated requests skip the work	16.4
Cache invalidation	Removing or refreshing cached entries when the underlying data changes	17.3
Calibration	Predicted probabilities matching observed frequencies; separate from ranking quality	6.5
Calibration within groups	A score of s means an s chance of the outcome in every group	19.2
Canary release	Giving a new model to a small slice of real traffic before ramping up	16.6
Candidate generation	First funnel stage that retrieves a few hundred plausible items from millions, judged by recall	13.8
Cannibalisation	A new feature stealing outcomes that would have happened anyway elsewhere on the site	18.9
Cardinality	Number of distinct values a categorical feature has	4.2
CART	Classification and Regression Trees: the binary-split tree algorithm used by scikit-learn	7.1
Cascade	Cheap checks on everything, expensive checks only on what earlier stages flag	18.3
Catalogue coverage	Share of all items that ever get recommended	13.11
CatBoost	Gradient-boosting library with ordered boosting and strong native categorical handling	7.4
Categorical cross-entropy	−ln of the probability the softmax gives the true class	10.3
Causal mask	Hides future positions so each token attends only to earlier ones (decoders)	12.7
CCPA / CPRA	California privacy laws giving rights to know, delete, correct and opt out	19.9
CDF (cumulative distribution function)	P(X ≤ x): share of values at or below x	10.2
Cell state (LSTM)	LSTM's long-term memory, updated additively so gradients survive many steps	12.4
Censored label	Outcome never observed because your own action (e.g. blocking a payment) prevented it	18.2
Central limit theorem (CLT)	Sample means are approximately normal around the true mean with spread σ/√n	2.10
Centroid	Mean point of a cluster	9.1
Chain rule	Derivative of nested functions = product of the local derivatives	2.4
Channel	One 2-D slice of an image or feature map (e.g. red, green, blue)	11.1
Chargeback	Card payment reversed after the cardholder disputes it, at the merchant's cost	4.1
Churn	Customers leaving or cancelling; a common prediction target	4.1
CI/CD	Continuous integration (automatic tests on every change) and continuous delivery (automated safe release)	16.10
Classification	Supervised task predicting a category	1.5
Class imbalance	One class much rarer than the other(s), as in fraud or disease	4.9
Class weights	Loss weights that make mistakes on the rare class cost more	4.9
CLIP	Image–text model trained contrastively so images and captions share one embedding space	18.6
[CLS] token	Special token whose final vector summarises the whole input for classification (BERT, ViT)	11.4
Clustering	Grouping similar examples together without labels	9.1
CNN (convolutional neural network)	Network built from convolution and pooling layers; the classic architecture for images	11.1
Cohen's kappa	Chance-corrected agreement between two annotators, (p_o − p_e)/(1 − p_e)	4.1
Cohort	Group of users sharing a trait (sign-up month, country); metrics per cohort reveal hidden failures	6.2
Cold start	Recommending with little or no history for a new user, item or system	13.10
Collaborative filtering (CF)	Recommending from other users' behaviour: people who liked what you liked also liked X	13.4
ColumnTransformer	scikit-learn tool applying different preprocessing to different column groups	3.5
COMPAS	US recidivism-risk score whose 2016 analysis sparked the fairness-impossibility debate	19.10
Compound scaling	Growing a network's depth, width and input resolution together in fixed ratios	11.4
Computational graph	Record of the operations that produced a value, used to compute gradients	10.4
Concept drift	Change in the relationship between inputs and the label, P(Y|X)	16.8
Conditional probability	Probability of A once you know B happened, P(A | B)	2.7
Condition number	Ratio of a matrix's largest to smallest stretch; large values make gradient descent slow and solutions unstable	5.4
Confidence (association rules)	Share of baskets with A that also contain B	9.8
Confidence interval (CI)	Range built by a procedure that captures the true value a stated share of the time	2.11

Term	Meaning	Where
Confounder	Hidden factor that drives two variables and makes them look causally linked	2.9
Confusion matrix	Table of TP, FP, FN and TN counts from which classification metrics are computed	6.3
Container	Packaged app plus its dependencies that runs the same anywhere (e.g. Docker)	16.10
Content-based filtering	Recommending items whose attributes resemble what the user liked	13.4
Contextual bandit	Bandit whose choice depends on context features (e.g. LinUCB)	14.4
Contextual embedding	Word vector that depends on the surrounding sentence (from models like BERT)	12.9
Context vector	Fixed-size summary of the input passed from encoder to decoder	12.6
Context window	Maximum number of tokens a model can read at once	12.8
Continuous training	Automatically retraining and validating models on new data	16.10
Contrastive loss	Loss pulling matching pairs together and pushing non-matching pairs apart (e.g. InfoNCE)	18.6
Conversion	The action an advertiser actually wants, such as a purchase or sign-up	18.1
Convex function	Bowl-shaped function with a single minimum, so gradient descent cannot get stuck in a worse one	2.5
Convolution	Sliding a small shared kernel over an input and summing element-wise products into a feature map	11.1
Co-occurrence	Items appearing together in the same sessions or baskets	17.1
Copy-on-write	pandas 3 behaviour where derived objects behave as copies, so edits never leak back silently	3.3
Core / border / noise point	DBSCAN roles: dense centre point / near a core point / isolated outlier labelled −1	9.3
Corpus	Collection of text used for training or analysis	12.8
Correlation	Strength of co-movement; Pearson for linear, Spearman for monotonic (ranks)	2.9
Cosine similarity	Dot product divided by both lengths; compares direction only	2.1
Cost-complexity pruning	Removing tree branches that don't pay for their leaves, controlled by ccp_alpha	7.1
Counterfactual explanation	Smallest actionable input change that flips a decision	19.5
Covariance matrix	Matrix of how each pair of features varies together; PCA takes its eigenvectors	9.5
CPC (cost per click)	Pricing where the advertiser pays only when the ad is clicked	18.1
CPU (central processing unit)	A computer's general-purpose processor; fine for classic ML and small models	3.7
Credit assignment	Working out which earlier actions deserve credit for a delayed reward	14.1
C (regularisation parameter)	Inverse regularisation strength: large C fits training data tightly, small C prefers a simpler boundary	8.2
Cross-attention	Attention with queries from the decoder and keys/values from the encoder	12.7
Cross-correlation	What deep-learning 'convolution' actually computes: sliding the kernel without flipping it	11.1
Cross-encoder	Model reading query and item together for an accurate but expensive relevance score	17.3
Cross-entropy	Average surprise $-\Sigma\ p \log q$ when reality is p and the prediction is q; the classification loss	2.12
Cross-fitting	Computing a feature (e.g. target encoding) for each fold from the other folds only, to avoid leakage	4.4
Cross-modal retrieval	Searching one kind of data (video) with another (text)	17.4
Cross-validation (CV)	Rotating the validation role across k folds and averaging the scores	6.6
CSAM	Child sexual abuse material; handled by a dedicated, legally governed process	18.3
CSAT (customer satisfaction)	Survey score of how satisfied customers are	15.1
CSV	Comma-separated values: a plain-text table format	3.3
CTR (click-through rate)	Clicks divided by impressions	18.1
CUPED	Variance-reduction method that adjusts experiment metrics with pre-experiment data	16.9
Curse of dimensionality	In high dimensions space empties and distances concentrate, so "nearest" loses meaning	8.1
Cyclical encoding	Encoding periodic values as (sin, cos) so December sits next to January	4.6
DAG (directed acyclic graph)	Steps connected by one-way dependencies with no cycles; how pipelines are described	16.10
Data augmentation	Creating extra training examples with label-preserving transformations	10.7
Data drift	Change in the input distribution P(X) (covariate shift)	16.8
DataFrame / Series	pandas table with labelled columns / a single labelled column	3.3
Data leakage	Information unavailable at prediction time reaching training or evaluation	4.8
Data minimisation	Collecting and keeping only the personal data you actually need	19.9
Data poisoning	Attack that corrupts training data, possibly planting a backdoor trigger	19.7

Term	Meaning	Where
Dataset / DataLoader	PyTorch classes that give single examples / shuffle and batch them for training	10.1
Datasheet for datasets	Document describing how a dataset was created, what it contains and how it should be used	19.8
Data validation (TFDV, Great Expectations)	Automatic checks that data matches the expected schema and ranges before it is used	16.10
Data versioning (DVC)	Tracking dataset versions like code so any model can be reproduced	16.10
Data warehouse / data lake	Store of cleaned tables for analysis / store of raw files of any kind	16.2
DAU / MAU	Daily / monthly active users	17.1
Davies–Bouldin / Calinski–Harabasz	Internal clustering scores: cluster overlap (lower is better) / between- vs within-cluster spread (higher is better)	9.9
DBSCAN	Density-based clustering: core points with ≥ min_samples neighbours within eps; others border or noise	9.3
DCG (discounted cumulative gain)	Sum of relevance scores, each divided by $\log_2(\text{position} + 1)$, so lower positions count less	6.4
Deadly triad	Function approximation + bootstrapping + off-policy learning, which can make RL diverge	14.8
Decision boundary	Where the classifier's decision changes; for logistic regression, $w \cdot x + b = 0$	5.8
Decision stump	Tree with a single split; the classic weak learner in boosting	7.4
Decision tree	Model predicting by a sequence of feature-threshold questions ending in leaves	7.1
Deepfake	Fake video, audio or image impersonating a real person	19.7
Deep learning	Machine learning with many-layer neural networks that learn their own features	1.1
Degenerate feedback loop	Model's own outputs narrowing its future data until it only sees what it already likes	16.8
Delayed feedback	Labels (conversions, chargebacks) arriving long after the prediction	18.1
Demographic parity	Fairness criterion requiring equal approval (selection) rates across groups	19.2
Dendrogram	Tree of agglomerative merges; slicing it at a chosen height yields the clusters	9.2
Dense (fully connected) layer	Layer where every output is connected to every input	10.1
Depthwise-separable convolution	Per-channel spatial convolution followed by a 1×1 convolution; far cheaper than a full convolution	11.1
Derivative	Rate at which a function's output changes as its input changes; the slope	2.4
Determinant	Single number saying how much a square matrix scales area or volume; 0 means it has no inverse	2.2
DETR	Transformer object detector that predicts a set of boxes directly, without anchors or NMS	11.7
Dice loss	Segmentation loss based on mask overlap, robust to small foreground areas	11.7
Differencing	Replacing a series by its step-to-step changes to remove trend	12.10
Differential privacy	Guarantee that an algorithm's output barely changes whether any one person is in the data (parameter ε)	19.6
Diffusion model	Generative model that learns to turn random noise into an image step by step	11.7
Dimension	Number of entries in a vector, or one axis of an array	2.1
Dimensionality reduction	Describing data with fewer numbers (PCA, t-SNE, UMAP)	9.5
Discount factor (γ)	Weight on future rewards; horizon ≈ $1/(1-\gamma)$	14.2
Disparate impact ratio	Selection rate of one group divided by another's; below 0.8 fails the four-fifths rule of thumb	19.2
Disparate treatment / impact	Treating people differently because of a protected attribute / a neutral rule with unequal effect	19.4
Distillation	Training a small student model to mimic a large teacher	16.6
Distributional hypothesis	Words that appear in similar contexts tend to have similar meanings	12.9
Distribution shift	Production data differing from training data	16.2
Domain randomisation	Randomising a simulator's details so a policy transfers to the real world	14.11
Dot product	Sum of element-wise products of two vectors; large when they point the same way	2.1
Double DQN / Dueling DQN	DQN variants that reduce over-estimated values / separate state value from action advantage	14.8
DPIA (data protection impact assessment)	GDPR risk assessment required before high-risk processing	19.9
DPO	Direct Preference Optimization: trains a policy on preference pairs without a separate reward model or RL loop	14.10
DP-SGD	Training with per-example gradient clipping plus Gaussian noise to get differential privacy	19.6
DQN	Deep Q-Network: neural Q-function trained with experience replay and a target network	14.8
Dropout	Randomly zeroing units during training so the network can't rely on co-adapted features	10.7
dtype	Data type of an array's elements (e.g. int64, float32); sets memory use and precision	3.2

Term	Meaning	Where
Dual form (SVM)	Rewriting the SVM problem so it uses only dot products between examples, which enables kernels	8.2
Dummy-variable trap	One-hot columns summing to 1 are collinear with the intercept; drop one for unregularised OLS	4.4
Dwell time	How long a user stays on an item or page	13.1
Dynamic programming (RL)	Computing optimal values from a known model by repeated Bellman backups	14.5
Early stopping	Halting training when validation loss stops improving for a set patience	6.10
ECOA / Regulation B	US credit law requiring lenders to give specific reasons for denials	19.9
EDA (exploratory data analysis)	First look at a dataset: shapes, types, missing values, distributions and relationships	4.2
Effect size	How big a difference is, separate from whether it is statistically significant	16.8
ε-greedy	Explore a random action with probability ε, otherwise exploit the best known	14.4
Eigenvector	Direction a matrix only stretches, $Av = \lambda v$; λ is the eigenvalue	2.3
ElasticNet	Linear regression with a mix of L1 and L2 penalties	5.7
Elbow method	Choosing k where adding clusters stops reducing inertia much	9.1
ELU / PReLU / SELU	ReLU variants with a smooth or learned slope for negative inputs	10.2
EM algorithm (expectation–maximisation)	Alternates estimating soft assignments (E-step) and refitting parameters (M-step); fits GMMs	9.4
Embedding	Short dense learned vector representing a discrete thing (word, user, item)	13.6
Encoder / decoder	Part of a network that compresses input into a representation / part that expands it into the output	11.7
Encoder-only / decoder-only model	Transformer that reads text for understanding (BERT) / that generates text left to right (GPT)	12.7
Ensemble	Combination of several models' predictions	7.5
Entropy	Average uncertainty of a distribution, $-\Sigma\ p \log_2 p$ (bits)	2.12
Episode	One run from a start state to an end state	14.1
Episodic vs continuing task	Task with a natural end / task that goes on forever	14.2
Epoch	One full pass over the training data	2.5
Equalized odds	Fairness criterion requiring equal TPR and FPR across groups	19.2
Equal opportunity	Fairness criterion requiring equal true-positive rates across groups	19.2
ESMM	Ads model predicting pCTR × pCVR on all impressions so the conversion head isn't trained only on clicked ads	18.1
Estimator	Any scikit-learn object with fit(); models and transformers are estimators	3.5
ETA (estimated time of arrival)	Predicted time until something arrives, e.g. a delivery	4.6
EU AI Act	EU regulation sorting AI systems into risk tiers; credit scoring and hiring are high-risk	19.9
Euclidean / Manhattan distance	Straight-line distance / sum of absolute coordinate differences	8.1
Evasion attack	Crafting inputs at prediction time so the model misclassifies them	19.7
Exact / perceptual hash	Fingerprint that changes with any edit / one that stays similar after small edits like crops	18.3
Expectation (expected value)	Probability-weighted average of a random variable, E[X]	2.6
Expected calibration error (ECE)	Bucket-size-weighted average gap between mean predicted probability and observed rate	18.1
Expected cost	Probability-weighted cost of a decision; used to set thresholds	18.2
Experience replay	Storing transitions and training on random mini-batches of them to decorrelate updates	14.8
Experiment tracking	Logging each run's code version, parameters, data and metrics (e.g. MLflow)	16.10
Explainability	Ability to say why a model made a prediction in human-checkable terms	19.5
Explained variance ratio	Share of total variance captured by each principal component	9.5
Explicit / implicit feedback	Direct ratings / behaviour such as clicks, plays and purchases	13.2
Exploding gradient	Gradients growing exponentially through layers or time steps, causing huge updates or NaN	10.6
Exploration–exploitation	Trade-off between trying options to learn and using the best-known option	14.4
Exponential smoothing (ETS)	Forecasting with weighted averages where recent values count more	12.10
Exposure bias	Logs only show outcomes for items the system chose to show	4.1
F1 score	Harmonic mean of precision and recall	6.3
Factorisation machine	Model scoring every pair of features through dot products of small learned vectors	18.1
Fail closed	On failure, block the action (don't publish, don't suggest) rather than proceed; used where errors harm people	18.7
Fail open	On failure, let the action proceed (approve, publish) rather than block it	16.2

Term	Meaning	Where
Fairlearn / AIF360	Open-source toolkits for measuring and mitigating unfairness (Microsoft / IBM origins)	19.3
Fairness audit	Computing metrics per group and comparing them	19.3
Fairness criterion	Precise statistical condition on predictions across groups	19.2
Fairness penalty	Loss term that grows with the gap between groups	19.4
Fairness through unawareness	Dropping the sensitive column; weak because proxies carry the same information	19.3
FAISS	Meta's library for fast exact and approximate vector search	8.1
Fallback	Pre-agreed cheaper answer used when the model fails (previous model, cached score, rule)	16.2
False negative / false positive	A real positive the model missed / a negative the model wrongly flagged	4.1
False-removal / appeal-overturn rate	Share of removals that were wrong (by audit) / share of appeals that win	18.3
Fan-out on write / read	Pushing a new post into followers' feeds at write time / pulling posts when a feed is read	17.2
Fβ score	Weighted harmonic mean of precision and recall; β > 1 favours recall	6.3
FCN (fully convolutional network)	Network with only convolutional layers, so it outputs a map (e.g. per-pixel labels)	11.7
Feature	Measurable input property used by a model	1.6
Feature cross	New feature combining two others (device × ad category)	13.8
Feature engineering	Creating new inputs that make the pattern easier to learn	4.6
Feature hashing	Mapping categories to a fixed number of buckets with a hash function (hashing trick)	4.4
Feature importance	Score of how much each feature contributes (impurity-based, permutation, SHAP)	7.3
Feature map	Output grid of one convolution kernel applied across the input	11.1
Feature store	System serving consistent features: offline history for training, online latest values for serving	16.4
FedAvg	Federated averaging: the server averages client model weights, weighted by client data size	19.6
Federated learning (FL)	Training across devices or institutions by averaging locally trained model updates (FedAvg)	19.6
Feedback loop	A model's outputs shaping its own future training data	16.8
Feed-forward network	Network where data flows one way from input to output with no loops	10.1
FGSM	Fast Gradient Sign Method: $x + \varepsilon \cdot \text{sign}(\nabla_x \text{ loss})$ creates an adversarial example	19.7
Filtered ANN	Nearest-neighbour search restricted by filters such as category or price	18.6
Filter (kernel)	Small learned weight grid slid over the input in a convolution	11.1
Fine-tuning	Continuing training of a pretrained model on your task with a small learning rate	11.5
Flatten	Reshaping a feature map into one long vector before dense layers	11.1
FLOPs	Floating-point operations: a count of arithmetic work, used to compare model compute cost	11.4
Focal loss	Detection loss that down-weights easy examples so training focuses on hard ones	18.7
Fold	One of the k parts data is split into for cross-validation	6.6
Forecast horizon	How far ahead a forecast predicts	12.10
Forget gate	LSTM gate deciding which cell-state entries to keep or erase	12.4
Forward pass	Running inputs through the network to get predictions and the loss	10.3
Foundation model	Large model pretrained on broad data and adapted to many tasks	11.8
Four-fifths rule	US rule of thumb flagging a selection-rate ratio below 0.8 between groups	19.2
FP-Growth	Frequent-itemset mining without candidate generation, using a prefix tree	9.8
Fraud ring	Group of colluding accounts, often visible as a dense cluster in a graph	18.2
Freezing layers	Keeping pretrained weights fixed while training only the new layers	11.5
Friends-of-friends (FoF)	Candidates two hops away in a social graph; closing triangles	18.4
FTRL	Follow-the-regularised-leader: online learning rule for huge sparse models such as ad CTR	18.1
Function approximation	Representing values or policies with a model (e.g. a neural net) instead of a table	14.8
Funnel	Multi-stage pipeline: cheap high-recall retrieval, heavy ranking, rule-based re-ranking	13.8
GAE (generalised advantage estimation)	Smoothed advantage estimate trading bias for variance with a parameter λ	14.6
GAM (generalised additive model)	Model that adds up one smooth function per feature; flexible yet readable	19.5
Gate	Sigmoid-controlled switch deciding how much information to let through	12.4
Gaussian process	Probabilistic model giving a prediction and an uncertainty for any input; a classic surrogate in Bayesian optimisation	6.9

Term	Meaning	Where
GBDT (gradient-boosted decision trees)	Gradient boosting with decision trees: shallow trees added one by one, each correcting the current errors	7.4
GDPR	EU General Data Protection Regulation: rules for processing personal data of people in the EU	19.9
GELU	Smooth ReLU-like activation $z \cdot \Phi(z)$ used in transformers	10.2
Generalisation	Performing well on new, unseen data	1.3
General-purpose AI model	EU AI Act term for broadly capable models such as LLMs, with their own documentation duties	19.9
Generative AI	Models that produce new content such as text, images, audio or code	1.1
Geohash / S2 cell	Codes naming cells of a hierarchical grid over the map, for fast location lookups	18.8
Gini coefficient	Inequality measure from 0 (perfectly even) to 1 (one item gets everything)	18.5
Gini impurity	$1 - \Sigma\, p_k^2$, the chance of mislabelling a random sample from the node's class mix	7.1
Global average pooling (GAP)	Averaging each feature map to one number before the classifier head	11.2
Global vs local explanation	Describes the model's overall behaviour / explains one prediction	19.5
GloVe / fastText	Word embeddings from global co-occurrence counts / from character n-grams (handles unseen words)	12.9
GMM (Gaussian mixture model)	Clustering as a mix of Gaussians fitted with EM, giving soft assignments	9.4
Gold questions	Items with known answers mixed into annotation work to check labeller quality	4.1
GPU (graphics processing unit)	Chip with thousands of small cores; does the matrix maths of deep learning far faster than a CPU	3.7
Gradient	Vector of partial derivatives; points in the direction of steepest increase	2.4
Gradient accumulation	Summing gradients over several small batches before one update, to mimic a bigger batch	10.8
Gradient boosting	Boosting where each new tree fits the negative gradient of the loss (residuals for MSE)	7.4
Gradient clipping	Rescaling gradients whose norm exceeds a threshold to prevent explosions	10.6
Gradient descent (GD)	Minimising a loss by repeated steps $w \leftarrow w - \eta \nabla L$	2.5
Graph-cluster randomisation	Assigning whole communities to one experiment arm to limit network interference	18.4
Graph embedding	Vectors for nodes of a graph learned so connected nodes are close (e.g. node2vec, PinSage)	13.6
Graph neural network (GNN)	Neural network over a graph of entities, used e.g. to spot fraud rings	18.2
Greedy algorithm	Takes the best-looking step now without looking ahead, as tree splitting does	7.1
Greedy decoding / sampling	Generating by always taking the most likely token / by drawing tokens from the predicted distribution	12.5
Grid search	Trying every combination of listed hyperparameter values with cross-validation	6.9
Gridworld	Toy RL environment of cells an agent moves between	14.1
Ground truth	The correct answer a prediction is judged against	11.7
GroupNorm	Normalisation over groups of channels in each example; works with tiny batches	10.7
Group-specific thresholds	Different decision thresholds per group, chosen to equalise a rate	19.4
Group split	Keeping all rows of one entity in the same split	4.7
GRPO	Group Relative Policy Optimization: compares several sampled answers per prompt against their group mean	14.10
GRU	Gated recurrent unit with update and reset gates; lighter than an LSTM	12.4
Guardrail metric	Metric that must not get worse in an experiment, even if the primary improves	16.9
Hallucination	Generated content that is false or unsupported	15.2
Hard-case mining	Collecting the examples the model gets wrong or is unsure about for labelling	18.7
Hard negative	Non-matching example that looks very like a match; sharpens what the model learns	4.1
Hard / soft voting	Combining classifiers by majority label / by averaging their probabilities	7.5
Hard- vs soft-margin SVM	No points allowed inside the margin / some violations allowed at a cost set by C	8.2
Hash function	Function mapping any input to a fixed-size number; the same input always gives the same output	16.9
HDBSCAN	Hierarchical DBSCAN: finds clusters of varying density without a single eps	9.3
Head (network)	Final task-specific layers on top of a shared network body (backbone)	11.2
Health check	Regular probe confirming a service is up and answering correctly	16.6
He initialisation	Weight variance 2/fan_in, designed for ReLU layers	10.6
Hessian	Matrix of second derivatives; describes the curvature of a loss surface	5.4
Heuristic	Simple rule of thumb that is usually good enough	1.2
Hidden layer	Layer between input and output whose values are not observed directly	10.1

Term	Meaning	Where
Hidden state	An RNN's memory vector, updated at each step	12.2
Hierarchical clustering	Clustering that builds a nested tree of merges (see agglomerative)	9.2
Hinge loss	max(0, 1 − y·f(x)); the SVM loss	8.2
HIPAA	US law protecting health information	19.9
Historical / representation / measurement bias	Bias from an unequal past / from under-sampled groups / from features measured unevenly across groups	19.1
Hit rate@k (Hit@K)	Share of cases where at least one relevant item is in the top k	13.11
HNSW	Hierarchical Navigable Small World graph index for approximate nearest-neighbour search	13.9
HOG (histogram of oriented gradients)	Hand-crafted image feature counting edge directions in small cells	8.2
Holdback	Small slice kept on the old experience for months to measure long-term effects	16.9
Homogeneity / differencing attack	Learning a secret because everyone in a group shares it / by subtracting two aggregate answers	19.6
Homoscedasticity	Errors having the same spread at every prediction level (a linear-regression assumption)	5.5
HTTP	The request–response protocol web services speak; 200 means success	15.1
Huber loss	Quadratic for small errors, linear for large; robust regression and DQN	10.3
Human oversight	People able to monitor, review and override an AI system's decisions	19.9
Hybrid recommender	Combining collaborative and content-based signals	13.4
Hybrid retrieval	Combining lexical and semantic retrieval results	17.3
Hydration (render-time)	Filling in the latest live data (stock, sold status, capacity) just before display	18.6
Hyperparameter	Setting chosen before training, e.g. learning rate, depth or C	1.6
Hyperplane	Flat boundary one dimension lower than the space (a line in 2-D, a plane in 3-D)	5.1
Hypothesis test	Checking whether an observed effect could plausibly be noise under a null hypothesis	2.11
ICE (individual conditional expectation)	One curve per row showing how its prediction changes as a feature varies	19.5
Idempotent	Safe to repeat: doing it twice has the same effect as once	18.4
Identity matrix	Square matrix with 1s on the diagonal and 0s elsewhere; multiplying by it changes nothing	2.2
i.i.d. (independent and identically distributed)	Examples drawn independently from the same distribution; the assumption behind random splits	6.6
ImageNet	Large labelled image dataset (1,000 classes) used to pretrain and benchmark vision models	11.4
Impossibility theorem (fairness)	With different base rates, calibration and equal error rates can't all hold unless the model is perfect	19.2
Impression	One showing of an item or ad on a user's screen	17.1
Impression discounting	Lowering scores of items shown many times without any action	18.4
Impurity	How mixed the classes in a tree node are (Gini, entropy) or how spread the targets are (variance)	7.1
Impurity-based importance (MDI)	Tree feature importance from total impurity decrease; biased toward high-cardinality features	7.3
Imputation	Filling missing values (mean, median, KNN, iterative)	4.3
In-batch negatives	Using the other items in a training batch as negatives for each user	13.7
Inception (GoogLeNet)	CNN with parallel branches of different filter sizes and 1×1 bottlenecks	11.4
Independence / separation / sufficiency	Fairness families: decision independent of group / errors independent of group given the truth / scores mean the same per group	19.2
Independence (statistical)	Two events are independent when knowing one tells you nothing about the other	2.7
Indexing lag	Delay between content arriving and becoming searchable	17.4
Individual / counterfactual fairness	Similar people get similar predictions / the prediction wouldn't change if only the person's group changed	19.2
Inertia (WCSS)	Sum of squared distances from points to their cluster centroid; lower is tighter	9.1
Inference	Using a trained model to make predictions	1.3
Inference path	Everything that runs to turn a request into a prediction	16.1
InfoNCE	Contrastive loss: a softmax over similarities that must pick the true partner out of the batch	18.6
Information gain	Entropy of the parent minus the weighted entropy of the children after a split	7.1
Instance-based learning	Predicting from stored similar examples rather than a fitted formula (e.g. k-NN)	1.8
Integrity layer	Rules and classifiers that demote or remove harmful content inside ranking	17.2

Term	Meaning	Where
Inter-annotator (inter-rater) agreement (IAA)	How often independent labellers agree; measured with e.g. Cohen's kappa	4.1
Interference (experiments)	Treated units affecting control units, which biases an A/B test	16.9
Interleaving	Merging two rankers' results into one list and seeing whose items get clicked	16.9
Interpretability	Synonym for explainability: how directly a person can understand how a model works	19.5
Intersectional group	Group defined by several attributes at once (e.g. older women)	19.2
Intra-list diversity	How different the items within one recommendation list are	13.11
Intrinsically interpretable model	Model readable directly, such as a linear model, small tree or GAM	19.5
Invalid traffic	Clicks and impressions from bots or click farms, removed before billing and training	18.1
Inverse propensity scoring / weighting (IPS / IPW)	Weighting each logged example by 1/P(it was shown/examined) to remove exposure, selection or position bias	17.3
Inverted dropout	Scaling surviving units by 1/(1 − p) during training so nothing changes at evaluation	10.7
Inverted index	Map from each word to the documents containing it; the core of keyword search	17.3
IoU (intersection over union)	Overlap of two boxes divided by their union; matches detections to ground truth	11.7
IQR (interquartile range)	Distance between the 25th and 75th percentiles; a spread measure that ignores extremes	2.8
Irreducible noise	Error no model can remove because the outcome is partly random	6.7
Isolation Forest	Anomaly detector: anomalies are isolated by random splits in fewer steps	9.7
Isotonic regression	Non-decreasing step-function calibration mapping; needs more data than Platt	6.5
Item2vec	word2vec's skip-gram applied to sessions or playlists to learn item embeddings	13.6
Iterative imputation (MICE)	Filling each missing column by predicting it from the others, repeated for several rounds	4.3
IVF (inverted file index)	ANN index that clusters vectors with k-means and searches only the nearest clusters (nprobe)	13.9
Jaccard similarity	$\|A \cap B\| / \|A \cup B\|$ for sets	13.3
Jupyter notebook	Interactive document mixing code cells, outputs and notes; good for exploration	3.1
k-anonymity	Every quasi-identifier combination shared by at least k records	19.6
KD-tree	Tree that splits space along coordinates to speed up nearest-neighbour search in low dimensions	8.1
Kernel PCA	PCA in an implicit non-linear feature space via the kernel trick	9.5
Kernel trick	Replacing dot products with a kernel $K(x, z) = \varphi(x)\cdot\varphi(z)$ without computing φ	8.2
Key-value (KV) store	Database that returns a stored value for a given key, very fast	18.4
k-fold cross-validation	Cross-validation with k equal folds, each used once for validation	6.6
Kill switch	Pre-built control that instantly turns a model or feature off	15.3
KL divergence	Extra surprise from predicting with q instead of p; non-negative and asymmetric	2.12
k-means	Clustering by alternately assigning points to the nearest centroid and recomputing centroids	9.1
k-means++	Initialisation picking centroids with probability ∝ squared distance to those already chosen	9.1
k-nearest neighbours (k-NN)	Predicting by majority vote or average of the k closest training points	8.1
KS test	Kolmogorov–Smirnov: maximum gap between two empirical CDFs, used for drift	16.8
KV cache	Stored keys and values of earlier tokens so a transformer doesn't recompute them when generating	12.7
L1 regularisation	Penalty $\lambda\Sigma\|w\|$ that drives some weights exactly to zero	5.7
L2 regularisation	Penalty $\lambda\Sigma w^2$ that shrinks all weights smoothly (weight decay)	5.7
Label	Target value to predict (y)	1.6
Label bias	Target that is a flawed or unevenly recorded stand-in for what you care about	19.1
Label encoding	Mapping categories to integer codes; `LabelEncoder` is meant for targets	4.4
Labelling function	Small rule or heuristic that votes a label; many are combined in weak supervision	4.1
Label shift	Change in the outcome distribution P(Y)	16.8
Lag feature	Past value of a series (yesterday, last week) used as an input	12.10
Lambda / kappa architecture	Separate batch and streaming paths / one streaming path for everything	16.4
LambdaMART	Gradient-boosted trees trained with a ranking objective that targets NDCG	17.3
Laplace mechanism	Adding Laplace noise with scale sensitivity/ε to achieve ε-differential privacy	19.6
Laplace smoothing	Adding α (usually 1) to every count so unseen words don't get zero probability	8.3
Lasso	Linear regression with an L1 penalty; performs feature selection	5.7

Term	Meaning	Where
Latency	Time from request to response; usually reported as percentiles	16.6
Latency budget	Milliseconds the whole prediction path may take, itemised by stage	16.6
Latent factors	Hidden dimensions learned by matrix factorisation that describe users' tastes and items' traits	13.5
Lawful basis	GDPR legal reason for processing personal data (consent, contract, legitimate interests ...)	19.9
LayerNorm	Normalisation across the features of each example; standard in transformers	10.7
Lazy learning	Doing no work at training time and all of it at prediction time (k-NN)	8.1
L-BFGS	Quasi-Newton optimiser using curvature estimates; scikit-learn's default logistic-regression solver	5.8
l-diversity / t-closeness	k-anonymity add-ons requiring varied sensitive values per group / a distribution close to the overall one	19.6
Leaf-wise tree growth	Splitting the leaf with the biggest gain next (LightGBM) rather than growing level by level	7.4
Leaky ReLU	ReLU variant with a small slope α for negative inputs	10.2
Learning curve	Training and validation score vs training-set size	6.8
Learning rate	Step size η of gradient descent	2.5
Learning-rate schedule	Changing the learning rate during training (step, cosine, warmup, one-cycle)	10.5
Learning to rank	Training models to order items (pointwise, pairwise, listwise losses)	17.3
LeCun initialisation	Weight variance 1/fan_in, suited to SELU and linear layers	10.6
Leetspeak / homoglyphs	Evasion by swapping letters for digits ('fr33') / for look-alike characters from other alphabets	18.3
LeNet / AlexNet / VGG	Early landmark CNNs (1998, 2012, 2014) that showed deep convolutional nets work	11.4
Lexical vs semantic retrieval	Matching exact words (BM25) / matching meaning via embeddings	17.3
Lift	Confidence(A → B) divided by support(B); above 1 means a positive association	9.8
LightGBM	Gradient-boosting library with leaf-wise histogram trees, GOSS and EFB	7.4
Light / heavy ranker	Cheap first-pass ranker over many candidates / expensive model over the survivors	17.2
Likelihood	How probable the observed data is under a given hypothesis or parameter value	2.7
LIME	Local explanation fitting a sparse linear surrogate around one prediction	19.5
Lineage	Record of which data, code and settings produced a model	16.2
Linear regression	Predicting a number as w·x + b by minimising squared error	5.2
Linkage (single / complete / Ward)	Rule for the distance between clusters: closest points / farthest points / increase in variance	9.2
Link prediction	Predicting which new edges will appear in a graph	18.4
LLM (large language model)	Very large transformer trained on huge amounts of text to predict the next token; powers chatbots and assistants	1.1
Load balancer	Component spreading incoming requests across many servers	15.1
Local differential privacy	Noise added on each user's device before data is sent	19.6
Local Outlier Factor (LOF)	Anomaly score comparing a point's local density with its neighbours'	9.7
Logistic regression (LR)	Linear score passed through a sigmoid to give a class probability	5.8
Logit	Raw pre-sigmoid or pre-softmax score; for logistic regression, the log-odds	5.8
Log loss	Negative log-likelihood of labels under predicted probabilities; binary cross-entropy	5.8
Log-odds	ln(p/(1 − p)); the quantity a logistic regression models linearly	5.8
logQ correction	Subtracting log(sampling probability) from logits so popular items aren't over-penalised as negatives	13.7
Loss function	Formula scoring how wrong a prediction is; training adjusts parameters to make it small (also called cost or objective)	1.6
LSTM	Recurrent cell with an additive cell state and forget, input and output gates	12.4
Macro / weighted average	Averaging per-class metrics equally / in proportion to class size	6.3
MAD (median absolute deviation)	Median distance from the median; an outlier-resistant spread measure	9.7
MAE	Mean absolute error; optimal constant prediction is the median	6.2
MAP	Mean average precision over queries or users	6.4
MAPE (mean absolute percentage error)	Average error as a percentage of the true value; breaks near zero	6.2
mAP (mean average precision)	Average precision averaged over classes (and IoU thresholds); the standard detection metric	11.7
Marginal recall	Extra relevant items a retrieval source finds that the other sources miss	17.1
Margin (SVM)	Gap between the decision boundary and the closest training points; SVMs maximise it	8.2

Term	Meaning	Where
Market-basket analysis	Finding items often bought together, via association rules	9.8
Markov property	The next state depends only on the current state and action, not the full history	14.2
matplotlib / seaborn	Python plotting libraries; seaborn adds statistical charts on top of matplotlib	3.1
Matrix	Rectangular grid of numbers; a dataset is a matrix with one row per example	2.2
Matrix factorisation (MF)	Approximating the user–item matrix by user and item factor vectors whose dot product predicts affinity	13.5
Matrix inverse	Matrix A^{-1} that undoes A, so $A^{-1}A = I$; in practice solve the system instead of computing it	2.2
Maximum likelihood estimation (MLE)	Choosing parameters that make the observed data most probable	5.2
Max pooling	Downsampling by keeping the largest value in each window	11.2
MCAR / MAR / MNAR	Missing completely at random / at random given other columns / not at random (depends on the missing value itself)	4.3
MDP (Markov decision process)	Formal RL setting: states, actions, transition probabilities, rewards and a discount factor	14.2
Mean / median / mode	Average value / middle value when sorted / most frequent value	2.8
Mean pooling	Averaging several vectors (e.g. frame embeddings) into one	17.4
Membership inference	Attack deciding whether a record was in the training set	19.7
Meta-model	Model trained on other models' predictions, as in stacking	7.5
Micro-batching	Grouping a few requests into one model call to use hardware efficiently	17.2
Mini-batch	Small subset of examples used for one gradient update	5.4
Minimum detectable effect (MDE)	Smallest lift an experiment is sized to detect; sample size $\propto 1/\text{MDE}^2$	16.9
Minkowski / Chebyshev / Hamming distance	General Lp distance / largest coordinate difference / count of positions that differ	8.1
Min-max scaling	Rescaling a feature to [0, 1] with (x − min)/(max − min)	4.5
Missing indicator	Extra 0/1 column marking where a value was missing, so the model can use missingness itself	4.3
Mixup / CutMix	Augmentations blending two images and their labels / pasting a patch of one image onto another	11.6
ML (machine learning)	Getting a computer to learn a task's rules from examples instead of having a person write them	1.1
ML objective	The measurable quantity the model predicts or optimises, chosen to serve the business goal	15.1
MLOps	DevOps practices (versioning, testing, CI/CD, monitoring) applied to data and models	16.10
MLP (multi-layer perceptron)	Feed-forward network of fully connected layers with non-linear activations	10.1
MMoE (multi-gate mixture of experts)	Multi-task architecture where each task head mixes shared expert sub-networks with its own gate	17.2
MMR (maximal marginal relevance)	Greedy re-ranking trading relevance against similarity to items already chosen	13.8
MNIST / CIFAR-10 / COCO	Classic benchmark datasets: handwritten digits / small colour images / everyday-object detection	11.3
MobileNet / EfficientNet	Efficient CNNs built from depthwise-separable convolutions / scaled with compound scaling	11.4
Model-based learning	Learning a compact model (e.g. a formula with weights) and throwing the training data away afterwards	1.8
Model card	Short document with a model's intended use, per-group evaluation and limits	19.8
Model extraction	Copying a model by querying its API and training a surrogate	19.7
Model-free RL	Learning values or policies from experience without knowing the transition probabilities	14.3
Model inversion	Attack reconstructing sensitive inputs or features from a model's outputs	19.7
Model registry	Store of model versions with metrics, lineage, stages and owners	16.10
Momentum	Optimizer that accumulates a velocity of past gradients	10.5
Monte Carlo (MC, RL)	Updating value estimates toward the actual return at the end of an episode	14.6
MRR (mean reciprocal rank)	Mean of 1/rank of the first relevant result	6.4
MSE	Mean squared error; optimal constant prediction is the mean	6.2
Multiclass vs multi-label	One label out of K classes / any number of labels per example	5.9
Multicollinearity	Features that are near-copies of each other, making coefficients unstable	5.5
Multi-head attention	Several attention heads in parallel subspaces, concatenated and projected	12.7
Multimodal	Using several kinds of input together (image, text, audio)	18.3
Multiple comparisons (p-hacking)	Running many tests makes some look significant by chance; correct with e.g. Bonferroni	2.11
Multi-task learning (MTL)	One shared network body with a head per task or event	17.2
Naive Bayes	Bayes' theorem with features assumed independent given the class	8.3
NaN	'Not a number': the floating-point marker for a missing or undefined value	3.3

Term	Meaning	Where
Natural / behavioural labels	Outcomes the world records anyway (delivery time) / labels from user behaviour (clicks)	16.3
NDCG	Normalised discounted cumulative gain: position-discounted graded relevance ÷ ideal	6.4
Negative downsampling	Keeping only a fraction of negatives to cut training cost; probabilities must then be corrected	4.9
Negative sampling	Training against a few sampled negatives instead of a full softmax	12.9
Nested cross-validation	Outer CV loop around the whole tuning procedure for an unbiased estimate	6.6
Nesterov momentum	Momentum that evaluates the gradient at the look-ahead position	10.5
Netflix Prize	2006–2009 competition on movie-rating prediction that popularised matrix factorisation	13.5
Neural architecture search (NAS)	Automatically searching for a good network design	11.4
Neural network	Model made of layers of simple units (neurons) whose weights are learned by gradient descent	10.1
Neuron (artificial)	Unit computing a weighted sum of inputs plus a bias, then applying an activation	10.1
NLP (natural language processing)	Field of ML working with human language	12.8
NMI (normalised mutual information)	Agreement between a clustering and known labels, from 0 (none) to 1 (perfect)	9.9
NMS (non-maximum suppression)	Keeping the most confident detection and removing boxes that overlap it above an IoU threshold	11.7
Node / branch / leaf	Point where a tree asks a question / one answer path / end point that gives the prediction	7.1
Non-parametric model	Model whose size grows with the data instead of being fixed in advance (k-NN, trees)	8.1
Non-stationarity	The environment or payoffs changing over time	14.4
Normal equation	Closed-form least-squares solution $\theta = (X^T X)^{-1} X^T y$	5.3
Normal (Gaussian) distribution	Symmetric bell-shaped distribution set by a mean μ and standard deviation σ	2.6
Normalised entropy	Log loss divided by the log loss of always predicting the average rate; lower is better	18.1
Norm (L1, L2, L∞)	Length of a vector: sum of absolute values (L1), straight-line length (L2) or largest entry (L∞)	2.1
Novelty effect	Temporary change in behaviour because something is new; fades over time	16.9
Novelty / serendipity	Recommending less-popular items / relevant items the user would not have found alone	13.11
n-step return / TD(λ)	Targets that look n steps ahead / blend all step counts with weight λ	14.6
Null hypothesis (H_0)	Default 'no effect' claim that a hypothesis test tries to reject	2.11
Numerical gradient check	Comparing a hand-coded gradient with (f(x+h) − f(x−h))/2h to catch bugs	2.4
NumPy	Python library for fast n-dimensional arrays and maths on them	3.2
NYC Local Law 144	New York City rule requiring annual bias audits of automated hiring tools	19.9
Object detection	Finding objects in an image as boxes with class labels and confidences	11.7
Object-level recall	Share of real objects caught in every view they appear in, not per box	18.7
Observation window	Waiting period before a missing event is recorded as a negative label	17.1
OCR (optical character recognition)	Reading text out of images or video frames	17.4
Odds	p/(1 − p): how much likelier an event is to happen than not	4.9
Odds ratio	e^w for a logistic-regression coefficient: factor by which the odds change per unit	5.8
Offline RL	Learning a policy only from logged data, without new interaction	14.12
Off-policy	Learning about one policy (e.g. greedy) while following another (e.g. ε-greedy)	14.7
OLS (ordinary least squares)	Linear regression fitted by minimising the sum of squared errors	5.3
One-hot encoding	One 0/1 column per category	4.4
One-vs-one	Training a binary classifier for every pair of classes and letting them vote	5.9
One-vs-rest	K binary classifiers, one per class against the rest	5.9
Online inference	Computing a prediction at request time	16.5
Online learning	Updating a model continuously from small chunks of data (`partial_fit`)	1.7
ONNX	Open format for exporting models so they run in other runtimes	16.6
On-policy	Learning about the same policy that is choosing the actions (e.g. SARSA)	14.7
OOV (out-of-vocabulary)	A word missing from the vocabulary; subword tokenisers avoid the problem	12.8
Open-weights model	Model whose trained weights are published so you can host it yourself	15.1
Operating point	The specific threshold, and its precision/recall, a deployed model actually uses	6.3
Optimizer	Algorithm that updates the parameters from the gradients (SGD, Adam, ...)	5.1
Optuna	Python library for hyperparameter tuning with Bayesian-style search and early pruning	6.9

Term	Meaning	Where
Orchestration	Scheduling and running pipeline steps in order (Airflow, Kubeflow, Dagster)	16.10
Ordered boosting	CatBoost trick computing each row's statistics only from earlier rows to avoid target leakage	7.4
Ordinal encoding	Mapping ordered categories to integers (small < medium < large)	4.4
Outlier	Value far from the rest; may be an error or the most interesting case	4.3
Outlier vs novelty detection	Finding anomalies inside the training data / flagging new points unlike a clean training set	9.7
Out-of-bag (OOB) score	Validation from the rows each bootstrap tree never saw	7.3
Out-of-core learning	Training on data too big for memory by streaming it in chunks	1.7
Out-of-fold prediction	Prediction for a row made by a model that did not train on that row's fold	7.5
Overfitting	Fitting training noise; low training error, much higher validation error	1.10
Oversampling / undersampling	Repeating rare-class rows / dropping common-class rows to rebalance training data	4.9
p95 / p99 latency	Response time that 95% / 99% of requests beat; captures the slow tail	16.6
Pacing	Control loop that spreads a campaign's spend over its planned period	18.1
Padding (same / valid)	Adding border pixels so the output keeps the input size ('same') or not adding any ('valid')	11.1
pandas	Python library for tables (DataFrames): loading, cleaning, grouping, joining	3.1
Parameter (model)	Number the model learns during training, such as a weight or bias	1.3
Partial dependence (PDP)	Average prediction as one feature is varied over a grid	19.5
Partial derivative	Derivative with respect to one input while holding the others fixed	2.4
Partial observability	The agent sees only part of the true state	14.9
PCA	Projection onto orthogonal directions of maximum variance	9.5
pCTR / pCVR	Predicted click-through rate / predicted conversion rate given a click	18.1
Pearson / Spearman correlation	Linear correlation of values / correlation of their ranks (catches any monotonic trend)	4.2
Percentile	Value below which a given share of the data falls; the 90th percentile (p90) beats 90% of values	2.8
Perceptron	Single neuron with a step activation; can't learn XOR	10.1
Permutation importance	Drop in held-out score when one feature's values are shuffled	7.3
Perplexity (t-SNE)	t-SNE setting roughly equal to how many neighbours each point considers	9.6
PGD (projected gradient descent attack)	Repeated FGSM-style steps kept within the budget; the standard strong attack	19.7
PII (personally identifiable information)	Data that identifies a person directly or in combination (name, email, ID number)	16.3
Pipeline	scikit-learn object chaining preprocessing and a model into one estimator	3.5
Platt scaling	Calibration by fitting a logistic curve on the model's scores	6.5
PMF / PDF	Probability mass function (discrete values) / probability density function (continuous values)	2.6
Point-in-time join	Joining each training example to feature values as they were at its timestamp	4.8
Pointwise / pairwise / listwise	Learning-to-rank losses on single items / on item pairs / on whole lists	17.3
Policy	Rule mapping states to actions (deterministic or stochastic)	14.3
Policy evaluation / improvement / iteration	Computing a policy's values / making it greedy on those values / repeating both until stable	14.5
Policy gradient	Gradient ascent on expected return, raising the log-probability of good actions	14.9
Pooling	Parameter-free downsampling of feature maps (max or average)	11.2
Popularity bias	Recommenders over-recommending already-popular items	13.12
Population / sample / statistic	Everyone you care about / the part you measured / a number computed from the sample	2.10
Positional encoding (PE)	Position information added to token embeddings because attention ignores order	12.7
Position bias	Items shown higher get more clicks regardless of relevance	17.3
Posterior	Updated belief after combining the prior with the data via Bayes' theorem	2.7
Post-hoc explanation	Method that explains an already-trained black-box model from outside	19.5
PPO	Proximal Policy Optimization: actor-critic that clips how far each update moves the policy	14.9
PR-AUC	Area under the precision–recall curve; the metric for rare positives	6.3
Pre-activation	A neuron's weighted sum z = w·x + b before the activation function	10.1
Precision	TP/(TP + FP): share of flagged items that are truly positive	6.3
Precision@k	Share of the top k results that are relevant	6.4

Term	Meaning	Where
Precision–recall (PR) curve	Plot of precision against recall as the threshold varies; best view for rare positives	6.3
Prediction (ŷ)	The model's output for one input	1.6
Predictive parity	Equal precision (PPV) across groups	19.2
Pre-ranking	Light scoring step between retrieval and the heavy ranker	13.8
Prevalence (of violating views)	Share of all views that land on violating content, estimated by sampling	18.3
Principal component	New axis in PCA; a linear combination of the original features	9.5
Prior	Belief about something before seeing the current data	2.7
Prioritised replay	Sampling replay transitions with large TD errors more often	14.8
Privacy budget (composition)	Total ε spent: each differentially private query adds to it	19.6
Probability distribution	Rule giving how likely each value of a random variable is	2.6
Product quantisation (PQ)	Compressing vectors into short codes for memory-efficient approximate search	13.9
Prompt / prompt injection	Text instructions sent to an LLM / an attack hiding instructions in input the model reads	15.2
Proper scoring rule	Metric (log loss, Brier) that is best when predicted probabilities equal the true ones	6.5
Protected (sensitive) attribute	Characteristic such as race, sex or age that should not drive decisions	19.1
Provenance	Verifiable record of where data or content came from and how it changed	19.7
Proxy feature	Feature that stands in for a sensitive attribute (e.g. ZIP region for group)	19.3
Proxy label	Easier-to-get stand-in for the outcome you care about, e.g. clicks for satisfaction	4.1
Proxy trap	Optimising an easy metric (clicks) that drifts away from the real goal (satisfaction)	17.1
Pruning (trees)	Stopping growth early or cutting back branches to reduce overfitting	7.1
Pseudonymisation	Replacing identifiers (e.g. hashing IDs); still personal data, not anonymous	19.6
PSI (population stability index)	Σ(cur − ref)·ln(cur/ref) over bins; > 0.25 means investigate	16.8
p-value	Probability of data at least this extreme if the null hypothesis were true	2.11
PYMK (People You May Know)	Social-network feature suggesting new connections	18.4
PyTorch	Python deep-learning framework with tensors, autograd and GPU support	3.7
Q-learning	Off-policy TD control updating Q toward r + γ·max Q(s', ·)	14.7
QPS (queries per second)	Request rate a service must handle; plan for peak, not average	15.3
Quantile	Cut point splitting sorted data at a given fraction (the 0.9 quantile = 90th percentile)	4.6
Quantisation	Storing weights/activations with fewer bits (e.g. int8) to shrink and speed up a model	16.6
Quasi-identifier	Attribute harmless alone but identifying in combination (ZIP code, birth date, sex)	19.6
Query, key, value (Q, K, V)	Attention's three projections: what a position looks for, what it offers to match, what it contributes	12.6
Query understanding	Normalising, spell-correcting, expanding and classifying a search query before retrieval	17.3
R^2	Coefficient of determination: share of target variance explained, 1 − SSE/SST	6.2
RAG (retrieval-augmented generation)	Retrieving relevant documents and giving them to an LLM so its answer is grounded in them	12.9
RAM	A computer's working memory; data must fit in it for fast processing	5.3
Random forest (RF)	Bagged decision trees with a random subset of features considered at each split	7.3
Random search	Tuning by sampling hyperparameter combinations from distributions	6.9
Random variable	Quantity whose value comes from a random process, such as a die roll	2.6
Ranking	Ordering items for a user or query; also the precise funnel stage after retrieval	13.8
RBF kernel	Gaussian similarity $\exp(-\gamma\|x - z\|^2)$; γ sets how local each point's influence is	8.2
Reach-aware re-scoring	Re-checking content with a stronger model each time its views cross a tier	18.3
Real-time bidding	Auctioning each ad impression in milliseconds as a page loads	14.11
Reason codes	Plain-language top reasons given to a customer for an adverse decision	19.5
Recall	TP/(TP + FN): share of true positives that were caught (sensitivity)	6.3
Recall@k	Share of relevant items that appear in the top k	6.4
Receptive field	Region of the input that can influence a unit	11.2
Reciprocal rank fusion (RRF)	Merging ranked lists by summing 1/(k + rank) per list (k ≈ 60); needs no training or score calibration	17.4
Recommender system	System predicting which items a user will want from a large catalogue	13.1

Term	Meaning	Where
Reconstruction error	How far data is from its compressed-then-restored version	9.5
Recourse	Telling a person what they could change to get a different decision	19.5
Red-teaming	Deliberately attacking your own system to find failures before others do	19.7
Reformulation / abandonment	User rewriting the query / leaving without clicking; both signal dissatisfaction	17.3
Regression	Supervised task predicting a number	1.5
Regression suite	Fixed test set re-run on every new version to catch quality going backwards	18.6
Regret	Total reward lost compared with always choosing the best option	14.4
Regularisation	Anything that limits a model's freedom to fit noise (penalties, dropout, early stopping)	5.7
REINFORCE	Basic policy-gradient algorithm weighting log-probability gradients by the return	14.9
Reinforcement learning	Learning a policy by trial and error to maximise long-run reward	14.1
Relevance floor	Minimum relevance a result (or ad) must clear before it can be shown	17.3
Reliability diagram	Plot of mean predicted probability vs observed frequency per bin	6.5
ReLU	max(0, z); the default hidden-layer activation	10.2
Replica	Extra copy of a service or data store, for capacity and safety	16.6
Reporter reputation	Historical precision of a user's reports, used to weight new reports	18.3
Representation (metric) learning	Learning embeddings where distance reflects similarity	18.6
Re-ranking	Final funnel stage applying diversity, freshness and business rules to the ranked list	13.8
Residual	Difference between a true value and the prediction	5.5
Residual connection	Skip path y = x + F(x) that gives gradients an identity highway	11.4
ResNet	CNN family with residual (skip) connections, trainable at 100+ layers	11.4
Response bias	Survey answers skewed by who chooses to respond or how the question is asked	17.2
Responsibility (GMM)	Probability that a point belongs to each Gaussian component	9.4
Retrieval	Candidate generation: narrowing millions of items to hundreds, graded on recall	17.1
Return (RL)	Discounted sum of future rewards $G_t = r_{t+1} + \gamma G_{t+1}$	14.2
Reward hacking	Agent maximising the reward as written rather than the intended goal	14.12
Reward model (RM)	Model trained on human preference comparisons to score responses in RLHF	14.10
Reweighing	Weighting (group, label) cells so group and label are independent in the weighted data	19.4
Rich-get-richer	Feedback loop where already-popular items or people get ever more exposure	18.4
Ridge regression	Linear regression with an L2 penalty	5.7
Right to erasure	GDPR right to have your personal data deleted ('right to be forgotten')	19.9
RLAIF / Constitutional AI	RLHF where an AI model, guided by written principles, supplies the preference labels	14.10
RLHF	Reinforcement learning from human feedback: SFT, reward model, then PPO with a KL penalty	14.10
RL with verifiable rewards	RL where rewards come from automatic checks (tests pass, maths answer correct)	14.10
RMSE	Root mean squared error, in the target's units; weighs big misses more	6.2
RMSProp	Optimizer dividing each step by a running RMS of recent gradients	10.5
RNN (recurrent neural network)	Network that reads a sequence step by step, carrying a hidden state	12.2
Robust scaling	Rescaling with the median and IQR so outliers have little influence	4.5
ROC curve	Plot of true-positive rate against false-positive rate as the threshold varies	6.3
ROI (return on investment)	Value gained per unit of money spent	18.9
Rollback	Returning quickly to the previous model, features or config after a bad release	13.7
Rolling deploy	Replacing servers with the new version a few at a time	16.6
Row subsampling	Training each boosting tree on a random fraction of rows to reduce overfitting	7.4
RPM (revenue per mille)	Revenue per thousand impressions	18.1
Rules engine	System applying explicit human-written conditions, alongside or before a model	18.2
Sampled metrics	Ranking metrics computed against a few sampled negatives instead of the full catalogue; can mislead	13.11
Sample inefficiency	Needing very many interactions to learn	14.7
Sample ratio mismatch (SRM)	Arms receiving a different share of users than planned; signals a broken experiment	16.9
SAM (Segment Anything Model)	Meta's promptable segmentation foundation model	11.7

Term	Meaning	Where
SARIMA	ARIMA with extra seasonal terms	12.10
SARSA	On-policy TD control updating Q toward r + γ·Q(s', a') for the action actually taken	14.7
Scaled dot-product attention	softmax(QKᵀ/√d_k)V	12.6
ScaNN / Annoy / hnswlib	Popular open-source ANN search libraries	13.9
Schema check	Validating that incoming data has the expected columns, types and ranges	16.8
scikit-learn	Python library of classic ML models, preprocessing and evaluation tools with a shared fit/predict API	3.5
Scree plot	Chart of explained variance per component, used to pick how many to keep	9.5
Seasonal naive	Forecast repeating the value from one season ago (e.g. same weekday last week)	12.10
Second-price / GSP / first-price auction	Winner pays the runner-up's bid / multi-slot version of that / winner pays its own bid	18.1
Secure aggregation	Protocol letting the server see only the sum of client updates, never one client's	19.6
Selection bias	Distortion caused by which examples ended up in the data	2.9
Selective labels	Outcomes seen only for people the system approved, so the rest stay unknown	19.1
Self-attention	Attention where queries, keys and values all come from the same sequence	12.7
Self-play	Training an agent by having it play against copies of itself	14.11
Self-supervised learning	Learning from labels generated from the data itself (e.g. next-token prediction)	1.4
Semantic / instance segmentation	Labelling every pixel with a class / separating each individual object's pixels	11.7
Semi-supervised learning	Learning from a few labelled examples plus many unlabelled ones	1.4
Sensitivity (DP)	Largest change one person's record can make to a query result	19.6
Sentence embedding	One vector representing a whole sentence or document	12.9
Seq2seq	Encoder–decoder model mapping an input sequence to an output sequence	12.5
Sequence data	Data whose order matters: text, audio, time series, event streams	12.1
Sequential recommendation	Predicting the next item from the order of a user's recent items	18.5
Sequential test	Experiment analysis that allows checking results early without inflating false positives	16.9
Session embeddings	Item vectors learned by skip-gram over browsing sessions, so items considered together are close	18.9
SFT (supervised fine-tuning)	Training a pretrained LLM on example prompts and good answers before RL	14.10
Shadow deployment	New model scores live traffic in parallel; outputs are logged but not used	16.6
SHAP	Shapley-value attributions splitting one prediction into additive feature contributions	19.5
Shapley values	Game-theory split of a prediction among features by average marginal contribution	19.5
Share velocity	How fast an item is being shared; a sign of fast-spreading content	18.3
Sigmoid	σ(z) = 1/(1 + e⁻ᶻ), squashing a score into (0, 1)	2.4
Significance level (α)	False-positive rate you accept in a hypothesis test, often 0.05	2.11
Silhouette score	Average over points of (b − a)/max(a, b), where a = own-cluster distance and b = nearest-other-cluster distance	9.1
SiLU (Swish)	Smooth activation z·σ(z), common in modern vision models and LLMs	10.2
Sim-to-real gap	Performance lost when a policy trained in simulation meets reality	14.12
Singular value	Stretch factor on the diagonal of Σ in the SVD; large ones carry most of the matrix's information	2.3
Skip-gram / CBOW	word2vec variants: predict context words from the centre word / the centre word from its context	12.9
SLA (service-level agreement)	Promise to a customer about a service level, often with penalties	18.2
SLO (service-level objective)	Internal target for a service, such as p99 latency under 100 ms	15.1
SMOTE	Oversampling by creating synthetic minority points between minority neighbours	4.9
Snorkel	Weak-supervision library that combines labelling functions into training labels	4.1
Softmax	e^zₖ/Σe^zⱼ: turns K scores into probabilities that sum to 1	5.9
Sparsity (user–item matrix)	Most user–item cells are empty because each user touches few items	13.1
Specificity	TN/(TN + FP): share of real negatives correctly left alone	6.3
Spectrogram	Image of how a sound's frequencies change over time	18.5
Splines	Piecewise polynomials joined smoothly, used to fit curves flexibly	5.6
Stacking	Meta-model trained on out-of-fold predictions of diverse base models	7.5
Staleness	How out of date a feature or prediction is	16.1

Term	Meaning	Where
Standard deviation (std)	Typical distance of values from their mean; the square root of the variance	2.8
Standard error	Spread of a sample statistic, σ/√n for a mean	2.10
Standardisation	Rescaling to mean 0, std 1: z = (x − μ)/σ	2.8
State / action / reward	Situation the agent is in / choice it makes / number telling it how good the result was	14.1
Stateless service	Service that keeps no per-user memory between requests, so any copy can answer	16.6
Stateless vs stateful retraining	Training from scratch each time / continuing from the previous model	16.8
State-space model	Sequence architecture with a linear recurrent state that scales to long inputs (e.g. Mamba)	12.7
State-value / action-value function	Expected return from a state V(s) / from taking an action in a state Q(s, a)	14.3
Stationarity	Statistical properties (mean, variance) not changing over time	12.10
Statistical power	Chance a test detects an effect that really exists, 1 − β	2.11
Step-up authentication	Asking for an extra identity check (one-time code, 3-D Secure) when risk is uncertain	18.2
Sticky assignment	Keeping each user in the same experiment arm or model version across requests	16.6
Stochastic gradient descent (SGD)	Gradient descent using one example or a mini-batch per step	2.5
Stop words / stemming / lemmatisation	Dropping very common words / cutting words to a crude stem / mapping words to their dictionary form	12.8
Stratified split	Split keeping class proportions equal in every part	4.7
Streaming inference	Scoring events as they flow through a stream	16.5
Stride	How many pixels a kernel moves between positions	11.1
Subword tokenisation	Splitting rare words into frequent pieces so any text can be encoded with a fixed vocabulary	12.8
Successive halving / Hyperband	Tuning that gives many candidates a small budget and keeps only the best for the next, bigger round	6.9
Supervised learning	Learning from inputs paired with correct labels	1.4
Support (association rules)	Fraction of baskets containing an itemset	9.8
Support vector	Training point on or inside the margin that defines an SVM's boundary	8.2
Surrogate model	Cheap model that approximates an expensive one (e.g. score vs hyperparameters in Bayesian optimisation)	6.9
SVD (singular value decomposition)	Factorising any matrix as $U\Sigma V^T$ (rotate, stretch, rotate); keeping the top singular values gives the best low-rank approximation	2.3
SVM (support vector machine)	Classifier finding the maximum-margin hyperplane, optionally with kernels	8.2
SVR (support vector regression)	SVM for numbers: fits a tube of width ε and ignores errors inside it	8.2
Switchback test	Experiment alternating treatment and control over time slots, for marketplaces with interference	16.9
Tabular data	Data in rows and columns, like a spreadsheet or database table	7.1
Tanh	Activation squashing values into (−1, 1); zero-centred cousin of the sigmoid	10.2
Target encoding	Replacing a category with a smoothed, out-of-fold mean of the target	4.4
Target leakage	A feature that secretly contains the answer, e.g. a field filled in only after the outcome	1.10
Target network	Frozen copy of the Q-network used to compute stable TD targets in DQN	14.8
TD error (δ)	r + γV(s') − V(s): the surprise between predicted and observed value	14.6
Teacher forcing	Feeding the true previous token to the decoder during training	12.5
Temperature (softmax)	Divisor applied to scores before a softmax; low values make the distribution sharper	13.7
Temporal difference (TD) learning	Updating a value toward reward + γ·(estimate of next value) after every step	14.6
Tensor	n-dimensional array, the basic data type of deep-learning frameworks	11.1
TensorFlow / Keras	Google's deep-learning framework and its high-level model-building API	3.7
Test set	Data used once at the end to estimate real-world performance	1.6
Test statistic	Number computed from the data to judge a hypothesis (e.g. a t- or z-value)	2.11
Test-time augmentation (TTA)	Averaging predictions over several augmented copies of a test input	11.6
TF-IDF	Term frequency × inverse document frequency: down-weights words common across documents	4.6
Thompson sampling	Sampling each arm's payoff from its posterior and pulling the best sample	14.4
Threat model	Statement of what an attacker wants, knows and can do	19.7
Threshold (decision)	Score cut-off turning probabilities into decisions; sets the precision–recall trade-off	6.3
Threshold moving	Changing the score cut-off instead of the model to trade precision for recall	4.9
Throughput	Amount of work completed per unit of time (e.g. predictions per second)	16.1

Term	Meaning	Where
Tiling	Cutting a large image into overlapping pieces so a detector sees each at normal size	18.7
Time-based split	Training on the past and testing on the future	4.7
Timeout	Maximum wait before a call is abandoned and a fallback is used	16.2
Time series	Measurements ordered in time, such as daily sales	12.10
Time step	One position in a sequence	12.1
Token	Unit of text a model reads (a word, subword or character), mapped to an id	12.1
Tokenization	Splitting text into tokens mapped to integer ids from a vocabulary	12.8
Tombstone	Marker saying an index entry is deleted, used instead of immediate removal	18.6
TPE (tree-structured Parzen estimator)	Bayesian-optimisation method used by Optuna	6.9
TP / FP / FN / TN	Counts of true positives, false positives, false negatives and true negatives in a confusion matrix	6.3
TPR / FPR / FNR / PPV	True-positive rate (recall) / false-positive rate / false-negative rate / positive predictive value (precision)	6.3
Training-data extraction	Getting a generative model to output memorised training text or images	19.7
Training pipeline	Automated steps that turn raw data into a validated, registered model	16.1
Training–serving skew	Features computed differently in production than in training	16.2
Training set	The examples a model learns its parameters from	1.6
Transfer learning	Reusing a model pretrained on a large task as the starting point for yours	11.5
Transformer	Architecture of stacked self-attention and feed-forward blocks with residuals and LayerNorm	12.7
Transition probability	P(s' | s, a): chance of landing in s' after action a in state s	14.2
Translation equivariance / invariance	Shifting the input shifts the output the same way / leaves the output unchanged	11.1
Transpose	Flipping a matrix so rows become columns (A^T)	2.2
Trend / seasonality	Long-run rise or fall / repeating pattern over a fixed period	12.10
Triangle closing	Suggesting C to A when A–B and B–C are already connected	18.4
Truncated BPTT	Backpropagating through only the last k steps to save memory on long sequences	12.2
Trustworthiness	Score of how well a low-dimensional map keeps true neighbours together	9.6
t-SNE	Non-linear 2-D visualisation preserving local neighbourhoods	9.6
TTL (time to live)	How long a cached entry stays valid before it expires	17.4
Two-sided marketplace	Platform serving two groups (guests and hosts, users and organisers) whose interests both count	18.8
Two-stage vs one-stage detector	Propose regions then classify them (Faster R-CNN) / predict boxes in a single pass (YOLO)	11.7
Two-tower model	Separate user and item encoders whose embeddings meet at a dot product	13.7
Type I / Type II error	False positive (rejecting a true H_0) / false negative (missing a real effect)	2.11
UCB (upper confidence bound)	Bandit strategy adding an optimism bonus that shrinks as an arm is tried	14.4
UMAP	Non-linear dimensionality reduction for visualisation that can transform new points	9.6
Underfitting	Model too simple to capture the pattern; training and validation error both high	1.10
U-Net	Encoder–decoder segmentation network with concatenated skip connections	11.7
Universal approximation theorem	One hidden layer with enough units can approximate any continuous function on a bounded region	10.1
Unsupervised learning	Finding structure in data without labels	1.4
Upsampling	Increasing a feature map's spatial size, as in a segmentation decoder	11.7
User-based / item-based CF	Recommending from similar users' choices / from items similar to ones the user liked	13.4
Validation curve	Training and validation score vs one hyperparameter	6.8
Validation set	Data used to compare models and tune hyperparameters	1.6
Value formula	Product-owned weighted sum of predicted probabilities used as the final ranking score	17.2
Value function	See state-value / action-value function	14.3
Value iteration	Repeating the Bellman optimality update until values converge	14.5
Vanishing gradient	Gradients shrinking exponentially through many layers or time steps	10.6
Variance (model)	How much the model changes when trained on a different sample	6.7
Variance (statistical)	Average squared distance from the mean; how spread out values are	2.6
Vector	Ordered list of numbers, e.g. one example's features	2.1

Term	Meaning	Where
Vector database	Database that stores embeddings and answers nearest-neighbour queries	12.9
Vectorisation	Replacing Python loops with whole-array operations	3.2
Vector search (similarity search)	Finding the stored vectors closest to a query vector	8.1
Velocity feature	Count of recent activity in a short window (transactions on this card in 10 minutes)	18.2
Version skew	A new user tower querying an index built by an old item tower; silently degrades quality	13.7
View vs copy (arrays)	A view shares memory with the original array (edits show through); a copy is independent	3.2
VIF (variance inflation factor)	How much a coefficient's variance is inflated by correlation with other features; a multicollinearity check	5.10
Vision Transformer (ViT)	Transformer applied to image patches treated as tokens	11.4
Vocabulary	Fixed list of tokens a model knows, each with an id	12.8
Vocabulary mismatch	Query and document using different words for the same thing	17.3
Watermarking	Embedding a hidden, detectable mark in generated content or in a model	19.7
Weak learner	Model only slightly better than chance, e.g. a stump, used inside boosting	7.4
Weak supervision	Many noisy labelling functions combined by a label model into training labels	4.1
Weight decay	Shrinking weights each step, equivalent to an L2 penalty for SGD	10.7
Weights and bias (intercept)	Learned multipliers on each feature, plus a constant offset added to the sum	5.2
Weight sharing	Reusing the same weights at every position (convolutions, RNN steps)	11.1
White-box / black-box attack	Attacker knows the weights and code / can only send queries and read answers	19.7
Wide & Deep / DCN / DLRM	Ranking networks combining memorised feature crosses with deep layers (Google, Google, Meta)	13.8
Winsorizing (clipping)	Capping extreme values at chosen percentiles instead of deleting them	4.3
Word2vec	Shallow model learning word embeddings from context (skip-gram or CBOW)	12.9
Word embedding	Learned vector for a word, placing words used in similar contexts near each other	12.9
WordPiece / SentencePiece	Subword tokenisers used by BERT / by many multilingual models	12.8
Xavier initialisation	Weight variance 2/(fan_in + fan_out), designed for tanh and sigmoid	10.6
XGBoost	Gradient-boosting library with a regularised second-order objective	7.4
XOR problem	Output 1 when exactly one input is 1; not linearly separable, so one perceptron can't learn it	10.1
YOLO	One-stage detector predicting boxes and classes over a grid in a single pass	11.7
Zero-shot	Handling a task or class with no task-specific training examples	11.8
Zipf distribution	A few items are extremely common and most are rare (a long tail)	17.3
z-score	(x − mean)/std: how many standard deviations from the mean	2.8

Chapter TL;DR

- **20.1–20.4**: the workflow (frame → data → prepare → train → evaluate → deploy → monitor), the math you need (dot products, gradients, Bayes, entropy), the NumPy/pandas/scikit-learn API and data preparation — split first, pipeline everything, fit on training data only.
- **20.5–20.8**: every algorithm with its knobs and costs, every loss, every metric with a worked number, and how to validate and tune honestly (CV, nested CV, calibration, early stopping).
- **20.9–20.12**: deep learning (activations, optimizers, init, regularisation, the debugging checklist), CNN/RNN/attention formulas, recommenders (MF, two-tower, funnel, ANN) and the RL update rules.
- **20.13–20.15**: the 7-step system-design framework with napkin numbers and the thirteen case studies, then the production/MLOps and responsible-ML checklists to run before every launch.
- **20.16–20.17**: decision tables for picking an algorithm and a metric, and 50 interview questions with crisp answers.
- **20.18**: the glossary — follow each **§** back to the chapter where the idea is explained properly.

Chapter 21 — Interview Question Bank

This chapter collects the machine-learning questions that come up again and again in ML-engineer and data-scientist interview loops, as gathered in the big public question lists on GitHub (listed at the end). Each question is worded the way interviewers usually say it, and each has a short answer in this book's own words, so you can revise the whole subject in one place and know you have not missed a classic.

How to use this bank

- **Wording.** Questions are phrased as they are typically asked ("What's the difference between bagging and boosting?"), so you recognise them on the day.
- **★ = asked very often.** A star marks the roughly 90 questions that appear across several of the sources listed at the end and are the classics of almost every ML loop. If you only have one evening, do the starred ones.
- **Answers you can say out loud.** Each answer is 2–6 sentences (or a short code snippet plus a sentence or two). That is about 30–60 seconds of speech, which is the right length for a first answer; the interviewer will then ask a follow-up.
- **→ More.** Where this book explains the topic in depth, a link points to the chapter and section (for example *§6.7* means section 6.7). Questions without a link are answered fully here.
- **Terms are defined at first use.** If a word is new, the answer explains it; the glossary in Ch 20 has the rest.
- **How to drill.** Cover the answer, say yours out loud, then check. Mark the ones you stumbled on and repeat only those the next day. For the coding tasks at the end, type the solution from a blank file in under 15 minutes.

Versions assumed: Python 3.14, scikit-learn 1.9, PyTorch 2.14, NumPy 2.5 (the same as the rest of the book).

---

1. ML basics and the workflow

Q1. ★ What is machine learning, and how is it different from traditional programming? In traditional programming a person writes the rules and the computer applies them to data. In machine learning (ML) you give the computer examples (data, often with the correct answers) and an algorithm learns the rules, stored as the numbers inside a *model*. You choose ML when the rules are too many, too fuzzy or too fast-changing to write by hand (spam, image recognition, recommendations) and when you have enough representative data. → *More:* Ch 1 §1.2 — Learning from data vs writing rules

Q2. ★ What's the difference between AI, machine learning and deep learning? Artificial intelligence (AI) is the broad goal of making machines behave intelligently, by any method, including hand-written rules and search. Machine learning is the subset of AI where behaviour is learned from data. Deep learning is the subset of ML that uses neural networks with many layers, which learn their own features from raw inputs such as pixels or text. → *More:* Ch 1 §1.1 — The AI family tree

Q3. ★ Explain supervised, unsupervised, semi-supervised, self-supervised and reinforcement learning. *Supervised*: every training example has a label (the correct answer) and the model learns input → label. *Unsupervised*: no labels; the model finds structure such as clusters or a compact representation. *Semi-supervised*: a few labels plus many unlabelled examples. *Self-supervised*: labels are made from the data itself, such as predicting the next word or a masked patch; this is how large language models are pretrained. *Reinforcement learning*: an agent acts in an environment and learns from rewards rather than labels. → *More:* Ch 1 §1.4 — Types of learning by supervision

Q4. ★ What's the difference between classification and regression? Give an example of each. Classification predicts a category (spam or not spam, which of 10 digits); regression predicts a number (house price, delivery time). Many models do both: a decision tree outputs a class in one case and a leaf average in the other. Logistic regression, despite its name, is a classification model because it outputs a class probability. → *More:* Ch 1 §1.5 — Classification vs regression

Q5. ★ What is the difference between a parameter and a hyperparameter? Parameters are learned from the training data: the weights of a linear model, the split thresholds of a tree. Hyperparameters are chosen before training and control how learning happens: learning rate, tree depth, number of neighbours k, regularisation strength. You tune hyperparameters by comparing scores on validation data, never on the test set. → *More:* Ch 1 §1.6 — Key terms

Q6. Walk me through the end-to-end machine-learning workflow. Frame the problem (what decision does the prediction drive, and which metric shows success); collect and label data; explore it (EDA, exploratory data analysis); clean it and build features; split into train/validation/test; train a simple baseline, then better models; tune on validation; evaluate once on test; deploy; monitor and retrain. Most real time goes into data and evaluation, not model choice. → *More:* Ch 1 §1.9 — The end-to-end ML workflow

Q7. What's the difference between batch (offline) learning and online learning? Batch learning trains on the full dataset at once and is retrained periodically (for example nightly). Online learning updates the model incrementally as each new example or mini-batch arrives, so it adapts quickly to change and handles data that does not fit in memory. Online learning needs guards against bad data quickly corrupting the model. → *More:* Ch 1 §1.7 — Batch vs online learning

Q8. What's the difference between instance-based and model-based learning? Instance-based methods memorise training examples and compare new inputs to them at prediction time (k-nearest neighbours). Model-based methods compress the data into a set of parameters during training and then discard the examples (linear regression, neural networks). Instance-based learning trains instantly but predicts slowly and needs the data stored. → *More:* Ch 1 §1.8 — Instance-based vs model-based learning

Q9. What's the difference between a generative and a discriminative model? A discriminative model learns P(y | x) directly, the boundary between classes (logistic regression, SVMs, most neural classifiers). A generative model learns how the data is produced, P(x | y) and P(y), or P(x) itself, and can create new samples (naive Bayes, Gaussian mixtures, GANs, diffusion models, language models). Discriminative models usually classify better with plenty of data; generative ones cope better with little data and can handle missing inputs. → *More:* Ch 8 §8.3 — Naive Bayes

Q10. ★ Why do we split data into training, validation and test sets? The training set fits the parameters. The validation set is used to choose hyperparameters and compare models. The test set is touched once at the end so its score is an unbiased estimate of performance on new data; if you tune on it, it stops being a fair exam. Typical splits are 70/15/15 or 80/10/10, or cross-validation instead of a fixed validation set when data is small. → *More:* Ch 4 §4.7 — Train, validation and test splits

Q11. What are the main challenges (failure modes) in machine learning? Not enough data, unrepresentative data (sampling bias), poor-quality data and labels, irrelevant features, overfitting, underfitting, and data leakage. In production add training/serving skew and drift (the

world changing after training). Most of these are data problems, which is why "get better data" usually beats "try a fancier model". → *More:* Ch 1 §1.10 — The main challenges of machine learning

Q12. How do you decide whether a problem needs ML at all? Ask whether simple rules or a lookup already solve it, whether there is a clear pattern to learn, whether enough labelled data exists or can be created, whether mistakes are tolerable and whether the prediction changes a decision. Always build a non-ML baseline (a rule, the majority class, last week's value); if ML cannot beat it by a margin that matters to the business, do not ship ML. → *More:* Ch 15 §15.4 — Scoping a vague AI question

---

2. Bias–variance, overfitting and regularisation

Q13. ⋆ Explain the bias–variance trade-off. Bias is error from wrong assumptions: a model too simple to capture the pattern (a straight line through a curve). Variance is error from sensitivity to the particular training sample: a model so flexible that it fits noise and changes a lot if the data changes. For squared error, expected test error = bias² + variance + irreducible noise. Making a model more complex lowers bias and raises variance, so test error is U-shaped and you pick the complexity at the bottom. → *More:* Ch 6 §6.7 — The bias–variance trade-off

Q14. ⋆ What is overfitting and underfitting? How do you detect them? Overfitting: the model learns noise in the training data, so it scores much better on training than on validation data. Underfitting: the model is too simple and scores poorly on both. Detect them by comparing training and validation scores, and with learning curves (score vs training-set size) or validation curves (score vs a complexity hyperparameter). → *More:* Ch 6 §6.8 — Learning curves and validation curves

Q15. ⋆ How do you prevent overfitting? Get more (or augmented) data; use a simpler model or fewer features; add regularisation (L1/L2 penalties, dropout, weight decay); stop training early when validation loss stops improving; use ensembles such as bagging; and validate with cross-validation so you do not overfit the validation set itself. Also check for leakage: a suspiciously good score is often leakage rather than skill. → *More:* Ch 1 §1.10, Ch 10 §10.7 — Regularization

Q16. How do you fix underfitting? Use a more flexible model (deeper trees, more layers, a kernel), add better features or feature interactions, reduce regularisation, and train longer. If the training score itself is low, more data will not help much; the model needs more capacity or better inputs. → *More:* Ch 6 §6.7

Q17. What is regularisation and why does it work? Regularisation is anything that discourages a model from becoming too complex, usually a penalty added to the loss such as $\lambda \cdot \Sigma w^2$ (L2) or $\lambda \cdot \Sigma |w|$ (L1). The penalty pulls weights toward zero, so the model cannot use huge weights to chase noise. This adds a little bias but removes a lot of variance. The strength λ is a hyperparameter chosen on validation data. → *More:* Ch 5 §5.7 — Regularization: Ridge, Lasso, ElasticNet

Q18. ⋆ What's the difference between L1 and L2 regularisation? Why does L1 give sparse weights? L2 (Ridge) adds the sum of squared weights; it shrinks all weights smoothly and spreads weight among correlated features. L1 (Lasso) adds the sum of absolute weights; it drives some weights exactly to zero, which is built-in feature selection. Geometrically, the L1 constraint region is a diamond whose corners lie on the axes, so the optimum often lands on a corner where some weights are zero; the L2 region is a circle with no corners. From the gradient view, L1's pull toward zero is constant even for tiny weights, while L2's pull shrinks as the weight shrinks. → *More:* Ch 5 §5.7

Q19. What is ElasticNet and when would you use it? ElasticNet combines L1 and L2 penalties with a mixing ratio. Use it when you want sparsity (like Lasso) but have groups of correlated features: plain Lasso tends to pick one of them arbitrarily, while the L2 part lets ElasticNet keep the group together and makes the solution more stable. → *More:* Ch 5 §5.7

Q20. Can you use L2 regularisation for feature selection? Not directly: L2 shrinks weights toward zero but almost never makes them exactly zero, so every feature stays in. You can rank by weight size after scaling, but that is a heuristic. For real selection use L1, tree importances, permutation importance or recursive feature elimination. → *More:* Ch 5 §5.7

Q21. What regularisation techniques exist for neural networks? Weight decay (L2 on the weights), dropout, early stopping, data augmentation, batch or layer normalisation (mild side effect), label smoothing (training on softened targets like 0.9/0.1 instead of 1/0), smaller architectures, and transfer learning from a pretrained model. In practice augmentation and early stopping are the cheapest big wins. → *More:* Ch 10 §10.7 — Regularization: fighting overfitting

Q22. What is early stopping? You track the validation loss during training and stop when it has not improved for a set number of epochs (the *patience*), then restore the best checkpoint. Training longer makes a flexible model fit noise, so the stopping epoch acts like a regularisation hyperparameter that costs nothing to tune. → *More:* Ch 6 §6.10 — Early stopping

Q23. Explain overfitting to a non-technical person. It is like a student who memorises last year's exam answers instead of learning the subject: perfect on questions they have seen, lost on new ones. The model has learned the quirks of its training examples rather than the general pattern, so it looks great in testing on familiar data and disappoints on real, new cases.

Q24. ⋆ What is the curse of dimensionality? As the number of features grows, the space grows exponentially, so data becomes sparse: you need vastly more examples to cover it, and distances between points become nearly equal, which breaks methods that rely on "nearby" points (k-NN, k-means). Models also find more spurious patterns. Remedies: feature selection, dimensionality reduction (PCA, embeddings), regularisation and models that do not depend on raw distances. → *More:* Ch 8 §8.1 — k-NN, Ch 9 §9.5 — PCA

Q25. What is double descent? In very over-parameterised models (huge neural nets), test error can fall, rise near the point where the model can exactly fit the training data, then fall again as the model gets even bigger. It does not contradict bias–variance; it shows that with enough parameters plus implicit regularisation from gradient descent, the model picks smooth solutions among the many that fit. It explains why giant networks generalise despite having more parameters than examples.

---

3. Data, feature engineering, leakage and imbalance

Q26. ⋆ How do you handle missing values? First ask why they are missing: completely at random (MCAR), explainable by other columns (MAR), or related to the missing value itself (MNAR, such as rich people skipping the income question). Then drop rows only if few and random; otherwise impute (median for numbers, a "missing" category for categories), optionally with model-based imputation (KNN, iterative), and add a "was missing" indicator, because missingness itself is often predictive. Fit the imputer on training data only, inside a pipeline. Gradient-boosted trees can handle NaN natively. → *More:* Ch 4 §4.3 — Data cleaning

Q27. How do you detect and handle outliers? Detect with plots (box plots, scatter plots), rules such as values beyond 1.5 × IQR (interquartile range) or $|z| > 3$, or anomaly detectors such as Isolation Forest. Then decide: an outlier may be a data error (fix or drop),

a genuine rare case (keep, maybe cap or log-transform), or the very thing you want to find (fraud). Robust choices include median instead of mean, MAE or Huber loss instead of MSE, and tree models. → *More:* Ch 4 §4.3, Ch 9 §9.7 — Anomaly detection

Q28. ⋆ How do you encode categorical variables? One-hot vs label vs target encoding? *One-hot* makes one 0/1 column per category; best for a small number of nominal (unordered) categories. *Label/ordinal* encoding maps categories to integers; fine for ordered categories or for trees, but misleading for linear models because it invents an order. *Target encoding* replaces each category with the average target for that category; great for high-cardinality features (many distinct values, like ZIP codes), but must be computed out-of-fold and smoothed to avoid leakage. *Hashing* maps categories into a fixed number of buckets for unbounded vocabularies; deep models use learned embeddings. → *More:* Ch 4 §4.4 — Encoding categorical variables

Q29. ⋆ What is feature scaling? Normalisation vs standardisation? Which models need it? Standardisation rescales a feature to mean 0 and standard deviation 1; min-max normalisation rescales it into [0, 1]. Models that use distances, gradients or weight penalties need scaling: k-NN, k-means, SVMs, PCA, regularised linear/logistic regression and neural networks. Tree-based models do not, because splits depend only on the order of values. Fit the scaler on training data and apply the same transform to validation, test and production data. → *More:* Ch 4 §4.5 — Feature scaling

Q30. What is feature engineering? Give examples. Creating inputs that make the pattern easier for the model to learn. Examples: ratios (debt / income), date parts (hour of day, day of week, is_holiday), log of skewed amounts, counts and aggregates over time windows ("number of transactions in the last hour"), text features (TF-IDF), interaction features (price × quantity) and distances (to the nearest store). Good features usually come from domain knowledge and often matter more than the model choice. → *More:* Ch 4 §4.6 — Feature creation

Q31. ⋆ What is feature selection and which methods do you know? Choosing a subset of features to reduce overfitting, cost and latency, and to improve interpretability. *Filter* methods score each feature independently (correlation, mutual information, chi-squared). *Wrapper* methods search subsets using a model (forward selection, recursive feature elimination). *Embedded* methods select during training (L1 penalty, tree importances). Do selection inside cross-validation, or the selection step leaks information from the validation folds. → *More:* Ch 4 §4.6, Ch 5 §5.7

Q32. ⋆ What is data leakage? Give examples and say how to prevent it. Leakage is when information that would not be available at prediction time gets into training or evaluation, so offline scores look great and production results disappoint. Examples: a feature recorded after the outcome (a "refund issued" flag for predicting fraud); scaling or feature selection fitted on the full dataset before splitting; random splits of time-series data so the model trains on the future; the same customer or duplicate rows in both train and test; target encoding computed with the row's own label. Prevent it by splitting first, putting all preprocessing inside a pipeline, using time-based or group-based splits, and building features "as of" the prediction time. → *More:* Ch 4 §4.8 — Data leakage

Q33. Your model has 99% validation accuracy on the first try. What do you check? Suspect leakage or a broken metric before celebrating. Check the class balance (99% may be the majority-class rate), look at the top features for anything that encodes the label or is recorded after it, check for duplicate or same-entity rows across splits, check that the split respects time, and verify the evaluation code by scoring a trivial baseline. Then test on a truly fresh, later slice of data. → *More:* Ch 4 §4.8

Q34. ⋆ How do you handle an imbalanced dataset? First, use the right metric: precision, recall, F1, PR-AUC, not accuracy. Then try, roughly in order: tune the decision threshold; use class weights in the loss; resample the training set (undersample the majority, oversample the minority, or SMOTE, which creates synthetic minority examples by interpolating between neighbours); collect more minority examples; or frame it as anomaly detection when positives are extremely rare. Resample only the training data, and if you change the class ratio, recalibrate the probabilities afterwards. → *More:* Ch 4 §4.9 — Class imbalance

Q35. Why is accuracy a bad metric for imbalanced data? If 99% of transactions are legitimate, a model that always says "legitimate" scores 99% accuracy while catching zero fraud. Accuracy is dominated by the majority class. Use the confusion matrix, precision and recall on the rare class, PR-AUC, or a cost-weighted metric. → *More:* Ch 6 §6.3 — Classification metrics

Q36. Does SMOTE always help? What are its pitfalls? Not always. SMOTE can create unrealistic points in noisy or overlapping regions, does not work naturally for categorical features, and inflates the minority rate so predicted probabilities become too high (they must be recalibrated). It must be applied only inside the training folds; applying it before splitting leaks synthetic copies of validation points into training. With strong models like gradient boosting, class weights plus threshold tuning often do as well. → *More:* Ch 4 §4.9

Q37. How do you split time-series data, and why not randomly? Split by time: train on the past, validate and test on later periods, ideally with a gap between them so features built from recent windows do not overlap. A random split lets the model see the future (and near-duplicate neighbouring days), so validation scores are optimistic. For cross-validation use expanding or rolling windows (scikit-learn's `TimeSeriesSplit`). → *More:* Ch 4 §4.7, Ch 6 §6.6

Q38. What would you do with a high-cardinality categorical feature such as user ID or ZIP code? Options: group rare values into "other"; frequency encoding (replace with how often the value appears); out-of-fold target encoding with smoothing; feature hashing; or a learned embedding in a neural network. Raw IDs often just memorise the training set, so check that the feature still helps on users or items not seen in training. → *More:* Ch 4 §4.4, Ch 13 §13.6 — Embeddings

Q39. Your target (for example price) is heavily right-skewed. What do you do? Prices, incomes and counts are usually right-skewed (a long tail of large values), not normally distributed. Train on log(1 + y) so the model is not dominated by a few huge values and errors become relative rather than absolute, then transform predictions back with exp(·) − 1 (noting this predicts something closer to the median than the mean). Alternatively use a loss suited to the shape, such as Poisson or Tweedie for counts and amounts. → *More:* Ch 4 §4.5

Q40. Why use a `Pipeline` in scikit-learn? A `Pipeline` chains preprocessing steps and the model into one estimator, so `fit` learns scalers, imputers and encoders on training data only and `predict` applies the identical steps to new data. This prevents leakage during cross-validation, keeps training and serving consistent, and lets you tune preprocessing and model hyperparameters together in one grid search. `ColumnTransformer` applies different steps to numeric and categorical columns. → *More:* Ch 4 §4.10 — `Pipeline` + `ColumnTransformer`

Q41. Where do labels come from, and what can go wrong with them? From real outcomes (a chargeback months later), from user behaviour (clicks, which are biased by what was shown and where), from human annotators (costly, inconsistent, so measure agreement), or from weak supervision and LLM labelling (cheap but with correlated errors). Watch for label delay, noisy labels, labels that only exist for items the old system chose to show (selection bias), and label definitions that drift over time. → *More:* Ch 4 §4.1, Ch 16 §16.3

Q42. What is data augmentation? Give examples. Creating extra training examples by applying label-preserving changes to existing ones. Images: flips, crops, rotations, colour jitter, mixup/cutmix. Text: synonym swaps, back-translation. Audio: noise, time shifts, pitch changes. Tabular: rarely useful beyond SMOTE-style methods. It teaches invariances (a cat is still a cat when flipped) and reduces overfitting; it must only produce changes that really keep the label. → *More:* Ch 11 §11.6 — Data augmentation for images

---

4. Evaluation, metrics, cross-validation and tuning

Q43. ⋆ What is a confusion matrix? A table of predicted class vs actual class. For binary problems it has four cells: true positives (TP, predicted positive and actually positive), false positives (FP, predicted positive but actually negative), false negatives (FN, missed positives) and true negatives (TN). Every classification metric (accuracy, precision, recall, F1, FPR) is a ratio of these four counts. → *More:* Ch 6 §6.3 — Classification metrics

Q44. ⋆ Define precision, recall and F1. When do you prefer precision over recall? Precision = TP / (TP + FP): of the items flagged, how many were right. Recall (sensitivity, true positive rate) = TP / (TP + FN): of the real positives, how many we caught. F1 is their harmonic mean, 2PR / (P + R), high only when both are high. Prefer precision when false alarms are costly (blocking a good customer's card, spam filter hiding real email); prefer recall when misses are costly (cancer screening, fraud you must catch, harmful content). → *More:* Ch 6 §6.3

Q45. ⋆ What is the ROC curve and what does AUC mean? The ROC (receiver operating characteristic) curve plots true positive rate against false positive rate (FP / (FP + TN)) as you sweep the decision threshold from high to low. AUC is the area under it: the probability that a randomly chosen positive is scored higher than a randomly chosen negative. 0.5 is random guessing, 1.0 is perfect ranking. It measures ranking quality across all thresholds and ignores calibration. → *More:* Ch 6 §6.3

Q46. ⋆ ROC-AUC vs PR-AUC: when is PR-AUC better? When positives are rare. FPR divides by the huge number of negatives, so even thousands of false alarms barely move the ROC curve and AUC looks great. Precision divides by the number flagged, so false alarms show up clearly. The baseline for PR-AUC (average precision) is the positive rate (say 0.001), which also makes improvements easier to read. Use ROC-AUC when classes are reasonably balanced or you care about both classes equally. → *More:* Ch 6 §6.3, Ch 16 §16.7

Q47. What are sensitivity, specificity, Type I and Type II errors? Sensitivity = recall = TP / (TP + FN). Specificity = true negative rate = TN / (TN + FP). A Type I error is a false positive (rejecting a true null hypothesis, a false alarm); a Type II error is a false negative (missing a real effect). Lowering the threshold trades Type II errors for Type I errors. → *More:* Ch 2 §2.11, Ch 6 §6.3

Q48. How do you choose the classification threshold? The default 0.5 is rarely right. Choose based on costs and constraints: with calibrated probabilities, flag when p > C_FP / (C_FP + C_FN), where C_FP and C_FN are the costs of a false positive and a false negative. Or pick the threshold that meets a business constraint ("recall ≥ 90%", "at most 500 manual reviews a day") on validation data. Many systems use two thresholds: auto-block above one, send to human review between them. → *More:* Ch 6 §6.3, Ch 6 §6.11

Q49. ⋆ What regression metrics do you know? MSE vs RMSE vs MAE vs R^2? MSE (mean squared error) averages squared errors and punishes large errors heavily; RMSE is its square root, back in the target's units. MAE (mean absolute error) averages absolute errors; it is robust to outliers and its optimal prediction is the median rather than the mean. R^2 is the fraction of variance explained compared with always predicting the mean (1 is perfect, 0 is no better than the mean, negative is worse). MAPE gives percentage error but breaks near zero targets. → *More:* Ch 6 §6.2 — Regression metrics

Q50. What is log loss (cross-entropy) and why use it to evaluate classifiers? Log loss is the average of −log(probability the model gave to the true class). It rewards confident correct predictions and punishes confident wrong ones very heavily (predicting 0.01 for something that happens costs −ln 0.01 ≈ 4.6). Unlike accuracy or AUC it evaluates the probabilities themselves, so it is the right metric when scores feed later calculations, such as bids in an ad auction. → *More:* Ch 2 §2.12, Ch 6 §6.3

Q51. What is model calibration and how do you fix a badly calibrated model? A model is calibrated when its probabilities match reality: of all cases scored 0.7, about 70% are positive. Check with a reliability diagram (predicted vs observed rate per bin) or the Brier score. Fix it on held-out data with Platt scaling (a logistic regression on the scores) or isotonic regression (a flexible monotonic mapping). Boosted trees, SVMs, naive Bayes and models trained on resampled data are often miscalibrated. → *More:* Ch 6 §6.5 — Calibration

Q52. ⋆ Explain k-fold cross-validation. How do you choose k? Split the data into k equal folds; train on k − 1 folds and validate on the remaining one; repeat k times so each fold is validated once; report the mean and standard deviation of the scores. k = 5 or 10 is the usual compromise: larger k uses more data for training each time (less bias) but costs more and gives more correlated estimates. Leave-one-out (k = n) is for tiny datasets only. → *More:* Ch 6 §6.6 — Cross-validation

Q53. What is stratified, group and time-series cross-validation? *Stratified* k-fold keeps the class proportions the same in every fold, important for imbalanced classes. *Group* k-fold keeps all rows of one entity (a patient, a user) in the same fold, so the model is tested on unseen entities. *Time-series* splitting always validates on data later than the training data. Pick the scheme that mimics how the model will meet new data in production. → *More:* Ch 6 §6.6

Q54. What is nested cross-validation and when do you need it? An inner cross-validation loop tunes hyperparameters, and an outer loop estimates how well the whole "tune then train" procedure generalises. You need it when you want an honest performance estimate from small data without a separate test set; reporting the best score from the same CV you tuned on is optimistically biased. → *More:* Ch 6 §6.6

Q55. ⋆ Grid search vs random search vs Bayesian optimisation? Grid search tries every combination of listed values: fine for one or two hyperparameters, wasteful beyond that. Random search samples combinations at random; for the same budget it tries many more distinct values of each important hyperparameter, so it usually wins when only a few hyperparameters really matter. Bayesian optimisation (Optuna, for example) builds a model of score vs hyperparameters and picks the next trial where improvement looks likely; best when each trial is expensive. Search learning rates and regularisation strengths on a log scale. → *More:* Ch 6 §6.9 — Hyperparameter tuning

Q56. What is a baseline and why should you always build one? A baseline is the simplest reasonable predictor: the majority class, the mean, "same as last week", a hand rule or a plain logistic regression. It tells you whether the problem is learnable, catches bugs (a fancy model that cannot beat the baseline is broken), and gives the business a reference for how much the ML adds. → *More:* Ch 6 §6.1 — Baselines

Q57. ⋆ How do you evaluate a ranking or recommendation model? Explain NDCG, MAP, MRR and precision@k. Precision@k is the fraction of the top k results that are relevant; recall@k is the fraction of all relevant items that appear in the top k. MRR (mean reciprocal rank) averages 1 / (rank of the first relevant result), good when one right answer matters. MAP (mean average precision) averages the precision at each position where a relevant item appears, then averages that over queries. NDCG (normalised discounted cumulative gain) sums graded relevance divided by $\log_2(\text{rank} + 1)$ and normalises by the best possible ordering, so it rewards putting the most relevant items at the top. → *More:* Ch 6 §6.4 — Ranking metrics, Ch 13 §13.11

Q58. Offline metrics improved but the A/B test showed no gain. Why might that be? Common causes: the offline metric is not aligned with the business metric (better log loss on clicks, no effect on revenue); training/serving skew (features differ online); the evaluation data was biased by the old model's choices; leakage inflated the offline score; novelty or position effects; the improvement is real but too small to detect with the traffic you had; or the change affects a segment too small to move the total. Check online feature parity and slice the experiment results. → *More:* Ch 16 §16.7, Ch 16 §16.9

Q59. How do you compare two models properly? Is a 0.5% gain real? Use the same splits for both, look at the spread across folds or bootstrap resamples, and use a paired test (for example a paired t-test on per-fold scores or McNemar's test on the same test cases). A gain smaller than the fold-to-fold noise is not convincing. Also weigh cost: a 0.5% gain may not be worth twice the latency. → *More:* Ch 6 §6.6, Ch 2 §2.11

Q60. What are learning curves and what do they tell you? They plot training and validation scores against the number of training examples. A large gap that narrows as data grows means high variance: more data will help. Both curves flat and low means high bias: more data will not help, you need a better model or features. → *More:* Ch 6 §6.8

Q61. Micro vs macro vs weighted averaging for multiclass metrics? Macro averaging computes the metric per class and takes the plain mean, so small classes count equally. Weighted averaging weights each class by its frequency. Micro averaging pools all TP/FP/FN counts first; for single-label multiclass problems micro-F1 equals accuracy. Use macro when rare classes matter. → *More:* Ch 6 §6.3

Q62. How do you translate an ML metric into a business decision? Attach costs and values to each cell of the confusion matrix (the value of catching a fraud, the cost of a manual review, the cost of an annoyed customer), pick the threshold that maximises expected profit on validation data, and report the result in business units ("saves $X per month at Y reviews per day"). Then confirm with an online experiment. → *More:* Ch 6 §6.11 — From ML metric to business decision

---

5. Linear and logistic regression

Q63. ⋆ Explain linear regression. What loss does it minimise? Linear regression predicts a number as a weighted sum of the features plus an intercept: $\hat{y} = w \cdot x + b$. It is fitted by minimising the mean squared error between predictions and targets (ordinary least squares). Each weight is the expected change in the prediction for a one-unit increase in that feature, holding the others fixed. → *More:* Ch 5 §5.2 — Linear regression: model and loss

Q64. ⋆ What are the assumptions of linear regression? Linearity (the target is a linear function of the features); independent errors; homoscedasticity (constant error variance across predictions); normally distributed errors (needed for confidence intervals and p-values, not for prediction); and no strong multicollinearity. Check them with residual plots (residuals vs fitted should look like random noise) and Q-Q plots. → *More:* Ch 5 §5.5 — Assumptions of linear regression

Q65. ⋆ Normal equation vs gradient descent: when would you use each? The normal equation $\theta = (X^TX)^{-1}X^Ty$ gives the exact solution in one step with no learning rate, but costs about $O(n \cdot d^2 + d^3)$ for n rows and d features (forming X^TX, then solving it) and needs the data in memory. Gradient descent iterates small steps downhill; it scales to millions of rows and features, works for any differentiable loss, and supports streaming mini-batches, but needs a learning rate and scaled features. Use the closed form for small d, gradient descent otherwise. (Libraries actually solve least squares with QR or SVD, which is more stable than inverting X^TX.) → *More:* Ch 5 §5.3 — The normal equation vs gradient descent

Q66. What is multicollinearity and why does it matter? Multicollinearity means features are strongly correlated with each other. Predictions can still be fine, but individual coefficients become unstable (large, sign-flipping, with huge standard errors), so you cannot interpret them. Detect it with the correlation matrix or VIF (variance inflation factor; above 5–10 is a warning). Fix it by dropping or combining features, PCA, or Ridge regularisation. → *More:* Ch 5 §5.5

Q67. Your data has columns x, y and z = x + y. What happens to linear regression? The columns are perfectly collinear, so X^TX is singular (not invertible) and there are infinitely many weight combinations giving the same predictions. The normal equation fails; a library using a pseudo-inverse or gradient descent returns one of the solutions, with meaningless individual weights. If z = x + y + small noise, the problem is solvable but the weights are unstable. Ridge regularisation or dropping z fixes both. → *More:* Ch 5 §5.5

Q68. ⋆ Explain logistic regression. Why is it called "regression" if it classifies? It computes a linear score $z = w \cdot x + b$ and passes it through the sigmoid $\sigma(z) = 1 / (1 + e^{-z})$ to get a probability between 0 and 1. It is trained by minimising log loss (equivalently, maximising the likelihood of the labels). The name comes from regressing the log-odds, $\log(p / (1 - p))$, linearly on the features; you get a class by thresholding the probability. → *More:* Ch 5 §5.8 — Logistic regression

Q69. Is logistic regression a linear model? Yes, as a classifier. The probability is a non-linear (sigmoid) function of the features, but the decision boundary $p = 0.5$ is where $w \cdot x + b = 0$, which is a straight line (a hyperplane in many dimensions). It can model curved boundaries only if you add non-linear features such as polynomial terms. → *More:* Ch 5 §5.8

Q70. ⋆ Why does logistic regression use log loss instead of mean squared error? With the sigmoid, MSE gives a non-convex loss surface (multiple flat regions and local minima) and very small gradients when the model is confidently wrong. Log loss is the maximum-likelihood loss for a Bernoulli outcome, is convex for logistic regression so gradient descent finds the global minimum, and gives the clean gradient $(p - y) \cdot x$. → *More:* Ch 5 §5.8, Ch 2 §2.12

Q71. ⋆ How do you interpret the coefficients of a logistic regression? A coefficient w is the change in log-odds for a one-unit increase in that feature, holding others fixed; e^w is the odds ratio. For example $w = 0.7$ means the odds multiply by $e^{0.7} \approx 2$. Compare magnitudes only after standardising features, and be careful with correlated features, whose coefficients share credit unpredictably. → *More:* Ch 5 §5.10 — Interpreting coefficients

Q72. If one weight is larger than another, is that feature more important? Not necessarily. Weights depend on the feature's units (income in dollars vs thousands), so compare only standardised weights. Even then, correlated features split or swap weight, and a big weight on a rarely non-zero feature may matter little overall. Use permutation importance or SHAP on held-out data for importance. → *More:* Ch 5 §5.10, Ch 19 §19.5

Q73. How does logistic regression handle multiple classes? What is softmax? Either one-vs-rest (train one binary classifier per class, pick the highest score) or multinomial (softmax) regression. Softmax turns a vector of K scores into probabilities that sum to 1: $p_i = e^{z_i} / \Sigma_j e^{z_j}$. It is trained with categorical cross-entropy and is also the standard output layer of neural classifiers. → *More:* Ch 5 §5.9 — One-vs-rest and softmax

Q74. What does the intercept (bias term) mean in a linear model? It is the prediction when every feature is zero, or, if features are standardised, the prediction for an average example. In logistic regression it sets the base log-odds, so it absorbs the class prior. Without it the fitted line is forced through the origin, which is almost never correct. → *More:* Ch 5 §5.10

Q75. What is polynomial regression, and is it still linear regression? You add powers and products of features (x^2, $x \cdot z$) and then fit ordinary linear regression on the expanded features. It is still linear in the parameters, so the same solver works, but it fits curves in the original inputs. High degrees overfit quickly and blow up the number of features, so combine with regularisation. → *More:* Ch 5 §5.6 — Polynomial regression

Q76. What's the difference between linear regression with MSE and with MAE or Huber loss? MSE fits the conditional mean and is very sensitive to outliers because errors are squared. MAE fits the conditional median and ignores how far away outliers are. Huber loss is quadratic for small errors and linear for large ones, a compromise. Quantile loss fits any chosen percentile, useful for prediction intervals such as "90th-percentile delivery time". → *More:* Ch 6 §6.2, Ch 20 §20.6 — Loss functions

---

6. Decision trees, random forests and boosting

Q77. ⋆ How does a decision tree decide where to split? At each node it tries every feature and candidate threshold and keeps the split that most reduces impurity: Gini impurity or entropy for classification, variance (MSE) for regression. It repeats greedily on each child until a stopping rule (maximum depth, minimum samples per leaf). Greedy means it picks the best split now without looking ahead, so the tree is good but not globally optimal. → *More:* Ch 7 §7.1 — Decision trees

Q78. ⋆ Gini impurity vs entropy (information gain)? Gini = $1 - \Sigma\, p_i^2$, the chance of misclassifying a random sample labelled by the node's class mix. Entropy = $-\Sigma\, p_i \log_2 p_i$; information gain is the parent's entropy minus the weighted entropy of the children. They choose the same split most of the time; Gini is slightly cheaper (no logarithm) and is scikit-learn's default. → *More:* Ch 7 §7.1

Q79. ⋆ Why do decision trees overfit, and how do you stop it? A tree can keep splitting until every leaf is pure, memorising the training data, including noise; a small change in data can produce a very different tree (high variance). Control it with max depth, min samples per leaf or split, max leaf nodes, cost-complexity pruning (`ccp_alpha`), or by averaging many trees in an ensemble. → *More:* Ch 7 §7.2 — Bias–variance of trees

Q80. What are the advantages and disadvantages of a single decision tree? Pros: easy to explain and draw, no scaling needed, handles mixed feature types and non-linear interactions, fast predictions. Cons: unstable and prone to overfitting, axis-aligned staircase boundaries, cannot extrapolate beyond the range of the training targets, and usually less accurate than ensembles. → *More:* Ch 7 §7.1

Q81. ⋆ What is bagging? How does a random forest work? Bagging (bootstrap aggregating) trains many models on bootstrap samples (random samples drawn with replacement) and averages their predictions, which cuts variance. A random forest bags deep decision trees and adds one more trick: at each split it considers only a random subset of features (`max_features`). This decorrelates the trees, so averaging removes more variance than bagging alone. → *More:* Ch 7 §7.3 — Bagging and Random Forests

Q82. Why does a random forest need feature randomness, not just bootstrap samples? If one feature is very strong, every bagged tree splits on it first and the trees become highly correlated; averaging correlated models barely reduces variance. Restricting each split to a random subset of features forces trees to use other features, making their errors less correlated, so the average is much better. → *More:* Ch 7 §7.3

Q83. What is the out-of-bag (OOB) error? Each bootstrap sample leaves out about 37% of the rows (the chance a row is never picked is $(1 - 1/n)^n \approx 1/e$). Each row can therefore be predicted by the trees that did not see it. The average of those predictions gives a free validation estimate without a separate hold-out set. → *More:* Ch 7 §7.3

Q84. Can a random forest overfit by adding more trees? How many trees do you need? Adding trees does not increase overfitting; the error levels off as the average stabilises, and extra trees only cost time and memory. Pick the point where the OOB or validation score stops improving (often a few hundred). A forest can still overfit through its individual trees (very deep trees on noisy data), which you control with depth and leaf-size limits. → *More:* Ch 7 §7.3

Q85. ⋆ What is boosting? Explain gradient boosting. Boosting builds models sequentially, each one focusing on the mistakes of the ensemble so far, and adds them up. Gradient boosting fits each new shallow tree to the negative gradient of the loss with respect to the current predictions (for squared error, simply the residuals), then adds it scaled by a learning rate (shrinkage). Many small corrective steps build a strong model; it mainly reduces bias. → *More:* Ch 7 §7.4 — Boosting

Q86. ⋆ What's the difference between bagging and boosting? Random forest vs gradient boosting? Bagging trains independent models in parallel on resampled data and averages them: it reduces variance, uses deep trees, and is robust with little tuning. Boosting trains models sequentially on the previous errors: it reduces bias, uses shallow trees, and is usually more accurate on tabular data but more sensitive to hyperparameters and noisy labels, and needs early stopping. Random forest is the "safe default"; gradient boosting (XGBoost, LightGBM, CatBoost) is the "win the benchmark" choice. → *More:* Ch 7 §7.2–7.4

Q87. How does AdaBoost differ from gradient boosting? AdaBoost reweights the training examples: after each weak learner (often a one-split "stump"), misclassified examples get higher weight so the next learner focuses on them, and each learner gets a vote based on its accuracy. Gradient boosting instead fits each new learner to the negative gradient of any differentiable loss. AdaBoost turns out to be the same as forward stagewise additive modelling with the exponential loss, so it can be seen as one member of the gradient-boosting family. → *More:* Ch 7 §7.4

Q88. ⋆ What makes XGBoost and LightGBM fast and accurate? XGBoost uses a second-order (gradient and Hessian) approximation of the loss, explicit regularisation on leaf weights (L1/L2) and on the number of leaves (γ), sparsity-aware handling of missing values (each split learns a default direction), and parallel split finding; since version 2.0 its default `tree_method` is `hist`, the same histogram binning LightGBM popularised. By default XGBoost grows trees depth-wise (level by level). LightGBM grows trees leaf-wise (always splitting the leaf with the largest gain), which reaches a lower loss with the same number of leaves but can overfit small data, so you cap `num_leaves` and `min_data_in_leaf`; it also adds GOSS (gradient-based one-side sampling: keep rows with large gradients, sample the rest) and exclusive feature bundling for sparse features. CatBoost adds ordered target statistics for categorical features and symmetric (oblivious) trees. All three support early stopping and native categorical features. → *More:* Ch 7 §7.4

Q89. What are the most important hyperparameters of gradient boosting, and how do you tune them? Number of trees and learning rate (they trade off: smaller learning rate needs more trees; set a small rate such as 0.05 and choose the number of trees by early stopping); tree depth or number of leaves (interaction complexity); minimum samples or child weight per leaf; row and column subsampling; L1/L2 penalties on leaves. Tune depth and leaves first, then subsampling and regularisation, with random or Bayesian search. → *More:* Ch 7 §7.7 — Hyperparameter cheat sheet

Q90. Can you parallelise random forest training? Gradient boosting? Random forest is trivially parallel: each tree is independent. Gradient boosting is sequential across trees because each tree depends on the previous ones, but the work inside one tree (finding the best split across features or histogram bins) is parallelised, which is what XGBoost and LightGBM do. → *More:* Ch 7 §7.3–7.4

Q91. ⋆ How do you get feature importance from tree models, and why can it mislead? Impurity-based importance sums how much each feature reduced impurity across all splits. It is computed on training data, favours high-cardinality and continuous features (they offer more split points), and splits credit unpredictably among correlated features. Better: permutation importance on held-out data (shuffle one

feature and measure the score drop) or SHAP values, which attribute each prediction fairly among features. → *More:* Ch 7 §7.3, Ch 19 §19.5 — Explainability

Q92. What are stacking and voting ensembles? Voting combines several different models by majority vote (hard) or averaged probabilities (soft). Stacking trains a meta-model (often a logistic regression) on the out-of-fold predictions of base models, so it learns how much to trust each one. Both help most when the base models make different kinds of errors. → *More:* Ch 7 §7.5 — Voting and stacking

Q93. Why do ensembles usually beat single models? Averaging several models whose errors are not perfectly correlated cancels part of the error: the variance of an average of n independent models falls by a factor of n, and even partly correlated ones help. Boosting additionally reduces bias. The gain depends on diversity, which is why random forests inject randomness and why stacking mixes different model families. → *More:* Ch 7 §7.2–7.5

Q94. When would you choose gradient-boosted trees over a neural network? For tabular data with mixed numeric and categorical columns, up to millions of rows, where features are already meaningful: boosted trees are usually more accurate, need less tuning, train on a CPU, handle missing values, and are easier to explain. Neural networks win on raw unstructured data (images, audio, text) and very large datasets where learned representations pay off. → *More:* Ch 7 §7.6 — When trees beat neural networks

Q95. Why can't tree models extrapolate, and why does that matter for time series? A regression tree predicts a leaf average of training targets, so it can never output a value outside the range it saw. If a trend pushes sales higher than ever before, a tree-based forecaster flattens out at the historical maximum. Remedies: model differences or ratios instead of levels, detrend first, or use a linear component. → *More:* Ch 7 §7.1, Ch 12 §12.10

---

7. k-NN, SVM and naive Bayes

Q96. ⋆ How does k-nearest neighbours work? How do you choose k? To predict for a new point, find the k closest training points (by Euclidean or another distance on scaled features) and take a majority vote (classification) or average (regression), optionally weighted by distance. Small k gives a jagged, high-variance boundary; large k smooths toward the majority (high bias). Choose k by cross-validation; an odd k avoids ties in binary problems. → *More:* Ch 8 §8.1 — k-Nearest Neighbours

Q97. What are the drawbacks of k-NN? Prediction is slow (O(n·d) per query without an index) and needs the whole training set in memory; it is sensitive to feature scaling and irrelevant features; and it degrades in high dimensions where distances become similar. Approximate nearest-neighbour indexes (HNSW, IVF) make it practical at scale, which is how embedding search works. → *More:* Ch 8 §8.1, Ch 13 §13.9

Q98. ⋆ Explain support vector machines. An SVM finds the hyperplane that separates two classes with the largest margin, the gap to the nearest points, which are called support vectors. A wider margin generalises better. The soft-margin version allows some points inside the margin or misclassified, with the hyperparameter C setting the penalty: large C fits training data tightly (low bias, high variance), small C prefers a wider margin. The loss is the hinge loss, max(0, 1 − y·f(x)). → *More:* Ch 8 §8.2 — Support Vector Machines

Q99. ⋆ What is the kernel trick? Many problems are not linearly separable in the original space but are after mapping to a higher-dimensional feature space φ(x). The SVM's dual formulation only needs dot products between examples, so you can replace x·z with a kernel K(x, z) = φ(x)·φ(z) computed directly, without ever building φ. Common kernels: linear, polynomial and RBF (Gaussian), K = exp(−γ‖x − z‖²). → *More:* Ch 8 §8.2

Q100. What do C and gamma control in an RBF SVM? C is the penalty for margin violations (high C: fewer training errors, more complex boundary). Gamma sets how far one training point's influence reaches: high gamma means very local influence and wiggly boundaries that can overfit; low gamma means smooth boundaries. They interact, so tune them together on a log-scale grid, with features standardised. → *More:* Ch 8 §8.2

Q101. When would you use an SVM vs logistic regression? Both are linear classifiers in their basic form; logistic regression gives calibrated-ish probabilities and scales to huge data, and the hinge loss of SVMs focuses only on boundary points. Kernel SVMs are strong on small-to-medium, high-dimensional data (text, bioinformatics) where a non-linear boundary helps, but training cost grows roughly quadratically or worse with the number of samples, so they are rarely used above a few hundred thousand rows. → *More:* Ch 8 §8.2, Ch 8 §8.4

Q102. ⋆ Explain naive Bayes. Why is it "naive", and why does it still work? It applies Bayes' theorem, P(class | features) ∝ P(class) · Π P(feature$_i$ | class), assuming features are independent given the class. That assumption is almost never true (words in an email are correlated), which is the "naive" part. It still classifies well because you only need the highest-scoring class to be right, not the exact probabilities; it is fast, needs little data and is a classic text baseline. Its probabilities, however, are usually over-confident. → *More:* Ch 8 §8.3 — Naive Bayes

Q103. What is Laplace (additive) smoothing in naive Bayes? If a word never appeared with a class in training, its estimated probability is zero, and multiplying by zero wipes out all other evidence. Laplace smoothing adds a small count α (often 1) to every word count so no probability is exactly zero. In practice you also sum log-probabilities rather than multiplying probabilities, to avoid numerical underflow. → *More:* Ch 8 §8.3

Q104. How do you choose among classic supervised algorithms? Consider data size and type, the need for interpretability, latency and the shape of the boundary. Start with a fast baseline (logistic or linear regression); use gradient-boosted trees for tabular data; naive Bayes or linear models on TF-IDF for quick text classifiers; k-NN when similarity is the whole point; kernel SVMs on small high-dimensional data. Always compare with cross-validation rather than reputation. → *More:* Ch 8 §8.4 — Choosing a classic supervised algorithm, Ch 20 §20.16

---

8. Unsupervised learning: clustering and dimensionality reduction

Q105. ⋆ How does k-means work? Choose k initial centroids; assign every point to its nearest centroid; move each centroid to the mean of its assigned points; repeat until assignments stop changing. This minimises within-cluster sum of squares (inertia). It always converges, but only to a local optimum, so use k-means++ initialisation (spread-out starting centroids) and several restarts, and scale features first. → *More:* Ch 9 §9.1 — k-means clustering

Q106. ⋆ How do you choose k in k-means? Elbow method: plot inertia against k and look for the bend where adding clusters stops helping much. Silhouette score: for each point, compares the average distance to its own cluster with the distance to the nearest other cluster (from −1 to 1, higher is better). Also gap statistic, stability across resamples, and above all whether the clusters are useful for the business purpose. → *More:* Ch 9 §9.1, Ch 9 §9.9

Q107. What are the limitations of k-means? It assumes roughly spherical clusters of similar size and density, needs k chosen in advance, is sensitive to initialisation, outliers and feature scale, and works only with means, so not with categorical data (k-modes or k-prototypes handle that). For elongated or nested shapes use DBSCAN, Gaussian mixtures or hierarchical clustering. → *More:* Ch 9 §9.1

Q108. How does DBSCAN work, and when would you choose it over k-means? DBSCAN (density-based spatial clustering of applications with noise) marks a point as a core point if at least `min_samples` points lie within radius ε; clusters grow from connected core points, and points reachable from none are labelled noise. Choose it when clusters have arbitrary shapes, when the number of clusters is unknown, or when you want outliers flagged. It struggles when clusters have very different densities and in high dimensions. → *More:* Ch 9 §9.3 — DBSCAN

Q109. What is hierarchical clustering? Agglomerative clustering starts with every point as its own cluster and repeatedly merges the two closest clusters, recording the merges in a tree (dendrogram). Cutting the tree at a height gives any number of clusters. The linkage rule defines "closest": single (nearest points, makes chains), complete (farthest points), average, or Ward (smallest increase in variance). It costs at least $O(n^2)$ memory, so it suits small datasets. → *More:* Ch 9 §9.2 — Hierarchical clustering

Q110. What is a Gaussian mixture model and how does EM fit it? A GMM assumes the data comes from k Gaussian distributions, each with its own mean, covariance and weight, giving soft (probabilistic) cluster membership and elliptical clusters. Expectation–maximisation alternates: the E-step computes each point's probability of belonging to each component; the M-step re-estimates means, covariances and weights from those soft assignments. k-means is the limit of a GMM with equal spherical covariances and hard assignments. → *More:* Ch 9 §9.4 — Gaussian mixture models and EM

Q111. ⋆ Explain PCA. How do you choose the number of components? Principal component analysis finds orthogonal directions (principal components) along which standardised data varies the most; they are the eigenvectors of the covariance matrix, or equivalently come from the SVD of the centred data. Projecting onto the first few components keeps most of the variance in fewer dimensions. Choose the number by cumulative explained variance (for example 95%), by a scree-plot elbow, or by downstream model performance. → *More:* Ch 9 §9.5 — PCA, Ch 2 §2.3

Q112. Why must you standardise data before PCA? PCA picks directions of maximum variance, and variance depends on units: income in dollars (values in the tens of thousands) would dominate age in years just because its numbers are bigger, and the same distance written in metres instead of kilometres would get a million times more variance. Standardising puts every feature on equal footing. Also centre the data (subtract the mean), which PCA requires. → *More:* Ch 9 §9.5

Q113. What are the limitations of PCA? It is linear, so it misses curved structure; it is unsupervised, so the directions of maximum variance may not be the ones that predict the target; components are mixtures of features and hard to interpret; and it is sensitive to scaling and outliers. For visualisation of non-linear structure use t-SNE or UMAP; for supervised reduction consider LDA. → *More:* Ch 9 §9.5–9.6

Q114. PCA vs LDA vs t-SNE vs UMAP? PCA: unsupervised, linear, keeps global variance, fast, used for compression and pre-processing. LDA (linear discriminant analysis): supervised, linear, finds directions that best separate known classes (at most classes − 1 dimensions). t-SNE: non-linear, preserves local neighbourhoods, for 2-D visualisation only; distances between clusters and cluster sizes in the plot are not meaningful. UMAP: similar purpose to t-SNE, faster, keeps somewhat more global structure and can transform new points. → *More:* Ch 9 §9.6 — t-SNE and UMAP

Q115. What is SVD and where is it used in ML? Singular value decomposition factors any matrix as $A = U\Sigma V^T$: orthogonal directions in the input and output spaces scaled by singular values. Uses: computing PCA, low-rank approximation and compression (keep the top singular values), latent semantic analysis of text, and matrix-factorisation recommenders, where a user–item matrix is approximated by low-rank user and item factors. → *More:* Ch 2 §2.3, Ch 13 §13.5

Q116. How would you detect anomalies without labels? Statistical rules (z-scores, IQR) for single features; Isolation Forest (anomalies are isolated by few random splits); local outlier factor (density lower than neighbours'); one-class SVM; autoencoder reconstruction error (normal data reconstructs well, anomalies do not); or distance to the nearest cluster centre. Validate with whatever labels you can get (a few confirmed cases) and tune the alert rate to what reviewers can handle. → *More:* Ch 9 §9.7 — Anomaly detection

Q117. How do you evaluate a clustering when you have no labels? Internal metrics: silhouette score, Davies–Bouldin index (lower is better), Calinski–Harabasz. Stability: do clusters stay the same when you resample or change the seed? Usefulness: can a domain expert name them, and do they improve a downstream task (targeting, features for a model)? If some labels exist, use adjusted Rand index or normalised mutual information. → *More:* Ch 9 §9.9 — Evaluating unsupervised models

Q118. What are association rules (support, confidence, lift)? Used in market-basket analysis ("people who buy bread and butter also buy milk"). Support: how often the item set appears. Confidence: P(milk | bread and butter). Lift: confidence divided by milk's overall frequency; lift above 1 means the items co-occur more than by chance. Apriori finds frequent item sets by pruning any set whose subsets are not frequent. → *More:* Ch 9 §9.8 — Association rules

---

9. Probability, statistics and math

Q119. ⋆ Explain Bayes' theorem with an example. $P(A \mid B) = P(B \mid A) \cdot P(A) / P(B)$: update a prior belief P(A) with evidence B. Example: a disease affects 1% of people; a test catches 90% of cases and gives 5% false positives. $P(\text{positive}) = 0.9 \times 0.01 + 0.05 \times 0.99 = 0.0585$, so $P(\text{sick} \mid \text{positive}) = 0.009 / 0.0585 \approx 15\%$. Most positives are false because the disease is rare: the base-rate effect, the same reason rare-event classifiers have low precision. → *More:* Ch 2 §2.7 — Conditional probability and Bayes' theorem

Q120. ⋆ What is a p-value? What is it not? The p-value is the probability of seeing data at least as extreme as what you observed if the null hypothesis (no effect) were true. A small p-value (below a pre-chosen α such as 0.05) means the data would be surprising under no effect. It is not the probability that the null hypothesis is true, not the size or importance of the effect, and it becomes misleading if you test many things or peek repeatedly and stop when significant. → *More:* Ch 2 §2.11 — Hypothesis tests, p-values and confidence intervals

Q121. ⋆ What is a confidence interval? A range computed from the sample by a procedure that, over many repeated samples, contains the true value a stated fraction of the time (for example 95%). It does not mean "there is a 95% chance the true value is in this particular interval" (that is the Bayesian credible interval's meaning). Its width shows the precision of the estimate; it shrinks roughly with $1 / \sqrt{n}$. → *More:* Ch 2 §2.11

Q122. ⋆ What is the central limit theorem and why does it matter? The average of many independent samples from almost any distribution with finite variance is approximately normally distributed, with standard error $\sigma / \sqrt{n}$, however skewed the original data. It is why z-tests and t-tests, confidence intervals and A/B-test significance calculations work on non-normal metrics like revenue per user, given enough samples. → *More:* Ch 2 §2.10 — Sampling and the central limit theorem

Q123. What is the normal distribution and why do we care? How do you check normality? The bell curve, defined by mean and standard deviation; about 68%, 95% and 99.7% of values fall within 1, 2 and 3 standard deviations. It appears through the central limit theorem and is the assumption behind many tests and the MSE loss (which is maximum likelihood under Gaussian noise). Check normality with a histogram, a Q-Q plot (points on a straight line), or tests such as Shapiro–Wilk (which flag tiny deviations on large samples). → *More:* Ch 2 §2.6 — Random variables and distributions

Q124. Name common probability distributions and when each appears. Bernoulli (one yes/no event, a click); binomial (number of successes in n trials); Poisson (count of events in a fixed interval, such as orders per minute); exponential (time between Poisson events); normal (sums and averages); uniform (equal chance); log-normal (incomes, prices); beta (a distribution over probabilities, used for click-rate priors and Thompson sampling). → *More:* Ch 2 §2.6

Q125. ⋆ Correlation vs causation? Correlation means two variables move together; causation means changing one changes the other. Correlation can come from a confounder (ice-cream sales and drownings both rise in summer), reverse causation or chance. To establish causation you need a randomised experiment (A/B test) or careful causal-inference methods. Predictive models exploit correlation, which is fine for prediction but risky when you use the model to decide on an intervention. → *More:* Ch 2 §2.9 — Correlation vs causation

Q126. Pearson vs Spearman correlation? Pearson measures linear association between two numeric variables and is sensitive to outliers. Spearman is Pearson computed on ranks, so it measures any monotonic relationship and is robust to outliers. Both range from −1 to 1; zero correlation does not mean independence ($y = x^2$ over symmetric x has Pearson ≈ 0). → *More:* Ch 2 §2.9

Q127. ⋆ What is maximum likelihood estimation (MLE)? How does it relate to common losses? MLE picks the parameters that make the observed data most probable. Minimising MSE is MLE under Gaussian noise; minimising log loss is MLE for Bernoulli/categorical outcomes. MAP (maximum a posteriori) estimation adds a prior over parameters: a Gaussian prior gives L2 regularisation and a Laplace prior gives L1. → *More:* Ch 2 §2.12, Ch 5 §5.8

Q128. Bayesian vs frequentist statistics? Frequentists treat parameters as fixed unknowns and probability as long-run frequency; they use p-values and confidence intervals. Bayesians treat parameters as random with a prior distribution and update it to a posterior with data, giving direct statements like "P(B is better than A) = 92%". Bayesian methods handle small data and prior knowledge naturally but require choosing priors and can be costlier to compute.

Q129. ⋆ What are entropy, cross-entropy and KL divergence? Entropy $H(p) = -\Sigma\ p \log p$ measures the uncertainty of a distribution (a fair coin has 1 bit). Cross-entropy $H(p, q) = -\Sigma\ p \log q$ is the average cost of encoding outcomes from p using a code built for q; with one-hot labels it is exactly the classification loss. KL divergence $KL(p\|q) = H(p, q) - H(p)$ measures how far q is from p; it is non-negative and not symmetric. KL appears in VAEs, knowledge distillation and RLHF penalties. → *More:* Ch 2 §2.12 — Information theory

Q130. ⋆ Explain gradient descent. What happens if the learning rate is too large or too small? Gradient descent repeatedly moves parameters a small step against the gradient (the direction of steepest increase of the loss): $w \leftarrow w - \eta \cdot \nabla L(w)$, where η is the learning rate. Too small: very slow progress, may stall. Too large: overshoots the minimum, oscillates or diverges (loss goes to infinity or NaN). Feature scaling makes the loss surface rounder so one learning rate suits all directions. → *More:* Ch 2 §2.5 — Gradient descent, Ch 5 §5.4

Q131. ⋆ Batch vs stochastic vs mini-batch gradient descent? Batch gradient descent computes the gradient on the full dataset per step: accurate but slow and memory-hungry. Stochastic (SGD) uses one example per step: cheap and noisy; the noise can help escape poor regions but makes convergence jumpy. Mini-batch (32–1024 examples) is the practical default: GPU-friendly vectorised maths with moderate noise. An *epoch* is one pass through all data; an *iteration* is one update. → *More:* Ch 5 §5.4 — Gradient descent in practice

Q132. What is a convex function, and why does convexity matter in ML? A function is convex if the straight line between any two points on its graph lies on or above the graph, so any local minimum is the global minimum. Linear regression with MSE, logistic regression with log loss and SVMs are convex, so gradient descent reliably finds the best solution. Neural network losses are non-convex, yet in practice gradient descent finds good-enough minima. → *More:* Ch 2 §2.5

Q133. What are eigenvalues and eigenvectors? Where do they appear in ML? An eigenvector v of a square matrix A is a direction that A only stretches: $Av = \lambda v$, where λ (the eigenvalue) is the stretch factor. In PCA the eigenvectors of the covariance matrix are the principal components and the eigenvalues are the variance along each. They also appear in PageRank, spectral clustering and the analysis of exploding/vanishing gradients in RNNs. → *More:* Ch 2 §2.3 — Eigenvalues, eigenvectors and SVD

Q134. What are the gradient, Jacobian and Hessian? The gradient is the vector of partial derivatives of a scalar function, pointing uphill. The Jacobian is the matrix of partial derivatives of a vector-valued function (each output with respect to each input), used when chaining layers in backpropagation. The Hessian is the matrix of second derivatives, describing curvature; Newton's method uses it, and XGBoost uses its diagonal. → *More:* Ch 2 §2.4 — Derivatives, partial derivatives and gradients

Q135. Variance, standard deviation and standard error: what's the difference? Variance is the average squared distance from the mean; standard deviation is its square root, in the data's units, describing the spread of individual values. Standard error is the standard deviation of an estimate (such as a sample mean), $\sigma / \sqrt{n}$, describing how precisely you know it. More data shrinks the standard error, not the standard deviation. → *More:* Ch 2 §2.8 — Descriptive statistics

Q136. Probability puzzle: a family has two children and at least one is a girl. What is the probability both are girls? List equally likely outcomes: GG, GB, BG, BB. "At least one girl" removes BB, leaving three, of which one is GG, so the answer is 1/3. (If instead you know the *older* child is a girl, the answer is 1/2.) The point is to condition on exactly the information given. → *More:* Ch 2 §2.7

Q137. Can two events with P(A) = 0.6 and P(B) = 0.7 be disjoint? No. Disjoint (mutually exclusive) events satisfy P(A or B) = P(A) + P(B) = 1.3, which exceeds 1, impossible. They must overlap by at least 0.3: P(A and B) ≥ 0.6 + 0.7 − 1.

Q138. What is the expected number of coin flips to get two heads in a row? Let E be the expected flips from the start. Flip once: tails (½) restarts; heads (½) then a second flip either finishes (½) or restarts (½). $E = ½(1 + E) + ½·½·2 + ½·½(2 + E)$, which gives $E = 6$. Interviewers want the method: set up states and solve linear equations for expectations.

Q139. ⋆ How do you decide the sample size for an A/B test? What is statistical power? Power is the probability of detecting a real effect of a given size (commonly 80%). Sample size per group grows with the metric's variance and shrinks with the square of the minimum detectable effect (MDE): roughly $n \approx 16 \cdot \sigma^2 / \delta^2$ per group for α = 0.05 and 80% power. Halving the effect you want to detect needs four times the traffic. Decide the size before starting and do not stop early when the result first looks significant. → *More:* Ch 16 §16.9 — Online experiments and A/B testing

Q140. What is the multiple-comparisons problem? Test 20 independent metrics at α = 0.05 and you expect one "significant" result by pure chance. Corrections: Bonferroni (divide α by the number of tests; conservative), Holm, or Benjamini–Hochberg (controls the false discovery rate). Better still, name one primary metric before the experiment. → *More:* Ch 2 §2.11

Q141. How would you sample uniformly from a stream of unknown length (reservoir sampling)? Keep the first k items. For the i-th item (counting from 1, i > k), draw a random integer j from 1 to i; if j ≤ k, replace slot j with the new item. So the new item enters with probability k / i and evicts a uniformly random current member. By induction, after n items every item is in the reservoir with probability exactly k / n. It uses O(k) memory and one pass, which is useful for sampling training data from logs.

---

10. Deep learning fundamentals

Q142. How does a feed-forward neural network work? Inputs pass through layers; each layer computes a weighted sum of the previous layer's outputs plus a bias and applies a non-linear activation function; the final layer produces the prediction (a number, or class probabilities via softmax). Training compares the prediction with the label using a loss, computes gradients with backpropagation, and updates the weights with an optimizer. Stacked layers learn increasingly abstract features. → *More:* Ch 10 §10.1 — From the perceptron to the MLP

Q143. ⋆ Explain backpropagation. Backpropagation computes the gradient of the loss with respect to every weight by applying the chain rule backwards through the network. The forward pass stores each layer's intermediate values; the backward pass starts from ∂loss/∂output and, layer by layer, multiplies by each layer's local derivative, reusing results so the whole gradient costs only a small constant multiple of a forward pass (roughly two forward passes' worth of compute). An optimizer such as SGD or Adam then uses the gradients to update the weights. Frameworks do this automatically (autograd). → *More:* Ch 10 §10.4 — Backpropagation

Q144. ⋆ Why do we need non-linear activation functions? A composition of linear layers is just one linear function, so without non-linearities a 100-layer network is no more powerful than logistic regression. Non-linear activations let networks represent curved decision boundaries and complex functions; with enough hidden units, a network with one hidden layer can approximate any continuous function (the universal approximation theorem), though depth makes this far more efficient. → *More:* Ch 10 §10.2 — Activation functions

Q145. ⋆ Compare sigmoid, tanh, ReLU, Leaky ReLU and GELU. Sigmoid squashes to (0, 1), saturates at both ends (gradients near zero) and is not zero-centred: use only for binary output probabilities or gates. Tanh squashes to (−1, 1), zero-centred, still saturates. ReLU, max(0, x), is cheap, does not saturate for positive inputs and is the default for hidden layers, but units can "die" (stuck at zero). Leaky ReLU keeps a small slope for negatives to avoid dead units. GELU and SiLU/Swish are smooth ReLU-like functions used in transformers. → *More:* Ch 10 §10.2

Q146. What is the dying ReLU problem? If a ReLU unit's input becomes negative for all examples (often after a large update from a high learning rate), its output and gradient are zero, so it never updates again. Fixes: a lower learning rate, Leaky ReLU/ELU/GELU, careful (He) initialisation, and normalisation layers. → *More:* Ch 10 §10.2

Q147. ⋆ What are vanishing and exploding gradients? How do you fix them? During backpropagation the gradient for early layers is a product of many per-layer factors. If those factors are mostly below 1 the gradient shrinks exponentially (vanishing) and early layers stop learning; the sigmoid's derivative is at most 0.25, so deep sigmoid networks vanish easily; if above 1 it grows exponentially (exploding) and training diverges. Fixes: ReLU-family activations, good initialisation (He/Xavier), batch or layer normalisation, residual (skip) connections, gated RNNs (LSTM/GRU), and gradient clipping for explosions. → *More:* Ch 10 §10.6 — Initialization and vanishing/exploding gradients

Q148. ⋆ What happens if you initialise all weights to zero? How should you initialise? Every neuron in a layer computes the same output and receives the same gradient, so they stay identical forever (the symmetry is never broken) and the layer acts like one neuron. Initialise randomly with a scale that keeps the variance of activations stable across layers: Xavier/Glorot (variance ∝ 1/fan-in, or 2/(fan-in + fan-out)) for tanh/sigmoid, He (variance 2/fan-in) for ReLU. Biases can start at zero. → *More:* Ch 10 §10.6

Q149. ⋆ Explain SGD with momentum, RMSProp and Adam. When would you use SGD instead of Adam? Momentum keeps a running average of past gradients (a velocity), smoothing noise and speeding progress along consistent directions. RMSProp divides each parameter's step by a running average of its squared gradients, giving per-parameter learning rates. Adam combines both, with bias correction for the early steps; it converges fast with little tuning and is the default for transformers. SGD with momentum often generalises slightly better for CNNs on vision tasks, given a good learning-rate schedule. → *More:* Ch 10 §10.5 — Optimizers and learning-rate schedules

Q150. What is AdamW and why is it preferred over Adam with L2? In Adam, an L2 penalty added to the loss gets divided by the adaptive denominator, so weights with large gradients are barely regularised. AdamW applies weight decay directly to the weights, separately from the gradient update (decoupled weight decay), which regularises uniformly and generalises better. It is the standard optimizer for training transformers. → *More:* Ch 10 §10.5

Q151. Why use a learning-rate schedule? What is warm-up? A larger learning rate early makes fast progress; a smaller one later settles into a good minimum. Common schedules: step decay, cosine decay and one-cycle. Warm-up starts with a tiny learning rate and ramps up over the first steps, which avoids unstable early updates when Adam's statistics and normalisation layers are not yet reliable; it is standard for transformers. A learning-rate range test (increase the rate until the loss blows up) helps pick the peak. → *More:* Ch 10 §10.5

Q152. ⋆ What is dropout and how does it behave at training vs inference time? During training, dropout sets each unit's output to zero with probability p (for example 0.1–0.5) and scales the survivors by 1/(1 − p), so the network cannot rely on any single co-adapted unit; it acts like training an ensemble of thinned networks. At inference dropout is switched off (`model.eval()` in PyTorch) and all units are used. Forgetting to switch modes is a classic bug. → *More:* Ch 10 §10.7 — Regularization

Q153. ⋆ What is batch normalisation and why does it help? Batch norm vs layer norm? Batch norm normalises each feature (each channel in a CNN, over the batch and spatial positions) to mean 0 and variance 1 over the current mini-batch, then applies a learned scale and shift. It stabilises training, allows higher learning rates, reduces sensitivity to initialisation and adds mild regularisation (batch statistics are noisy). The original "internal covariate shift" explanation is disputed; a smoother loss landscape is the more accepted reason it helps. At inference it uses running averages collected in training. Layer norm normalises across the features of each single example instead, so it does not depend on batch size, which is why transformers and RNNs use it. → *More:* Ch 10 §10.7

Q154. Which loss functions do you use for which tasks? Regression: MSE (or MAE/Huber for outliers). Binary classification: binary cross-entropy on a sigmoid output (in PyTorch use `BCEWithLogitsLoss` on raw logits). Multiclass: categorical cross-entropy on softmax (`CrossEntropyLoss` takes logits). Multilabel: independent sigmoids with BCE. Ranking: pairwise or listwise losses. Embeddings: contrastive or triplet loss. Imbalanced detection: weighted cross-entropy or focal loss. → *More:* Ch 10 §10.3 — The forward pass and loss functions, Ch 20 §20.6

Q155. Epoch vs batch vs iteration? A batch is the group of examples used for one gradient update; an iteration (step) is one update; an epoch is one full pass over the training data. With 10,000 examples and batch size 100, one epoch is 100 iterations. → *More:* Ch 10 §10.8 — Practical training habits

Q156. How does batch size affect training? Larger batches give less noisy gradients and better hardware use, but need more memory, often need a proportionally larger learning rate (with warm-up), and can generalise slightly worse. Smaller batches add noise that can act as regularisation but make training slower per example. When memory is short, use gradient accumulation to simulate a larger batch. → *More:* Ch 10 §10.8

Q157. ⋆ Your network's training loss is not decreasing. How do you debug it? First try to overfit a tiny batch (10 examples): if it cannot reach near-zero loss, there is a bug. Check: data and labels line up (plot some), inputs are normalised, the loss matches the output layer (logits vs probabilities, label format), the learning rate (try ten times smaller and larger), gradients are flowing (no `detach`, `zero_grad` called each step, no dead ReLUs, no NaNs), and `model.train()`/`model.eval()` modes. Then add complexity back one step at a time. → *More:* Ch 10 §10.8

Q158. Your loss becomes NaN during training. What could cause it? A learning rate that is too high (exploding updates); exploding gradients (clip them); log of zero or division by zero in a hand-written loss (use the library's numerically stable versions that take logits); bad input values (NaN or infinite features, unnormalised huge values); or mixed-precision overflow (use a gradient scaler or bfloat16). → *More:* Ch 10 §10.6, Ch 10 §10.8

Q159. What are residual (skip) connections and why do they matter? A residual block outputs x + F(x): the layers learn only a correction to the identity. Gradients flow straight through the addition, so very deep networks (100+ layers) train without vanishing gradients, and adding a layer can never make things much worse because it can learn F ≈ 0. Introduced with ResNet; every transformer layer uses them. → *More:* Ch 11 §11.4 — Famous architectures

Q160. What is an autoencoder? Give applications. A network that compresses the input into a small bottleneck vector (encoder) and reconstructs it (decoder), trained to minimise reconstruction error. Uses: dimensionality reduction, denoising, anomaly detection (high reconstruction error means unusual), and pretraining. Variational autoencoders (VAEs) make the bottleneck a probability distribution so you can sample new data. → *More:* Ch 9 §9.7

Q161. What are the components of a GAN, and what makes GANs hard to train? A generative adversarial network has a generator (turns random noise into fake samples) and a discriminator (tells real from fake); they train in a two-player game until fakes are indistinguishable. Training is unstable: mode collapse (the generator produces little variety), oscillation, and one side overpowering the other. Diffusion models, which learn to remove noise step by step, have largely replaced GANs for image generation because they train more stably.

Q162. What is knowledge distillation? Train a small "student" model to match the softened output probabilities of a large "teacher" model (softmax with a temperature above 1), usually alongside the true labels. The soft targets carry information about which wrong classes are similar, so the student learns more than from hard labels. It is used to make models small and fast enough for serving. → *More:* Ch 16 §16.6 — Model serving basics

Q163. How do you make a model smaller or faster for inference? Quantisation (store weights and compute in 8-bit or 4-bit integers instead of 32-bit floats), pruning (remove small weights or whole channels), knowledge distillation, efficient architectures, batching requests, caching, compiling the model (`torch.compile`, ONNX Runtime, TensorRT) and running on suitable hardware. Measure the accuracy loss on your own validation set after each step. → *More:* Ch 16 §16.6

Q164. What is the difference between `model.train()` / `model.eval()` and `torch.no_grad()` in PyTorch? `model.train()` and `model.eval()` switch layer behaviour: dropout on/off and batch norm using batch statistics vs running statistics. `torch.no_grad()` (or `torch.inference_mode()`) stops autograd from recording operations, saving memory and time. For inference you want both: `model.eval()` and a no-grad context. → *More:* Ch 10 §10.9 — PyTorch basics

---

11. CNNs, RNNs, attention and transformers

Q165. How does a convolutional layer work? What determines the output shape? A small filter (kernel), say 3×3 across all input channels, slides over the image; at each position it takes a weighted sum of the patch, producing one value of an output feature map. A layer learns many filters, one output channel each. For a grayscale input the filter is 3×3×1; for RGB it is 3×3×3. Output width = ⌊(W − K + 2P) / S⌋ + 1 for input width W, kernel K, padding P and stride S; output depth = number of filters. → *More:* Ch 11 §11.1 — Convolution

Q166. ⋆ Why use convolutions for images instead of fully connected layers? Weight sharing: the same filter is applied everywhere, so a 3×3 filter has 9 weights per channel instead of millions of connections. Local connectivity matches the fact that nearby pixels are related. Translation equivariance: a pattern is detected wherever it appears. This means far fewer parameters, less overfitting and better generalisation; stacking layers grows the receptive field from edges to textures to objects. → *More:* Ch 11 §11.1

Q167. What is pooling and why use it? Are CNNs invariant to rotation? Pooling downsamples feature maps: max pooling keeps the strongest activation in each window, average pooling the mean. It reduces computation, enlarges the receptive field and adds a little translation invariance. CNNs are not naturally invariant to rotation or scale: a rotated image can change the prediction, which is why rotation and scale augmentations are used in training. → *More:* Ch 11 §11.2 — Pooling and the CNN architecture

Q168. What is a receptive field, and why use several 3×3 kernels instead of one large kernel? The receptive field is the region of the input that affects one output unit. Two stacked 3×3 convolutions see 5×5 and three see 7×7, with fewer parameters (3 × 9 = 27 vs 49 per channel pair) and more non-linearities in between, so the network is both cheaper and more expressive. This was the key insight of VGG. → *More:* Ch 11 §11.2, Ch 11 §11.4

Q169. Name famous CNN architectures and their main idea. LeNet (early small CNN for digits); AlexNet (deep CNN on GPUs with ReLU and dropout, won ImageNet 2012); VGG (stacks of 3×3 convolutions); GoogLeNet/Inception (parallel filter sizes); ResNet (skip connections enable very deep nets); MobileNet/EfficientNet (efficient convolutions and scaling for mobile); Vision Transformers (ViT) treat image patches as tokens. → *More:* Ch 11 §11.4 — Famous architectures

Q170. ⋆ What is transfer learning and how do you fine-tune a pretrained model? Reuse a model pretrained on a large dataset (ImageNet, a large text corpus) whose early layers already detect general features. Replace the output head with one for your classes; first train only the head with the backbone frozen, then optionally unfreeze some or all layers and fine-tune with a small learning rate. It gives strong results with few labels and little compute. → *More:* Ch 11 §11.5 — Transfer learning and fine-tuning

Q171. Classification vs object detection vs segmentation? What are IoU and non-maximum suppression? Classification gives one label per image; detection gives labelled bounding boxes (Faster R-CNN, YOLO); semantic segmentation labels every pixel with a class, and instance segmentation separates individual objects (Mask R-CNN). IoU (intersection over union) is the overlap of two boxes divided by their combined area, used to match predictions to ground truth. Non-maximum suppression keeps the highest-scoring box and removes others that overlap it above an IoU threshold. → *More:* Ch 11 §11.7 — Vision tasks

Q172. ⋆ How does an RNN work, and why is it hard to train on long sequences? A recurrent neural network processes a sequence one step at a time, updating a hidden state $h_t = f(W \cdot h_{t-1} + U \cdot x_t)$ that carries information forward. Training unrolls the network over time (backpropagation through time), so gradients are products over many steps and vanish or explode; the network then forgets long-range dependencies. It is also sequential, so it cannot be parallelised across time steps. → *More:* Ch 12 §12.2–12.3 — RNNs and vanishing gradients through time

Q173. ⋆ How do LSTMs and GRUs solve the vanishing-gradient problem? An LSTM adds a cell state updated additively, $c_t = f \odot c_{t-1} + i \odot \tilde{c}_t$, controlled by sigmoid gates: forget (f), input (i) and output. Because the update is additive and the forget gate can stay near 1, information and gradients can flow across many steps. A GRU merges the gates into update and reset gates, with fewer parameters and similar accuracy. → *More:* Ch 12 §12.4 — LSTM and GRU

Q174. What is a sequence-to-sequence (encoder–decoder) model? An encoder reads the input sequence into a representation and a decoder generates the output sequence token by token, used for translation, summarisation and speech recognition. Early versions squeezed the whole input into one fixed vector, a bottleneck for long inputs; attention removed it by letting the decoder look back at all encoder states. During training the decoder is fed the true previous token (teacher forcing). → *More:* Ch 12 §12.5 — Sequence-to-sequence models

Q175. ⋆ Explain the attention mechanism. What are Q, K and V? Attention lets each position build its output as a weighted average of other positions' information, with weights computed from similarity. Each token produces a query (what I am looking for), a key (what I contain) and a value (what I pass on). Scores are the dot products of a query with all keys; softmax turns them into weights; the output is the weighted sum of values: Attention(Q, K, V) = softmax(QK^T / √d_k) · V. → *More:* Ch 12 §12.6 — The attention mechanism

Q176. ⋆ Why divide by √d_k in scaled dot-product attention? For random vectors of dimension d_k, dot products have variance proportional to d_k, so with large dimensions the scores become large and softmax saturates: one weight near 1, the rest near 0, with tiny gradients. Dividing by √d_k keeps the scores at unit scale so softmax stays smooth and trainable. → *More:* Ch 12 §12.6

Q177. ⋆ Explain the transformer architecture. A stack of identical blocks, each with multi-head self-attention (every token attends to every other token, in several heads that can learn different relations) followed by a position-wise feed-forward network, both wrapped with residual connections and layer normalisation. Positional encodings add order information, since attention itself ignores order. Encoder-only models (BERT) read text bidirectionally; decoder-only models (GPT-style) use causal masking so each token sees only earlier ones and generate text; encoder–decoder models (T5) translate between sequences. → *More:* Ch 12 §12.7 — Self-attention and the transformer

Q178. Why did transformers replace RNNs? What's the downside? Transformers process all positions in parallel (fast training on GPUs) and connect any two tokens in one step, so long-range dependencies are easy to learn; RNNs process tokens one after another and pass information through many steps. The downside: self-attention costs $O(n^2)$ time and memory in the sequence length n, which is why long-context models use tricks such as FlashAttention, sparse or sliding-window attention and KV caching. → *More:* Ch 12 §12.7

Q179. Why do transformers need positional encoding? Self-attention has no built-in notion of order: if you permute the input tokens, the outputs are just permuted the same way, so without extra information "dog bites man" and "man bites dog" look alike. Positional encodings (fixed sinusoids, learned position embeddings, or rotary embeddings, RoPE, which rotate queries and keys by position) inject order. → *More:* Ch 12 §12.7

Q180. What is multi-head attention? Instead of one attention with large vectors, the model projects Q, K and V into h smaller subspaces and runs attention in each head in parallel, then concatenates and projects the results. Different heads can specialise (one tracks syntax, another coreference) at the same total cost as a single full-size head. → *More:* Ch 12 §12.7

Q181. ⋆ What are word embeddings? How does word2vec work? Embeddings are dense vectors (say 100–1,000 numbers) for words, learned so that words used in similar contexts get similar vectors, enabling analogies like king − man + woman ≈ queen. Word2vec trains a shallow network either to predict a word from its neighbours (CBOW) or neighbours from a word (skip-gram), using negative sampling to keep it cheap. Static embeddings give one vector per word; transformer models give contextual embeddings that differ by sentence ("bank" of a river vs a bank account). → *More:* Ch 12 §12.9 — Word embeddings

Q182. What is TF-IDF and bag-of-words? What are their limitations? Bag-of-words represents a document by word counts, ignoring order. TF-IDF weights each count by inverse document frequency, log(N / number of documents containing the word), so common words like "the" count little and distinctive words count a lot. Adding n-grams (word pairs) captures some order. Limitations: huge sparse vectors, no notion of synonyms, and no word order beyond n-grams. Still a strong, fast baseline with logistic regression. → *More:* Ch 12 §12.8 — Text preprocessing and tokenization

Q183. What is tokenization? Stemming vs lemmatization? Why subword tokenizers? Tokenization splits text into units (tokens). Stemming chops word endings by rules ("studies" → "studi"); lemmatization maps to the dictionary form using vocabulary and grammar ("studies" → "study"). Modern models use subword tokenizers (BPE, WordPiece, SentencePiece) that split rare words into frequent pieces, so the vocabulary stays fixed-size yet any word, typo or name can be represented without "unknown" tokens. → *More:* Ch 12 §12.8

Q184. BERT vs GPT: what's the difference? BERT is an encoder-only transformer pretrained by masked-language modelling (predict hidden words using context on both sides); it is good for understanding tasks such as classification, entity tagging and search embeddings. GPT is decoder-only, pretrained to predict the next token left to right; it is good at generation and, at scale, at following instructions and in-context learning. → *More:* Ch 12 §12.7

Q185. What is retrieval-augmented generation (RAG)? Before a language model answers, the system retrieves relevant documents (usually by embedding the question and searching a vector index, often combined with keyword search) and puts them into the prompt, so the model answers from current, private or citable sources instead of only its training memory. Quality depends on chunking, the embedding model, re-ranking and evaluation of both retrieval and the final answer. → *More:* Ch 13 §13.9 — ANN search, Ch 17 §17.4

Q186. How would you forecast a time series? How is it different from ordinary regression? Observations are ordered and correlated over time, often with trend and seasonality, so you must split by time and avoid features that peek into the future. Baselines: last value, seasonal naive (same day last week), moving averages. Classical models: exponential smoothing, ARIMA. ML: gradient boosting on lag and rolling-window features plus calendar features; deep models (LSTMs, temporal transformers) for many related series. Evaluate with rolling-origin backtests. → *More:* Ch 12 §12.10 — Time-series forecasting

Q187. How do RLHF and reinforcement learning relate to large language models? After pretraining and supervised fine-tuning on example answers, RLHF (reinforcement learning from human feedback) collects human preference rankings between answers, trains a reward model on them, then optimises the language model to maximise that reward (for example with PPO), with a KL penalty that keeps it close to the original model. DPO (direct preference optimisation) skips the separate reward model and trains on preference pairs directly. → *More:* Ch 14 §14.10 — RLHF

Q188. Explore vs exploit: what is a multi-armed bandit, and Q-learning vs SARSA? A bandit repeatedly chooses among options with unknown payoffs, balancing trying options (explore) against using the best so far (exploit), with ε-greedy, UCB (optimism under uncertainty) or

Thompson sampling (sample from the posterior). In full reinforcement learning, Q-learning updates toward the best next action's value (off-policy, learns the optimal policy), while SARSA updates toward the action actually taken (on-policy, safer while exploring). → *More:* Ch 14 §14.4, Ch 14 §14.7

---

12. Recommender systems and embeddings

Q189. ⋆ Content-based vs collaborative filtering? Content-based filtering recommends items similar to what the user liked, using item attributes (genre, text, image embeddings); it works for new items but tends to recommend more of the same. Collaborative filtering uses the behaviour of many users ("people who liked X also liked Y"), discovering surprising, high-quality items without knowing what they are, but it fails for new users and items with no interactions. Production systems are hybrids. → *More:* Ch 13 §13.4 — Content-based vs collaborative filtering

Q190. Explicit vs implicit feedback? How do you use implicit feedback? Explicit feedback is a direct rating (stars, thumbs up), scarce but clear. Implicit feedback is behaviour (clicks, watch time, purchases, skips), plentiful but noisy: a missing interaction may mean "not seen" rather than "disliked", and clicks are biased by position and by what the old system showed. Use implicit signals as weighted positives (longer watch time = stronger), sample negatives from items shown but not engaged, and correct for position bias. → *More:* Ch 13 §13.2 — Explicit vs implicit feedback

Q191. How does matrix factorisation work for recommendations? Represent the sparse user–item interaction matrix R as the product of two low-rank matrices: each user gets a vector u_i and each item a vector v_j (say 64 numbers), and the predicted affinity is their dot product $u_i \cdot v_j$. Learn the vectors by minimising the error on observed entries plus L2 regularisation, with SGD or alternating least squares (ALS: fix items, solve users exactly, then swap). The learned vectors are embeddings in which similar users and items end up close. → *More:* Ch 13 §13.5 — Matrix factorisation

Q192. ⋆ What is the cold-start problem and how do you handle it? New users or items have no interaction history, so collaborative methods cannot score them. For new users: popular or trending items, onboarding questions, context (location, device, time), and fast session-based updates. For new items: content features (text, image and metadata embeddings) mapped into the same space, plus controlled exploration (a small share of traffic, bandits) to gather interactions quickly. → *More:* Ch 13 §13.10 — Cold start

Q193. What is a two-tower model and how is it used for candidate retrieval? One neural network (tower) embeds the user and context, another embeds the item, and the score is the dot product of the two vectors. Because items do not depend on the user, all item vectors are computed offline and loaded into an approximate-nearest-neighbour index; at request time only the user tower runs, and the index returns the top few hundred items in milliseconds. Training uses positives from engagement and in-batch negatives (other items in the batch), with a correction so popular items are not over-penalised. → *More:* Ch 13 §13.7 — Two-tower retrieval

Q194. Why do recommenders use a multi-stage funnel (retrieval → ranking → re-ranking)? Scoring hundreds of millions of items with a heavy model per request is impossible within a ~100 ms budget. Retrieval cheaply narrows to hundreds or thousands of candidates from several sources (two-tower, co-visitation, trending); a ranking model with rich features and cross features scores those precisely; re-ranking applies business rules, diversity, freshness, de-duplication and policy filters. Each stage trades accuracy for speed differently. → *More:* Ch 13 §13.8 — The funnel

Q195. What is approximate nearest-neighbour (ANN) search? Name some methods. Exact nearest-neighbour search over millions of vectors is too slow, so ANN indexes accept a small loss in recall for huge speed-ups. HNSW builds a multi-layer graph and walks greedily toward the query; IVF clusters vectors and searches only the nearest clusters; product quantisation compresses vectors to save memory. Libraries: FAISS, ScaNN, and vector databases. Tune for recall@k against latency. → *More:* Ch 13 §13.9 — ANN search

Q196. How do you evaluate a recommender system offline and online? Offline: split by time, then measure recall@k and NDCG@k (ranking quality), plus coverage (share of the catalogue ever recommended), diversity and novelty. Online: A/B test the business metrics (watch time, purchases, retention, long-term satisfaction) with guardrails (latency, complaints). Offline gains often shrink online because logged data reflects the old system's choices, so treat offline metrics as a filter, not the verdict. → *More:* Ch 13 §13.11 — Evaluation

Q197. What are popularity bias and feedback loops in recommenders? How do you counter them? The system shows popular items, they collect more clicks, the next model learns they are even better, and the rich get richer while niche items and new creators never get exposure. Counter with exploration traffic, diversity constraints in re-ranking, debiasing (inverse propensity weighting of logged data), and metrics such as coverage and long-tail share. → *More:* Ch 13 §13.12 — Popularity bias and feedback loops

Q198. What are good similarity measures for embeddings? Cosine vs dot product vs Euclidean? Cosine similarity compares only direction (it ignores vector length), good for text embeddings. Dot product also rewards length, which models can use to encode popularity or confidence; two-tower retrieval often uses it. Euclidean distance is equivalent to cosine for normalised vectors. Jaccard similarity suits sets (shared items between two users). → *More:* Ch 13 §13.3 — Similarity measures

---

13. ML system design

Q199. ⋆ How do you approach an ML system design interview? What framework do you use? Use a repeatable structure and say it out loud: (1) clarify the goal and frame it as an ML task (inputs, output, what decision it drives); (2) define success: an offline metric and the online business metric plus guardrails; (3) data and labels: sources, how labels are made, volumes; (4) features and a baseline model, then a stronger model; (5) serving: batch vs online, latency budget, the multi-stage funnel if catalogues are large; (6) evaluation, A/B testing, monitoring and retraining; (7) failure modes, fairness and cost. Spend most time where the problem is hardest, and state assumptions with napkin numbers. → *More:* Ch 15 §15.3 — The seven-step framework

Q200. How does ML system design differ from classic system design? Classic design focuses on storage, APIs, scaling and consistency. ML design adds data and labels as first-class components, a training pipeline separate from the serving path, an objective that must be translated from business goals, offline and online evaluation, and the fact that a system can silently degrade as data drifts while every server stays healthy. Interviewers expect you to talk about metrics, labels and feedback loops, not just boxes. → *More:* Ch 15 §15.1 — ML design vs classic system design

Q201. ⋆ Design a video (or product) recommendation system. Goal: maximise long-term satisfaction, measured online by watch time, retention and explicit feedback, not just clicks (which reward clickbait). Data: impressions, watches with watch fraction, likes, skips, search history, item metadata. Architecture: candidate generation from several sources (two-tower embeddings with an ANN index, co-watch, subscriptions, trending) → a ranker (gradient-boosted trees or a deep multi-task network predicting click, watch time and like, combined in

a weighted score) → re-ranking for diversity, freshness and policy. Cold start via content embeddings and exploration. Offline recall@k/NDCG on a time split; online A/B test; monitor drift and popularity bias; retrain daily, with near-real-time features for session context. → *More:* Ch 17 §17.1 — Video recommendations

Q202. ⋆ Design a news-feed ranking system. Rank candidate posts from friends, groups and pages for each user. Predict several engagement probabilities (click, like, comment, share, hide/report) with a multi-task model and combine them into a value score with tuned weights, including negative terms for hides and reports. Features: author affinity, post type and age, past engagement, content embeddings. Re-rank for diversity and integrity. Watch for engagement-bait, position bias in labels and long-term effects; evaluate with A/B tests on meaningful-interaction metrics and long-run retention. → *More:* Ch 17 §17.2 — Social feed ranking

Q203. ⋆ Design a search ranking system (e-commerce or web search). Two stages: retrieval (keyword/BM25 and embedding-based semantic search, merged) then learning-to-rank on the top few hundred results. Labels from clicks, add-to-cart and purchases (corrected for position bias, for example with randomised position experiments or click models), plus human relevance judgements. Features: query–document text match, embedding similarity, item quality and popularity, personalisation, price. Losses: pointwise, pairwise (RankNet) or listwise (LambdaMART). Offline NDCG@10 and MRR; online conversion and search abandonment; interleaving experiments detect ranking differences with less traffic. → *More:* Ch 17 §17.3 — E-commerce search ranking

Q204. ⋆ Design an ad click-through-rate (CTR) prediction system. Predict P(click | user, ad, context) for every ad in an auction; expected value = bid × pCTR, so predictions must be calibrated, not just ranked. Data is huge (billions of impressions/day) with rare positives (~1%), very sparse categorical features (user, ad, publisher IDs) and fast-changing ads. Models: logistic regression with hashed and crossed features, factorisation machines, or deep models with embeddings and explicit feature crosses (DCN, DeepFM). Train continuously or online on fresh data; negative downsampling with probability correction; evaluate with normalised log loss and calibration, then online revenue, CTR and advertiser ROI. Handle delayed clicks/conversions and click fraud. → *More:* Ch 18 §18.1 — Ad click prediction

Q205. ⋆ Design a real-time payment fraud-detection system. Score each transaction within ~100 ms before approval. Labels come from chargebacks and investigations, arriving weeks late, so mature labels lag and recent fraud patterns are learned slowly. Features: transaction amount, merchant, device and IP, velocity features from a streaming feature store ("number of cards used on this device in the last hour"), graph features linking accounts. Model: gradient-boosted trees, plus anomaly detection and hand rules for new attack patterns. Decisions: thresholds by expected cost, with approve / step-up authentication / review / block bands. Metrics: PR-AUC, recall at a fixed false-positive rate, dollars saved. Adversaries adapt, so monitor drift closely and retrain often. → *More:* Ch 18 §18.2 — Payment fraud detection

Q206. Design a harmful-content (content moderation) detection system. Multimodal classifiers (text, image, video, plus account signals) score each post at upload; high-confidence violations are removed automatically, uncertain ones go to human reviewers prioritised by predicted harm × reach, and appeals feed back as labels. Use multi-label outputs per policy, precision-oriented thresholds for automatic action, and recall measured by sampling. Handle adversarial obfuscation, many languages, label disagreement among reviewers and fairness across dialects. → *More:* Ch 18 §18.3 — Harmful content detection

Q207. Design "People You May Know" (friend recommendations). Treat it as link prediction on the social graph. Candidates from friends-of-friends, shared schools/workplaces and contact uploads; a ranker using graph features (mutual friends, common neighbours), profile similarity and interaction signals, or a graph neural network. Label: a sent and accepted request. Precompute candidates in batch for most users and refresh on new connections. Watch privacy (do not reveal sensitive relationships) and spam. → *More:* Ch 18 §18.4 — People You May Know

Q208. Design a visual search ("search by photo") system. Embed every catalogue image with a CNN or vision transformer fine-tuned with contrastive learning so the same or similar products land close together; store vectors in an ANN index; at query time detect and crop the object, embed it, retrieve nearest neighbours and re-rank by price, availability and metadata. Evaluate with recall@k on labelled pairs and online click-through and purchases. → *More:* Ch 18 §18.6 — Visual search

Q209. Design an ETA (estimated time of arrival) prediction system. Regression on route features (distance, road segments, historical and live speeds), time of day, weather and restaurant or pickup preparation time; a routing-engine estimate plus an ML correction is a strong design. Loss and metric: MAE or a quantile loss if being late hurts more than being early. Real-time features from streaming traffic data; evaluate per city and per time slot; monitor drift during holidays and events. → *More:* Ch 16 §16.11 — An ML platform for ETA prediction

Q210. What napkin math should you do in an ML design interview? Requests per second (daily active users × actions per day / 86,400, times 2–3 for peak), candidate counts per stage, latency budget per stage, feature-store reads per request, model size in memory, embedding index size (items × dimensions × 4 bytes for float32; 100 M × 128 × 4 ≈ 51 GB), and training data volume per day. It shows whether online scoring is feasible and where caching or batch precomputation is needed. → *More:* Ch 20 §20.13 — The 7-step framework and napkin math

Q211. How do you choose the right ML objective and metric for a product problem? Start with the business goal (revenue, retention, safety), find an observable proxy that the model can predict quickly (click, watch time, purchase), and add guardrail metrics to catch gaming (hides, complaints, latency). Check the proxy actually correlates with the goal; clicks alone reward clickbait. Combine several predicted outcomes in a tunable score so product owners can adjust weights without retraining. → *More:* Ch 15 §15.3, Ch 16 §16.7

Q212. What are the most common mistakes in ML design interviews? Jumping to a model before framing the problem and metric; ignoring how labels are obtained; forgetting latency and scale; not having a baseline; no plan for monitoring, retraining or cold start; ignoring position bias and feedback loops; and not discussing trade-offs. Interviewers reward explicit assumptions and choices you can justify. → *More:* Ch 15 §15.5 — Common mistakes

---

14. MLOps: deployment, monitoring, drift and A/B testing

Q213. ⋆ Batch vs online (real-time) vs streaming inference: how do you choose? Batch inference scores everything on a schedule and stores results (nightly recommendations, churn scores): cheap and simple, but stale and wasteful for users who never show up. Online inference scores on request through an API: fresh and personalised, but needs low-latency serving and features available in real time. Streaming inference scores events as they flow through a queue (fraud on each transaction). Many systems mix them: batch-precomputed candidates plus online ranking. → *More:* Ch 16 §16.5 — Batch vs online vs streaming inference

Q214. ⋆ What is training–serving skew and how do you prevent it? A difference between the data or feature logic the model saw in training and what it receives in production: features computed by different code paths, different time windows, missing values handled differently, or features available offline that are not available in time online. Prevent it by sharing one feature definition for both paths (a feature store), logging the exact features used at serving time and training on those logs, and checking online vs offline feature distributions. → *More:* Ch 16 §16.2 — Training vs serving

Q215. What is a feature store and why use one? A system that defines features once and serves them consistently: an offline store with full history for building training sets with point-in-time-correct joins (only feature values known before each label's timestamp), and an online low-latency store for serving the latest values. It removes training–serving skew, prevents leakage, and lets teams reuse features. → *More:* Ch 16 §16.4 — Features and feature stores

Q216. ⋆ What is data drift vs concept drift? How do you detect them? Data (covariate) drift: the distribution of inputs changes (new user demographics, a new device type). Concept drift: the relationship between inputs and the label changes (fraudsters change tactics, so the same features now mean something else). Label/prior drift: the class balance changes. Detect input drift with statistics per feature (population stability index, PSI; Kolmogorov–Smirnov test; null rates), prediction drift by monitoring the score distribution, and concept drift only through performance once labels arrive. Often the "drift" is actually a broken upstream pipeline. → *More:* Ch 16 §16.8 — Monitoring, drift and retraining

Q217. ⋆ How do you monitor a model in production? In layers: (1) system health: latency, errors, throughput, resource use; (2) data quality: schema checks, missing values, ranges, feature drift; (3) prediction distribution and its drift; (4) model performance on delayed labels, overall and per segment; (5) business metrics. Set alerts with sensible thresholds, keep dashboards per model version, and have a runbook: roll back, fall back to rules, or retrain. → *More:* Ch 16 §16.8

Q218. When and how do you retrain a model? On a schedule (daily, weekly), when monitoring shows a measured drop in performance or strong drift, or continuously for fast-changing domains like ads. Retrain through an automated pipeline: build a fresh dataset with point-in-time features, train, validate against the current model on a recent hold-out, then promote through shadow and canary stages. Keep versions so you can roll back. → *More:* Ch 16 §16.8

Q219. How do you safely deploy a new model? Shadow, canary, blue-green? Shadow deployment: the new model scores live traffic but its outputs are only logged, not used; compare with the current model and check latency. Canary: send a small share of real traffic (say 1–5%) to the new model and watch metrics before ramping up. Blue-green: run old and new environments side by side and switch traffic at once, with instant rollback. Then a proper A/B test measures the business impact. → *More:* Ch 16 §16.9–16.10

Q220. ⋆ How would you design an A/B test for a new model? Choose one primary metric plus guardrails; randomise by user (not by request) so each person has a consistent experience; compute sample size from the baseline variance and minimum detectable effect; run for whole weekly cycles; check the split ratio matches what you configured (sample-ratio mismatch signals a bug); analyse with a pre-registered test and confidence intervals; and watch for novelty effects. Ship if the primary metric improves and no guardrail degrades. → *More:* Ch 16 §16.9 — Online experiments and A/B testing

Q221. What problems can invalidate an A/B test? Peeking and stopping early when results look significant; testing many metrics without correction; sample-ratio mismatch; network effects or interference (in marketplaces and social networks, treated users affect control users; use cluster or switchback randomisation); novelty and learning effects; seasonality; and too little traffic for the effect size. Recommenders also suffer from shared training data between arms, since both models learn from the same logs. → *More:* Ch 16 §16.9

Q222. What is interleaving and when is it better than A/B testing? For ranking, interleaving mixes results from models A and B into one list for each user and counts which model's results get clicked. Because each user compares both, it needs far less traffic (often 10–100×) to detect which ranker is better. It tells you which is preferred, not the effect on business metrics, so it is used as a fast filter before a full A/B test. → *More:* Ch 16 §16.9, Ch 17 §17.3

Q223. What are multi-armed bandits used for in production, and how do they compare with A/B tests? Bandits shift traffic toward better-performing variants during the test, reducing the cost of showing a poor variant; good for short-lived choices (headlines, promotions) and for exploration in recommenders. A/B tests keep a fixed split and give cleaner, unbiased estimates of effect size, which matters for long-term product decisions. → *More:* Ch 14 §14.4, Ch 16 §16.9

Q224. What is MLOps? What should an ML pipeline include? MLOps applies DevOps practice to ML: versioned data, code and models; automated, reproducible training pipelines; experiment tracking (parameters, metrics, artefacts, for example MLflow); a model registry with stages; continuous integration tests for data and models; automated deployment with gates; and monitoring that can trigger retraining. The aim is that any model in production can be traced back and rebuilt. → *More:* Ch 16 §16.10 — MLOps

Q225. How do you serve a model with low latency? Keep the model small (distillation, quantisation), precompute what you can (item embeddings, batch features), cache frequent results, batch requests on GPUs, use an optimised runtime (ONNX Runtime, TensorRT, TorchServe/Triton), fetch features in parallel from an online store, set a per-stage latency budget, and define a fallback (cached or rule-based answer) when the model times out. → *More:* Ch 16 §16.6 — Model serving basics

Q226. Your model's performance dropped in production. How do you investigate? Confirm it is real (label delay? metric bug?). Check data pipelines and feature freshness first: schema changes, upstream outages, a feature suddenly null. Compare feature and prediction distributions to training. Slice performance by segment, country, device and time to localise it. Check for external causes (holiday, marketing campaign, new product, competitor) and for concept drift. Then fix the pipeline, roll back, or retrain on recent data. → *More:* Ch 16 §16.8

Q227. How do you handle delayed labels (for example fraud chargebacks arriving weeks later)? Monitor proxies in the meantime (prediction drift, early signals such as disputes or manual review results); train on matured labels while using a short window of recent data with partial labels carefully; use survival-style or importance-weighted corrections; and add fast-feedback components (rules, anomaly detectors) for new attack patterns. → *More:* Ch 16 §16.3, Ch 18 §18.2

Q228. How do you scale training to very large data or models? Data parallelism (copies of the model on many GPUs, each on a different mini-batch, with gradients averaged); model, tensor or pipeline parallelism when a model does not fit on one device; sharded optimisers (FSDP/ZeRO); mixed precision; gradient checkpointing; and efficient data loading. For classic ML, use distributed gradient boosting (XGBoost/LightGBM on Spark or Dask) or sample the data.

---

15. Responsible ML

Q229. Where does bias in ML systems come from? Historical bias (the world was unfair, and labels record it), representation and sampling bias (some groups under-represented), measurement and label bias (proxies like "arrests" for "crime"), aggregation bias (one model for groups that behave differently), evaluation bias (benchmarks that do not reflect users), and feedback loops after deployment. Removing the sensitive attribute does not help when other features act as proxies (ZIP code for race). → *More:* Ch 19 §19.1 — Where bias comes from

Q230. What fairness definitions do you know, and can they all be satisfied? Demographic parity: equal positive-prediction rates across groups. Equal opportunity: equal true positive rates. Equalised odds: equal true and false positive rates. Predictive parity / calibration

within groups: equal precision or calibration. When base rates differ between groups, a non-perfect classifier cannot satisfy calibration and equalised odds simultaneously (the impossibility results), so you choose the criterion by the harm in your context and document why. → *More:* Ch 19 §19.2 — Fairness definitions and the impossibility trade-off

Q231. How do you mitigate bias? Before training (pre-processing): collect more representative data, reweigh or resample groups, fix label problems. During training (in-processing): add fairness constraints or penalties to the loss. After training (post-processing): group-specific thresholds or calibration. Always measure per-group metrics before and after, and check the accuracy cost. → *More:* Ch 19 §19.4 — Mitigating bias

Q232. ⋆ How do you explain a model's predictions? SHAP vs LIME vs permutation importance? Global: permutation importance (score drop when a feature is shuffled) and partial dependence plots (average prediction as one feature varies). Local: SHAP assigns each feature a contribution to a single prediction based on Shapley values from game theory, consistent and additive (contributions sum to the prediction minus the baseline); LIME fits a simple surrogate model around one prediction, faster but less stable. Explanations show what the model uses, not what causes the outcome. → *More:* Ch 19 §19.5 — Explainability

Q233. What is differential privacy and federated learning? Differential privacy adds calibrated noise (to data, gradients as in DP-SGD, or query results) so that the output barely changes whether or not any one person is in the data, with the privacy budget ε quantifying the guarantee. Federated learning trains on users' devices and sends only model updates to a server, so raw data never leaves the device; it is often combined with secure aggregation and differential privacy because updates can still leak information. → *More:* Ch 19 §19.6 — Privacy

Q234. What attacks on ML models do you know? Adversarial examples (tiny input changes that flip predictions, for example FGSM, which steps in the sign of the input gradient); data poisoning (injecting bad training data or backdoors); model extraction (copying a model through its API); model inversion and membership inference (recovering training data or learning whether a person was in it); and, for LLMs, prompt injection and jailbreaks. Defences include adversarial training, data provenance checks, rate limiting, output rounding and differential privacy. → *More:* Ch 19 §19.7 — Security: attacks on ML models

Q235. What is a model card? A short document published with a model that states its intended use and out-of-scope uses, training data, evaluation results including per-group metrics, known limitations, ethical considerations and versioning. Datasheets do the same for datasets. They make limits visible to users and auditors and are increasingly expected by regulation such as the EU AI Act. → *More:* Ch 19 §19.8 — Model cards and datasheets, Ch 19 §19.9 — Regulation overview

---

16. Coding tasks commonly asked

These come up in "ML coding" rounds: implement from scratch with NumPy, usually in 20–30 minutes, then discuss complexity and edge cases. Snippets assume `import numpy as np`.

Q236. ⋆ Implement k-means from scratch.

```
def kmeans(X, k, iters=100, seed=0):
    rng = np.random.default_rng(seed)
    C = X[rng.choice(len(X), k, replace=False)]           # random initial centroids
    for _ in range(iters):
        d = ((X[:, None, :] - C[None, :, :]) ** 2).sum(-1)  # n x k squared distances
        labels = d.argmin(1)
        newC = np.array([X[labels == j].mean(0) if (labels == j).any() else C[j] for j in range(k)])
        if np.allclose(newC, C): break
        C = newC
    return C, labels
```

Each iteration costs O(n·k·d). Mention k-means++ initialisation, multiple restarts, empty-cluster handling (kept here by reusing the old centroid) and feature scaling. → *More:* Ch 9 §9.1

Q237. ⋆ Implement logistic regression with gradient descent.

```
def sigmoid(z): return 1 / (1 + np.exp(-z))

def fit_logreg(X, y, lr=0.1, epochs=1000, l2=0.0):
    w, b = np.zeros(X.shape[1]), 0.0
    for _ in range(epochs):
        p = sigmoid(X @ w + b)
        w -= lr * (X.T @ (p - y) / len(y) + l2 * w)   # gradient of mean log loss + L2
        b -= lr * (p - y).mean()
    return w, b
```

The gradient (p − y)·x falls out of log loss with a sigmoid. Follow-ups: scale features, use `np.clip` or a stable sigmoid for large |z|, add mini-batches, and check convergence by tracking the loss. → *More:* Ch 5 §5.8

Q238. Implement linear regression (closed form and gradient descent).

```
def fit_ols(X, y):
    Xb = np.c_[np.ones(len(X)), X]                    # add intercept column
    return np.linalg.lstsq(Xb, y, rcond=None)[0]      # stable least squares

def fit_gd(X, y, lr=0.01, epochs=5000):
    Xb = np.c_[np.ones(len(X)), X]; th = np.zeros(Xb.shape[1])
    for _ in range(epochs):
        th -= lr * 2 * Xb.T @ (Xb @ th - y) / len(y)  # gradient of MSE
    return th
```

Mention why `lstsq` beats explicitly inverting X^TX (numerical stability) and that gradient descent needs scaled features. → *More:* Ch 5 §5.3

Q239. ⋆ Compute precision, recall, F1 and accuracy from scratch.

```python
def prf(y_true, y_pred):
    tp = np.sum((y_pred == 1) & (y_true == 1)); fp = np.sum((y_pred == 1) & (y_true == 0))
    fn = np.sum((y_pred == 0) & (y_true == 1)); tn = np.sum((y_pred == 0) & (y_true == 0))
    p = tp / (tp + fp) if tp + fp else 0.0
    r = tp / (tp + fn) if tp + fn else 0.0
    f1 = 2 * p * r / (p + r) if p + r else 0.0
    return p, r, f1, (tp + tn) / len(y_true)
```

Handle the zero-division cases explicitly; interviewers check for them. → *More:* Ch 6 §6.3

Q240. ⋆ Compute ROC-AUC from scratch.

```python
def roc_auc(y, s):
    order = np.argsort(s, kind="mergesort")
    ranks = np.empty(len(s)); ranks[order] = np.arange(1, len(s) + 1)
    for v in np.unique(s):                      # average ranks for tied scores
        m = s == v; ranks[m] = ranks[m].mean()
    n_pos, n_neg = y.sum(), len(y) - y.sum()
    return (ranks[y == 1].sum() - n_pos * (n_pos + 1) / 2) / (n_pos * n_neg)
```

This uses the rank (Mann–Whitney) identity: AUC is the fraction of positive–negative pairs ranked correctly, computed in O(n log n) instead of $O(n^2)$ pair comparisons. → *More:* Ch 6 §6.3

Q241. Implement k-nearest neighbours classification.

```python
def knn_predict(Xtr, ytr, Xte, k=5):
    d = ((Xte[:, None, :] - Xtr[None, :, :]) ** 2).sum(-1)   # m x n distances
    idx = np.argpartition(d, k - 1, axis=1)[:, :k]           # k nearest, unsorted
    return np.array([np.bincount(ytr[row]).argmax() for row in idx])  # labels 0..C-1
```

`argpartition` finds the k smallest in O(n) per query instead of a full sort (using `k - 1` as the pivot also works when k equals the training-set size). Mention memory (m × n matrix; process in chunks), scaling, ties, and KD-trees or ANN indexes for speed. → *More:* Ch 8 §8.1

Q242. Implement a numerically stable softmax and cross-entropy.

```python
def softmax(z):
    z = z - z.max(axis=-1, keepdims=True)        # subtract max: avoids overflow in exp
    e = np.exp(z); return e / e.sum(axis=-1, keepdims=True)

def cross_entropy(logits, y):                    # y = integer class labels
    z = logits - logits.max(1, keepdims=True)
    logp = z - np.log(np.exp(z).sum(1, keepdims=True))   # log-softmax
    return -logp[np.arange(len(y)), y].mean()
```

Subtracting the maximum does not change the result but prevents `exp` overflow; computing log-softmax directly avoids log(0). → *More:* Ch 10 §10.3

Q243. Implement a decision-tree split search (best threshold by Gini) for one feature.

```python
def gini(y): p = np.bincount(y) / len(y); return 1 - (p ** 2).sum()

def best_split(x, y):
    best = (None, np.inf)
    for t in np.unique(x)[:-1]:                  # candidate thresholds
        L, R = y[x <= t], y[x > t]
        score = (len(L) * gini(L) + len(R) * gini(R)) / len(y)
        if score < best[1]: best = (t, score)
    return best
```

This is O(u·n) for u unique values; real implementations sort once and update class counts incrementally for O(n log n). A full tree applies this to every feature and recurses. → *More:* Ch 7 §7.1

Q244. Implement a two-layer neural network forward and backward pass in NumPy.

```python
def step(X, Y, W1, b1, W2, b2, lr=0.1):          # Y one-hot, n x classes
    H = np.maximum(0, X @ W1 + b1)               # ReLU hidden layer
    P = softmax(H @ W2 + b2)
    dZ2 = (P - Y) / len(X)                       # softmax + cross-entropy gradient
    dW2, db2 = H.T @ dZ2, dZ2.sum(0)
    dH = dZ2 @ W2.T; dH[H <= 0] = 0              # backprop through ReLU
    dW1, db1 = X.T @ dH, dH.sum(0)
    for p, g in ((W1, dW1), (b1, db1), (W2, dW2), (b2, db2)): p -= lr * g
```

Shows backpropagation as matrix calculus: each gradient is the upstream gradient times a local derivative. Check it with numerical gradient checking on a tiny network. → *More:* Ch 10 §10.4

Q245. Implement scaled dot-product attention.

```python
def attention(Q, K, V, mask=None):               # Q: n x d, K: m x d, V: m x dv
    scores = Q @ K.T / np.sqrt(Q.shape[-1])
    if mask is not None: scores = np.where(mask, scores, -1e9)   # causal / padding mask
    return softmax(scores) @ V
```

Follow-ups: multi-head (reshape into h heads), causal mask (`np.tril`), and the $O(n^2)$ memory of the score matrix. → *More:* Ch 12 §12.6

Q246. Implement PCA with NumPy.

```python
def pca(X, n_components):
    Xc = X - X.mean(0)                          # centre (standardise first if units differ)
    U, S, Vt = np.linalg.svd(Xc, full_matrices=False)
    explained = S ** 2 / (S ** 2).sum()
    return Xc @ Vt[:n_components].T, explained[:n_components]
```

The rows of Vt are the principal directions; squared singular values are proportional to the variance each explains. SVD avoids forming the covariance matrix explicitly. → *More:* Ch 9 §9.5

Q247. Implement NDCG@k.

```python
def ndcg_at_k(rels, k):                         # rels: relevance in ranked order
    rels = np.asarray(rels, float)
    disc = 1 / np.log2(np.arange(2, k + 2))
    dcg = (rels[:k] * disc[:len(rels[:k])]).sum()
    ideal = np.sort(rels)[::-1][:k]
    idcg = (ideal * disc[:len(ideal)]).sum()
    return dcg / idcg if idcg > 0 else 0.0
```

Some definitions use 2^rel − 1 as the gain; say which one you use. → *More:* Ch 6 §6.4

Q248. Implement stratified train/test split or k-fold indices without scikit-learn.

```python
def stratified_kfold(y, k=5, seed=0):
    rng = np.random.default_rng(seed); folds = [[] for _ in range(k)]
    for c in np.unique(y):
        idx = rng.permutation(np.where(y == c)[0])
        for i, chunk in enumerate(np.array_split(idx, k)): folds[i].extend(chunk)
    all_idx = np.arange(len(y))
    return [(np.setdiff1d(all_idx, f), np.array(f)) for f in folds]
```

Each class is shuffled and dealt evenly across folds, so every fold keeps the class ratio. → *More:* Ch 6 §6.6

Q249. Implement mini-batch SGD for linear regression with early stopping.

```python
def sgd(X, y, Xv, yv, lr=0.01, bs=32, epochs=100, patience=5, seed=0):
    rng = np.random.default_rng(seed); w = np.zeros(X.shape[1]); best, bad = (np.inf, w), 0
    for _ in range(epochs):
        for i in np.array_split(rng.permutation(len(X)), max(1, len(X) // bs)):
            w = w - lr * 2 * X[i].T @ (X[i] @ w - y[i]) / len(i)
        v = ((Xv @ w - yv) ** 2).mean()
        if v < best[0]: best, bad = (v, w.copy()), 0
        elif (bad := bad + 1) >= patience: break
    return best[1]
```

Shows shuffling each epoch, mini-batches, validation tracking and restoring the best weights. → *More:* Ch 5 §5.4, Ch 6 §6.10

Q250. Implement a Gaussian or multinomial naive Bayes classifier.

```python
def fit_mnb(X, y, alpha=1.0):                   # X: word-count matrix
    classes = np.unique(y)
    log_prior = np.log(np.array([(y == c).mean() for c in classes]))
    counts = np.array([X[y == c].sum(0) for c in classes]) + alpha    # Laplace smoothing
    log_lik = np.log(counts / counts.sum(1, keepdims=True))
    return classes, log_prior, log_lik

def predict_mnb(X, classes, log_prior, log_lik):
    return classes[(X @ log_lik.T + log_prior).argmax(1)]  # sum of logs, no underflow
```

Working in log space turns the product of many small probabilities into a sum, so it never underflows to zero; α > 0 keeps unseen words from zeroing out a class. → *More:* Ch 8 §8.3

Q251. How would you write a train/evaluate script for a new tabular problem in a live-coding round? Load data; split first (stratified or by time); build a `Pipeline` with a `ColumnTransformer` (impute + scale numerics, impute + one-hot categoricals); fit a baseline (logistic regression or `DummyClassifier`) and a gradient-boosted model (`HistGradientBoostingClassifier`); evaluate with cross-validation on the training set using the right metric; tune a few hyperparameters with randomised search; evaluate once on the test set; print a confusion matrix and permutation importances. Narrate each choice. → *More:* Ch 4 §4.10, Ch 1 §1.11 — A first end-to-end project

Q252. Which LeetCode-style problems are common in ML-engineer coding rounds? Besides ML-from-scratch tasks, MLE loops reuse standard data-structure problems: Top K Frequent Elements and K Closest Points to Origin (heaps, directly related to nearest-neighbour search), Merge Intervals, LRU Cache, Moving Average from Data Stream, Random Pick with Weight (weighted sampling via prefix sums and binary search), Kth Largest Element (quickselect), sparse-vector dot product, and matrix or graph traversal (BFS/DFS, connected components). Expect also SQL window-function questions for data roles.

Sources of the question lists

Question wording only (no answers) was gathered from these public lists; every answer above is written for this book.

- alexeygrigorev/data-science-interviews — "Theoretical interview questions" (GitHub, `theory.md`)
- andrewekhalel/MLQuestions — "Machine Learning Interview Questions and Answers" (GitHub)
- khangich/machine-learning-interview — "Minimum Viable Study Plan for Machine Learning Interviews" (GitHub; probability questions and the ML system design prompts: video recommendation, feed ranking, ad click prediction, delivery-time estimation, search ranking, fraud detection)
- LeetCode problem titles listed in Q252 (Top K Frequent Elements, K Closest Points to Origin, Random Pick with Weight, LRU Cache, Merge Intervals, Moving Average from Data Stream)